
dmx-learn

Release 1.1.0

Adam Walder

Aug 25, 2026

CONTENTS

1	Contents	3
	Python Module Index	739
	Index	741

dmx-learn is a Python package for distributed heterogeneous density estimation. With only a few lines of code you can specify and fit complex models on variable-length heterogeneous data.

CONTENTS

1.1 Installation

dmx-learn is a Python package for distributed heterogeneous density estimation. With only a few lines of code you can specify and fit complex models on variable-length heterogeneous data.

To install *dmx-learn* with make sure you have Python 3.10+ (less than 3.14) then run

```
$ git clone https://github.com/LLNL/dmx-learn.git
$ pip ./dmx-learn
```

1.2 Docstring style guide

dmx-learn uses Google-style docstrings, parsed by Sphinx Napoleon and checked with *pydocstyle*. Write for callers: describe the contract, statistical meaning, and non-obvious constraints without restating the implementation.

1.2.1 Structure and section order

Start with a short imperative or descriptive summary that ends with punctuation. Add a blank line and explanatory prose only when it helps a caller. When applicable, use sections in this order:

1. Args
2. Attributes (for public class attributes)
3. Returns or Yields
4. Raises
5. Note
6. Examples

Omit empty sections. Use the exact section names and a colon, indent entries by four spaces, and leave a blank line before and after each section. Keep constructor arguments in **Args** on the class docstring. A function that only has a useful summary does not need an **Args** or **Returns** section.

1.2.2 One-line docstrings

A one-line docstring is sufficient when the complete caller-facing contract is obvious from the name, signature, annotations, and surrounding protocol. This normally applies to simple accessors, trivial properties, and small protocol methods with no special shapes, side effects, or failure modes:

```
def name(self) -> str:
    """Return the distribution name."""
```

Do not expand a clear one-line docstring merely to satisfy a template. Use a longer docstring when a caller needs parameterization, support, shape, mutation, randomness, device or dtype behavior, initialization semantics, or exceptions to use the API correctly.

1.2.3 Repository policies

`__init__`

Put the complete construction contract and **Args** section on the class so autodoc presents it in one place. When pydocstyle requires a constructor docstring, use a short compliant description such as `"""Initialize the Gaussian distribution."""` rather than duplicating the class documentation.

Magic methods

Give public magic methods a concise, useful description when pydocstyle requires one. Describe the observable result, for example, `"""Return a reproducible representation of the distribution."""`. Do not document Python mechanics that are already self-evident.

Inherited protocol methods

Document what the concrete implementation does. State its accepted input form, result, shapes, or implementation-specific behavior; do not copy the abstract base class text without adapting it. For example, a concrete `seq_log_density` docstring should identify that implementation's encoded sequence layout and output shape.

Private helpers

A private helper needs a docstring only when its contract, mutation, shape transformation, or algorithm is not obvious. Keep it short and useful to maintainers. Public APIs always require documentation even if a related private helper is documented.

Types

Prefer function annotations as the source of types. Do not repeat a type in a docstring unless the prose adds a constraint that the annotation cannot express, such as accepted dtypes or scalar-versus-array behavior.

Shapes

State shapes whenever they are part of the contract. Use names such as `n` for observations, `d` for features, and `k` for components, define them in prose, and distinguish scalar input from encoded sequence input. Also document sufficient-statistic layouts exposed through `keys` and device or dtype rules in `torch_stats`.

Raises

List exceptions that the function deliberately raises as part of its public contract and say precisely when they occur. Do not invent failure cases or list every incidental exception that a dependency might propagate.

Raw docstrings

Prefix a docstring with `r` whenever it contains backslashes, especially LaTeX commands such as `\sum` or `\sigma`. This prevents Python escape processing and avoids invalid-escape warnings. Sphinx roles and directives still work normally inside a raw docstring.

1.2.4 Statistical API content

For distributions, estimators, accumulators, samplers, and encoders, document the caller-visible items that apply:

- parameterization and support;
- scalar and encoded-sequence input forms;
- array or tensor shapes;
- public sufficient-statistic layout;
- pseudo-count and initialization semantics;
- device and dtype behavior for `torch_stats`; and

- exceptional inputs or failure conditions.

Keep derivations in narrative documentation unless a short equation is necessary to define the API.

1.2.5 Examples

Distribution

This class docstring records parameterization, support, and attributes without repeating annotated types:

```
class BernoulliDistribution:
    """Represent a Bernoulli distribution on zero and one.

    ``prob`` is the probability of observing one. Scalar density methods
    accept values in ``{0, 1}``; sequence methods consume the representation
    returned by this distribution's data encoder.

    Args:
        prob: Probability of one. Must lie in ``[0, 1]``.
        name: Optional identifier used when composing models.

    Attributes:
        prob: Probability of one.
        name: Identifier used when composing models.
    """

    def __init__(self, prob: float, name: str | None = None) -> None:
        """Initialize the Bernoulli distribution."""
```

Estimator

An estimator should explain the statistic layout, initialization semantics, and meaningful failure modes:

```
class BernoulliEstimator:
    """Estimate a Bernoulli distribution from weighted observations.

    Args:
        pseudo_count: Effective sample size of the optional prior. A value
            of ``None`` disables smoothing.
    """

    def estimate(
        self,
        nobs: float | None,
        suff_stat: tuple[float, float],
    ) -> BernoulliDistribution:
        """Estimate a distribution from sufficient statistics.

        Args:
            nobs: Effective observation count, or ``None`` when the count
                is encoded in ``suff_stat``.
            suff_stat: Pair ``(sum_of_ones, total_weight)``.

        Returns:
```

(continues on next page)

(continued from previous page)

```

        Distribution with its probability set to the weighted mean.

    Raises:
        ValueError: If the total weight is not positive.
    """

```

Utility function

For array utilities, annotations give the type while prose defines the shape contract:

```

def normalize_rows(values: np.ndarray) -> np.ndarray:
    """Normalize each row to sum to one.

    Args:
        values: Nonnegative array of shape ``(n, d)`, where ``n`` is the
            number of observations and ``d`` is the feature dimension.

    Returns:
        New array of shape ``(n, d)`` with unit row sums.

    Raises:
        ValueError: If ``values`` is not two-dimensional or contains a row
            whose sum is zero.
    """

```

Sphinx and MathJax math

Use `:math:` for short inline expressions and `.. math::` for a displayed definition. Define every symbol, and use a raw docstring because LaTeX uses backslashes:

```

def log_density(self, x: float) -> float:
    r"""Evaluate the Gaussian log density at :math:`x`.

    The mean is :math:`\mu` and the positive variance is :math:`\sigma^2`:

    .. math::

        \log p(x) = -\frac{1}{2}
        \left[ \log(2\pi\sigma^2)
        + \frac{(x-\mu)^2}{\sigma^2} \right].

    Args:
        x: Scalar observation on the real line.

    Returns:
        Log density at ``x``.
    """

```

1.2.6 Validation

Run pydocstyle on every Python source file changed by an issue, using the smallest relevant target while iterating:

```
poetry run pydocstyle src/dmx/stats/gaussian.py
poetry run pydocstyle src/dmx/utils
```

The repository configuration selects the Google convention, ignores module rule D100, and excludes `__init__.py` files. Consequently, manually review module and package overview docstrings even when pydocstyle reports no violations. Run the repository-wide check used by CI and pre-commit with:

```
poetry run pydocstyle src/dmx
```

Build all documentation with warnings treated as errors to validate Napoleon, cross-references, and MathJax markup:

```
poetry run sphinx-build -W -b html docs/ docs/_build/html
```

Inspect the rendered HTML for representative signatures, lists, and equations. Do not commit generated files under `docs/_build`.

1.3 dmx.stats API

NumPy/SciPy probability models and classical estimation interfaces.

This is the primary non-Bayesian backend for local or PySpark-based density estimation. The package root re-exports its supported distributions and estimators together with helpers for encoding observations, initialization, likelihood evaluation, and parameter estimation. Lower-level protocols, samplers, accumulators, and non-re-exported models remain available from their defining `dmx.stats` submodules.

Models in this package use NumPy-oriented encoded data and should be paired with `dmx.utils` for local workflow helpers or `dmx.mpi4py.stats` for MPI collectives. Use `dmx.bstats` for prior-aware Bayesian and variational models, or `dmx.torch_stats` for tensor-backed execution. Although the backends share terminology, their distributions, estimators, and encodings are separate protocols rather than interchangeable implementations.

The guides below combine model introductions with the API reference pages for the most commonly used distributions. The remaining groups expose specialized models and integration helpers directly from their cleaned source docstrings.

1.3.1 Package-level workflows

`dmx.stats.initialize(data, estimator, rng, p=0.1)`

Randomly initialize a model corresponding to `ParameterEstimator` for iid observations data.

Parameters

- **data** (*Union[Sequence[T], RDD]*) – Set of iid observations compatible with ‘estimator’.
- **estimator** (*ParameterEstimator*) – `ParameterEstimator` object for desired model to be estimated from data.
- **rng** (*np.random.RandomState*) – `RandomState` object for setting seed.
- **p** (*float, optional*) – Proportion of data to randomly sample for initializing model. Defaults to 0.1.

Return type

`dmx.stats.pdist.SequenceEncodableProbabilityDistribution`

Returns

`SequenceEncodableProbabilityDistribution` – Initialized model.

Parameters

- **data** (*Sequence[T] | RDD*)
- **estimator** (*ParameterEstimator*)
- **rng** (*RandomState*)
- **p** (*float*)

`dmx.stats.estimate(data, estimator, prev_estimate=None)`

Perform E-step in EM algorithm by iterating over all observations in 'data'.

Parameters

- **data** (*Union[Sequence[T], RDD]*) – Sequence of iid observations of data type consistent with 'estimator' and/or 'prev_estimate'.
- **estimator** (*ParameterEstimator*) – Model to be estimated from 'data'.
- **prev_estimate** (*Optional[SequenceEncodableProbabilityDistribution], optional*) – Previous estimate of EM algorithm. Must be included for distributions that require initialization. Defaults to None.

Return type

dmx.stats.pdist.SequenceEncodableProbabilityDistribution

Returns

SequenceEncodableProbabilityDistribution – Next iteration of EM algorithm.

Parameters

- **data** (*Sequence[T] | RDD*)
- **estimator** (*ParameterEstimator*)
- **prev_estimate** (*SequenceEncodableProbabilityDistribution | None*)

`dmx.stats.seq_encode(data, encoder=None, estimator=None, model=None, num_chunks=1, chunk_size=None)`

Sequence encode a sequence of iid observations from a distribution corresponding to 'encoder'.

Parameters

- **data** (*Union[Sequence[T], RDD]*) – Sequence of iid observations of data type consistent with 'encoder'.
- **encoder** (*Optional[DataSequenceEncoder], optional*) – A DataSequenceEncoder object for sequence encoding iid sequences.
- **estimator** (*Optional[ParameterEstimator], optional*) – An estimator to create DataSequenceEncoder from.
- **model** (*Optional[SequenceEncodableProbabilityDistribution], optional*) – A distribution to create DataSequenceEncoder from.
- **num_chunks** (*int, optional*) – Number of chunks to split the data into. Defaults to 1.
- **chunk_size** (*Optional[int], optional*) – Approximate size of chunks to determine num_chunks above.

Return type

typing.Union[pyspark.core.rdd.RDD, typing.List[typing.Tuple[int, dmx.stats.pdist.EncodedDataSequence]]]

Returns

Union[RDD, List[Tuple[int, EncodedDataSequence]]] – Encoded data.

Parameters

- **data** (*Sequence[T] | RDD*)
- **encoder** (*DataSequenceEncoder | None*)
- **estimator** (*ParameterEstimator | None*)
- **model** (*SequenceEncodableProbabilityDistribution | None*)
- **num_chunks** (*int*)
- **chunk_size** (*int | None*)

`dmx.stats.seq_log_density(enc_data, estimate)`

Vectorized evaluation of ‘estimate’ log-density for each observation in `enc_data`.

Notes

If ‘estimate’ is input as a List of numpy arrays. Each list entry corresponds to the `seq_log_density` calls of all the encoded data for each List entry of estimate.

If ‘estimate’ is a single `SequenceEncodableProbabilityDistribution` instance. The `log_density` of every observation in the ‘`enc_data`’ data set is returned as a list.

`E = Union[List[Tuple[int, EncodedDataSequence]], RDD]`

Parameters

- **enc_data** (*E*) – Sequence encoded data of format matching output of `seq_encode()` function.
- **estimate** (*SequenceEncodableProbabilityDistribution*) – Distribution to use for `log_density` evaluations.

Return type

`typing.List[numpy.ndarray]`

Returns

`Union[List[np.ndarray[float]], List[float]]`

Parameters

- **enc_data** (*List[Tuple[int, EncodedDataSequence]] | RDD*)
- **estimate** (*Sequence[SequenceEncodableProbabilityDistribution] | SequenceEncodableProbabilityDistribution*)

`dmx.stats.seq_log_density_sum(enc_data, estimate)`

Evaluate and sum vectorized log densities over encoded data.

Notes

Returns a Tuple containing the sum of all observations in `enc_data`, and the sum of the `log_density` evaluated at all encoded data observations in `enc_data`. This is a fully vectorized evaluation.

Parameters

- **enc_data** (*Union[List[Tuple[int, T]], RDD]*) – Sequence encoded data of format matching output of `seq_encode()` function.
- **estimate** (*SequenceEncodableProbabilityDistribution*) – Distribution to use for `log_density` evaluations. Must be consistent with `enc_data`.

Return type

`typing.Tuple[float, float]`

Returns

Tuple[float, float]

Parameters

- **enc_data** (*List[Tuple[int, EncodedDataSequence]]* | *RDD*)
- **estimate** (*SequenceEncodableProbabilityDistribution*)

`dmx.stats.seq_estimate(enc_data, estimator, prev_estimate)`

Perform vectorized E-step in EM algorithm for encoded sequence of observations in ‘enc_data’.

Notes

Arg estimator must be consistent with prev_estimate. That is, prev_estimate must be an estimate that could be obtained from estimator.

Arg enc_data must type consistent with estimator and prev_estimate (result of seq_encode() call).

Returns the next iteration of EM algorithm with vectorized calls to “seq_update()” of the corresponding SequenceEncodableStatisticAccumulator objects.

E = Union[List[Tuple[int, EncodedDataSequence]], RDD]

Parameters

- **enc_data** (*E*) – Sequence encoded data of format matching output of seq_encode() function.
- **estimator** (*ParameterEstimator*) – Model to be estimated from ‘enc_data’.
- **prev_estimate** (*SequenceEncodableProbabilityDistribution*) – Previous estimate of EM algorithm.

Return type

typing.TypeVar(T_D, bound= `dmx.stats.pdist.SequenceEncodableProbabilityDistribution`)

Returns

SequenceEncodableProbabilityDistribution

Parameters

- **enc_data** (*List[Tuple[int, EncodedDataSequence]]* | *RDD*)
- **estimator** (*ParameterEstimator*)
- **prev_estimate** (*T_D*)

`dmx.stats.seq_initialize(enc_data, estimator, rng, p=0.1)`

Initialize a model from encoded independent observations.

Notes

Arg enc_data must type consistent with estimator (result of seq_encode() call). Arg estimator must be of data type consistent with encoded sequence data type in ‘enc_data’.

Vectorized initialization of SequenceEncodableProbabilityDistribution corresponding to ‘estimator’ from enc_data. Observations in the encoded sequence enc_data are kept with probability p.

This functions relies on calls to SequenceEncodableStatisticAccumulator.seq_initialize(), which is a vectorized initialization of the SequenceEncodableStatisticAccumulator object.

This method should produce the same initialized model as a call to initialize() if the data sets are the same.

E = Union[List[Tuple[int, EncodedDataSequence]], RDD]

Parameters

- **enc_data** (*E*) – Sequence encoded data of format matching output of `seq_encode()` function.
- **estimator** (*ParameterEstimator*) – Model to be estimated from ‘enc_data’.
- **rng** (*RandomState*) – *RandomState* object for setting seed.
- **p** (*float*) – Proportion of data to randomly sample for initializing model.

Return type*dmx.stats.pdist.SequenceEncodableProbabilityDistribution***Returns***SequenceEncodableProbabilityDistribution***Parameters**

- **enc_data** (*List[Tuple[int, EncodedDataSequence]] | RDD*)
- **estimator** (*ParameterEstimator*)
- **rng** (*RandomState*)
- **p** (*float*)

1.3.2 Core interfaces and model guides

Abstract Classes

dmx-learn captures most distributions in the exponential family. A detailed walkthrough on defining a custom distribution class can be found in *User Defined Classes*. We list the abstract classes that exist in dmx-learn below.

ProbabilityDistribution

class `dmx.stats.pdist.ProbabilityDistribution`

Defines ProbabilityDistribution Abstract Class.

Note

This is generally used as an inherited class for *SequenceEncodableProbabilityDistribution*.

__init__()

Initialize the probability distribution.

Return type

None

abstractmethod density(*x*)

Compute the probability density at *x*.

Parameters

x – Input value to evaluate density.

Return type

float

Returns

Probability density value.

Parameters**x** (*Any*)**abstractmethod estimator**(*pseudo_count=None*)

Create a parameter estimator for this distribution.

Parameters**pseudo_count** (*Optional[float]*) – Regularize sufficient statistics in the estimation step.**Return type***dmx.stats.pdist.ParameterEstimator***Returns**

ParameterEstimator

Parameters**pseudo_count** (*float | None*)**abstractmethod log_density**(*x*)

Evaluate the log-density of distribution.

Return type

float

Returns

float

Parameters**x** (*Any*)**abstractmethod sampler**(*seed=None*)

Create a sampler for a probability distribution.

Parameters**seed** (*Optional[int]*) – Set seed for drawing samples from distribution.**Return type***dmx.stats.pdist.DistributionSampler***Parameters****seed** (*int | None*)**SequenceEncodableProbabilityDistribution****class** *dmx.stats.pdist.SequenceEncodableProbabilityDistribution*

Extends the ProbabilityDistribution to handle vectorized calls.

abstractmethod dist_to_encoder()

Create a data sequence encoder for this distribution.

Return type*dmx.stats.pdist.DataSequenceEncoder***Returns**

DataSequenceEncoder

seq_ld_lambda()

Legacy method stub for compatibility.

Returns the vectorized log-density callable list used by older helpers.

Return type

list[typing.Any]

abstractmethod seq_log_density(x)

Vectorized evaluation of the log density.

Parameters**x** – Encoded sequence for the corresponding probability distribution.**Return type**

numpy.ndarray

Returns

np.ndarray

Parameters**x** (Any)**seq_log_density_lambda()**

Return a list containing the sequence log density method.

Return type

list[typing.Any]

Returns*List with single element* – the seq_log_density method.**DistributionSampler****class dmx.stats.pdist.DistributionSampler(dist, seed=None)**

DistributionSampler is an Abstract class for distribution samplers.

Parameters

- **dist** (Any)
- **seed** (int | None)

dist

Distribution to sample from.

rng

Random number generator.

Type

RandomState

__init__(dist, seed=None)

Initialize DistributionSampler.

Parameters

- **dist** – Distribution to sample from.
- **seed** (Optional[int]) – Used to set seed on rng.

Parameters

- **dist** (Any)
- **seed** (int | None)

Return type

None

new_seed()

Generate a new random seed from the random number generator.

Return type

int

Returns

A new random seed integer.

abstractmethod sample(size=None)

Generate samples from distribution.

Parameters

size (*Optional[int]*) – Number of samples to generate.

Return type

typing.Any

Returns

Samples from distribution.

Parameters

size (*int* | *None*)

ConditionalSampler

class dmx.stats.pdist.ConditionalSampler

AbstractClass for ConditionalSampler.

Note

This is only implemented for samples of conditional distributions.

abstractmethod sample_given(x)

Sample at conditional value.

Parameters

x (*Any*) – Conditioned on x, sample from dist.

Return type

typing.Any

Returns

Sample from conditional distribution.

Parameters

x (*Any*)

StatisticAccumulator

class dmx.stats.pdist.StatisticAccumulator

Abstract base class for sufficient statistic accumulators.

Accumulators maintain and update sufficient statistics for parameter estimation.

Type Parameters:

SS: Type of the sufficient statistics.

abstractmethod combine(*suff_stat*)

Method for combining aggregated sufficient statistics.

Parameters

suff_stat (*SS*) – Sufficient statistics.

Return type

dmx.stats.pdist.StatisticAccumulator

Returns

None

Parameters

suff_stat (*SS*)

abstractmethod from_value(*x*)

Set sufficient statistics equal to passed value.

Parameters

x (*SS*) – Generic sufficient statistic for instance of StatisticAccumulator.

Return type

dmx.stats.pdist.SequenceEncodableStatisticAccumulator

Parameters

x (*SS*)

initialize(*x*, *weight*, *rng*)

Initialize sufficient statistics for a single data observation.

Note

Used for debugging only.

Parameters

- **x** (*Any*) – Data type corresponding to StatisticAccumulator object.
- **weight** (*float*) – Weight associated with single observation.
- **rng** – Seed for initialization. Unused in the base implementation.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **rng** (*Any*)

abstractmethod key_merge(*stats_dict*)

Merge sufficient statistics with matching keys.

Parameters

stats_dict (*Dict[str, Any]*) – Dict mapping keys to sufficient statistic values or accumulators.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**abstractmethod key_replace**(*stats_dict*)

Set sufficient statistics of accumulator instance to key'd values.

Parameters**stats_dict** (*Dict[str, Any]*) – Dict mapping keys to sufficient statistic values or accumulators.**Return type**

None

Parameters**stats_dict** (*Dict[str, Any]*)**abstractmethod update**(*x, weight, estimate*)

Accumulate sufficient statistics for a single data observation.

Note

Used for debugging only.

Parameters

- **x** (*Any*) – Data type corresponding to StatisticAccumulator object.
- **weight** (*float*) – Weight associated with single observation.
- **estimate** – Previous estimate of distribution.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **estimate** (*Any*)

abstractmethod value()

Return sufficient statistics of StatisticAccumulator.

Return type`typing.TypeVar(SS)`**SequenceEncodableStatisticAccumulator****class** `dmx.stats.pdist.SequenceEncodableStatisticAccumulator`

Statistic accumulator with support for sequence-based updates.

Extends StatisticAccumulator to handle vectorized updates over sequences of encoded data.

Type Parameters:

SS: Type of the sufficient statistics.

abstractmethod acc_to_encoder()

Create a data sequence encoder for this accumulator.

Return type*dmx.stats.pdist.DataSequenceEncoder***get_seq_lambda()**

Legacy method stub for compatibility.

Returns the vectorized update callable list used by older helpers.

Return type*list[typing.Any]***abstractmethod seq_initialize(*x, weights, rng*)**

Vectorized initialization of sufficient statistics.

Parameters

- **x** – Encoded sequence for this accumulator type.
- **weights** (*np.ndarray*) – weights for observations.
- **rng** – RandomState used to set the initialization seed.

Return type*None***Parameters**

- **x** (*Any*)
- **weights** (*ndarray*)
- **rng** (*Any*)

abstractmethod seq_update(*x, weights, estimate*)

Vectorized accumulation of sufficient statistics for EM updates.

Parameters

- **x** – Encoded sequence for this accumulator type.
- **weights** (*np.ndarray*) – weights for observations.
- **estimate** – Optional previous estimate of distribution.

Return type*None***Parameters**

- **x** (*Any*)
- **weights** (*ndarray*)
- **estimate** (*Any*)

ParameterEstimator**class dmx.stats.pdist.ParameterEstimator(*args)**

Abstract class for ParameterEstimator object.

Parameters*args (Any)*

abstractmethod `__init__(*args)`

Initialize the ParameterEstimator.

Parameters

***args** – Variable length argument list for initialization.

Parameters

args (*Any*)

Return type

None

abstractmethod `accumulator_factory()`

Create SequenceEncodableStatisticAccumulator object.

Return type

`dmx.stats.pdist.StatisticAccumulatorFactory`

abstractmethod `estimate(nobs, suff_stat)`

Estimate a probability distribution from sufficient statistics.

Parameters

- **nobs** (*Optional[float]*) – Weighted number of observations.
- **suff_stat** (*Tuple[int, np.ndarray, np.ndarray, np.ndarray]*) – Sufficient statistics for a Dirichlet distribution.

Return type

`dmx.stats.pdist.SequenceEncodableProbabilityDistribution`

Returns

SequenceEncodableProbabilityDistribution

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*SS*)

DataSequenceEncoder

class `dmx.stats.pdist.DataSequenceEncoder`

Abstract base class for encoding data sequences.

Encoders transform raw data sequences into encoded representations suitable for probability distribution operations.

abstractmethod `seq_encode(x)`

Create an encoded sequence from IID observations.

Parameters

x (*Any*) – Sequence of observations from corresponding distribution.

Return type

`dmx.stats.pdist.EncodedDataSequence`

Returns

EncodedDataSequence

Parameters

x (*Any*)

EncodedDataSequence

class dmx.stats.pdist.**EncodedDataSequence**(*data*)

Container for encoded data sequences.

EncodedDataSequence is the output data structure from DataSequenceEncoder. This object is used for vectorized functions and type checks.

Parameters

data (*Any*)

data

The encoded data for vectorized calls.

__init__(*data*)

Create instance of EncodedDataSequence.

Parameters

data – Store the data encoded for vectorized calls.

Parameters

data (*Any*)

Return type

None

Base Distributions

Below is a list of distributions arranged by data type. This should help pick a suitable probability distribution for you data. Note that there may be restrictions on the support of each distribution. This information will be contained in the associated references for each individual distribution.

Data Type: int

Binomial

Data Type: int

The binomial distribution is used for modeling the number of successes for a given number of trials n . The distribution has support on the integers between $[min_val, min_val + n]$, where min_val is supplied by the user.

The probability mass function is given by

$$f(x|n, p) = \binom{n}{x} p^x (1 - p)^{n-x}, \quad min_val \leq x < min_val + n.$$

BinomialDistribution

class dmx.stats.binomial.**BinomialDistribution**(*p, n, min_val=None, name=None, keys=None*)

Represent a shifted binomial distribution.

If min_val is omitted, the support is $\{0, \dots, n\}$. Otherwise the support is $\{min_val, \dots, min_val + n\}$. The probability mass function is that of $x - min_val$ under a binomial distribution with n trials and success probability p .

Parameters

- **p** (*float*)
- **n** (*int*)

- **min_val** (*int* / *None*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

p

Probability of success, strictly between zero and one.

Type
float

log_p

Logarithm of p.

Type
float

log_1p

Logarithm of 1-p.

Type
float

n

Nonnegative number of trials.

Type
int

min_val

Lower endpoint of the support, or *None* for zero.

Type
Optional[int]

name

Name of the distribution.

Type
Optional[str]

keys

Key for identifying equivalent distributions.

Type
Optional[str]

__init__ (*p*, *n*, *min_val=None*, *name=None*, *keys=None*)

Initialize BinomialDistribution.

Parameters

- **p** – Probability of success, strictly between zero and one.
- **n** – Nonnegative number of trials.
- **min_val** – Lower endpoint of the support. *None* is equivalent to zero.
- **name** – Optional name for the distribution.
- **keys** – Optional key for tying sufficient statistics.

Raises

ValueError – If *p* is not strictly between zero and one, or if *n* is negative or non-finite.

Parameters

- *p* (*float*)
- *n* (*int*)
- *min_val* (*int* | *None*)
- *name* (*str* | *None*)
- *keys* (*str* | *None*)

Return type

None

density(*x*)

Return the probability mass at integer value *x*.

Parameters

x – Integer observation in the shifted support.

Return type

float

Returns

Probability mass at *x*.

Parameters

x (*int*)

dist_to_encoder()

Return a *BinomialDataEncoder*.

Return type

`dmx.stats.binomial.BinomialDataEncoder`

estimator(*pseudo_count=None*)

Create an estimator initialized from this distribution.

With *pseudo_count*, the estimator uses this distribution's *n*, *min_val*, and expected success count as prior information. Without it, the estimator infers the support endpoints from accumulated observations.

Parameters

pseudo_count (*Optional[float]*, *optional*) – Pseudo-count for prior. Defaults to *None*.

Return type

`dmx.stats.binomial.BinomialEstimator`

Returns

BinomialEstimator – Estimator object.

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Return the log-probability mass at integer value *x*.

Parameters

x – Integer observation in the shifted support.

Return type

float

Returns

Log-probability mass at **x**.

Parameters

x (*int*)

sampler (*seed=None*)

Return a BinomialSampler for this distribution.

Parameters

seed (*Optional[int], optional*) – Seed for RNG. Defaults to None.

Return type

dmx.stats.binomial.BinomialSampler

Returns

BinomialSampler – Sampler for this distribution.

Parameters

seed (*int | None*)

seq_log_density (*x*)

Evaluate log-probability masses for an encoded sequence.

Parameters

x – Encoded sequence containing N observations.

Return type

numpy.ndarray

Returns

Array of shape (N,) containing one log-probability mass per observation.

Parameters

x (*BinomialEncodedDataSequence*)

BinomialEstimator

```
class dmx.stats.binomial.BinomialEstimator(max_val=None, min_val=0, pseudo_count=None,  
                                           suff_stat=None, name=None, keys=None)
```

Estimate a shifted binomial distribution from weighted statistics.

The trial count is `max_val - min_val`. The success probability is the shifted weighted mean divided by that trial count. A pseudo-count either shrinks the success count toward `suff_stat` or, when `suff_stat` is absent, toward probability 0.5. Empty or zero-width data also produce probability 0.5.

Parameters

- **max_val** (*int | None*)
- **min_val** (*int | None*)
- **pseudo_count** (*float | None*)
- **suff_stat** (*float | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

max_val

Max value encountered.

Type
Optional[int]

min_val

Min value for BinomialDistribution.

Type
Optional[int]

pseudo_count

Pseudo-count for prior.

Type
Optional[float]

suff_stat

Sufficient statistic for prior.

Type
Optional[float]

name

Name of the estimator.

Type
Optional[str]

keys

Key for merging estimators.

Type
Optional[str]

__init__ (*max_val=None, min_val=0, pseudo_count=None, suff_stat=None, name=None, keys=None*)

Initialize BinomialEstimator.

Parameters

- **max_val** (*Optional[int], optional*) – Max value encountered. Defaults to None.
- **min_val** (*Optional[int], optional*) – Min value for BinomialDistribution. Defaults to 0.
- **pseudo_count** (*Optional[float], optional*) – Pseudo-count for prior. Defaults to None.
- **suff_stat** (*Optional[float], optional*) – Sufficient statistic for prior. Defaults to None.
- **name** (*Optional[str], optional*) – Name of the estimator. Defaults to None.
- **keys** (*Optional[str], optional*) – Key for merging estimators. Defaults to None.

Raises

TypeError – If keys is not a string or None.

Parameters

- **max_val** (*int | None*)
- **min_val** (*int | None*)
- **pseudo_count** (*float | None*)
- **suff_stat** (*float | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

accumulator_factory()

Return a BinomialAccumulatorFactory.

Return type`dmx.stats.binomial.BinomialAccumulatorFactory`**estimate(nobs, suff_stat)**

Estimate a BinomialDistribution from sufficient statistics.

Parameters

- **nobs** (*Optional[float]*) – Not used.
- **suff_stat** (*Tuple[float, float, Optional[int], Optional[int]]*) – (count, sum, min_val, max_val).

Return type`dmx.stats.binomial.BinomialDistribution`**Returns***BinomialDistribution* – Estimated distribution.**Parameters**

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[float, float, int | None, int | None]*)

BinomialSampler**class** `dmx.stats.binomial.BinomialSampler`(*dist, seed=None*)

Sampler for BinomialDistribution.

Parameters

- **dist** (*BinomialDistribution*)
- **seed** (*int | None*)

dist

Distribution to sample from.

Type*BinomialDistribution***rng**

Random number generator.

Type

RandomState

sample(size=None)

Draw samples from the distribution.

Parameters**size** (*Optional[int], optional*) – Number of samples to draw. Defaults to None.**Return type**`typing.Union[int, typing.List[int]]`

Returns*Union[int, List[int]]* – Single sample or list of samples.**Parameters****size** (*int* | *None*)**Geometric**

Data Type: int

The geometric distribution is used to model the number of trials until the first success. The probability mass function is given by

$$f(x|n, p) = p * (1 - p)^{n-1}, 1 \leq x.$$

For more info see [Geometric Distribution](#).

GeometricDistribution

class dmx.stats.geometric.**GeometricDistribution**(*p*, *name=None*, *keys=None*)

Represent a one-based geometric distribution.

The support is {1, 2, ...}, and p is the probability of success on each trial. The constructor clips p to the closed interval [0, 1].

Parameters

- **p** (*float*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

p

Success probability clipped to [0, 1].

Type

float

log_p

Log of probability of success p.

Type

float

log_1p

Log of 1-p (probability of failure).

Type

float

name

Name for the GeometricDistribution object.

Type

Optional[str]

keys

Key for parameter p.

Type

Optional[str]

__init__(*p*, *name=None*, *keys=None*)

Initialize GeometricDistribution.

Parameters

- **p** – Success probability; values outside `[0, 1]` are clipped.
- **name** – Optional name for the distribution.
- **keys** – Optional key for tying sufficient statistics.

Parameters

- **p** (*float*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

Return type

None

density(*x*)

Evaluate the probability mass at an integer observation.

Parameters

x (*int*) – Observed geometric value (1,2,3,...).

Return type

float

Returns

Probability mass at **x**.

Parameters

x (*int*)

dist_to_encoder()

Return a GeometricDataEncoder for this distribution.

Return type

`dmx.stats.geometric.GeometricDataEncoder`

Returns

GeometricDataEncoder – Encoder object.

estimator(*pseudo_count=None*)

Create an estimator initialized from this distribution.

With *pseudo_count*, estimation shrinks the weighted success count toward this distribution's success probability.

Parameters

pseudo_count (*Optional[float]*, *optional*) – Pseudo-count for regularization.

Return type

`dmx.stats.geometric.GeometricEstimator`

Returns

GeometricEstimator – Estimator object.

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Evaluate the log-probability mass at an integer observation.

Parameters

x (*int*) – Must be a natural number (1,2,3,...).

Return type

float

Returns

Log-probability mass at **x**.

Parameters

x (*int*)

sampler(*seed=None*)

Return a GeometricSampler for this distribution.

Parameters

seed (*Optional[int], optional*) – Seed for random number generator.

Return type

`dmx.stats.geometric.GeometricSampler`

Returns

GeometricSampler – Sampler object.

Parameters

seed (*int | None*)

seq_log_density(*x*)

Evaluate log-probability masses for an encoded sequence.

Parameters

x – Encoded sequence containing N observations.

Return type

`numpy.ndarray`

Returns

Array of shape (N,) containing one log-probability mass per observation.

Parameters

x (*GeometricEncodedDataSequence*)

GeometricEstimator

```
class dmx.stats.geometric.GeometricEstimator(pseudo_count=None, suff_stat=None, name=None,  
                                              keys=None)
```

Estimate a geometric success probability from weighted statistics.

Without prior information, the maximum-likelihood estimate is $p = \text{count} / \text{sum_x}$. A pseudo-count adds weighted prior successes from **suff_stat**; when no prior statistic is supplied, it adds the same amount to the numerator and denominator.

Parameters

- **pseudo_count** (*float | None*)
- **suff_stat** (*float | None*)
- **name** (*str | None*)

- **keys** (*str* | *None*)

pseudo_count

Pseudo-count for regularization.

Type

Optional[float]

suff_stat

Probability of success (value between (0,1)).

Type

Optional[float]

name

Name for the estimator.

Type

Optional[str]

keys

Key for merging sufficient statistics.

Type

Optional[str]

__init__ (*pseudo_count=None, suff_stat=None, name=None, keys=None*)

Initialize GeometricEstimator.

Parameters

- **pseudo_count** (*Optional[float], optional*) – Pseudo-count for regularization.
- **suff_stat** (*Optional[float], optional*) – Probability of success (value between (0,1)).
- **name** (*Optional[str], optional*) – Name for the estimator.
- **keys** (*Optional[str], optional*) – Key for merging sufficient statistics.

Raises

TypeError – If keys is not a string or None.

Parameters

- **pseudo_count** (*float* | *None*)
- **suff_stat** (*float* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

Return type

None

accumulator_factory()

Return a GeometricAccumulatorFactory for this estimator.

Return type

`dmx.stats.geometric.GeometricAccumulatorFactory`

Returns

GeometricAccumulatorFactory – Factory object.

estimate(*nobs*, *suff_stat*)

Estimate a GeometricDistribution from sufficient statistics.

Parameters

- **nobs** (*Optional[float]*) – Number of observations (not used).
- **suff_stat** (*Tuple[float, float]*) – (count, sum) sufficient statistics.

Return type

dmx.stats.geometric.GeometricDistribution

Returns

GeometricDistribution – Estimated distribution.

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[float, float]*)

GeometricSampler

class *dmx.stats.geometric.GeometricSampler*(*dist*, *seed=None*)

Sampler for the geometric distribution.

Parameters

- **dist** (*GeometricDistribution*)
- **seed** (*int | None*)

rng

RandomState with seed set for sampling.

Type

RandomState

dist

GeometricDistribution to sample from.

Type

GeometricDistribution

sample(*size=None*)

Generate iid samples from geometric distribution.

Parameters

size (*Optional[int]*, *optional*) – Number of iid samples to draw. If None, returns a single sample.

Return type

typing.Union[int, numpy.ndarray]

Returns

Union[int, np.ndarray] – Single sample (int) or numpy array of ints.

Parameters

size (*int | None*)

Poisson

Data Type: int

The Poisson distribution is used to model counts. The probability mass function is given by

$$f(x|\lambda) = \frac{\lambda^x e^{-x}}{x!}, \quad 0 \leq x.$$

For more info see [Poisson Distribution](#).

PoissonDistribution

class dmx.stats.poisson.PoissonDistribution(*lam*, *name=None*, *keys=None*)

Represent a Poisson distribution with rate *lam*.

For nonnegative integer *x*,

$$\log p(x \mid \lambda) = x \log \lambda - \log(x!) - \lambda.$$

Parameters

- **lam** (*float*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

lam

Mean of Poisson distribution.

Type

float

name

String name for object instance.

Type

Optional[str]

log_lambda

Log of attribute lam.

Type

float

keys

Keys for lambda.

Type

Optional[str]

__init__(*lam*, *name=None*, *keys=None*)

Initialize a Poisson distribution.

Parameters

- **lam** (*float*) – Positive real-valued number.
- **name** (*Optional[str]*) – String name for object instance.
- **keys** (*Optional[str]*) – Key for lambda.

Parameters

- **lam** (*float*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

Return type

None

density(*x*)Evaluate the density of Poisson distribution at observation *x*.**Notes**See `log_density()`.**Parameters***x* (*int*) – Must be a non-negative integer value (0,1,2,...).**Return type**

float

Returns*float* – Density of Poisson distribution evaluated at *x*.**Parameters***x* (*int*)**dist_to_encoder**()

Create an encoder for nonnegative count observations.

Return type`dmx.stats.poisson.PoissonDataEncoder`**estimator**(*pseudo_count=None*)

Create an estimator centered on this rate when regularized.

Return type`dmx.stats.poisson.PoissonEstimator`**Parameters****pseudo_count** (*float* / *None*)**log_density**(*x*)Evaluate the Poisson log density at *x*.

$$\log f(x| \lambda) = -x \log \lambda - \lambda + \log x!$$

Parameters*x* (*int*) – Must be a non-negative integer value (0,1,2,...).**Return type**

float

Returns

float – Log-density of Poisson distribution evaluated at *x*.

Parameters

x (*int*)

sampler (*seed=None*)

Create a sampler, optionally initialized with *seed*.

Return type

dmx.stats.poisson.PoissonSampler

Parameters

seed (*int* | *None*)

seq_log_density (*x*)

Evaluate log densities for an encoded sequence.

Both encoded arrays and the returned array have shape *(n,)*.

Return type

numpy.ndarray

Parameters

x (*PoissonEncodedDataSequence*)

PoissonEstimator

```
class dmx.stats.poisson.PoissonEstimator(pseudo_count=None, suff_stat=None, name=None,
                                         keys=None)
```

Estimate a Poisson rate from weighted sufficient statistics.

When both regularization arguments are present, *pseudo_count* adds that much effective weight at the target rate *suff_stat*.

Parameters

- **pseudo_count** (*float* | *None*)
- **suff_stat** (*float* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

pseudo_count

Re-weight *suff_stat*.

Type

Optional[*float*]

suff_stat

Mean of Poisson if not *None*.

Type

Optional[*float*]

name

String name of *PoissonEstimator* instance.

Type

Optional[*str*]

keys

String keys of PoissonEstimator instance for combining sufficient statistics.

Type

Optional[str]

__init__(*pseudo_count=None, suff_stat=None, name=None, keys=None*)

Initialize the estimator and optional rate regularization.

Parameters

- **pseudo_count** (*Optional[float]*) – Optional non-negative float.
- **suff_stat** (*Optional[float]*) – Optional non-negative float.
- **name** (*Optional[str]*) – Assign a name to PoissonEstimator.
- **keys** (*Optional[str]*) – Assign keys to PoissonEstimator for combining sufficient statistics.

Parameters

- **pseudo_count** (*float | None*)
- **suff_stat** (*float | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

accumulator_factory()

Create a matching accumulator factory.

Return type

`dmx.stats.poisson.PoissonAccumulatorFactory`

estimate(*nobs, suff_stat*)

Estimate the rate from a (count, sum) statistic.

The nobs argument is overwritten by the statistic count.

Return type

`dmx.stats.poisson.PoissonDistribution`

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[float, float]*)

PoissonSampler

class `dmx.stats.poisson.PoissonSampler`(*dist, seed=None*)

Draw independent samples from a Poisson distribution.

Parameters

- **dist** (`PoissonDistribution`)
- **seed** (*int | None*)

rng

RandomState with seed set for sampling.

Type

RandomState

dist

PoissonDistribution to sample from.

Type

GeometricDistribution

sample(*size=None*)

Generate iid samples from Poisson distribution.

Generates a single Poisson sample (int) if size is None, else a numpy array of integers of length size containing iid samples, from the Poisson distribution.

Parameters

size (*Optional[int]*) – Number of iid samples to draw. If None, assumed to be 1.

Return type

typing.Union[int, typing.Sequence[int]]

Returns

If size is None, int, else size length numpy array of ints.

Parameters

size (*int | None*)

Integer Categorical

Data Type: int

The Integer Categorical distribution is a categorical distribution defined on an integer support. The probability mass function is given by

$$f(x|\mathbf{p}) = \begin{cases} p_i, & x = k \\ 0, & x \notin [\min_val, \min_val + n) \end{cases}$$

where $\sum_i p_i = 1$ and n is a user-defined length. A categorical distribution defined over sets of objects can be used if the user does not want map a given set of values to the integers (see [Categorical Distribution](#).)

For more info see [Integer Categorical Distribution](#).

IntegerCategoricalDistribution

class dmx.stats.inrange.IntegerCategoricalDistribution(*min_val, p_vec, name=None, keys=None*)

Represent a categorical distribution on a contiguous integer range.

Entry i of $\mathbf{p_vec}$ is the mass for category $\min_val + i$; the inclusive support ends at $\min_val + \text{len}(\mathbf{p_vec}) - 1$.

Parameters

- **min_val** (*int*)
- **p_vec** (*List[float] | ndarray*)
- **name** (*str | None*)
- **keys** (*str | None*)

p_vec

Must sum to 1.0. First probability is probability for `p_mat(x_mat=min_val)`.

Type

`np.ndarray[float]`

min_val

Minimum value in support of integer categorical

Type

`int`

max_val

Maximum value in support of integer categorical set to `min_val + length(p_vec) - 1`.

Type

`int`

log_p_vec

Log of `p_vec`.

Type

`np.ndarray[float]`

num_vals

Total number of values in support of `IntegerCategoricalDistribution` instance.

Type

`int`

name

Name for object.

Type

`Optional[str]`

keys

Key for parameter.

Type

`Optional[str]`

__init__(*min_val*, *p_vec*, *name=None*, *keys=None*)

Initialize probabilities and the support origin.

Parameters

- **min_val** (*int*) – Minimum value of the integer categorical support.
- **p_vec** (*Union[List[float], np.ndarray]*) – Probability vector containing probability of each integer in the support range.
- **name** (*Optional[str]*) – Assign name to `IntegerCategoricalDistribution` object.
- **keys** (*Optional[str]*) – Key for parameter.

Parameters

- **min_val** (*int*)
- **p_vec** (*List[float] | ndarray*)
- **name** (*str | None*)

- **keys** (*str* / *None*)

Return type*None***density**(*x*)

Evaluate the density of the integer categorical at observation *x*.

Parameters

x (*int*) – Integer value.

Return type*float***Returns**

float – Density at *x*.

Parameters

x (*int*)

dist_to_encoder()

Create the compatible integer encoder.

Return type

dmx.stats.intrange.IntegerCategoricalDataEncoder

estimator(*pseudo_count=None*)

Create an estimator, optionally using current probabilities as a prior.

Return type

dmx.stats.intrange.IntegerCategoricalEstimator

Parameters

pseudo_count (*float* / *None*)

log_density(*x*)

Evaluate the log-density of the integer categorical at observation *x*.

Parameters

x (*int*) – Integer value.

Return type*float***Returns**

float – Log-density at *x*.

Parameters

x (*int*)

sampler(*seed=None*)

Create a sampler for this distribution.

Return type

dmx.stats.intrange.IntegerCategoricalSampler

Parameters

seed (*int* / *None*)

seq_log_density(*x*)

Evaluate log masses for an encoded array of *N* integers.

Return type`numpy.ndarray`**Returns**

Array of shape (N,) with one log mass per observation.

Parameters**x** (*IntegerCategoricalEncodedDataSequence*)**IntegerCategoricalEstimator**

```
class dmx.stats.inrange.IntegerCategoricalEstimator(min_val=None, max_val=None,
                                                    pseudo_count=None, suff_stat=None,
                                                    name=None, keys=None)
```

Estimate an integer categorical distribution from weighted counts.

Notes

Must set either min_val and max_val, or suff_stat must be passed as arg.

Parameters

- **min_val** (*int* / *None*)
- **max_val** (*int* / *None*)
- **pseudo_count** (*float* / *None*)
- **suff_stat** (*Tuple[int, ndarray]* / *None*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

min_val

Minimum value of integer categorical.

Type

Optional[int]

max_val

Maximum value of integer categorical.

Type

Optional[int]

pseudo_count

Used to re-weight suff_stat when merged with new aggregated data.

Type

Optional[float]

suff_stat

min value and prob vec

Type

Tuple[int, np.ndarray]

name

Name to IntegerCategoricalEstimator object.

Type

Optional[str]

keys

Keys for accumulating merging statistics of IntegerCategoricalAccumulator objects.

Type

Optional[str]

__init__(*min_val=None, max_val=None, pseudo_count=None, suff_stat=None, name=None, keys=None*)

Initialize support bounds, smoothing, and metadata.

Parameters

- **min_val** (*Optional[int]*) – Set minimum value of integer categorical.
- **max_val** (*Optional[int]*) – Set maximum value of integer categorical.
- **pseudo_count** (*Optional[float]*) – Used to re-weight suff_stat member variables in merging of sufficient statistics
- **suff_stat** – Set sufficient statistics. See above for details.
- **name** (*Optional[str]*) – Assign a name to IntegerCategoricalEstimator object.
- **keys** (*Optional[str]*) – Set keys for accumulating merging statistics of IntegerCategoricalAccumulator objects.

Parameters

- **min_val** (*int | None*)
- **max_val** (*int | None*)
- **pseudo_count** (*float | None*)
- **suff_stat** (*Tuple[int, ndarray] | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

accumulator_factory()

Create a compatible accumulator factory.

Return type`dmx.stats.intrange.IntegerCategoricalAccumulatorFactory`**estimate**(*nobs, suff_stat*)

Estimate probabilities from (minimum, count_vector) statistics.

Return type`dmx.stats.intrange.IntegerCategoricalDistribution`**Parameters**

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[int, ndarray] | None*)

IntegerCategoricalSampler

class dmx.stats.inrange.IntegerCategoricalSampler(*dist*, *seed=None*)

Draw integers from an integer categorical distribution.

Parameters

- **dist** (*IntegerCategoricalDistribution*)
- **seed** (*int* | *None*)

dist

IntegerCategoricalDistribution instance to sample from.

Type

IntegerCategoricalDistribution

rng

RandomState object with seed set if passed.

Type

RandomState

sample(*size=None*)

Draw iid samples from IntegerCategoricalSampler object.

Notes

If size is None, a single sample is returned as an integer. If size > 0, a List of integers with length equal to size is returned.

Parameters

size (*Optional[int]*) – Number of iid samples to draw.

Return type

typing.Union[int, typing.List[int]]

Returns

Integer or List[int] of iid samples from IntegerCategoricalSampler instance.

Parameters

size (*int* | *None*)

Spike and Slab Distribution

Data Type: int

The spike and slab distribution places a spike of probability p on integer value k in the range of values $[min_val, max_val]$. The remaining $n-1$ follow a uniform distribution. This distribution is great for cases where you encounter integer valued data with a spike on certain values in the range.

$$f(x|k, p) = \begin{cases} p, & x = k \\ \frac{1-p}{n-1}, & x \neq k \text{ \& } x \in [min_val, max_val] \\ 0, & else \end{cases}$$

In the above we have assumed the length of $[min_val, max_val]$ is n .

SpikeAndSlabDistribution

```
class dmx.stats.int_spike.SpikeAndSlabDistribution(k, num_vals, p, min_val=0, name=None,
                                                  keys=None)
```

Represent a point spike and uniform slab over integers.

Sampling uses integers in $[\text{min_val}, \text{min_val} + \text{num_vals})$. For legacy compatibility, scalar and sequence scoring also treat the cached endpoint $\text{min_val} + \text{num_vals}$ as an in-range slab value.

Parameters

- **k** (*int*)
- **num_vals** (*int*)
- **p** (*float*)
- **min_val** (*int* / *None*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

p

Probability of drawing from k.

Type

float

min_val

Lower bound for the range.

Type

int

max_val

Max value for the range.

Type

int

k

Integer to place the spike on.

Type

int

log_p

Log of p.

Type

float

log_1p

Log of 1-p

Type

float

num_vals

Total number of integers in range.

Type
int

name

Name for object instance.

Type
Optional[str]

keys

Key for parameters.

Type
Optional[str]

__init__(*k, num_vals, p, min_val=0, name=None, keys=None*)

Initialize the spike, support, and component probability.

Parameters

- **k** (*int*) – Integer value to place spike on. Must be within [min_val,min_val+num_vals)
- **num_vals** (*int*) – Number of integers in the range.
- **p** (*float*) – Probability of drawing k. (1-p)/(num_vals-1) to draw any other integer in range.
- **min_val** (*Optional[int]*) – Defaults to 0. Set bottom of integer range.
- **name** (*Optional[str]*) – Set name for object.
- **keys** (*Optional[str]*) – Key for parameters.

Parameters

- **k** (*int*)
- **num_vals** (*int*)
- **p** (*float*)
- **min_val** (*int | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type
None

density(*x*)

Evaluate the probability mass at integer x.

Return type
float

Parameters
x (*int*)

dist_to_encoder()

Create the compatible integer encoder.

Return type
dmx.stats.int_spike.SpikeAndSlabDataEncoder

estimator(*pseudo_count=None*)

Create an estimator retaining support and optional smoothing.

Return type

dmx.stats.int_spike.SpikeAndSlabEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Evaluate the log-probability mass at integer *x*.

Return type

float

Parameters

x (*int*)

sampler(*seed=None*)

Create a sampler for this distribution.

Return type

dmx.stats.int_spike.SpikeAndSlabSampler

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Evaluate log masses for an encoded array of *N* integers.

Return type

numpy.ndarray

Parameters

x (*SpikeAndSlabEncodedDataSequence*)

SpikeAndSlabEstimator

class *dmx.stats.int_spike.SpikeAndSlabEstimator*(*min_val=None, max_val=None, pseudo_count=None, suff_stat=None, name=None, keys=None*)

Estimate a spike location and mass from contiguous integer counts.

Parameters

- **min_val** (*int* | *None*)
- **max_val** (*int* | *None*)
- **pseudo_count** (*float* | *None*)
- **suff_stat** (*Tuple[int, float | None]* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

pseudo_count

Regularize value *k*.

Type

Optional[float]

min_val

Smallest integer value in the range. Defaults to 0.

Type

int

max_val

Set to the min val plus number of values - 1.

Type

int

suff_stat

Tuple of k to regularize and optional value of p for k.

Type

Optional[Tuple[int, Optional[float]]]

name

Set name for object instance.

Type

Optional[str]

keys

Set keys for object instance.

Type

Optional[str]

__init__(*min_val=None, max_val=None, pseudo_count=None, suff_stat=None, name=None, keys=None*)

Initialize support, pseudo-count settings, and metadata.

Parameters

- **min_val** (*Optional[int]*) – Smallest integer value in the range.
- **max_val** (*Optional[int]*) – Largest represented integer value.
- **pseudo_count** (*Optional[float]*) – Regularize value k.
- **suff_stat** (*Optional[Tuple[int, Optional[float]]]*) – Tuple of k to regularize and optional value of p for k.
- **name** (*Optional[str]*) – Set name for object instance.
- **keys** (*Optional[str]*) – Set keys for object instance.

Parameters

- **min_val** (*int | None*)
- **max_val** (*int | None*)
- **pseudo_count** (*float | None*)
- **suff_stat** (*Tuple[int, float | None] | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

accumulator_factory()

Create a compatible accumulator factory.

Return type

`dmx.stats.int_spike.SpikeAndSlabAccumulatorFactory`

estimate(*nobs*, *suff_stat*)

Estimate a distribution from (*minimum*, *count_vector*).

Return type

`dmx.stats.int_spike.SpikeAndSlabDistribution`

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*int*, *ndarray*])

SpikeAndSlabSampler

class `dmx.stats.int_spike.SpikeAndSlabSampler`(*dist*, *seed=None*)

Draw integers from a spike-and-uniform-slab distribution.

Parameters

- **dist** (`SpikeAndSlabDistribution`)
- **seed** (*int* | *None*)

rng

`RandomState` for seeding samples.

Type

`RandomState`

dist

`SpikeAndSlabDistribution` to sample from.

Type

`SpikeAndSlabDistribution`

non_k

All integers outside of the spiked value 'k'.

Type

`np.ndarray`

sample(*size=None*)

Draw one integer or an integer array of shape (*size*,).

Return type

`typing.Union`[*int*, `numpy.ndarray`]

Parameters

size (*int* | *None*)

Data Type: float**Gaussian (Normal)**

Data Type: float

The Gaussian distribution is a symmetric bell-shaped distribution on the real line. The probability density function is given by

$$f(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}, \quad x \in \mathbb{R}.$$

For more info see [Gaussian Distribution](#).

GaussianDistribution

class `dmx.stats.gaussian.GaussianDistribution`(*mu*, *sigma2*, *name=None*, *keys=None*)

Represent a univariate Gaussian distribution.

The support is the real line. `mu` is the mean and `sigma2` is the variance, not the standard deviation. A nonpositive or non-finite variance is replaced with `1.0` by the constructor.

Parameters

- **mu** (*float*)
- **sigma2** (*float*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

mu

Mean of the Gaussian distribution.

Type

`float`

sigma2

Variance of the Gaussian distribution.

Type

`float`

name

Name of the object.

Type

`Optional[str]`

const

Normalizing constant of the Gaussian (depends on `sigma2`).

Type

`float`

log_const

Log of the normalizing constant.

Type

`float`

keys

Key for the distribution.

Type

`Optional[str]`

__init__(*mu*, *sigma2*, *name=None*, *keys=None*)

Initialize GaussianDistribution.

Parameters

- **mu** – Mean of the distribution.
- **sigma2** – Variance. Nonpositive, NaN, or infinite values are replaced with 1.0.
- **name** – Optional name for the distribution.
- **keys** – Optional key for tying sufficient statistics.

Parameters

- **mu** (*float*)
- **sigma2** (*float*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

Return type

None

density(*x*)

Evaluate the density of the Gaussian distribution at *x*.

Parameters

x – Scalar real-valued observation.

Return type

float

Returns

Probability density at *x*.

Parameters

x (*float*)

dist_to_encoder()

Return a GaussianDataEncoder for this distribution.

Return type

`dmx.stats.gaussian.GaussianDataEncoder`

Returns

GaussianDataEncoder – Encoder object.

estimator(*pseudo_count=None*)

Create an estimator initialized from this distribution.

With *pseudo_count*, the mean and variance estimates are both shrunk toward this distribution's corresponding parameter.

Parameters

pseudo_count (*Optional[float]*, *optional*) – Pseudo-count for regularization.

Return type

`dmx.stats.gaussian.GaussianEstimator`

Returns

GaussianEstimator – Estimator object.

Parameters**pseudo_count** (*float* | *None*)**log_density(*x*)**Evaluate the log-density of the Gaussian distribution at *x*.**Parameters****x** – Scalar real-valued observation.**Return type***float***Returns**Log-density at *x*.**Parameters****x** (*float*)**sampler**(*seed=None*)

Return a GaussianSampler for this distribution.

Parameters**seed** (*Optional[int]*, *optional*) – Seed for random number generator.**Return type***dmx.stats.gaussian.GaussianSampler***Returns***GaussianSampler* – Sampler object.**Parameters****seed** (*int* | *None*)**seq_ld_lambda()**

Return a list containing the seq_log_density method.

Return type*typing.List[typing.Callable]***seq_log_density(*x*)**

Vectorized log-density for encoded data.

Parameters**x** – Encoded sequence containing *N* observations.**Return type***numpy.ndarray***Returns**Array of shape (*N*,) containing one log-density per observation.**Parameters****x** (*GaussianEncodedDataSequence*)**GaussianEstimator**

```
class dmx.stats.gaussian.GaussianEstimator(pseudo_count=(None, None), suff_stat=(None, None),  
                                           name=None, keys=None)
```

Estimate a Gaussian mean and variance from weighted statistics.

The mean is the weighted first moment and the variance is the weighted second central moment. The two pseudo-counts independently shrink them toward the corresponding entries of `suff_stat`. Empty mean or variance statistics use zero, after which the distribution constructor normalizes a zero variance to one.

Parameters

- **pseudo_count** (*Tuple[float | None, float | None]*)
- **suff_stat** (*Tuple[float | None, float | None]*)
- **name** (*str | None*)
- **keys** (*str | None*)

pseudo_count

Weights for sufficient statistics.

Type

Tuple[Optional[float], Optional[float]]

suff_stat

Tuple of mean (μ) and variance (σ^2).

Type

Tuple[Optional[float], Optional[float]]

name

Name of the estimator.

Type

Optional[str]

keys

Key for mean and variance.

Type

Optional[str]

__init__ (*pseudo_count=(None, None), suff_stat=(None, None), name=None, keys=None*)

Initialize GaussianEstimator.

Parameters

- **pseudo_count** (*Tuple[Optional[float], Optional[float]]*) – Tuple of two positive floats.
- **suff_stat** (*Tuple[Optional[float], Optional[float]]*) – Tuple of mean and variance.
- **name** (*Optional[str], optional*) – Name for the estimator.
- **keys** (*Optional[str], optional*) – Key for mean and variance.

Raises

TypeError – If keys is not a string or None.

Parameters

- **pseudo_count** (*Tuple[float | None, float | None]*)
- **suff_stat** (*Tuple[float | None, float | None]*)
- **name** (*str | None*)
- **keys** (*str | None*)

accumulator_factory()

Return a GaussianAccumulatorFactory for this estimator.

Return type

`dmx.stats.gaussian.GaussianAccumulatorFactory`

Returns

GaussianAccumulatorFactory – Factory object.

estimate(nobs, suff_stat)

Estimate a GaussianDistribution from sufficient statistics.

Parameters

- **nobs** (*Optional[float]*) – Number of observations (not used).
- **suff_stat** (*Tuple[float, float, float, float]*) – (sum, sum2, count, count2) sufficient statistics.

Return type

`dmx.stats.gaussian.GaussianDistribution`

Returns

GaussianDistribution – Estimated distribution.

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[float, float, float, float]*)

GaussianSampler

class `dmx.stats.gaussian.GaussianSampler`(*dist, seed=None*)

Sampler for drawing samples from a GaussianDistribution instance.

Parameters

- **dist** (*GaussianDistribution*)
- **seed** (*int | None*)

dist

GaussianDistribution instance to sample from.

Type

GaussianDistribution

rng

Random number generator.

Type

RandomState

sample(size=None)

Draw iid samples from the Gaussian distribution.

Parameters

size (*Optional[int], optional*) – Number of samples to draw. If None, returns a single sample.

Return type

`typing.Union[float, numpy.ndarray]`

Returns

Union[float, np.ndarray] – Single sample or array of samples.

Parameters**size** (*int* / *None*)**Exponential**

Data Type: float

The exponential distribution is can be used to model arrival times between events. The distribution has support on the positive real line. The probability density function is given by

$$f(x|\beta) = \frac{1}{\beta} e^{-\frac{1}{\beta}x}, x \geq 0.$$

The above is the scale parameterization of the exponential distribution. For more info see [Exponential Distribution](#).

ExponentialDistribution**class** dmx.stats.exponential.**ExponentialDistribution**(*beta, name=None, keys=None*)

Represent an exponential distribution with scale beta.

The support is the nonnegative real line and the density is $\exp(-x / \text{beta}) / \text{beta}$. Thus the rate is $1 / \text{beta}$.

Parameters

- **beta** (*float*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

beta

Positive real number defining the scale of the exponential distribution.

Type

float

log_beta

Logarithm of the beta parameter.

Type

float

name

Name for the ExponentialDistribution object.

Type

Optional[str]

keys

Key for parameters.

Type

Optional[str]

__init__ (*beta, name=None, keys=None*)

Initialize ExponentialDistribution.

Parameters

- **beta** – Positive scale parameter.

- **name** – Optional name for the distribution.
- **keys** – Optional key for tying sufficient statistics.

Parameters

- **beta** (*float*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

Return type

None

density(*x*)Evaluate the density of the exponential distribution at *x*.**Parameters***x* – Nonnegative scalar observation.**Return type**

float

ReturnsProbability density at *x*, or zero below the support.**Parameters***x* (*float*)**dist_to_encoder()**

Return an ExponentialDataEncoder for this distribution.

Return type`dmx.stats.exponential.ExponentialDataEncoder`**Returns***ExponentialDataEncoder* – Encoder object.**estimator(*pseudo_count=None*)**

Create an estimator initialized from this distribution.

With *pseudo_count*, the estimate of *beta* is shrunk toward this distribution's scale.**Parameters****pseudo_count** (*Optional[float]*, *optional*) – Pseudo-count for regularization.**Return type**`dmx.stats.exponential.ExponentialEstimator`**Returns***ExponentialEstimator* – Estimator object.**Parameters****pseudo_count** (*float* / *None*)**log_density(*x*)**Evaluate the log-density of the exponential distribution at *x*.**Parameters***x* – Nonnegative scalar observation.**Return type**

float

Returns

Log-density at **x**, or negative infinity below the support.

Parameters

x (*float*)

sampler (*seed=None*)

Return an ExponentialSampler for this distribution.

Parameters

seed (*Optional[int], optional*) – Seed for random number generator.

Return type

dmx.stats.exponential.ExponentialSampler

Returns

ExponentialSampler – Sampler object.

Parameters

seed (*int | None*)

seq_log_density (*x*)

Vectorized log-density for encoded data.

Parameters

x – Encoded sequence containing N observations.

Return type

numpy.ndarray

Returns

Array of shape (N,) containing one log-density per observation.

Parameters

x (*ExponentialEncodedDataSequence*)

ExponentialEstimator

class *dmx.stats.exponential.ExponentialEstimator* (*pseudo_count=None, suff_stat=None, name=None, keys=None*)

Estimate an exponential scale from weighted sufficient statistics.

Without prior information, beta is the weighted sample mean. A pseudo-count shrinks that mean toward *suff_stat* when supplied, or toward one otherwise. Empty statistics produce scale one.

Parameters

- **pseudo_count** (*float | None*)
- **suff_stat** (*float | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

pseudo_count

Used to weight sufficient statistics.

Type

Optional[float]

suff_stat

Positive float value for scale of exponential distribution.

Type

Optional[float]

name

Name for the estimator.

Type

Optional[str]

keys

Key for combining sufficient statistics.

Type

Optional[str]

__init__(*pseudo_count=None, suff_stat=None, name=None, keys=None*)

Initialize ExponentialEstimator.

Parameters

- **pseudo_count** (*Optional[float], optional*) – Used to weight sufficient statistics.
- **suff_stat** (*Optional[float], optional*) – Positive float value for scale of exponential distribution.
- **name** (*Optional[str], optional*) – Name for the estimator.
- **keys** (*Optional[str], optional*) – Key for combining sufficient statistics.

Raises

TypeError – If keys is not a string or None.

Parameters

- **pseudo_count** (*float | None*)
- **suff_stat** (*float | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

accumulator_factory()

Return an ExponentialAccumulatorFactory for this estimator.

Return type

`dmx.stats.exponential.ExponentialAccumulatorFactory`

Returns

ExponentialAccumulatorFactory – Factory object.

estimate(*nobs, suff_stat*)

Estimate an ExponentialDistribution from sufficient statistics.

Parameters

- **nobs** (*Optional[float]*) – Number of observations (not used).
- **suff_stat** (*Tuple[float, float]*) – (count, sum) sufficient statistics.

Return type*dmx.stats.exponential.ExponentialDistribution***Returns***ExponentialDistribution* – Estimated distribution.**Parameters**

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*float*, *float*])

ExponentialSampler**class** `dmx.stats.exponential.ExponentialSampler`(*dist*, *seed=None*)

Sampler for the exponential distribution.

Parameters

- **dist** (*ExponentialDistribution*)
- **seed** (*int* | *None*)

dist*ExponentialDistribution* instance to sample from.**Type***ExponentialDistribution***rng**

Random number generator.

Type*RandomState***sample**(*size=None*)

Draw iid samples from the exponential distribution.

Parameters**size** (*Optional*[*int*], *optional*) – Number of samples to draw. If *None*, returns a single sample.**Return type***typing.Union*[*float*, *numpy.ndarray*]**Returns***Union*[*float*, *np.ndarray*] – Single sample or array of samples.**Parameters****size** (*int* | *None*)**Gamma**Data Type: *float*

The gamma distribution is a generalization of the exponential distribution. The distribution has support on the positive real line. The probability density function is given by

$$f(x|k, \theta) = \frac{1}{\Gamma(k)\theta^k} x^{k-1} e^{-\frac{1}{\theta}x}, \quad x \geq 0.$$

The above is the scale parameterization of the gamma distribution. For more info see [Gamma Distribution](#).

GammaDistribution

class `dmx.stats.gamma.GammaDistribution(k, theta, name=None, keys=None)`

Represent a gamma distribution in the shape-scale parameterization.

The support is the positive real line. `k` is the positive shape and `theta` is the positive scale, so the mean is `k * theta` and the rate is `1 / theta`.

Parameters

- `k` (*float*)
- `theta` (*float*)
- `name` (*str* | *None*)
- `keys` (*str* | *None*)

`k`

Positive real-valued shape parameter.

Type

float

`theta`

Positive real-valued scale parameter.

Type

float

`name`

Name for the GammaDistribution instance.

Type

Optional[*str*]

`log_const`

Normalizing constant of gamma distribution.

Type

float

`keys`

Key for parameters of distribution.

Type

Optional[*str*]

`__init__`(*k*, *theta*, *name*=None, *keys*=None)

Initialize GammaDistribution.

Parameters

- `k` – Positive shape parameter.
- `theta` – Positive scale parameter.
- `name` – Optional name for the distribution.
- `keys` – Optional key for tying sufficient statistics.

Parameters

- `k` (*float*)

- **theta** (*float*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

Return type

None

density(*x*)

Evaluate the density of the gamma distribution at *x*.

Parameters

x – Positive scalar observation.

Return type

float

Returns

Probability density at *x*.

Parameters

x (*float*)

dist_to_encoder()

Return a GammaDataEncoder for this distribution.

Return type

`dmx.stats.gamma.GammaDataEncoder`

Returns

GammaDataEncoder – Encoder object.

estimator(*pseudo_count=None*)

Create an estimator initialized from this distribution.

With *pseudo_count*, the estimator receives this distribution's arithmetic and geometric means as its two prior statistics.

Parameters

pseudo_count (*Optional[float]*, *optional*) – Pseudo-count for regularization.

Return type

`dmx.stats.gamma.GammaEstimator`

Returns

GammaEstimator – Estimator object.

Parameters

pseudo_count (*float* / *None*)

log_density(*x*)

Evaluate the log-density of the gamma distribution at *x*.

Parameters

x – Positive scalar observation.

Return type

float

Returns

Log-density at *x*.

Parameters**x** (*float*)**sampler** (*seed=None*)

Return a GammaSampler for this distribution.

Parameters**seed** (*Optional[int], optional*) – Seed for random number generator.**Return type***dmx.stats.gamma.GammaSampler***Returns***GammaSampler* – Sampler object.**Parameters****seed** (*int | None*)**seq_log_density** (*x*)

Vectorized log-density for encoded data.

Parameters**x** – Encoded sequence containing N observations.**Return type***numpy.ndarray***Returns**

Array of shape (N,) containing one log-density per observation.

Parameters**x** (*GammaEncodedDataSequence*)**GammaEstimator**

```
class dmx.stats.gamma.GammaEstimator(pseudo_count=(0.0, 0.0), suff_stat=(1.0, 0.0), threshold=1.0e-8,  
                                     name=None, keys=None)
```

Estimate gamma shape and scale from weighted sufficient statistics.

The shape solves $\log(k) - \text{digamma}(k) = \log(\text{mean}(x)) - \text{mean}(\log(x))$ by Newton iteration. The scale is then derived from the adjusted arithmetic mean. Separate pseudo-counts augment `sum_x` and `sum_log_x` with the corresponding entries of `suff_stat`; empty statistics produce `GammaDistribution(1, 1)`.

Parameters

- **pseudo_count** (*Tuple[float, float]*)
- **suff_stat** (*Tuple[float, float]*)
- **threshold** (*float*)
- **name** (*str | None*)
- **keys** (*str | None*)

pseudo_count

Values used to re-weight sufficient statistics.

Type*Tuple[float, float]*

suff_stat

Prior arithmetic and geometric means.

Type

`Tuple[float, float]`

threshold

Threshold for estimating the shape of gamma.

Type

`float`

name

Name for the estimator.

Type

`Optional[str]`

keys

Key for combining sufficient statistics.

Type

`Optional[str]`

`__init__` (*pseudo_count*=(0.0, 0.0), *suff_stat*=(1.0, 0.0), *threshold*=1.0e-8, *name*=None, *keys*=None)

Initialize GammaEstimator.

Parameters

- **`pseudo_count`** (*Tuple[float, float]*, *optional*) – Values used to re-weight sufficient statistics.
- **`suff_stat`** – Prior arithmetic and geometric means used with the two pseudo-counts.
- **`threshold`** (*float*, *optional*) – Threshold for estimating the shape of gamma.
- **`name`** (*Optional[str]*, *optional*) – Name for the estimator.
- **`keys`** (*Optional[str]*, *optional*) – Key for combining sufficient statistics.

Raises

`TypeError` – If `keys` is not a string or None.

Parameters

- **`pseudo_count`** (*Tuple[float, float]*)
- **`suff_stat`** (*Tuple[float, float]*)
- **`threshold`** (*float*)
- **`name`** (*str | None*)
- **`keys`** (*str | None*)

Return type

`None`

`accumulator_factory`()

Return a GammaAccumulatorFactory for this estimator.

Return type

`dmx.stats.gamma.GammaAccumulatorFactory`

Returns

GammaAccumulatorFactory – Factory object.

estimate(*nobs*, *suff_stat*)

Estimate a GammaDistribution from sufficient statistics.

Parameters

- **nobs** (*Optional[float]*) – Number of observations (not used).
- **suff_stat** (*Tuple[float, float, float]*) – (nobs, sum, sum_of_logs) sufficient statistics.

Return type

dmx.stats.gamma.GammaDistribution

Returns

GammaDistribution – Estimated distribution.

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[float, float, float]*)

static estimate_shape(*avg_sum*, *avg_sum_of_logs*, *threshold*)

Estimate the shape parameter of the GammaDistribution.

Parameters

- **avg_sum** (*float*) – Weighted mean of gamma observations.
- **avg_sum_of_logs** (*float*) – Weighted mean of log gamma observations.
- **threshold** (*float*) – Threshold for convergence of shape estimation.

Return type

float

Returns

float – Estimate of shape parameter 'k'.

Parameters

- **avg_sum** (*float*)
- **avg_sum_of_logs** (*float*)
- **threshold** (*float*)

GammaSampler

class *dmx.stats.gamma.GammaSampler*(*dist*, *seed=None*)

Sampler for the gamma distribution.

Parameters

- **dist** (*GammaDistribution*)
- **seed** (*int | None*)

rng

Random number generator.

Type

RandomState

dist

GammaDistribution to sample from.

Type*GammaDistribution***seed**

Seed for random number generator.

Type

Optional[int]

sample(size=None)

Draw iid samples from the gamma distribution.

Parameters**size** (*Optional[int]*, *optional*) – Number of iid samples to draw. If None, returns a single sample.**Return type**

typing.Union[float, typing.List[float]]

Returns*Union[float, List[float]]* – Single sample (float) if size is None, else a list of samples.**Parameters****size** (*int* | *None*)

Log-Gaussian (Log-Normal)

Data Type: float

The log-Gaussian distribution is used to model data that is normal on the log scale. The probability density function is given by

$$f(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}x} e^{-\frac{(\log x - \mu)^2}{2\sigma^2}}, \quad x \in \mathbb{R}.$$

For more info see [log-Gaussian Distribution](#).

LogGaussianDistribution

class dmx.stats.log_gaussian.LogGaussianDistribution(mu, sigma2, name=None, keys=None)

Represent a univariate log-normal distribution.

The log observation has mean mu and variance sigma2:

$$p(x) = \frac{1}{x\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(\log x - \mu)^2}{2\sigma^2}\right], \quad x > 0.$$

A nonpositive or nonfinite sigma2 is replaced by 1.0.

Parameters

- **mu** (*float*)
- **sigma2** (*float*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

mu

Mean of the log observation.

sigma2

Variance of the log observation.

constGaussian normalizing constant before the $1 / x$ factor.**log_const**

Logarithm of const.

name

Optional instance name.

keys

Optional key for sharing sufficient statistics.

__init__(*mu*, *sigma2*, *name=None*, *keys=None*)

Initialize a log-normal distribution.

Parameters

- **mu** – Mean of $\log(X)$.
- **sigma2** – Variance of $\log(X)$; invalid values are replaced by 1.0 .
- **name** – Optional instance name.
- **keys** – Optional key for sharing sufficient statistics.

Parameters

- **mu** (*float*)
- **sigma2** (*float*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

Return type

None

density(*x*)Density of Log-Gaussian distribution at observation *x*.See `log_density()` for details.**Parameters****x** (*float*) – Positive real-valued number.**Return type***float***Returns***float* – Density of Log-Gaussian at *x*.**Parameters****x** (*float*)

dist_to_encoder()

Create an encoder for positive scalar observations.

Return type

`dmx.stats.log_gaussian.LogGaussianDataEncoder`

estimator(*pseudo_count=None*)

Create an estimator centered on this distribution when regularized.

Return type

`dmx.stats.log_gaussian.LogGaussianEstimator`

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Log-density of log-Gaussian distribution at observation *x*.

Parameters

x (*float*) – Positive valued observation of log-Gaussian.

Return type

float

Returns

float – Log-density at observation *x*.

Parameters

x (*float*)

sampler(*seed=None*)

Create a sampler, optionally initialized with *seed*.

Return type

`dmx.stats.log_gaussian.LogGaussianSampler`

Parameters

seed (*int* | *None*)

seq_ld_lambda()

Return the vectorized log-density callable.

Return type

`typing.List[typing.Callable]`

seq_log_density(*x*)

Evaluate log densities for encoded log observations.

The encoded data array and returned array both have shape `(n,)`.

Return type

`numpy.ndarray`

Parameters

x (*LogGaussianEncodedDataSequence*)

LogGaussianEstimator

```
class dmx.stats.log_gaussian.LogGaussianEstimator(pseudo_count=(None, None), suff_stat=(None, None), name=None, keys=None)
```

Estimate log-normal mean and variance from weighted statistics.

`pseudo_count` independently regularizes the log mean and log variance. The corresponding `suff_stat` entries are the centering mean and variance.

Parameters

- **pseudo_count** (*Tuple[float | None, float | None]*)
- **suff_stat** (*Tuple[float | None, float | None]*)
- **name** (*str | None*)
- **keys** (*str | None*)

pseudo_count

Weights for `suff_stat`.

Type

Tuple[Optional[float], Optional[float]]

suff_stat

Tuple of mean (`mu`) and variance (`sigma2`).

Type

Tuple[Optional[float], Optional[float]]

name

String name of `LogGaussianEstimator` instance.

Type

Optional[str]

keys

String keys of `LogGaussianEstimator` instance for combining sufficient statistics.

Type

Optional[str]

__init__ (*pseudo_count=(None, None), suff_stat=(None, None), name=None, keys=None*)

Initialize the estimator and optional regularization targets.

Parameters

- **pseudo_count** (*Tuple[Optional[float], Optional[float]]*) – Tuple of two positive floats.
- **suff_stat** (*Tuple[Optional[float], Optional[float]]*) – Tuple of float and positive float.
- **name** (*Optional[str]*) – Assign a name to `LogGaussianEstimator`.
- **keys** (*Optional[str]*) – Assign keys to `LogGaussianEstimator` for combining sufficient statistics.

Parameters

- **pseudo_count** (*Tuple[float | None, float | None]*)
- **suff_stat** (*Tuple[float | None, float | None]*)
- **name** (*str | None*)
- **keys** (*str | None*)

accumulator_factory()

Create a matching accumulator factory.

Return type

`dmx.stats.log_gaussian.LogGaussianAccumulatorFactory`

estimate(*nobs*, *suff_stat*)

Estimate a distribution from four log-domain statistics.

nobs is accepted for protocol compatibility; the two counts in *suff_stat* control estimation.

Return type

`dmx.stats.log_gaussian.LogGaussianDistribution`

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*float*, *float*, *float*, *float*])

LogGaussianSampler

class `dmx.stats.log_gaussian.LogGaussianSampler(dist, seed=None)`

Draw independent samples from a log-normal distribution.

Parameters

- **dist** (`LogGaussianDistribution`)
- **seed** (*int* | *None*)

dist

Distribution to sample from.

rng

Pseudorandom number generator.

sample(*size*=*None*)

Draw ‘size’ iid samples from LogGaussianSampler object.

Numpy array of length ‘size’ from log-Gaussian distribution with scale beta if size not None. Else a single sample is returned as float.

Parameters

size (*Optional*[*int*]) – Treated as 1 if None is passed.

Return type

`typing.Union`[*float*, `numpy.ndarray`]

Returns

‘size’ iid samples from Gaussian distribution.

Parameters

size (*int* | *None*)

Data Type: Sequence[*float*]**Diagonal Multivariate Gaussian**

Data Type: Sequence[*float*]

A d-dimensional random variable $(X_1, X_2, \dots, X_d)$ follows a diagonal multivariate normal distribution if each $X_i \sim N(0, \sigma_i^2)$ and $Cov(X_i, X_j) = 0$ for each $i \neq j$.

The probability density function is given by

$$f(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\sigma}) = \left(\frac{1}{2\pi}\right)^{d/2} \prod_{i=1}^d \frac{1}{\sigma_i} \exp - \left(\frac{(x_i - \mu_i)^2}{2\sigma_i^2}\right)$$

For more info see [Multivariate Normal Distribution](#).

DiagonalGaussianDistribution

class `dmx.stats.dmvn.DiagonalGaussianDistribution(mu, covar, name=None, keys=(None, None))`

Represent a multivariate Gaussian with diagonal covariance.

The support is $\mathbb{R}^D$. `mu` and `covar` are length-D vectors, and `covar[i]` is the positive variance of coordinate `i`; it is not a standard deviation or a full covariance matrix.

Parameters

- **mu** (*Sequence[float] | ndarray*)
- **covar** (*Sequence[float] | ndarray*)
- **name** (*str | None*)
- **keys** (*Tuple[str | None, str | None]*)

dim

Dimension of the multivariate Gaussian.

Type

int

mu

Mean of the Gaussian.

Type

np.ndarray

covar

Variance for each component.

Type

np.ndarray

name

Name of object instance.

Type

Optional[str]

log_c

Normalizing constant for diagonal Gaussian.

Type

float

ca

Term for likelihood calculation.

Type

np.ndarray

cb

Term for likelihood calculation.

Type

np.ndarray

cc

Term for likelihood calculation.

Type

np.ndarray

keys

Key for mean and covariance.

Type

Tuple[Optional[str], Optional[str]]

__init__(*mu*, *covar*, *name=None*, *keys=(None, None)*)

Initialize a DiagonalGaussianDistribution object.

Parameters

- **mu** – Mean vector with shape (D,).
- **covar** – Diagonal variance vector with shape (D,).
- **name** – Optional name for the distribution.
- **keys** – Optional keys for tying mean and variance statistics separately.

Parameters

- **mu** (*Sequence[float] | ndarray*)
- **covar** (*Sequence[float] | ndarray*)
- **name** (*str | None*)
- **keys** (*Tuple[str | None, str | None]*)

Return type

None

density(*x*)

Evaluate the density of the DiagonalGaussianDistribution.

Parameters**x** – One observation with shape (D,).**Return type**

float

ReturnsProbability density at **x**.**Parameters****x** (*Sequence[float] | ndarray*)

dist_to_encoder()

Return a DiagonalGaussianDataEncoder for this distribution.

Return type

`dmx.stats.dmvn.DiagonalGaussianDataEncoder`

Returns

DiagonalGaussianDataEncoder – Encoder object.

estimator(pseudo_count=None)

Create an estimator initialized from this distribution.

A supplied `pseudo_count` is applied to both mean and variance estimation. The estimator returned here has no prior sufficient-statistic vectors, so those pseudo-counts affect estimates only if priors are later supplied directly to the estimator.

Parameters

pseudo_count (*Optional[float], optional*) – Pseudo-count for regularization.

Return type

`dmx.stats.dmvn.DiagonalGaussianEstimator`

Returns

DiagonalGaussianEstimator – Estimator object.

Parameters

pseudo_count (*float | None*)

log_density(x)

Evaluate the log-density of the DiagonalGaussianDistribution.

Parameters

x – One observation with shape (D,).

Return type

`float`

Returns

Log-density at x.

Parameters

x (*Sequence[float] | ndarray*)

sampler(seed=None)

Return a DiagonalGaussianSampler for this distribution.

Parameters

seed (*Optional[int], optional*) – Seed for random number generator.

Return type

`dmx.stats.dmvn.DiagonalGaussianSampler`

Returns

DiagonalGaussianSampler – Sampler object.

Parameters

seed (*int | None*)

seq_log_density(x)

Vectorized log-density for encoded data.

Parameters

x – Encoded sequence backed by an array with shape (N, D).

Return type`numpy.ndarray`**Returns**

Array of shape (N,) containing one log-density per observation.

Parameters

x (*DiagonalGaussianEncodedDataSequence*)

DiagonalGaussianEstimator

```
class dmx.stats.dmvn.DiagonalGaussianEstimator(dim=None, pseudo_count=(None, None),
                                              suff_stat=(None, None), name=None, keys=(None,
                                              None))
```

Estimate diagonal Gaussian means and variances from weighted statistics.

The mean is the weighted first moment and the variance is the weighted second central moment, coordinate by coordinate. The two pseudo-counts independently regularize the mean toward `prior_mu` and the variance toward `prior_covar` when the corresponding prior is present.

Parameters

- **dim** (*int* | *None*)
- **pseudo_count** (*Tuple*[*float* | *None*, *float* | *None*])
- **suff_stat** (*Tuple*[*ndarray* | *None*, *ndarray* | *None*])
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*])

name

Name for object instance.

Type

`Optional[str]`

dim

Dimension of Gaussian, either set or determined from `suff_stat` arg.

Type

`int`

prior_mu

Set from `suff_stat[0]`.

Type

`Optional[np.ndarray]`

prior_covar

Set from `suff_stat[1]`.

Type

`Optional[np.ndarray]`

pseudo_count

Re-weight the sum of observations and sum of squared observations in estimation.

Type

`Tuple`[`Optional`[`float`], `Optional`[`float`]]

keys

Key for mean and covariance.

Type

`Tuple[Optional[str], Optional[str]]`

__init__ (*dim=None, pseudo_count=(None, None), suff_stat=(None, None), name=None, keys=(None, None)*)

Initialize DiagonalGaussianEstimator.

Parameters

- **dim** (*Optional[int], optional*) – Optional dimension of Gaussian.
- **pseudo_count** (*Tuple[Optional[float], Optional[float]], optional*) – Re-weight the sum of observations and sum of squared observations in estimation.
- **suff_stat** (*Tuple[Optional[np.ndarray], Optional[np.ndarray]], optional*) – Sum of observations and sum of squared observations both having same dimension.
- **name** (*Optional[str], optional*) – Name for object instance.
- **keys** (*Tuple[Optional[str], Optional[str]], optional*) – Key for mean and covariance.

Raises

TypeError – If keys is not a tuple of two strings or None.

Parameters

- **dim** (*int | None*)
- **pseudo_count** (*Tuple[float | None, float | None]*)
- **suff_stat** (*Tuple[ndarray | None, ndarray | None]*)
- **name** (*str | None*)
- **keys** (*Tuple[str | None, str | None]*)

Return type

None

accumulator_factory()

Return a DiagonalGaussianAccumulatorFactory for this estimator.

Return type

`dmx.stats.dmvn.DiagonalGaussianAccumulatorFactory`

Returns

DiagonalGaussianAccumulatorFactory – Factory object.

estimate(nobs, suff_stat)

Estimate a DiagonalGaussianDistribution from sufficient statistics.

Parameters

- **nobs** – Ignored; counts are read from `suff_stat`.
- **suff_stat** – (`sum_x`, `sum_x2`, `count`, `count2`). The arrays have shape `(D,)`; this implementation uses `count` for both estimates.

Return type

`dmx.stats.dmvn.DiagonalGaussianDistribution`

Returns

DiagonalGaussianDistribution – Estimated distribution.

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray* | *None*, *ndarray* | *None*, *float*])

DiagonalGaussianSampler

class dmx.stats.dmvn.**DiagonalGaussianSampler**(*dist*, *seed=None*)

Sampler for *DiagonalGaussianDistribution*.

Parameters

- **dist** (*DiagonalGaussianDistribution*)
- **seed** (*int* | *None*)

dist

Object instance to sample from.

Type

DiagonalGaussianDistribution

rng

Random number generator.

Type

RandomState

sample(*size=None*)

Generate samples from *Diagonal Gaussian* distribution.

Parameters

size (*Optional*[*int*], *optional*) – Number of samples to generate.

Return type

typing.Union[*numpy.ndarray*, *typing.List*[*numpy.ndarray*]]

Returns

Union[*np.ndarray*, *List*[*np.ndarray*]] – Single sample or list of samples.

Parameters

size (*int* | *None*)

Multivariate Gaussian

Data Type: *Sequence*[*float*]

The probability density function of a *d*-dimensional multivariate Gaussian random variable $(X_1, X_2, \dots, X_d)$ with mean μ and postive definite covariance matrix Σ is given by

$$f(\mathbf{x}|\mu, \Sigma) = \left(\frac{1}{2\pi}\right)^{d/2} |\Sigma|^{d/2} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^t \Sigma^{-1} (\mathbf{x} - \mu)\right).$$

If you are assuming $Cov(X_i, X_j) = 0$ $i \neq j$, a faster and more efficient option is the *Diagonal Multivariate Gaussian*.

For more info see [Multivariate Normal Distribution](#).

MultivariateGaussianDistribution

class dmx.stats.mvn.MultivariateGaussianDistribution(*mu*, *covar*, *name=None*, *keys=None*)

Represent a multivariate Gaussian with full covariance.

mu has shape (d,) and *covar* has shape (d, d). The covariance is interpreted directly, rather than as a precision or Cholesky factor, and must be symmetric positive definite so SciPy can factor it.

Parameters

- **mu** (*List[float]* | *ndarray*)
- **covar** (*List[List[float]]* | *ndarray*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

dim

N is the dim of multivariate normal.

Type

int

mu

Length N numpy array

Type

np.ndarray

covar

N by N numpy array for Covariance matrix.

Type

np.ndarray

chol

Cholesky decomposition of covar.

Type

np.ndarray

lower

Flag for lower (False for upper)

Type

bool

name

Set name to object.

Type

Optional[str]

keys

Set keys for distribution.

Type

Optional[str]

self.use_lstsq

Cholesky does not exist so use least squares approx.

Type

bool

self.chol_const

det from covar if lstsq is to be used.

Type

float

__init__(*mu, covar, name=None, keys=None*)

Initialize a multivariate Gaussian distribution.

Parameters

- **mu** (*Union[List[float], np.ndarray]*) – N-dimensional mean.
- **covar** (*Union[List[List[float]], np.ndarray]*) – Covariance matrix, should be N by N and positive definite.
- **name** (*Optional[str]*) – Set name to object.
- **keys** (*Optional[str]*) – Set keys for distribution.

Parameters

- **mu** (*List[float] | ndarray*)
- **covar** (*List[List[float]] | ndarray*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

density(*x*)

Evaluate the density at x.

Parameters

x (*np.ndarray*) – Observation from multivariate Gaussian distribution.

Return type

float

Returns

float – Density at x.

Parameters

x (*ndarray*)

dist_to_encoder()

Create an encoder for vectors of this distribution's dimension.

Return type

`dmx.stats.mvn.MultivariateGaussianDataEncoder`

estimator(*pseudo_count=None*)

Create an estimator centered on this distribution when regularized.

Return type

`dmx.stats.mvn.MultivariateGaussianEstimator`

Parameters**pseudo_count** (*float* | *None*)**log_density(x)**

Evaluate the log-density at x.

Parameters**x** (*np.ndarray*) – Observation from multivariate Gaussian distribution.**Return type***float***Returns***float* – Log-density at x.**Parameters****x** (*ndarray*)**sampler(seed=None)**

Create a sampler, optionally initialized with seed.

Return type*dmx.stats.mvn.MultivariateGaussianSampler***Parameters****seed** (*int* | *None*)**seq_log_density(x)**

Evaluate log densities for an encoded observation matrix.

Encoded data has shape (n, d) and the result has shape (n,).

Return type*numpy.ndarray***Parameters****x** (*MultivariateGaussianEncodedDataSequence*)**MultivariateGaussianEstimator**

```
class dmx.stats.mvn.MultivariateGaussianEstimator(dim=None, pseudo_count=(None, None),  
                                                suff_stat=(None, None), name=None, keys=None)
```

Estimate a full-covariance multivariate Gaussian.

The two pseudo-count entries independently regularize the mean and covariance toward `suff_stat`. Without them, estimation uses weighted maximum likelihood and therefore assumes positive total weight and a positive-definite estimated covariance.

Parameters

- **dim** (*int* | *None*)
- **pseudo_count** (*Tuple[*float* | *None*, *float* | *None*] | *None*)*
- **suff_stat** (*Tuple[*ndarray* | *None*, *ndarray* | *None*]*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

dim

Dimension of multivariate normal.

Type

int

pseudo_count

Regularize mean and/or covariance.

Type

Optional[Tuple[Optional[float], Optional[float]]]

prior_mu

Mean from prior data or used to regularize.

Type

Optional[np.ndarray]

prior_covar

Covariance matrix from prior data or used to regularize.

Type

Optional[np.ndarray]

name

Set name to object.

Type

Optional[str]

keys

Keys for merging sufficient statistics.

Type

Optional[str]

__init__ (*dim=None, pseudo_count=(None, None), suff_stat=(None, None), name=None, keys=None*)

Initialize the estimator and optional regularization targets.

Parameters

- **dim** (*Optional[int]*) – Dimension of multivariate normal. Inferred from ‘suff_stat’ if None.
- **pseudo_count** (*Optional[Tuple[Optional[float], Optional[float]]]*) – Regularize mean and/or covariance.
- **suff_stat** (*Optional[Tuple[Optional[np.ndarray], Optional[np.ndarray]]]*) – Mean and covariance estimated from previous data or used to regularize.
- **name** (*Optional[str]*) – Set name for object instance.
- **keys** (*Optional[str]*) – Set keys for estimator.

Parameters

- **dim** (*int | None*)
- **pseudo_count** (*Tuple[float | None, float | None] | None*)
- **suff_stat** (*Tuple[ndarray | None, ndarray | None]*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

accumulator_factory()

Create a matching accumulator factory.

Return type`dmx.stats.mvn.MultivariateGaussianAccumulatorFactory`**estimate(nobs, suff_stat)**

Estimate a distribution from (sum, sum2, count).

nobs is accepted for protocol compatibility and replaced by the statistic count.

Return type`dmx.stats.mvn.MultivariateGaussianDistribution`**Parameters**

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray* | *None*, *ndarray* | *None*, *float*])

MultivariateGaussianSampler**class** `dmx.stats.mvn.MultivariateGaussianSampler`(*dist*, *seed=None*)

Draw independent samples from a multivariate Gaussian.

Parameters

- **dist** (`MultivariateGaussianDistribution`)
- **seed** (*int* | *None*)

rng

Sets seed for generating samples.

Type`RandomState`**dist**

MultivariateGaussianDistribution to sample from.

Type`MultivariateGaussianDistribution`**sample(size=None)**

Generate samples from MultivariateGaussianDistribution.

Parameters**size** (*Optional*[*int*]) – Number of samples to generate.**Return type**`numpy.ndarray`**Returns***np.ndarray* – Size by dim number of samples.**Parameters****size** (*int* | *None*)

Dirichelt Distribution

Data Type: Sequence[float]

The Dirichlet distribution is a distribution on the simplex. A d-dimensinoal Dirichlet random variable $(X_1, X_2, \dots, X_d)$ has a density function is given by

$$f(\mathbf{x}|\boldsymbol{\alpha}) = \frac{1}{B(\boldsymbol{\alpha})} \prod_{i=1}^d x_i^{\alpha_i-1}, \quad 0 \leq x_i \leq 1$$

where $B((\alpha)) = \frac{\prod_{i=1}^d \Gamma(\alpha_i)}{\Gamma(\sum_{i=1}^d \alpha_i)}$ and $\sum_{i=1}^d x_i = 1$.

For more info see [Dirichlet Distribution](#).

DirichletDistribution

class `dmx.stats.dirichlet.DirichletDistribution(alpha, name=None, keys=None)`

Represent a Dirichlet distribution with concentration vector **alpha**.

The standard support is the K-simplex: length-K nonnegative vectors whose entries sum to one. Positive entries of **alpha** are ordinary concentration parameters. The implementation also permits nonpositive entries and treats their coordinates specially when evaluating or sampling.

Parameters

- **alpha** (*List[float] | ndarray*)
- **name** (*str | None*)
- **keys** (*str | None*)

dim

Number of simplex coordinates.

Type

int

alpha

Concentration parameters with shape (K,).

Type

np.ndarray

alpha_ma

Boolean mask for positive alpha entries.

Type

np.ndarray

log_const

Logarithm of the beta-function normalizer.

Type

float

has_invalid

True if any alpha are less than or equal to 0.

Type

bool

name

Optional name for object instance.

Type

Optional[str]

keys

Optional key for merging sufficient statistics.

Type

Optional[str]

__init__(*alpha*, *name*=None, *keys*=None)

Initialize DirichletDistribution.

Parameters

- **alpha** – One-dimensional concentration vector with shape (K,).
- **name** – Optional name for the distribution.
- **keys** – Optional key for tying sufficient statistics.

Parameters

- **alpha** (*List[float]* | *ndarray*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

Return type

None

density(*x*)

Evaluate the density of a Dirichlet observation.

Parameters

x – One simplex observation with shape (K,).

Return type

float

Returns

Probability density at **x**.

Parameters

x (*List[float]* | *ndarray*)

dist_to_encoder()

Create an encoder for sequences of Dirichlet observations.

Return type

`dmx.stats.dirichlet.DirichletDataEncoder`

Returns

DirichletDataEncoder – Encoder object.

estimator(*pseudo_count*=None)

Create an estimator initialized from this distribution.

With *pseudo_count*, the log of the normalized concentration vector is used as a prior mean-log sufficient statistic.

Parameters

pseudo_count (*Optional[float], optional*) – Pseudo-count for regularization.

Return type

`dmx.stats.dirichlet.DirichletEstimator`

Returns

DirichletEstimator – Estimator object.

Parameters

pseudo_count (*float | None*)

log_density(x)

Evaluate the log-density of one simplex observation.

For concentration vector alpha, the log-density is $-\log(B(\alpha)) + \sum((\alpha - 1) * \log(x))$.

Parameters

x – One simplex observation with shape (K,).

Return type

`float`

Returns

Log-density at x.

Parameters

x (*List[float] | ndarray*)

sampler(seed=None)

Return a DirichletSampler for this distribution.

Parameters

seed (*Optional[int], optional*) – Seed for random number generator.

Return type

`dmx.stats.dirichlet.DirichletSampler`

Returns

DirichletSampler – Sampler object.

Parameters

seed (*int | None*)

seq_log_density(x)

Vectorized log-density for encoded data.

Parameters

x – Encoded sequence containing N length-K observations.

Return type

`numpy.ndarray`

Returns

Array of shape (N,) containing one log-density per observation.

Parameters

x (*DirichletEncodedDataSequence*)

DirichletEstimator

class dmx.stats.dirichlet.**DirichletEstimator**(*dim*, *pseudo_count=None*, *suff_stat=None*, *delta=1.0e-8*, *keys=None*, *use_mpe=False*, *name=None*)

Estimate a Dirichlet concentration vector from weighted statistics.

Without a pseudo-count, the estimator initializes `alpha` from the first moment, then solves the mean-log equations by fixed-point iteration. `use_mpe=True` selects the minimal-polynomial extrapolation implemented by `find_alpha()`. Pseudo-counts augment the mean-log statistic with either a symmetric prior or the supplied `suff_stat` vector.

Parameters

- **dim** (*int*)
- **pseudo_count** (*float* | *None*)
- **suff_stat** (*ndarray* | *None*)
- **delta** (*float* | *None*)
- **keys** (*str* | *None*)
- **use_mpe** (*bool*)
- **name** (*str* | *None*)

dim

Dimension of Dirichlet distribution to estimate.

Type

`int`

pseudo_count

Pseudo count for sufficient statistics.

Type

`Optional[float]`

delta

Tolerance for shape estimation from sufficient statistics.

Type

`Optional[float]`

suff_stat

Sufficient statistics.

Type

`Optional[np.ndarray]`

keys

Optional key string for shape parameter.

Type

`Optional[str]`

use_mpe

Whether to use minimal polynomial extrapolation.

Type

`bool`

name

Name for object.

Type

Optional[str]

__init__(*dim*, *pseudo_count*=None, *suff_stat*=None, *delta*=1.0e-8, *keys*=None, *use_mpe*=False, *name*=None)

Initialize DirichletEstimator.

Parameters

- **dim** (*int*) – Dimension of Dirichlet distribution to estimate.
- **pseudo_count** (*Optional[float]*, *optional*) – Pseudo count for sufficient statistics.
- **suff_stat** (*Optional[np.ndarray]*, *optional*) – Sufficient statistics.
- **delta** (*Optional[float]*, *optional*) – Tolerance for shape estimation from sufficient statistics.
- **keys** (*Optional[str]*, *optional*) – Optional key string for shape parameter.
- **use_mpe** (*bool*, *optional*) – Whether to use minimal polynomial extrapolation.
- **name** (*Optional[str]*, *optional*) – Name for object.

Raises

TypeError – If keys is not a string or None.

Parameters

- **dim** (*int*)
- **pseudo_count** (*float* | *None*)
- **suff_stat** (*ndarray* | *None*)
- **delta** (*float* | *None*)
- **keys** (*str* | *None*)
- **use_mpe** (*bool*)
- **name** (*str* | *None*)

Return type

None

accumulator_factory()

Return a DirichletAccumulatorFactory for this estimator.

Return type

dmx.stats.dirichlet.DirichletAccumulatorFactory

Returns

DirichletAccumulatorFactory – Factory object.

estimate(*nobs*, *suff_stat*)

Estimate a DirichletDistribution from sufficient statistics.

Parameters

- **nobs** – Ignored; the weighted count is read from *suff_stat*.
- **suff_stat** – (count, sum_log_x, sum_x, sum_x2), where each array has shape (K,).

Return type*dmx.stats.dirichlet.DirichletDistribution***Returns***DirichletDistribution* – Estimated distribution.**Parameters**

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*float*, *ndarray*, *ndarray*, *ndarray*])

DirichletSampler**class** `dmx.stats.dirichlet.DirichletSampler`(*dist*, *seed=None*)

DirichletSampler object for drawing samples from Dirichlet distribution.

Parameters

- **dist** (*DirichletDistribution*)
- **seed** (*int* | *None*)

rng

RandomState object for generating seeded samples.

Type

RandomState

dist

DirichletDistribution object to draw samples from.

Type*DirichletDistribution***sample**(*size=None*)

Draw samples from Dirichlet distribution.

Parameters**size** (*Optional*[*int*], *optional*) – Number of samples to draw.**Return type**`numpy.ndarray`**Returns**

One sample with shape (K,) when size is None; otherwise an array with shape (size, K).

Parameters**size** (*int* | *None*)**Data Type: str****Categorical**

Data Type: str

The Categorical distribution, also known as the multinomial distribution, is a probability distribution over a set of possible categories. Although this distribution is claimed to support data type str, this distribution can be used to define a distribution over any set of objects. The probability mass function is given by over a set of values $V = \{v_1, \dots, v_k\}$ is given by

$$f(x|\mathbf{p}) = \begin{cases} p_i, & x = v_i \\ 0, & x \notin V \end{cases}$$

where $\sum_{i=1}^k p_i = 1$. Note that any set of values V can be enumerated and mapped to the set of integers $0, 1, \dots, k - 1$. The user can then refer to the faster *Integer Categorical Distribution*.

For more info see [Categorical Distribution](#).

CategoricalDistribution

```
class dmx.stats.categorical.CategoricalDistribution(pmap, default_value=0.0, name=None,
                                                    keys=None)
```

Represent a categorical distribution over hashable values.

`pmap` maps explicit support values to their probability weights. A value absent from `pmap` receives weight `default_value`. Scalar and sequence likelihoods divide either weight by $1 + \text{default_value}$; callers normally provide an explicit probability map whose values sum to one.

Parameters

- **pmap** (*Dict[Any, float]*)
- **default_value** (*float*)
- **name** (*str | None*)
- **keys** (*str | None*)

name

Optional name for the distribution.

Type

Optional[str]

pmap

Explicit category-to-weight mapping.

Type

Dict[Any, float]

default_value

Weight assigned to values outside `pmap`.

Type

float

no_default

Whether the supplied default weight is nonzero.

Type

bool

log_default_value

Logarithm of the default weight.

Type

float

log1p_default_value

Logarithm of $1 + \text{default_value}$.

Type

float

keys

Optional key for tying sufficient statistics.

Type

Optional[str]

__init__(*pmap*, *default_value*=0.0, *name*=None, *keys*=None)

Initialize a categorical distribution.

Parameters

- **pmap** – Mapping from hashable category labels to probability weights.
- **default_value** – Weight for any category absent from **pmap**. The stored value is clipped to [0, 1].
- **name** – Optional name for the distribution.
- **keys** – Optional key for tying sufficient statistics.

Parameters

- **pmap** (*Dict*[Any, *float*])
- **default_value** (*float*)
- **name** (*str* | None)
- **keys** (*str* | None)

Return type

None

density(*x*)

Evaluate the probability mass at a category.

Parameters

x – Hashable category label.

Return type

float

Returns

Probability mass at **x**.

Parameters

x (*Any*)

dist_to_encoder()

Returns a CategoricalDataEncoder for this distribution.

Return type

dmx.stats.categorical.CategoricalDataEncoder

Returns

CategoricalDataEncoder – Encoder for categorical data.

estimator(*pseudo_count*=None)

Create an estimator initialized from this distribution.

When supplied, **pseudo_count** weights this distribution's probability map as prior category proportions.

Parameters

pseudo_count (*Optional*[float], *optional*) – If set, inflates counts for currently set sufficient statistic (**pmap**). Defaults to None.

Return type*dmx.stats.categorical.CategoricalEstimator***Returns***CategoricalEstimator* – Estimator object for the distribution.**Parameters****pseudo_count** (*float* | *None*)**log_density(*x*)**

Evaluate the log-probability mass at a category.

Parameters**x** – Hashable category label.**Return type***float***Returns**Log-probability mass at **x**.**Parameters****x** (*Any*)**sampler(*seed=None*)**

Creates a CategoricalSampler for sampling from the CategoricalDistribution.

Parameters**seed** (*Optional[int]*, *optional*) – Seed for setting random number generator used to sample.
Defaults to *None*.**Return type***dmx.stats.categorical.CategoricalSampler***Returns***CategoricalSampler* – Sampler object for the distribution.**Parameters****seed** (*int* | *None*)**seq_log_density(*x*)**

Evaluate log-probability masses for an encoded sequence.

Parameters**x** – Encoded sequence containing *N* categorical observations.**Return type***numpy.ndarray***Returns**Array of shape (*N*,) containing one log-probability mass per observation.**Parameters****x** (*CategoricalEncodedDataSequence*)**CategoricalEstimator**

```
class dmx.stats.categorical.CategoricalEstimator(pseudo_count=None, suff_stat=None,  
                                                default_value=False, name=None, keys=None)
```

Estimate categorical probabilities from weighted category counts.

Without a pseudo-count, counts are normalized directly. With a pseudo-count but no prior map, symmetric smoothing is spread over observed categories. With both, the union of observed and prior categories is normalized after adding the weighted prior map. `default_value=True` also assigns unseen values the implementation's data-dependent default weight.

Parameters

- **pseudo_count** (*float* | *None*)
- **suff_stat** (*Dict[Any, float]* | *None*)
- **default_value** (*bool*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

pseudo_count

Inflate sufficient statistic counts by pseudo_count.

Type

Optional[float]

suff_stat

Dictionary with category labels and probabilities as values.

Type

Optional[Dict[Any, float]]

default_value

True is default value should be set.

Type

bool

name

Assign name to be passed to Distribution, Accumulator, ect.

Type

Optional[str]

keys

Assign key to Estimator designating all same key estimators to later be combined, in accumulation.

Type

Optional[str]

__init__(*pseudo_count=None, suff_stat=None, default_value=False, name=None, keys=None*)

Initializes a CategoricalEstimator object.

Parameters

- **pseudo_count** (*Optional[float], optional*) – Inflate sufficient statistic counts by pseudo_count. Defaults to None.
- **suff_stat** (*Optional[Dict[Any, float]], optional*) – Dictionary with category labels and probabilities as values. Defaults to None.
- **default_value** (*bool, optional*) – True if default value should be set. Defaults to False.
- **name** (*Optional[str], optional*) – Assign name to be passed to Distribution, Accumulator, etc. Defaults to None.

- **keys** (*Optional[str], optional*) – Assign key to Estimator designating all same key estimators to later be combined, in accumulation. Defaults to None.

Parameters

- **pseudo_count** (*float | None*)
- **suff_stat** (*Dict[Any, float] | None*)
- **default_value** (*bool*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

accumulator_factory()

Returns a CategoricalAccumulatorFactory for this estimator.

Return type`dmx.stats.categorical.CategoricalAccumulatorFactory`**Returns***CategoricalAccumulatorFactory* – Factory for creating accumulators.**estimate(nobs, suff_stat)**

Estimates a CategoricalDistribution from sufficient statistics.

Parameters

- **nobs** (*Optional[float]*) – Not used. Kept for consistency with ParameterEstimator.estimate.
- **suff_stat** (*Dict[Any, float]*) – Dict with categories as keys and counts as values from accumulated data.

Return type`dmx.stats.categorical.CategoricalDistribution`**Returns***CategoricalDistribution* – Estimated distribution.**Parameters**

- **nobs** (*float | None*)
- **suff_stat** (*Dict[Any, float]*)

CategoricalSampler**class** `dmx.stats.categorical.CategoricalSampler`(*dist, seed=None*)

CategoricalSampler object used to generate samples from CategoricalDistribution.

Parameters

- **dist** (`CategoricalDistribution`)
- **seed** (*int | None*)

rng

RandomState with seed set to seed if provided. Else just RandomState().

Type

RandomState

levels

Category labels for the CategoricalDistribution.

Type

List[Any]

probs

Probabilities for each category in CategoricalDistribution.

Type

List[float]

num_levels

Total number of categories. I.e. len(levels).

Type

int

sample(size=None)

Draws samples from the CategoricalSampler object.

Parameters

size (*Optional[int], optional*) – Number of samples to draw. If None, draws a single sample. Defaults to None.

Return type

typing.Union[typing.Any, typing.List[typing.Any]]

Returns

Union[Any, List[Any]] – List of levels if size > 1, else a single sample from levels with prob probs.

Parameters

size (*int | None*)

Data Type: Sequence[int]**Integer Bernoulli Set Distribution**

Data Type: Sequence[int]

The integer Bernoulli set distribution is distribution over the power sets of size n . Each integer between $[0, n)$ is included in the set with probability p_i . Note there is no constraint $\sum_i p_i = 1$, as each p_i simply models the probability that integer i is included in the set. Let $x = (x_0, x_1, \dots, x_{n-1})$ be a tuple of binary variables indicating set membership. The probability mass function for a Bernoulli set distribution is given by

$$f(x|p) = \prod_{i=0}^{n-1} p_i^{x_i} (1 - p_i)^{1-x_i}$$

See [Bernoulli Set Distribution](#) for a more generic implementation over sets of any objects.

IntegerBernoulliSetDistribution

```
class dmx.stats.intsetdist.IntegerBernoulliSetDistribution(log_pvec, log_nvec=None,
                                                         name=None, keys=None)
```

Represent independent inclusions on integers $[\emptyset, K)$.

log_pvec[k] is the log probability that integer k is present. log_nvec[k], when supplied, is the log probability that it is absent; otherwise absence probabilities are derived as $\log(1 - \exp(\log_pvec))$.

Parameters

- **log_pvec** (*Sequence[float] | ndarray*)
- **log_nvec** (*Sequence[float] | ndarray | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

name

Name for object instance.

Type

Optional[str]

log_pvec

Shape (K,) log inclusion probabilities.

Type

np.ndarray

log_nvec

Optional log absence probabilities of shape (K,).

Type

Optional[Union[Sequence[float], np.ndarray]]

log_dvec

Shape (K,) inclusion log-odds.

Type

np.ndarray

log_nsum

Sum of finite log absence probabilities.

Type

float

keys

Key for tying sufficient statistics.

Type

Optional[str]

__init__ (*log_pvec, log_nvec=None, name=None, keys=None*)

Initialize an integer Bernoulli set distribution.

Parameters

- **log_pvec** (*Union[Sequence[float], np.ndarray]*) – Shape (K,) log inclusion probabilities, indexed directly by integers 0 to K - 1.
- **log_nvec** (*Optional[Union[Sequence[float], np.ndarray]]*) – Optional Shape (K,) log absence probabilities.
- **name** (*Optional[str]*) – Set name to object instance.
- **keys** (*Optional[str]*) – Set keys for object instance.

Parameters

- **log_pvec** (*Sequence[float] | ndarray*)

- **log_nvec** (*Sequence[float] | ndarray | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

density(*x*)

Evaluate the probability mass of one integer set.

Return type

float

Parameters**x** (*Sequence[int] | ndarray*)**dist_to_encoder**()

Create the compatible set-data encoder.

Return type`dmx.stats.intsetdist.IntegerBernoulliSetDataEncoder`**estimator**(*pseudo_count=None*)

Create an estimator initialized with this support size.

Return type`dmx.stats.intsetdist.IntegerBernoulliSetEstimator`**Parameters****pseudo_count** (*float | None*)**log_density**(*x*)

Evaluate the log-probability mass of one integer set.

Return type

float

Parameters**x** (*Sequence[int] | ndarray*)**sampler**(*seed=None*)

Create a sampler for this distribution.

Return type`dmx.stats.intsetdist.IntegerBernoulliSetSampler`**Parameters****seed** (*int | None*)**seq_log_density**(*x*)

Evaluate log masses for N encoded set observations.

Return type`numpy.ndarray`**Returns**

Array of shape (N,) with one log mass per set.

Parameters**x** (*IntegerBernoulliSetEncodedDataSequence*)

IntegerBernoulliSetEstimator

```
class dmx.stats.intsetdist.IntegerBernoulliSetEstimator(num_vals, min_prob=1.0e-128,  
                                                         pseudo_count=None, suff_stat=None,  
                                                         name=None, keys=None)
```

Estimate Bernoulli inclusion probabilities from weighted counts.

Parameters

- **num_vals** (*int*)
- **min_prob** (*float*)
- **pseudo_count** (*float* | *None*)
- **suff_stat** (*ndarray* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

num_vals

Number of values in integer range for the set.

Type

int

keys

Keys for merging sufficient statistics with matching key'd objects.

Type

Optional[str]

pseudo_count

Re-weight suff stats in estimation.

Type

Optional[float]

suff_stat

Probability for integer inclusion.

Type

Optional[np.ndarray]

name

Set name for object instance.

Type

Optional[str]

min_prob

Minimum probability for an integer in range of set dist.

Type

float

```
__init__(num_vals, min_prob=1.0e-128, pseudo_count=None, suff_stat=None, name=None, keys=None)
```

Initialize estimator support, smoothing, and metadata.

Parameters

- **num_vals** (*int*) – Number of values in integer range for the set.

- **min_prob** (*float*) – Minimum probability for an integer in range of set dist.
- **pseudo_count** (*Optional[float]*) – Re-weight suff stats in estimation.
- **suff_stat** (*Optional[np.ndarray]*) – Probability for integer inclusion.
- **name** (*Optional[str]*) – Set name for object instance.
- **keys** (*Optional[str]*) – Keys for merging sufficient statistics with matching key'd objects.

Parameters

- **num_vals** (*int*)
- **min_prob** (*float*)
- **pseudo_count** (*float | None*)
- **suff_stat** (*ndarray | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

accumulator_factory()

Create a compatible accumulator factory.

Return type`dmx.stats.intsetdist.IntegerBernoulliSetAccumulatorFactory`**estimate**(*nobs, suff_stat=None*)

Estimate log inclusion and absence probabilities.

Parameters

- **nobs** – Ignored legacy observation count.
- **suff_stat** – (positive_counts, total_weight) statistic, where positive_counts has shape (K,).

Return type`dmx.stats.intsetdist.IntegerBernoulliSetDistribution`**Returns**

Fitted distribution on integers [0, K).

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*ndarray | None*)

IntegerBernoulliSetSampler**class** `dmx.stats.intsetdist.IntegerBernoulliSetSampler`(*dist, seed=None*)

Draw independent integer sets from a Bernoulli set distribution.

Parameters

- **dist** (`IntegerBernoulliSetDistribution`)
- **seed** (*int | None*)

rng

RandomState object with seed set if passed in args.

Type

RandomState

dist

Object instance to sample from.

Type

IntegerBernoulliSetDistribution

sample(size=None)

Draw one integer set or a list of size sets.

Return type

typing.Union[typing.List[typing.Sequence[int]], typing.Sequence[int]]

Parameters

size (int | None)

Data Type: Sequence[str]**Bernoulli Set Distribution**

Data Type: Sequence[str]

The Bernoulli set distribution is distribution over the power sets of elements $V = \{v_0, v_1, \dots, v_{n-1}\}$. Each element v_i is included in the set with probability p_i . Note there is no constraint $\sum_i p_i = 1$, as each p_i simply models the probability that element v_i is included in the set. Let $\mathbf{x}$ be a subset of V . The probability mass function for a Bernoulli set distribution is given by

$$f(\mathbf{x}|\mathbf{p}) = \prod_{i=0}^{n-1} [v_i \in \mathbf{x}]p_i + [v_i \notin \mathbf{x}](1 - p_i).$$

For speed, the user can map observed values $v_i \rightarrow i$ and use the *Integer Categorical Distribution*.

BernoulliSetDistribution

```
class dmx.stats.setdist.BernoulliSetDistribution(pmap, min_prob=1.0e-128, name=None,
                                                keys=None)
```

Model an unordered set by independent element-inclusion probabilities.

Each key of pmap defines one possible element. Duplicate values in an input are counted repeatedly, so callers must supply collections with unique elements.

Parameters

- **pmap** (Dict[Any, float])
- **min_prob** (float)
- **name** (str | None)
- **keys** (str | None)

keys

Keys for object instance.

Type

Optional[str]

name

Name to object instance.

Type

Optional[str]

pmap

Maps elements in support to probabilities.

Type

Dict[Any, float]

required

An observation must contain this subset of elements. Else, return probability 0.0.

Type

Set

nlog_sum

Normalizing term for computing numerically stable likelihood.

Type

float

log_dmap

Map from elements to their corrected log probability

Type

Dict[Any, float]

of inclusion in the set.**min_prob**

Minimum probability for elements. Corrects for prob = 0.

Type

float

num_required

Number of required elements in a subset. Corrected if min_prob was non-zero.

Type

int

__init__ (*pmap*, *min_prob*=1.0e-128, *name*=None, *keys*=None)

Initialize an independent Bernoulli set distribution.

Parameters

- **pmap** (*Dict*[Any, *float*]) – Maps values to probabilities.
- **min_prob** (*float*) – Minimum probability for numerical stability in log prob calculations.
- **name** (*Optional*[*str*]) – Set name to object instance.
- **keys** (*Optional*[*str*]) – Set keys for object instance.

Parameters

- **pmap** (*Dict*[Any, *float*])
- **min_prob** (*float*)
- **name** (*str* | None)

- **keys** (*str* / *None*)

Return type*None***density**(*x*)

Evaluate the probability of the unordered set *x*.

Return type*float***Parameters***x* (*Sequence*[*Any*])**dist_to_encoder**()

Create an encoder for batches of unordered set observations.

Return type*dmx.stats.setdist.BernoulliSetDataEncoder***estimator**(*pseudo_count=None*)

Create an estimator, optionally centered on this distribution.

Return type*dmx.stats.setdist.BernoulliSetEstimator***Parameters***pseudo_count* (*float* / *None*)**log_density**(*x*)

Evaluate the log probability of the unordered set *x*.

Input order is ignored. Every value must be a key in *pmap* and callers must exclude duplicates, which the implementation would count repeatedly.

Return type*float***Parameters***x* (*Sequence*[*Any*])**sampler**(*seed=None*)

Create a sampler using independent inclusion draws.

Return type*dmx.stats.setdist.BernoulliSetSampler***Parameters***seed* (*int* / *None*)**seq_log_density**(*x*)

Evaluate log probabilities for encoded set observations.

Return type*numpy.ndarray***Parameters***x* (*BernoulliSetEncodedDataSequence*)

BernoulliSetEstimator

```
class dmx.stats.setdist.BernoulliSetEstimator(min_prob=1.0e-128, pseudo_count=None,  
                                              suff_stat=None, name=None, keys=None)
```

Estimate inclusion probabilities from weighted element counts.

Parameters

- **min_prob** (*float*)
- **pseudo_count** (*float | None*)
- **suff_stat** (*Dict[Any, float] | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

min_prob

Minimum probability for elements estimated with prob = 0.

Type

float

pseudo_count

Used to re-weight suff_stats in estimation.

Type

Optional[float]

suff_stat

Optional dictionary containing value to probability mapping.

Type

Optional[Dict[Any, float]]

name

Set name for object instance.

Type

Optional[str]

keys

Set key for merging sufficient statistics.

Type

Optional[str]

```
__init__(min_prob=1.0e-128, pseudo_count=None, suff_stat=None, name=None, keys=None)
```

Initialize the Bernoulli set estimator.

Parameters

- **min_prob** (*float*) – Minimum probability for elements estimated with prob = 0.
- **pseudo_count** (*Optional[float]*) – Used to re-weight suff_stats in estimation.
- **suff_stat** (*Optional[Dict[Any, float]]*) – Optional dictionary containing value to probability mapping.
- **name** (*Optional[str]*) – Set name for object instance.
- **keys** (*Optional[str]*) – Set key for merging sufficient statistics.

Parameters

- **min_prob** (*float*)
- **pseudo_count** (*float* | *None*)
- **suff_stat** (*Dict[Any, float]* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

Return type*None***accumulator_factory()**

Create a factory for compatible sufficient-statistic accumulators.

Return type*dmx.stats.setdist.BernoulliSetAccumulatorFactory***estimate**(*nobs*, *suff_stat*)

Estimate a distribution from element counts and total set weight.

nobs is ignored. With no prior statistics, each inclusion probability is its weighted count divided by the total weight. A pseudo-count alone adds a symmetric half-present, half-absent prior for each observed element.

Return type*dmx.stats.setdist.BernoulliSetDistribution***Parameters**

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple[Dict[Any, float], float]*)

BernoulliSetSampler

```
class dmx.stats.setdist.BernoulliSetSampler(dist, seed=None)
```

Generate unordered-set observations from a Bernoulli set distribution.

Parameters

- **dist** (*BernoulliSetDistribution*)
- **seed** (*int* | *None*)

dist

Object instance to sample from.

Type*BernoulliSetDistribution***seed**

Set seed for random number generator.

Type*Optional[int]***sample**(*size=None*)

Draw one set, or *size* independent sets.

Return type

`typing.Union[typing.Sequence[typing.Any],
Sequence[typing.Any]]` `typing.List[typing.`

Parameters

size (*int* | *None*)

Data Type: `Sequence[Tuple[int, float]]`

Integer Multinomial Distribution

Data Type: `Sequence[Tuple[int, float]]`

The integer multinomial distribution is a generalization of the binomial distribution to k classes. The multinomial give the probability of observing n_k success of class k in $n = \sum_i n_i$ trials. The probability mass function is given by

$$f(\mathbf{x}|n, \mathbf{p}) = \frac{n!}{x_1! \dots x_k!} p_1^{x_1} \dots p_k^{x_k},$$

where $\sum_{i=1}^k p_i = 1$ and $\sum_{i=1}^k x_i = n$.

For more info see [Multinomial Distribution](#).

IntegerMultinomialDistribution

```
class dmx.stats.intmultinomial.IntegerMultinomialDistribution(min_val=0, p_vec=None,  
                                                             len_dist=NoneDistribution(),  
                                                             name=None, keys=None)
```

Represent multinomial counts on consecutive integer categories.

This implementation scores the product of category probabilities raised to their counts, together with the total-count distribution. It intentionally omits the multinomial combinatorial coefficient.

Parameters

- **min_val** (*int*)
- **p_vec** (*List[float]* | *ndarray* | *None*)
- **len_dist** (*SequenceEncodableProbabilityDistribution* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

p_vec

Probability of each integer category for a trial.

Type

`ndarray`

min_val

Smallest integer value for category range. Defaults to 0.

Type

`int`

max_val

Largest value of category range. Set by `min_val + len(p_vec) - 1`.

Type

`int`

log_p_vec

Log of p_vec member instance.

Type

ndarray

num_vals

Total number of integer valued categories.

Type

int

len_dist

Distribution for number of trials. Set to NullDistribution if None.

Type

SequenceEncodableProbabilityDistribution

keys

Keys for distribution passed when ParameterEstimator is created.

Type

Optional[str]

name

Name for object instance.

Type

Optional[str]

__init__(min_val=0, p_vec=None, len_dist=None, name=None, keys=None)

Initialize category probabilities and the total-count model.

Parameters

- **min_val** (*int*) – Set the minimum value on range of values.
- **p_vec** (*Union[List[float], np.ndarray]*) – Probabilities for values. Length determines number of categories.
- **len_dist** (*Optional[SequenceEncodableProbabilityDistribution]*) – Optional length distributions serving as for the number of trials.
- **name** (*Optional[str]*) – Set name for object instance.
- **keys** (*Optional[str]*) – Set key for distribution.

Parameters

- **min_val** (*int*)
- **p_vec** (*List[float] | ndarray | None*)
- **len_dist** (*SequenceEncodableProbabilityDistribution | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

density(*x*)

Evaluate the density of IntegerMultinomialDistribution at observed value *x*.

Parameters

x (*Sequence[Tuple[int, float]]*) – Sequence of Tuple(s) containing the integer category value and number of successes.

Return type

float

Returns

float – Density at *x*.

Parameters

x (*Sequence[Tuple[int, float]]*)

dist_to_encoder()

Create an encoder compatible with the total-count distribution.

Return type

`dmx.stats.intmultinomial.IntegerMultinomialDataEncoder`

estimator(*pseudo_count=None*)

Create an estimator retaining support and optional smoothing.

Return type

`dmx.stats.intmultinomial.IntegerMultinomialEstimator`

Parameters

pseudo_count (*float | None*)

log_density(*x*)

Evaluate the log mass of one sparse count observation.

Parameters

x (*Sequence[Tuple[int, float]]*) – Sequence of Tuple(s) containing the integer category value and number of successes.

Return type

float

Returns

float – Log-density at *x*.

Parameters

x (*Sequence[Tuple[int, float]]*)

sampler(*seed=None*)

Create a sampler when a total-count distribution is configured.

Return type

`dmx.stats.intmultinomial.IntegerMultinomialSampler`

Parameters

seed (*int | None*)

seq_log_density(*x*)

Evaluate log masses for *N* encoded count observations.

Return type

`numpy.ndarray`

Parameters**x** (*IntegerMultinomialEncodedDataSequence*)**IntegerMultinomialEstimator**

```
class dmx.stats.intmultinomial.IntegerMultinomialEstimator(min_val=None, max_val=None,
                                                            len_estimator=None, len_dist=None, name=None,
                                                            pseudo_count=None, suff_stat=None,
                                                            keys=None)
```

Estimate category and total-count distributions from statistics.

Parameters

- **min_val** (*int* | *None*)
- **max_val** (*int* | *None*)
- **len_estimator** (*ParameterEstimator* | *None*)
- **len_dist** (*SequenceEncodableProbabilityDistribution* | *None*)
- **name** (*str* | *None*)
- **pseudo_count** (*float* | *None*)
- **suff_stat** (*Tuple[int, ndarray]* | *None*)
- **keys** (*str* | *None*)

min_val

Set minimum value integer multinomial.

Type*Optional[int]***max_val**

Set maximum value for integer multinomial.

Type*Optional[int]***len_estimator**

ParameterEstimator for number of trials, set to *NullEstimator()* if *None* is passed as arg.

Type*ParameterEstimator***len_dist**

Optional SequenceEncodableProbabilityDistribution for fixing distribution on number of trials.

Type*Optional[SequenceEncodableProbabilityDistribution]***name**

Set name for object instance.

Type*Optional[str]*

pseudo_count

Used to re-weight sufficient statistics if `suff_stat` is passed.

Type

Optional[float]

suff_stat

Set minimum value and counts for categories. If 'min_val' and 'max_val' are both not None, this is ignored in estimation.

Type

Optional[Tuple[int, np.ndarray]]

keys

Set key for merging sufficient statistics of objects with matching keys.

Type

Optional[str]

__init__(*min_val=None, max_val=None, len_estimator=None, len_dist=None, name=None, pseudo_count=None, suff_stat=None, keys=None*)

Initialize support, smoothing, length estimation, and metadata.

Parameters

- **min_val** (*Optional[int]*) – Set minimum value integer multinomial.
- **max_val** (*Optional[int]*) – Set maximum value for integer multinomial.
- **len_estimator** (*Optional[ParameterEstimator]*) – Optional ParameterEstimator for number of trials.
- **len_dist** (*Optional[SequenceEncodableProbabilityDistribution]*) – Optional SequenceEncodableProbabilityDistribution for fixing distribution on number of trials.
- **name** (*Optional[str]*) – Set name for object instance.
- **pseudo_count** (*Optional[float]*) – Used to re-weight sufficient statistics if `suff_stat` is passed.
- **suff_stat** (*Optional[Tuple[int, np.ndarray]]*) – Set minimum value and counts for categories.
- **keys** (*Optional[str]*) – Set key for merging sufficient statistics of objects with matching keys.

Parameters

- **min_val** (*int | None*)
- **max_val** (*int | None*)
- **len_estimator** (*ParameterEstimator | None*)
- **len_dist** (*SequenceEncodableProbabilityDistribution | None*)
- **name** (*str | None*)
- **pseudo_count** (*float | None*)
- **suff_stat** (*Tuple[int, ndarray] | None*)
- **keys** (*str | None*)

Return type

None

accumulator_factory()

Create a compatible accumulator factory.

Return type`dmx.stats.intmultinomial.IntegerMultinomialAccumulatorFactory`**estimate(nobs, suff_stat)**

Estimate a model from category and length sufficient statistics.

Return type`dmx.stats.intmultinomial.IntegerMultinomialDistribution`**Parameters**

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*int*, *ndarray*, *SS0* | *None*])

IntegerMultinomialSampler**class** `dmx.stats.intmultinomial.IntegerMultinomialSampler`(*dist*, *seed=None*)

Draw sparse integer-category count observations.

Parameters

- **dist** (*IntegerMultinomialDistribution*)
- **seed** (*int* | *None*)

dist*IntegerMultinomialDistribution* object instance to sample from.**Type***IntegerMultinomialDistribution***rng**

RandomState set with seed if passed.

Type

RandomState

len_sampler

DistributionSampler object for number of trials.

Type*DistributionSampler***sample(size=None)**

Draw independent samples from an integer multinomial distribution.

Parameters**size** (*Optional*[*int*]) – Number of samples to draw.**Return type**`typing.Union[typing.List[typing.Tuple[int, float]], typing.List[typing.List[typing.Tuple[int, float]]]`**Returns**List of samples. If size is None, returns one sample as a `List[Tuple[int, float]]`.

Parameters**size** (*int* / *None*)**Data Type:** Sequence[Tuple[int, str]]**Multinomial Distribution**

Data Type: Sequence[Tuple[int, str]]

The multinomial distribution is a generalization of the binomial distribution to k classes. The multinomial give the probability of observing x_k success/counts of class object v_k in $n = \sum_i x_i$ trials. The probability mass function is given by

$$f(\mathbf{x}|\mathbf{n}, \mathbf{p}) = \frac{n!}{x_1! \dots x_k!} p_1^{x_1} \dots p_k^{x_k},$$

where $\sum_{i=1}^k p_i = 1$ and $\sum_{i=1}^k x_i = n$. Here we have allowed the classes to be represented by an object/string v_i in the set $V = \{v_1, \dots, v_k\}$. If the user maps the objects to the set of integers, the *Integer Multinomial Distribution* can be used instead.

For more info see [Multinomial Distribution](#).

MultinomialDistribution

```
class dmx.stats.catmultinomial.MultinomialDistribution(dist, len_dist=None, len_normalized=False,
                                                    name=None, keys=None)
```

Represent multinomial counts over a countable category support.

The scalar likelihood multiplies category masses raised to their counts and the mass assigned to the total count. It intentionally omits the multinomial combinatorial coefficient. With `len_normalized`, category contributions are divided by the total count.

Parameters

- **dist** (*SequenceEncodableProbabilityDistribution*) – Distribution with at most a countable support.
- **len_dist** (*Optional[SequenceEncodableProbabilityDistribution]*, *optional*) – Distribution for the number of trials. Defaults to `NullDistribution()`.
- **len_normalized** (*bool*, *optional*) – Take geometric mean of the density of observation. Defaults to `False`.
- **name** (*Optional[str]*, *optional*) – Name for the object instance. Defaults to `None`.
- **keys** (*Optional[str]*, *optional*) – Keys for merging sufficient statistics. Defaults to `None`.

Parameters

- **dist** (*SequenceEncodableProbabilityDistribution*)
- **len_dist** (*Optional[SequenceEncodableProbabilityDistribution]*)
- **len_normalized** (*bool*)
- **name** (*Optional[str]*)
- **keys** (*Optional[str]*)

dist

Distribution with at most a countable support.

Type*SequenceEncodableProbabilityDistribution***len_dist**

Distribution for the number of trials.

TypeOptional[*SequenceEncodableProbabilityDistribution*]**len_normalized**

Take geometric mean of the density of observation.

Type

bool

name

Name for the object instance.

Type

Optional[str]

keys

Keys for merging sufficient statistics.

Type

Optional[str]

__init__(*dist*, *len_dist*=None, *len_normalized*=False, *name*=None, *keys*=None)

Initialize category and total-count distributions.

Parameters

- **dist** (*SequenceEncodableProbabilityDistribution*) – Distribution with at most a countable support.
- **len_dist** (*Optional[SequenceEncodableProbabilityDistribution]*, *optional*) – Distribution for the number of trials. Defaults to `NullDistribution()`.
- **len_normalized** (*bool*, *optional*) – Take geometric mean of the density of observation. Defaults to `False`.
- **name** (*Optional[str]*, *optional*) – Name for the object instance. Defaults to `None`.
- **keys** (*Optional[str]*, *optional*) – Keys for merging sufficient statistics. Defaults to `None`.

Parameters

- **dist** (*SequenceEncodableProbabilityDistribution*)
- **len_dist** (*SequenceEncodableProbabilityDistribution* / *None*)
- **len_normalized** (*bool*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

Return type

None

density(*x*)Returns the density of multinomial evaluated at observation *x*.

Parameters

x (*Sequence[Tuple[T, float]]*) – Tuples of observed multinomial values and successes such that the successes sum to the number of trials.

Return type

float

Returns

float – Density evaluated at x.

Parameters

x (*Sequence[Tuple[T, float]]*)

dist_to_encoder()

Get a data encoder for this distribution.

Return type

`dmx.stats.catmultinomial.MultinomialDataEncoder`

Returns

MultinomialDataEncoder – Encoder for this distribution.

estimator(pseudo_count=None)

Create a MultinomialEstimator object from this distribution.

Parameters

pseudo_count (*Optional[float], optional*) – Re-weight member sufficient statistics when estimating from aggregated data.

Return type

`dmx.stats.catmultinomial.MultinomialEstimator`

Returns

MultinomialEstimator – Estimator for this distribution.

Parameters

pseudo_count (*float | None*)

log_density(x)

Returns the log-density of multinomial evaluated at observation x.

Parameters

x (*Sequence[Tuple[T, float]]*) – Tuples of observed multinomial values and successes such that the successes sum to the number of trials.

Return type

float

Returns

float – Log-density evaluated at x.

Parameters

x (*Sequence[Tuple[T, float]]*)

sampler(seed=None)

Create a MultinomialSampler object from this distribution.

Parameters

seed (*Optional[int], optional*) – Seed for sampling. Defaults to None.

Return type

`dmx.stats.catmultinomial.MultinomialSampler`

Returns

MultinomialSampler – Sampler for this distribution.

Raises

Exception – If `len_dist` is a `NullDistribution`.

Parameters

`seed` (*int* | *None*)

seq_log_density(*x*)

Vectorized log-density for encoded data.

Parameters

x (*MultinomialEncodedDataSequence*) – Encoded sequence.

Return type

`numpy.ndarray`

Returns

np.ndarray – Log-densities for each observation.

Raises

Exception – If input is not a `MultinomialEncodedDataSequence`.

Parameters

x (*MultinomialEncodedDataSequence*)

MultinomialEstimator

```
class dmx.stats.catmultinomial.MultinomialEstimator(estimator, len_estimator=NoneEstimator(),
                                                    pseudo_count=None, len_dist=None,
                                                    len_normalized=False, name=None,
                                                    keys=None)
```

Estimate category and total-count distributions from statistics.

Parameters

- **estimator** (*ParameterEstimator*)
- **len_estimator** (*Optional* [*ParameterEstimator*])
- **pseudo_count** (*Optional* [*float*])
- **len_dist** (*Optional* [*SequenceEncodableProbabilityDistribution*])
- **len_normalized** (*bool*)
- **name** (*Optional* [*str*])
- **keys** (*Optional* [*str*])

estimator

ParameterEstimator for distribution of values.

Type

ParameterEstimator

len_estimator

ParameterEstimator for the number of trials, defaults to the `NullEstimator` if `None` is passed.

Type

ParameterEstimator

pseudo_count

Regularizer estimator and len_estimator.

Type

Optional[float]

len_dist

If None, distribution for number of trials will be estimated from 'len_estimator'.

Type

Optional[*SequenceEncodableProbabilityDistribution*]

len_normalized

Take geometric mean of density.

Type

Optional[bool]

name

Name of object instance.

Type

Optional[str]

keys

Keys of object instance for merging sufficient statistics.

Type

Optional[str]

__init__(*estimator*, *len_estimator*=*NullEstimator*(), *pseudo_count*=*None*, *len_dist*=*None*,
len_normalized=*False*, *name*=*None*, *keys*=*None*)

Initialize category and length estimators.

Parameters

- **estimator** (*ParameterEstimator*) – ParameterEstimator for distribution of values.
- **len_estimator** (*Optional[ParameterEstimator]*) – Optional ParameterEstimator for the number of trials.
- **pseudo_count** (*Optional[float]*) – Regularizer estimator and len_estimator.
- **len_dist** (*Optional[SequenceEncodableProbabilityDistribution]*) – Set distribution for the number of trials.
- **len_normalized** (*bool*) – Take geometric mean of density.
- **name** (*Optional[str]*) – Set name to object instance.
- **keys** (*Optional[str]*) – Set keys to object instance for merging sufficient statistics.

Parameters

- **estimator** (*ParameterEstimator*)
- **len_estimator** (*ParameterEstimator* | *None*)
- **pseudo_count** (*float* | *None*)
- **len_dist** (*SequenceEncodableProbabilityDistribution* | *None*)
- **len_normalized** (*bool*)
- **name** (*str* | *None*)

- **keys** (*str* | *None*)

Return type

None

accumulator_factory()

Create a compatible accumulator factory.

Return type

dmx.stats.catmultinomial.MultinomialAccumulatorFactory

estimate(*nobs*, *suff_stat*)

Estimate a distribution from category and length statistics.

Parameters

- **nobs** (*Optional[float]*) – Number of observations used in aggregation of ‘suff_stat’.
- **suff_stat** (*Tuple[SS1, Optional[SS2]]*) – Tuple of sufficient statistics for distribution of values and trial distribution.

Return type

dmx.stats.catmultinomial.MultinomialDistribution

Returns

MultinomialDistribution – Estimate from sufficient statistics.

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple[SS1, SS2 | None]*)

MultinomialSampler

class *dmx.stats.catmultinomial.MultinomialSampler*(*dist*, *seed=None*)

MultinomialSampler object for sampling from multinomial distribution.

Parameters

- **dist** (*MultinomialDistribution*)
- **seed** (*Optional[int]*)

dist

An instance of a MultinomialDistribution object.

Type

MultinomialDistribution

rng

RandomState with seed set if passed.

Type

RandomState

dist_sampler

DistributionSampler object for sampling category values.

Type

DistributionSampler

len_sampler

DistributionSampler object for sampling number of trials in multinomial.

Type

DistributionSampler

sample(size=None)

Draw samples from multinomial distribution.

Note: If len_sampler can draw n=0, an empty list is returned for that sample.

Parameters

size (*Optional[int]*) – Number of iid samples to draw from multinomial.

Return type

`typing.Union[typing.Sequence[typing.Sequence[typing.Tuple[typing.Any, float]]], typing.Sequence[typing.Tuple[typing.Any, float]]]`

Returns

Union[Sequence[Sequence[Tuple[Any, float]]], Sequence[Tuple[Any, float]]] – Sequence of ‘size’ iid observations if size is not None, else a single multinomial sample.

Parameters

size (*int | None*)

Data Type: Sequence[Any]**Markov Chain**

A Markov chain is a stochastic process that undergoes transitions from one state to another within a finite or countable number of possible states. It is characterized by the property that the future state depends only on the current state and not on the sequence of events that preceded it. This property is known as the Markov property.

Mathematically, a Markov chain can be defined as follows:

- Let S be a finite set of states.
- Let P be the transition matrix, where P_{ij} represents the probability of moving from state i to state j .

The transition probabilities must satisfy the following conditions:

$$\sum_{j \in S} P_{ij} = 1 \quad \text{for all } i \in S$$

Likelihood Evaluation

Assume we have observed a sequence $\mathbf{x} = (x_1, \dots, x_n)$ of length n . The likelihood is given by

$$p(\mathbf{x}_i) = p(N = n)p(x_1) \prod_{k=1}^n p(x_k | x_{k-1})$$

In the above equation, $p(x_k | x_{k-1})$ is defined by the transition matrix, $p(x_1)$ is defined by the initial state vector π , and $p(N = n)$ is a length distribution on the integers.

MarkovChainDistribution

```
class dmx.stats.markovchain.MarkovChainDistribution(init_prob_map, transition_map,
                                                    len_dist=NullDistribution(), default_value=0.0,
                                                    name=None, keys=None)
```

Represent a first-order Markov chain over states of type T.

Each observation is a complete state sequence. Initial and transition maps need only contain explicitly modeled states; `default_value` supplies mass for missing entries using the implementation's fallback normalization.

Parameters

- **init_prob_map** (*Dict[T, float]*)
- **transition_map** (*Dict[T, Dict[T, float]]*)
- **len_dist** (*SequenceEncodableProbabilityDistribution / None*)
- **default_value** (*float*)
- **name** (*str / None*)
- **keys** (*str / None*)

init_prob_map

Probability of each initial values of data type T.

Type

Dict[T, float]

transition_map

Transition probability map.

Type

Dict[T, Dict[T, float]]

len_dist

Length distribution for length of observation sequence.

Type

Optional[*SequenceEncodableProbabilityDistribution*]

default_value

Default probability for value outside support.

Type

float

name

Set name to MarkovChainDistribution object.

Type

Optional[*str*]

all_vals

Set of all values in state-space.

Type

Set[T]

loginit_prob_map

Dictionary mapping initial state value to log probability.

Type

Dict[T, float]

log_transition_map

Dictionary mapping state to state transition log-probabilities.

Type

Dict[T, Dict[T, float]]

log_dv

Log default value.

Type

float

log_dtv

Log of default value scaled by number of state-values + 1.

Type

float

log1p_dv

Log of 1 plus default_value.

Type

float

key_map

Maps each state-value in all_vals to integer [1, len(all_vals)+1]

Type

Dict[T, int]

inv_key_map

List of all state-values (keys).

Type

List[T]

num_keys

Number of state-values (len(keys)).

Type

int

init_log_pvec

Log-probabilities of each initial value. Entry 0, is log_dv. (len == num_keys+1).

Type

ndarray

trans_log_pvec

Dictionary of keys for sparse log transition probabilities.

Type

dok_matrix

keys

Key used to share accumulated sufficient statistics.

Type

Optional[str]

__init__(*init_prob_map, transition_map, len_dist=NullDistribution(), default_value=0.0, name=None, keys=None*)

Initialize a generic Markov-chain distribution.

Parameters

- **init_prob_map** (*Dict[T, float]*) – Probability of each initial values of data type T.
- **transition_map** (*Dict[T, Dict[T, float]]*) – Transition probability map.
- **len_dist** (*Optional[SequenceEncodableProbabilityDistribution]*) – Length distribution for length of observation sequence.
- **default_value** (*float*) – Default probability for value outside support.
- **name** (*Optional[str]*) – Set name to MarkovChainDistribution object.
- **keys** (*Optional[str]*) – Key used to share accumulated sufficient statistics.

Parameters

- **init_prob_map** (*Dict[T, float]*)
- **transition_map** (*Dict[T, Dict[T, float]]*)
- **len_dist** (*SequenceEncodableProbabilityDistribution | None*)
- **default_value** (*float*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

density(*x*)

Return density of MarkovChainDistribution at observed sequence x.

Returns exponential of log_density(x). See log_density() for details.

Parameters

x (*List[T]*) – An observed Markov chain sequence of data type T.

Return type

float

Returns

float – Density of Markov chain at x.

Parameters

x (*List[T]*)

dist_to_encoder()

Create an encoder for batches of state sequences.

Return type

dmx.stats.markovchain.MarkovChainDataEncoder

estimator(*pseudo_count=None*)

Create an estimator using this distribution's length model.

Return type

dmx.stats.markovchain.MarkovChainEstimator

Parameters**pseudo_count** (*float* | *None*)**log_density(x)**

Return log-density of MarkovChainDistribution at observed sequence x.

Parameters**x** (*List[T]*) – An observed Markov chain sequence of data type T.**Return type***float***Returns***float* – Log-density of Markov chain at x.**Parameters****x** (*List[T]*)**sampler(seed=None)**

Create a sampler for complete state sequences.

Return type*dmx.stats.markovchain.MarkovChainSampler***Parameters****seed** (*int* | *None*)**seq_log_density(x)**

Evaluate log densities for an encoded batch of state sequences.

Parameters**x** – Batch produced by MarkovChainDataEncoder.**Return type***numpy.ndarray***Returns**

One log density per input sequence.

Raises**TypeError** – If x is not a compatible encoded batch.**Parameters****x** (*MarkovChainEncodedDataSequence*)**MarkovChainEstimator**

```
class dmx.stats.markovchain.MarkovChainEstimator(pseudo_count=None, levels=None,
                                                  len_estimator=NoneEstimator(), name=None,
                                                  keys=None)
```

Estimate a generic Markov chain from aggregated weighted counts.

Notes

With `pseudo_count=None`, initial-state and transition probabilities are normalized from observed counts only. With a pseudo-count, the estimator builds a smoothing support from observed initial states, observed transition targets, and any values supplied in `levels`. Unseen states receive pseudo-count mass only if they are included in that support, so pass `levels` when a known state space should be available during initialization or sparse fitting.

The length distribution is estimated by `len_estimator` and is not smoothed by the Markov-chain pseudo-count.

Parameters

- **pseudo_count** (*float* | *None*)
- **levels** (*Iterable[T]* | *None*)
- **len_estimator** (*ParameterEstimator* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

pseudo_count

Smoothing mass for initial-state and transition probabilities.

Type

*Optional[*float*]*

levels

Optional state values to include in the smoothing support.

Type

*Optional[*Iterable[T]*]*

len_estimator

NullEstimator if no length distribution is to be estimated.

Type

ParameterEstimator

name

Name for instance of *MarkovChainEstimator*.

Type

*Optional[*str*]*

keys

Keys for merging sufficient statistics of *MarkovChainAccumulator* objects.

Type

*Optional[*str*]*

__init__(*pseudo_count=None, levels=None, len_estimator=None, name=None, keys=None*)

Initialize a Markov-chain estimator.

Parameters

- **pseudo_count** (*Optional[*float*]*) – Smoothing mass for initial-state and transition probabilities. If set, mass is spread uniformly over the support from observed states plus **levels**.
- **levels** (*Optional[*Iterable[T]*]*) – Optional state values to include in the smoothing support even if they are not observed.
- **len_estimator** (*Optional[*ParameterEstimator*]*) – *ParameterEstimator* for length of Markov sequences.
- **name** (*Optional[*str*]*) – Set a name for instance of *MarkovChainEstimator*.
- **keys** (*Optional[*str*]*) – Set keys for merging sufficient statistics of *MarkovChainAccumulator* objects.

Parameters

- **pseudo_count** (*float* | *None*)

- **levels** (*Iterable[T] | None*)
- **len_estimator** (*ParameterEstimator | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

accumulator_factory()

Create a factory for compatible sufficient-statistic accumulators.

Return type`dmx.stats.markovchain.MarkovChainAccumulatorFactory[typing.TypeVar(T)]`**estimate(nobs, suff_stat)**

Estimate parameters, applying pseudo-count smoothing when configured.

Return type`dmx.stats.markovchain.MarkovChainDistribution[typing.TypeVar(T)]`**Parameters**

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[Dict[T, float], Dict[T, Dict[T, float]], Any | None]*)

estimate0(nobs, suff_stat)

Estimate an unsmoothed Markov chain from sufficient statistics.

Maximum likelihood estimates for initial state probabilities, transition probabilities, and the length distribution are obtained directly from aggregated data in 'suff_stat'.

Arg suff_stat is a Tuple of length three containing,

suff_stat[0] (Dict[T, float]): Maps initial state values to their aggregated counts. suff_stat[1] (Dict[T, Dict[T, List[float]]]): Maps state to state transition counts. suff_stat[2] (T1): Sufficient statistic value of length accumulator. (Assumed type T1).

Parameters

- **nobs** (*Optional[float]*) – Number of observations. Passed to estimate1() or estimate2().
- **suff_stat** – Seed above for details.

Return type`dmx.stats.markovchain.MarkovChainDistribution`**Returns**

MarkovChainDistribution object.

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[Dict[T, float], Dict[T, Dict[T, float]], Any | None]*)

estimate1(*nobs*, *suff_stat*)

Estimate a pseudo-count-smoothed chain from sufficient statistics.

Maximum likelihood estimates for initial state probabilities, transition probabilities, and the length distribution are obtained by a weighted aggregation of sufficient statistics in 'suff_stat', and member variables of MarkovChainEstimator object.

The smoothing support is the union of observed initial states, observed transition targets, and levels. States absent from that union do not receive pseudo-count mass.

Arg suff_stat is a Tuple of length three containing,

suff_stat[0] (Dict[T, float]): Maps initial state values to their aggregated counts. suff_stat[1] (Dict[T, Dict[T, List[float]]]): Maps state to state transition counts. suff_stat[2] (T1): Sufficient statistic value of length accumulator. (Assumed type T1).

Parameters

- **nobs** (*Optional[float]*) – Number of observations. Passed to estimate1() or estimate2().
- **suff_stat** – Seed above for details.

Return type

`dmx.stats.markovchain.MarkovChainDistribution[typing.TypeVar(T)]`

Returns

MarkovChainDistribution object.

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[Dict[T, float], Dict[T, Dict[T, float]], Any | None]*)

MarkovChainSampler

class `dmx.stats.markovchain.MarkovChainSampler`(*dist*, *seed=None*)

Sample complete sequences from a generic Markov chain.

Parameters

- **dist** (`MarkovChainDistribution[T]`)
- **seed** (*int | None*)

rng

RandomState object for setting seed of random sampler.

Type

RandomState

init_prob

Tuple of initial state-values and probabilities.

Type

Tuple[List[T], List[float]]

trans_prob

Dictionary mapping transition probabilities from state i to state j.

Type

Dict[T, Tuple[List[T], List[float]]]

len_sampler

Sample length of Markov chain sequence. Must be a `DistributionSampler` with support on non-negative integers.

Type

DistributionSampler

sample(*size=None*)

Draw iid samples from Markov chain distribution.

If *size* is `None`, sample *N* from `len_sampler()` and return a `List[T]` of length *N*, where *T* is the data type of the Markov chain. If *size* > 0, return a list of length *size*, containing `List[T]` data types.

Parameters

size (*Optional[int]*) – Number of samples to draw. Draws 1 sample if `None`.

Return type

`typing.Union[typing.List[typing.TypeVar(T)], typing.List[typing.List[typing.TypeVar(T)]]]`

Returns

`List[T]` or `List[List[T]]`, depending on *size* arg.

Parameters

size (*int* | *None*)

sample_seq(*size=None*, *v0=None*)

Sample a fixed-length chain, optionally from a supplied state.

If *size* is `None`, draw a sequence of length 1, returning as type *T*.

If *size* is not `None`, draw a sequence of length *size*, returning as type `List[T]`.

If *v0* is `None`, *v0* is sampled from member variable ‘*init_prob*’.

Parameters

- **size** (*Optional[int]*) – Length of Markov chain sequence to sample.
- **v0** (*Optional[T]*) – Initial state of Markov chain sequence to sample from.

Return type

`typing.Union[typing.TypeVar(T), typing.List[typing.TypeVar(T)]]`

Returns

T or `List[T]` depending on arg *size*.

Parameters

- **size** (*int* | *None*)
- **v0** (*T* | *None*)

Combinator Distributions

dmx-learn was designed to handle heterogenous tuples of data. The *base distributions* provide suitable building blocks for a wide array of data types for each component of an observed data tuple. The *Combinator* distributions provide a means to stitch together the individual tuple components.

Contents

Composite Distribution

The composite distribution is the staple distribution of *dmx-learn* that allows for distributions over heterogeneous tuples of data. Assume we have observed a d -dimensional tuple $x = (x_1, x_2, \dots, x_d)$ with component-wise data types $(T_1, T_2, \dots, T_d)$. The composite distribution models the tuple with a likelihood

$$f(x_1, \dots, x_d | \theta_1, \dots, \theta_k) = \prod_{i=1}^d f(x_i | \theta_i)$$

where $f(x_i | \theta_i)$ are distributions compatible with component data type T_i .

CompositeDistribution

class `dmx.stats.composite.CompositeDistribution`(*dists*, *name=None*, *keys=None*)

Model a tuple as independent draws from positional child distributions.

The support is the Cartesian product of the child supports and the density is the product of the child densities. Consequently, the wrapper is normalized when every child is normalized. Scoring, encoding, sampling, accumulation, and estimation are all delegated position by position; children do not share state unless their own estimators or the composite `keys` contract says otherwise. At least one child is required, and every observation tuple must have the same number and order of fields as `dists`.

Parameters

- **dists** (`Sequence[SequenceEncodableProbabilityDistribution]`)
- **name** (`str` | `None`)
- **keys** (`str` | `None`)

dists

Distributions for each component.

Type

`Tuple[SequenceEncodableProbabilityDistribution, ...]`

count

Number of components (i.e. `len(dists)`).

Type

`int`

name

Name of object.

Type

`Optional[str]`

keys

Key for marking shared parameters.

Type

`Optional[str]`

__init__(*dists*, *name=None*, *keys=None*)

Create an instance of `CompositeDistribution`.

Parameters

- **dists** (*Sequence[SequenceEncodableProbabilityDistribution]*) – Component distributions.
- **name** (*Optional[str], optional*) – Name of object. Defaults to None.
- **keys** (*Optional[str], optional*) – Key for marking shared parameters. Defaults to None.

Parameters

- **dists** (*Sequence[SequenceEncodableProbabilityDistribution]*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

density(x)

Evaluate density of CompositeDistribution for a single observation tuple x.

Parameters

x (*Tuple[Any, ...]*) – Tuple of length = len(dists), the k-th data type must be consistent with dists[k].

Return type

float

Returns

float – Density value.

Parameters

x (*Tuple[Any, ...]*)

dist_to_encoder()

Return a CompositeDataEncoder for this distribution.

Return type

`dmx.stats.composite.CompositeDataEncoder`

Returns

CompositeDataEncoder – Encoder object.

estimator(pseudo_count=None)

Create CompositeEstimator for estimating CompositeDistribution.

Parameters

pseudo_count (*Optional[float], optional*) – Used to inflate sufficient statistics in estimation.

Return type

`dmx.stats.composite.CompositeEstimator`

Returns

CompositeEstimator – Estimator object.

Parameters

pseudo_count (*float | None*)

log_density(x)

Evaluate the sum of child log densities for one observation tuple.

Parameters

x (*Tuple[Any, ...]*) – Tuple of length = len(dists), the k-th data type must be consistent with dists[k].

Return type

float

Returns*float* – Log-density value.**Parameters****x** (*Tuple*[*Any*, ...])**sampler**(*seed=None*)

Create CompositeSampler for sampling from CompositeDistribution instance.

Parameters**seed** (*Optional*[*int*], *optional*) – Seed to set for sampling with RandomState. Defaults to None.**Return type***dmx.stats.composite.CompositeSampler***Returns***CompositeSampler* – Sampler object.**Parameters****seed** (*int* | *None*)**seq_log_density**(*x*)

Vectorized evaluation of log density for CompositeEncodedDataSequence.

Parameters**x** (*CompositeEncodedDataSequence*) – EncodedDataSequence for Composite Distribution.**Return type***numpy.ndarray***Returns***np.ndarray* – Log-density evaluated at all encoded data points.**Raises****Exception** – If input is not a CompositeEncodedDataSequence.**Parameters****x** (*CompositeEncodedDataSequence*)

CompositeEstimator

class *dmx.stats.composite.CompositeEstimator*(*estimators*, *keys=None*, *name=None*)

Estimator for CompositeDistribution.

Notes

A Composite estimator-level **keys** value shares the whole tuple-shaped sufficient statistic with other Composite estimators using the same key. The merge is positional: field **i** in one composite is pooled with field **i** in another.

Composite accumulators also recurse into their child accumulators. That means child estimator keys can still share individual fields even when the Composite estimator itself is unkeyed. Use a Composite-level key only when every field position has the same meaning across the estimators being fit; otherwise prefer keys on selected child estimators.

Parameters

- **estimators** (*Sequence*[*ParameterEstimator*])

- **keys** (*str* / *None*)

- **name** (*str* / *None*)

estimators

Estimators for each component.

Type

Sequence[*ParameterEstimator*]

keys

Key used for sharing the complete positional tuple of component sufficient statistics.

Type

Optional[str]

count

Number of components.

Type

int

name

Name of the object.

Type

Optional[str]

__init__ (*estimators*, *keys=None*, *name=None*)

Initialize CompositeEstimator.

Parameters

- **estimators** (*Sequence[ParameterEstimator]*) – Estimators for each component.
- **keys** (*Optional[str]*, *optional*) – Key used to share the complete composite sufficient statistic with matching Composite estimators. Sharing is positional, so component *i* is pooled with component *i*. Use child estimator keys instead when only selected fields should be tied. Defaults to None.
- **name** (*Optional[str]*, *optional*) – Name of the object. Defaults to None.

Raises

TypeError – If keys is not a string or None.

Parameters

- **estimators** (*Sequence[ParameterEstimator]*)
- **keys** (*str* / *None*)
- **name** (*str* / *None*)

Return type

None

accumulator_factory()

Return a CompositeAccumulatorFactory for this estimator.

Return type

dmx.stats.composite.CompositeAccumulatorFactory

Returns

CompositeAccumulatorFactory – Factory object.

estimate(*nobs*, *suff_stat*)

Estimate a CompositeDistribution from aggregated sufficient statistics.

Parameters

- **nobs** (*Optional[float]*) – Weighted number of observations used to form *suff_stat*.
- **suff_stat** (*Tuple[Any, ...]*) – Tuple of sufficient statistics for each estimator.

Return type

dmx.stats.composite.CompositeDistribution

Returns

CompositeDistribution – Estimated distribution.

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[Any, ...]*)

CompositeSampler

class *dmx.stats.composite.CompositeSampler*(*dist*, *seed=None*)

CompositeSampler used to generate samples from CompositeDistribution.

Parameters

- **dist** (*CompositeDistribution*)
- **seed** (*int | None*)

dist

CompositeDistribution to draw samples from.

Type

CompositeDistribution

rng

RandomState with seed set if provided.

Type

RandomState

dist_samplers

List of DistributionSamplers for each component.

Type

List[*DistributionSampler*]

sample(*size=None*)

Generate independent samples from a CompositeDistribution.

If size is None, draw one sample and return as Tuple of length = len(dists). If size > 0, draw size samples and return a list of length size containing tuples of len(dists).

Parameters

size (*Optional[int], optional*) – If None, draw 1 sample. Else, draw size number of iid samples.

Return type

typing.Union[typing.List[typing.Tuple[typing.Any, ...]], typing.Tuple[typing.Any, ...]]

Returns

Union[List[Tuple[Any, ...]], Tuple[Any, ...]] – A tuple of length = len(dists) or a list of length size containing tuples of length = len(dists).

Parameters

size (*int* | *None*)

Sequence Distribution

The sequence distribution is used to model independent and identitically distributed (*iid*) sequences of observations or varying lengths. We can also model the distribution for the lengths of the sequences.

Assume $x_i = (x_{i1}, \dots, x_{in_i})$ is a sequence of length n_i having data type T . The sequence distribution models each $x_{i,j}$ with a distribution compatible with type T data, $g(x_i|\theta)$. The lengths of the sequences n_i are modeled with a distribution on the integers $h(n_i|\phi)$. The likelihood for a set of observed sequences $X = ([x_{1,1}, \dots, x_{1,n_1}], \dots, [x_{N,1}, \dots, x_{N,n_N}])$ is

$$f(X) = \prod_{i=1}^N g(x_{i,1}, \dots, x_{i,n_i}|\theta) h(n_i|\phi).$$

SequenceDistribution

```
class dmx.stats.sequence.SequenceDistribution(dist, len_dist=NullDistribution(),
                                             len_normalized=False, name=None, keys=None)
```

Model an ordered sequence using independent item and length factors.

Parameters

- **dist** (*SequenceEncodableProbabilityDistribution*)
- **len_dist** (*SequenceEncodableProbabilityDistribution* | *None*)
- **len_normalized** (*bool* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

dist

Base distribution of sequence (compatible with T).

Type

SequenceEncodableProbabilityDistribution

len_dist

Length distribution for modeling lengths of sequences of observations (compatible with type int). Set to NullDistribution if None is passed.

Type

Optional[*SequenceEncodableProbabilityDistribution*]

len_normalized

If True, take geometric mean density for any density evaluation.

Type

Optional[bool]

name

Name to instance of SequenceDistribution.

Type

Optional[str]

null_len_dist

True if 'len_dist' is set to instance of NullDistribution.

Type

bool

keys

Key for parameters of sequence distribution.

Type

Optional[str]

__init__(*dist*, *len_dist*=NullDistribution(), *len_normalized*=False, *name*=None, *keys*=None)

Initialize a sequence distribution.

Parameters

- **dist** (*SequenceEncodableProbabilityDistribution*) – Set base distribution of sequence (compatible with T).
- **len_dist** (*Optional[SequenceEncodableProbabilityDistribution]*) – Length distribution for modeling lengths of sequences of observations (compatible with type int).
- **len_normalized** (*Optional[bool]*) – If True, take geometric mean density for any density evaluation.
- **name** (*Optional[str]*) – Set name to instance of SequenceDistribution.
- **keys** (*Optional[str]*) – Key for parameters of sequence distribution.

Parameters

- **dist** (*SequenceEncodableProbabilityDistribution*)
- **len_dist** (*SequenceEncodableProbabilityDistribution | None*)
- **len_normalized** (*bool | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

density(*x*)

Evaluate the density of SequenceDistribution at observed sequence *x*.

Parameters

x (*Sequence[T]*) – Sequence of iid observations from base distribution of SequenceDistribution.

Return type

float

Returns

float – Density evaluated at observation *x*.

Parameters**x** (*Sequence*[*T*])**dist_to_encoder()**

Create an encoder composed from the item and length encoders.

Return type*dmx.stats.sequence.SequenceDataEncoder***estimator**(*pseudo_count=None*)

Create an estimator by delegating to both child distributions.

Return type*dmx.stats.sequence.SequenceEstimator***Parameters****pseudo_count** (*float* | *None*)**log_density**(*x*)Evaluate the log-density of *SequenceDistribution* at observed sequence *x*.**Parameters****x** (*Sequence*[*T*]) – Sequence of iid observations from base distribution of *SequenceDistribution*.**Return type***float***Returns***float* – Log-density evaluated at observation *x*.**Parameters****x** (*Sequence*[*T*])**sampler**(*seed=None*)

Create a sampler, requiring a non-null length distribution.

Return type*dmx.stats.sequence.SequenceSampler***Parameters****seed** (*int* | *None*)**seq_ld_lambda()**

Return vectorized log-density callables for the active child models.

Return type*typing.List*[*typing.Any*]**seq_log_density**(*x*)

Evaluate log densities for a batch of encoded ordered sequences.

Return type*numpy.ndarray***Parameters****x** (*SequenceEncodedDataSequence*)

SequenceEstimator

```
class dmx.stats.sequence.SequenceEstimator(estimator, len_estimator=NullEstimator(), len_dist=None,  
                                           len_normalized=False, name=None, keys=None)
```

Estimate item and length models from their paired statistics.

Notes

Requires arg ‘estimator’ to be ParameterEstimator of data type T, compatible with the observed entry values of SequenceDistribution.

If arg ‘len_estimator’ is passed, it must be a ParameterEstimator object compatible with non-negative integers.

If len_estimator is NullEstimator() or None, len_dist is used as length distribution in estimation.

A Sequence estimator-level **keys** value shares the wrapper sufficient statistic, which includes both the repeated-item statistic and the sequence length statistic when a length estimator is present. The accumulator also recurses into the item and length accumulators, so their own keys can be used for more targeted sharing.

Use a Sequence-level key only when both the item distribution and length behavior should be tied across the keyed estimators. If only lengths or only item parameters should be shared, put keys on the corresponding child estimator instead.

Sequence initialization delegates to the item accumulator and, when present, the length accumulator. Pseudo-counts and prior targets should be set on those child estimators; the Sequence wrapper itself only controls how sequence weights are distributed to items through **len_normalized**.

Parameters

- **estimator** (ParameterEstimator)
- **len_estimator** (ParameterEstimator / None)
- **len_dist** (SequenceEncodableProbabilityDistribution / None)
- **len_normalized** (bool / None)
- **name** (str / None)
- **keys** (str / None)

estimator

ParameterEstimator for base distribution.

Type

ParameterEstimator

len_estimator

ParameterEstimator for length distribution. If None, set to NullEstimator.

Type

Optional[*ParameterEstimator*]

len_dist

Set a fixed length distribution.

Type

Optional[*SequenceEncodableProbabilityDistribution*]

len_normalized

Take geometric mean of density if True.

Type

Optional[bool]

name

Name of SequenceEstimator instance.

Type

Optional[str]

keys

Key for sharing the Sequence wrapper sufficient statistic.

Type

Optional[str]

__init__ (*estimator, len_estimator=None, len_dist=None, len_normalized=False, name=None, keys=None*)

Initialize a sequence estimator and its optional length model.

Parameters

- **estimator** (*ParameterEstimator*) – Set ParameterEstimator for base distribution.
- **len_estimator** (*Optional[ParameterEstimator]*) – Set ParameterEstimator for length distribution.
- **len_dist** (*Optional[SequenceEncodableProbabilityDistribution]*) – Set a fixed length distribution.
- **len_normalized** (*Optional[bool]*) – Take geometric mean of density if True.
- **name** (*Optional[str]*) – Set name to SequenceEstimator instance.
- **keys** (*Optional[str]*) – Key used to share the Sequence wrapper sufficient statistic with matching Sequence estimators. This shares item and length statistics together, when length statistics are being accumulated. Use child estimator keys for narrower sharing.

Parameters

- **estimator** (*ParameterEstimator*)
- **len_estimator** (*ParameterEstimator / None*)
- **len_dist** (*SequenceEncodableProbabilityDistribution / None*)
- **len_normalized** (*bool / None*)
- **name** (*str / None*)
- **keys** (*str / None*)

Return type

None

accumulator_factory()

Create a factory backed by both child estimator factories.

Return type

dmx.stats.sequence.SequenceAccumulatorFactory

estimate(*nobs, suff_stat*)

Estimate child distributions and assemble a sequence distribution.

A null length estimator leaves **len_dist** fixed (or absent). Otherwise the second sufficient statistic is passed to the length estimator. The first is always passed to the item estimator.

Return type*dmx.stats.sequence.SequenceDistribution***Parameters**

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*Any*, *Any* | *None*])

SequenceSampler**class** `dmx.stats.sequence.SequenceSampler`(*dist*, *len_dist*, *seed=None*)

SequenceSampler object for sampling from an SequenceDistribution instance.

Parameters

- **dist** (*SequenceEncodableProbabilityDistribution*)
- **len_dist** (*SequenceEncodableProbabilityDistribution*)
- **seed** (*int* | *None*)

dist

The Base distribution for the sequences (data type T).

Type*SequenceEncodableProbabilityDistribution***len_dist**

Length distribution for the length of the sequences (support on positive integers).

Type*SequenceEncodableProbabilityDistribution***rng**

RandomState object for random sampling.

Type

RandomState

dist_sampler

DistributionSampler instance from base distribution.

Type*DistributionSampler***len_sampler**

DistributionSampler instance from length distribution.

Type*DistributionSampler***sample**(*size=None*)

Generate iid samples from SequenceSampler object.

If size is None, the length ‘n’ of the iid sequence is sampled from len_sampler. Then ‘n’ iid samples are drawn from the base dist sampled ‘dist_sampler’.

If size > 0, above is repeated size times and a List of size List[T] is returned.

Parameters**size** (*Optional*[*int*])

Return type`typing.List[typing.Any]`**Returns**`List[T]` or `List[List[T]]` with `length(size)`.**Parameters****size** (`int` | `None`)**Conditional Distribution**

The Conditional distribution is used to model conditional dependencies between two random variables. This can be used for separate data types.

Assume we have observed (x_i, y_i) where x_i has data type T_1 and y_i has data type T_2 . The Conditional distribution is used to model conditional dependencies between two random variables. This can be used for separate data types.

Assume we have observed (x_i, y_i) where x_i has data type T_1 and y_i has data type T_2 . Choosing a compatible *given* distribution $f(x_i|\theta_1)$ for x_i and a distribution $g(y_i|\theta_2)$, the conditional density is given by

$$f((x_i, y_i)) = g(y_i|x_i, \theta_2)h(x_i|\theta_1).$$

Note that each value of x_i emits a distribution over the support of y values.

ConditionalDistribution

```
class dmx.stats.conditional.ConditionalDistribution(dmap, default_dist=NoneDistribution(),
                                                    given_dist=NoneDistribution(), name=None,
                                                    keys=None)
```

Model pairs with a keyed conditional law and an optional given law.

For a pair (`x0`, `x1`), the wrapper multiplies the density of `given_dist` at `x0` by the child density selected from `dmap` at `x1`. Unknown keys use `default_dist` when supplied and otherwise have scalar log density negative infinity. The result is normalized when the given law is normalized and every conditional law reachable from its support is normalized. A null given law contributes a constant factor and is only a placeholder, so it does not by itself define a normalized joint law over arbitrary conditioning values.

Encoding groups responses by conditioning value and delegates each group to its selected child encoder. Sampling first draws the conditioning value and then uses its keyed or default child. Estimation maintains independent child, default, and given sufficient statistics.

Parameters

- **dmap** (`Dict[Any, SequenceEncodableProbabilityDistribution]` | `List[SequenceEncodableProbabilityDistribution]`)
- **default_dist** (`SequenceEncodableProbabilityDistribution` | `None`)
- **given_dist** (`SequenceEncodableProbabilityDistribution` | `None`)
- **name** (`str` | `None`)
- **keys** (`str` | `None`)

dmap

Mapping from `T0` to distributions.

Type`Dict[T0, SequenceEncodableProbabilityDistribution]`

default_dist

Default distribution if key not in dmap.

Type

SequenceEncodableProbabilityDistribution

given_dist

Distribution for given variable.

Type

SequenceEncodableProbabilityDistribution

has_default

True if default_dist is not NullDistribution.

Type

bool

has_given

True if given_dist is not NullDistribution.

Type

bool

name

Name assigned to object.

Type

Optional[str]

keys

All ConditionalDistribution objects with same keys value are the same distribution.

Type

Optional[str]

__init__(dmap, default_dist=NullDistribution(), given_dist=NullDistribution(), name=None, keys=None)

Initialize ConditionalDistribution.

Parameters

- **dmap** – Mapping from conditioning values to distributions, or a list of distributions keyed by their index.
- **default_dist** – Distribution for cases where x[0] is not a key in dmap.
- **given_dist** – Distribution for the given variable.
- **name** – Name assigned to object.
- **keys** – All ConditionalDistribution objects with the same keys value are the same distribution.

Parameters

- **dmap** (*Dict[Any, SequenceEncodableProbabilityDistribution] / List[SequenceEncodableProbabilityDistribution]*)
- **default_dist** (*SequenceEncodableProbabilityDistribution / None*)
- **given_dist** (*SequenceEncodableProbabilityDistribution / None*)
- **name** (*str / None*)

- **keys** (*str* | *None*)

Return type

None

density(*x*)Evaluate density of ConditionalDistribution at Tuple *x*.**Parameters***x* (*Tuple*[*T0*, *T1*]) – *T0* data type must match keys of dmap, *T1* must match value of dmap distribution for key value.**Return type**

float

Returns*float* – Density of ConditionalDistribution at Tuple *x*.**Parameters***x* (*Tuple*[*T0*, *T1*])**dist_to_encoder**()

Return a ConditionalDistributionDataEncoder for this distribution.

Return type`dmx.stats.conditional.ConditionalDistributionDataEncoder`**Returns***ConditionalDistributionDataEncoder* – Encoder object.**estimator**(*pseudo_count=None*)

Create ConditionalDistributionEstimator from sufficient statistics.

Parameters**pseudo_count** (*Optional*[float], *optional*) – Used to inflate the sufficient statistics.**Return type**`dmx.stats.conditional.ConditionalDistributionEstimator`**Returns***ConditionalDistributionEstimator* – Estimator object.**Parameters****pseudo_count** (*float* | *None*)**log_density**(*x*)Evaluate log-density of ConditionalDistribution at Tuple *x*.**Parameters***x* (*Tuple*[*T0*, *T1*]) – *T0* data type must match keys of dmap, *T1* must match value of dmap distribution for key value.**Return type**

float

Returns*float* – Log-density of ConditionalDistribution at Tuple *x*.**Parameters***x* (*Tuple*[*T0*, *T1*])

sampler(*seed=None*)

Create ConditionalDistributionSampler for sampling from this distribution.

Parameters

seed (*Optional[int], optional*) – Seed for random number generator.

Return type

dmx.stats.conditional.ConditionalDistributionSampler

Returns

ConditionalDistributionSampler – Sampler object.

Parameters

seed (*int | None*)

seq_log_density(*x*)

Vectorized log-density for encoded data.

Parameters

x (*ConditionalEncodedDataSequence*) – Encoded data sequence.

Return type

numpy.ndarray

Returns

np.ndarray – Log-density values.

Raises

Exception – If input is not a ConditionalEncodedDataSequence.

Parameters

x (*ConditionalEncodedDataSequence*)

ConditionalDistributionEstimator

```
class dmx.stats.conditional.ConditionalDistributionEstimator(estimator_map,  
                                                            default_estimator=NullEstimator(),  
                                                            given_estimator=NullEstimator(),  
                                                            name=None, keys=None)
```

Estimator for ConditionalDistribution.

Notes

The top-level `keys` value is retained on the estimator and resulting distribution, but the current Conditional accumulator does not use it to merge the whole conditional sufficient statistic. During keyed merging it delegates to the accumulators in `estimator_map`, `default_estimator`, and `given_estimator`.

To share conditional submodels, put keys on the child estimators. For example, use matching keys on selected values in `estimator_map` to share those conditional distributions, or on `given_estimator` to share the model for the conditioning variable. Do not rely on the top-level `keys` argument to pool the full mapping.

Parameters

- **estimator_map** (*Dict[Any, ParameterEstimator]*)
- **default_estimator** (*ParameterEstimator | None*)
- **given_estimator** (*ParameterEstimator | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

estimator_map

Estimators for each conditional distribution.

Type

Dict[T0, *ParameterEstimator*]

default_estimator

Estimator for default_distribution.

Type

ParameterEstimator

given_estimator

Estimator for given_distribution.

Type

ParameterEstimator

name

Name for object.

Type

Optional[str]

keys

Metadata key propagated to the estimator and fitted distribution; it is not used by the current accumulator to share the full conditional mapping.

Type

Optional[str]

__init__(*estimator_map*, *default_estimator*=NullEstimator(), *given_estimator*=NullEstimator(),
name=None, *keys*=None)

Initialize ConditionalDistributionEstimator.

Parameters

- **estimator_map** (*Dict*[T0, *ParameterEstimator*]) – Estimators for each conditional distribution.
- **default_estimator** (*Optional*[*ParameterEstimator*]) – Estimator for default_distribution.
- **given_estimator** (*Optional*[*ParameterEstimator*]) – Estimator for given_distribution.
- **name** (*Optional*[str], *optional*) – Name for object.
- **keys** (*Optional*[str], *optional*) – Metadata key propagated to the estimator and fitted distribution. Current keyed aggregation is controlled by keys on the child estimators, not by this top-level value.

Raises

TypeError – If keys is not a string or None.

Parameters

- **estimator_map** (*Dict*[Any, *ParameterEstimator*])
- **default_estimator** (*ParameterEstimator* | None)
- **given_estimator** (*ParameterEstimator* | None)
- **name** (str | None)
- **keys** (str | None)

Return type

None

accumulator_factory()

Return a ConditionalDistributionAccumulatorFactory for this estimator.

Return type`dmx.stats.conditional.ConditionalDistributionAccumulatorFactory`**Returns***ConditionalDistributionAccumulatorFactory* – Factory object.**estimate(*nobs*, *suff_stat*)**

Estimate a ConditionalDistribution from aggregated data.

Parameters

- **nobs** (*Optional[float]*) – Not used. Kept for consistency.
- **suff_stat** (*Tuple[Dict[T0, SS0], Optional[SS1], Optional[SS2]]*) – Sufficient statistics.

Return type`dmx.stats.conditional.ConditionalDistribution`**Returns***ConditionalDistribution* – Estimated distribution.**Parameters**

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[Dict[Any, SS0], SS1 | None, SS2 | None]*)

ConditionalDistributionSampler**class** `dmx.stats.conditional.ConditionalDistributionSampler(dist, seed=None)`

Sampler for ConditionalDistribution.

Parameters

- **dist** (*ConditionalDistribution*)
- **seed** (*int | None*)

dist

ConditionalDistribution object to draw samples from.

Type*ConditionalDistribution***default_sampler**

Sampler for default_dist.

Type*DistributionSampler***has_default_sampler**

True if default_sampler is not NullDistribution.

Type

bool

given_sampler

Sampler for given_dist.

Type

DistributionSampler

has_given_sampler

True if given sampler is not NullDistribution.

Type

bool

samplers

Dictionary of samplers for each key.

Type

Dict[T0, *DistributionSampler*]

sample(size=None)

Sample independent samples from ConditionalDistribution.

Parameters

size (*Optional[int], optional*) – Number of samples to draw. If None, returns a single sample.

Return type

typing.Union[typing.Tuple[typing.Any, typing.Any], typing.List[typing.Tuple[typing.Any, typing.Any]]]

Returns

Union[Tuple[Any, Any], List[Tuple[Any, Any]] – A tuple or list of tuples of (T0, T1).

Parameters

size (*int | None*)

sample_given(x)

Sample from conditional distribution given value x.

Parameters

x (*T0*) – Value of given/conditional variable.

Return type

typing.Any

Returns

Any – Single sample from the conditional distribution for x.

Parameters

x (*T0*)

single_sample()

Generate a single sample from the ConditionalDistribution.

Return type

typing.Tuple[typing.Any, typing.Any]

Returns

Tuple[Any, Any] – (T0, T1) as defined from dmap and given_distribution types in dist.

Optional Distribution

The Optional distribution assigns a probability (p) to data being missing. With probability $(1-p)$ the data is assumed to come from a base distribution set by the user.

Assuming the data follows a distribution $g(x_i|\theta)$, the likelihood for the Optional distribution is given by

$$f(x|\theta, p) = \begin{cases} p, & x \text{ is missing} \\ (1-p)g(x|\theta), & \text{else} \end{cases}$$

We allow for the user to define the *missing value*.

OptionalDistribution

```
class dmx.stats.optional.OptionalDistribution(dist, p=None, missing_value=None, name=None,
                                             keys=None)
```

Add a distinguished missing value to a child distribution.

When p is supplied, the missing marker has mass p and non-missing values have child density scaled by $1 - p$. The wrapper is normalized when the child is normalized on the non-missing support. When p is `None`, missing values instead contribute a neutral density of one, non-missing values retain the child density, and the result is generally not normalized.

Encoding separates missing positions from values sent to the child encoder. Sampling delegates entirely to the child when p is absent; otherwise it independently chooses missing markers or child draws. Estimation always fits the child from non-missing values and only estimates missingness when `est_prob` is enabled.

Parameters

- **dist** ([SequenceEncodableProbabilityDistribution](#))
- **p** (*float* | *None*)
- **missing_value** (*Any*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

dist

Base distribution.

Type

[SequenceEncodableProbabilityDistribution](#)

p

Probability that dist has missing_value.

Type

float

has_p

True if distribution has arg p passed.

Type

bool

log_p

log of p.

Type

float

log_pn

$\log(1-p)$.

Type

float

missing_value_is_nan

True if the missing value is nan.

Type

bool

missing_value

Missing value from dist.

Type

Any

name

Set a name for the object instance.

Type

Optional[str]

keys

Keys for parameters.

Type

Optional[str]

__init__(*dist*, *p=None*, *missing_value=None*, *name=None*, *keys=None*)

Initialize a missing-value wrapper around *dist*.

Parameters

- **dist** (*SequenceEncodableProbabilityDistribution*) – Base distribution.
- **p** (*Optional[float]*) – Probability that *dist* has *missing_value*.
- **missing_value** (*Any*) – Missing value from *dist*.
- **name** (*Optional[str]*) – Set a name for the object instance.
- **keys** (*Optional[str]*) – Keys for parameters.

Parameters

- **dist** (*SequenceEncodableProbabilityDistribution*)
- **p** (*float | None*)
- **missing_value** (*Any*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

density(*x*)

Evaluate the density of the Optional distribution at *x*.

Notes

See `log_density()`.

Parameters

$\mathbf{x} (T)$ – Observation from base dist or missing value.

Return type

float

Returns

float – Log-density at $\mathbf{x}$.

Parameters

$\mathbf{x} (T)$

dist_to_encoder()

Wrap the child encoder with missing-value partitioning.

Return type

`dmx.stats.optional.OptionalDataEncoder`

estimator(*pseudo_count=None*)

Create an estimator for the child and, when configured, missingness.

Return type

`dmx.stats.optional.OptionalEstimator`

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Evaluate the log density of the Optional distribution at $\mathbf{x}$.

Notes

If $\mathbf{x}$ is a missing value: return $\log(p)$ if p is not *None*, else return 0.0 If $\mathbf{x}$ is not the missing_value: if p is not *None*, return the `log_density(x)` at base dist + $\log(1-p)$ else: return `log_density(x)`.

Parameters

$\mathbf{x} (T)$ – Observation from base dist or missing value.

Return type

float

Returns

float – Log-density at $\mathbf{x}$.

Parameters

$\mathbf{x} (T)$

sampler(*seed=None*)

Create a missingness-aware child sampler.

Return type

`dmx.stats.optional.OptionalSampler`

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Score missing positions and child-encoded observed positions.

Parameters

x – Encoding produced by `OptionalDataEncoder`.

Return type

`numpy.ndarray`

Returns

One log density per original observation.

Raises

TypeError – If **x** is not an optional encoded sequence.

Parameters

x (*OptionalEncodedDataSequence*)

OptionalEstimator

```
class dmx.stats.optional.OptionalEstimator(estimator, missing_value=None, est_prob=False,
                                           pseudo_count=None, name=None, keys=None)
```

OptionalEstimator for estimating OptionalDistribution from sufficient statistics.

Notes

An Optional estimator-level **keys** value shares the Optional wrapper sufficient statistic with other Optional estimators using the same key. That statistic contains both missing/non-missing counts and the wrapped estimator's sufficient statistic for non-missing observations.

The Optional accumulator does not separately recurse into the wrapped accumulator for keyed sharing. If **keys** is `None`, keys on the wrapped estimator are not applied through this wrapper. Use the Optional-level key when missingness and non-missing parameters should be tied together.

Parameters

- **estimator** (*ParameterEstimator*)
- **missing_value** (*Any*)
- **est_prob** (*bool*)
- **pseudo_count** (*float | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

estimator

Estimator for base distribution.

Type

ParameterEstimator

missing_value

Missing_value specification.

Type

Any

est_prob

If true estimate the probability of a missing value.

Type

bool

pseudo_count

Regularize estimate of missing data.

Type

Optional[float]

name

Set name to object.

Type

Optional[str]

keys

Key for sharing missingness counts and wrapped sufficient statistics together.

Type

Optional[str]

__init__ (*estimator*, *missing_value*=None, *est_prob*=False, *pseudo_count*=None, *name*=None, *keys*=None)

Initialize missingness and child estimation settings.

Parameters

- **estimator** (*ParameterEstimator*) – Estimator for base distribution.
- **missing_value** (*Any*) – Missing_value specification.
- **est_prob** (*bool*) – If true estimate the probability of a missing value.
- **pseudo_count** (*Optional[float]*) – Regularize estimate of missing data.
- **name** (*Optional[str]*) – Set name to object.
- **keys** (*Optional[str]*) – Key used to share Optional wrapper sufficient statistics with matching Optional estimators. The shared statistic includes missing/non-missing counts and the wrapped estimator's statistic for observed values.

Parameters

- **estimator** ([ParameterEstimator](#))
- **missing_value** (*Any*)
- **est_prob** (*bool*)
- **pseudo_count** (*float | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

Return type

None

accumulator_factory()

Create a factory for the paired wrapper and child statistics.

Return type

`dmx.stats.optional.OptionalEstimatorAccumulatorFactory`

estimate(*nobs*, *suff_stat*)

Fit the child from observed data and optionally fit missingness.

The child estimator receives the observed weight as *nobs*. If missingness is estimated, its probability is the weighted missing fraction, with a symmetric pseudo-count added to both outcomes when configured.

Return type

dmx.stats.optional.OptionalDistribution

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple[List[float], SS]* | *None*)

OptionalSampler

class *dmx.stats.optional.OptionalSampler*(*dist*, *seed=None*)

OptionalSampler object for generating samples from OptionalDistribution.

Parameters

- **dist** (*OptionalDistribution*)
- **seed** (*int* | *None*)

dist

OptionalDistribution to sample from.

Type

OptionalDistribution

rng

Seeded RandomState object.

Type

RandomState

sampler

DistributionSampler for base distribution.

Type

DistributionSampler

sample(*size=None*)

Generate samples from OptionalDistribution.

Notes

Returns a missing_value or a sample from the base distribution (type T).

Parameters

size (*Optional[int]*) – Number of samples to generate.

Return type

typing.Any

Returns

Union[Union[Any, T], Sequence[Union[Any, T]]

Parameters

size (*int* | *None*)

Ignored Distribution

The IgnoredDistribution and IgnoredEstimator classes provide a way of either ignoring particular features or specifying distributions that are known in advance. This allows users to fix a certain component of the model (i.e. fix a component of a composite distribution).

IgnoredDistribution

class `dmx.stats.ignored.IgnoredDistribution(dist, name=None, keys=None)`

Preserve a child distribution while excluding it from estimation.

The wrapper has exactly the child's support, density, normalization, encoding, and sampling behavior. Its accumulator deliberately records no sufficient statistics, and its estimator always returns a new wrapper around the original child without fitting that child. Thus "ignored" applies only to estimation.

Parameters

- **dist** (`SequenceEncodableProbabilityDistribution` / `None`)
- **name** (`str` / `None`)
- **keys** (`str` / `None`)

dist

Distribution to be ignored.

Type

`SequenceEncodableProbabilityDistribution`

name

Set name for object instance.

Type

`Optional[str]`

keys

Keys for distribution (just a place holder).

Type

`Optional[str]`

__init__ (`dist`, `name=None`, `keys=None`)

Initialize an estimation-ignored child distribution.

Parameters

- **dist** (`Optional[SequenceEncodableProbabilityDistribution]`) – Distribution to be ignored.
- **name** (`Optional[str]`) – Set name for object instance.
- **keys** (`Optional[str]`) – Keys for distribution (just a place holder).

Parameters

- **dist** (`SequenceEncodableProbabilityDistribution` / `None`)
- **name** (`str` / `None`)
- **keys** (`str` / `None`)

Return type

`None`

density(*x*)

Evaluate the density of the IgnoredDistribution at *x*.

Parameters

x (*T*) – Type corresponding to attribute ‘dist’.

Return type

float

Returns

float – Density of attribute ‘dist’ at *x*

Parameters

x (*T*)

dist_to_encoder()

Wrap the child’s sequence encoder.

Return type

`dmx.stats.ignored.IgnoredDataEncoder`

estimator(*pseudo_count=None*)

Create an estimator that preserves, rather than fits, the child.

Return type

`dmx.stats.ignored.IgnoredEstimator`

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Evaluate the log-density of the IgnoredDistribution at *x*.

Parameters

x (*T*) – Type corresponding to attribute ‘dist’.

Return type

float

Returns

float – log-density of attribute ‘dist’ at *x*.

Parameters

x (*T*)

sampler(*seed=None*)

Create a sampler that delegates to the child sampler.

Return type

`dmx.stats.ignored.IgnoredSampler`

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Delegate vectorized scoring to the child distribution.

Parameters

x – An ignored-wrapper encoding or the child’s encoding directly.

Return type

`numpy.ndarray`

Returns

The child's vector of log densities.

Raises

TypeError – If `x` is not an encoded data sequence.

Parameters

x ([EncodedDataSequence](#))

IgnoredEstimator

```
class dmx.stats.ignored.IgnoredEstimator(dist=NoneDistribution(), pseudo_count=None, suff_stat=None,
                                         keys=None, name=None)
```

Return the original child regardless of accumulated data.

`pseudo_count` and `suff_stat` are placeholders. The produced accumulator is a no-op and `estimate` never invokes an estimator on `dist`.

Parameters

- **dist** ([SequenceEncodableProbabilityDistribution](#) / `None`)
- **pseudo_count** (`float` / `None`)
- **suff_stat** (`Any` / `None`)
- **keys** (`str` / `None`)
- **name** (`str` / `None`)

dist

Distribution to be ignored.

Type

[SequenceEncodableProbabilityDistribution](#)

pseudo_count

Place holder for consistency.

Type

Optional[`float`]

suff_stat

Place holder for consistency.

Type

Optional[`Any`]

keys

Place holder for consistency.

Type

Optional[`str`]

name

Set name for object instance.

Type

Optional[`str`]

__init__(*dist=NullDistribution(), pseudo_count=None, suff_stat=None, keys=None, name=None*)

Initialize an estimator that freezes **dist**.

Parameters

- **dist** (*Optional[SequenceEncodableProbabilityDistribution]*) – Distribution to be ignored.
- **pseudo_count** (*Optional[float]*) – Place holder for consistency.
- **suff_stat** (*Optional[Any]*) – Place holder for consistency.
- **keys** (*Optional[str]*) – Place holder for consistency.
- **name** (*Optional[str]*) – Set name for object instance.

Parameters

- **dist** (*SequenceEncodableProbabilityDistribution | None*)
- **pseudo_count** (*float | None*)
- **suff_stat** (*Any | None*)
- **keys** (*str | None*)
- **name** (*str | None*)

Return type

None

accumulator_factory()

Create a no-op accumulator factory using the child's encoder.

Return type

`dmx.stats.ignored.IgnoredAccumulatorFactory`

estimate(*nobs, suff_stat*)

Wrap the unchanged child, ignoring counts and sufficient statistics.

Return type

`dmx.stats.ignored.IgnoredDistribution`

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*Any*)

IgnoredSampler

class `dmx.stats.ignored.IgnoredSampler`(*dist, seed=None*)

IgnoredSampler object for generating samples from Ignored distribution.

Parameters

- **dist** (*IgnoredDistribution*)
- **seed** (*int | None*)

dist_sampler

DistributionSampler for ignored distribution.

Type

`DistributionSampler`

null_sampler

True if IgnoredDistribution is the NullDistribution.

Type

bool

sample(size=None)

Draw from the child, preserving its scalar or sized return contract.

Return type

typing.Any

Parameters

size (int | None)

Mixture Models

Mixture models are used to infer properties about sub-populations when presented with pooled population data. Several applications such as topic modeling, clustering, and rich-prior distribution selection can be handled with mixture models. *dmx-learn* offers a flexible implementation of the mixture model. *dmx-learn*'s mixture model API combined with *base distributions* and *combinators*, allows for the specification of deep nested graphical models on heterogenous data types.

Mixture Models

Mixture Distribution

Mixture distributions are useful when a statistical population contains two or more subpopulations that are unobserved. *dmx-learn* allows for the specification of any model for the *components* of the mixture distribution. Assuming we have an observation x of data type T (any heterogenous form), the data generating process for a K-component mixture model is given by

$$\begin{aligned} z &\sim \pi \\ x|z &\sim f_k(x|\theta_k) \end{aligned}$$

where π_k representing the probability of x being drawn from component distribution $f_k(x|\theta_k)$. For more details see [Mixture Distribution](#).

MixtureDistribution

class dmx.stats.mixture.**MixtureDistribution**(components, w, name=None, keys=(None, None))

Represent a finite mixture of homogeneous component distributions.

components[k] defines f_k and w[k] is its weight π_k . Component log densities exclude mixture weights; posterior methods return normalized responsibilities

$$r_k(x) = P(Z = k | X = x) = \frac{\pi_k f_k(x)}{\sum_j \pi_j f_j(x)}.$$

Parameters

- **components** (Sequence[SequenceEncodableProbabilityDistribution])
- **w** (ndarray | Sequence[float])
- **name** (str | None)
- **keys** (Tuple[str | None, str | None])

components

List of component distributions (data type T).

Type

Sequence[*SequenceEncodableProbabilityDistribution*]

w

Mixture weights assigned from args (w).

Type

ndarray[float]

zw

True if a weight is 0.0, else False.

Type

ndarray[bool]

log_w

Log of weights (w). set to -np.inf, where zw is True.

Type

ndarray[float]

num_components

Number of components in MixtureDistribution instance.

Type

int

name

String name to MixtureDistribution object.

Type

Optional[str]

keys

Set keys for the weights and component distributions.

Type

Tuple[Optional[str], Optional[str]]

__init__(*components*, *w*, *name=None*, *keys=(None, None)*)

Initialize a mixture distribution.

Parameters

- **components** (*Sequence[SequenceEncodableProbabilityDistribution]*) – Component distributions.
- **w** (*ndarray[float]*) – Length-K mixture weights, ordered like **components** and expected to sum to one.
- **name** (*Optional[str]*) – Assign string name to MixtureDistribution object.
- **keys** (*Tuple[Optional[str], Optional[str]]*) – Set keys for the weights and component distributions.

Parameters

- **components** (*Sequence[SequenceEncodableProbabilityDistribution]*)
- **w** (*ndarray | Sequence[float]*)

- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*])

Return type*None***component_log_density(*x*)**

Evaluate every component log density at one observation.

Entry *k* is $\ell_k(x) = \log f_k(x)$ and does not include $\log \pi_k$.

Parameters

x (*T*) – Single observation from mixture distribution. *T* is data type of components.

Return type*numpy.ndarray***Returns**

np.ndarray – Component log densities with shape (*K*,).

Parameters*x* (*T*)**density(*x*)**

Evaluate density of Mixture distribution at observation *x*.

See `log_density` for details.

Parameters

x (*T*) – Single observation from mixture distribution. *T* is data type of components.

Return type*float***Returns**

float – Density at *x*.

Parameters*x* (*T*)**dist_to_encoder()**

Create a sequence encoder using the first component's encoder.

Return type*dmx.stats.mixture.MixtureDataEncoder***Returns**

MixtureDataEncoder – Encoder shared by all homogeneous components.

estimator(*pseudo_count=None*)

Create an estimator compatible with this mixture.

When `pseudo_count` is supplied, it smooths the outer mixture weights toward uniform weights. Each component estimator is also constructed with $1 / K$ as its pseudo-count, matching the existing convenience behavior.

Parameters

pseudo_count (*Optional*[*float*]) – Outer weight pseudo-count, or *None* for maximum-likelihood weight estimation.

Return type*dmx.stats.mixture.MixtureEstimator*

Returns*MixtureEstimator* – Estimator with one estimator per component.**Parameters****pseudo_count** (*float* | *None*)**log_density(x)**

Evaluate the mixture log density at one observation.

$$\log f(x) = \log \left(\sum_{k=0}^{K-1} \pi_k f_k(x) \right).$$

Parameters**x** (*T*) – Single observation from mixture distribution. T is data type of components.**Return type***float***Returns***float* – Log-density at x.**Parameters****x** (*T*)**posterior(x)**

Compute posterior responsibilities for one observation.

$$r_k(x) = \frac{\pi_k f_k(x)}{\sum_{j=0}^{K-1} \pi_j f_j(x)}.$$

Parameters**x** (*T*) – Single observation from mixture distribution. T is data type of components.**Return type***numpy.ndarray***Returns***np.ndarray* – Responsibilities in component order with shape (K,).**Parameters****x** (*T*)**sampler(seed=None)**

Create a sampler for this mixture.

Parameters**seed** (*Optional[int]*) – Seed used to initialize component selection and component samplers.**Return type***dmx.stats.mixture.MixtureSampler***Returns***MixtureSampler* – Sampler bound to this distribution.**Parameters****seed** (*int* | *None*)**seq_component_log_density(x)**

Evaluate component log densities for an encoded iid sequence.

Parameters

x (*MixtureEncodedDataSequence*) – EncodedDataSequence for mixture component.

Return type

`numpy.ndarray`

Returns

np.ndarray –

Matrix with shape (N, K). Entry (n, k) is the
unweighted component log density for observation n and component k.

Parameters

x (*MixtureEncodedDataSequence*)

seq_log_density(x)

Evaluate mixture log densities for an encoded iid sequence.

Parameters

x (*MixtureEncodedDataSequence*) – Encoded sequence of N observations.

Return type

`numpy.ndarray`

Returns

np.ndarray – Mixture log densities with shape (N,).

Parameters

x (*MixtureEncodedDataSequence*)

seq_posterior(x)

Compute posterior responsibilities for an encoded iid sequence.

Parameters

x (*MixtureEncodedDataSequence*) – EncodedDataSequence for mixture component.

Return type

`numpy.ndarray`

Returns

np.ndarray – Responsibility matrix with shape (N, K).

Parameters

x (*T1*)

MixtureEstimator

```
class dmx.stats.mixture.MixtureEstimator(estimators, fixed_weights=None, suff_stat=None,  
                                           pseudo_count=None, name=None, keys=(None, None))
```

Estimate a mixture from aggregated sufficient statistics.

Component count `counts[k]` is passed as `nobs` to estimator `k`. Unless weights are fixed, weights are normalized component counts, optionally smoothed by `pseudo_count`.

Notes

`keys` controls which sufficient-statistic blocks are pooled with other Mixture estimators that use the same key values.

- `keys[0]` shares the outer mixture-weight counts.

- `keys[1]` shares component sufficient statistics by component index.

This allows partial sharing. For example, `keys=(None, "comps0")` keeps each outer mixture's weights separate while tying the component estimators positionally. Unkeyed blocks are still estimated independently.

Use keyed sharing only when component positions have the same meaning across the models being fit. If the component ordering or intended semantics differ, the shared key will over-pool statistics and can force the wrong parameters to match.

Parameters

- **estimators** (*Sequence*[*ParameterEstimator*])
- **fixed_weights** (*Sequence*[*float*] | *ndarray* | *None*)
- **suff_stat** (*ndarray* | *None*)
- **pseudo_count** (*float* | *None*)
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*])

estimators

Sequence of *ParameterEstimator* objects for the mixture components.

Type

Sequence[*ParameterEstimator*]

fixed_weights

Treat mixture weights as fixed values. Must sum to 1.0.

Type

Optional[*np.ndarray*]

suff_stat

Weights of the mixture. Must sum to 1.0.

Type

Optional[*np.ndarray*]

pseudo_count

Used to re-weight the member variable sufficient statistics in estimation.

Type

Optional[*float*]

name

Name for *MixtureEstimator* object.

Type

Optional[*str*]

keys

Keys for the outer mixture-weight counts and the component sufficient statistics.

Type

Tuple[*Optional*[*str*], *Optional*[*str*]]

__init__ (*estimators*, *fixed_weights*=*None*, *suff_stat*=*None*, *pseudo_count*=*None*, *name*=*None*, *keys*=(*None*, *None*))

Initialize a mixture estimator.

Parameters

- **estimators** (*Sequence[ParameterEstimator]*) – Sequence of ParameterEstimator objects for the mixture components.
- **fixed_weights** (*Optional[Union[Sequence[float], np.ndarray]]*) – Set fixed values for mixture weights.
- **suff_stat** (*Optional[np.ndarray]*) – Numpy array of floats with length equal to length of estimators.
- **pseudo_count** (*Optional[float]*) – Used to re-weight the member variable sufficient statistics in estimation.
- **name** (*Optional[str]*) – Set a name to the MixtureEstimator object.
- **keys** (*Tuple[Optional[str], Optional[str]]*) – Keys that control sharing of sufficient statistics across Mixture estimators with matching key values. `keys[0]` shares outer mixture-weight counts. `keys[1]` shares component sufficient statistics by component index. Use `keys=(None, "shared_components")` when only the inner components should be tied. Shared keys assume aligned component ordering; if the component positions do not mean the same thing, the fit will over-share statistics.

Parameters

- **estimators** (*Sequence[ParameterEstimator]*)
- **fixed_weights** (*Sequence[float] | ndarray | None*)
- **suff_stat** (*ndarray | None*)
- **pseudo_count** (*float | None*)
- **name** (*str | None*)
- **keys** (*Tuple[str | None, str | None]*)

Return type

None

accumulator_factory()

Create a compatible accumulator factory.

Return type

`dmx.stats.mixture.MixtureAccumulatorFactory`

estimate(nobs, suff_stat)

Estimate components and mixture weights from sufficient statistics.

With no pseudo-count, normalized component counts are used, falling back to uniform weights when their sum is zero. A pseudo-count without `suff_stat` adds `pseudo_count / K` to every component count. When prior `suff_stat` is supplied, `pseudo_count * suff_stat` is added as the weight prior.

Parameters

- **nobs** (*Optional[float]*) – Total observation count; ignored because component counts are contained in `suff_stat`.
- **suff_stat** – Component counts with shape `(K,)` and a length-K tuple of component sufficient statistics.

Return type

`dmx.stats.mixture.MixtureDistribution`

Returns

MixtureDistribution – Distribution estimated in component order.

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *Tuple*[*Any*, ...]])

MixtureSampler

class `dmx.stats.mixture.MixtureSampler`(*dist*, *seed=None*)

Draw observations from a mixture distribution.

Sampling first draws *Z* from the categorical distribution with probabilities `dist.w`, then draws one observation from component *Z*.

Parameters

- **dist** (*MixtureDistribution*)
- **seed** (*int* | *None*)

dist

MixtureDistribution to draw samples from.

Type

MixtureDistribution

rng

Seeded *RandomState* for sampling.

Type

RandomState

comp_samplers

List of *DistributionSampler* objects for each mixture component.

Type

Sequence[*DistributionSamplers*]

sample(*size=None*)

Draw iid samples from a mixture distribution.

The data type drawn from ‘*comp_samplers*’ is type *T*, corresponding to the data type of the mixture components.

If *size* is *None*, a single sample (of data type *T*) is drawn and returned. If *size* is not *None*, ‘*size*’-iid mixture samples are drawn and returned as a *Sequence* with data type *List*[*T*].

Parameters

size (*Optional*[*int*]) – Number of iid samples to draw.

Return type

typing.Union[*typing.Sequence*[*typing.Any*], *typing.Any*]

Returns

Data type *T* or *Sequence*[*T*].

Parameters

size (*int* | *None*)

Heterogeneous Mixture Distribution

The Heterogeneous mixture distribution can be used to assign heterogeneous mixture components to the *Mixture Distribution*. For example, consider observing a positive float x . We can define a two component mixture to be $f_1(x|\lambda) \sim \text{Exp}(\lambda)$ and $f_2(x|\mu, \sigma) \sim \text{LogNormal}(\mu, \sigma)$. The only requirement for the components of the heterogeneous mixture is that the components distributions have the same support as the data type of x .

HeterogeneousMixtureDistribution

```
class dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution(components, w,  
                                                                    name=None,  
                                                                    keys=(None, None))
```

Define a mixture whose components may use different encoders.

Each observation is a single value x accepted by every component, while the components may belong to different distribution families. The latent variable Z is a categorical component index and its posterior probabilities are the usual component responsibilities $p(Z=k \mid x)$.

Parameters

- **components** (*Sequence*[*SequenceEncodableProbabilityDistribution*])
- **w** (*Sequence*[*float*] | *ndarray*)
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*])

components

List of component distributions (data type T).

Type

Sequence[*SequenceEncodableProbabilityDistribution*]

w

Mixture weights assigned from args (w).

Type

np.ndarray

name

String name for the HeterogeneousMixtureDistribution object.

Type

Optional[*str*]

zw

True if a weight is 0.0, else False.

Type

np.ndarray

log_w

Log of weights (w). Set to $-\text{np.inf}$ where zw is True.

Type

np.ndarray

num_components

Number of components in HeterogeneousMixtureDistribution instance.

Type
int

keys

Keys for weights and components.

Type
Tuple[Optional[str], Optional[str]]

__init__(*components*, *w*, *name*=None, *keys*=(None, None))

Initialize HeterogeneousMixtureDistribution.

Parameters

- **components** (*Sequence[SequenceEncodableProbabilityDistribution]*) – Set component distributions. Must all be compatible with type T.
- **w** (*Union[Sequence[float], np.ndarray]*) – Mixture weights, must sum to 1.0.
- **name** (*Optional[str], optional*) – Assign string name to HeterogeneousMixtureDistribution object.
- **keys** (*Tuple[Optional[str], Optional[str]], optional*) – Keys for weights and components.

Parameters

- **components** (*Sequence[SequenceEncodableProbabilityDistribution]*)
- **w** (*Sequence[float] | ndarray*)
- **name** (*str | None*)
- **keys** (*Tuple[str | None, str | None]*)

Return type
None

component_log_density(*x*)

Evaluate component-wise log-density at an observation.

Returns a num_components-dimensional array with $\log(f_k(x))$ in each entry.

Parameters

x (*T*) – Single observation from mixture distribution. T is data type of components.

Return type

numpy.ndarray

Returns

np.ndarray – Component-wise log-density at x.

Parameters

x (*T*)

density(*x*)

Evaluate density of heterogeneous mixture distribution at observation x.

Parameters

x (*T*) – Single observation from heterogeneous mixture distribution. T is data type of components.

Return type

float

Returns

float – Density at x.

Parameters

x (*T*)

dist_to_encoder()

Return a `HeterogeneousMixtureDataEncoder` for this distribution.

Return type

`dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDataEncoder`

Returns

HeterogeneousMixtureDataEncoder – Encoder object.

estimator(pseudo_count=None)

Return a `HeterogeneousMixtureEstimator` for this distribution.

Parameters

pseudo_count (*Optional[float], optional*) – Pseudo-count for regularization.

Return type

`dmx.stats.heterogeneous_mixture.HeterogeneousMixtureEstimator`

Returns

HeterogeneousMixtureEstimator – Estimator object.

Parameters

pseudo_count (*float | None*)

log_density(x)

Evaluate log-density of heterogeneous mixture distribution at observation x.

$$\log f(x) = \log \left(\sum_{k=1}^K f_k(x) p_k \right).$$

Parameters

x (*T*) – Single observation from mixture distribution. T is data type of components.

Return type

float

Returns

float – Log-density at x.

Parameters

x (*T*)

posterior(x)

Obtain component responsibilities at an observation.

$$\frac{f(z = k | \text{vertex})}{\sum_{k=1}^K f_k(x)} p_k$$

Parameters

x (*T*) – Single observation from mixture distribution. T is data type of components.

Return type

`numpy.ndarray`

Returns

`np.ndarray` – Posterior distribution at observation x.

Parameters

x (*T*)

sampler(*seed=None*)

Return a HeterogeneousMixtureSampler for this distribution.

Parameters

seed (*Optional[int], optional*) – Seed for random number generator.

Return type

`dmx.stats.heterogeneous_mixture.HeterogeneousMixtureSampler`

Returns

`HeterogeneousMixtureSampler` – Sampler object.

Parameters

seed (*int | None*)

seq_component_log_density(*x*)

Vectorized evaluation of component-wise log-density for encoded sequence x.

Parameters

x (*HeterogeneousMixtureEncodedDataSequence*) – EncodedDataSequence for Heterogeneous Mixture.

Return type

`numpy.ndarray`

Returns

`np.ndarray` – 2-d array of shape (n_samples, n_components).

Parameters

x (*HeterogeneousMixtureEncodedDataSequence*)

seq_log_density(*x*)

Vectorized evaluation of log-density for encoded sequence x.

Parameters

x (*HeterogeneousMixtureEncodedDataSequence*) – EncodedDataSequence for Heterogeneous Mixture.

Return type

`numpy.ndarray`

Returns

np.ndarray – log_density of each observation in encoded sequence.

Parameters

x (*HeterogeneousMixtureEncodedDataSequence*)

seq_posterior(x)

Evaluate component responsibilities for an encoded sequence.

Parameters

x (*HeterogeneousMixtureEncodedDataSequence*) – EncodedDataSequence for Heterogeneous Mixture.

Return type

numpy.ndarray

Returns

np.ndarray – Posterior probabilities for each observation in encoded sequence.

Parameters

x (*HeterogeneousMixtureEncodedDataSequence*)

HeterogeneousMixtureEstimator

```
class dmx.stats.heterogeneous_mixture.HeterogeneousMixtureEstimator(estimators,  
                                                                    fixed_weights=None,  
                                                                    suff_stat=None,  
                                                                    pseudo_count=None,  
                                                                    name=None, keys=(None,  
                                                                    None))
```

Estimate a heterogeneous mixture from sufficient statistics.

Notes

The outer sufficient statistic is a component-count vector. If *pseudo_count* is provided without *suff_stat*, the estimator smooths mixture weights uniformly across components. If both are provided, *suff_stat* is treated as a prior weight target and mixed with the accumulated component counts. *fixed_weights* bypasses this outer-weight update entirely.

keys[0] shares the outer component-count vector. *keys*[1] shares child component sufficient statistics by component index. Shared component keys assume component position *i* has the same meaning in every estimator with that key.

Parameters

- **estimators** (*Sequence*[*ParameterEstimator*])
- **fixed_weights** (*ndarray* | *None*)
- **suff_stat** (*ndarray* | *None*)
- **pseudo_count** (*float* | *None*)
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*])

estimators

Estimators for the mixture components.

Type

Sequence[*ParameterEstimator*]

fixed_weights

Fixed weights for the mixture (if any).

Type

Optional[np.ndarray]

suff_stat

Prior target for the outer mixture weights.

Type

Optional[np.ndarray]

pseudo_count

Pseudo-count mass for the outer mixture weights.

Type

Optional[float]

name

Name for the estimator.

Type

Optional[str]

keys

Keys for the weights and component distributions.

Type

Tuple[Optional[str], Optional[str]]

__init__ (*estimators*, *fixed_weights=None*, *suff_stat=None*, *pseudo_count=None*, *name=None*, *keys=(None, None)*)

Initialize HeterogeneousMixtureEstimator.

Parameters

- **estimators** (*Sequence[ParameterEstimator]*) – Estimators for the mixture components.
- **fixed_weights** (*Optional[np.ndarray]*, *optional*) – Fixed weights for the mixture.
- **suff_stat** (*Optional[np.ndarray]*, *optional*) – Prior target for the outer mixture weights. Used only when **pseudo_count** is not **None**.
- **pseudo_count** (*Optional[float]*, *optional*) – Pseudo-count mass for the outer mixture weights. With no **suff_stat** it smooths uniformly across components; with **suff_stat** it shrinks toward that target.
- **name** (*Optional[str]*, *optional*) – Name for the estimator.
- **keys** (*Tuple[Optional[str], Optional[str]]*, *optional*) – Keys for the weights and component distributions.

Raises

TypeError – If keys is not a tuple of two strings or **None**.

Parameters

- **estimators** (*Sequence[ParameterEstimator]*)
- **fixed_weights** (*ndarray | None*)
- **suff_stat** (*ndarray | None*)
- **pseudo_count** (*float | None*)

- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*])

Return type*None***accumulator_factory()**

Return a *HeterogeneousMixtureAccumulatorFactory* for this estimator.

Return type*dmx.stats.heterogeneous_mixture.HeterogeneousMixtureAccumulatorFactory***Returns***HeterogeneousMixtureAccumulatorFactory* – Factory object.**estimate(nobs, suff_stat)**

Estimate a *HeterogeneousMixtureDistribution* from sufficient statistics.

Parameters

- **nobs** (*Optional*[*float*]) – Number of observations (not used).
- **suff_stat** (*Tuple*[*np.ndarray*, *Tuple*[*Any*, ...]]) – Component counts and one child sufficient-statistic value per heterogeneous component.

Return type*dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution***Returns***HeterogeneousMixtureDistribution* – Estimated distribution.**Parameters**

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *Tuple*[*Any*, ...]])

HeterogeneousMixtureSampler

```
class dmx.stats.heterogeneous_mixture.HeterogeneousMixtureSampler(dist, seed=None)
```

Sampler for *HeterogeneousMixtureDistribution*.

Parameters

- **dist** (*HeterogeneousMixtureDistribution*)
- **seed** (*int* | *None*)

dist

Distribution to sample from.

Type*HeterogeneousMixtureDistribution***rng**

Seeded *RandomState* for sampling.

Type*RandomState*

comp_samplers

List of DistributionSampler objects for each mixture component.

Type

List[[DistributionSampler](#)]

sample(size=None)

Draw iid samples from a heterogeneous mixture distribution.

Parameters

size (*Optional[int], optional*) – Number of iid samples to draw.

Return type

typing.Union[typing.Any, typing.Sequence[typing.Any]]

Returns

Any or Sequence[Any] – Single sample or list of samples.

Parameters

size (*int | None*)

Joint Mixture Distribution

The Joint Mixture Distribution is a mixture of mixtures (see [Mixture Distribution](#)). This model is particularly useful when observations can belong to multiple latent groups simultaneously. This model can capture mutli-level clustering and dependencies. For $K_1 = K_2$, this model can be viewed as a single-step [Hidden Markov Distribution](#). The generative process for a Joint Mixture Model with K_1 outer-states and K_2 inner-states is described as

$$\begin{aligned} z_1 &\sim \pi \\ z_2|z_1 = k_1 &\sim \tau_{k_1} \\ x|z_2 = k_2 &\sim f_k(x|\theta_{k_2}) \end{aligned}$$

where the initial group membership is drawn $P(Z_1 = k_1) = \pi_{k_1}$ and transition probability is given by $P(Z_2 = k_2|Z_1 = k_1) = \tau_{k_1, k_2}$.

JointMixtureDistribution

class dmx.stats.jmixture.**JointMixtureDistribution**(components1, components2, w1, w2, taus12, taus21, keys=(None, None, None), name=None)

JointMixtureDistribution object for defining a joint mixture distribution.

Notes

Data type is Tuple[T0, T1] where all components1 entries and component2 entries are compatible with T0 and T1 respectively.

Parameters

- **components1** (*Sequence[SequenceEncodableProbabilityDistribution]*)
- **components2** (*Sequence[SequenceEncodableProbabilityDistribution]*)
- **w1** (*Sequence[float] | ndarray*)
- **w2** (*Sequence[float] | ndarray*)
- **taus12** (*List[List[float]] | ndarray*)
- **taus21** (*List[List[float]] | ndarray*)
- **keys** (*Tuple[str | None, str | None, str | None] | None*)

- **name** (*str* / *None*)

components1

Mixture components for mixture of X1.

Type

Sequence[*SequenceEncodableProbabilityDistribution*]

components2

Mixture components for mixture X2.

Type

Sequence[*SequenceEncodableProbabilityDistribution*]

w1

Probability of drawing X1 from component i.

Type

np.ndarray

w2

Probability of drawing X2 from component j.

Type

np.ndarray

num_components1

Number of mixture components for X1.

Type

int

num_components2

Number of mixture components for X2.

Type

int

taus12

2-d Numpy array with probabilities of drawing X2 from comp j given X1 was drawn from comp i. Rows are component X1 state.

Type

np.ndarray

taus21

2-d Numpy array with probabilities of drawing X1 from comp i given X2 was drawn from comp j. Rows are component X1 state.

Type

np.ndarray

log_w1

Log-probability of drawing X1 from component i.

Type

np.ndarray

log_w2

Log-probability of drawing X2 from component j.

Type

np.ndarray

log_taus12

2-d Numpy array with log-probabilities of drawing X2 from comp j given X1 was drawn from comp i. Rows are component X1 state.

Type

np.ndarray

log_taus21

2-d Numpy array with log-probabilities of drawing X1 from comp i given X2 was drawn from comp j. Rows are component X1 state.

Type

np.ndarray

keys

Set keys for weights, mixture components of X1, mixture components of X2.

Type

Optional[Tuple[Optional[str], Optional[str], Optional[str]]]

name

Set name to object.

Type

Optional[str]

__init__(*components1*, *components2*, *w1*, *w2*, *taus12*, *taus21*, *keys*=(None, None, None), *name*=None)

Initialize a joint mixture distribution.

Parameters

- **components1** (*Sequence[SequenceEncodableProbabilityDistribution]*) – Mixture components for mixture of X1.
- **components2** (*Sequence[SequenceEncodableProbabilityDistribution]*) – Mixture components for mixture X2.
- **w1** (*np.ndarray*) – Probability of drawing X1 from component i.
- **w2** (*np.ndarray*) – Probability of drawing X2 from component j.
- **taus12** (*np.ndarray*) – 2-d Numpy array with probabilities of drawing X2 from comp j given X1 was drawn from comp i. Rows are component X1 state.
- **taus21** (*np.ndarray*) – 2-d Numpy array with probabilities of drawing X1 from comp i given X2 was drawn from comp j. Rows are component X1 state.
- **keys** (*Optional[Tuple[Optional[str], Optional[str], Optional[str]]]*) – Set keys for weights, mixture components of X1, mixture components of X2.
- **name** (*Optional[str]*) – Set name to object.

Parameters

- **components1** (*Sequence[SequenceEncodableProbabilityDistribution]*)
- **components2** (*Sequence[SequenceEncodableProbabilityDistribution]*)
- **w1** (*Sequence[float] | ndarray*)
- **w2** (*Sequence[float] | ndarray*)

- **taus12** (*List[List[float]] | ndarray*)
- **taus21** (*List[List[float]] | ndarray*)
- **keys** (*Tuple[str | None, str | None, str | None] | None*)
- **name** (*str | None*)

Return type

None

density(*x*)

Evaluate the density of one paired observation.

Return type

float

Parameters

x (*Tuple[T0, T1]*)

dist_to_encoder()

Return an encoder that separates the two observed views.

Return type

dmx.stats.pdist.DataSequenceEncoder

estimator(*pseudo_count=None*)

Return an estimator for both component banks and their coupling.

Return type

dmx.stats.jmixture.JointMixtureEstimator

Parameters

pseudo_count (*float | None*)

log_density(*x*)

Evaluate the log-density of one paired observation.

Return type

float

Parameters

x (*Tuple[T0, T1]*)

sampler(*seed=None*)

Return a sampler for paired observations.

Return type

dmx.stats.jmixture.JointMixtureSampler

Parameters

seed (*int | None*)

seq_log_density(*x*)

Evaluate log-densities for encoded paired observations.

Return type

numpy.ndarray

Parameters

x (*JointMixtureEncodedDataSequence*)

JointMixtureEstimator

```
class dmx.stats.jmixture.JointMixtureEstimator(estimators1, estimators2, suff_stat=None,  
                                              pseudo_count=None, keys=(None, None, None),  
                                              name=None)
```

Estimate a joint mixture from aggregated sufficient statistics.

Notes

`keys` identifies the statistic blocks that may be shared with other JointMixture estimators using the same key values.

- `keys[0]` corresponds to the joint latent-count block used for the marginal weights and cross-view transition matrices.
- `keys[1]` shares the X1 component sufficient statistics by component index.
- `keys[2]` shares the X2 component sufficient statistics by component index.

Tuple positions allow partial sharing; they do not tie the whole joint mixture unless all relevant positions use matching keys. Component-bank keys assume component index `i` has the same meaning in every estimator that shares the key. Treat keyed sharing of the joint latent-count block carefully and verify it in the fitted model before relying on it.

Parameters

- **`estimators1`** (*Sequence[ParameterEstimator]*)
- **`estimators2`** (*Sequence[ParameterEstimator]*)
- **`suff_stat`** (*Tuple[ndarray, ndarray, ndarray, Tuple[E0, ...], Tuple[E1, ...]] | None*)
- **`pseudo_count`** (*Tuple[float, float, float] | None*)
- **`keys`** (*Tuple[str | None, str | None, str | None] | None*)
- **`name`** (*str | None*)

`estimators1`

Estimators for mixture component of X1.

Type

Sequence[ParameterEstimator]

`estimators2`

Estimators for mixture component of X2.

Type

Sequence[ParameterEstimator]

`suff_stat`

Optional reference statistics.

Type

Optional[Tuple[Any, ...]]

`pseudo_count`

Used to re-weight the state counts in estimation.

Type

Optional[Tuple[float, float, float]]

keys

Keys for joint latent counts, X1 component statistics, and X2 component statistics.

Type

Optional[Tuple[Optional[str], Optional[str], Optional[str]]]

name

Set name to object.

Type

Optional[str]

__init__ (*estimators1, estimators2, suff_stat=None, pseudo_count=None, keys=(None, None, None), name=None*)

Initialize a joint-mixture estimator.

Parameters

- **estimators1** (*Sequence[ParameterEstimator]*) – Estimators for mixture component of X1.
- **estimators2** (*Sequence[ParameterEstimator]*) – Estimators for mixture component of X2.
- **suff_stat** (*Optional[Tuple[Any, ...]]*) – Optional reference statistics.
- **pseudo_count** (*Optional[Tuple[float, float, float]]*) – Used to re-weight the state counts in estimation.
- **keys** (*Optional[Tuple[Optional[str], Optional[str], Optional[str]]]*) – Keys that control which sufficient-statistic blocks are shared. **keys[0]** refers to the joint latent-count block used for weights and cross-view transition matrices. **keys[1]** and **keys[2]** share the X1 and X2 component accumulators positionally. Shared component keys require aligned component ordering across estimators.
- **name** (*Optional[str]*) – Set name to object.

Parameters

- **estimators1** (*Sequence[ParameterEstimator]*)
- **estimators2** (*Sequence[ParameterEstimator]*)
- **suff_stat** (*Tuple[ndarray, ndarray, ndarray, Tuple[E0, ...], Tuple[E1, ...]] | None*)
- **pseudo_count** (*Tuple[float, float, float] | None*)
- **keys** (*Tuple[str | None, str | None, str | None] | None*)
- **name** (*str | None*)

Return type

None

accumulator_factory()

Return a factory for the two component-estimator banks.

Return type

`dmx.stats.jmixture.JointMixtureEstimatorAccumulatorFactory`

estimate (*nobs, suff_stat*)

Estimate component banks, marginals, and conditional matrices.

Return type

`dmx.stats.jmixture.JointMixtureDistribution`

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *ndarray*, *ndarray*, *Tuple*[*E0*, ...], *Tuple*[*E1*, ...]])

JointMixtureSampler

class dmx.stats.jmixture.**JointMixtureSampler**(*dist*, *seed=None*)

JointMixtureSampler object for sampling from a joint mixture distribution.

Parameters

- **dist** (*JointMixtureDistribution*)
- **seed** (*int* | *None*)

rng

RandomState for seeding samples.

Type

RandomState

dist

Distribution to sample from.

Type

JointMixtureDistribution

comp_sampler1

Inner-mixture sampler.

Type

DistributionSampler

comp_sampler2

Outer-mixture sampler.

Type

DistributionSampler

sample(*size=None*)

Draw one paired observation or a collection of pairs.

Return type

typing.Union[*typing.Tuple*[*typing.Any*, *typing.Any*], *typing.Sequence*[*typing.Tuple*[*typing.Any*, *typing.Any*]]]

Parameters

size (*int* | *None*)

Hierarchical Mixture Distribution

A Hierarchical Mixture Model is a statistical model that combines multiple layers of mixture models to capture complex data distributions. It is particularly useful in scenarios where data can be grouped into subpopulations, each of which may follow its own mixture distribution. This model is often referred to as a “mixture of mixtures.” The data format required for using the Hierarchical mixture requires a sequence of observations of the form $\mathbf{X}_i = (X_{i,1}, \dots, X_{i,n_i})$. This can be thought of as independent draws from a topic, where the topic distribution is also a mixture model.

Table 1: Hierarchical Mixture Model Features

Feature	Sym- bol	Description
Outer-State Distribution	π	Represents the distribution over the K_1 outer states.
Inner-State Probabilities	τ_k	Distribution over the K_2 inner-states.
Component Distributions	$f_k(x)$	The probability distribution of the observed data given a specific inner state.
Length Distribution	$g(\cdot)$	Distribution for the lengths of each observation.

The generative process for data $\mathbf{X}_i = (X_{i,1}, \dots, X_{i,n_i})$ is as follows,

$$\begin{aligned}
 n_i &\sim g(\cdot) \\
 Z_i &\sim \pi \\
 U_{i,j}|Z_i &\sim \tau_{Z_i} \\
 X_{i,j}|U_{i,j} &\sim f_{U_{i,j}}(\cdot)
 \end{aligned}$$

HierarchicalMixtureDistribution

```
class dmx.stats.hmixture.HierarchicalMixtureDistribution(topics, mixture_weights, topic_weights,
                                                         len_dist=NoneDistribution(), name=None,
                                                         keys=(None, None))
```

Define a sequence mixture whose outer components share topic distributions.

Parameters

- **topics** (*Sequence*[*SequenceEncodableProbabilityDistribution*])
- **mixture_weights** (*List*[*float*] | *ndarray*)
- **topic_weights** (*List*[*List*[*float*]] | *ndarray*)
- **len_dist** (*SequenceEncodableProbabilityDistribution* | *None*)
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*] | *None*)

topics

Topic distributions shared in hierarchical mixture distribution.

Type

Sequence[*SequenceEncodableProbabilityDistribution*]

num_topics

Number of topic distributions (i.e. sets number of inner-mixture weights).

Type

int

num_mixtures

Number of weights in outter-mixture (i.e. sets numer of top-layer mixture weights.)

Type

int

w

1-d numpy array of outer-mixture weights. Should sum to 1.

Type

np.ndarray

log_w

Numpy array of the log of w above.

Type

np.ndarray

taus

2-d array of dimension (num_mixtures by num_topics).

Type

np.ndarray

log_taus

2-d array of the log of tau above.

Type

np.ndarray

len_dist

Distribution for the sequence length on topics. Defaults to the NullDistribution if None is passed.

Type*SequenceEncodableProbabilityDistribution***name**

Name for object instance.

Type

Optional[str]

keys

Keys for the weights and topics.

Type

Tuple[Optional[str], Optional[str]]

__init__ (*topics, mixture_weights, topic_weights, len_dist=NullDistribution(), name=None, keys=(None, None)*)

Initialize a hierarchical mixture distribution.

Parameters

- **topics** (*Sequence[SequenceEncodableProbabilityDistribution]*) – Topic distributions shared in hierarchical mixture distribution.
- **mixture_weights** (*Union[List[float], np.ndarray]*) – One-d array of floats for weights on components of mixtures. Should sum to 1.0.
- **topic_weights** (*Union[List[List[float]], np.ndarray]*) – 2-d array with rows containing weights for each component mixture distribution. All rows should sum to 1.0.
- **len_dist** (*Optional[SequenceEncodableProbabilityDistribution]*) – Distribution for the length on the sequence distribution for the component mixtures
- **name** (*Optional[str]*) – Set name for object instance.
- **keys** (*Optional[Tuple[Optional[str], Optional[str]]]*) – Set keys for the weights and topics.

Parameters

- **topics** (*Sequence[SequenceEncodableProbabilityDistribution]*)
- **mixture_weights** (*List[float] | ndarray*)
- **topic_weights** (*List[List[float]] | ndarray*)
- **len_dist** (*SequenceEncodableProbabilityDistribution | None*)
- **name** (*str | None*)
- **keys** (*Tuple[str | None, str | None] | None*)

Return type

None

component_log_density(*x*)

Evaluate outer-component log-densities for one sequence.

Parameters

x (*Sequence[T]*) – An observation from a hierarchical mixture model.

Return type

numpy.ndarray

Returns

np.ndarray – Numpy array length of ‘num_mixtures’.

Parameters

x (*Sequence[T]*)

density(*x*)

Evaluate the density of one sequence observation.

Parameters

x (*Sequence[T]*) – A sequence of type data type T’s.

Return type

float

Returns

float – Density evaluated at x.

Parameters

x (*Sequence[T]*)

dist_to_encoder()

Return an encoder for flattened items and sequence lengths.

Return type

dmx.stats.hmixture.HierarchicalMixtureDataEncoder

estimator(*pseudo_count=None*)

Return an estimator using one shared estimator per topic.

Return type

dmx.stats.hmixture.HierarchicalMixtureEstimator

Parameters

pseudo_count (*float | None*)

log_density(*x*)

Evaluate the log-density of one sequence observation.

Note: Observation is a sequence.

Parameters $\mathbf{x}$ (*Sequence[T]*) – A sequence of type data type T's.**Return type**

float

Returns*float* – Log-density evaluated at $\mathbf{x}$.**Parameters** $\mathbf{x}$ (*Sequence[T]*)**posterior($\mathbf{x}$)**

Compute outer-component responsibilities for one sequence.

Parameters $\mathbf{x}$ (*Sequence[T]*) – An observed sequence of data type T.**Return type**

numpy.ndarray

Returns*np.ndarray* – Numpy array of length 'num_mixtures'.**Parameters** $\mathbf{x}$ (*Sequence[T]*)**sampler($seed=None$)**

Return a sampler for this hierarchical mixture.

Return type*dmx.stats.hmixture.HierarchicalMixtureSampler***Parameters****seed** (*int* | *None*)**seq_component_log_density($\mathbf{x}$)**

Evaluate outer-component log-densities for encoded observations.

This returns a numpy array with shape (rv[0], 'num_mixtures').

Note

This density is a Mixture of Sequence of Mixture, so the data must be bin-counted as last step in code.

Parameters $\mathbf{x}$ (*HierarchicalMixtureEncodedDataSequence*) – EncodedDataSequence for Hierarchical mixture observations.**Return type**

numpy.ndarray

Returns*np.ndarray* – Numpy array of dimensions 'rv[0]' by 'num_mixtures', containing the log-density for each component of the outer mixture.**Parameters** $\mathbf{x}$ (*HierarchicalMixtureEncodedDataSequence*)

seq_log_density(*x*)

Evaluate log-densities for encoded sequence observations.

Return type

`numpy.ndarray`

Parameters

x (*HierarchicalMixtureEncodedDataSequence*)

seq_posterior(*x*)

Evaluate outer-component responsibilities for encoded observations.

Parameters

x (*HierarchicalMixtureEncodedDataSequence*) – *EncodedDataSequence* for Hierarchical mixture observations.

Return type

`numpy.ndarray`

Returns

`np.ndarray` – dimension (`x[0]`, ‘num_mixtures’) containing posteriors for each observation.

Parameters

x (*HierarchicalMixtureEncodedDataSequence*)

to_mixture()

Returns a *MixtureDistribution* object created from object instance.

Return type

`dmx.stats.mixture.MixtureDistribution`

HierarchicalMixtureEstimator

```
class dmx.stats.hmixture.HierarchicalMixtureEstimator(estimators, num_mixtures,  
                                                    len_estimator=NullEstimator(),  
                                                    len_dist=None, suff_stat=None,  
                                                    pseudo_count=None, name=None,  
                                                    keys=(None, None))
```

Estimate a hierarchical mixture from sufficient statistics.

Notes

The accumulated count matrix has shape (`num_mixtures`, `num_components`): rows correspond to outer mixtures and columns correspond to topic distributions. If `pseudo_count` is provided without `suff_stat`, the count matrix is smoothed uniformly across all cells before being normalized into outer mixture weights and per-outer-mixture topic weights. If both are provided, `suff_stat` is treated as a prior target for that same matrix. The target affects both the outer weights and the conditional topic weights after normalization, so its shape and topic ordering must match the estimator.

`keys` controls sharing for two statistic blocks:

- `keys[0]` shares the outer-mixture by topic count matrix, which drives both the outer mixture weights and the per-outer-mixture topic weights.
- `keys[1]` shares the topic sufficient statistics by topic index.

The length estimator is not controlled by this tuple; use `keys` on the length estimator itself if length statistics should be shared. Shared topic keys assume topic index `i` has the same meaning across the hierarchical mixtures being fit. If topic roles are not aligned, keyed sharing will over-pool statistics.

Parameters

- **estimators** (*Sequence*[*ParameterEstimator*])
- **num_mixtures** (*int*)
- **len_estimator** (*ParameterEstimator* / *None*)
- **len_dist** (*SequenceEncodableProbabilityDistribution* / *None*)
- **suff_stat** (*ndarray* / *None*)
- **pseudo_count** (*float* / *None*)
- **name** (*str* / *None*)
- **keys** (*Tuple*[*str* / *None*, *str* / *None*] / *None*)

num_components

Number of topic distributions (inner-mixture).

Type

int

num_mixtures

Number of outer-mixture components.

Type

int

estimators

ParameterEstimator objects for the topics.

Type

Sequence[*ParameterEstimator*]

pseudo_count

Pseudo-count mass for the outer-mixture by topic count matrix.

Type

Optional[*float*]

suff_stat

2-d numpy array of dimension (num_mixtures, num_components). Prior target for the outer-mixture by topic count matrix.

Type

Optional[*np.ndarray*]

len_estimator

Estimator for the length of inner mixture sequences.

Type

Optional[*ParameterEstimator*]

keys

Keys for the mixture-by-topic count matrix and the topic sufficient statistics.

Type

Optional[*Tuple*[*Optional*[*str*], *Optional*[*str*]]]

len_dist

Fix the length on inner-mixture sequence distribution.

Type

Optional[[SequenceEncodableProbabilityDistribution](#)]

name

Name for object instance.

Type

Optional[str]

__init__ (*estimators*, *num_mixtures*, *len_estimator*=*NullEstimator*(), *len_dist*=*None*, *suff_stat*=*None*, *pseudo_count*=*None*, *name*=*None*, *keys*=(*None*, *None*))

Initialize a hierarchical-mixture estimator.

Parameters

- **estimators** (*Sequence*[[ParameterEstimator](#)]) – [ParameterEstimator](#) objects for the topics.
- **num_mixtures** (*int*) – Number of outer-mixture components.
- **len_estimator** (*Optional*[[ParameterEstimator](#)]) – Estimator for the length of inner mixture sequences.
- **len_dist** (*Optional*[[SequenceEncodableProbabilityDistribution](#)]) – Fix the length on inner-mixture sequence distribution.
- **suff_stat** (*Optional*[*np.ndarray*]) – 2-d numpy array of dimension (num_mixtures, num_components). Prior target for the outer-mixture by topic count matrix. The target is normalized into both outer weights and per-outer-mixture topic weights during estimation.
- **pseudo_count** (*Optional*[*float*]) – Pseudo-count mass applied to the outer-mixture by topic count matrix. With no **suff_stat** it smooths uniformly across all cells; with **suff_stat** it shrinks toward that target.
- **name** (*Optional*[*str*]) – Set a name to object instance.
- **keys** (*Optional*[*Tuple*[*Optional*[*str*], *Optional*[*str*]]]) – Keys that control sharing of sufficient statistics across [HierarchicalMixture](#) estimators with matching key values. **keys**[0] shares the outer-mixture by topic count matrix. **keys**[1] shares topic sufficient statistics by topic index. The tuple does not control the length estimator.

Parameters

- **estimators** (*Sequence*[[ParameterEstimator](#)])
- **num_mixtures** (*int*)
- **len_estimator** ([ParameterEstimator](#) | *None*)
- **len_dist** ([SequenceEncodableProbabilityDistribution](#) | *None*)
- **suff_stat** (*ndarray* | *None*)
- **pseudo_count** (*float* | *None*)
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*] | *None*)

Return type

None

accumulator_factory()

Return a factory sharing one child estimator per topic.

Return type

`dmx.stats.hmixture.HierarchicalMixtureEstimatorAccumulatorFactory`

estimate(*nobs*, *suff_stat*)

Estimate a HierarchicalMixtureDistribution from sufficient statistics.

Parameters

- **nobs** (*Optional[float]*) – Number of observations (not used).
- **suff_stat** (*Tuple[np.ndarray, Sequence[Any], Optional[Any]]*) – Count matrix, topic sufficient statistics, and optional length sufficient statistics.

Return type

`dmx.stats.hmixture.HierarchicalMixtureDistribution`

Returns

HierarchicalMixtureDistribution – Estimated distribution.

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[ndarray, Sequence[Any], Any | None]*)

HierarchicalMixtureSampler

class `dmx.stats.hmixture.HierarchicalMixtureSampler(dist, seed=None)`

HierarchicalMixtureSampler object for sampling from a hierarchical mixture model.

Parameters

- **dist** (*HierarchicalMixtureDistribution*)
- **seed** (*int | None*)

rng

RandomState object with seed set is passed as arg.

Type

RandomState

dist

HierarchicalMixtureDistribution instance to sample from.

Type

HierarchicalMixtureDistribution

sampler

Convert ‘dist’ to a MixtureDistribution for sampling.

Type

MixtureDistributionSampler

sample(*size=None*)

Draw one sequence or a collection of sequences.

Return type

`typing.Union[typing.Sequence[typing.Any], typing.Any]`

Parameters**size** (*int* | *None*)**Hidden Markov Distribution**

Hidden Markov Models (HMMs) are statistical models used to represent systems that are assumed to be a Markov process with hidden (unobserved) states. They are particularly useful in scenarios where the system being modeled is not directly observable, but can be inferred through observable outputs.

Table 2: Summary of Hidden Markov Models

Feature	Symbol	Description
Initial States	π	Finite set of initial hidden states representing possible initial conditions of the system.
Observations	$\mathbf{Y}_i = (y_i(0), \dots, y_i(t_i - 1))$	Outputs produced by hidden states according to a probability distribution.
Transition Probabilities	τ , and $\mathbf{S}$ by $\mathbf{S}$ matrix with entries $P(Z(t) = j Z(t-1) = i)$	Probabilities associated with transitioning from one hidden state to another.
Emission Probabilities	$f_k(y(t) Z(t) = k)$	Likelihood of producing each possible observation from hidden states.

The generative process for the Hidden Markov model is described as follows, for the initial value

$$\begin{aligned} Z(0) &\sim \pi \\ y(0) &\sim f_{Z(0)} \end{aligned}$$

for time points $1, 2, \dots, t-1$,

$$\begin{aligned} Z(t) | Z(t-1) &\sim \tau_{Z(t)} \\ Y(t) | Z(t) &\sim f_{Z(t)}(\cdot) \end{aligned}$$

HiddenMarkovModelDistribution

```
class dmx.stats.hidden_markov.HiddenMarkovModelDistribution(topics, w, transitions, taus=None,
                                                             len_dist=NullDistribution(),
                                                             name=None, keys=(None, None,
                                                             None), terminal_values=None,
                                                             use_numba=False)
```

Represent a finite-state HMM with observations of generic type T .

Without *taus*, *topics*[*i*] is the emission distribution for hidden state *i* and the number of topics equals the number of states. With *taus*, rows select topic-mixture weights for each state. The vectorized likelihood and estimation paths assume the one-topic-per-state form.

Parameters

- **topics** (*Sequence*[*SequenceEncodableProbabilityDistribution*])
- **w** (*Sequence*[*float*] | *ndarray*)
- **transitions** (*List*[*List*[*float*]] | *ndarray*)
- **taus** (*List*[*List*[*float*]] | *ndarray* | *None*)
- **len_dist** (*SequenceEncodableProbabilityDistribution* | *None*)
- **name** (*str* | *None*)

- **keys** (*Tuple[str | None, str | None, str | None] | None*)
- **terminal_values** (*Set[T] | None*)
- **use_numba** (*bool*)

topics

Emission distributions all having type T.

Type

Sequence[*SequenceEncodableProbabilityDistribution*]

n_topics

Number of emission distributions.

Type

int

n_states

Number of hidden states.

Type

int

w

Initial state probabilities.

Type

np.ndarray

log_w

Initial state log-probabilities.

Type

np.ndarray

transitions

2-d Numpy array of hidden state transition probabilities. (n_states by n_states).

Type

np.ndarray

log_transitions

Log of above.

Type

np.ndarray

taus

Emission distributions are a Mixture over topics. Hidden states govern transitions between mixture weights.

Type

Optional[np.ndarray]

log_taus

Log probabilities of taus above.

Type

Optional[np.ndarray]

has_topics

True if taus is passed.

Type

bool

len_dist

Distribution of the observed sequence length.

Type

SequenceEncodableProbabilityDistribution

name

Set name to object instance.

Type

Optional[str]

terminal_values

Define terminating emission outputs of the HMM.

Type

Optional[Set[T]]

use_numba

If True, use numba package for encoding and vectorized operations.

Type

bool

keys

Keys for initial states, transitions counts, and emission distributions. Defaults to Tuple of (None, None, None).

Type

Tuple[Optional[str], Optional[str], Optional[str]]

__init__ (*topics, w, transitions, taus=None, len_dist=NoneDistribution(), name=None, keys=(None, None, None), terminal_values=None, use_numba=False*)

Initialize a hidden Markov model.

Parameters

- **topics** (*Sequence[SequenceEncodableProbabilityDistribution]*) – Emission distributions all having type T.
- **w** (*Union[Sequence[float], np.ndarray]*) – Initial state probabilities.
- **transitions** (*Union[List[List[float]], np.ndarray]*) – 2-d array of hidden state transition probabilities.
- **taus** (*Optional[Union[Sequence[float], np.ndarray]]*) – Emission distributions are a Mixture over topics. Hidden states govern transitions between mixture weights.
- **len_dist** (*Optional[SequenceEncodableProbabilityDistribution]*) – Distribution of sequence lengths on the nonnegative integers.
- **name** (*Optional[str]*) – Set name to object instance.
- **keys** (*Tuple[Optional[str], Optional[str], Optional[str]]*) – Keys for initial states, transitions counts, and emission distributions. Defaults to Tuple of (None, None, None).
- **terminal_values** (*Optional[Set[T]]*) – Define terminating emission outputs of the HMM.

- **use_numba** (*bool*) – If True, use numba package for encoding and vectorized operations.

Parameters

- **topics** (*Sequence[SequenceEncodableProbabilityDistribution]*)
- **w** (*Sequence[float] | ndarray*)
- **transitions** (*List[List[float]] | ndarray*)
- **taus** (*List[List[float]] | ndarray | None*)
- **len_dist** (*SequenceEncodableProbabilityDistribution | None*)
- **name** (*str | None*)
- **keys** (*Tuple[str | None, str | None, str | None] | None*)
- **terminal_values** (*Set[T] | None*)
- **use_numba** (*bool*)

Return type

None

density(*x*)

Returns the density of HMM for an observed sequence *x*.

See ‘HiddenMarkovDistribution.log_density()’ for details.

Parameters

x (*Sequence[T]*) – Observed sequence of HMM emissions.

Return type

float

Returns

float – Density of HMM for observed sequence *x*.

Parameters

x (*Sequence[T]*)

dist_to_encoder()

Create an encoder for batches of observation sequences.

Return type

`dmx.stats.hidden_markov.HiddenMarkovDataEncoder`

estimator(*pseudo_count=None*)

Create an estimator for the Markov and emission parameters.

Return type

`dmx.stats.hidden_markov.HiddenMarkovEstimator`

Parameters

pseudo_count (*float | None*)

log_density(*x*)

Evaluate the HMM log density of one observation sequence.

For the one-topic-per-state form, this marginalizes the latent path with a scaled forward recursion and then adds the sequence-length log density. An empty or None sequence contributes only the length term.

Parameters

x (*Sequence[T]*) – Observed sequence of HMM emissions.

Return type

float

Returns*float* – Log-density of observed HMM sequence *x*.**Parameters****x** (*Sequence*[*T*])**sampler**(*seed=None*)

Create a sampler using sequence lengths or terminal values.

Return type*dmx.stats.hidden_markov.HiddenMarkovSampler***Parameters****seed** (*int* | *None*)**seq_log_density**(*x*)

Evaluate log densities for an encoded batch using forward recursions.

The encoder representation selects either the vectorized NumPy routine or the flattened Numba routine. Both return one value per observation sequence, including its length log density.

Return type*numpy.ndarray***Parameters****x** (*HiddenMarkovEncodedDataSequence*)**seq_posterior**(*x*)

Compute normalized forward state probabilities for encoded sequences.

Despite the historical method name, each returned row is the filtered probability $P(z_t = i \mid x_0, \dots, x_t)$, not the smoothed posterior conditioned on the complete sequence. This method requires the flattened Numba encoding.

Parameters**x** (*HiddenMarkovEncodedDataSequence*) – Numba encoded sequence of HMM observations.**Return type***typing.List*[*numpy.ndarray*]**Returns***List*[*np.ndarray*] – A list of posterior probabilities for each latent state for each observation sequence.**Parameters****x** (*HiddenMarkovEncodedDataSequence*)**seq_viterbi**(*x*)

Return flattened per-time maximizers from vectorized Viterbi scores.

As in [viterbi\(\)](#), this performs no predecessor traceback. The historical implementation requires `numba_enc=True` while consuming the banded encoding fields.

Notes

This takes a numba encoded sequence of HMM observations and returns back the flattened 1-d sequence of Viterbi states.

Parameters

x (*HiddenMarkovEncodedDataSequence*) – Numba EncodedDataSequence for Hidden Markov Model.

Return type

`numpy.ndarray`

Returns

Flattened zero-based hidden-state indices in encoded observation order.

Parameters

x (*HiddenMarkovEncodedDataSequence*)

viterbi(x)

Return per-time maximizing states from the Viterbi score table.

The method performs the max-product forward recurrence but does not store predecessor states or run traceback. Consequently, the returned state at each position maximizes that position's dynamic score and the full array is not guaranteed to be the globally maximizing state path.

Parameters

x (*Sequence[T]*) – Single HMM sequence.

Return type

`numpy.ndarray`

Returns

One zero-based hidden-state index per observation.

Parameters

x (*Sequence[T]*)

HiddenMarkovEstimator

```
class dmx.stats.hidden_markov.HiddenMarkovEstimator(estimators, len_estimator=NoneEstimator(),
                                                    pseudo_count=(None, None), name=None,
                                                    keys=(None, None, None), use_numba=False)
```

Estimate an HMM from expected sufficient statistics.

The statistic tuple contains the number of states, expected initial-state counts, expected state occupancies, expected transition counts, one emission statistic per state, and a length statistic. Emissions and lengths are delegated to their component estimators. Initial and transition counts are normalized separately, with optional uniform pseudo-count mass. Without transition smoothing, a row with no expected transitions remains all zero.

Notes

`keys` lets multiple HMM estimators share selected statistic blocks during fitting instead of sharing the entire model.

- `keys[0]` shares initial-state counts.
- `keys[1]` shares transition-count matrices.
- `keys[2]` shares emission sufficient statistics by hidden-state index.

The length estimator is not controlled by this tuple; any sharing for sequence lengths must come from keys on the length estimator itself.

A common pattern is `keys=("init_key", "trans_key", None)` to tie the Markov dynamics while leaving emissions independent. Only use `keys[2]` when state index `i` has the same semantic role in every model, otherwise the emission updates will be pooled across mismatched states.

`pseudo_count` has two slots: initial-state counts and transition counts. They smooth toward uniform initial-state probabilities and uniform transition probabilities, respectively. Emission regularization is controlled by the child estimators in `estimators`; the generic HMM estimator does not have a third emission pseudo-count slot. Length regularization is controlled by `len_estimator`.

Parameters

- **estimators** (*List*[*ParameterEstimator*])
- **len_estimator** (*ParameterEstimator* | *None*)
- **pseudo_count** (*Tuple*[*float* | *None*, *float* | *None*] | *None*)
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*, *str* | *None*] | *None*)
- **use_numba** (*bool*)

estimators

Estimators for emission distributions.

Type

List[*ParameterEstimator*]

len_estimator

Estimator for length distribution.

Type

ParameterEstimator

pseudo_count

Pseudo counts for initial-state and transition-count smoothing.

Type

Tuple[*Optional*[*float*], *Optional*[*float*]]

name

Name for the object instance.

Type

Optional[*str*]

keys

Keys for initial-state counts, transition counts, and emission sufficient statistics.

Type

Tuple[*Optional*[*str*], *Optional*[*str*], *Optional*[*str*]]

use_numba

Whether to use Numba for sequence encoding and vectorized functions.

Type

bool

__init__(*estimators*, *len_estimator*=*NullEstimator*(), *pseudo_count*=(*None*, *None*), *name*=*None*, *keys*=(*None*, *None*, *None*), *use_numba*=*False*)

Initialize an HMM estimator.

Parameters

- **estimators** – Estimators for emission distributions.
- **len_estimator** – Estimator for length distribution.
- **pseudo_count** – Two-slot tuple for initial-state and transition-count smoothing. Emission and length regularization must be configured on the child emission estimators and length estimator.
- **name** – Name for the object instance.
- **keys** – Keys that control sharing of sufficient statistics across HMM estimators with matching key values. **keys**[0] shares initial-state counts, **keys**[1] shares transition-count matrices, and **keys**[2] shares emission sufficient statistics by hidden-state index. This tuple does not affect the length estimator. Use **keys**=(*"init_key"*, *"trans_key"*, *None*) to tie the Markov dynamics while keeping emissions separate. If emission keys are shared, the hidden-state ordering must already be aligned across the models.
- **use_numba** – Whether to use Numba for sequence encoding and vectorized functions.

Raises

TypeError – If **keys** is not a tuple of three optional strings.

Parameters

- **estimators** (*List*[*ParameterEstimator*])
- **len_estimator** (*ParameterEstimator* | *None*)
- **pseudo_count** (*Tuple*[*float* | *None*, *float* | *None*] | *None*)
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*, *str* | *None*] | *None*)
- **use_numba** (*bool*)

Return type

None

accumulator_factory()

Return a factory for compatible HMM accumulators.

Return type

dmx.stats.hidden_markov.HiddenMarkovAccumulatorFactory

Returns

HiddenMarkovAccumulatorFactory – The accumulator factory.

estimate(*nobs*, *suff_stat*)

Estimate an HMM from aggregated expected counts.

Parameters

- **nobs** – Number of observations.
- **suff_stat** – Tuple containing number of states, initial-state counts, state counts, transition counts, child emission sufficient statistics, and length sufficient statistics.

Return type

dmx.stats.hidden_markov.HiddenMarkovModelDistribution

Returns

HiddenMarkovModelDistribution – The estimated distribution.

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*int*, *ndarray*, *ndarray*, *ndarray*, *List*[*T1*], *T2* | *None*])

HiddenMarkovSampler

class `dmx.stats.hidden_markov.HiddenMarkovSampler`(*dist*, *seed=None*)

Sample observation sequences and their implicit latent paths.

If ‘dist.len_dist’ is set, samples HMM sequences with sequence lengths generated from ‘len_dist’. If ‘dist.len_dist’ is *NullDistribution*, ‘dist.terminal_values’ is must be set. Samples are generated until a terminal value is reached.

Parameters

- **dist** (*HiddenMarkovModelDistribution*)
- **seed** (*int* | *None*)

num_states

Number of hidden states in ‘dist’ object.

Type

int

dist

HiddenMarkovModelDistribution object instance to sample from.

Type

HiddenMarkovModelDistribution

rng

RandomState object with seed set for sampling.

Type

RandomState

obs_samplers

List of *DistributionSampler* objects corresponding to the emission distributions of ‘dist’. Taken to be *MixtureSampler* objects if ‘dist.has_topics’ is *True*.

Type

List[*DistributionSampler*]

len_sampler

DistributionSampler object with data type *int* and support on non-negative integers for sampling HMM observation sequence lengths.

Type

Optional[*DistributionSampler*]

terminal_set

Set of values to terminate HMM sampling when calling ‘sample_seq()’.

Type

Optional[Set[T]]

state_sampler

MarkovChainSampler for sampling states of HMM.

Type

MarkovChainSampler

sample(size=None)

Draw iid samples from HMM.

If a ‘len_sampler’ is set, call ‘sample_seq()’ (See HiddenMarkovSampler.sample_seq() for details). If ‘len_sampler’ is the NullDistributionSampler(), ‘sample_terminal()’ is called. (See HiddenMarkovSampler.sample_terminal() for details).

Parameters

size (Optional[int]) – Number of iid HMM sequences to sample.

Return type

typing.Union[typing.List[typing.TypeVar(T)], typing.List[typing.List[typing.TypeVar(T)]]]

Returns

List[T] or List[List[T]] depending on arg size.

Parameters

size (int | None)

sample_seq(size=None)

Sample iid HMM sequences.

If size is None, 1 sample is drawn and a List[T] is returned. If size > 0, ‘size’ samples are drawn and a List of length ‘size’ with HMM sequences (List[T]) is returned.

Parameters

size (Optional[int]) – Number of iid HMM sequences to sample.

Return type

typing.Union[typing.List[typing.Any], typing.List[typing.List[typing.Any]]]

Returns

List[T] or List[List[T]] depending on size arg.

Parameters

size (int | None)

sample_terminal(terminal_set)

Sample through and including the first terminal observation.

Parameters

terminal_set (Set[T]) – Set values to terminate the HMM sequence.

Return type

typing.List[typing.TypeVar(T)]

Returns

List[T] with length determined by samples to reach the first terminating value.

Parameters**terminal_set** (*Set* [*T*])

1.3.3 Additional distributions and wrappers

Dirac Length Mixture

Mixture of a Dirac delta at v and a length distribution.

The DiracMixtureDistribution is defined by the density:

$$P(Y) = p \cdot P_1(Y) + (1 - p) \cdot \Delta_v(Y)$$

where:

- $P_1()$ is a length distribution with support on non-negative integers (or a subset thereof)
- $\Delta_v(x) = 1$ if $x = v$, else 0
- p is the probability of drawing from the length distribution, with $0 < p \leq 1$

class `dmx.stats.dirac_length.DiracMixtureAccumulator`(*accumulator*, *v=0*, *keys=(None, None)*,
name=None)

Bases: [*SequenceEncodableStatisticAccumulator*](#)

DiracMixtureAccumulator object for aggregating sufficient statistics.

Parameters

- **accumulator** ([*SequenceEncodableStatisticAccumulator*](#))
- **v** (*int*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*])
- **name** (*str* | *None*)

accumulator

Accumulator object for distribution with integer support.

Type[*SequenceEncodableStatisticAccumulator*](#)**comp_counts**

Sufficient statistics for mixture components.

Type`np.ndarray`**weight_key**

Key for weights of mixture.

Type`Optional[str]`**comp_key**

Key for components of mixture.

Type`Optional[str]`

v

Dirac spike value.

Type

int

name

Name for object.

Type

Optional[str]

acc_to_encoder()

Return a DiracMixtureDataEncoder for this accumulator.

Return type*dmx.stats.dirac_length.DiracMixtureDataEncoder***Returns***DiracMixtureDataEncoder* – Encoder object.**combine(suff_stat)**

Combine another accumulator's sufficient statistics into this one.

Parameters**suff_stat** (*Tuple[np.ndarray, SS0]*) – Sufficient statistics to combine.**Return type***dmx.stats.dirac_length.DiracMixtureAccumulator***Returns***DiracMixtureAccumulator* – Self after combining.**Parameters****suff_stat** (*Tuple[ndarray, SS0]*)**from_value(x)**

Set the sufficient statistics from a tuple.

Parameters**x** (*Tuple[np.ndarray, SS0]*) – Sufficient statistics.**Return type***dmx.stats.dirac_length.DiracMixtureAccumulator***Returns***DiracMixtureAccumulator* – Self after setting values.**Parameters****x** (*Tuple[ndarray, SS0]*)**initialize(x, weight, rng)**

Initialize accumulator with a new observation.

Parameters

- **x** (*int*) – Observation.
- **weight** (*float*) – Weight for the observation.
- **rng** (*np.random.RandomState*) – Random number generator.

Return type

None

Parameters

- **x** (*int*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge this accumulator into a dictionary by key.

Parameters**stats_dict** (*Dict[str, Any]*) – Dictionary of accumulators.**Return type**

None

Parameters**stats_dict** (*Dict[str, Any]*)**key_replace**(*stats_dict*)

Replace this accumulator's values with those from a dictionary by key.

Parameters**stats_dict** (*Dict[str, Any]*) – Dictionary of accumulators.**Return type**

None

Parameters**stats_dict** (*Dict[str, Any]*)**seq_initialize**(*x, weights, rng*)

Vectorized initialization for encoded data.

Parameters

- **x** (*DiracMixtureEncodedDataSequence*) – Encoded data sequence.
- **weights** (*np.ndarray*) – Weights for each observation.
- **rng** (*np.random.RandomState*) – Random number generator.

Return type

None

Parameters

- **x** (*DiracMixtureEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x, weights, estimate*)

Vectorized update for encoded data.

Parameters

- **x** (*DiracMixtureEncodedDataSequence*) – Encoded data sequence.
- **weights** (*np.ndarray*) – Weights for each observation.
- **estimate** (*DiracMixtureDistribution*) – Distribution estimate for update.

Return type

None

Parameters

- **x** ([DiracMixtureEncodedDataSequence](#))
- **weights** (*ndarray*)
- **estimate** ([DiracMixtureDistribution](#))

update(*x, weight, estimate*)

Update accumulator with a new observation.

Parameters

- **x** (*int*) – Observation.
- **weight** (*float*) – Weight for the observation.
- **estimate** ([DiracMixtureDistribution](#)) – Distribution estimate for update.

Return type

None

Parameters

- **x** (*int*)
- **weight** (*float*)
- **estimate** ([DiracMixtureDistribution](#))

value()

Return the sufficient statistics as a tuple.

Return type

`typing.Tuple[numpy.ndarray, typing.Any]`

Returns

`Tuple[np.ndarray, Any]` – (comp_counts, accumulator value)

```
class dmx.stats.dirac_length.DiracMixtureAccumulatorFactory(factory, v=0, keys=(None, None),  
                                                         name=None)
```

Bases: `StatisticAccumulatorFactory`

Create accumulators for the point-mass mixture and its child model.

Parameters

- **factory** (`StatisticAccumulatorFactory`)
- **v** (*int*)
- **keys** (`Tuple[str | None, str | None]`)
- **name** (`str | None`)

factory

`StatisticAccumulatorFactory` for mixture components.

Type

`StatisticAccumulatorFactory`

v

Dirac integer value.

Type

`int`

keys

Keys for weights and mixture components.

Type

`Tuple[Optional[str], Optional[str]]`

name

Name for object.

Type

`Optional[str]`

make()

Create a new DiracMixtureAccumulator.

Return type

`dmx.stats.dirac_length.DiracMixtureAccumulator`

Returns

DiracMixtureAccumulator – New accumulator instance.

class `dmx.stats.dirac_length.DiracMixtureDataEncoder`(*encoder*, *v=0*)

Bases: *DataSequenceEncoder*

Encode lengths for the mixture and its child distribution.

The encoded tuple identifies observations equal and unequal to the spike value and also contains the child encoder's representation of every observation.

Parameters

- **encoder** (*DataSequenceEncoder*)
- **v** (*int*)

encoder

DataSequenceEncoder for distribution with support on the integers.

Type

DataSequenceEncoder

v

Dirac spike value.

Type

`int`

seq_encode(x)

Encode a sequence of integers for DiracMixtureDistribution.

Parameters

x (*Sequence[int]*) – Sequence of integer observations.

Return type

`dmx.stats.dirac_length.DiracMixtureEncodedDataSequence`

Returns

DiracMixtureEncodedDataSequence – Encoded data sequence.

Parameters

x (*Sequence[int]*)

```
class dmx.stats.dirac_length.DiracMixtureDistribution(dist, p, v=0, name=None, keys=(None, None))
```

Bases: [SequenceEncodableProbabilityDistribution](#)

Mix a length distribution with a point mass at one integer.

Observations are scalar integer lengths. With probability *p* an observation is drawn from *dist*; with probability $1 - p$ it equals *v*. At *v* the two component probabilities are added, so *dist* may itself assign mass there.

Parameters

- **dist** ([SequenceEncodableProbabilityDistribution](#))
- **p** (*float*)
- **v** (*int*)
- **name** (*str* | *None*)
- **keys** (*Tuple[str | None, str | None]*)

p

Probability of being drawn from length distribution. Must be between 0 and 1.

Type

float

dist

Distribution with support on non-negative integers.

Type

[SequenceEncodableProbabilityDistribution](#)

v

Dirac spike value.

Type

int

name

Name for object instance.

Type

Optional[str]

keys

Keys for weights and components of mixture.

Type

Tuple[Optional[str], Optional[str]]

component_log_density(*x*)

Evaluate the log density for the components of the Dirac mixture.

The components are Dirac spike and *dist*.

Parameters

x (*int*) – Integer value with support on mixture components.

Return type

numpy.ndarray

Returns

np.ndarray – Log-densities for each component.

Parameters**x** (*int*)**density**(*x*)

Evaluate density of Dirac mixture distribution at observation *x*.

Parameters**x** (*int*) – Integer value.**Return type***float***Returns***float* – Density at *x*.**Parameters****x** (*int*)**dist_to_encoder**()

Return a DiracMixtureDataEncoder for this distribution.

Return type*dmx.stats.dirac_length.DiracMixtureDataEncoder***Returns***DiracMixtureDataEncoder* – Encoder object.**estimator**(*pseudo_count=None*)

Return a DiracMixtureEstimator for this distribution.

Parameters**pseudo_count** (*Optional[float], optional*) – Pseudo-count for regularization.**Return type***dmx.stats.dirac_length.DiracMixtureEstimator***Returns***DiracMixtureEstimator* – Estimator object.**Parameters****pseudo_count** (*float | None*)**log_density**(*x*)

Evaluate the log-density of Dirac mixture distribution at observation *x*.

$$\log(P(x)) = \log(p * P_1(x) + (1-p) * \text{Delta}_{\{v\}}(x))$$
Parameters**x** (*int*) – Integer value.**Return type***float***Returns***float* – log-density at *x*.**Parameters****x** (*int*)**posterior**(*x*)

Evaluate the posterior for the components of the Dirac mixture.

The components are Dirac spike and *dist*.

Parameters

x (*int*) – Integer value with support on mixture components.

Return type

`numpy.ndarray`

Returns

np.ndarray – Posterior probabilities for each component.

Parameters

x (*int*)

sampler(*seed=None*)

Return a DiracMixtureSampler for this distribution.

Parameters

seed (*Optional[int], optional*) – Seed for random number generator.

Return type

`dmx.stats.dirac_length.DiracMixtureSampler`

Returns

DiracMixtureSampler – Sampler object.

Parameters

seed (*int | None*)

seq_component_log_density(*x*)

Evaluate component log densities for encoded observations.

The components are Dirac spike and *dist*.

Parameters

x (*DiracMixtureEncodedDataSequence*) – EncodedDataSequence for DiracMixtureDistribution.

Return type

`numpy.ndarray`

Returns

np.ndarray – Log-densities for each component.

Parameters

x (*DiracMixtureEncodedDataSequence*)

seq_log_density(*x*)

Vectorized log-density for DiracMixtureEncodedDataSequence.

Parameters

x (*DiracMixtureEncodedDataSequence*) – Encoded data sequence.

Return type

`numpy.ndarray`

Returns

np.ndarray – Log-density values.

Parameters

x (*DiracMixtureEncodedDataSequence*)

seq_posterior(*x*)

Evaluate component posterior probabilities for encoded observations.

The components are Dirac spike and *dist*.

Parameters

x (*DiracMixtureEncodedDataSequence*) – EncodedDataSequence for DiracMixtureDistribution.

Return type

`numpy.ndarray`

Returns

`np.ndarray` – Posterior probabilities for each component.

Parameters

x (*DiracMixtureEncodedDataSequence*)

class `dmx.stats.dirac_length.DiracMixtureEncodedDataSequence`(*data*)

Bases: *EncodedDataSequence*

DiracMixtureEncodedDataSequence object for use with vectorized function calls.

Parameters

data (*Tuple[int, ndarray, ndarray, EncodedDataSequence]*)

data

Encoded sequence of iid Dirac mixture observations.

Type

Tuple[int, np.ndarray, np.ndarray, EncodedDataSequence]

class `dmx.stats.dirac_length.DiracMixtureEstimator`(*estimator, v=0, fixed_p=None, suff_stat=None, pseudo_count=None, name=None, keys=(None, None)*)

Bases: *ParameterEstimator*

DiracMixtureEstimator object for estimating DiracMixtureDistribution.

Parameters

- **estimator** (*ParameterEstimator*)
- **v** (*int*)
- **fixed_p** (*int | None*)
- **suff_stat** (*float | None*)
- **pseudo_count** (*float | None*)
- **name** (*str | None*)
- **keys** (*Tuple[str | None, str | None]*)

estimator

Estimator for components of mixture. Should have support on integers.

Type

ParameterEstimator

v

Spiked value.

Type

`int`

pseudo_count

Regularize sufficient statistics.

Type

Optional[float]

suff_stat

Regularize estimation on the Dirac probability.

Type

Optional[float]

keys

Keys for weights and components of mixture.

Type

Tuple[Optional[str], Optional[str]]

name

Assign a name to the object.

Type

Optional[str]

fixed_p_vec

Fixed Dirac spike probability.

Type

Optional[np.ndarray]

accumulator_factory()

Return a DiracMixtureAccumulatorFactory for this estimator.

Return type

dmx.stats.dirac_length.DiracMixtureAccumulatorFactory

Returns

DiracMixtureAccumulatorFactory – Factory object.

estimate(nobs, suff_stat)

Estimate a DiracMixtureDistribution from sufficient statistics.

Parameters

- **nobs** (*Optional[float]*) – Not used.
- **suff_stat** (*Tuple[np.ndarray, SS0]*) – Sufficient statistics.

Return type

dmx.stats.dirac_length.DiracMixtureDistribution

Returns

DiracMixtureDistribution – Estimated distribution.

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[ndarray, SS0]*)

class *dmx.stats.dirac_length.DiracMixtureSampler*(*dist, seed=None*)

Bases: *DistributionSampler*

DiracMixtureSampler used to generate samples.

Parameters

- **dist** ([DiracMixtureDistribution](#))
- **seed** (*int* | *None*)

rng

Seeded RandomState for sampling.

Type

RandomState

p

Probability of drawing from length distribution.

Type

float

dist_sampler

Sampler for the length distribution.

Type

[DistributionSampler](#)

v

Dirac location.

Type

int

sample (*size=None*)

Draw iid samples from a DiracMixture distribution.

Parameters

size (*Optional[int]*, *optional*) – Number of iid samples to draw.

Return type

`typing.Union[typing.List[int], int]`

Returns

Union[int, List[int]] – Single sample if size is None, else list of samples.

Parameters

size (*int* | *None*)

Integer Set Edit Distribution

Model one edit between two unordered subsets of a finite integer universe.

An observation is a pair (*before*, *after*) of collections representing sets of unique integers from 0 through *N* - 1. Collection order is ignored. For each integer, the edit independently chooses absence or presence in *after* conditional on absence or presence in *before*. The joint probability is the product of those *N* transition probabilities and *init_dist*'s probability for *before*.

Rows of *log_edit_pmat* correspond to integers. Two-column input gives log probabilities for *present* | *absent* and *present* | *present*; their complements are constructed. Four-column input is ordered as *absent* | *absent*, *absent* | *present*, *present* | *absent*, and *present* | *present*. The initial-set child distribution is used independently for scoring, sampling, encoding, accumulation, and estimation.

```
class dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator(num_vals,  
                                                                init_acc=NullAccumulator(),  
                                                                name=None, keys=None)
```


Bases: *SequenceEncodableStatisticAccumulator*

Accumulate removal, addition, retention, and initial-set statistics.

For each integer, the three explicit columns count `absent | present`, `present | absent`, and `present | present`. The complementary `absent | absent` count is derived from total observation weight. Initial sets are also forwarded to `init_acc`.

Parameters

- **num_vals** (*int*)
- **init_acc** (*SequenceEncodableStatisticAccumulator | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

pcnt

Counts for sufficient statistics.

Type

`np.ndarray`

keys

Keys for parameters of distribution.

Type

`Optional[str]`

num_vals

Number of values in the distribution.

Type

`int`

init_acc

Initial accumulator object.

Type

SequenceEncodableStatisticAccumulator

tot_sum

Total sum of weights.

Type

`float`

acc_to_encoder()

Create an edit encoder using the child accumulator's encoder.

Return type

dmx.stats.int_edit_setdist.IntegerBernoulliEditDataEncoder

combine(suff_stat)

Merge edit counts, total weight, and child statistics.

Return type

dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator

Parameters

suff_stat (*Tuple[ndarray, float, SS1 | None]*)

from_value(*x*)

Restore edit and child statistics from a serialized value.

Return type

dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator

Parameters

x (*Tuple*[*ndarray*, *float*, *SS1* | *None*])

initialize(*x*, *weight*, *rng*)

Initialize from one set pair and delegate its initial set.

Return type

None

Parameters

- **x** (*Tuple*[*Sequence*[*int*] | *ndarray*, *Sequence*[*int*] | *ndarray*])
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge keyed edit statistics and then delegate to the child.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

key_replace(*stats_dict*)

Replace keyed edit statistics and then delegate to the child.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

seq_initialize(*x*, *weights*, *rng*)

Initialize from weighted encoded edits and initial sets.

Return type

None

Parameters

- **x** (*IntegerBernoulliEditEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Add weighted encoded edit categories and initial sets.

Return type

None

Parameters

- **x** (*IntegerBernoulliEditEncodedDataSequence*)

- **weights** (*ndarray*)
- **estimate** (*IntegerBernoulliEditDistribution* | *None*)

update(*x, weight, estimate*)

Add one weighted set-pair observation.

Return type

None

Parameters

- **x** (*Tuple[Sequence[int] | ndarray, Sequence[int] | ndarray]*)
- **weight** (*float*)
- **estimate** (*IntegerBernoulliEditDistribution* | *None*)

value()

Return edit counts, total weight, and initial-set statistics.

Return type

typing.Tuple[numpy.ndarray, float, typing.Optional[typing.Any]]

```
class dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulatorFactory(num_vals,
                                                                    init_factory=NoneAccumulatorFactory(),
                                                                    keys=None,
                                                                    name=None)
```

Bases: *StatisticAccumulatorFactory*

Factory for creating Integer Bernoulli Edit Accumulators.

Parameters

- **num_vals** (*int*)
- **init_factory** (*StatisticAccumulatorFactory* | *None*)
- **keys** (*str* | *None*)
- **name** (*str* | *None*)

keys

Keys for parameters of distribution.

Type

Optional[str]

init_factory

Initial factory for creating accumulators.

Type

StatisticAccumulatorFactory

num_vals

Number of values in the distribution.

Type

int

name

Name for object.

Type

Optional[str]

make()

Create an empty edit accumulator with a new child accumulator.

Return type*dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator***class** dmx.stats.int_edit_setdist.**IntegerBernoulliEditDataEncoder**(*init_encoder*)Bases: *DataSequenceEncoder*

Encode set pairs as flattened removal, addition, and retention events.

Parameters**init_encoder** (*DataSequenceEncoder*)**init_encoder**

Initial encoder for the distribution.

Type*DataSequenceEncoder***seq_encode(x)**

Encode a batch of unordered (before, after) set pairs.

Parameters**x** (*Sequence[T]*) – Set pairs containing unique integers in the modeled universe.**Return type***dmx.stats.int_edit_setdist.IntegerBernoulliEditEncodedDataSequence***Returns***IntegerBernoulliEditEncodedDataSequence* –**Flattened transition events plus**

the child encoding of every initial set.

Parameters**x** (*Sequence[Tuple[Sequence[int] | ndarray, Sequence[int] | ndarray]*)**class** dmx.stats.int_edit_setdist.**IntegerBernoulliEditDistribution**(*log_edit_pmat*,
init_dist=NullDistribution(),
name=None, keys=None)Bases: *SequenceEncodableProbabilityDistribution*

Model a pair of unordered integer sets with independent edit transitions.

Parameters

- **log_edit_pmat** (*Sequence[Tuple[float, float]] | ndarray*)
- **init_dist** (*SequenceEncodableProbabilityDistribution | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

name

Name for object.

Type

Optional[str]

init_dist

Initial probability distribution.

Type

SequenceEncodableProbabilityDistribution

num_vals

Number of values in the distribution.

Type

int

orig_log_edit_pmat

Original log probabilities for the edit matrix.

Type

np.ndarray

log_edit_pmat

Log probabilities for the edit matrix, expanded to 4 columns.

Type

np.ndarray

log_nsum

Logarithmic sum of probabilities for missing values.

Type

float

log_dvec

Difference vector for log probabilities.

Type

np.ndarray

keys

Keys for parameters of distribution.

Type

Optional[str]

density(*x*)

Evaluate the joint probability of a (before, after) set pair.

Return type

float

Parameters

x (*Tuple*[*Sequence*[int] | ndarray, *Sequence*[int] | ndarray])

dist_to_encoder()

Create an encoder using the initial distribution's child encoder.

Return type

dmx.stats.int_edit_setdist.IntegerBernoulliEditDataEncoder

estimator(*pseudo_count=None*)

Create an estimator and delegate initial-set estimation to the child.

Return type

dmx.stats.int_edit_setdist.IntegerBernoulliEditEstimator

Parameters**pseudo_count** (*float* | *None*)**log_density**(*x*)

Evaluate the joint log probability of a (before, after) set pair.

Return type*float***Parameters****x** (*Tuple*[*Sequence*[*int*] | *ndarray*, *Sequence*[*int*] | *ndarray*])**sampler**(*seed=None*)

Create a sampler for initial sets and one-step edits.

Return type*dmx.stats.int_edit_setdist.IntegerBernoulliEditSampler***Parameters****seed** (*int* | *None*)**seq_log_density**(*x*)

Evaluate joint log probabilities for encoded set-pair observations.

Return type*numpy.ndarray***Parameters****x** (*IntegerBernoulliEditEncodedDataSequence*)**class** *dmx.stats.int_edit_setdist.IntegerBernoulliEditEncodedDataSequence*(*data*)Bases: *EncodedDataSequence*

Store flattened edit events and encoded initial sets.

Parameters**data** (*Tuple*[*int*, *ndarray*, *ndarray*, *ndarray*, *Tuple*[*ndarray*, *ndarray*, *ndarray*], *EncodedDataSequence*])**data** (*Tuple*[*int*, *np.ndarray*, *np.ndarray*, *np.ndarray*, *Tuple*[*np.ndarray*, *np.ndarray*, *np.ndarray*], *EncodedDataSequence*])

Encoded data containing size, indices, values, and other metadata.

class *dmx.stats.int_edit_setdist.IntegerBernoulliEditEstimator*(*num_vals*,
init_estimator=NullEstimator(),
min_prob=1.0e-128,
pseudo_count=None,
suff_stat=None, *name=None*,
keys=None)Bases: *ParameterEstimator*

Estimator for the Integer Bernoulli Edit Set Distribution.

Parameters

- **num_vals** (*int*)
- **init_estimator** (*ParameterEstimator* | *None*)
- **min_prob** (*float*)
- **pseudo_count** (*float* | *None*)

- **suff_stat** (*ndarray* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

num_vals

Number of values in the distribution.

Type

int

keys

Keys for parameters of distribution.

Type

Optional[str]

pseudo_count

Pseudo-count for smoothing.

Type

Optional[float]

suff_stat

Sufficient statistics for estimation.

Type

Optional[np.ndarray]

name

Name for object.

Type

Optional[str]

min_prob

Minimum probability value.

Type

float

init_est

Initial estimator object.

Type

ParameterEstimator

accumulator_factory()

Create a compatible factory using the initial-set child estimator.

Return type

dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulatorFactory

estimate(nobs, suff_stat)

Estimate transition probabilities and the initial-set distribution.

nobs is ignored. The three explicit transition counts plus total weight determine all four transition outcomes for every integer. *init_est* receives the third sufficient-statistic component independently.

Return type

dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[ndarray, float, SS1 | None]*)

class dmx.stats.int_edit_setdist.**IntegerBernoulliEditSampler**(*dist, seed=None*)

Bases: *DistributionSampler*

Sampler for the Integer Bernoulli Edit Set Distribution.

Parameters

- **dist** (*IntegerBernoulliEditDistribution*)
- **seed** (*int | None*)

rng

Random number generator for sampling.

Type

RandomState

dist

The distribution to sample from.

Type

IntegerBernoulliEditDistribution

init_rng

Initial sampler for the distribution.

Type

DistributionSampler

next_rng

Random number generator for subsequent samples.

Type

RandomState

sample(*size=None*)

Draw one initial/edited set pair, or size independent pairs.

Return type

*typing.Union[typing.List[typing.Tuple[typing.List[int], typing.List[int]]],
typing.Tuple[typing.List[int], typing.List[int]]]*

Parameters

size (*int | None*)

sample_given(*x*)

Samples from the distribution given a prior subset.

Parameters

x (*Sequence[Sequence[int]]*) – Prior subset.

Return type

typing.List[int]

Returns

List[int] – Sampled subset.

Parameters**x** (*Sequence[Sequence[int]]*)**Stepwise Integer Set Edit Distribution**

Model one edit between integer sets with stepwise transition estimates.

Observations are pairs (*before*, *after*) of unordered collections containing unique integers from 0 through $N - 1$. Each integer transitions independently between absence and presence, and *init_dist* models the initial set. Two-column edit input contains log probabilities for *present | absent* and *present | present*; four-column input lists both complementary outcomes first.

The distribution and encoder use the same per-integer edit semantics as *int_edit_setdist*. The estimator differs by projecting each fitted transition type onto two probability levels: it sorts per-integer probabilities and selects the split with the greatest Bernoulli likelihood.

```
class dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditAccumulator(num_vals,
                                                                    init_acc=NoneAccumulator(),
                                                                    name=None,
                                                                    keys=None)
```

Bases: *SequenceEncodableStatisticAccumulator*

Accumulate removal, addition, retention, and initial-set statistics.

The three count columns represent *absent | present*, *present | absent*, and *present | present*. Counts for *absent | absent* are derived from total weight. Initial sets are forwarded to the child accumulator.

Parameters

- **num_vals** (*int*)
- **init_acc** (*SequenceEncodableStatisticAccumulator* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

acc_to_encoder()

Create an edit encoder using the child accumulator's encoder.

Return type

dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditDataEncoder

combine(suff_stat)

Merge edit counts, total weight, and child statistics.

Return type

dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditAccumulator

Parameters

suff_stat (*Tuple[ndarray, float, SS1 | None]*)

from_value(x)

Restore edit and child statistics from a serialized value.

Return type

dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditAccumulator

Parameters

x (*Tuple[ndarray, float, SS1 | None]*)

initialize(*x, weight, rng*)

Initialize from one set pair and delegate its initial set.

Return type

None

Parameters

- **x** (*Tuple[Sequence[int] | ndarray, Sequence[int] | ndarray]*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge keyed edit statistics and then delegate to the child.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**key_replace**(*stats_dict*)

Replace keyed edit statistics and then delegate to the child.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**seq_initialize**(*x, weights, rng*)

Initialize from weighted encoded edits and initial sets.

Return type

None

Parameters

- **x** (*IntegerStepBernoulliEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x, weights, estimate*)

Add weighted encoded edit categories and initial sets.

Return type

None

Parameters

- **x** (*IntegerStepBernoulliEncodedDataSequence*)
- **weights** (*ndarray*)
- **estimate** (*IntegerStepBernoulliEditDistribution | None*)

update(*x, weight, estimate*)

Add one weighted set-pair observation.

Return type

None

Parameters

- **x** (*Tuple[Sequence[int] | ndarray, Sequence[int] | ndarray]*)
- **weight** (*float*)
- **estimate** (*IntegerStepBernoulliEditDistribution | None*)

value()

Return edit counts, total weight, and initial-set statistics.

Return type

typing.Tuple[numpy.ndarray, float, typing.Optional[typing.Any]]

```
class dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditAccumulatorFactory(num_vals,
                                                                              init_factory=NoneAccumulator,
                                                                              name=None,
                                                                              keys=None)
```

Bases: *StatisticAccumulatorFactory*

Create step-edit accumulators with compatible initial-set children.

Parameters

- **num_vals** (*int*)
- **init_factory** (*StatisticAccumulatorFactory | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

make()

Create an empty edit accumulator with a new child accumulator.

Return type

dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditAccumulator

```
class dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditDataEncoder(init_encoder)
```

Bases: *DataSequenceEncoder*

Encode set pairs as flattened removal, addition, and retention events.

Parameters

init_encoder (*DataSequenceEncoder*)

seq_encode(x)

Encode a batch of unordered (before, after) set pairs.

Each changed or retained integer becomes a flattened event categorized as removal, addition, or retention. Integers absent from both sets are implicit. The initial sets are also encoded by `init_encoder`.

Return type

dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEncodedDataSequence

Parameters

x (*Sequence[Tuple[Sequence[int] | ndarray, Sequence[int] | ndarray]]*)

```
class dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditDistribution(log_edit_pmat,
                                                                           init_dist=NoneDistribution(),
                                                                           name=None,
                                                                           keys=None)
```

Bases: [SequenceEncodableProbabilityDistribution](#)

Model an unordered integer-set pair with independent edit transitions.

Parameters

- **log_edit_pmat** ([Sequence\[Tuple\[float, float\]\]](#) | [ndarray](#))
- **init_dist** ([SequenceEncodableProbabilityDistribution](#) | [None](#))
- **name** ([str](#) | [None](#))
- **keys** ([str](#) | [None](#))

density(*x*)

Evaluate the joint probability of a (before, after) set pair.

Return type

[float](#)

Parameters

x ([Tuple\[Sequence\[int\]\]](#) | [ndarray](#), [Sequence\[int\]](#) | [ndarray](#))

dist_to_encoder()

Create an encoder using the initial distribution's child encoder.

Return type

[dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditDataEncoder](#)

estimator(*pseudo_count=None*)

Create a step-constrained estimator with the initial-set child estimator.

Return type

[dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditEstimator](#)

Parameters

pseudo_count ([float](#) | [None](#))

log_density(*x*)

Evaluate the joint log probability of a (before, after) set pair.

Return type

[float](#)

Parameters

x ([Tuple\[Sequence\[int\]\]](#) | [ndarray](#), [Sequence\[int\]](#) | [ndarray](#))

sampler(*seed=None*)

Create a sampler for initial sets and one-step edits.

Return type

[dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditSampler](#)

Parameters

seed ([int](#) | [None](#))

seq_log_density(*x*)

Evaluate joint log probabilities for encoded set-pair observations.

Return type

[numpy.ndarray](#)

Parameters**x** ([IntegerStepBernoulliEncodedDataSequence](#))

```
class dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditEstimator(num_vals,
                                                                    init_estimator=NoneEstimator(),
                                                                    min_prob=1.0e-128,
                                                                    pseudo_count=None,
                                                                    suff_stat=None,
                                                                    name=None,
                                                                    keys=None)
```

Bases: [ParameterEstimator](#)

Estimate edit probabilities and project each transition onto two levels.

Parameters

- **num_vals** (*int*)
- **init_estimator** ([ParameterEstimator](#) | *None*)
- **min_prob** (*float*)
- **pseudo_count** (*float* | *None*)
- **suff_stat** (*ndarray* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

accumulator_factory()

Create a compatible factory using the initial-set child estimator.

Return type[dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditAccumulatorFactory](#)**estimate**(*nobs*, *suff_stat*)

Estimate a step-constrained edit model and its initial-set model.

nobs is ignored. Unconstrained removal and addition probabilities are first obtained from the edit counts. For each transition type, integers are sorted by probability and the maximum-likelihood split into one high and one low level is selected. Initial-set statistics are estimated independently by `init_est`.

Return type[dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditDistribution](#)**Parameters**

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *float*, *SS1* | *None*])

```
class dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditSampler(dist, seed=None)
```

Bases: [DistributionSampler](#)

Sample initial sets and their independently edited successors.

Parameters

- **dist** ([IntegerStepBernoulliEditDistribution](#))
- **seed** (*int* | *None*)

sample(*size=None*)

Draw one initial/edited set pair, or size independent pairs.

Return type`typing.Union[typing.List[typing.Tuple[typing.List[int], typing.List[int]]],
typing.Tuple[typing.List[int], typing.List[int]]]`**Parameters****size** (*int* | *None*)**sample\_given**(*x*)Draw an edited set conditional on the final set in *x*.**Return type**`typing.Sequence[int]`**Parameters****x** (*Sequence[Sequence[int]]*)**class** `dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEncodedDataSequence`(*data*)Bases: [`EncodedDataSequence`](#)

Store flattened edit events and encoded initial sets.

Parameters**data** (*Tuple[int, ndarray, ndarray, ndarray, Tuple[ndarray, ndarray,
ndarray], EncodedDataSequence]*)

Null Distribution

Create, estimate, and sample from a null distribution.

Defines the `NullDistribution`, `NullSampler`, `NullAccumulatorFactory`, `NullAccumulator`, `NullEstimator`, and the `NullDataEncoder` classes for use with `dmx-learn`.The `NullDistribution` object and its related classes are space filling objects meant for consistency in type hints.

Notes

The density evaluates to 1.0 for any value (Any data type). The sampler generates `None` for any size input. Sequence encodings return `None` for any input.**class** `dmx.stats.null_dist.NullAccumulator`(*keys=None*)Bases: [`SequenceEncodableStatisticAccumulator`](#)Implement the accumulator protocol with an invariant `None` statistic.

Notes

All functions do nothing. They are kept for consistency with other classes to ensure type checks.

Parameters**keys** (*str* | *None*)**keys**

Set key for distribution.

Type`Optional[str]`

acc_to_encoder()

Return an encoder that discards every observation.

Return type

dmx.stats.null_dist.NullDataEncoder

combine(suff_stat)

Ignore another statistic and return this accumulator.

Return type

dmx.stats.null_dist.NullAccumulator

Parameters

suff_stat (*Any* | *None*)

from_value(x)

Ignore a supplied value and return this accumulator.

Return type

dmx.stats.null_dist.NullAccumulator

Parameters

x (*Any* | *None*)

initialize(x, weight, rng)

Ignore one initialization observation.

Return type

None

Parameters

- **x** (*Any* | *None*)
- **weight** (*float*)
- **rng** (*RandomState* | *None*)

key_merge(stats_dict)

Register *None* under the configured key if it is absent.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

key_replace(stats_dict)

Leave this stateless accumulator unchanged.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

seq_initialize(x, weights, rng)

Ignore an encoded initialization sequence.

Return type

None

Parameters

- **x** ([NullEncodedDataSequence](#))
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x, weights, estimate*)

Ignore an encoded sequence, weights, and estimate.

Return type

None

Parameters

- **x** ([NullEncodedDataSequence](#))
- **weights** (*ndarray*)
- **estimate** ([NullDistribution](#) / *None*)

update(*x, weight, estimate*)

Ignore one observation, its weight, and its estimate.

Return type

None

Parameters

- **x** (*Any* / *None*)
- **weight** (*float*)
- **estimate** ([NullDistribution](#) / *None*)

value()

Return the invariant *None* sufficient statistic.

Return type

None

class `dmx.stats.null_dist.NullAccumulatorFactory`(*keys=None*)

Bases: `StatisticAccumulatorFactory`

`NullAccumulatorFactory` object for creating `NullAccumulator` objects.

Notes

All functions do nothing. They are kept for consistency with other classes to ensure type checks.

Parameters

keys (*str* / *None*)

keys

Set key for distribution.

Type

`Optional[str]`

make()

Create a null accumulator with the configured key.

Return type

`dmx.stats.null_dist.NullAccumulator`

class `dmx.stats.null_dist.NullDataEncoder`

Bases: *DataSequenceEncoder*

NullDataEncoder object for consistency with DataSequenceEncoders.

Notes

This enables consistency in type-hints and type-checks for other encodings.

seq_encode(*x*)

Discard the input sequence and return a null encoding.

Return type

dmx.stats.null_dist.NullEncodedDataSequence

Parameters

x (*Any*)

class `dmx.stats.null_dist.NullDistribution`(*name=None*)

Bases: *SequenceEncodableProbabilityDistribution*

Provide a neutral placeholder for an absent distribution component.

The scalar density is the constant one on every supplied value and the log density is zero. This is a neutral likelihood factor, not a normalized distribution on an arbitrary non-singleton support. Sampling always returns *None*. Encoding discards all observations, so vectorized scoring returns the implementation's fixed one-element zero array rather than preserving batch size.

Parameters

name (*str* / *None*)

name

Name for object.

Type

Optional[*str*]

density(*x*)

Density for NullDistribution.

Parameters

x (*Optional[Any]*) – Can pass any value.

Return type

float

Returns

float – Always evaluates to 1.0.

Parameters

x (*Any* / *None*)

dist_to_encoder()

Return an encoder that discards its input.

Return type

dmx.stats.null_dist.NullDataEncoder

estimator(*pseudo_count=None*)

Create an estimator whose result is another null distribution.

Return type*dmx.stats.null_dist.NullEstimator***Parameters****pseudo_count** (*float* | *None*)**log_density**(*x*)

Log-density for NullDistribution.

Parameters**x** (*Optional[Any]*) – Can pass any value.**Return type***float***Returns***float* – Always evaluates to 0.0.**Parameters****x** (*Any* | *None*)**sampler**(*seed=None*)Create a sampler that always returns *None*.**Return type***dmx.stats.null_dist.NullSampler***Parameters****seed** (*int* | *None*)**seq_log_density**(*x*)

Return a one-element zero vector regardless of encoded input.

Return type*numpy.ndarray***Parameters****x** (*NullEncodedDataSequence*)**class** *dmx.stats.null_dist.NullEncodedDataSequence*(*data*)Bases: *EncodedDataSequence**NullEncodedDataSequence* object for vectorized calls.**Notes**

This enables consistency in type-hints and type-checks for other encodings.

Parameters**data** (*None*)**data***None* is passed as placeholder.**Type***None***class** *dmx.stats.null_dist.NullEstimator*(*pseudo_count=None, suff_stat=None, name=None, keys=None*)Bases: *ParameterEstimator*

Produce a null distribution without using any observations.

Notes

Always estimates to same NullDistribution object. This is simply a placeholder.

Parameters

- **pseudo_count** (*float* | *None*)
- **suff_stat** (*Any* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

pseudo_count

Regularize sufficient statistics (ignored).

Type

Optional[float]

suff_stat

Can pass anything, is simply ignored.

Type

Optional[Any]

keys

Key for distribution (not meaningful as all estimates are NullDistribution())

Type

Optional[str]

name

Name for estimator.

Type

Optional[str]

accumulator_factory()

Create a factory for invariant null sufficient statistics.

Return type

dmx.stats.null_dist.NullAccumulatorFactory

estimate(*nobs*, *suff_stat=None*)

Return a named null distribution, ignoring all supplied statistics.

Return type

dmx.stats.null_dist.NullDistribution

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Any* | *None*)

class *dmx.stats.null_dist.NullSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*

NullSampler object, always generates None as sample type.

Note

This generally serves as a place-holder for consistency with other classes. Try to remove it before sampling.

Parameters

- **dist** (*NullDistribution*)
- **seed** (*int* | *None*)

rng

For consistency with other samplers.

Type

RandomState

dist

For consistency with other samplers.

Type

NullDistribution

sample (*size=None*)

Generate samples from *NullDistribution*.

Notes

Always returns *None* regardless of size.

Parameters

size (*Optional[int]*) – For consistency, does not control number of samples.

Return type

None

Returns

None

Parameters

size (*int* | *None*)

Select Distribution

Create, estimate, and sample from a select distribution.

Defines the *SelectDistribution*, *SelectSampler*, *SelectAccumulatorFactory*, *SelectAccumulator*, *SelectEstimator*, and the *SelectDataEncoder* classes for use with dmx-learn.

The *SelectDistribution* samples from a set of *SequenceEncodableProbabilityDistribution* objects. The a choice function maps an observation a distribution from the set of distributions.

class `dmx.stats.select.SelectDataEncoder` (*encoders, choice_function*)

Bases: *DataSequenceEncoder*

Partition observations by route and invoke the matching child encoders.

The encoded representation stores only routes present in the input together with their original positions. Equality compares child encoders but, for historical compatibility, does not compare the choice functions. Vectorized consumers treat populated groups positionally, so batches must populate routes contiguously from child zero and in that order.

Parameters

- **encoders** (*Sequence*[*DataSequenceEncoder*])
- **choice_function** (*Callable*[[*Any*], *int*])

seq_encode(*x*)

Group observations by selected index and encode each populated group.

Return type

dmx.stats.select.SelectEncodedDataSequence

Parameters

x (*Sequence*[*Any*])

class *dmx.stats.select.SelectDistribution*(*dists*, *choice_function*)

Bases: *SequenceEncodableProbabilityDistribution*

Route each observation to one of several child distributions.

choice_function(*x*) must return a valid child index. Scoring uses only that child, so the support is the union of the routed pieces of the child supports. This piecewise density is normalized only when the integrals of the children on their routed regions sum to one; normalization of every child alone is not sufficient.

Encoding and estimation partition observations with the same choice function. Vectorized operations consume populated route groups positionally; agreement with scalar operations therefore requires those groups to occur in child-index order starting at zero. This is stricter than scalar scoring, which accepts any valid child index directly. Sampling is deliberately different: it returns one draw from every child (or a zip of sized child draws), because no distribution over route indices exists. It therefore does not generate ordinary observations for this routed density.

Parameters

- **dists** (*Sequence*[*SequenceEncodableProbabilityDistribution*])
- **choice_function** (*Callable*[[*Any*], *int*])

density(*x*)

Evaluate the density of the child selected for *x*.

Return type

float

Parameters

x (*Any*)

dist_to_encoder()

Create an encoder that partitions observations by route.

Return type

dmx.stats.select.SelectDataEncoder

estimator(*pseudo_count=None*)

Create one child estimator per route.

Return type

dmx.stats.select.SelectEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Evaluate the log density of the child selected for *x*.

Return type
float

Parameters
x (*Any*)

sampler(*seed=None*)

Create a sampler that draws from every child.

Return type
dmx.stats.select.SelectSampler

Parameters
seed (*int* | *None*)

seq_log_density(*x*)

Score each encoded route group and restore original observation order.

Raises
TypeError – If *x* was not produced by a select encoder.

Return type
numpy.ndarray

Parameters
x (*SelectEncodedDataSequence*)

class *dmx.stats.select.SelectEncodedDataSequence*(*data*)

Bases: *EncodedDataSequence*

Store route positions, route indices, and routed child encodings.

Parameters
data (*Tuple*[*Tuple*[*ndarray*, ...], *Tuple*[*int*, ...],
Tuple[*EncodedDataSequence*, ...]])

class *dmx.stats.select.SelectEstimator*(*estimators*, *choice_function*)

Bases: *ParameterEstimator*

Estimate each routed child independently from its grouped statistic.

Parameters
• **estimators** (*Sequence*[*ParameterEstimator*[*Any*]])
• **choice_function** (*Callable*[[*Any*], *int*])

accumulator_factory()

Create a factory preserving child order and routing.

Return type
dmx.stats.select.SelectEstimatorAccumulatorFactory

estimate(*nobs*, *suff_stat*)

Fit each child using its route weight as that child's observation count.

Return type
dmx.stats.select.SelectDistribution

Parameters
• **nobs** (*float* | *None*)
• **suff_stat** (*Any*)

class `dmx.stats.select.SelectEstimatorAccumulator`(*accumulators*, *choice_function*)

Bases: `SequenceEncodableStatisticAccumulator`

Maintain route weights and one sufficient statistic per child.

Scalar and encoded observations are sent only to the accumulator selected by *choice_function*. Scalar updates pass `None` as the child estimate, whereas encoded updates pass the corresponding child distribution. The public value is a one-shot zip iterator of (*route_weight*, *child_statistic*) pairs. Key merge and replacement are no-ops; child key contracts are not traversed by this wrapper.

Parameters

- **accumulators** (`Sequence[SequenceEncodableStatisticAccumulator]`)
- **choice_function** (`Callable[[Any], int]`)

acc_to_encoder()

Build a routed encoder from the child accumulator encoders.

Return type

`dmx.stats.select.SelectDataEncoder`

combine(*suff_stat*)

Combine route weights and child statistics positionally.

Return type

`dmx.stats.select.SelectEstimatorAccumulator`

Parameters

suff_stat (*Any*)

from_value(*x*)

Restore route weights and child statistics from an iterable.

Return type

`dmx.stats.select.SelectEstimatorAccumulator`

Parameters

x (*Any*)

initialize(*x*, *weight*, *rng*)

Initialize the selected child and add its observation weight.

Return type

`None`

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Leave the shared-statistics dictionary unchanged.

Return type

`None`

Parameters

stats_dict (`Dict[str, Any]`)

key_replace(*stats_dict*)

Leave all routed child statistics unchanged.

Return type

None

Parameters**stats_dict** (*Dict*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *rng*)

Initialize every encoded route group with independent randomness.

Return type

None

Parameters

- **x** (*SelectEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Update every encoded route group with its corresponding estimate.

Return type

None

Parameters

- **x** (*SelectEncodedDataSequence*)
- **weights** (*ndarray*)
- **estimate** (*SelectDistribution*)

update(*x*, *weight*, *estimate*)

Update the selected child and its total observation weight.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **estimate** (*SelectDistribution* / *None*)

value()

Return a zip iterator of route-weight and child-statistic pairs.

Return type*typing.Any***class** `dmx.stats.select.SelectEstimatorAccumulatorFactory`(*estimators*, *choice_function*)Bases: *StatisticAccumulatorFactory*

Create a routed accumulator from child estimators.

Parameters

- **estimators** (*Sequence*[*ParameterEstimator*[*Any*]])

- **choice_function** (*Callable*[[*Any*], *int*])

make()

Create one accumulator for every child estimator.

Return type

dmx.stats.select.SelectEstimatorAccumulator

class *dmx.stats.select.SelectSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*

Draw independently from every child rather than selecting a route.

Parameters

- **dist** (*SelectDistribution*)
- **seed** (*int* | *None*)

sample(*size=None*)

Return a tuple of scalar child draws or a zip of sized child draws.

Return type

typing.Any

Parameters

size (*int* | *None*)

Spearman Ranking Distribution

Provide a finite distribution over complete rankings.

The support is all permutations of `range(dim)`. A ranking x has mass proportional to $\exp(-\rho\|x - \sigma\|^2)$, where `sigma` is a reference rank vector and `rho` controls concentration. Normalization and sampling enumerate every permutation, so this implementation is suitable only for small dimensions. The estimator accumulates weighted rank sums, sets `sigma` to their ordering, and fixes `rho` to one when data are present; it is a ranking-center estimator, not a general correlation fit.

class *dmx.stats.spearman_rho.SpearmanRankingAccumulator*(*dim*, *name=None*, *keys=None*)

Bases: *SequenceEncodableStatisticAccumulator*

Accumulate weighted rank sums for the ranking-center estimator.

Parameters

- **dim** (*int*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

sum

Suff stat counts

Type

np.ndarray

count

Suff stat total weight count.

Type

float

keys

Key for distribution.

Type

Optional[str]

name

Name for object.

Type

Optional[str]

acc_to_encoder()

Return the ranking encoder.

Return type

dmx.stats.spearman_rho.SpearmanRankingDataEncoder

combine(suff_stat)

Combine rank-sum sufficient statistics.

Return type

dmx.stats.spearman_rho.SpearmanRankingAccumulator

Parameters

suff_stat (*Tuple[float, ndarray]*)

from_value(x)

Restore total weight and rank sums from a tuple.

Return type

dmx.stats.spearman_rho.SpearmanRankingAccumulator

Parameters

x (*Tuple[float, ndarray]*)

initialize(x, weight, rng)

Accumulate a weighted encoded ranking batch.

Return type

None

Parameters

- **x** (*List[int] | ndarray*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(stats_dict)

Merge keyed rank-sum sufficient statistics.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

key_replace(stats_dict)

Replace keyed rank-sum sufficient statistics.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**seq_initialize**(*x, weights, rng*)

Initialize from an encoded ranking batch.

Return type

None

Parameters

- **x** (*SpearmanRankingEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x, weights, estimate*)

Initialize from an encoded ranking batch.

Return type

None

Parameters

- **x** (*SpearmanRankingEncodedDataSequence*)
- **weights** (*ndarray*)
- **estimate** (*SpearmanRankingDistribution* / *None*)

update(*x, weight, estimate*)

Initialize from one ranking without randomization.

Return type

None

Parameters

- **x** (*List[int]* / *ndarray*)
- **weight** (*float*)
- **estimate** (*SpearmanRankingDistribution* / *None*)

value()

Return total weight and rank-sum vector.

Return type*typing.Tuple*[*float*, *numpy.ndarray*]**class** *dmx.stats.spearman_rho.SpearmanRankingAccumulatorFactory*(*dim, name=None, keys=None*)Bases: *StatisticAccumulatorFactory*

Create rank-sum accumulators for a fixed ranking dimension.

Parameters

- **dim** (*int*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

dim

Dimension of rankings.

Type

int

keys

Key for distribution.

Type

Optional[str]

name

Name for object.

Type

Optional[str]

make()

Create a fresh ranking accumulator.

Return type

dmx.stats.spearman_rho.SpearmanRankingAccumulator

class dmx.stats.spearman_rho.SpearmanRankingDataEncoder

Bases: *DataSequenceEncoder*

Encoder for sequences of Spearman rho observations.

seq_encode(*x*)

Encode rankings as a two-dimensional integer array.

Return type

dmx.stats.spearman_rho.SpearmanRankingEncodedDataSequence

Parameters

x (*Sequence[List[int]]*)

class dmx.stats.spearman_rho.SpearmanRankingDistribution(*sigma, rho=1.0, name=None, keys=None*)

Bases: *SequenceEncodableProbabilityDistribution*

Represent a squared-rank-distance distribution over permutations.

Parameters

- **sigma** (*Sequence[float] | ndarray*)
- **rho** (*float*)
- **name** (*str | None*)
- **keys** (*str | None*)

sigma

Numpy array of means for the rank variables.

Type

np.ndarray]

rho

Decay rate on variance of ranks.

Type

float

name

Name for object instance.

Type

Optional[str]

dim

Dimension of the rank variable.

Type

int

keys

Set keys for object instance.

Type

Optional[str]

density(*x*)

Return the probability of one complete ranking.

Return type

float

Parameters

x (*List[int]*)

dist_to_encoder()

Return the encoder for complete-ranking batches.

Return type

dmx.stats.spearman_rho.SpearmanRankingDataEncoder

estimator(*pseudo_count=None*)

Create the ranking-center estimator.

Return type

dmx.stats.spearman_rho.SpearmanRankingEstimator

Parameters

pseudo_count (*float | None*)

log_density(*x*)

Return the log probability of one complete ranking.

Return type

float

Parameters

x (*List[int]*)

sampler(*seed=None*)

Create an exact enumerative ranking sampler.

Return type

dmx.stats.spearman_rho.SpearmanRankingSampler

Parameters**seed** (*int* / *None*)**seq_log_density**(*x*)

Return log probabilities for an encoded ranking batch.

Return type*numpy.ndarray***Parameters****x** (*SpearmanRankingEncodedDataSequence*)**class** *dmx.stats.spearman_rho.SpearmanRankingEncodedDataSequence*(*data*)Bases: *EncodedDataSequence*

Hold encoded rankings with shape (N, dim).

Parameters**data** (*ndarray*)**data**

Iid observations from spearman rho ranking distribution.

Type*np.ndarray***class** *dmx.stats.spearman_rho.SpearmanRankingEstimator*(*dim*, *pseudo_count=None*, *suff_stat=None*,
name=None, *keys=None*)Bases: *ParameterEstimator*

Estimate a ranking center from weighted rank sums.

Parameters

- **dim** (*int*)
- **pseudo_count** (*float* / *None*)
- **suff_stat** (*Tuple[float, ndarray]* / *None*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

dim

Dimension of rankings.

Type*int***psuedo_count**

Regularize suff stat for estimates.

Type*Optional[float]***suff_stat**

Suff stat for regularization.

Type*Optional[Tuple[float, np.ndarray]]*

keys

Key for distribution.

Type

Optional[str]

name

Name for object.

Type

Optional[str]

accumulator_factory()

Return a factory for rank-sum statistics.

Return type

dmx.stats.spearman_rho.SpearmanRankingAccumulatorFactory

estimate(nobs, suff_stat)

Estimate the reference ranking and fixed concentration parameter.

Return type

dmx.stats.spearman_rho.SpearmanRankingDistribution

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*float*, *ndarray*])

class `dmx.stats.spearman_rho.SpearmanRankingSampler`(*dist*, *seed=None*)

Bases: *DistributionSampler*

Sample rankings by enumerating the finite permutation support.

Parameters

- **dist** (*SpearmanRankingDistribution*)
- **seed** (*int* | *None*)

rng

Seed samples.

Type

RandomState

dist

Distribution to draw samples from.

Type

SpearmanRankingDistribution

perms

List of all possible rankings.

Type

List[List[int]]

probs

Probability of each permutation.

Type

np.ndarray

sample(*size=None*)

Draw one ranking, or size independently drawn rankings.

Return type

typing.Union[typing.List[int], typing.Sequence[typing.List[int]]]

Parameters**size** (*int* | *None*)**Von Mises–Fisher Distribution**

Provide von Mises–Fisher distributions and estimation utilities.

VonMisesFisherDistribution models unit vectors on S^{p-1} with mean direction μ and concentration κ . The module also provides sampling, weighted sufficient statistics, maximum-likelihood estimation, and sequence encoding for directional observations.

class dmx.stats.vmf.VonMisesFisherAccumulator(*dim=None, name=None, keys=None*)Bases: [SequenceEncodableStatisticAccumulator](#)

Accumulator for sufficient statistics of the vmf distribution.

Parameters

- **dim** (*int* | *None*)
- **name** (*str* | *None*)
- **keys** (*str* | *None*)

dim

Dimension of the data.

Type

Optional[int]

count

Total weight of observations.

Type

float

ssum

Sum of weighted observations.

Type

np.ndarray

key

Key for the accumulator instance.

Type

Optional[str]

name

Name for the accumulator instance.

Type

Optional[str]

acc_to_encoder()

Create the matching directional data encoder.

Return type

dmx.stats.vmf.VonMisesFisherDataEncoder

combine(suff_stat)

Add a (count, vector_sum) statistic to this accumulator.

Return type

dmx.stats.vmf.VonMisesFisherAccumulator

Parameters

suff_stat (*Tuple[float, ndarray | None]*)

from_value(x)

Replace this accumulator from a sufficient-statistic tuple.

Return type

dmx.stats.vmf.VonMisesFisherAccumulator

Parameters

x (*Tuple[float, ndarray | None]*)

initialize(x, weight, rng)

Initialize from one direction vector; rng is unused.

Return type

None

Parameters

- **x** (*Sequence[float] | ndarray*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(stats_dict)

Merge this accumulator into stats_dict under its key.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

key_replace(stats_dict)

Replace this accumulator from stats_dict when keyed.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

seq_initialize(x, weights, rng)

Initialize from encoded observations; rng is unused.

Return type

None

Parameters

- **x** (`VonMisesFisherEncodedDataSequence`)
- **weights** (`ndarray`)
- **rng** (`RandomState`)

seq_update(*x, weights, estimate*)

Add an encoded (*n*, *p*) matrix with weights of shape (*n*,).

Return type

`None`

Parameters

- **x** (`VonMisesFisherEncodedDataSequence`)
- **weights** (`ndarray`)
- **estimate** (`VonMisesFisherDistribution` / `None`)

update(*x, weight, estimate*)

Add one weighted direction vector.

Return type

`None`

Parameters

- **x** (`Sequence[float]` / `ndarray`)
- **weight** (`float`)
- **estimate** (`VonMisesFisherDistribution` / `None`)

value()

Return the (*count*, *vector_sum*) sufficient statistic.

Return type

`typing.Tuple[float, typing.Optional[numpy.ndarray]]`

class `dmx.stats.vmf.VonMisesFisherAccumulatorFactory`(*dim=None, name=None, keys=None*)

Bases: `StatisticAccumulatorFactory`

Factory class for creating `VonMisesFisherAccumulator` instances.

Parameters

- **dim** (`int` / `None`)
- **name** (`str` / `None`)
- **keys** (`str` / `None`)

dim

Dimension of the data.

Type

`Optional[int]`

name

Name for the factory instance.

Type

`Optional[str]`

keys

Key for the factory instance.

Type

Optional[str]

make()

Create a new von Mises–Fisher accumulator.

Return type

dmx.stats.pdist.SequenceEncodableStatisticAccumulator

Returns

SequenceEncodableStatisticAccumulator – A new accumulator instance.

class dmx.stats.vmf.VonMisesFisherDataEncoder

Bases: *DataSequenceEncoder*

Encode direction vectors as an observation matrix of shape (n, p).

seq_encode(x)

Encode a sequence of direction vectors.

Parameters

x (*Union[Sequence[float], np.ndarray]*) – Input data sequence.

Return type

dmx.stats.vmf.VonMisesFisherEncodedDataSequence

Returns

VonMisesFisherEncodedDataSequence – Encoded data sequence.

Parameters

x (*Sequence[float] | ndarray*)

class dmx.stats.vmf.VonMisesFisherDistribution(mu, kappa, name=None, keys=None)

Bases: *SequenceEncodableProbabilityDistribution*

Represent a von Mises–Fisher distribution on the unit sphere.

For unit vectors **x** and **mu** in $\mathbb{R}^p$, the log density is

$$\log p(x \mid \mu, \kappa) = \log C_p(\kappa) + \kappa \mu^\top x.$$

kappa is the nonnegative concentration: zero gives the uniform density and larger values concentrate mass around **mu**. The constructor records, but does not normalize or validate, the supplied direction and concentration.

Parameters

- **mu** (*Sequence[float] | ndarray*)
- **kappa** (*float*)
- **name** (*str | None*)
- **keys** (*str | None*)

name

Name for the distribution instance.

Type

Optional[str]

dim

Dimension of the mean direction vector.

Type

int

mu

Mean direction vector (norm should be 1.0).

Type

np.ndarray

kappa

Positive concentration parameter.

Type

float

log_const

Normalizing constant for the vmf distribution.

Type

float

keys

Optional keys for the distribution instance.

Type

Optional[str]

density(*x*)

Evaluate the density at a unit vector of shape (*p*,).

Return type

float

Parameters

x (*Sequence[float] | ndarray*)

dist_to_encoder()

Create an encoder for directional vectors.

Return type

dmx.stats.vmf.VonMisesFisherDataEncoder

estimator(*pseudo_count=None*)

Create an unregularized estimator.

pseudo_count is accepted for protocol compatibility but is not used.

Return type

dmx.stats.vmf.VonMisesFisherEstimator

Parameters

pseudo_count (*float | None*)

log_density(*x*)

Evaluate the log density at a unit vector of shape (*p*,).

Return type

float

Parameters**x** (*Sequence[float] | ndarray*)**sampler** (*seed=None*)

Create a sampler, optionally initialized with seed.

Return type*dmx.stats.vmf.VonMisesFisherSampler***Parameters****seed** (*int | None*)**seq_log_density** (*x*)Evaluate log densities for an encoded (*n*, *p*) matrix.**Return type***numpy.ndarray***Parameters****x** (*VonMisesFisherEncodedDataSequence*)**class** *dmx.stats.vmf.VonMisesFisherEncodedDataSequence* (*data*)Bases: *EncodedDataSequence*Store an encoded directional matrix of shape (*n*, *p*).**Parameters****data** (*ndarray*)**data**

Encoded data array.

Type*np.ndarray***class** *dmx.stats.vmf.VonMisesFisherEstimator* (*dim=None, pseudo_count=None, name=None, keys=None*)Bases: *ParameterEstimator*

Estimate direction and concentration by weighted maximum likelihood.

Estimation normalizes the weighted vector sum for the mean direction and solves the standard mean-resultant-length equation for concentration. *pseudo_count* is stored for protocol compatibility but is not applied.**Parameters**

- **dim** (*int | None*)
- **pseudo_count** (*float | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

dim

Dimension of the data.

Type*Optional[int]***pseudo_count**

Pseudo count for estimation.

Type

Optional[float]

name

Name for the estimator instance.

Type

Optional[str]

keys

Key for the estimator instance.

Type

Optional[str]

accumulator_factory()

Create a matching accumulator factory.

Return type*dmx.stats.vmf.VonMisesFisherAccumulatorFactory***Returns***VonMisesFisherAccumulatorFactory* – A factory instance.**estimate(nobs, suff_stat)**

Estimate parameters from (count, vector_sum).

nobs is accepted for protocol compatibility. A zero resultant or nonpositive count yields the uniform distribution with a fixed default direction.

Parameters

- **nobs** (*Optional[float]*) – Number of observations.
- **suff_stat** (*Tuple[float, np.ndarray]*) – Sufficient statistics for estimation.

Return type*dmx.stats.vmf.VonMisesFisherDistribution***Returns***VonMisesFisherDistribution* – Estimated vmf distribution.**Parameters**

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[float, ndarray | None]*)

class *dmx.stats.vmf.VonMisesFisherSampler*(*dist, seed=None*)Bases: *DistributionSampler*

Sampler for the von Mises-Fisher distribution.

Parameters

- **dist** (*VonMisesFisherDistribution*)
- **seed** (*int | None*)

rng

Random number generator.

Type

RandomState

dist

The vmf distribution instance.

Type

VonMisesFisherDistribution

sample(*size=None*)

Generate samples from the von Mises–Fisher distribution.

Parameters

size (*Optional[int]*) – Number of samples to generate. If None, generates a single sample.

Return type

`numpy.ndarray`

Returns

np.ndarray – Generated samples.

Parameters

size (*int | None*)

dmx.stats.vmf.lniv(*v, ln_z*)

Compute the log modified Bessel function of the first kind.

Parameters

- **v** (*float*) – Order of the Bessel function.
- **ln_z** (*float*) – Logarithm of z.

Return type

`float`

Returns

float – Logarithm of the modified Bessel function.

Parameters

- **v** (*float*)
- **ln_z** (*float*)

dmx.stats.vmf.lniv_h(*v, ln_z*)

Compute the modified Bessel approximation for small z.

Parameters

- **v** (*float*) – Order of the Bessel function.
- **ln_z** (*float*) – Logarithm of z.

Return type

`float`

Returns

float – Approximation of the modified Bessel function.

Parameters

- **v** (*float*)
- **ln_z** (*float*)

`dmx.stats.vmf.lniv_z(v, ln_z)`

Compute the modified Bessel approximation for large z .

Parameters

- `v (float)` – Order of the Bessel function.
- `ln_z (float)` – Logarithm of z .

Return type

`float`

Returns

`float` – Approximation of the modified Bessel function.

Parameters

- `v (float)`
- `ln_z (float)`

Weighted Distribution

Provide a wrapper for observations carrying multiplicative weights.

This Distribution simply allows from weights on observations. I.e. Data type D is observed and an associated score/weight is assigned to the data. This simply passes the weights and data downstream in aggregation.

Likelihood evals are equivalent to normal likelihood calls to the base distribution.

class `dmx.stats.weighted.WeightedAccumulator(accumulator, keys=None, name=None)`

Bases: [`SequenceEncodableStatisticAccumulator`](#)

Delegate statistics using external weight times embedded weight.

The wrapper adds no statistic of its own: `value`, `combine`, and `from_value` use the child's statistic directly. A wrapper key shares that child statistic but does not recurse through the child's own key contract.

Parameters

- **accumulator** ([`SequenceEncodableStatisticAccumulator`](#))
- **keys** (`str` | `None`)
- **name** (`str` | `None`)

accumulator

Accumulator for base distribution.

Type

[`SequenceEncodableStatisticAccumulator`](#)

keys

Key for sufficient statistics of base distribution.

Type

`Optional[str]`

name

Optional name for distribution.

Type

`Optional[str]`

acc_to_encoder()

Wrap the child accumulator's encoder.

Return type

dmx.stats.weighted.WeightedDataEncoder

combine(suff_stat)

Combine a child sufficient statistic and return this wrapper.

Return type

dmx.stats.weighted.WeightedAccumulator

Parameters

suff_stat (SS)

from_value(x)

Restore the child sufficient statistic and return this wrapper.

Return type

dmx.stats.weighted.WeightedAccumulator

Parameters

x (SS)

initialize(x, weight, rng)

Initialize the child using the product of both weights.

Return type

None

Parameters

- **x** (*Tuple*[*T*, *float*])
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(stats_dict)

Merge the child statistic under the wrapper key when configured.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

key_replace(stats_dict)

Replace the child statistic from the wrapper key when available.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

seq_initialize(x, weights, rng)

Initialize encoded child values using elementwise weight products.

Return type

None

Parameters

- **x** (`WeightedEncodedDataSequence`)
- **weights** (`ndarray`)
- **rng** (`RandomState`)

seq_update(*x, weights, estimate*)

Update encoded child values using elementwise weight products.

Return type

`None`

Parameters

- **x** (`WeightedEncodedDataSequence`)
- **weights** (`ndarray`)
- **estimate** (`WeightedDistribution`)

update(*x, weight, estimate*)

Update the child using the product of both weights.

Return type

`None`

Parameters

- **x** (`Tuple[T, float]`)
- **weight** (`float`)
- **estimate** (`WeightedDistribution`)

value()

Return the child sufficient statistic unchanged.

Return type

`typing.Any`

class `dmx.stats.weighted.WeightedAccumulatorFactory`(*factory, keys=None, name=None*)

Bases: `StatisticAccumulatorFactory`

`WeightedAccumulatorFactory` object for creating `WeightedAccumulator` objects.

Parameters

- **factory** (`StatisticAccumulatorFactory`)
- **keys** (`str` | `None`)
- **name** (`str` | `None`)

factory

Accumulator for base distribution.

Type

`StatisticAccumulatorFactory`

keys

Optional keys for base distribution.

Type

`Optional[str]`

name

Name for object.

Type

Optional[str]

make()

Create a weighted wrapper around a new child accumulator.

Return type

dmx.stats.weighted.WeightedAccumulator

class `dmx.stats.weighted.WeightedDataEncoder`(*encoder*)

Bases: *DataSequenceEncoder*

WeightedDataEncoder object for encoding iid sequences of WeightedDistribution.

Parameters

encoder (*DataSequenceEncoder*)

encoder

DataSequenceEncoder for the base distribution.

Type

DataSequenceEncoder

seq_encode(x)

Encode child values and store embedded weights as a float array.

Return type

dmx.stats.weighted.WeightedEncodedDataSequence

Parameters

x (*Sequence[Tuple[T, float]]*)

class `dmx.stats.weighted.WeightedDistribution`(*dist*, *name=None*, *keys=None*)

Bases: *SequenceEncodableProbabilityDistribution*

Scale a child's log likelihood by an observation's embedded weight.

Observations are (value, weight) pairs and scoring returns `weight * child.log_density(value)`. Thus the density is the child density raised to `weight`; it is an objective contribution and is not generally a normalized joint distribution over value-weight pairs. Child support is preserved for the value field, subject to the numeric embedded weight.

Encoding separates child values from a float weight vector. Accumulation multiplies the caller's weight by each embedded weight before delegating to the child. Sampling delegates directly to the child and therefore returns unweighted child values, not value-weight pairs.

Notes

Distribution acts only on the value for likelihood calls and treats weight as number of replicates.

Parameters

- **dist** (*SequenceEncodableProbabilityDistribution*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

dist

Distribution for values.

Type

SequenceEncodableProbabilityDistribution

name

Name for distribution.

Type

Optional[str]

keys

Keys for parameters of dist.

Type

Optional[str]

density(*x*)

Return the exponentiated weighted child log density.

Return type

float

Parameters

x (*Tuple*[*T*, *float*])

dist_to_encoder()

Create an encoder for child values and embedded weights.

Return type

dmx.stats.weighted.WeightedDataEncoder

estimator(*pseudo_count=None*)

Wrap the child's estimator, forwarding any pseudo-count.

Return type

dmx.stats.weighted.WeightedEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Multiply the child's log density by the embedded weight.

Return type

float

Parameters

x (*Tuple*[*T*, *float*])

sampler(*seed=None*)

Return the child sampler without adding embedded weights.

Return type

dmx.stats.pdist.DistributionSampler

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Multiply vectorized child log densities by encoded weights.

Raises

TypeError – If *x* is not a weighted encoded sequence.

Return type

`numpy.ndarray`

Parameters

x (`WeightedEncodedDataSequence`)

class `dmx.stats.weighted.WeightedEncodedDataSequence(data)`

Bases: `EncodedDataSequence`

WeightedEncodedDataSequence object for vectorized calls.

Parameters

data (`Tuple[EncodedDataSequence, ndarray]`)

data

EncodedDataSequence for base distribution and array of counts.

Type

`Tuple[EncodedDataSequence, np.ndarray]`

class `dmx.stats.weighted.WeightedEstimator(estimator, keys=None, name=None)`

Bases: `ParameterEstimator`

Fit a child from already weighted sufficient statistics.

The wrapper passes *nobs* through unchanged to the child estimator; embedded weights affect the accumulated child statistic but do not independently replace that argument. The fitted child is returned inside a new weighted wrapper.

Parameters

- **estimator** (`ParameterEstimator`)
- **keys** (`str` | `None`)
- **name** (`str` | `None`)

estimator

Estimator for the base distribution.

Type

`ParameterEstimator`

keys

Keys for the base distribution.

Type

`Optional[str]`

name

Optional name for object.

Type

`Optional[str]`

accumulator_factory()

Create a weighted factory around the child's factory.

Return type

dmx.stats.weighted.WeightedAccumulatorFactory

estimate(nobs, suff_stat)

Fit the child with the supplied count and child statistic.

Return type

dmx.stats.weighted.WeightedDistribution

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*SS*)

1.3.4 Mixture and topic models

Diagonal Gaussian Mixture

Create, estimate, and sample diagonal multivariate normal mixtures.

For $Z \in \{0, \dots, K-1\}$, weights $\pi_k = P(Z = k)$, means μ_k , and diagonal covariance matrices $\text{diag}(\mathbf{v}_k)$, this module implements

$$f(\mathbf{x}) = \sum_{k=0}^{K-1} \pi_k \mathcal{N}(\mathbf{x}; \mu_k, \text{diag}(\mathbf{v}_k)).$$

The public classes cover distribution evaluation, posterior responsibilities, sampling, sufficient-statistic accumulation, estimation, and sequence encoding.

```
class dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator(num_components, dim,  
                                                                tied=False, keys=(None, None),  
                                                                name=None)
```

Bases: *SequenceEncodableStatisticAccumulator*

Accumulate sufficient statistics for a diagonal-MVN mixture.

Parameters

- **num_components** (*int*)
- **dim** (*int*)
- **tied** (*bool*)
- **keys** (*Tuple[str | None, str | None]*)
- **name** (*str* | *None*)

num_components

Number of mixture components.

Type

int

dim

Dimensionality of the data.

Type

int

tied

If True, the covariance of each mixture component is tied.

Type

bool

keys

Set keys for weights and parameters.

Type

Tuple[Optional[str], Optional[str]]

name

Name for object.

Type

Optional[str]

acc_to_encoder()

Return a DiagonalGaussianMixtureDataEncoder for this accumulator.

Return type

dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDataEncoder

Returns

DiagonalGaussianMixtureDataEncoder – Encoder object.

combine(suff_stat)

Combine another accumulator's sufficient statistics into this one.

Parameters

suff_stat (*SS*) – Sufficient statistics to combine.

Return type

dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator

Returns

DiagonalGaussianMixtureAccumulator – Self after combining.

Parameters

suff_stat (*Tuple[ndarray, ndarray, ndarray, ndarray, ndarray]*)

from_value(x)

Set the sufficient statistics from a tuple.

Parameters

x (*SS*) – Sufficient statistics.

Return type

dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator

Returns

DiagonalGaussianMixtureAccumulator – Self after setting values.

Parameters

x (*Tuple[ndarray, ndarray, ndarray, ndarray, ndarray]*)

initialize(x, weight, rng)

Randomly initialize statistics from one weighted observation.

A Dirichlet draw splits a nonzero weight across the K latent components.

Parameters

- **x** (*Union[List[float], np.ndarray]*) – Observation.
- **weight** (*float*) – Weight for the observation.
- **rng** (*np.random.RandomState*) – Random number generator.

Return type

None

Parameters

- **x** (*List[float] | ndarray*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge this accumulator into a dictionary by key.

Parameters

stats_dict (*Dict[str, Any]*) – Dictionary of accumulators.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

key_replace(*stats_dict*)

Replace this accumulator's values with those from a dictionary by key.

Parameters

stats_dict (*Dict[str, Any]*) – Dictionary of accumulators.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

seq_initialize(*x, weights, rng*)

Randomly initialize statistics from an encoded iid sequence.

Each positive observation weight is independently split across components by a symmetric Dirichlet draw; nonpositive weights receive zero allocation.

Parameters

- **x** (*DiagonalGaussianMixtureEncodedDataSequence*) – Encoded data sequence.
- **weights** (*np.ndarray*) – Observation weights with shape (N,).
- **rng** (*np.random.RandomState*) – Random number generator.

Return type

None

Parameters

- **x** (*DiagonalGaussianMixtureEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Vectorized update for encoded data.

Parameters

- **x** (*DiagonalGaussianMixtureEncodedDataSequence*) – Encoded data sequence.
- **weights** (*np.ndarray*) – Weights for each observation.
- **estimate** (*DiagonalGaussianMixtureDistribution*) – Distribution estimate for update.

Return type

None

Parameters

- **x** (*DiagonalGaussianMixtureEncodedDataSequence*)
- **weights** (*ndarray*)
- **estimate** (*DiagonalGaussianMixtureDistribution*)

update(*x*, *weight*, *estimate*)

Update accumulator with a new observation.

Parameters

- **x** (*Union[List[float], np.ndarray]*) – Observation.
- **weight** (*float*) – Weight for the observation.
- **estimate** (*DiagonalGaussianMixtureDistribution*) – Distribution estimate for update.

Return type

None

Parameters

- **x** (*List[float] | ndarray*)
- **weight** (*float*)
- **estimate** (*DiagonalGaussianMixtureDistribution*)

value()

Return the sufficient statistics as a tuple.

Return type

typing.Tuple[*numpy.ndarray*, *numpy.ndarray*, *numpy.ndarray*, *numpy.ndarray*, *numpy.ndarray*]

Returns

SS – Tuple of sufficient statistics.

```
class dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulatorFactory(num_components, dim,
                                                                    keys=(None, None),
                                                                    name=None,
                                                                    tied=False)
```

Bases: *StatisticAccumulatorFactory*

Factory for creating accumulators for diagonal Gaussian mixture distributions.

Parameters

- **num_components** (*int*)

- **dim** (*int*)
- **keys** (*Tuple[str | None, str | None]*)
- **name** (*str | None*)
- **tied** (*bool*)

num_components

Number of mixture components.

Type

int

dim

Dimensionality of the data.

Type

int

keys

Set keys for weights and parameters.

Type

Tuple[Optional[str], Optional[str]]

name

Name for object.

Type

Optional[str]

tied

If True, the covariance of each mixture component is tied.

Type

bool

make()

Create a new *DiagonalGaussianMixtureAccumulator*.

Return type

dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator

Returns

DiagonalGaussianMixtureAccumulator – New accumulator instance.

class *dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDataEncoder*

Bases: *DataSequenceEncoder*

Encoder for data sequences in diagonal Gaussian mixture distributions.

seq_encode(*x*)

Encode a sequence of data points.

Parameters

x (*Union[Sequence[Sequence[float]], np.ndarray]*) – Sequence of data points.

Return type

dmx.stats.dmvn_mixture.DiagonalGaussianMixtureEncodedDataSequence

Returns

DiagonalGaussianMixtureEncodedDataSequence – Encoded data sequence.

Parameters**x** (*Sequence*[*Sequence*[*float*]] | *ndarray*)

```
class dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution(mu, covar, w, tied=False,
                                                                name=None, keys=(None,
                                                                None))
```

Bases: *SequenceEncodableProbabilityDistribution*

Represent a mixture of diagonal multivariate normal components.

$\mu[k]$ is μ_k , $\text{covar}[k]$ contains the diagonal variances v_k , and $w[k]$ is π_k . Component log densities exclude mixture weights. Posterior methods return responsibilities $r_k(x) = P(Z = k \mid \mathbf{X} = x)$.

Parameters

- **mu** (*Sequence*[*Sequence*[*float*]] | *ndarray*)
- **covar** (*Sequence*[*float*] | *Sequence*[*Sequence*[*float*]] | *ndarray*)
- **w** (*ndarray* | *List*[*float*])
- **tied** (*bool*)
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*])

mu

Means of the mixture components (K x D).

Type

np.ndarray

covar

Covariances of the mixture components (K x D).

Type

np.ndarray

w

Mixture weights (length K).

Type

np.ndarray

tied

If True, the covariance of each mixture component is tied.

Type

bool

name

Name for object.

Type

Optional[str]

keys

Set keys for weights and parameters.

Type

Tuple[Optional[str], Optional[str]]

dim

Dimensionality of the data.

Type

int

num_components

Number of mixture components.

Type

int

zw

Boolean mask for zero-weight components.

Type

np.ndarray

log_w

Logarithm of mixture weights.

Type

np.ndarray

log_c

Constant term in log-density.

Type

float

component_log_density(*x*)

Compute unweighted component log densities for one observation.

Parameters

x (*Union[Sequence[float], np.ndarray]*) – Input data point.

Return type

numpy.ndarray

Returns

np.ndarray – Component log densities with shape $(K,)$.

Parameters

x (*Sequence[float] | ndarray*)

density(*x*)

Compute the density of the distribution at a given point.

Parameters

x (*Union[Sequence[float], np.ndarray]*) – Input data point.

Return type

float

Returns

float – Density value.

Parameters

x (*Sequence[float] | ndarray*)

dist_to_encoder()

Return a `DiagonalGaussianMixtureDataEncoder` for this distribution.

Return type

`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDataEncoder`

Returns

`DiagonalGaussianMixtureDataEncoder` – Encoder object.

estimator(pseudo_count=None)

Return a `DiagonalGaussianMixtureEstimator` for this distribution.

Parameters

pseudo_count (*Optional[float], optional*) – Scalar copied into the estimator’s weight, mean, and covariance pseudo-count slots. This convenience method does not supply prior mean or covariance sufficient statistics, so those slots only affect later estimation if prior statistics are supplied to the returned estimator. The weight slot smooths mixture weights toward uniform weights when no weight prior is supplied.

Return type

`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureEstimator`

Returns

`DiagonalGaussianMixtureEstimator` – Estimator object.

Parameters

pseudo_count (*float | None*)

fast_seq_posterior(x)

Compute sequence responsibilities using the Numba kernel.

Parameters

x (*DiagonalGaussianMixtureEncodedDataSequence*) – Encoded data sequence containing input data points.

Return type

`numpy.ndarray`

Returns

`np.ndarray` – Posterior probabilities with shape (N, K), where N is the number of samples and K is the number of mixture components.

Parameters

x (*DiagonalGaussianMixtureEncodedDataSequence*)

log_density(x)

Compute the mixture log density at one vector observation.

Parameters

x (*Union[Sequence[float], np.ndarray]*) – Input data point.

Return type

`float`

Returns

`float` – Log density value.

Parameters

x (*Sequence[float] | ndarray*)

posterior(x)

Compute posterior responsibilities for one vector observation.

Parameters

x (*Union[Sequence[float], np.ndarray]*) – Input data point.

Return type

`numpy.ndarray`

Returns

np.ndarray – Responsibilities with shape (K,).

Parameters

x (*Sequence[float] | ndarray*)

sampler(seed=None)

Return a `DiagonalGaussianMixtureSampler` for this distribution.

Parameters

seed (*Optional[int], optional*) – Seed for random number generator.

Return type

dmx.stats.dmvn_mixture.DiagonalGaussianMixtureSampler

Returns

DiagonalGaussianMixtureSampler – Sampler object.

Parameters

seed (*int | None*)

seq_component_log_density(x)

Compute component log densities for an encoded iid sequence.

Parameters

x (*DiagonalGaussianMixtureEncodedDataSequence*) – Encoded data sequence.

Return type

`numpy.ndarray`

Returns

np.ndarray – Unweighted component log densities with shape (N, K).

Parameters

x (*DiagonalGaussianMixtureEncodedDataSequence*)

seq_log_density(x)

Compute mixture log densities for an encoded iid sequence.

Parameters

x (*DiagonalGaussianMixtureEncodedDataSequence*) – Encoded data sequence.

Return type

`numpy.ndarray`

Returns

np.ndarray – Mixture log densities with shape (N,).

Parameters

x (*DiagonalGaussianMixtureEncodedDataSequence*)

seq_posterior(x)

Compute posterior responsibilities for an encoded iid sequence.

Parameters

x (*DiagonalGaussianMixtureEncodedDataSequence*) – Encoded data sequence containing input data points.

Return type

`numpy.ndarray`

Returns

`np.ndarray` – Responsibilities with shape (N, K).

Parameters

x (*DiagonalGaussianMixtureEncodedDataSequence*)

class `dmx.stats.dmvn_mixture.DiagonalGaussianMixtureEncodedDataSequence(data)`

Bases: *EncodedDataSequence*

Encoded data sequence for diagonal Gaussian mixture distributions.

Parameters

data (*ndarray*)

data

Encoded data points.

Type

`np.ndarray`

class `dmx.stats.dmvn_mixture.DiagonalGaussianMixtureEstimator(num_components, dim, fixed_weights=None, suff_stat=(None, None, None), pseudo_count=(None, None, None), tied=False, keys=(None, None), name=None)`

Bases: *ParameterEstimator*

Estimator for diagonal Gaussian mixture distributions.

Notes

`pseudo_count` and `suff_stat` are three-tuples ordered as (weights, means, diagonal_covariances). Weight pseudo-counts smooth toward uniform mixture weights when `suff_stat[0]` is `None` and shrink toward `suff_stat[0]` when it is provided. Mean and diagonal covariance pseudo-counts only take effect when the matching prior targets are provided.

`suff_stat[1]` and `suff_stat[2]` are reshaped to (num_components, dim) and should be prior targets for component means and diagonal covariance values. Use them for regularized initialization when component locations or scales are known. `fixed_weights` bypasses the outer weight update, but does not disable mean or covariance regularization.

Parameters

- **num_components** (*int*)
- **dim** (*int*)
- **fixed_weights** (*List[float] | ndarray | None*)
- **suff_stat** (*Tuple[ndarray | None, ndarray | None, ndarray | None]*)
- **pseudo_count** (*Tuple[float | None, float | None, float | None]*)
- **tied** (*bool*)

- **keys** (*Tuple[str | None, str | None]*)
- **name** (*str | None*)

num_components

Number of mixture components.

Type

int

dim

Dimensionality of the data.

Type

int

fixed_weights

Fixed mixture weights, if provided.

Type

Optional[Union[List[float], np.ndarray]]

suff_stat (*Tuple[Optional[np.ndarray], Optional[np.ndarray],*

Optional[np.ndarray]]): Prior targets for weights, means, and diagonal covariances.

pseudo_count

Pseudo counts for weights, means, and diagonal covariances, respectively.

Type

Tuple[Optional[float], Optional[float], Optional[float]]

tied

If True, the covariance of each mixture component is tied.

Type

bool

keys

Set keys for weights and parameters.

Type

Tuple[Optional[str], Optional[str]]

name

Name for object.

Type

Optional[str]

accumulator_factory()

Return a DiagonalGaussianMixtureAccumulatorFactory for this estimator.

Return type

dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulatorFactory

Returns

DiagonalGaussianMixtureAccumulatorFactory – Factory object.

estimate (*nobs, suff_stat*)

Estimate a DiagonalGaussianMixtureDistribution from sufficient statistics.

Parameters

- **nobs** (*Optional[float]*) – Number of observations (not used).
- **suff_stat** (*SS*) – Tuple containing component counts, weighted vector sums, weighted squared vector sums, effective component counts, and auxiliary counts accumulated from data.

Return type*dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution***Returns***DiagonalGaussianMixtureDistribution* – Estimated distribution.**Parameters**

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[ndarray, ndarray, ndarray, ndarray, ndarray]*)

```
class dmx.stats.dmvn_mixture.DiagonalGaussianMixtureSampler(dist, seed=None)
```

Bases: *DistributionSampler*

Draw observations from a diagonal-MVN mixture.

Each draw first selects *Z* according to *dist.w* and then draws from the selected diagonal multivariate normal component.**Parameters**

- **dist** (*DiagonalGaussianMixtureDistribution*)
- **seed** (*int | None*)

dist

The distribution to sample from.

Type*DiagonalGaussianMixtureDistribution***rng**

Random number generator for sampling.

Type

RandomState

```
sample(size=None)
```

Draw iid vectors from the diagonal-MVN mixture.

Parameters

size (*Optional[int], optional*) – Number of samples to generate. If *None*, generates a single sample.

Return type*typing.Union[float, numpy.ndarray]***Returns***Union[float, np.ndarray]* –**An array with shape (1, D) when size is***None*; otherwise a list-like collection with shape (*size*, *D*).**Parameters****size** (*int | None*)

`dmx.stats.dmvn_mixture.fast_seq_posterior(x, mu, covar, log_w, zw, out)`

Compute sequence responsibilities using Numba.

Parameters

- **x** (*np.ndarray*) – Input data points (N x D).
- **mu** (*np.ndarray*) – Means of the mixture components (K x D).
- **covar** (*np.ndarray*) – Covariances of the mixture components (K x D).
- **log_w** (*np.ndarray*) – Log mixture weights (length K).
- **zw** (*np.ndarray*) – Boolean array indicating zero-weight components.
- **out** (*np.ndarray*) – Output array for posterior probabilities (N x K).

Return type

None

Parameters

- **x** (*ndarray*)
- **mu** (*ndarray*)
- **covar** (*ndarray*)
- **log_w** (*ndarray*)
- **zw** (*ndarray*)
- **out** (*ndarray*)

Gaussian Mixture

Create, estimate, and sample univariate Gaussian mixture models.

For $Z \in \{0, \dots, K-1\}$, weights $\pi_k = P(Z = k)$, means μ_k , and variances σ_k^2 , this module implements

$$f(x) = \sum_{k=0}^{K-1} \pi_k \mathcal{N}(x; \mu_k, \sigma_k^2).$$

Unlike a generic [MixtureDistribution](#) of Gaussian components, this implementation can estimate one variance shared across components while retaining component-specific means.

```
class dmx.stats.gmm.GaussianMixtureAccumulator(num_components, tied=False, keys=(None, None),
                                                name=None)
```

Bases: [SequenceEncodableStatisticAccumulator](#)

Accumulator for sufficient statistics of the Gaussian mixture distribution.

Parameters

- **num_components** (*int*)
- **tied** (*bool*)
- **keys** (*Tuple[str | None, str | None]*)
- **name** (*str | None*)

num_components

Number of mixture components.

Type

int

tied

If variance is the same across all mixture components.

Type

bool

comp_counts

Sufficient statistics for component counts.

Type

np.ndarray

xw

Sufficient statistics for means.

Type

np.ndarray

x2w

Sufficient statistics for variances.

Type

np.ndarray

wcnt

Sufficient statistics for means.

Type

np.ndarray

wcnt2

Sufficient statistics for variances.

Type

np.ndarray

weight_key

Key for weights.

Type

Optional[str]

comp_key

Key for components.

Type

Optional[str]

name

Name for the accumulator.

Type

Optional[str]

keys

Keys for weights and parameters.

Type

`Tuple[Optional[str], Optional[str]]`

acc_to_encoder()

Return a GaussianMixtureDataEncoder for this accumulator.

Return type

`dmx.stats.gmm.GaussianMixtureDataEncoder`

Returns

GaussianMixtureDataEncoder – Encoder object.

combine(suff_stat)

Aggregate sufficient statistics with this accumulator.

Parameters

suff_stat (*SS*) – Sufficient statistics to combine.

Return type

`dmx.stats.gmm.GaussianMixtureAccumulator`

Returns

GaussianMixtureAccumulator – Self after combining.

Parameters

suff_stat (`Tuple[ndarray, ndarray, ndarray, ndarray, ndarray]`)

from_value(x)

Set the sufficient statistics from a tuple.

Parameters

x (*SS*) – Sufficient statistics.

Return type

`dmx.stats.gmm.GaussianMixtureAccumulator`

Returns

GaussianMixtureAccumulator – Self after setting values.

Parameters

x (`Tuple[ndarray, ndarray, ndarray, ndarray, ndarray]`)

initialize(x, weight, rng)

Randomly initialize statistics from one weighted observation.

A Dirichlet draw splits a nonzero weight across the *K* latent components.

Parameters

- **x** (*float*) – Observation.
- **weight** (*float*) – Weight for the observation.
- **rng** (*RandomState*) – Random number generator.

Return type

`None`

Parameters

- **x** (*float*)

- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge this accumulator into a dictionary by key.

Parameters

stats_dict (*Dict[str, Any]*) – Dictionary of accumulators.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

key_replace(*stats_dict*)

Replace this accumulator's values with those from a dictionary by key.

Parameters

stats_dict (*Dict[str, Any]*) – Dictionary of accumulators.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

seq_initialize(*x, weights, rng*)

Randomly initialize statistics from an encoded iid sequence.

Each positive observation weight is independently split across components by a symmetric Dirichlet draw; nonpositive weights receive zero allocation.

Parameters

- **x** (*GaussianMixtureEncodedDataSequence*) – Encoded data sequence.
- **weights** (*np.ndarray*) – Observation weights with shape (N,).
- **rng** (*RandomState*) – Random number generator.

Return type

None

Parameters

- **x** (*GaussianMixtureEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x, weights, estimate*)

Vectorized update for encoded data.

Parameters

- **x** (*GaussianMixtureEncodedDataSequence*) – Encoded data sequence.
- **weights** (*np.ndarray*) – Weights for each observation.
- **estimate** (*GaussianMixtureDistribution*) – Distribution for posterior calculation.

Return type

None

Parameters

- **x** (`GaussianMixtureEncodedDataSequence`)
- **weights** (`ndarray`)
- **estimate** (`GaussianMixtureDistribution`)

update(*x, weight, estimate*)

Update accumulator with a new observation.

Parameters

- **x** (*float*) – Observation.
- **weight** (*float*) – Weight for the observation.
- **estimate** (`GaussianMixtureDistribution`) – Distribution for posterior calculation.

Return type

None

Parameters

- **x** (*float*)
- **weight** (*float*)
- **estimate** (`GaussianMixtureDistribution`)

value()

Return the sufficient statistics as a tuple.

Return type`typing.Tuple[numpy.ndarray, numpy.ndarray, numpy.ndarray, numpy.ndarray, numpy.ndarray]`**Returns**

SS – Tuple of sufficient statistics.

```
class dmx.stats.gmm.GaussianMixtureAccumulatorFactory(num_components, keys=(None, None),  
                                                    name=None, tied=False)
```

Bases: `StatisticAccumulatorFactory`Factory for creating `GaussianMixtureAccumulator` objects.**Parameters**

- **num_components** (*int*)
- **keys** (`Tuple[str | None, str | None]`)
- **name** (*str | None*)
- **tied** (*bool*)

num_components

Number of mixture components.

Type`int`**keys**

Keys for weights and parameters.

Type

Tuple[Optional[str], Optional[str]]

tied

If variance is the same across all mixture components.

Type

bool

name

Name for the factory.

Type

Optional[str]

make()

Create a new GaussianMixtureAccumulator.

Return type

dmx.stats.gmm.GaussianMixtureAccumulator

Returns

GaussianMixtureAccumulator – New accumulator instance.

class dmx.stats.gmm.GaussianMixtureDataEncoder

Bases: *DataSequenceEncoder*

Encoder for sequences of iid Gaussian mixture observations.

seq_encode(x)

Encode a sequence of Gaussian mixture observations.

Parameters

x (*Union[Sequence[float], np.ndarray]*) – Sequence of observations.

Return type

dmx.stats.gmm.GaussianMixtureEncodedDataSequence

Returns

GaussianMixtureEncodedDataSequence – Encoded data sequence.

Parameters

x (*Sequence[float] | ndarray*)

class dmx.stats.gmm.GaussianMixtureDistribution(mu, sigma2, w, name=None, keys=(None, None))

Bases: *SequenceEncodableProbabilityDistribution*

Represent a mixture of univariate Gaussian distributions.

mu[k] is μ_k , sigma2[k] is σ_k^2 , and w[k] is π_k . A scalar sigma2 gives every component the same variance. Component log densities exclude mixture weights, while posterior methods return responsibilities $r_k(x) = P(Z = k | X = x)$.

Parameters

- **mu** (*Sequence[float] | ndarray*)
- **sigma2** (*Sequence[float] | ndarray | float*)
- **w** (*ndarray | List[float]*)
- **name** (*str | None*)
- **keys** (*Tuple[str | None, str | None]*)

mu

Means of each mixture component.

Type

np.ndarray

sigma2

Variance for each Gaussian.

Type

Union[float, np.ndarray]

w

Weights for each mixture component.

Type

np.ndarray

zw

Indicator for zero weights.

Type

np.ndarray

log_w

Log of mixture weights.

Type

np.ndarray

log_c

Constant for Gaussian distributions.

Type

Union[float, np.ndarray]

num_components

Number of mixture components.

Type

int

_tied

True if sigma2 is the same across all components.

Type

bool

name

Name for the object.

Type

Optional[str]

keys

Keys for weights and parameters.

Type

Tuple[Optional[str], Optional[str]]

component_log_density(*x*)

Evaluate unweighted component log densities at one observation.

Parameters

x (*float*) – Observation.

Return type

`numpy.ndarray`

Returns

np.ndarray – Component log densities with shape (1, K).

Parameters

x (*float*)

density(*x*)

Evaluate the density of the mixture at x.

Parameters

x (*float*) – Observation.

Return type

`float`

Returns

float – Density at x.

Parameters

x (*float*)

dist_to_encoder()

Return a GaussianMixtureDataEncoder for this distribution.

Return type

`dmx.stats.gmm.GaussianMixtureDataEncoder`

Returns

GaussianMixtureDataEncoder – Encoder object.

estimator(*pseudo_count=None*)

Return a GaussianMixtureEstimator for this distribution.

Parameters

pseudo_count (*Optional[float], optional*) – Scalar copied into the estimator’s weight, mean, and variance pseudo-count slots. This convenience method does not supply prior mean or variance sufficient statistics, so those slots only affect later estimation if prior statistics are supplied to the returned estimator. The weight slot smooths mixture weights toward uniform weights when no weight prior is supplied.

Return type

`dmx.stats.gmm.GaussianMixtureEstimator`

Returns

GaussianMixtureEstimator – Estimator object.

Parameters

pseudo_count (*float | None*)

log_density(*x*)

Evaluate the mixture log density at one scalar observation.

Parameters

x (*float*) – Observation.

Return type

float

Returns

float – Log-density at x.

Parameters

x (*float*)

posterior(x)

Compute posterior responsibilities for one observation.

Parameters

x (*float*) – Observation.

Return type

numpy.ndarray

Returns

np.ndarray – Responsibilities with shape (1, K).

Parameters

x (*float*)

sampler(seed=None)

Return a GaussianMixtureSampler for this distribution.

Parameters

seed (*Optional[int], optional*) – Seed for random number generator.

Return type

dmx.stats.gmm.GaussianMixtureSampler

Returns

GaussianMixtureSampler – Sampler object.

Parameters

seed (*int | None*)

seq_component_log_density(x)

Evaluate component log-density terms for an encoded iid sequence.

Parameters

x (*GaussianMixtureEncodedDataSequence*) – Encoded data sequence.

Return type

numpy.ndarray

Returns

np.ndarray –

Component matrix with shape (N, K). Zero-weight components are masked with negative infinity.

Parameters

x (*GaussianMixtureEncodedDataSequence*)

seq_log_density(x)

Evaluate mixture log densities for an encoded iid sequence.

Parameters

x (*GaussianMixtureEncodedDataSequence*) – Encoded data sequence.

Return type

`numpy.ndarray`

Returns

np.ndarray – Mixture log densities with shape $(N,)$.

Parameters

x (*GaussianMixtureEncodedDataSequence*)

seq_posterior(x)

Compute posterior responsibilities for an encoded iid sequence.

Parameters

x (*GaussianMixtureEncodedDataSequence*) – Encoded data sequence.

Return type

`numpy.ndarray`

Returns

np.ndarray – Responsibilities with shape (N, K) .

Parameters

x (*GaussianMixtureEncodedDataSequence*)

class `dmx.stats.gmm.GaussianMixtureEncodedDataSequence`(*data*)

Bases: *EncodedDataSequence*

Encoded data sequence for use with vectorized function calls.

Parameters

data (*ndarray*)

data

Sequence of iid Gaussian mixture observations.

Type

`np.ndarray`

class `dmx.stats.gmm.GaussianMixtureEstimator`(*num_components*, *fixed_weights=None*, *suff_stat=(None, None, None)*, *pseudo_count=(None, None, None)*, *tied=False*, *keys=(None, None)*, *name=None*)

Bases: *ParameterEstimator*

Estimator for GaussianMixtureDistribution from aggregated sufficient statistics.

Notes

Set equal variance with *tied* parameter.

pseudo_count and *suff_stat* are three-tuples ordered as (*weights*, *means*, *variances*). Each position controls a separate parameter update. Weight pseudo-counts smooth toward uniform mixture weights when *suff_stat*[0] is *None* and shrink toward *suff_stat*[0] when it is provided. Mean and variance pseudo-counts only take effect when their matching prior targets, *suff_stat*[1] and *suff_stat*[2], are provided.

Use the prior targets in *suff_stat* for regularized initialization when a reasonable starting mean or variance is known. Use *fixed_weights* when the outer mixture weights should not be estimated; component mean and variance regularization still follows the tuple slots above. With *tied=True*, the variance update pools variance mass across components.

Parameters

- **num_components** (*int*)
- **fixed_weights** (*List[float] | ndarray | None*)
- **suff_stat** (*Tuple[ndarray | None, ndarray | None, ndarray | None]*)
- **pseudo_count** (*Tuple[float | None, float | None, float | None]*)
- **tied** (*bool*)
- **keys** (*Tuple[str | None, str | None]*)
- **name** (*str | None*)

num_components

Number of mixture components.

Type

int

pseudo_count

Pseudo-counts for weights, means, and variances, respectively.

Type

Tuple[Optional[float], Optional[float], Optional[float]]

suff_stat (*Tuple[Optional[np.ndarray], Optional[np.ndarray],*

Optional[np.ndarray]]): Prior targets for weights, means, and variances. The mean and variance targets are not raw observation counts.

tied

If True, assume each Gaussian mixture has the same variance.

Type

bool

keys

Keys for weights and mixture components.

Type

Tuple[Optional[str], Optional[str]]

fixed_weights

If not None, weights of the mixture are assumed fixed.

Type

Optional[np.ndarray]

name

Name for the estimator.

Type

Optional[str]

accumulator_factory()

Return a GaussianMixtureAccumulatorFactory for this estimator.

Return type

dmx.stats.gmm.GaussianMixtureAccumulatorFactory

Returns

GaussianMixtureAccumulatorFactory – Factory object.

estimate(*nobs*, *suff_stat*)

Estimate a GaussianMixtureDistribution from sufficient statistics.

Parameters

- **nobs** (*Optional[float]*) – Number of observations (not used).
- **suff_stat** (*SS*) – Tuple containing component counts, weighted sums, weighted squared sums, effective component counts, and auxiliary counts accumulated from data.

Return type

dmx.stats.gmm.GaussianMixtureDistribution

Returns

GaussianMixtureDistribution – Estimated distribution.

Parameters

- **nobs** (*float | None*)
- **suff_stat** (*Tuple[ndarray, ndarray, ndarray, ndarray, ndarray]*)

class *dmx.stats.gmm.GaussianMixtureSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*

Draw observations from a univariate Gaussian mixture.

Each draw first selects *Z* according to *dist.w* and then draws from the selected univariate Gaussian component.

Parameters

- **dist** (*GaussianMixtureDistribution*)
- **seed** (*int | None*)

rng

Random number generator.

Type

RandomState

dist

Distribution to sample from.

Type

GaussianMixtureDistribution

_tied

True if variance is tied across components.

Type

bool

sample(*size=None*)

Draw iid samples from the Gaussian mixture distribution.

Parameters

size (*Optional[int], optional*) – Number of samples to draw. If *None*, returns a single sample.

Return type

typing.Union[float, typing.List[float]]

Returns

Union[float, List[float]] – Single sample or list of samples.

Parameters**size** (*int* | *None*)**Probabilistic Latent Semantic Indexing**

Provide probabilistic latent semantic indexing for integer tokens.

An observation is a document identifier and a sparse bag of token counts, $(d, [(v_0, c_0), \dots, (v_m, c_m)])$. Document and token identifiers must be zero-based integers. For topic z , the model parameters are the token probabilities $\phi_{vz} = P(v | z)$, document-topic probabilities $\theta_{dz} = P(z | d)$, and document probabilities $\pi_d = P(d)$. With $n = \sum_j c_j$, the implemented document mass is

$$P(d, x) = P_{\text{len}}(n) \pi_d \prod_j \left(\sum_z \phi_{v_j z} \theta_{dz} \right)^{c_j}.$$

The module supplies the distribution, sampler, sufficient-statistic accumulator, estimator, and vectorized data encoder. Estimation performs responsibility-weighted updates of the three probability tables; initialization instead assigns token counts to topics randomly from a symmetric Dirichlet draw.

```
class dmx.stats.int_plsi.IntegerPLSIAccumulator(num_vals, num_states, num_docs,  
                                              len_acc=NoneAccumulator(), name=None,  
                                              keys=(None, None, None))
```

Bases: [SequenceEncodableStatisticAccumulator](#)

Accumulate responsibility-weighted PLSI sufficient statistics.

Note: Keys in order, words/values, states, documents.

Parameters

- **num_vals** (*int*)
- **num_states** (*int*)
- **num_docs** (*int*)
- **len_acc** ([SequenceEncodableStatisticAccumulator](#) | *None*)
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*, *str* | *None*] | *None*)

num_vals

Number of words in the corpus.

Type

int

num_states

Number of word-topics or mixture components.

Type

int

num_docs

Number of authors (doc_ids) in the corpus.

Type

int

word_count

Numpy array of shape num_states by num_vals for aggregating state/word counts.

Type

ndarray

comp_count

Numpy array (num_docs by num_states) for aggregating doc/state counts.

Type

ndarray

doc_count

Numpy array for aggregating counts of authors (prior on doc_ids).

Type

ndarray

name

Name of object instance.

Type

Optional[str]

wc_key

Key for merging 'word_count' with objects containing matching keys.

Type

Optional[str]

sc_key

Key for merging 'comp_count' with objects containing matching keys.

Type

Optional[str]

dc_key

Key for merging 'doc_count' with objects containing matching keys.

Type

Optional[str]

len_acc

Accumulator object for the lengths of documents (total word counts). Defaults to the NullAccumulator if None is passed.

Type

SequenceEncodableStatisticAccumulator

_init_rng

True if RandomState objects for accumulator have been initialized.

Type

bool

_acc_rng

RandomState object for initializing the PLSI model.

Type

Optional[RandomState]

_len_rng

RandomState object for initializing the length accumulator.

Type

Optional[RandomState]

acc_to_encoder()

Return an IntegerPLSIDataEncoder object.

Return type

dmx.stats.int_plsi.IntegerPLSIDataEncoder

combine(suff_stat)

Add another accumulator's sufficient statistics in place.

Return type

dmx.stats.int_plsi.IntegerPLSIAccumulator

Parameters

suff_stat (Tuple[ndarray, ndarray, ndarray, SS1 | None])

from_value(x)

Replace the accumulated sufficient statistics and return this object.

Return type

dmx.stats.int_plsi.IntegerPLSIAccumulator

Parameters

x (Tuple[ndarray, ndarray, ndarray, SS1 | None])

initialize(x, weight, rng)

Initialize one document with random token-topic allocations.

A symmetric Dirichlet draw independently divides each distinct token count among topics. The supplied observation weight scales all resulting counts.

Return type

None

Parameters

- **x** (Tuple[int, Sequence[Tuple[int, float]]])
- **weight** (float)
- **rng** (RandomState)

key_merge(stats_dict)

Merge sufficient statistics into a keyed statistics dictionary.

Return type

None

Parameters

stats_dict (Dict[str, Any])

key_replace(stats_dict)

Replace sufficient statistics from matching dictionary keys.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**seq_initialize**(*x*, *weights*, *rng*)

Initialize encoded documents with random token-topic allocations.

Return type

None

Parameters

- **x** (*IntegerPLSEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Accumulate expected topic counts for encoded documents.

Return type

None

Parameters

- **x** (*IntegerPLSEncodedDataSequence*)
- **weights** (*ndarray*)
- **estimate** (*IntegerPLSIDistribution*)

update(*x*, *weight*, *estimate*)

Accumulate expected topic counts under a current PLSI estimate.

Return type

None

Parameters

- **x** (*Tuple[int, Sequence[Tuple[int, float]]]*)
- **weight** (*float*)
- **estimate** (*IntegerPLSIDistribution*)

value()

Return token-topic, document-topic, document, and length statistics.

Return type

typing.Tuple[numpy.ndarray, numpy.ndarray, numpy.ndarray, typing.Optional[typing.Any]]

```
class dmx.stats.int_plsi.IntegerPLSIAccumulatorFactory(num_vals, num_states, num_docs,
                                                    len_factory=NullAccumulatorFactory(),
                                                    keys=(None, None, None), name=None)
```

Bases: *StatisticAccumulatorFactory*

IntegerPLSIAccumulatorFactory object for creating IntegerPLSIAccumulator objects.

Parameters

- **num_vals** (*int*)
- **num_states** (*int*)
- **num_docs** (*int*)

- **len_factory** (*StatisticAccumulatorFactory* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*, *str* | *None*] | *None*)
- **name** (*str* | *None*)

num_vals

Number of words/values in PLSI.

Type

int

num_states

Number of states in PLSI.

Type

int

num_docs

Number of doc_ids (authors) in PLSI.

Type

int

len_factory

Accumulator factory object for length distribution. Defaults to the `NullAccumulatorFactory()`. Should have support on non-negative integers.

Type

StatisticsAccumulatorFactory

keys

Set keys for merging word, state, and doc sufficient statistics with matching keys.

Type

Tuple[*Optional*[*str*], *Optional*[*str*], *Optional*[*str*]]

name

Set name for object.

Type

Optional[*str*]

make()

Create a PLSI sufficient-statistic accumulator.

Return type

dmx.stats.int_plsi.IntegerPLSIAccumulator

class `dmx.stats.int_plsi.IntegerPLSIDataEncoder`(*len_encoder=NullDataEncoder()*)

Bases: *DataSequenceEncoder*

Encode sparse integer-token documents for vectorized PLSI operations.

Parameters

len_encoder (*DataSequenceEncoder* | *None*)

len_encoder

DataSequenceEncoder for the total number of words in each document, defaulting to `NullDataEncoder` if `None` is passed.

Type

DataSequenceEncoder

seq_encode(*x*)

Encode independent PLSI observations for vectorized operations.

Parameters

x – Observations represented as document identifiers paired with sparse (token, count) sequences.

Return type

dmx.stats.int_plsi.IntegerPLSEncodedDataSequence

Returns

Encoded lengths plus flattened token values, counts, document identifiers, observation indexes, and per-document totals.

Parameters

x (*Sequence*[*Tuple*[*int*, *Sequence*[*Tuple*[*int*, *float*]]]])

```
class dmx.stats.int_plsi.IntegerPLSIDistribution(state_word_mat, doc_state_mat, doc_vec,
                                             len_dist=NullDistribution(), name=None,
                                             keys=(None, None, None))
```

Bases: *SequenceEncodableProbabilityDistribution*

Represent an integer-token PLSI distribution.

Parameters

- **state_word_mat** (*List*[*List*[*float*]] | *ndarray*)
- **doc_state_mat** (*List*[*List*[*float*]] | *ndarray*)
- **doc_vec** (*List*[*float*] | *ndarray*)
- **len_dist** (*SequenceEncodableProbabilityDistribution* | *None*)
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*, *str* | *None*])

prob_mat

Token-topic probabilities $P(v \mid z)$ with shape (V, S); each topic column sums to one.

Type

np.ndarray

state_mat

Document-topic probabilities $P(z \mid d)$ with shape (D, S); each document row sums to one.

Type

np.ndarray

doc_vec

Document probabilities $P(d)$ with shape (D,).

Type

np.ndarray

log_doc_vec

1-d numpy array of the $\log(p_mat(doc=d))$.

Type

np.ndarray

num_vals

Number of total words in corpus. (Number of rows in prob_mat).

Type
int

num_states

Number of word topics (mixture components). (Number of columns in prob_mat/state_mat).

Type
int

num_docs

Total number of document ids in corpus. (Number of rows in state_mat).

Type
int

name

Optional name for object instance.

Type
Optional[str]

len_dist

Distribution object for the number of words per document. Defaults to the NullDistribution if None is passed.

Type
SequenceEncodableProbabilityDistribution

keys

Keys for word_probs, state_probs, and doc_probs.

Type
Tuple[Optional[str], Optional[str], Optional[str]]

component_log_density(x)

Evaluate the log-density for each state in the PLSI.

Returns $\text{count} \cdot \log(p_{\text{mat}}(W|S))$ for each word-count pair in the document. Returned value is S by 1 where S is the number of components in the model.

Parameters

x (*Tuple[int, Sequence[Tuple[int, float]]]*) – Single PLSI observation of form (doc_id, [(value_id, count_for_value)]).

Return type

numpy.ndarray

Returns

np.ndarray – Numpy array of length S (num_states).

Parameters

x (*Tuple[int, Sequence[Tuple[int, float]]]*)

density(x)

Evaluate the density of PLSI model for an observation x.

See log_density() for details on the density evaluation.

Parameters

$\mathbf{x}$ (*Tuple*[*int*, *Sequence*[*Tuple*[*int*, *float*]]]) – Single observation of integer PLSI.

Return type

float

Returns

float – Density evaluated at observed value x .

Parameters

$\mathbf{x}$ (*Tuple*[*int*, *Sequence*[*Tuple*[*int*, *float*]]])

dist_to_encoder()

Create a data encoder compatible with this distribution.

Return type

dmx.stats.int_plsi.IntegerPLSIDataEncoder

estimator(*pseudo_count=None*)

Create an estimator, optionally regularized toward these parameters.

Return type

dmx.stats.int_plsi.IntegerPLSEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(x)

Evaluate the document log-likelihood.

For $\mathbf{x} = (d, [(v_j, c_j), \dots])$, this returns

$$\log \pi_d + \log P_{\text{len}}(\sum_j c_j) + \sum_j c_j \log (\sum_z \phi_{v_j z} \theta_{dz}) .$$

Parameters

$\mathbf{x}$ (*Tuple*[*int*, *Sequence*[*Tuple*[*int*, *float*]]]) – (*doc_id*, [(*value_id*, *count_for_value*)]). See above for details.

Return type

float

Returns

float – Log-density evaluated at a single observation x .

Parameters

$\mathbf{x}$ (*Tuple*[*int*, *Sequence*[*Tuple*[*int*, *float*]]])

sampler(*seed=None*)

Create a sampler for this distribution.

Return type

dmx.stats.int_plsi.IntegerPLSISampler

Parameters

seed (*int* | *None*)

seq_component_log_density(x)

Evaluate per-topic token log-likelihoods for encoded documents.

See `component_log_density()` function for details on component log-likelihood evaluation.

Parameters

$\mathbf{x}$ (*IntegerPLSEncodedDataSequence*) – *EncodedDataSequence* of PLSI observations.

Return type`numpy.ndarray`**Returns***np.ndarray* – 2-d numpy array containing N rows of num_state sized arrays.**Parameters****x** (`IntegerPLSIEncodedDataSequence`)**seq_log_density(x)**

Evaluate log-likelihoods for an encoded sequence of documents.

Return type`numpy.ndarray`**Parameters****x** (`IntegerPLSIEncodedDataSequence`)**class** `dmx.stats.int_plsi.IntegerPLSIEncodedDataSequence(data)`Bases: `EncodedDataSequence`

IntegerPLSIEncodedDataSequence object for vectorized function calls.

Notes`E = Tuple[EncodedDataSequence, Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray]]`**Parameters****data** (`Tuple[EncodedDataSequence, Tuple[ndarray, ndarray, ndarray, ndarray, ndarray, ndarray]]`)**data**

Encoded sequence of PLSI observations.

Type`E`**class** `dmx.stats.int_plsi.IntegerPLSIEstimator(num_vals, num_states, num_docs, len_estimator=None, pseudo_count=(None, None, None), suff_stat=(None, None, None), name=None, keys=(None, None, None))`Bases: `ParameterEstimator`

Estimate integer PLSI parameters from sufficient statistics.

Parameters

- **num_vals** (`int`)
- **num_states** (`int`)
- **num_docs** (`int`)
- **len_estimator** (`ParameterEstimator` | `None`)
- **pseudo_count** (`Tuple[float | None, float | None, float | None]` | `None`)
- **suff_stat** (`Tuple[ndarray | None, ndarray | None, ndarray | None]` | `None`)
- **name** (`str` | `None`)
- **keys** (`Tuple[str | None, str | None, str | None]` | `None`)

num_vals

Number of words/values in PLSI.

Type

int

num_states

Number of states in PLSI.

Type

int

num_docs

Number of doc_ids (authors) in PLSI.

Type

int

len_estimator

Optional ParameterEstimator object for the length of documents. Should have support on non-negative integers. Defaults to NullEstimator() if None is passed.

Type

ParameterEstimator

pseudo_count

Optional re-weight sufficient statistics in 'estimate()' function. Defaults to (None, None, None) if None is passed.

Type

Tuple[Optional[float], Optional[float], Optional[float]]

suff_stat (Tuple[Optional[np.ndarray], Optional[np.ndarray],

Optional[np.ndarray]): Optional Tuple of numpy arrays containing 'word_counts' (num_states by num_vals), 'state_counts' (num_docs by num_states), and doc_counts (length num_docs). Defaults to (None, None, None) if None is passed.

name

Name of object instance.

Type

Optional[str]

keys

Keys for merging word, state, and doc sufficient statistics with matching keys.

Type

Tuple[Optional[str], Optional[str], Optional[str]]

accumulator_factory()

Create a factory configured for this estimator's model dimensions.

Return type

dmx.stats.int_plsi.IntegerPLSIAccumulatorFactory

estimate(nobs, suff_stat)

Normalize sufficient statistics into a PLSI distribution.

Parameters

- **nobs** – Ignored; document mass is obtained from doc_count.

- **suff_stat** – Token-topic counts of shape (S, V), document-topic counts of shape (D, S), document counts of shape (D,), and length sufficient statistics.

Return type

dmx.stats.int_plsi.IntegerPLSIDistribution

Returns

A distribution whose probability tables are the normalized counts, including configured pseudo-count regularization.

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *ndarray*, *ndarray*, *SS1* | *None*])

class *dmx.stats.int_plsi.IntegerPLSISampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*

IntegerPLSISampler object for sampling from IntegerPLSIDistribution.

Parameters

- **dist** (*IntegerPLSIDistribution*)
- **seed** (*int* | *None*)

rng

RandomState object with seed set if passed.

Type

RandomState

dist

IntegerPLSIDistribution instance to sampler from.

Type

IntegerPLSIDistribution

size_rng

RandomState object for sampling the length of documents.

Type

RandomState

sample(*size=None*)

Generate iid samples from PLSI model.

Parameters

size (*Optional*[*int*]) – Number of samples to generate. Defaults to 0 if size is None.

Return type

typing.Union[*typing.Tuple*[*int*, *typing.Sequence*[*typing.Tuple*[*int*, *float*]]],
typing.Sequence[*typing.Tuple*[*int*, *typing.Sequence*[*typing.Tuple*[*int*,
float]]]]]

Returns

Sequence of iid PLSI samples if size is not None, else a single sample from PLSI model.

Parameters

size (*int* | *None*)

`dmx.stats.int_plsi.bincount(x, w, out)`

Add scalar weights to indexed entries of *out*.

Return type

`typing.Any`

Parameters

- *x* (*Any*)
- *w* (*Any*)
- *out* (*Any*)

`dmx.stats.int_plsi.fast_seq_component_log_density(xv, xc, xd, xi, xm, wmat, out)`

Accumulate per-topic token log-likelihoods using Numba.

Return type

`None`

Parameters

- *xv* (*Any*)
- *xc* (*Any*)
- *xd* (*Any*)
- *xi* (*Any*)
- *xm* (*Any*)
- *wmat* (*Any*)
- *out* (*Any*)

`dmx.stats.int_plsi.fast_seq_log_density(xv, xc, xd, xi, xm, wmat, smat, dvec, out)`

Accumulate PLSI log-likelihoods into *out* using Numba.

Return type

`None`

Parameters

- *xv* (*Any*)
- *xc* (*Any*)
- *xd* (*Any*)
- *xi* (*Any*)
- *xm* (*Any*)
- *wmat* (*Any*)
- *smat* (*Any*)
- *dvec* (*Any*)
- *out* (*Any*)

`dmx.stats.int_plsi.fast_seq_update(xv, xc, xd, xi, xm, weights, wmat, smat, wcnt, scnt, dcnt)`

Accumulate responsibility-weighted PLSI counts using Numba.

Return type

`None`

Parameters

- **xv** (*Any*)
- **xc** (*Any*)
- **xd** (*Any*)
- **xi** (*Any*)
- **xm** (*Any*)
- **weights** (*Any*)
- **wmat** (*Any*)
- **smat** (*Any*)
- **wcnt** (*Any*)
- **scnt** (*Any*)
- **dcnt** (*Any*)

`dmx.stats.int_plsi.index_dot(x, xi, y, yi, out)`

Store indexed row-wise dot products in `out`.

Return type

`typing.Any`

Parameters

- **x** (*Any*)
- **xi** (*Any*)
- **y** (*Any*)
- **yi** (*Any*)
- **out** (*Any*)

`dmx.stats.int_plsi.vec_bincount1(x, w, out)`

Add matrix rows to indexed rows of `out`.

Return type

`typing.Any`

Parameters

- **x** (*Any*)
- **w** (*Any*)
- **out** (*Any*)

`dmx.stats.int_plsi.vec_bincount2(x, w, y, out)`

Add selected matrix rows to indexed rows of `out`.

Return type

`typing.Any`

Parameters

- **x** (*Any*)
- **w** (*Any*)
- **y** (*Any*)

- **out** (*Any*)

`dmx.stats.int_plsi.vec_bincount3(x, w, out)`

Numba bincount on the rows of matrix w for groups x.

Used to update comp counts for word/state probabilities.

$N = \text{len}(x)$ S = number of states. U = unique values in x can take on (unique words in corpus).

Parameters

- **x** (`np.ndarray[np.float64]`) – Group ids of columns of w.
- **w** (`np.ndarray[np.float64]`) – S by N numpy array with cols corresponding to x
- **out** (`np.ndarray[np.float64]`) – S by U matrix.

Return type

`typing.Any`

Returns

Numpy 2-d array.

Parameters

- **x** (*Any*)
- **w** (*Any*)
- **out** (*Any*)

`dmx.stats.int_plsi.vec_bincount4(x, w, out)`

Numba bincount on the rows of matrix w for groups x.

Used to initialize doc/state counts.

$N = \text{len}(x)$ S = number of states. U = unique values in x can take on. (Unique number of authors).

Parameters

- **x** (`np.ndarray[np.float64]`) – Group ids of columns of w.
- **w** (`np.ndarray[np.float64]`) – S by N numpy array with cols corresponding to x
- **out** (`np.ndarray[np.float64]`) – U by S matrix.

Return type

`typing.Any`

Returns

Numpy 2-d array.

Parameters

- **x** (*Any*)
- **w** (*Any*)
- **out** (*Any*)

Latent Dirichlet Allocation

Provide latent Dirichlet allocation for sparse counted documents.

A document is a sparse sequence $[(x_0, c_0), \dots, (x_m, c_m)]$ whose values are accepted by each topic distribution and whose nonnegative counts give token multiplicity. For K topics, the model draws document proportions

$\theta_d \sim \text{Dirichlet}(\alpha)$, then assigns each token a topic $z_{dn} \sim \text{Categorical}(\theta_d)$ and draws its value from the corresponding topic distribution. A separate length distribution supplies the number of tokens when sampling.

The distribution's `log_density` methods return the variational evidence lower bound (ELBO) computed by the document-posterior iteration, not the exact marginal log-likelihood. Accumulator updates use the resulting token-topic responsibilities. Random initialization draws document-topic proportions from the previous `alpha` (or an all-ones vector) and randomly allocates topic statistics.

class `dmx.stats.lda.LDADataEncoder(encoder)`

Bases: [`DataSequenceEncoder`](#)

Encode sparse counted documents for vectorized LDA operations.

Parameters

encoder ([`DataSequenceEncoder`](#))

seq_encode(x)

Encode sparse documents for vectorized LDA operations.

The encoded payload is `(num_documents, idx, counts, gammas, enc_data)`. `idx` maps each flattened distinct token to its document, `counts` stores its multiplicity, `gammas` is initially `None`, and `enc_data` is the wrapped encoder's representation of the flattened token values.

Parameters

x ([`Sequence\[Sequence\[Tuple\[int, float\]\]\]`](#)) – Sequence of LDA documents.

Return type

[`dmx.stats.lda.LDAEncodedDataSequence`](#)

Returns

The encoded document sequence.

Parameters

x ([`Sequence\[Sequence\[Tuple\[int, float\]\]\]`](#))

class `dmx.stats.lda.LDADistribution(topics, alpha, len_dist=NoneDistribution(), gamma_threshold=1.0e-8, keys=(None, None), name=None)`

Bases: [`SequenceEncodableProbabilityDistribution`](#)

Represent a latent Dirichlet allocation model.

Model definition:

`topics[k]` is the conditional value distribution for latent topic `k`; `alpha[k]` is the positive Dirichlet concentration for its document-level proportion. `gamma_threshold` controls only variational inference and is not a model parameter.

Parameters

- **topics** ([`Sequence\[SequenceEncodableProbabilityDistribution\]`](#))
- **alpha** ([`Sequence\[float\]`](#) | [`ndarray`](#))
- **len_dist** ([`SequenceEncodableProbabilityDistribution`](#) | `None`)
- **gamma_threshold** (`float`)
- **keys** ([`Tuple\[str | None, str | None\]`](#))
- **name** (`str` | `None`)

topics

Topic distributions for the LDA.

Type

Sequence[*SequenceEncodableProbabilityDistribution*]

alpha

Parameter to the prior Dirichlet for which topics are drawn.

Type

np.ndarray

len_dist

Distribution for length of documents. Must be set to non-negative support distribution for sampling. Default to NullDistribution.

Type

SequenceEncodableProbabilityDistribution

gamma_threshold

For numerical stability in estimation.

Type

float

density(*x*)

Exponentiate the variational lower bound for one document.

Return type

float

Parameters

x (Sequence[Tuple[int, float]])

dist_to_encoder()

Create a document encoder using the first topic's value encoder.

Return type

dmx.stats.lda.LDADataEncoder

estimator(*pseudo_count=None*)

Create an estimator for the topic models and Dirichlet parameter.

Return type

dmx.stats.lda.LDAEstimator

Parameters

pseudo_count (float | None)

log_density(*x*)

Return the variational evidence lower bound for one document.

Return type

float

Parameters

x (Sequence[Tuple[int, float]])

sampler(*seed=None*)

Create a sampler for this LDA distribution.

Return type`dmx.stats.lda.LDASampler`**Parameters**`seed(int | None)`**seq_component_log_density(*x*)**

Return count-weighted value log-densities for each document and topic.

Return type`numpy.ndarray`**Parameters**`x(LDAEncodedDataSequence)`**seq_log_density(*x*)**

Return the variational evidence lower bound for encoded documents.

The returned vector has one entry per document. Despite the protocol method name, these values are ELBOs obtained after iterative variational posterior updates, rather than exact marginal log-likelihoods.

Return type`numpy.ndarray`**Parameters**`x(LDAEncodedDataSequence)`**seq_posterior(*x*)**

Return normalized variational document-topic parameters.

Return type`numpy.ndarray`**Parameters**`x(LDAEncodedDataSequence)`**class dmx.stats.lda.LDAEncodedDataSequence(*data*)**

Bases: [`EncodedDataSequence`](#)

Store flattened documents and optional variational initialization values.

Parameters`data(Tuple[int, ndarray, ndarray, ndarray | None, EncodedDataSequence])`**class dmx.stats.lda.LDAEstimator(*estimators*, *suff_stat*=None, *pseudo_count*=None, *name*=None, *keys*=(None, None), *fixed_alpha*=None, *gamma_threshold*=1.0e-8, *alpha_threshold*=1.0e-8)**

Bases: [`ParameterEstimator`](#)

Estimate an LDADistribution from aggregated document and topic statistics.

Notes

keys controls two different sharing decisions for LDA estimators.

- `keys[0]` shares the statistics used to update `alpha`: accumulated expected log-topic proportions, document counts, and the previous `alpha` iterate.
- `keys[1]` shares topic sufficient statistics by topic index.

This means `keys=(None, "shared_topics")` ties the topic-word models while allowing each LDA estimator to keep its own document-topic prior. In contrast, `keys=("shared_alpha", None)` shares only the `alpha` update.

Shared topic keys assume topic index *i* represents the same topic role across the models being fit. If topic order is not aligned, keyed sharing will pool the wrong topic statistics. When `fixed_alpha` is set, alpha-sharing keys do not change the final alpha because the estimator uses the fixed value.

Parameters

- **estimators** (*Sequence*[*ParameterEstimator*])
- **suff_stat** (*Any* | *None*)
- **pseudo_count** (*Tuple*[*float*, *float*] | *None*)
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*] | *None*)
- **fixed_alpha** (*ndarray* | *None*)
- **gamma_threshold** (*float*)
- **alpha_threshold** (*float*)

accumulator_factory()

Create a factory for compatible LDA accumulators.

Return type

dmx.stats.lda.LDAEstimatorAccumulatorFactory

estimate(nobs, suff_stat)

Estimate topic distributions and the Dirichlet concentration.

Parameters

- **nobs** – Ignored; weighted document mass is included in `suff_stat`.
- **suff_stat** – Previous alpha, summed expected log topic proportions, weighted document mass, topic responsibility totals, and one set of sufficient statistics per topic estimator.

Return type

dmx.stats.lda.LDADistribution

Returns

An LDA distribution with updated topics and either an updated or fixed Dirichlet concentration.

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray* | *None*, *ndarray*, *float*, *ndarray*, *Sequence*[*Any*]])

class *dmx.stats.lda.LDAEstimatorAccumulator*(*accumulators*, *name=None*, *keys=(None, None)*, *prev_alpha=None*)

Bases: *SequenceEncodableStatisticAccumulator*

Accumulate variational LDA sufficient statistics.

The stored values are the previous Dirichlet parameter, sums of expected log document-topic proportions, weighted document mass, topic responsibility mass, and the sufficient statistics of each topic model.

Parameters

- **accumulators** (*Sequence*[*SequenceEncodableStatisticAccumulator*])
- **name** (*str* | *None*)

- **keys** (*Tuple[str | None, str | None] | None*)
- **prev_alpha** (*ndarray | None*)

acc_to_encoder()

Create an LDA encoder from the first topic accumulator.

Return type

dmx.stats.lda.LDADataEncoder

combine(suff_stat)

Add another accumulator's sufficient statistics in place.

Return type

dmx.stats.lda.LDAEstimatorAccumulator

Parameters

suff_stat (*Tuple[ndarray | None, ndarray, float, ndarray, Sequence[SS0]]*)

from_value(x)

Replace the sufficient statistics and return this accumulator.

Return type

dmx.stats.lda.LDAEstimatorAccumulator

Parameters

x (*Tuple[ndarray | None, ndarray, float, ndarray, Sequence[SS0]]*)

initialize(x, weight, rng)

Randomly initialize sufficient statistics for one sparse document.

Return type

None

Parameters

- **x** (*Sequence[Tuple[Any, float]]*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(stats_dict)

Merge alpha-update and topic statistics under configured keys.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

key_replace(stats_dict)

Replace alpha-update and topic statistics from configured keys.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

seq_initialize(*x*, *weights*, *rng*)

Randomly initialize sufficient statistics for encoded documents.

Document proportions are drawn from `prev_alpha`, defaulting to an all-ones vector, and token-topic weights are randomized before being passed to each topic accumulator.

Return type

None

Parameters

- **x** (`LDAEncodedDataSequence`)
- **weights** (`ndarray`)
- **rng** (`RandomState`)

seq_update(*x*, *weights*, *estimate*)

Update statistics from variational token-topic responsibilities.

Return type

None

Parameters

- **x** (`LDAEncodedDataSequence`)
- **weights** (`ndarray`)
- **estimate** (`LDADistribution`)

update(*x*, *weight*, *estimate*)

Leave scalar observations unchanged; LDA updates require encoding.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **estimate** (*Any*)

value()

Return alpha-update, document, and per-topic sufficient statistics.

Return type

`typing.Tuple[typing.Optional[numpy.ndarray], numpy.ndarray, float, numpy.ndarray, typing.Sequence[typing.Any]]`

```
class dmx.stats.lda.LDAEstimatorAccumulatorFactory(factories, dim, name=None, keys=(None, None),
                                                    prev_alpha=None)
```

Bases: `StatisticAccumulatorFactory`

Create LDA sufficient-statistic accumulators.

Parameters

- **factories** (`Sequence[StatisticAccumulatorFactory]`)
- **dim** (*int*)
- **name** (*str* | *None*)

- **keys** (*Tuple*[*str* | *None*, *str* | *None*] | *None*)
- **prev_alpha** (*ndarray* | *None*)

make()

Create an LDA sufficient-statistic accumulator.

Return type

dmx.stats.lda.LDAEstimatorAccumulator

class *dmx.stats.lda.LDASampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*

Sample sparse documents from an LDA distribution.

Parameters

- **dist** (*LDADistribution*)
- **seed** (*int* | *None*)

sample(*size=None*)

Draw one document or a sequence of documents.

Return type

typing.Union[*typing.Sequence*[*typing.List*[*typing.Tuple*[*typing.Any*, *int*]]],
typing.List[*typing.Tuple*[*typing.Any*, *int*]]]

Parameters

size (*int* | *None*)

dmx.stats.lda.alpha_seq_lambda(*mean_log_p*)

Create the fixed-point map for a Dirichlet concentration update.

Return type

typing.Callable[[*numpy.ndarray*], *numpy.ndarray*]

Parameters

mean_log_p (*ndarray*)

dmx.stats.lda.find_alpha(*current_alpha*, *mlp*, *thresh*)

Estimate a Dirichlet concentration using extrapolated fixed-point updates.

Return type

typing.Tuple[*numpy.ndarray*, *int*]

Parameters

- **current_alpha** (*ndarray*)
- **mlp** (*ndarray*)
- **thresh** (*float*)

dmx.stats.lda.mpe(*x0*, *f*, *eps*)

Iterate a fixed-point map with minimal-polynomial extrapolation.

Return type

typing.Tuple[*numpy.ndarray*, *int*]

Parameters

- **x0** (*ndarray*)
- **f** (*Callable*[[*ndarray*], *ndarray*])

- **eps** (*float*)

`dmx.stats.lda.mpe_update(x_mat, y, min_size=2)`

Append a fixed-point iterate and compute a minimal-polynomial extrapolation.

Return type

`typing.Tuple[numpy.ndarray, numpy.ndarray]`

Parameters

- **x_mat** (*ndarray* | *None*)
- **y** (*ndarray*)
- **min_size** (*int*)

`dmx.stats.lda.seq_posterior(estimate, x)`

Compute the mean-field variational posterior for encoded documents.

Parameters

- **estimate** – LDA model supplying topics, alpha, and convergence threshold.
- **x** – Encoded payload (`num_documents`, `idx`, `counts`, `gammas`, `enc_data`).

Return type

`typing.Tuple[numpy.ndarray, numpy.ndarray, numpy.ndarray]`

Returns

Token-topic responsibilities of shape (M, K), document Dirichlet parameters of shape (D, K), and topic log-densities of shape (M, K), where M is the number of distinct encoded document-token pairs.

Parameters

- **estimate** (*LDADistribution*)
- **x** (*Tuple[int, ndarray, ndarray, Any | None, E0]*)

`dmx.stats.lda.seq_posterior2(estimate, x)`

Delegate to `seq_posterior()` for backward compatibility.

Return type

`typing.Tuple[numpy.ndarray, numpy.ndarray, numpy.ndarray]`

Parameters

- **estimate** (*LDADistribution*)
- **x** (*Tuple[int, ndarray, ndarray, Any | None, E0]*)

`dmx.stats.lda.update_alpha(alpha_curr, mean_log_p, alpha_threshold)`

Solve the Dirichlet fixed-point update for mean log proportions.

Parameters

- **alpha_curr** – Positive starting concentration vector.
- **mean_log_p** – Mean expected log topic proportions.
- **alpha_threshold** – Relative L1 convergence threshold.

Return type

`typing.Tuple[numpy.ndarray, int]`

Returns

The updated concentration vector and iteration count.

Parameters

- **alpha_curr** (*ndarray*)
- **mean_log_p** (*ndarray*)
- **alpha_threshold** (*float*)

Semi-supervised Mixture

Model observations with optional priors on their mixture labels.

An observation is (*datum*, *prior*). *prior* is either *None* or a sequence of (*component_index*, *probability*) pairs. With no prior, the latent component *Z* has the global mixture weights. With a prior, only listed components are eligible and their supplied probabilities reweight the global weights before normalization. Responsibilities combine this conditional label information with component likelihoods. The sampler returns bare data because label priors are conditioning input, not generated observations.

Unlike *heterogeneous_mixture*, all components use one encoder. Unlike *hmixture*, each observation has one latent component rather than sequence- and item-level variables. Unlike *jmixture*, the second tuple entry is supervision, not a second modeled view.

class `dmx.stats.ss_mixture.SemiSupervisedMixtureDataEncoder`(*encoder*)

Bases: *DataSequenceEncoder*

Encode data and sparse optional label priors.

For batch size *B* the result is (*B*, *enc_data*, *prior_info*, *raw_data*). *prior_info* is (*prior_mat*, *prior_sum*, *has_prior*). *prior_mat* contains four aligned one-dimensional arrays: observation indices, component indices, prior probabilities, and log probabilities. *prior_sum* and *has_prior* have shape (*B*,). *raw_data* is retained for scalar initialization.

Parameters

encoder (*DataSequenceEncoder*)

seq_encode(*x*)

Encode a batch of data and optional sparse label priors.

Return type

dmx.stats.ss_mixture.SemiSupervisedMixtureEncodedDataSequence

Parameters

x (*Sequence[Tuple[T0, Sequence[Tuple[int, float]] | None]]*)

class `dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution`(*components*, *w*, *name=None*)

Bases: *SequenceEncodableProbabilityDistribution*

Define a mixture conditioned by optional per-observation label priors.

Parameters

- **components** (*Sequence[SequenceEncodableProbabilityDistribution]*)
- **w** (*List[float] | ndarray*)
- **name** (*str | None*)

components

Mixture components.

Type

Sequence[SequenceEncodableProbabilityDistribution]

num_components

Number of mixture components.

Type

int

zw

Bool numpy array, True where weights are 0.0.

Type

np.ndarray

log_w

Log of weights. Set to -np.inf where weights are 0.

Type

np.ndarray

w

Mixture weights. Should sum to 1.0.

Type

np.ndarray

name

Set name for object.

Type

Optional[str]

density(*x*)

Evaluate density for a datum and optional label prior.

Return type

float

Parameters

x (*Tuple*[*T0*, *Sequence*[*Tuple*[*int*, *float*]] | *None*)

dist_to_encoder()

Return an encoder for data and sparse label priors.

Return type

dmx.stats.ss_mixture.SemiSupervisedMixtureDataEncoder

estimator(*pseudo_count=None*)

Return an estimator for mixture weights and shared-type components.

Return type

dmx.stats.ss_mixture.SemiSupervisedMixtureEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Evaluate log-density for a datum and optional label prior.

Return type

float

Parameters

x (*Tuple*[*T0*, *Sequence*[*Tuple*[*int*, *float*]] | *None*)

posterior(*x*)

Compute component responsibilities subject to an optional label prior.

Return type

`numpy.ndarray`

Parameters

x (`Tuple[T0, Sequence[Tuple[int, float]] | None]`)

sampler(*seed=None*)

Return a sampler that generates unlabeled component data.

Return type

`dmx.stats.ss_mixture.SemiSupervisedMixtureSampler`

Parameters

seed (`int | None`)

seq_log_density(*x*)

Evaluate log-densities for encoded conditional observations.

Return type

`numpy.ndarray`

Parameters

x (`SemiSupervisedMixtureEncodedDataSequence`)

seq_posterior(*x*)

Evaluate responsibilities for encoded conditional observations.

Return type

`numpy.ndarray`

Parameters

x (`SemiSupervisedMixtureEncodedDataSequence`)

class `dmx.stats.ss_mixture.SemiSupervisedMixtureEncodedDataSequence`(*data*)

Bases: `EncodedDataSequence`

Store encoded data, sparse label-prior arrays, and original observations.

Notes

`E1 = Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]` `E = Tuple[int, EncodedDataSequence, Tuple[E1, np.ndarray, np.ndarray], T]`

Parameters

data (`Tuple[int, EncodedDataSequence, Tuple[Tuple[ndarray, ndarray, ndarray, ndarray], ndarray, ndarray], Sequence[Tuple[T0, Sequence[Tuple[int, float]] | None]]]`)

data

Encoded sequence of semi-supervised mixture observations.

Type

`E`

class `dmx.stats.ss_mixture.SemiSupervisedMixtureEstimator`(*estimators, suff_stat=None, pseudo_count=None, keys=(None, None), name=None*)

Bases: *ParameterEstimator*

Estimate a semi-supervised mixture from sufficient statistics.

Label priors affect responsibilities during accumulation; estimation then uses the resulting component-count vector and one child statistic per component. `keys[0]` shares counts and `keys[1]` shares child statistics positionally.

Parameters

- **estimators** (*Sequence*[*ParameterEstimator*])
- **suff_stat** (*ndarray* | *None*)
- **pseudo_count** (*float* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*] | *None*)
- **name** (*str* | *None*)

estimators

Sequence of *ParameterEstimators* objects for the components of the mixture. All must be of the same class compatible with data type T.

Type

Sequence[*ParameterEstimator*]

suff_stat

Mixture weights for components obtained from prev estimation or for regularization.

Type

Optional[*np.ndarray*]

pseudo_count

Re-weight sufficient statistics, i.e. penalize sufficient statistics.

Type

Optional[*float*]

keys

Set keys for the weights and components.

Type

Optional[*Tuple*[*Optional*[*str*], *Optional*[*str*]]]

name

Set name for object.

Type

Optional[*str*]

accumulator_factory()

Return a factory sharing child statistics by component index.

Return type

dmx.stats.ss_mixture.SemiSupervisedMixtureEstimatorAccumulatorFactory

estimate(*nobs*, *suff_stat*)

Estimate component distributions and mixture weights.

Return type

dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *Tuple*[*SS0*, ...]])

```
class dmx.stats.ss_mixture.SemiSupervisedMixtureEstimatorAccumulator(accumulators,  
                                                                    keys=(None, None),  
                                                                    name=None)
```

Bases: [*SequenceEncodableStatisticAccumulator*](#)

Accumulate label-conditioned component responsibilities and statistics.

Parameters

- **accumulators** (*Sequence*[[*SequenceEncodableStatisticAccumulator*](#)])
- **keys** (*Tuple*[*str* | *None*, *str* | *None*] | *None*)
- **name** (*str* | *None*)

acc_to_encoder()

Return the encoder required by the child accumulators.

Return type

[*dmx.stats.ss_mixture.SemiSupervisedMixtureDataEncoder*](#)

combine(*suff_stat*)

Merge another semi-supervised mixture sufficient statistic.

Return type

[*dmx.stats.ss_mixture.SemiSupervisedMixtureEstimatorAccumulator*](#)

Parameters

suff_stat (*Tuple*[*ndarray*, *Tuple*[*SS0*, ...]])

from_value(*x*)

Replace accumulated statistics from a serialized value.

Return type

[*dmx.stats.ss_mixture.SemiSupervisedMixtureEstimatorAccumulator*](#)

Parameters

x (*Tuple*[*ndarray*, *Tuple*[*SS0*, ...]])

initialize(*x*, *weight*, *rng*)

Initialize component statistics using priors or a random assignment.

Return type

None

Parameters

- **x** (*Tuple*[*T0*, *Sequence*[*Tuple*[*int*, *float*]] | *None*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge keyed component counts and child statistics.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**key_replace**(*stats_dict*)

Replace statistics from matching keyed values.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**seq_initialize**(*x, weights, rng*)

Initialize encoded observations through the scalar initialization path.

Return type

None

Parameters

- **x** (*SemiSupervisedMixtureEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x, weights, estimate*)

Accumulate posterior-weighted statistics for encoded observations.

Return type

None

Parameters

- **x** (*SemiSupervisedMixtureEncodedDataSequence*)
- **weights** (*ndarray*)
- **estimate** (*SemiSupervisedMixtureDistribution*)

update(*x, weight, estimate*)

Update statistics for one conditionally labeled observation.

Return type

None

Parameters

- **x** (*Tuple[T0, Sequence[Tuple[int, float]] | None]*)
- **weight** (*float*)
- **estimate** (*SemiSupervisedMixtureDistribution*)

value()

Return component counts and child sufficient statistics.

Return type*typing.Tuple[*numpy.ndarray*, *typing.Tuple[typing.Any, ...]*]*

```
class dmx.stats.ss_mixture.SemiSupervisedMixtureEstimatorAccumulatorFactory(factories, dim,
                                                                           keys=(None,
                                                                           None),
                                                                           name=None)
```


Bases: *SequenceEncodableStatisticAccumulator*

Accumulate expected latent assignments and component statistics.

The sufficient statistic is (conditional, source, length). During an update, posterior assignment probabilities distribute each target count across source values before updating the conditional component.

Parameters

- **cond_acc** (*ConditionalDistributionAccumulator*)
- **given_acc** (*SequenceEncodableStatisticAccumulator* | *None*)
- **size_acc** (*SequenceEncodableStatisticAccumulator* | *None*)
- **name** (*str* | *None*)
- **keys** (*Tuple[str | None, str | None]* | *None*)

acc_to_encoder()

Create the pass-through encoder used by this accumulator.

Return type

dmx.stats.hidden_association.HiddenAssociationDataEncoder

combine(suff_stat)

Merge conditional, source, and length sufficient statistics.

Return type

dmx.stats.hidden_association.HiddenAssociationAccumulator

Parameters

suff_stat (*Tuple[SS1, SS2 | None, SS3 | None]*)

from_value(x)

Replace all component sufficient statistics with x.

Return type

dmx.stats.hidden_association.HiddenAssociationAccumulator

Parameters

x (*Tuple[SS1, SS2 | None, SS3 | None]*)

initialize(x, weight, rng)

Initialize statistics with random source assignments per target value.

Return type

None

Parameters

- **x** (*Tuple[List[Tuple[T, float]], List[Tuple[T, float]]]*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(stats_dict)

Merge keyed component statistics into stats_dict.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**key_replace**(*stats_dict*)Replace keyed component statistics from *stats_dict*.**Return type**

None

Parameters**stats_dict** (*Dict[str, Any]*)**seq_initialize**(*x, weights, rng*)

Initialize statistics from an encoded weighted batch.

Return type

None

Parameters

- **x** (*HiddenAssociationEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x, weights, estimate*)

Add an encoded weighted batch using posterior assignments.

Return type

None

Parameters

- **x** (*HiddenAssociationEncodedDataSequence*)
- **weights** (*ndarray*)
- **estimate** (*HiddenAssociationDistribution*)

update(*x, weight, estimate*)

Add a weighted paired-bag observation using posterior assignments.

Return type

None

Parameters

- **x** (*Tuple[List[Tuple[T, float]], List[Tuple[T, float]]]*)
- **weight** (*float*)
- **estimate** (*HiddenAssociationDistribution*)

value()

Return conditional, source, and target-length statistics.

Return type*typing.Tuple[typing.Any, typing.Optional[typing.Any], typing.Optional[typing.Any]]*

```
class dmx.stats.hidden_association.HiddenAssociationAccumulatorFactory(cond_factory,  
                                                                    given_factory=NullAccumulatorFactory(),  
                                                                    len_factory=NullAccumulatorFactory(),  
                                                                    name=None,  
                                                                    keys=(None, None))
```

Bases: `StatisticAccumulatorFactory`

Create generic hidden-association accumulators.

Parameters

- **cond_factory** (`ConditionalDistributionAccumulatorFactory`)
- **given_factory** (`StatisticAccumulatorFactory` | `None`)
- **len_factory** (`StatisticAccumulatorFactory` | `None`)
- **name** (`str` | `None`)
- **keys** (`Tuple[str | None, str | None]` | `None`)

make()

Create a new accumulator with fresh component accumulators.

Return type

`dmx.stats.hidden_association.HiddenAssociationAccumulator`

class `dmx.stats.hidden_association.HiddenAssociationDataEncoder`

Bases: `DataSequenceEncoder`

Wrap paired bags without transforming their generic values.

seq_encode(x)

Wrap a batch of paired source and target bags for vectorized calls.

Return type

`dmx.stats.hidden_association.HiddenAssociationEncodedDataSequence`

Parameters

x (`Sequence[Tuple[List[Tuple[T, float]], List[Tuple[T, float]]]`)

class `dmx.stats.hidden_association.HiddenAssociationDistribution`(`cond_dist`,
`given_dist=NullDistribution()`,
`len_dist=NullDistribution()`,
`name=None`, `keys=(None, None)`)

Bases: `SequenceEncodableProbabilityDistribution`

Model associations between source and target bags of generic values.

The latent source assignment for each target occurrence is marginalized in density evaluation and sampling returns grouped target counts.

Parameters

- **cond_dist** (`ConditionalDistribution`)
- **given_dist** (`SequenceEncodableProbabilityDistribution` | `None`)
- **len_dist** (`SequenceEncodableProbabilityDistribution` | `None`)
- **name** (`str` | `None`)
- **keys** (`Tuple[str | None, str | None]` | `None`)

cond_dist

`ConditionalDistribution` defining target-value distributions conditioned on source values.

Type*ConditionalDistribution***given_dist**

Distribution for the grouped source bag. Defaults to `NullDistribution`.

Type*SequenceEncodableProbabilityDistribution***len_dist**

Distribution for the length of the target bag, measured by its total count.

Type*SequenceEncodableProbabilityDistribution***name**

Name for object instance.

Type`Optional[str]`**keys**

Compatibility keys forwarded to the estimator and accumulator factory.

Type`Tuple[Optional[str], Optional[str]]`**density(*x*)**

Evaluate the density of a paired source and target bag.

Return type`float`**Parameters**`x (Tuple[List[Tuple[T, float]], List[Tuple[T, float]])`**dist_to_encoder()**

Create the pass-through encoder used by vectorized operations.

Return type*dmx.stats.hidden_association.HiddenAssociationDataEncoder***estimator(*pseudo_count=None*)**

Create an estimator from the three component estimators.

The `pseudo_count` argument is accepted for protocol compatibility but is not forwarded by this implementation.

Return type*dmx.stats.hidden_association.HiddenAssociationEstimator***Parameters**`pseudo_count (float | None)`**log_density(*x*)**

Evaluate the log density after marginalizing source assignments.

Return type`float`**Parameters**`x (Tuple[List[Tuple[T, float]], List[Tuple[T, float]])`

sampler(*seed=None*)

Create a sampler for paired source and target bags.

Return type

`dmx.stats.hidden_association.HiddenAssociationSampler`

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Evaluate log densities for an encoded batch of paired bags.

Return type

`numpy.ndarray`

Parameters

x (`HiddenAssociationEncodedDataSequence`)

class `dmx.stats.hidden_association.HiddenAssociationEncodedDataSequence`(*data*)

Bases: `EncodedDataSequence`

Store a batch of generic paired-bag observations.

Parameters

data (`Sequence[Tuple[List[Tuple[T, float]], List[Tuple[T, float]]]`)

data

iid obs.

Type

`Sequence[Tuple[List[Tuple[T, float]], List[Tuple[T, float]]]`

class `dmx.stats.hidden_association.HiddenAssociationEstimator`(*cond_estimator*,
given_estimator=None,
len_estimator=None,
pseudo_count=None, *name=None*,
keys=(None, None))

Bases: `ParameterEstimator`

Estimate a generic hidden-association distribution.

The conditional, source-bag, and target-length components are estimated independently from their corresponding sufficient statistics. Assignment expectations are computed earlier by accumulator updates.

Parameters

- **cond_estimator** (`ConditionalDistributionEstimator`)
- **given_estimator** (`ParameterEstimator` | *None*)
- **len_estimator** (`ParameterEstimator` | *None*)
- **pseudo_count** (*float* | *None*)
- **name** (*str* | *None*)
- **keys** (`Tuple[str | None, str | None]` | *None*)

cond_estimator

Estimator for the conditional emission of values in set 2 given states.

Type

`ConditionalDistributionEstimator`

given_estimator

Estimator for the given values. Should be compatible with `Tuple[T, float]` where `T` is the type for the values.

Type

ParameterEstimator

len_estimator

Estimator for the length of the observed set 2 values.

Type

ParameterEstimator

pseudo_count

Kept for consistency.

Type

`Optional[float]`

name

Set name for object instance.

Type

`Optional[str]`

keys

Set keys for weights and transitions.

Type

`Optional[Tuple[Optional[str], Optional[str]]]`

accumulator_factory()

Create a factory for compatible sufficient-statistic accumulators.

Return type

dmx.stats.hidden_association.HiddenAssociationAccumulatorFactory

estimate(*nobs*, *suff_stat*)

Estimate all model components from aggregated statistics.

Return type

dmx.stats.hidden_association.HiddenAssociationDistribution

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*SS1*, *SS2* | *None*, *SS3* | *None*])

class `dmx.stats.hidden_association.HiddenAssociationSampler`(*dist*, *seed=None*)

Bases: *DistributionSampler*

Sample paired bags or target bags conditional on a source bag.

Parameters

- **dist** (*HiddenAssociationDistribution*)
- **seed** (*int* | *None*)

sample(*size=None*)

Draw one paired-bag observation or a batch of observations.

Return type

```
typing.Union[typing.Sequence[typing.Tuple[typing.List[typing.
    Tuple[typing.Any, float]], typing.List[typing.Tuple[typing.Any, float]]],
    typing.Tuple[typing.List[typing.Tuple[typing.Any, float]], typing.
        List[typing.Tuple[typing.Any, float]]]
```

Parameters

```
size (int | None)
```

sample_given(x)

Draw a grouped target bag conditional on source bag x.

Return type

```
typing.List[typing.Tuple[typing.Any, float]]
```

Parameters

```
x (List[Tuple[T, float]])
```

Chow–Liu Tree

Provide integer-valued Chow–Liu tree distributions.

An observation is a fixed-length vector $x = (x_0, \dots, x_{F-1})$ of nonnegative categories. `dependency_list` gives a rooted feature tree as (feature, parent) pairs, with one None parent. The model is

$$p(x) = p(x_r) \prod_{(j,p) \in \mathcal{T}, p \neq \text{None}} p(x_j \mid x_p).$$

The accumulator collects feature marginals and pairwise category tables. The estimator uses pairwise mutual information to find a maximum-dependence spanning tree, roots it at feature zero, then estimates the conditional PMFs. This is the Chow–Liu tree approximation (Chow and Liu, 1968).

```
class dmx.stats.icltree.ICLTreeAccumulator(num_features, num_states, keys=None, name=None)
```

Bases: [*SequenceEncodableStatisticAccumulator*](#)

Accumulate marginal and pairwise counts for Chow–Liu fitting.

Parameters

- **num_features** (int | None)
- **num_states** (int | None)
- **keys** (str | None)
- **name** (str | None)

acc_to_encoder()

Return the encoder for accumulated integer vectors.

Return type

```
dmx.stats.icltree.ICLTreeDataEncoder
```

combine(suff_stat)

Combine another marginal-and-pairwise count tuple.

Return type

```
dmx.stats.icltree.ICLTreeAccumulator
```

Parameters

```
suff_stat (Tuple[int, int, ndarray, ndarray])
```

from_value(*x*)

Restore count arrays from a sufficient-statistic tuple.

Return type

dmx.stats.icltree.ICLTreeAccumulator

Parameters

x (*Tuple[int, int, ndarray, ndarray]*)

initialize(*x, weight, rng*)

Initialize counts from one observation without randomization.

Return type

None

Parameters

- **x** (*Sequence[int] | ndarray*)
- **weight** (*float*)
- **rng** (*RandomState | None*)

key_merge(*stats_dict*)

Leave keyed merging unimplemented for this accumulator.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

key_replace(*stats_dict*)

Leave keyed replacement unimplemented for this accumulator.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

seq_initialize(*x, weights, rng*)

Initialize counts from an encoded batch without randomization.

Return type

None

Parameters

- **x** (*ICLTreeEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState | None*)

seq_update(*x, weights, estimate*)

Accumulate weighted counts from an encoded batch.

Return type

None

Parameters

- **x** (*ICLTreeEncodedDataSequence*)

- **weights** (*ndarray*)
- **estimate** (*ICLTreeDistribution* | *None*)

update(*x*, *weight*, *estimate*)

Accumulate weighted counts from one feature vector.

Return type

None

Parameters

- **x** (*Sequence[int]* | *ndarray*)
- **weight** (*float*)
- **estimate** (*ICLTreeDistribution* | *None*)

value()

Return feature count, category count, and count arrays.

Return type

typing.Tuple[int, int, numpy.ndarray, numpy.ndarray]

class `dmx.stats.icltree.ICLTreeAccumulatorFactory`(*num_features=None*, *num_states=None*,
keys=None, *name=None*)

Bases: *StatisticAccumulatorFactory*

Create accumulators for integer Chow–Liu sufficient statistics.

Parameters

- **num_features** (*int* | *None*)
- **num_states** (*int* | *None*)
- **keys** (*str* | *None*)
- **name** (*str* | *None*)

make()

Create a fresh ICL tree accumulator.

Return type

dmx.stats.icltree.ICLTreeAccumulator

class `dmx.stats.icltree.ICLTreeDataEncoder`

Bases: *DataSequenceEncoder*

ICLTreeDataEncoder object for encoding sequences of iid ICL observations.

seq_encode(*x*)

Encode vectors as an integer array with shape (N, F).

Return type

dmx.stats.icltree.ICLTreeEncodedDataSequence

Parameters

x (*List[int]* | *ndarray*)

class `dmx.stats.icltree.ICLTreeDistribution`(*dependency_list*, *conditional_log_densities*,
feature_order=None, *name=None*, *keys=None*)

Bases: *SequenceEncodableProbabilityDistribution*

Represent a directed Chow–Liu tree over integer feature vectors.

Parameters

- **dependency_list** (*Sequence[Tuple[int, int | None]]*)
- **conditional_log_densities** (*Sequence[Any]*)
- **feature_order** (*Sequence[int] | None*)
- **name** (*str | None*)
- **keys** (*str | None*)

feature_order

Ordering of features. If None, ordering is assumed as entered.

Type

Sequence[int]

dependency_list

List of Tuples containing features order id and Tuple of feature and feature dep.

Type

List[Tuple[int, Tuple[int, Optional[int]]]

conditional_log_densities

Conditional log densities for each features' dependency split.

Type

Union[Sequence[float], np.ndarray]

conditional_densities

Conditional densities as numpy array.

Type

np.ndarray

num_features

Total number of features.

Type

int

name

Name for object instance.

Type

Optional[str]

keys

Keys for parameters of model.

Type

Optional[str]

density(*x*)

Return the probability of one integer feature vector.

Return type

float

Parameters

x (*Sequence[int] | ndarray*)

dist_to_encoder()

Return the encoder for fixed-length integer vectors.

Return type

dmx.stats.icltree.ICLTreeDataEncoder

estimator(pseudo_count=None)

Create an estimator; this convenience method ignores pseudo-counts.

Return type

dmx.stats.icltree.ICLTreeEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(x)

Return the log probability of one integer feature vector.

Return type

float

Parameters

x (*Sequence[int]* | *ndarray*)

sampler(seed=None)

Create a sampler that draws features in tree order.

Return type

dmx.stats.icltree.ICLTreeSampler

Parameters

seed (*int* | *None*)

seq_log_density(x)

Return log probabilities for an encoded batch of feature vectors.

Return type

numpy.ndarray

Parameters

x (*ICLTreeEncodedDataSequence*)

class dmx.stats.icltree.ICLTreeEncodedDataSequence(data)

Bases: *EncodedDataSequence*

Hold encoded integer vectors with batch-first shape (N, F).

Parameters

data (*ndarray*)

data

Numpy array of observations.

Type

np.ndarray

class dmx.stats.icltree.ICLTreeEstimator(num_features=None, num_states=None, pseudo_count=None, suff_stat=None, keys=None, name=None)

Bases: *ParameterEstimator*

Fit a mutual-information spanning-tree approximation to integer data.

Parameters

- **num_features** (*int* | *None*)
- **num_states** (*int* | *None*)
- **pseudo_count** (*float* | *None*)
- **suff_stat** (*Any* | *None*)
- **keys** (*str* | *None*)
- **name** (*str* | *None*)

accumulator_factory()

Return a factory for the estimator's count arrays.

Return type

dmx.stats.icltree.ICLTreeAccumulatorFactory

estimate(*nobs*, *suff_stat*)

Estimate a rooted Chow–Liu tree and its conditional PMFs.

Return type

dmx.stats.icltree.ICLTreeDistribution

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*int*, *int*, *ndarray*, *ndarray*])

class *dmx.stats.icltree.ICLTreeSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*

Sample independent integer vectors from an ICL tree.

Parameters

- **dist** (*ICLTreeDistribution*)
- **seed** (*int* | *None*)

rng

RandomState for setting sampling seed.

Type

RandomState

dist

ICL Tree distribution to sample from.

Type

ICLTreeDistribution

sample(*size=None*)

Draw one vector, or size independently drawn vectors.

Return type

typing.Union[*typing.List*[*typing.Optional*[*int*]], *typing.Sequence*[*typing.List*[*typing.Optional*[*int*]]]

Parameters

size (*int* | *None*)

Integer Hidden Association

Provide low-rank hidden-association models for paired integer bags.

An observation is (source, target); each bag contains (value, count) pairs. Source values lie in $\{0, \dots, V_1 - 1\}$, target values in $\{0, \dots, V_2 - 1\}$, and latent states in $\{0, \dots, K - 1\}$. Write source counts as c_v , target counts as d_y , and $M = \sum_v c_v$. For every target occurrence, a latent assignment selects source value v with probability c_v/M , then a latent state H and target value are drawn using `cond_weights[v, h]` and `state_prob_mat[h, y]`. With uniform-mixture weight `alpha`, the marginalized target probability is

$$q(y \mid \text{source}) = \frac{\alpha}{V_2} + (1 - \alpha) \sum_v \frac{c_v}{M} \sum_h W_{vh} Q_{hy}.$$

The joint log density adds `prev_dist.log_density(source)` and the log density of the total target count under `len_dist`. EM updates accumulate expected (source value, latent state) and (latent state, target value) counts for the low-rank association term.

```
class dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator(num_vals1,
                                                                           num_vals2,
                                                                           num_states,
                                                                           prev_acc=NoneAccumulator(),
                                                                           size_acc=NoneAccumulator(),
                                                                           use_numba=False,
                                                                           keys=(None,
                                                                           None))
```

Bases: [SequenceEncodableStatisticAccumulator](#)

Accumulate expected latent assignments and component statistics.

`weight_count[v, h]` records expected source-to-state assignments and `state_count[h, y]` records expected state-to-target assignments. `init_count` records source counts but is not used by the estimator. Source-bag and target-length statistics are delegated to their component accumulators.

Update responsibilities are normalized over source values and latent states for each target value. They describe the low-rank association component; the fixed uniform component selected by `alpha` is not included in these responsibilities.

Parameters

- `num_vals1` (*int*)
- `num_vals2` (*int*)
- `num_states` (*int*)
- `prev_acc` ([SequenceEncodableStatisticAccumulator](#) / *None*)
- `size_acc` ([SequenceEncodableStatisticAccumulator](#) / *None*)
- `use_numba` (*bool*)
- `keys` (*Tuple[str | None, str | None] | None*)

`acc_to_encoder()`

Create an encoder compatible with this accumulator.

Return type

[dmx.stats.pdist.DataSequenceEncoder](#)

`combine(suff_stat)`

Merge association, source-bag, and length statistics.

Return type*dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator***Parameters****suff_stat** (*Tuple*[*ndarray*, *ndarray*, *ndarray*, *SS1* | *None*, *SS2* | *None*])**from_value**(*x*)Replace all sufficient statistics with *x*.**Return type***dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator***Parameters****x** (*Tuple*[*ndarray*, *ndarray*, *ndarray*, *SS1* | *None*, *SS2* | *None*])**initialize**(*x*, *weight*, *rng*)

Initialize association statistics with random latent allocations.

Return type*None***Parameters**

- **x** (*Tuple*[*List*[*Tuple*[*int*, *float*]], *List*[*Tuple*[*int*, *float*]]])
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge keyed association and component statistics.

Return type*None***Parameters****stats_dict** (*Dict*[*str*, *Any*])**key_replace**(*stats_dict*)

Replace keyed association and component statistics.

Return type*None***Parameters****stats_dict** (*Dict*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *rng*)

Initialize statistics from an encoded weighted batch.

Return type*None***Parameters**

- **x** (*IntegerHiddenAssociationEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Add an encoded weighted batch using posterior assignment counts.

Return type

None

Parameters

- **x** (`IntegerHiddenAssociationEncodedDataSequence`)
- **weights** (`ndarray`)
- **estimate** (`IntegerHiddenAssociationDistribution`)

update(*x, weight, estimate*)

Add a weighted paired bag using posterior assignment counts.

Return type

None

Parameters

- **x** (`Tuple[List[Tuple[int, float]], List[Tuple[int, float]]]`)
- **weight** (`float`)
- **estimate** (`IntegerHiddenAssociationDistribution`)

value()

Return all association and component sufficient statistics.

Return type

`typing.Tuple[numpy.ndarray, numpy.ndarray, numpy.ndarray, typing.Optional[typing.Any], typing.Optional[typing.Any]]`

```
class dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulatorFactory(num_vals1,
                                                                                  num_vals2,
                                                                                  num_states,
                                                                                  prev_factory=NoneAccumulatorFactory,
                                                                                  len_factory=NoneAccumulatorFactory,
                                                                                  use_numba=False,
                                                                                  keys=(None, None))
```

Bases: `StatisticAccumulatorFactory`

Create integer hidden-association accumulators.

Parameters

- **num_vals1** (`int`)
- **num_vals2** (`int`)
- **num_states** (`int`)
- **prev_factory** (`StatisticAccumulatorFactory | None`)
- **len_factory** (`StatisticAccumulatorFactory | None`)
- **use_numba** (`bool`)
- **keys** (`Tuple[str | None, str | None]`)

make()

Create a new accumulator with fresh component accumulators.

Return type`dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator`

```
class dmx.stats.int_hidden_association.IntegerHiddenAssociationDataEncoder(prev_encoder,  
                                                                           len_encoder,  
                                                                           use_numba)
```

Bases: [DataSequenceEncoder](#)

Encode paired integer bags for vectorized or Numba-backed operations.

The regular representation stores four arrays per observation. The Numba representation flattens values and counts across observations and records per-observation unique-value counts and total source counts. Both forms also carry encoded source bags and target-count totals.

Parameters

- **prev_encoder** ([DataSequenceEncoder](#))
- **len_encoder** ([DataSequenceEncoder](#))
- **use_numba** (*bool*)

seq_encode(*x*)

Sequence encoding for integer hidden association observations.

If numba is not used see `_seq_encode()`. Else the following is returned a Tuple of the following form is returned None, ((s0, s1, x0, x1, c0, c1, w0), xv, nn) with,

s0 (np.ndarray): Numpy array of lengths for length of x[i][0] s1 (np.ndarray): Numpy array of lengths for length of x[i][1]. x0 (np.ndarray): Flattened numpy array of values from x[i][0]. x1 (np.ndarray): Flattened numpy array of values from x[i][1]. c0 (np.ndarray): Flattened numpy array of counts from x[i][0]. c1 (np.ndarray): Flattened numpy array of counts from x[i][1]. w0 (np.ndarray): Numpy array of sum of counts for each x[i][0]. xv (E1): Sequence encoded flattened values of x[i][0]. nn (E2): Sequence encoded values of lengths (counts).

Parameters

x – Sequence of iid integer hidden association observations.

Return type

[dmx.stats.int_hidden_association.IntegerHiddenAssociationEncodedDataSequence](#)

Returns

See above.

Parameters

x (*Sequence[Tuple[List[Tuple[int, float]], List[Tuple[int, float]]]*)

```
class dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution(state_prob_mat,  
                                                                           cond_weights,  
                                                                           alpha=0.0,  
                                                                           prev_dist=NullDistribution(),  
                                                                           len_dist=NullDistribution(),  
                                                                           name=None,  
                                                                           keys=(None,  
                                                                           None),  
                                                                           use_numba=False)
```

Bases: [SequenceEncodableProbabilityDistribution](#)

Model associations between finite-domain source and target bags.

`cond_weights` has shape (V1, K) and contains latent-state probabilities given source values. `state_prob_mat` has shape (K, V2) and contains target probabilities given latent states.

Parameters

- **state_prob_mat** (*List[List[float]]* | *ndarray*)
- **cond_weights** (*List[List[float]]* | *ndarray*)
- **alpha** (*float*)
- **prev_dist** (*SequenceEncodableProbabilityDistribution* | *None*)
- **len_dist** (*SequenceEncodableProbabilityDistribution* | *None*)
- **name** (*str* | *None*)
- **keys** (*Tuple[str | None, str | None]*)
- **use_numba** (*bool*)

cond_weights

States given words in S1.

Type

np.ndarray

state_prob_mat

Words in S2 given States.

Type

np.ndarray

len_dist

Distribution for length of observations. Should be compatible with type *Tuple[int, int]*.

Type

SequenceEncodableProbabilityDistribution

prev_dist

Distribution for given P(S1). Should be compatible with *Tuple[int, float]*.

Type

SequenceEncodableProbabilityDistribution

has_prev_dist

True if there is a non-null prev_dist specified.

Type

bool

num_vals2

Number of values in S2.

Type

int

num_vals1

Number of values in S1.

Type

int

num_states

Number of hidden states.

Type

int

alpha

Probability of drawing from uniform vs transition density.

Type

float

name

Set name for object.

Type

Optional[str]

keys

Keys for the weights and states.

Type

Tuple[Optional[str], Optional[str]]

init_prob_vec

initial prob vector.

Type

np.ndarray

use_numba

If True, numba is used for encoding and estimation.

Type

bool

density(*x*)

Evaluate the density of a paired source and target bag.

Return type

float

Parameters

x (Tuple[List[Tuple[int, float]], List[Tuple[int, float]]])

dist_to_encoder()

Create an encoder with this distribution's component encoders.

Return type

dmx.stats.int_hidden_association.IntegerHiddenAssociationDataEncoder

estimator(*pseudo_count=None*)

Create an estimator with matching domains and latent-state count.

Return type

dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator

Parameters

pseudo_count (float | None)

log_density(*x*)

Evaluate log density after marginalizing assignments and latent states.

Return type

float

Parameters

x (Tuple[List[Tuple[int, float]], List[Tuple[int, float]]])

sampler(*seed=None*)

Create a sampler, requiring non-null source and length models.

Return type

`dmx.stats.int_hidden_association.IntegerHiddenAssociationSampler`

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Evaluate log densities for an encoded batch of paired integer bags.

The regular encoding follows `log_density()`. The historical Numba kernel evaluates only the low-rank association term and does not apply the alpha uniform mixture.

Return type

`numpy.ndarray`

Parameters

x (`IntegerHiddenAssociationEncodedDataSequence`)

class `dmx.stats.int_hidden_association.IntegerHiddenAssociationEncodedDataSequence`(*data*,
use_numba=False)

Bases: `EncodedDataSequence`

Store an encoded batch of paired integer-bag observations.

Notes

`E0 = Tuple[List[Tuple[np.ndarray, ...]], EncodedDataSequence, EncodedDataSequence]` `E1 = Tuple[Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray], EncodedDataSequence, EncodedDataSequence]`

Parameters

- **data** (`Tuple[List[Tuple[ndarray, ...]], EncodedDataSequence, EncodedDataSequence]` | `Tuple[Tuple[ndarray, ndarray, ndarray, ndarray, ndarray, ndarray], EncodedDataSequence, EncodedDataSequence]`)
- **use_numba** (*bool*)

data

Encoded data. E1 is for numba use.

Type

`Union[E0, E1]`

numba_enc

If True, a numba encoding was passed.

Type

`bool`

```
class dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator(num_vals,
                                                                    num_states,
                                                                    alpha=0.0,
                                                                    prev_estimator=NoneEstimator(),
                                                                    len_estimator=NoneEstimator(),
                                                                    suff_stat=None,
                                                                    pseudo_count=None,
                                                                    use_numba=False,
                                                                    name=None,
                                                                    keys=(None, None))
```

Bases: [ParameterEstimator](#)

Estimate an integer hidden-association distribution.

Rows of the source-to-state and state-to-target expected-count matrices are normalized independently. If `pseudo_count` is set, uniform mass is added to both matrices in place before normalization. Source-bag and target-length distributions are estimated from their component statistics; `init_count` and `suff_stat` are retained for compatibility but do not affect the fitted distribution.

Parameters

- **num_vals** (*List[int] | Tuple[int, int] | int*)
- **num_states** (*int*)
- **alpha** (*float*)
- **prev_estimator** (*ParameterEstimator | None*)
- **len_estimator** (*ParameterEstimator | None*)
- **suff_stat** (*Any | None*)
- **pseudo_count** (*float | None*)
- **use_numba** (*bool*)
- **name** (*str | None*)
- **keys** (*Tuple[str | None, str | None] | None*)

num_vals

Number of values in S1 and S2. Either length 2, if int value is set to num_vals1 and num_vals2.

Type

Union[List[int], Tuple[int, int], int]

num_states

Number of hidden states.

Type

int

alpha

Prob of drawing from uniform, (1-alpha) draw from transition density.

Type

float

prev_estimator

Estimator for the previous word set. Must be compatible with Tuple[int, float]. Defaults to NoneEstimator().

Type*ParameterEstimator***len_estimator**

Estimator for the length of observations. Must be compatible with `Tuple[int, int]`. Defaults to `NullEstimator()`.

Type*ParameterEstimator***suff_stat**

Kept for consistency.

Type`Optional[Any]`**pseudo_count**

Kept for consistency.

Type`Optional[float]`**use_numba**

If true Numba is used for encoding and vectorized function calls.

Type`bool`**name**

Set a name to the object instance.

Type`Optional[str]`**keys**

Set the keys for weights and transitions.

Type`Tuple[Optional[str], Optional[str]]`**num_vals1**

Number of values in set 1.

Type`int`**num_vals2**

Number of values in set 2.

Type`int`**accumulator_factory()**

Create a factory for compatible sufficient-statistic accumulators.

Return type*`dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulatorFactory`***estimate(*nobs*, *suff_stat*)**

Estimate model matrices and component distributions.

Return type

`dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution`

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *ndarray*, *ndarray*, *SS1* | *None*, *SS2* | *None*])

class `dmx.stats.int_hidden_association.IntegerHiddenAssociationSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`

Sample paired bags or target bags conditional on a source bag.

Parameters

- **dist** (`IntegerHiddenAssociationDistribution`)
- **seed** (*int* | *None*)

sample(*size=None*)

Draw one paired-bag observation or a batch of observations.

Return type

`typing.Union[typing.Sequence[typing.Tuple[typing.List[typing.Tuple[int, float]], typing.List[typing.Tuple[int, float]]], typing.Tuple[typing.List[typing.Tuple[int, float]], typing.List[typing.Tuple[int, float]]]]`

Parameters

size (*int* | *None*)

sample_given(*x*)

Draw a grouped target bag conditional on source bag *x*.

Return type

`typing.List[typing.Tuple[int, float]]`

Parameters

x (*List*[*Tuple*[*int*, *float*]])

`dmx.stats.int_hidden_association.numba_seq_log_density`(*num_states*, *max_len1*, *t0*, *t1*, *x0*, *x1*, *c0*, *c1*, *w0*, *cond_weights*, *state_prob_mat*, *init_prob_vec*, *a*, *b*, *out*)

Evaluate low-rank association log densities into *out*.

Return type

None

Parameters

- **num_states** (*int*)
- **max_len1** (*int*)
- **t0** (*ndarray*)
- **t1** (*ndarray*)
- **x0** (*ndarray*)
- **x1** (*ndarray*)
- **c0** (*ndarray*)
- **c1** (*ndarray*)

- **w0** (*ndarray*)
- **cond_weights** (*ndarray*)
- **state_prob_mat** (*ndarray*)
- **init_prob_vec** (*ndarray*)
- **a** (*float*)
- **b** (*float*)
- **out** (*ndarray*)

`dmx.stats.int_hidden_association.numba_seq_update`(*num_states*, *max_len1*, *t0*, *t1*, *x0*, *x1*, *c0*, *c1*, *w0*, *cond_weights*, *state_prob_mat*, *weight_count*, *state_count*, *init_count*, *weights*)

Accumulate Numba-backed expected association counts in place.

Return type

None

Parameters

- **num_states** (*int*)
- **max_len1** (*int*)
- **t0** (*ndarray*)
- **t1** (*ndarray*)
- **x0** (*ndarray*)
- **x1** (*ndarray*)
- **c0** (*ndarray*)
- **c1** (*ndarray*)
- **w0** (*ndarray*)
- **cond_weights** (*ndarray*)
- **state_prob_mat** (*ndarray*)
- **weight_count** (*ndarray*)
- **state_count** (*ndarray*)
- **init_count** (*ndarray*)
- **weights** (*ndarray*)

`dmx.stats.int_hidden_association.vec_bincount1`(*x*, *w*, *out*)

Numba bincount on the rows of matrix w for groups x.

Parameters

- **x** (*np.ndarray[np.float64]*) – Group ids of rows
- **w** (*np.ndarray[np.float64]*) – N by S numpy array with rows corresponding to x
- **out** (*np.ndarray[np.float64]*) – Unique values in support of x by S.

Return type

`numpy.ndarray`

Returns

Numpy 2-d array.

Parameters

- **x** (*ndarray*)
- **w** (*ndarray*)
- **out** (*ndarray*)

`dmx.stats.int_hidden_association.vec_bincount2(x, w, out)`

Numba bincount on the rows of matrix w for groups x.

N = len(x) S = number of states. U = unique values in x can take on.

Parameters

- **x** (*np.ndarray[np.float64]*) – Group ids of columns of w.
- **w** (*np.ndarray[np.float64]*) – S by N numpy array with cols corresponding to x
- **out** (*np.ndarray[np.float64]*) – S by U matrix.

Return type

`numpy.ndarray`

Returns

Numpy 2-d array.

Parameters

- **x** (*ndarray*)
- **w** (*ndarray*)
- **out** (*ndarray*)

Integer Hidden Markov Model

Provide finite-state HMMs with categorical integer emissions.

This module specializes `dmx.stats.hidden_markov` to observations in `range(num_words)`. It uses the same zero-based latent states, initial probabilities `w`, row-oriented transition matrix `transitions`, and sequence-length model. The integer-specific emission matrix satisfies `pmat[value, state] = P(Xt=value | Zt=state)`.

```
class dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator(num_words, num_states,
                                                                len_accumulator=NoneAccumulator(),
                                                                keys=(None, None, None),
                                                                name=None)
```

Bases: [SequenceEncodableStatisticAccumulator](#)

Accumulate expected statistics for integer-emission HMM sequences.

The sufficient statistic is `(init_counts, state_counts, trans_counts, emission_counts, length_statistic)`. `emission_counts` has shape `(num_words, num_states)`. Forward-backward updates fill the initial, transition, and emission counts; `state_counts` is retained for protocol compatibility but is not used by the integer estimator.

Parameters

- **num_words** (*int*)
- **num_states** (*int*)
- **len_accumulator** ([SequenceEncodableStatisticAccumulator](#) / *None*)

- **keys** (*Tuple[str | None, str | None, str | None]*)
- **name** (*str | None*)

num_words

Size of the integer observation domain.

Type
int

wcnts

Expected integer-emission counts by latent state.

Type
np.ndarray

num_states

Total number of hidden states.

Type
int

init_counts

Track $\gamma_i(0)$, or first time point γ for each component in Baum-Welch.

Type
ndarray

trans_counts

2-d matrix tracking transition updates from Baum-Welch ($\sum_t \psi_{ij}(t) / \sum_t \gamma_i(t)$).

Type
ndarray

state_counts

Expected number of times state is observed in sequence from $t=0$ to $t=T-2$.

Type
ndarray

len_accumulator

SequenceEncodableStatisticAccumulator object for the length distribution. Set to NullAccumulator is None is passed.

Type
SequenceEncodableStatisticAccumulator

init_key

Key for initial states.

Type
Optional[str]

trans_key

Key for state transitions.

Type
Optional[str]

state_key

Key for emission accumulators..

Type

Optional[str]

name

Name for object.

Type

Optional[str]

_init_rng

True if RandomState objects have been initialized

Type

bool

_len_rng

RandomState for initializing length accumulator.

Type

Optional[RandomState]

_idx_rng

RandomState for initializing initial state draws.

Type

Optional[RandomState]

acc_to_encoder()

Return a IntegerHiddenMarkovDataEncoder for this accumulator.

Return type

dmx.stats.int_hidden_markov.IntegerHiddenMarkovDataEncoder

Returns

IntegerHiddenMarkovDataEncoder – Encoder object.

combine(suff_stat)

Aggregate sufficient statistics with this accumulator.

Parameters

suff_stat – Initial-state, state, transition, emission, and length sufficient statistics to combine.

Return type

dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator

Returns

This accumulator after combining.

Parameters

suff_stat (*Tuple*[ndarray, ndarray, ndarray, ndarray, T2 | None])

from_value(x)

Set the sufficient statistics from a tuple.

Parameters

x (*Tuple*[np.ndarray, np.ndarray, np.ndarray, np.ndarray, Optional[T2]]) – Sufficient statistics.

Return type*dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator***Returns***IntegerHiddenMarkovAccumulator* – Self after setting values.**Parameters****x** (*Tuple*[*ndarray*, *ndarray*, *ndarray*, *ndarray*, *T2* | *None*])**initialize**(*x*, *weight*, *rng*)

Initialize the accumulator with a single HMM observation sequence.

Parameters

- **x** (*Sequence*[*int*]) – Observed sequence.
- **weight** (*float*) – Weight for the observation.
- **rng** (*RandomState*) – Random number generator.

Return type*None***Parameters**

- **x** (*Sequence*[*int*])
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge this accumulator into a dictionary by key.

Parameters**stats_dict** (*Dict*[*str*, *Any*]) – Dictionary of accumulators.**Return type***None***Parameters****stats_dict** (*Dict*[*str*, *Any*])**key_replace**(*stats_dict*)

Replace this accumulator's values with those from a dictionary by key.

Parameters**stats_dict** (*Dict*[*str*, *Any*]) – Dictionary of accumulators.**Return type***None***Parameters****stats_dict** (*Dict*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *rng*)

Vectorized initialization for encoded HMM data.

Initialization assigns a random hidden state to each encoded integer observation, then accumulates initial-state counts, transition counts, state counts, integer emission counts, and length statistics from those sampled assignments. It is a randomized start, not a posterior update from an existing HMM estimate.

Parameters

- **x** (*HiddenMarkovEncodedDataSequence*) – Encoded HMM data sequence.

- **weights** (*np.ndarray*) – Weights for each sequence.
- **rng** (*np.random.RandomState*) – Random number generator.

Return type

None

Parameters

- **x** (*IntegerHiddenMarkovEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x, weights, estimate*)

Apply weighted forward-backward updates to an encoded batch.

Return type

None

Parameters

- **x** (*IntegerHiddenMarkovEncodedDataSequence*)
- **weights** (*ndarray*)
- **estimate** (*IntegerHiddenMarkovModelDistribution*)

update(*x, weight, estimate*)

Update the accumulator with a single HMM observation sequence.

Parameters

- **x** (*Sequence[int]*) – Observed sequence.
- **weight** (*float*) – Weight for the observation.
- **estimate** (*HiddenMarkovModelDistribution*) – Current HMM distribution estimate.

Return type

None

Parameters

- **x** (*Sequence[int]*)
- **weight** (*float*)
- **estimate** (*IntegerHiddenMarkovModelDistribution*)

value()

Return the sufficient statistics as a tuple.

Return type

typing.Tuple[*numpy.ndarray*, *numpy.ndarray*, *numpy.ndarray*, *numpy.ndarray*,
typing.Optional[*typing.Any*]]

Returns

Tuple[*np.ndarray*, *np.ndarray*, *np.ndarray*, *Optional*[*Any*]] – Sufficient statistics.

```
class dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulatorFactory(num_words,
                                                                       num_states,
                                                                       len_factory=NullAccumulatorFactory(),
                                                                       keys=(None, None,
                                                                       None), name=None)
```

Bases: `StatisticAccumulatorFactory`

Create integer-HMM sufficient-statistic accumulators.

Parameters

- **num_words** (*int*)
- **num_states** (*int*)
- **len_factory** (*StatisticAccumulatorFactory*)
- **keys** (*Tuple[str | None, str | None, str | None] | None*)
- **name** (*str | None*)

num_words

Size of the integer observation domain.

num_states

Number of latent states.

len_factory

Factory for the length distribution.

Type

StatisticAccumulatorFactory

keys

Keys for initial states, transitions, and emissions.

Type

Tuple[Optional[str], Optional[str], Optional[str]]

name

Name for the object.

Type

Optional[str]

make()

Create a new integer-HMM accumulator.

Return type

dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator

Returns

IntegerHiddenMarkovAccumulator – A new accumulator instance.

class `dmx.stats.int_hidden_markov.IntegerHiddenMarkovDataEncoder` (*len_encoder=NullDataEncoder()*)

Bases: *DataSequenceEncoder*

Encode batches of integer HMM sequences into flat arrays.

Observations are concatenated into one integer array `xs`. The boundary array `tz` has length `num_sequences + 1` and selects sequence `i` as `xs[tz[i]:tz[i + 1]]`. The third field is the component encoding of the original sequence lengths.

Parameters

len_encoder (*DataSequenceEncoder | None*)

len_encoder

DataSequenceEncoder object for the length of sequences. Should have support of non-negative integers. Set to NullDataEncoder if None.

Type

DataSequenceEncoder

seq_encode(*x*)

Encode independent integer HMM observation sequences.

Parameters

x (*Sequence[Sequence[int]]*) – Batch of integer observation sequences.

Return type

dmx.stats.int_hidden_markov.IntegerHiddenMarkovEncodedDataSequence

Returns

Flattened observations, cumulative sequence boundaries, and encoded sequence lengths.

Parameters

x (*Sequence[Sequence[int]]*)

class *dmx.stats.int_hidden_markov.IntegerHiddenMarkovEncodedDataSequence*(*data*)

Bases: *EncodedDataSequence*

Store flattened integer observations, boundaries, and encoded lengths.

Notes

E is *Tuple[np.ndarray, np.ndarray, EncodedDataSequence]*.

Parameters

data (*Tuple[np.ndarray, np.ndarray, EncodedDataSequence]*)

data

Flattened observations, boundaries, and encoded lengths.

Type

E

class *dmx.stats.int_hidden_markov.IntegerHiddenMarkovEstimator*(*num_words*, *num_states*,
len_estimator=*NullEstimator*(),
pseudo_count=(*None*, *None*,
None), *name*=*None*, *keys*=(*None*,
None, *None*))

Bases: *ParameterEstimator*

Estimate an integer-emission HMM from expected counts.

Initial-state and transition estimation matches the generic HMM estimator. The integer specialization additionally normalizes each state column of the (*num_words*, *num_states*) emission-count matrix directly.

Notes

Unlike the generic *HiddenMarkovEstimator*, this specialized integer HMM has a three-slot *pseudo_count* tuple:

- *pseudo_count*[0] smooths initial-state counts uniformly across states.
- *pseudo_count*[1] smooths transition counts uniformly across state pairs.
- *pseudo_count*[2] smooths integer emission counts uniformly across *num_words* for each hidden state.

Length regularization is controlled by `len_estimator`. The emitted integer probabilities are estimated directly from the accumulated emission-count matrix, so there are no child emission estimators to configure.

Parameters

- **num_words** (*int*)
- **num_states** (*int*)
- **len_estimator** (*ParameterEstimator | None*)
- **pseudo_count** (*Tuple[float | None, float | None, float | None] | None*)
- **name** (*str | None*)
- **keys** (*Tuple[str | None, str | None, str | None] | None*)

num_words

Size of the integer observation domain.

Type

int

num_states

Number of latent states.

Type

int

len_estimator

Estimator for length distribution.

Type

ParameterEstimator

pseudo_count

Pseudo counts for initial states, transitions, and integer emissions.

Type

Tuple[Optional[float], Optional[float], Optional[float]]

name

Name for the object instance.

Type

Optional[str]

keys

Keys for initial states, transitions, and emissions.

Type

Tuple[Optional[str], Optional[str], Optional[str]]

accumulator_factory()

Return a factory for compatible integer-HMM accumulators.

Return type

dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulatorFactory

Returns

IntegerHiddenMarkovAccumulatorFactory – The accumulator factory.

estimate(*nobs*, *suff_stat*)

Estimate an integer-emission HMM from aggregated statistics.

Parameters

- **nobs** – Number of observations.
- **suff_stat** – Tuple containing initial-state counts, state counts, transition counts, integer emission counts, and length sufficient statistics.

Return type*dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution***Returns***IntegerHiddenMarkovModelDistribution* – The estimated distribution.**Parameters**

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *ndarray*, *ndarray*, *ndarray*, *T2* | *None*])

```
class dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution(pmat, w, transitions,  
                                                                    len_dist=NullDistribution(),  
                                                                    name=None,  
                                                                    keys=(None, None,  
                                                                    None), terminal_values=None)
```

Bases: *SequenceEncodableProbabilityDistribution*

Represent an HMM whose observations index a categorical emission matrix.

Valid observations are integer sequences with every value in $0 \leq \text{value} < \text{num_words}$. *pmat* has shape (*num_words*, *n_states*); each column is the categorical distribution emitted by one latent state. Other HMM conventions match *HiddenMarkovModelDistribution*.

Parameters

- **pmat** (*List*[*List*[*float*]] | *ndarray*)
- **w** (*Sequence*[*float*] | *ndarray*)
- **transitions** (*List*[*List*[*float*]] | *ndarray*)
- **len_dist** (*SequenceEncodableProbabilityDistribution* | *None*)
- **name** (*str* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*, *str* | *None*] | *None*)
- **terminal_values** (*Set*[*T*] | *None*)

pmatEmission probability matrix where *pmat*[*i*,*j*] is the probability of emitting integer *i* from state *j*.**Type***np.ndarray***log_pmat**

Log of emission probability matrix.

Type*np.ndarray*

n_words

Number of possible integer emissions (vocabulary size).

Type

int

n_states

Number of hidden states.

Type

int

w

Initial state probabilities.

Type

np.ndarray

log_w

Initial state log-probabilities.

Type

np.ndarray

transitions

2-d array of hidden state transition probabilities with shape (n_states, n_states).

Type

np.ndarray

log_transitions

Log of transition probabilities.

Type

np.ndarray

terminal_values

Set of values that terminate the HMM sequence when sampling.

Type

Optional[Set[T]]

name

Name of the distribution instance.

Type

Optional[str]

len_dist

Distribution for sequence lengths. Defaults to NullDistribution.

Type

SequenceEncodableProbabilityDistribution

keys

Keys for initial states, transitions counts, and emission distributions.

Type

Tuple[Optional[str], Optional[str], Optional[str]]

density(*x*)

Returns the density of HMM for an observed sequence *x*.

See [`log\_density\(\)`](#) for details.

Parameters

x (*Sequence[int]*) – Observed sequence of HMM emissions.

Return type

float

Returns

float – Density of HMM for observed sequence *x*.

Parameters

x (*Sequence[int]*)

dist_to_encoder()

Create an encoder for batches of integer observation sequences.

Return type

dmx.stats.int_hidden_markov.IntegerHiddenMarkovDataEncoder

estimator(*pseudo_count=None*)

Create an estimator with uniform smoothing for all parameter blocks.

Return type

dmx.stats.int_hidden_markov.IntegerHiddenMarkovEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Returns the log-density of HMM for observed sequence *x*.

Parameters

x (*Sequence[int]*) – Observed sequence of HMM emissions.

Return type

float

Returns

float – Log-density of observed HMM sequence *x*.

Parameters

x (*Sequence[int]*)

sampler(*seed=None*)

Create a sampler using sequence lengths or terminal values.

Return type

dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Evaluate log densities for a flattened batch of integer sequences.

Return type

numpy.ndarray

Parameters**x** (`IntegerHiddenMarkovEncodedDataSequence`)**seq_posterior(x)**

Compute smoothed latent-state probabilities for encoded sequences.

Each returned matrix has shape (sequence_length, n_states) and entry [t, i] equal to $P(Z_t = i \mid x_0, \dots, x_{n-1})$.

Parameters**x** (`IntegerHiddenMarkovEncodedDataSequence`) – Encoded integer HMM sequences.**Return type**`typing.List[numpy.ndarray]`**Returns**

`List[np.ndarray]` – A list of posterior probabilities for each latent state for each observation sequence.

Parameters**x** (`IntegerHiddenMarkovEncodedDataSequence`)**viterbi(x)**

Return the globally maximizing latent-state path.

This integer implementation stores predecessor states and performs a full traceback. An empty observation sequence returns an empty integer array.

Parameters**x** (`Sequence[int]`) – Single HMM sequence.**Return type**`numpy.ndarray`**Returns**

One zero-based latent-state index per observation.

Parameters**x** (`Sequence[int]`)

class `dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`

Sample integer observation sequences from the HMM.

With a length sampler, each call first samples the sequence length and latent path, then draws integer observations from the corresponding columns of `pmat`. Terminal sampling stops after emitting the first value in the supplied terminal set.

Parameters

- **dist** (`IntegerHiddenMarkovModelDistribution`)
- **seed** (`int` | `None`)

num_states

Number of hidden states in ‘dist’ object.

Type`int`

dist

HiddenMarkovModelDistribution object instance to sample from.

Type

HiddenMarkovModelDistribution

rng

RandomState object with seed set for sampling.

Type

RandomState

len_sampler

DistributionSampler object with data type int and support on non-negative integers for sampling HMM observation sequence lengths.

Type

Optional[*DistributionSampler*]

terminal_set

Set of values to terminate HMM sampling when calling 'sample_seq()'.

Type

Optional[Set[T]]

state_sampler

MarkovChainSampler for sampling states of HMM.

Type

MarkovChainSampler

sample(size=None)

Draw iid samples from HMM.

If a 'len_sampler' is set, call 'sample_seq()' (See HiddenMarkovSampler.sample_seq() for details). If 'len_sampler' is the NullDistributionSampler(), 'sample_terminal()' is called. (See HiddenMarkovSampler.sample_terminal() for details).

Parameters

size (Optional[int]) – Number of iid HMM sequences to sample.

Return type

typing.Union[list[int], list[list[int]]]

Returns

List[int] or List[List[int]] depending on arg size.

Parameters

size (int | None)

sample_int(i)

Sample one integer observation from latent state i.

Parameters

i (int) – Zero-based latent-state index.

Return type

int

Returns

An integer in range(num_words).

Parameters**i** (*int*)**sample_seq**(*size=None*)

Sample iid HMM sequences.

If size is None, 1 sample is drawn and a List[T] is returned. If size > 0, 'size' samples are drawn and a List of length 'size' with HMM sequences (List[T]) is returned.

Parameters**size** (*Optional[int]*) – Number of iid HMM sequences to sample.**Return type**

typing.Union[list[int], list[list[int]]]

Returns

List[T] or List[List[T]] depending on size arg.

Parameters**size** (*int | None*)**sample_terminal**(*terminal_set*)

Sample through and including the first terminal integer.

Parameters**terminal_set** (*Set[int]*) – Set values to terminate the HMM sequence.**Return type**

typing.List[int]

Returns

List[int] with length determined by samples to reach the first terminating value.

Parameters**terminal_set** (*Set[T]*)**dmx.stats.int_hidden_markov.fast_log_density**(*xs, tz, log_pmat, log_pi, log_A, out*)

Compute log densities for flattened categorical-emission sequences.

Parameters

- **xs** – Flattened integer observations.
- **tz** – Cumulative sequence boundaries.
- **log_pmat** – Log emission probabilities with shape (V, S).
- **log_pi** – Log initial-state probabilities with shape (S,).
- **log_A** – Log transition probabilities with shape (S, S).
- **out** – Output log densities with one entry per sequence.

Return type

None

Parameters

- **xs** (*ndarray*)
- **tz** (*ndarray*)
- **log_pmat** (*ndarray*)
- **log_pi** (*ndarray*)

- **log_A** (*ndarray*)
- **out** (*ndarray*)

`dmx.stats.int_hidden_markov.fast_seq_initialize(xs, tz, weights, states, icnts, scnts, tcnts, wcnts)`

Process data sequentially to ensure deterministic results.

Parameters

- **xs** – Flattened array of observations
- **tz** – Cumulative indices marking sequence boundaries
- **weights** – Weights for each sequence
- **states** – States for each observation
- **icnts** – Initial state counts buffer with shape (num_states)
- **scnts** – State counts buffer with shape (num_states)
- **tcnts** – Transition counts buffer with shape (num_states, num_states)
- **wcnts** – Emission-count buffer with shape (num_words, num_states).

Return type

None

Parameters

- **xs** (*ndarray*)
- **tz** (*ndarray*)
- **weights** (*ndarray*)
- **states** (*ndarray*)
- **icnts** (*ndarray*)
- **scnts** (*ndarray*)
- **tcnts** (*ndarray*)
- **wcnts** (*ndarray*)

`dmx.stats.int_hidden_markov.forward_backward(xs_seq, init_pvec, tran_mat, pmat, log_alpha, log_beta)`

Run the log-space forward-backward algorithm for one sequence.

Parameters

- **xs_seq** – numpy array of shape (T,) containing emission observations
- **init_pvec** – numpy array of shape (S,) initial state distribution
- **tran_mat** – numpy array of shape (S, S) transition matrix
- **pmat** – numpy array of shape (W, S) emission matrix $P(W=i | Z=z)$
- **log_alpha** – pre-allocated output array (T, S) for log forward probabilities
- **log_beta** – pre-allocated output array (T, S) for log backward probabilities

Return type

None

Parameters

- **xs_seq** (*ndarray*)

- **init_pvec** (*ndarray*)
- **tran_mat** (*ndarray*)
- **pmat** (*ndarray*)
- **log_alpha** (*ndarray*)
- **log_beta** (*ndarray*)

`dmx.stats.int_hidden_markov.logsumexp_2d_rows(arr, result)`

Logsumexp along axis=1 for 2D array.

Return type

None

Parameters

- **arr** (*ndarray*)
- **result** (*ndarray*)

`dmx.stats.int_hidden_markov.logsumexp_numba(arr)`

Numerically stable logsumexp for 1D array.

Return type

float

Parameters

arr (*ndarray*)

`dmx.stats.int_hidden_markov.numba_baum_welch(xs, tz, weights, init_pvec, tran_mat, pmat, init_counts, tran_counts, emit_counts)`

Compute batched Baum-Welch sufficient statistics in parallel.

Each sequence's posterior initial, transition, and integer-emission counts are multiplied by its observation weight.

Parameters

- **xs** – numpy array int32 of emission observations
- **tz** – numpy array int64 containing start/end indices (len(tz)-1 sequences)
- **weights** – Weight for each encoded observation sequence.
- **init_pvec** – numpy array float64 of shape (S,) initial state distribution
- **tran_mat** – numpy array float64 of shape (S, S) transition matrix
- **pmat** – numpy array float64 of shape (W, S) emission matrix
- **init_counts** – output array float64 of shape (S,) for initial state counts
- **tran_counts** – output array float64 of shape (S, S) for transition counts
- **emit_counts** – output array float64 of shape (W, S) for emission counts

Return type

None

Parameters

- **xs** (*ndarray*)
- **tz** (*ndarray*)
- **weights** (*ndarray*)

- **init_pvec** (*ndarray*)
- **tran_mat** (*ndarray*)
- **pmat** (*ndarray*)
- **init_counts** (*ndarray*)
- **tran_counts** (*ndarray*)
- **emit_counts** (*ndarray*)

`dmx.stats.int_hidden_markov.numba_baum_welch_alphas`(*num_states*, *tz*, *prob_mat*, *init_pvec*, *tran_mat*, *weights*, *alpha_loc*, *xi_acc*, *pi_acc*)

Parallelized Baum-Welch forward-backward algorithm for HMM parameter estimation.

Parameters

- **num_states** (*int*) – Number of hidden states.
- **tz** (*np.ndarray*) – Cumulative sum of sequence lengths.
- **prob_mat** (*np.ndarray*) – Observation likelihoods for each state.
- **init_pvec** (*np.ndarray*) – Initial state probabilities.
- **tran_mat** (*np.ndarray*) – State transition matrix.
- **weights** (*np.ndarray*) – Sequence weights.
- **alpha_loc** (*np.ndarray*) – Forward probabilities.
- **xi_acc** (*np.ndarray*) – Accumulator for transition probabilities.
- **pi_acc** (*np.ndarray*) – Accumulator for initial state probabilities.

Return type

None

Parameters

- **num_states** (*int*)
- **tz** (*ndarray*)
- **prob_mat** (*ndarray*)
- **init_pvec** (*ndarray*)
- **tran_mat** (*ndarray*)
- **weights** (*ndarray*)
- **alpha_loc** (*ndarray*)
- **xi_acc** (*ndarray*)
- **pi_acc** (*ndarray*)

Integer Markov Chain

Provide fixed-lag Markov chains over a finite integer state space.

An observation is a finite state sequence $x = (x_0, \dots, x_{n-1})$ with $x_i \in \{0, \dots, K-1\}$. For lag L and $n \geq L$,

$$p(x) = p_N(n) p_0(x_{0:L}) \prod_{i=0}^{n-L-1} p(x_{i+L} \mid x_{i:i+L}).$$

`init_dist` models the initial length-lag block, while row `ravel_multi_index(context, [num_values] * lag)` of `cond_dist` models the next state. Scalar density evaluation and sampling interpret `len_dist` as the distribution of `n`. Accumulator updates and encoded operations instead pass `max(n - lag + 1, 0)` to the length component; random accumulator initialization passes `n - lag` for sequences of at least `lag` states and does not update it for shorter sequences. A sequence shorter than `lag` has no initial or transition term, and sampling such a requested length returns the empty sequence. These historical semantics are intentionally unchanged.

The module provides the distribution, sampler, estimator, sufficient-statistic accumulator, and vectorized data encoder.

```
class dmx.stats.int_markovchain.IntegerMarkovChainAccumulator(lag,
                                                             init_accumulator=NoneAccumulator(),
                                                             len_accumulator=NoneAccumulator(),
                                                             keys=None, name=None)
```

Bases: [*SequenceEncodableStatisticAccumulator*](#)

Accumulate weighted statistics for fixed-lag integer chains.

The sufficient statistic contains transition counts keyed by `(context_tuple, next_state)`, followed by the initial-block and length component statistics. Only sequences of at least `lag` states contribute an initial block, and only later states contribute transitions.

Parameters

- **lag** (*int*)
- **init_accumulator** ([*SequenceEncodableStatisticAccumulator*](#) / *None*)
- **len_accumulator** ([*SequenceEncodableStatisticAccumulator*](#) / *None*)
- **keys** (*str* / *None*)
- **name** (*str* / *None*)

lag

The lag for the Markov chain.

Type

int

trans_count_map

Dictionary for tracking transition counts.

Type

Dict[Tuple[Sequence[int], int], float]

init_accumulator

Accumulator for the initial distribution. Should be a sequence compatible accumulator with support on the integers. Defaults to the `NullAccumulator`.

Type

[*SequenceEncodableStatisticAccumulator*](#)

len_accumulator

Accumulator for the length of the observed sequences. Should be a sequence compatible accumulator with support on the non-negative integers. Defaults to the `NullAccumulator`.

Type

[*SequenceEncodableStatisticAccumulator*](#)

max_value

Largest value encountered when accumulating sufficient statistics.

Type

int

keys

Set key for merging sufficient statistics with objects possessing matching key.

Type

Optional[str]

name

Set name for object.

Type

Optional[str]

_init_rng

True if RandomState objects for accumulator have been initialized.

Type

bool

_acc_rng

RandomState object for initializing the init accumulator.

Type

Optional[RandomState]

_len_rng

RandomState object for initializing the length accumulator.

Type

Optional[RandomState]

acc_to_encoder()

Create an encoder compatible with this accumulator.

Return type

dmx.stats.int_markovchain.IntegerMarkovChainDataEncoder

combine(suff_stat)

Merge another set of sufficient statistics into this accumulator.

Return type

dmx.stats.int_markovchain.IntegerMarkovChainAccumulator

Parameters

suff_stat (*Tuple*[*Dict*[*Tuple*[*Tuple*[int, ...], int], float], *SS1* | *None*, *SS2* | *None*])

from_value(x)

Replace the accumulated sufficient statistics with x.

Return type

dmx.stats.int_markovchain.IntegerMarkovChainAccumulator

Parameters

x (*Tuple*[*Dict*[*Tuple*[*Tuple*[int, ...], int], float], *SS1* | *None*, *SS2* | *None*])

initialize(*x*, *weight*, *rng*)

Initialize statistics from a weighted integer state sequence.

Return type

None

Parameters

- **x** (*Sequence[int]*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge statistics into entries selected by configured keys.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

key_replace(*stats_dict*)

Replace statistics from entries selected by configured keys.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

seq_initialize(*x*, *weights*, *rng*)

Initialize statistics from an encoded weighted batch.

Return type

None

Parameters

- **x** (*IntegerMarkovChainEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Add an encoded weighted batch to the statistics.

Return type

None

Parameters

- **x** (*IntegerMarkovChainEncodedDataSequence*)
- **weights** (*ndarray*)
- **estimate** (*IntegerMarkovChainDistribution* / *None*)

update(*x*, *weight*, *estimate*)

Add a weighted integer state sequence to the statistics.

Return type

None

Parameters

- **x** (*Sequence[int]*)
- **weight** (*float*)
- **estimate** (*IntegerMarkovChainDistribution* / *None*)

value()

Return transition, initial-block, and length statistics.

Return type

*typing.Tuple[typing.Dict[typing.Tuple[typing.Tuple[int, ...], int], float],
typing.Optional[typing.Any], typing.Optional[typing.Any]]*

```
class dmx.stats.int_markovchain.IntegerMarkovChainAccumulatorFactory(lag,  
                                                                    init_factory=NoneAccumulatorFactory(),  
                                                                    len_factory=NoneAccumulatorFactory(),  
                                                                    keys=None, name=None)
```

Bases: *StatisticAccumulatorFactory*

Create accumulators for fixed-lag integer Markov chains.

Parameters

- **lag** (*int*)
- **init_factory** (*StatisticAccumulatorFactory* / *None*)
- **len_factory** (*StatisticAccumulatorFactory* / *None*)
- **keys** (*str* / *None*)
- **name** (*str* / *None*)

lag

Length of lag in Markov chain.

Type

int

init_factory

StatisticAccumulatorFactory object for the init distribution. Should be compatible with sequences of integers. Defaults to *NullAccumulatorFactory* if *None*.

Type

StatisticAccumulatorFactory

len_factory

StatisticAccumulatorFactory object for the length of Markov chain sequence. Requires support on non-negative integers. Defaults to *NullAccumulatorFactory* if *None*.

Type

StatisticAccumulatorFactory

keys

Set key for merging sufficient statistics, including the sufficient statistics of *init_dist* and *len_dist*.

Type

Optional[str]

name

Set name for object.

Type

Optional[str]

make()

Create a new empty integer Markov-chain accumulator.

Return type

dmx.stats.int_markovchain.IntegerMarkovChainAccumulator

```
class dmx.stats.int_markovchain.IntegerMarkovChainDataEncoder(lag,
                                                             init_encoder=NoneDataEncoder(),
                                                             len_encoder=NoneDataEncoder())
```

Bases: *DataSequenceEncoder*

Encode batches of fixed-lag integer state sequences.

The encoding records sequence indices for initial blocks and transitions, deduplicates (context, next_state) transitions, and delegates initial blocks and effective lengths to their component encoders.

Parameters

- **lag** (*int*)
- **init_encoder** (*DataSequenceEncoder*)
- **len_encoder** (*DataSequenceEncoder*)

lag

Integer valued length of lag.

Type

int

init_encoder

DataSequenceEncoder object for initial lagged value. Should be a DataSequenceEncoder for a Sequence of distribution with support on integers.

Type

DataSequenceEncoder

len_encoder

DataSequenceEncoder for the length of observed sequences. Should be a DataSequenceEncoder with support on the integers.

Type

DataSequenceEncoder

seq_encode(x)

Encode independent observations from an integer Markov chain.

Parameters

x – Integer-valued Markov-chain observations.

Return type

dmx.stats.int_markovchain.IntegerMarkovChainEncodedDataSequence

Returns

Encoded sequence lengths, observation indexes, unique transitions, transition indexes, initial values, and sequence lengths.

Parameters**x** (*List*[*Sequence*[*int*]])

```
class dmx.stats.int_markovchain.IntegerMarkovChainDistribution(num_values, cond_dist, lag=1,
                                                             init_dist=NoneDistribution(),
                                                             len_dist=NoneDistribution(),
                                                             keys=None, name=None)
```

Bases: [SequenceEncodableProbabilityDistribution](#)Represent a fixed-lag chain on states 0 through `num_values - 1`.Each observation is a complete integer state sequence. Sequence duration is supplied by `len_dist`; there is no terminal state.**Parameters**

- **num_values** (*int*)
- **cond_dist** (*List*[*List*[*float*]] | *ndarray*)
- **lag** (*int*)
- **init_dist** ([SequenceEncodableProbabilityDistribution](#) | *None*)
- **len_dist** ([SequenceEncodableProbabilityDistribution](#) | *None*)
- **keys** (*str* | *None*)
- **name** (*str* | *None*)

num_values

Total number of values in support.

Type*int***cond_dist**Should be `num_vals ** lag` by `num_vals` with transition probabilities for each lagged length tuple `(v_0,v_1,...,v_{lag})`.**Type**

Array-like

lag

Lag length for conditional density.

Type*int***init_dist**Optional distribution for initial states of Markov chain (with length `lag`). Should be a distribution compatible with `Sequences`.**Type**Optional[[SequenceEncodableProbabilityDistribution](#)]**len_dist**

Optional distribution for the length of observations.

TypeOptional[[SequenceEncodableProbabilityDistribution](#)]

name

Set name for object instance.

Type

Optional[str]

keys

Set keys for merging sufficient statistics, including the sufficient statistics of `init_dist` and `len_dist`.

Type

Optional[str]

density(*x*)

Density of integer Markov chain evaluated at *x*.

See `log_density()` for details.

Parameters

x (*Sequence[int]*) – An integer markov chain observation.

Return type

float

Returns

float – Density evaluated at *x*.

Parameters

x (*Sequence[int]*)

dist_to_encoder()

Create an encoder for batches of integer state sequences.

Return type

dmx.stats.int_markovchain.IntegerMarkovChainDataEncoder

estimator(*pseudo_count=None*)

Create an estimator using this model's lag and component estimators.

Return type

dmx.stats.int_markovchain.IntegerMarkovChainEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Evaluate the scalar log density of an integer state sequence.

For a sequence of length *n* $\geq$ *lag*, this sums the initial-block log density, the log conditional probability of each later state given its preceding *lag* states, and `len_dist.log_density(n)`. A shorter sequence contributes only `len_dist.log_density(n)`.

Parameters

x (*Sequence[int]*) – An integer markov chain observation.

Return type

float

Returns

float – Log-density evaluated at *x*.

Parameters

x (*Sequence[int]*)

sampler(*seed=None*)

Create a sampler for complete integer state sequences.

Return type

`dmx.stats.int_markovchain.IntegerMarkovChainSampler`

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Evaluate log densities for an encoded batch of integer state sequences.

Return type

`numpy.ndarray`

Parameters

x (`IntegerMarkovChainEncodedDataSequence`)

class `dmx.stats.int_markovchain.IntegerMarkovChainEncodedDataSequence`(*data*)

Bases: `EncodedDataSequence`

Store an encoded batch of fixed-lag integer-chain observations.

Notes

E = Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray, np.ndarray, EncodedDataSequence, EncodedDataSequence]

Parameters

data (`Tuple`[`ndarray`, `ndarray`, `ndarray`, `ndarray`, `ndarray`, `EncodedDataSequence`, `EncodedDataSequence`])

data

Encoded sequence of integer Markov chain observations.

Type

E

class `dmx.stats.int_markovchain.IntegerMarkovChainEstimator`(*num_values*, *lag=1*,
init_estimator=NullEstimator(),
len_estimator=NullEstimator(),
init_dist=None, *len_dist=None*,
pseudo_count=None, *name=None*,
keys=None)

Bases: `ParameterEstimator`

Estimate a fixed-lag integer chain from aggregated statistics.

Transition counts are arranged into a dense $K \times \text{lag}$ by K matrix and normalized row-wise. `pseudo_count`, when supplied, is added to every matrix cell before normalization. Initial-block and length distributions are either held fixed by `init_dist` and `len_dist` or estimated from their corresponding component statistics. During estimation, the matrix state count is inferred as one plus the largest integer present in a transition key; the configured `num_values` is not used for that step.

Parameters

- **num_values** (*int*)
- **lag** (*int*)
- **init_estimator** (`ParameterEstimator` | *None*)

- **len_estimator** (*ParameterEstimator* / *None*)
- **init_dist** (*SequenceEncodableProbabilityDistribution* / *None*)
- **len_dist** (*SequenceEncodableProbabilityDistribution* / *None*)
- **pseudo_count** (*float* / *None*)
- **name** (*str* / *None*)
- **keys** (*str* / *None*)

num_values

Number of values in Markov chain support.

Type

int

lag

Length of conditional dependence.

Type

int

init_estimator

Optional *ParameterEstimator* object compatible with sequences of integers. Defaults to *NullEstimator*.

Type

ParameterEstimator

len_estimator

ParameterEstimator object compatible with the non-negative integers. Defaults to the *NullEstimator*.

Type

ParameterEstimator

init_dist

If passed, *init_dist* is fixed and not estimated. Must be compatible with sequences of integers.

Type

Optional[*SequenceEncodableProbabilityDistribution*]

len_dist

If passed, *len_dist* is fixed and not estimated. Must be compatible with non-negative integers.

Type

Optional[*SequenceEncodableProbabilityDistribution*]

pseudo_count

If passed sufficient statistics are re-weighted in estimation step.

Type

Optional[*float*]

name

Set name to object instance.

Type

Optional[*str*]

key

Set key for merging sufficient statistics, including the sufficient statistics of `init_dist` and `len_dist`.

Type

`Optional[str]`

accumulator_factory()

Create a factory for compatible sufficient-statistic accumulators.

Return type

`dmx.stats.int_markovchain.IntegerMarkovChainAccumulatorFactory`

estimate(*nobs*, *suff_stat*)

Estimate a distribution from transition and component statistics.

Return type

`dmx.stats.int_markovchain.IntegerMarkovChainDistribution`

Parameters

- **nobs** (`float` | `None`)
- **suff_stat** (`Tuple[Dict[Tuple[Tuple[int, ...], int], float], SS1` | `None, SS2` | `None`])

class `dmx.stats.int_markovchain.IntegerMarkovChainSampler(dist, seed)`

Bases: `DistributionSampler`

Sample complete sequences from a fixed-lag integer Markov chain.

Parameters

- **dist** (`IntegerMarkovChainDistribution`)
- **seed** (`int` | `None`)

dist

Integer Markov chain to sample from.

Type

`IntegerMarkovChainDistribution`

rng

RandomState object with seed set if passed.

Type

`RandomState`

trans_sampler

RandomState object for sampling transitions.

Type

`RandomState`

sample(*size=None*)

Draw iid samples from an integer Markov chain distribution.

Parameters

size (`Optional[int]`) – If `None`, size is taken to be 0.

Return type

`typing.Union[typing.List[typing.Sequence[int]], typing.Sequence[int]]`

Returns

Sequence[int] if size is None, else List[Sequence[int]] with length equal to size.

Parameters

size (*int* | *None*)

sample_given(*x*)

Sample from the Markov chain conditioned on a given value ‘x’.

Parameters

x (*Sequence[int]*) – Sample from Markov chain conditioned on observing ‘x’.

Return type

int

Returns

Single sample transition from integer Markov chain.

Parameters

x (*Sequence[int]*)

single_sample()

Returns a single sample from the integer Markov chain distribution.

Return type

typing.Sequence[int]

Look-back Hidden Markov Model

Provide a hidden Markov model whose emissions look back over observations.

Unlike the canonical HMM, this model’s latent state at step t selects an emission distribution for a window of observations rather than one observation. Let the raw sequence be $x = (x_0, \dots, x_{m-1})$ and let the lag be L . The model represents the sequence as

- an initial block $x_{0:L}$ generated by an initial distribution for each state; and
- $x_{t-L:t+1}$ windows, for $t = L, \dots, m-1$, generated by the state selected after a Markov transition.

Thus the latent path has $m - L + 1$ states, while the returned observation sequence still has m values. The initial state has probability w ; transitions use `transitions[i, j]` for $P(Z_{t+1} = j \mid Z_t = i)$. A subsequent emission scores the whole window $p(x_{t-L:t} \mid Z_t)$, as supplied by `topics`; overlapping windows therefore carry the observation history explicitly. The length distribution is evaluated at $m - L + 1$, the number of latent states (an empty input is the special case scored at length zero).

Inference uses scaled forward recursions for likelihoods and posterior state probabilities, and Viterbi dynamic programming for the most likely latent path. Sequence encoding stores initial blocks and look-back windows in separate encoded arrays so these operations can be vectorized over sequences. The estimator performs Baum–Welch updates for initial-state and transition counts, weighted topic and initial-block sufficient statistics, and the latent-path length distribution. Pseudo-counts are applied to initial and transition probabilities in the estimator.

This is intentionally not the canonical HMM notation: in particular, `t` indexes latent windows and the first latent state emits a block. Please give focused review to the notation and indexing in this module’s documentation.

class `dmx.stats.look_back_hmm.LookBackHMMEncodedDataSequence(data)`

Bases: [*EncodedDataSequence*](#)

Hold the split, vectorized representation used by look-back inference.

Parameters

data (*Tuple[Tuple[ndarray, ndarray, ndarray, ndarray, ndarray,*

```
EncodedDataSequence, EncodedDataSequence], EncodedDataSequence,  
Sequence[Sequence[T]]])
```

```
class dmx.stats.look_back_hmm.LookbackHiddenMarkovDataEncoder(encoder, lag,  
                                                                len_encoder=NoneDataEncoder(),  
                                                                init_encoder=NoneDataEncoder())
```

Bases: [DataSequenceEncoder](#)

Encode initial blocks and look-back windows for batched inference.

Parameters

- **encoder** ([DataSequenceEncoder](#))
- **lag** (*int*)
- **len_encoder** ([DataSequenceEncoder](#) | *None*)
- **init_encoder** ([DataSequenceEncoder](#) | *None*)

seq_encode(*x*)

Encode raw sequences into initial blocks and look-back windows.

Return type

[dmx.stats.look_back_hmm.LookBackHMMEncodedDataSequence](#)

Parameters

x (*Sequence[Sequence[T]]*)

```
class dmx.stats.look_back_hmm.LookbackHiddenMarkovDistribution(topics, w, transitions, lag=0,  
                                                                init_dist=None,  
                                                                len_dist=NoneDistribution(),  
                                                                name=None)
```

Bases: [SequenceEncodableProbabilityDistribution](#)

Represent a finite-state HMM with fixed-length look-back emissions.

lag=L makes each state after the initial state emit the window ending at the current observation and containing the preceding L values. The initial distribution for state *i* emits *x*[:L]. Consequently a raw sequence of length *m* has *m* - L + 1 latent states and its length distribution is evaluated at that value.

Parameters

- **topics** (*Sequence[SequenceEncodableProbabilityDistribution]*)
- **w** (*ndarray*)
- **transitions** (*Sequence[Sequence[float]]* | *ndarray*)
- **lag** (*int*)
- **init_dist** (*Sequence[SequenceEncodableProbabilityDistribution]* | *None*)
- **len_dist** ([SequenceEncodableProbabilityDistribution](#) | *None*)
- **name** (*str* | *None*)

density(*x*)

Return the density of an observed look-back sequence.

Return type

float

Parameters

x (*Sequence[T]*)

dist_to_encoder()

Return the encoder matching this distribution's sequence layout.

Return type

dmx.stats.look_back_hmm.LookbackHiddenMarkovDataEncoder

estimator(pseudo_count=None)

Create an estimator for topic and length parameters.

Return type

dmx.stats.look_back_hmm.LookbackHiddenMarkovEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(x)

Return the log density using a scaled forward recursion.

Return type

float

Parameters

x (*Sequence[T]*)

sampler(seed=None)

Create a sampler for this distribution.

Return type

dmx.stats.look_back_hmm.LookbackHiddenMarkovSampler

Parameters

seed (*int* | *None*)

seq_log_density(x)

Return log densities for an encoded batch of sequences.

Return type

numpy.ndarray

Parameters

x (*LookBackHMMEncodedDataSequence*)

seq_posterior(x)

Return posterior state probabilities for each encoded sequence.

Return type

typing.List[numpy.ndarray]

Parameters

x (*LookBackHMMEncodedDataSequence*)

viterbi_sequence(x)

Return the most likely latent state path for x.

Return type

numpy.ndarray

Parameters

x (*Sequence[T]*)

```
class dmx.stats.look_back_hmm.LookbackHiddenMarkovEstimator(estimators, lag=0,
                                                             init_estimators=None,
                                                             len_estimator=NoneEstimator(),
                                                             suff_stat=None,
                                                             pseudo_count=(None, None),
                                                             name=None, keys=(None, None,
                                                             None))
```

Bases: [ParameterEstimator](#)

Estimate look-back HMM parameters from Baum–Welch statistics.

Parameters

- **estimators** ([Sequence](#)[[ParameterEstimator](#)])
- **lag** ([int](#))
- **init_estimators** ([Sequence](#)[[ParameterEstimator](#)] | [None](#))
- **len_estimator** ([ParameterEstimator](#) | [None](#))
- **suff_stat** ([Any](#) | [None](#))
- **pseudo_count** ([Tuple](#)[[float](#) | [None](#), [float](#) | [None](#)] | [None](#))
- **name** ([str](#) | [None](#))
- **keys** ([Tuple](#)[[str](#) | [None](#), [str](#) | [None](#), [str](#) | [None](#)] | [None](#))

accumulator_factory()

Return a factory for this estimator’s sufficient statistics.

Return type

[dmx.stats.look_back_hmm.LookbackHiddenMarkovEstimatorAccumulatorFactory](#)

estimate(*nobs*, *suff_stat*)

Estimate distributions, initial weights, transitions, and lengths.

Return type

[dmx.stats.look_back_hmm.LookbackHiddenMarkovDistribution](#)

Parameters

- **nobs** ([float](#) | [None](#))
- **suff_stat** ([Any](#))

```
class dmx.stats.look_back_hmm.LookbackHiddenMarkovEstimatorAccumulator(seq_accumulators,
                                                                           init_accumulators=None,
                                                                           lag=0,
                                                                           len_accumulator=NoneAccumulator(),
                                                                           keys=(None, None,
                                                                           None))
```

Bases: [SequenceEncodableStatisticAccumulator](#)

Accumulate weighted sufficient statistics for a look-back HMM.

Parameters

- **seq_accumulators** ([Sequence](#)[[SequenceEncodableStatisticAccumulator](#)])
- **init_accumulators** ([Sequence](#)[[SequenceEncodableStatisticAccumulator](#)] | [None](#))

- **lag** (*int*)
- **len_accumulator** (*SequenceEncodableStatisticAccumulator* | *None*)
- **keys** (*Tuple[str | None, str | None, str | None]*)

acc_to_encoder()

Return the encoder corresponding to these accumulators.

Return type

dmx.stats.look_back_hmm.LookbackHiddenMarkovDataEncoder

combine(suff_stat)

Combine another accumulator's sufficient statistics into this one.

Return type

dmx.stats.look_back_hmm.LookbackHiddenMarkovEstimatorAccumulator

Parameters

suff_stat (*Any*)

from_value(x)

Restore sufficient statistics from a previously returned tuple.

Return type

dmx.stats.look_back_hmm.LookbackHiddenMarkovEstimatorAccumulator

Parameters

x (*Any*)

initialize(x, weight, rng)

Initialize statistics with randomized latent-state responsibilities.

Return type

None

Parameters

- **x** (*Sequence[T]*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(stats_dict)

Merge keyed sufficient statistics into stats_dict.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

key_replace(stats_dict)

Replace keyed sufficient statistics from stats_dict.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

seq_initialize(*x*, *weights*, *rng*)

Initialize statistics for each sequence in an encoded batch.

Return type

None

Parameters

- **x** ([LookBackHMMEncodedDataSequence](#))
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Accumulate Baum–Welch statistics for an encoded batch.

Return type

None

Parameters

- **x** ([LookBackHMMEncodedDataSequence](#))
- **weights** (*ndarray*)
- **estimate** ([LookbackHiddenMarkovDistribution](#))

update(*x*, *weight*, *estimate*)

Accumulate one raw sequence with the supplied weight.

Return type

None

Parameters

- **x** (*Sequence*[*T*])
- **weight** (*float*)
- **estimate** ([LookbackHiddenMarkovDistribution](#))

value()

Return sufficient statistics in the estimator’s tuple format.

Return type*typing.Tuple*[*typing.Any*, ...]

```
class dmx.stats.look_back_hmm.LookbackHiddenMarkovEstimatorAccumulatorFactory(lag,  
                                                                           seq_factories,  
                                                                           init_factories=None,  
                                                                           len_factory=NullAccumulatorFactory,  
                                                                           keys=(None,  
                                                                           None, None))
```

Bases: [StatisticAccumulatorFactory](#)

Build accumulators for look-back HMM estimation.

Parameters

- **lag** (*int*)
- **seq_factories** (*Sequence*[*StatisticAccumulatorFactory*])
- **init_factories** (*Sequence*[*StatisticAccumulatorFactory*] | None)

- **len_factory** (*StatisticAccumulatorFactory* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*, *str* | *None*] | *None*)

make()

Create a fresh look-back HMM statistic accumulator.

Return type

dmx.stats.look_back_hmm.LookbackHiddenMarkovEstimatorAccumulator

class *dmx.stats.look_back_hmm.LookbackHiddenMarkovSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*

Sample latent paths and observations from a look-back HMM.

Parameters

- **dist** (*LookbackHiddenMarkovDistribution*)
- **seed** (*int* | *None*)

sample(*size=None*)

Draw one sequence, or size independently drawn sequences.

Return type

typing.Union[*typing.List*[*typing.TypeVar*(*T*)], *typing.List*[*typing.List*[*typing.TypeVar*(*T*)]]]

Parameters

size (*int* | *None*)

dmx.stats.look_back_hmm.numba_baum_welch(*num_states*, *tz*, *prob_mat*, *init_pvec*, *tran_mat*, *weights*, *alpha_loc*, *xi_acc*, *pi_acc*, *beta_buff*, *xi_buff*)

Accumulate forward-backward counts for look-back sequences.

Return type

None

Parameters

- **num_states** (*int*)
- **tz** (*ndarray*)
- **prob_mat** (*ndarray*)
- **init_pvec** (*ndarray*)
- **tran_mat** (*ndarray*)
- **weights** (*ndarray*)
- **alpha_loc** (*ndarray*)
- **xi_acc** (*ndarray*)
- **pi_acc** (*ndarray*)
- **beta_buff** (*ndarray*)
- **xi_buff** (*ndarray*)

dmx.stats.look_back_hmm.numba_baum_welch2(*num_states*, *tz*, *prob_mat*, *init_pvec*, *tran_mat*, *weights*, *alpha_loc*, *xi_acc*, *pi_acc*)

Accumulate weighted forward-backward counts for look-back sequences.

Return type

None

Parameters

- **num_states** (*int*)
- **tz** (*ndarray*)
- **prob_mat** (*ndarray*)
- **init_pvec** (*ndarray*)
- **tran_mat** (*ndarray*)
- **weights** (*ndarray*)
- **alpha_loc** (*ndarray*)
- **xi_acc** (*ndarray*)
- **pi_acc** (*ndarray*)

`dmx.stats.look_back_hmm.numba_baum_welch_alphas(num_states, tz, prob_mat, init_pvec, tran_mat, weights, alpha_loc, xi_acc, pi_acc)`

Compute posterior state probabilities for encoded sequences.

Return type

None

Parameters

- **num_states** (*int*)
- **tz** (*ndarray*)
- **prob_mat** (*ndarray*)
- **init_pvec** (*ndarray*)
- **tran_mat** (*ndarray*)
- **weights** (*ndarray*)
- **alpha_loc** (*ndarray*)
- **xi_acc** (*ndarray*)
- **pi_acc** (*ndarray*)

`dmx.stats.look_back_hmm.numba_seq_log_density(num_states, tz, prob_mat, init_pvec, tran_mat, max_ll, next_alpha_mat, alpha_buff_mat, out)`

Compute scaled forward log densities for encoded sequences.

Return type

None

Parameters

- **num_states** (*int*)
- **tz** (*ndarray*)
- **prob_mat** (*ndarray*)
- **init_pvec** (*ndarray*)
- **tran_mat** (*ndarray*)

- **max_ll** (*ndarray*)
- **next_alpha_mat** (*ndarray*)
- **alpha_buff_mat** (*ndarray*)
- **out** (*ndarray*)

Sparse Markov Association

Provide a sparse lexical association model for two weighted token bags.

Defines the `SparseMarkovAssociationDistribution`, `SparseMarkovAssociationSampler`, `SparseMarkovAssociationAccumulatorFactory`, `SparseMarkovAssociationAccumulator`, `SparseMarkovAssociationEstimator`, and the `SparseMarkovAssociationDataEncoder` classes for use with dmx-learn.

Data type: `Tuple[List[Tuple[int, float]], List[Tuple[int, float]]]`.

The `SparseMarkovAssociation` model is a generative model for two sets of words $S_1 = \{w_{\{1,1\}}, \dots, w_{\{1,n\}}\}$ and $S_2 = \{w_{\{2,1\}}, \dots, w_{\{2,m\}}\}$ over W possible words. The model assumes a hidden set of assignments $A_2 = \{a_{\{2,1\}}, \dots, a_{\{2,m\}}\}$ where $a_{\{2,j\}}$ takes on values in $\{1, 2, \dots, m\}$. The observed likelihood function is computed from $P(S_1, S_2) = P(S_2 | S_1) P(S_1)$, where

- (1) $\log(P(S_2 | S_1)) = \sum_{i=1}^m \log(P(w_{\{2,i\}} | w_{\{1,1\}}, \dots, w_{\{1,n\}}))$
 $= \sum_{i=1}^m \log((1/m) * \sum_{j=1}^n (1-\alpha) * P(w_{\{2,i\}} | w_{\{1,j\}}) + \alpha/W).$
- (2) $\log(P(S_1)) = \sum_{j=1}^n \log((1-\alpha) * P(w_{\{1,j\}}) + \alpha/W).$

An observation is (S_1, S_2) , where each bag contains `(token_id, weight)` pairs. `init_prob_vec` scores tokens in S_1 and `sparse_cond_prob_mat[target, source]` maps source tokens to target tokens. For each target token, the likelihood averages its conditional probability across the source bag and mixes it with a uniform `alpha` background. The model is an alignment-marginal approximation: it does not retain an explicit alignment variable in its public input. Estimation accumulates expected sparse source-target associations and normalizes rows into conditional probabilities. `low_memory` selects an equivalent encoded representation that trades memory for repeated sparse operations.

```
class dmx.stats.sparse_markov_transform.SparseMarkovAssociationAccumulator(num_vals,
                                                                           size_acc=NoneAccumulator(),
                                                                           keys=(None,
                                                                           None),
                                                                           low_memory=True)
```

Bases: `SequenceEncodableStatisticAccumulator`

Accumulate sparse expected association and initial-token counts.

Parameters

- **num_vals** (*int*)
- **size_acc** (`SequenceEncodableStatisticAccumulator` / *None*)
- **keys** (`Tuple[str | None, str | None]`)
- **low_memory** (*bool*)

acc_to_encoder()

Return the encoder corresponding to this accumulator.

Return type

`dmx.stats.sparse_markov_transform.SparseMarkovAssociationDataEncoder`

combine(suff_stat)

Combine initial, sparse association, and length statistics.

Return type*dmx.stats.sparse_markov_transform.SparseMarkovAssociationAccumulator***Parameters****suff_stat** (*Tuple*[*ndarray*, *lil_matrix* | *csr_matrix* | *None*, *SS1*])**from_value**(*x*)

Restore sufficient statistics from a tuple.

Return type*dmx.stats.sparse_markov_transform.SparseMarkovAssociationAccumulator***Parameters****x** (*Tuple*[*ndarray*, *lil_matrix* | *csr_matrix* | *None*, *SS1*])**initialize**(*x*, *weight*, *rng*)

Initialize statistics from one observation pair.

Return type*None***Parameters**

- **x** (*Tuple*[*List*[*Tuple*[*int*, *float*]], *List*[*Tuple*[*int*, *float*]]])
- **weight** (*float*)
- **rng** (*RandomState*)

initialize_rng(*rng*)

Initialize independent random streams for component statistics.

Return type*None***Parameters****rng** (*RandomState*)**key_merge**(*stats_dict*)

Merge keyed sparse association statistics.

Return type*None***Parameters****stats_dict** (*Dict*[*str*, *Any*])**key_replace**(*stats_dict*)

Replace keyed sparse association statistics.

Return type*None***Parameters****stats_dict** (*Dict*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *rng*)

Initialize statistics from an encoded observation batch.

Return type*None***Parameters**

- **x** (`SparseMarkovAssociationEncodedDataSequence`)
- **weights** (`ndarray`)
- **rng** (`RandomState`)

seq_update(*x, weights, estimate*)

Accumulate expected sparse alignments for an encoded batch.

Return type

None

Parameters

- **x** (`SparseMarkovAssociationEncodedDataSequence`)
- **weights** (`ndarray`)
- **estimate** (`SparseMarkovAssociationDistribution`)

update(*x, weight, estimate*)

Accumulate statistics from one weighted source-target bag pair.

Return type

None

Parameters

- **x** (`Tuple[List[Tuple[int, float]], List[Tuple[int, float]]]`)
- **weight** (`float`)
- **estimate** (`SparseMarkovAssociationDistribution`)

value()

Return sufficient statistics for sparse association estimation.

Return type

`typing.Tuple[numpy.ndarray, typing.Union[scipy.sparse._lil.lil_matrix, scipy.sparse._csr.csr_matrix, None], typing.Any]`

```
class dmx.stats.sparse_markov_transform.SparseMarkovAssociationAccumulatorFactory(num_vals,
                                                                                   len_factory=NoneAccumulatorFactory,
                                                                                   low_memory=True,
                                                                                   keys=(None,
                                                                                   None))
```

Bases: `StatisticAccumulatorFactory`

Create sparse association statistic accumulators.

Parameters

- **num_vals** (`int`)
- **len_factory** (`StatisticAccumulatorFactory | None`)
- **low_memory** (`bool`)
- **keys** (`Tuple[str | None, str | None]`)

make()

Create a fresh sparse association accumulator.

Return type

`dmx.stats.sparse_markov_transform.SparseMarkovAssociationAccumulator`

```
class dmx.stats.sparse_markov_transform.SparseMarkovAssociationDataEncoder(len_encoder,  
                                                                           low_memory)
```

Bases: [DataSequenceEncoder](#)

Encode weighted source-target bags for sparse association inference.

Parameters

- **len_encoder** ([DataSequenceEncoder](#))
- **low_memory** (*bool*)

seq_encode(*x*)

Encode a batch of weighted source-target bag pairs.

Return type

[dmx.stats.sparse_markov_transform.SparseMarkovAssociationEncodedDataSequence](#)

Parameters

x ([Sequence\[Tuple\[List\[Tuple\[int, float\]\], List\[Tuple\[int, float\]\]\]](#))

```
class dmx.stats.sparse_markov_transform.SparseMarkovAssociationDistribution(init_prob_vec,  
                                                                           cond_prob_mat,  
                                                                           alpha=0.0,  
                                                                           len_dist=NoneDistribution(),  
                                                                           low_memory=False)
```

Bases: [SequenceEncodableProbabilityDistribution](#)

Represent a sparse source-to-target bag association distribution.

Parameters

- **init_prob_vec** ([Sequence\[float\]](#) | *ndarray*)
- **cond_prob_mat** (*csr_matrix*)
- **alpha** (*float*)
- **len_dist** ([SequenceEncodableProbabilityDistribution](#) | *None*)
- **low_memory** (*bool*)

init_prob_vec

Probabilities for the first set of words S1.

Type

np.ndarray

cond_prob_mat

Sparse matrix defining the probabilities for mapping words in S1 to S2. Its dimensions are the S2 vocabulary size by the S1 vocabulary size.

Type

csr_matrix

alpha

Regularization parameter (should be between 0 and 1).

Type

float

len_dist

Distribution for length of words. Must be compatible with `Tuple[int, int]`

Type

SequenceEncodableProbabilityDistribution

low_memory

If True, uses low_memory function calls.

Type

bool

density(*x*)

Return the density of one weighted source-target bag pair.

Return type

float

Parameters

x (`Tuple[List[Tuple[int, float]], List[Tuple[int, float]]]`)

dist_to_encoder()

Return an encoder using this distribution's memory mode.

Return type

dmx.stats.sparse_markov_transform.SparseMarkovAssociationDataEncoder

estimator(*pseudo_count=None*)

Create an estimator for initial and conditional probabilities.

Return type

dmx.stats.sparse_markov_transform.SparseMarkovAssociationEstimator

Parameters

pseudo_count (`float | None`)

log_density(*x*)

Return the log density of one weighted source-target bag pair.

Return type

float

Parameters

x (`Tuple[List[Tuple[int, float]], List[Tuple[int, float]]]`)

sampler(*seed=None*)

Create a sampler for sparse source-target associations.

Return type

dmx.stats.sparse_markov_transform.SparseMarkovAssociationSampler

Parameters

seed (`int | None`)

seq_log_density(*x*)

Return log densities for an encoded batch of bag pairs.

Return type

`numpy.ndarray`

Parameters

x (`SparseMarkovAssociationEncodedDataSequence`)

```
class dmx.stats.sparse_markov_transform.SparseMarkovAssociationEncodedDataSequence(data)
```

Bases: [EncodedDataSequence](#)

Hold sparse association batch indices, weights, and length encodings.

Parameters

data (Tuple[List[Tuple[ndarray, ndarray, ndarray, ndarray]],
[EncodedDataSequence](#), ndarray, Tuple[ndarray, ndarray, ndarray, ndarray,
ndarray, ndarray, ndarray, ndarray, ndarray] | None])

```
class dmx.stats.sparse_markov_transform.SparseMarkovAssociationEstimator(num_vals,  
                                                                           alpha=0.0,  
                                                                           len_estimator=NoneEstimator(),  
                                                                           suff_stat=None,  
                                                                           pseudo_count=None,  
                                                                           low_memory=True,  
                                                                           keys=(None, None))
```

Bases: [ParameterEstimator](#)

Estimate sparse initial and conditional association probabilities.

Parameters

- **num_vals** (*int*)
- **alpha** (*float*)
- **len_estimator** ([ParameterEstimator](#) | None)
- **suff_stat** (*Any* | None)
- **pseudo_count** (*float* | None)
- **low_memory** (*bool*)
- **keys** (Tuple[str | None, str | None])

num_vals

Number of values in S1.

Type

int

alpha

Regularization parameter (should be between 0 and 1).

Type

float

len_estimator

ParameterEstimator object for the length of observations.

Type

[ParameterEstimator](#)

suff_stat

Kept for consistency with estimate function.

Type

Optional[Any]

pseudo_count

Regularize sufficient statistics.

Type

Optional[float]

low_memory

If True, use low_memory options.

Type

bool

keys

Keys for initial distribution and state transition stats.

Type

Tuple[Optional[str], Optional[str]]

accumulator_factory()

Return a factory for sparse association sufficient statistics.

Return type

dmx.stats.sparse_markov_transform.SparseMarkovAssociationAccumulatorFactory

estimate(nobs, suff_stat)

Normalize accumulated sparse counts into a distribution.

Return type

dmx.stats.sparse_markov_transform.SparseMarkovAssociationDistribution

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *lil_matrix* | *csr_matrix* | *None*, *SS1*])

class *dmx.stats.sparse_markov_transform.SparseMarkovAssociationSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*

Sample weighted token-bag association observations.

Parameters

- **dist** (*SparseMarkovAssociationDistribution*)
- **seed** (*int* | *None*)

sample(size=None)

Draw one observation pair, or size independent pairs.

Return type

typing.Union[*typing.Tuple*[*typing.List*[*typing.Tuple*[*int*, *float*]], *typing.List*[*typing.Tuple*[*int*, *float*]]], *typing.Sequence*[*typing.Tuple*[*typing.List*[*typing.Tuple*[*int*, *float*]], *typing.List*[*typing.Tuple*[*int*, *float*]]]]]

Parameters

size (*int* | *None*)

Tree Hidden Markov Model

Provide hidden Markov models over rooted, ordered node arrays.

One observed tree is a sequence $[(d_0, y_0), \dots, (d_{n-1}, y_{n-1})]$. Each descriptor $d_v = (v, p_v)$ gives a node index and its parent index; the root is $(0, -1)$ and every non-root parent precedes its child. Each node v has an emission y_v and a latent state $Z_v \in \{0, \dots, K-1\}$. With $\pi_i = P(Z_0 = i)$, $A_{ij} = P(Z_c = j \mid Z_p = i)$, and $b_i(y) = p(y \mid Z_v = i)$, the state-and-emission model is

$$p(y, z \mid d) = \pi_{z_0} b_{z_0}(y_0) \prod_{v=1}^{n-1} A_{z_{p_v}, z_v} b_{z_v}(y_v).$$

`w` stores π with shape $(K,)$; `transitions` stores A with shape (K, K) ; and `topics[i]` provides b_i . A posterior for one tree has shape (n, K) and a Viterbi result has shape $(n,)$, both ordered exactly as the input nodes.

The encoder converts a batch of trees into flattened node emissions plus integer parent/child and tree-boundary arrays. It supports equivalent NumPy and Numba layouts; those arrays are implementation details, while callers continue to pass the tree sequence described above. Inference marginalizes or maximizes over node states with tree upward/downward recursions. Estimation collects root counts, parent-to-child transition counts, state-weighted emission statistics, and child-count statistics. `len_dist` governs child counts while sampling and is updated from those statistics, but it is not an additional factor in `log_density` or `seq_log_density`.

Please give focused review to this tree notation, the parent-index convention, and the rendered math before treating it as canonical public documentation.

```
class dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator(accumulators,
                                                    len_accumulator=NoneAccumulator(),
                                                    keys=(None, None, None), name=None,
                                                    use_numba=True)
```

Bases: [SequenceEncodableStatisticAccumulator](#)

Accumulate root, transition, emission, and child-count statistics.

Parameters

- **accumulators** ([Sequence](#)[[SequenceEncodableStatisticAccumulator](#)])
- **len_accumulator** ([SequenceEncodableStatisticAccumulator](#) | `None`)
- **keys** ([Tuple](#)[`str` | `None`, `str` | `None`, `str` | `None`])
- **name** (`str` | `None`)
- **use_numba** (`bool`)

acc_to_encoder()

Return the tree encoder corresponding to these accumulators.

Return type

[dmx.stats.tree_hmm.TreeHiddenMarkovDataEncoder](#)

combine(suff_stat)

Combine a sufficient-statistic tuple into this accumulator.

Return type

[dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator](#)

Parameters

suff_stat ([Tuple](#)[`int`, `ndarray`, `ndarray`, `ndarray`, [Sequence](#)[`SS0`], `SS1` | `None`])

from_value(*x*)

Restore this accumulator from a sufficient-statistic tuple.

Return type

dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator

Parameters

x (*Tuple*[*int*, *ndarray*, *ndarray*, *ndarray*, *Sequence*[*SS0*], *SS1* | *None*])

initialize(*x*, *weight*, *rng*)

Initialize statistics for one tree with random state assignments.

Return type

None

Parameters

- **x** (*Sequence*[*Tuple*[*Tuple*[*int*, *int* | *None*], *T*]])
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge keyed sufficient statistics into *stats_dict*.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

key_replace(*stats_dict*)

Replace keyed sufficient statistics from *stats_dict*.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

seq_initialize(*x*, *weights*, *rng*)

Initialize statistics for a batch of encoded trees.

Return type

None

Parameters

- **x** (*TreeHiddenMarkovEncodedDataSequence*)
- **weights** (*ndarray*)
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Accumulate weighted tree forward-backward sufficient statistics.

Return type

None

Parameters

- **x** (*TreeHiddenMarkovEncodedDataSequence*)

- **weights** (*ndarray*)
- **estimate** (*TreeHiddenMarkovModelDistribution*)

update(*x, weight, estimate*)

Accumulate weighted sufficient statistics for one raw tree.

Return type

None

Parameters

- **x** (*Sequence[Tuple[Tuple[int, int | None], T]]*)
- **weight** (*float*)
- **estimate** (*TreeHiddenMarkovModelDistribution*)

value()

Return state counts and component statistics for estimation.

Return type

typing.Tuple[int, numpy.ndarray, numpy.ndarray, numpy.ndarray, typing.Sequence[typing.Any], typing.Optional[typing.Any]]

```
class dmx.stats.tree_hmm.TreeHiddenMarkovAccumulatorFactory(factories,  
                                                         len_factory=NullAccumulatorFactory(),  
                                                         keys=(None, None, None),  
                                                         name=None, use_numba=True)
```

Bases: *StatisticAccumulatorFactory*

Build statistic accumulators for tree HMM estimation.

Parameters

- **factories** (*Sequence[StatisticAccumulatorFactory]*)
- **len_factory** (*StatisticAccumulatorFactory*)
- **keys** (*Tuple[str | None, str | None, str | None] | None*)
- **name** (*str | None*)
- **use_numba** (*bool*)

make()

Create a fresh tree HMM statistic accumulator.

Return type

dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator

```
class dmx.stats.tree_hmm.TreeHiddenMarkovDataEncoder(emission_encoder,  
                                                    len_encoder=NullDataEncoder(),  
                                                    use_numba=True)
```

Bases: *DataSequenceEncoder*

Encode tree batches as flattened nodes and parent-child index arrays.

The raw tree contract remains (*node_index*, *parent_index*), *emission*. The encoded data flattens all node emissions, retains cumulative tree boundaries, and stores parent, child, level, and leaf indices. In either encoder layout, node-level arrays have length equal to the batch's total node count and transition-pair arrays have length equal to its total edge count.

Parameters

- **emission_encoder** (`DataSequenceEncoder`)
- **len_encoder** (`DataSequenceEncoder` | `None`)
- **use_numba** (`bool`)

seq_encode(*x*)

Encode a batch of rooted node sequences for tree inference.

Return type

`dmx.stats.tree_hmm.TreeHiddenMarkovEncodedDataSequence`

Parameters

x (`Sequence[Sequence[Tuple[Tuple[int, int | None], T]]]`)

class `dmx.stats.tree_hmm.TreeHiddenMarkovEncodedDataSequence`(*data*)

Bases: `EncodedDataSequence`

Hold a NumPy- or Numba-oriented encoded batch of rooted trees.

Parameters

data (`Tuple[bool, Tuple[ndarray, Tuple[int, ndarray, ndarray, ndarray], Tuple[ndarray, ndarray, ndarray, ndarray, ndarray, ndarray, ndarray], EncodedDataSequence, Tuple[ndarray, EncodedDataSequence]]]` | `Tuple[bool, Tuple[int, ndarray, Tuple[ndarray, ndarray, ndarray], Tuple[ndarray, ndarray, ndarray, ndarray, List[ndarray], List[ndarray], List[ndarray], List[ndarray], ndarray], EncodedDataSequence, Tuple[ndarray, EncodedDataSequence]` | `None]]`)

class `dmx.stats.tree_hmm.TreeHiddenMarkovEstimator`(*estimators*, *len_estimator*=`NullEstimator()`, *pseudo_count*=(`None`, `None`), *name*=`None`, *keys*=(`None`, `None`, `None`), *use_numba*=`True`)

Bases: `ParameterEstimator`

Estimate tree HMM parameters from accumulated sufficient statistics.

Parameters

- **estimators** (`List[ParameterEstimator]`)
- **len_estimator** (`ParameterEstimator` | `None`)
- **pseudo_count** (`Tuple[float | None, float | None]` | `None`)
- **name** (`str` | `None`)
- **keys** (`Tuple[str | None, str | None, str | None]` | `None`)
- **use_numba** (`bool`)

accumulator_factory()

Return a factory for this estimator's sufficient statistics.

Return type

`dmx.stats.tree_hmm.TreeHiddenMarkovAccumulatorFactory`

estimate(*nobs*, *suff_stat*)

Estimate emissions, root weights, transitions, and child counts.

Return type

`dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution`

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*int*, *ndarray*, *ndarray*, *ndarray*, *Sequence*[*SS0*], *SS1* | *None*])

```
class dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution(topics, w, transitions,  
                                                         len_dist=NullDistribution(),  
                                                         terminal_level=10, name=None,  
                                                         use_numba=False)
```

Bases: [SequenceEncodableProbabilityDistribution](#)

Represent a finite-state HMM whose observations are rooted trees.

Parameters

- **topics** (*Sequence*[[SequenceEncodableProbabilityDistribution](#)])
- **w** (*Sequence*[*float*] | *ndarray*)
- **transitions** (*List*[*List*[*float*]] | *ndarray*)
- **len_dist** ([SequenceEncodableProbabilityDistribution](#) | *None*)
- **terminal_level** (*int*)
- **name** (*str* | *None*)
- **use_numba** (*bool*)

topics

State-indexed emission distributions.

num_states

Number of latent states, K.

w

Root-state probabilities with shape (K,).

transitions

Parent-to-child probabilities with shape (K, K).

len_dist

Child-count distribution used by sampling and estimation.

terminal_level

Maximum depth used only while sampling.

use_numba

Whether to use the Numba-oriented encoded layout.

density(*x*)

Return the marginal density of one observed tree.

Return type

float

Parameters

x (*Sequence*[*Tuple*[*Tuple*[*int*, *int* | *None*], *T*]])

dist_to_encoder()

Return the encoder matching this distribution's tree representation.

Return type*dmx.stats.tree_hmm.TreeHiddenMarkovDataEncoder***estimator**(*pseudo_count=None*)

Create an estimator for emissions, root weights, and transitions.

Return type*dmx.stats.tree_hmm.TreeHiddenMarkovEstimator***Parameters****pseudo_count** (*float* | *None*)**log_density**(*x*)

Return the marginal log density of one observed tree.

Return type*float***Parameters****x** (*Sequence*[*Tuple*[*Tuple*[*int*, *int* | *None*], *T*]])**sampler**(*seed=None*)

Create a tree sampler, requiring a non-null child-count distribution.

Return type*dmx.stats.tree_hmm.TreeHiddenMarkovSampler***Parameters****seed** (*int* | *None*)**seq_log_density**(*x*)

Return one marginal log density per encoded tree.

Return type*numpy.ndarray***Parameters****x** (*TreeHiddenMarkovEncodedDataSequence*)**seq_posterior**(*x*)

Return node-state posterior arrays of shape (n, K) per tree.

Return type*typing.List*[*numpy.ndarray*]**Parameters****x** (*TreeHiddenMarkovEncodedDataSequence*)**seq_viterbi**(*x*)

Return one most-likely state vector of shape (n,) per tree.

Return type*typing.List*[*numpy.ndarray*]**Parameters****x** (*TreeHiddenMarkovEncodedDataSequence*)**viterbi**(*x*)

Return the most likely state vector, with shape (n,).

Return type*numpy.ndarray*

Parameters**x** (*Sequence*[*Tuple*[*Tuple*[*int*, *int* | *None*], *T*]])**class** `dmx.stats.tree_hmm.TreeHiddenMarkovSampler`(*dist*, *seed=None*)Bases: *DistributionSampler*

Sample rooted trees, latent states, and node emissions.

Parameters

- **dist** (*TreeHiddenMarkovModelDistribution*)
- **seed** (*int* | *None*)

sample(*size=None*)

Sample trees using the configured child-count distribution.

Return type`typing.Union[typing.List[typing.Tuple[typing.Tuple[int, typing.Optional[int]], typing.Any]], typing.List[typing.List[typing.Tuple[typing.Tuple[int, typing.Optional[int]], typing.Any]]]`**Parameters****size** (*int* | *None*)**sample_state**(*given_state*, *size=None*)

Sample one or more child states conditioned on a parent state.

Return type`typing.Union[int, numpy.ndarray]`**Parameters**

- **given_state** (*int*)
- **size** (*int* | *None*)

sample_tree(*size=None*)

Sample one tree, or a list of independently sampled trees.

Return type`typing.Union[typing.List[typing.Tuple[typing.Tuple[int, typing.Optional[int]], typing.Any]], typing.List[typing.List[typing.Tuple[typing.Tuple[int, typing.Optional[int]], typing.Any]]]`**Parameters****size** (*int* | *None*)`dmx.stats.tree_hmm.find_level`(*parents*)

Find the level in the tree for nodes, given an array of parents.

Parameters**parents** (*np.ndarray*) – Numpy array of integers with first entry -1.**Return type**`typing.List[int]`**Returns**

Level of each node in the tree excluding the first entry which is the root (level = 0).

Parameters**parents** (*ndarray*)

`dmx.stats.tree_hmm.level_state_prob(levels, num_states, tr_mat, init_prob, out)`

Fill per-level marginal state probabilities with shape (L, K).

Return type

None

Parameters

- **levels** (*int*)
- **num_states** (*int*)
- **tr_mat** (*ndarray*)
- **init_prob** (*ndarray*)
- **out** (*ndarray*)

`dmx.stats.tree_hmm.numba_baum_welch(num_states, tz, txz, tp, tpz, tlnz, xp, xc, xl, xbi, xln, xlnl, pr_obs, p_level, tr_mat, weights, betas, etas, alphas, xi_acc, pi_acc)`

Accumulate tree forward-backward state and transition statistics.

Return type

None

Parameters

- **num_states** (*int*)
- **tz** (*ndarray*)
- **txz** (*ndarray*)
- **tp** (*ndarray*)
- **tpz** (*ndarray*)
- **tlnz** (*ndarray*)
- **xp** (*ndarray*)
- **xc** (*ndarray*)
- **xl** (*ndarray*)
- **xbi** (*ndarray*)
- **xln** (*ndarray*)
- **xlnl** (*ndarray*)
- **pr_obs** (*ndarray*)
- **p_level** (*ndarray*)
- **tr_mat** (*ndarray*)
- **weights** (*ndarray*)
- **betas** (*ndarray*)
- **etas** (*ndarray*)
- **alphas** (*ndarray*)
- **xi_acc** (*ndarray*)
- **pi_acc** (*ndarray*)

```
dmx.stats.tree_hmm.numba_initialize(tz, txz, tp, tpz, xp, xc, states, weights, init_counts, state_counts,  
                                   trans_counts)
```

Accumulate statistics from randomly initialized tree states.

Return type

None

Parameters

- **tz** (*ndarray*)
- **txz** (*ndarray*)
- **tp** (*ndarray*)
- **tpz** (*ndarray*)
- **xp** (*ndarray*)
- **xc** (*ndarray*)
- **states** (*ndarray*)
- **weights** (*ndarray*)
- **init_counts** (*ndarray*)
- **state_counts** (*ndarray*)
- **trans_counts** (*ndarray*)

```
dmx.stats.tree_hmm.numba_posteriors(num_states, tz, txz, tp, tpz, tlnz, xp, xc, xl, xbi, xln, xlnl, pr_obs,  
                                   p_level, tr_mat, betas, etas)
```

Compute posterior node-state probabilities for encoded trees.

Return type

None

Parameters

- **num_states** (*int*)
- **tz** (*ndarray*)
- **txz** (*ndarray*)
- **tp** (*ndarray*)
- **tpz** (*ndarray*)
- **tlnz** (*ndarray*)
- **xp** (*ndarray*)
- **xc** (*ndarray*)
- **xl** (*ndarray*)
- **xbi** (*ndarray*)
- **xln** (*ndarray*)
- **xlnl** (*ndarray*)
- **pr_obs** (*ndarray*)
- **p_level** (*ndarray*)
- **tr_mat** (*ndarray*)

- **betas** (*ndarray*)
- **etas** (*ndarray*)

`dmx.stats.tree_hmm.numba_seq_log_density(num_states, tz, txz, tp, tpz, tlnz, xp, xc, xl, xbi, xln, xlnl, pr_obs, p_level, tr_mat, pr_max0, betas, etas, out)`

Compute scaled marginal log densities for encoded tree batches.

Return type

None

Parameters

- **num_states** (*int*)
- **tz** (*ndarray*)
- **txz** (*ndarray*)
- **tp** (*ndarray*)
- **tpz** (*ndarray*)
- **tlnz** (*ndarray*)
- **xp** (*ndarray*)
- **xc** (*ndarray*)
- **xl** (*ndarray*)
- **xbi** (*ndarray*)
- **xln** (*ndarray*)
- **xlnl** (*ndarray*)
- **pr_obs** (*ndarray*)
- **p_level** (*ndarray*)
- **tr_mat** (*ndarray*)
- **pr_max0** (*ndarray*)
- **betas** (*ndarray*)
- **etas** (*ndarray*)
- **out** (*ndarray*)

`dmx.stats.tree_hmm.numba_viterbi(num_states, tz, txz, tp, tpz, tlnz, xp, xc, xl, xbi, xln, log_pr_obs, log_init_p, log_tr_mat, betas, etas, out)`

Compute most-likely node states for encoded tree batches.

Return type

None

Parameters

- **num_states** (*int*)
- **tz** (*ndarray*)
- **txz** (*ndarray*)
- **tp** (*ndarray*)
- **tpz** (*ndarray*)

- **tlnz** (*ndarray*)
- **xp** (*ndarray*)
- **xc** (*ndarray*)
- **xl** (*ndarray*)
- **xbi** (*ndarray*)
- **xln** (*ndarray*)
- **log_pr_obs** (*ndarray*)
- **log_init_p** (*ndarray*)
- **log_tr_mat** (*ndarray*)
- **betas** (*ndarray*)
- **etas** (*ndarray*)
- **out** (*ndarray*)

`dmx.stats.tree_hmm.vec_bincount(idx, ll, out)`

Add values into out at parallel integer-indexed positions.

Return type

`numpy.ndarray`

Parameters

- **idx** (*ndarray*)
- **ll** (*ndarray*)
- **out** (*ndarray*)

1.3.6 Distributed helpers

Spark RDD Sampling

Sample distributions into Spark RDD partitions.

The functions require a PySpark `SparkContext` and its RDD APIs. They broadcast or pickle the supplied distribution, construct one sampler per partition seed, and produce independent samples within that partition. The helpers neither fit nor score distributions; they are distributed sampling adapters. Reproducibility is scoped to the supplied seed, partition count, and Spark partition execution order.

`dmx.stats.rdd_sampler.sample_rdd(sc, dist, count_per_split, num_splits, seed=None)`

Sample independent observations into a partitioned RDD.

Parameters

- **sc** – Spark context that creates and broadcasts the work.
- **dist** – Distribution whose sampler implements `sample`.
- **count_per_split** – Observations generated by each partition.
- **num_splits** – Number of output partitions.
- **seed** – Optional base seed expanded into partition-local seeds.

Return type

`typing.Any`

Returns

An RDD containing `count_per_split * num_splits` observations.

Parameters

- `sc` (*SparkContext*)
- `dist` (*SequenceEncodableProbabilityDistribution*)
- `count_per_split` (*int*)
- `num_splits` (*int*)
- `seed` (*int | None*)

`dmx.stats.rdd_sampler.sample_seq_as_rdd(sc, dist, seq_len, count_per_split, num_splits, seed=None)`

Sample fixed-length sequences into a partitioned RDD.

Parameters

- `sc` – Spark context that creates and broadcasts the work.
- `dist` – Distribution whose sampler implements `sample_seq`.
- `seq_len` – Length of each sampled sequence.
- `count_per_split` – Sequences generated by each partition.
- `num_splits` – Number of output partitions.
- `seed` – Optional base seed expanded into partition-local seeds.

Return type

`typing.Any`

Returns

An RDD containing `count_per_split * num_splits` sequences.

Parameters

- `sc` (*SparkContext*)
- `dist` (*SequenceEncodableProbabilityDistribution*)
- `seq_len` (*int*)
- `count_per_split` (*int*)
- `num_splits` (*int*)
- `seed` (*int | None*)

`dmx.stats.rdd_sampler.take_sample(rdd, with_replacement, n, seed=None)`

Take an ordered random sample from an RDD.

Parameters

- `rdd` – Input RDD supporting `zipWithUniqueId` and `takeSample`.
- `with_replacement` – Whether sampling is performed with replacement.
- `n` – Requested sample size.
- `seed` – Optional seed for sampling and output shuffling.

Return type

`list[typing.Any]`

Returns

Locally collected records, randomly reordered after sampling.

Parameters

- **rdd** (*Any*)
- **with_replacement** (*bool*)
- **n** (*int*)
- **seed** (*int* | *None*)

1.4 dmx.bstats API

Bayesian distributions, priors, and variational fitting interfaces.

This package is the Bayesian counterpart to the NumPy/SciPy-oriented [dmx.stats](#) backend. It provides prior-aware distributions and estimators, including Dirichlet-process mixtures, and re-exports the commonly used model classes plus package-level encoding, likelihood, initialization, and estimation helpers. Import specialized priors and protocols from their defining [dmx.bstats](#) submodules when they are not re-exported here.

The package-level fitting helpers operate on local sequences and PySpark RDDs; some scalar paths also accept pandas DataFrames when the accumulator implements the DataFrame protocol. Use [dmx.torch_stats](#) instead when tensor device and dtype behavior is required, and [dmx.mpi4py.bstats](#) for collective MPI operations. Those neighboring backends use distinct model and encoded-data protocols and are not interchangeable implicitly.

This backend is available with the core package. Spark-backed workflows require a working PySpark environment; MPI variants are documented separately under [MPI API](#).

1.4.1 Protocols and Estimation

Core protocols

Define the core interfaces used by Bayesian statistical models.

The classes in this module describe the contracts shared by distributions, samplers, sufficient-statistic accumulators, estimators, and sequence encoders in [dmx.bstats](#). Methods raise `NotImplementedError` when no safe general implementation exists. The classes deliberately avoid abstract base class enforcement because legacy distributions implement partial protocols.

class [dmx.bstats.pdist.DataFrameEncodableAccumulator](#)

Bases: [StatisticAccumulator](#)[X, SS, V]

Accumulator that reads observations from a named DataFrame column.

df_initialize(*df*, *weights*, *rng*)

Initialize statistics from a DataFrame column.

Parameters

- **df** – DataFrame containing the named observation column.
- **weights** – Observation weights.
- **rng** – Random state for randomized initialization.

Raises

ValueError – If the accumulator has no column name.

Return type

`None`

Parameters

- **df** (*DataFrame*)
- **weights** (*Iterable[float]*)
- **rng** (*RandomState*)

df_update(*df*, *weights*, *estimate*)

Update statistics from a DataFrame column.

Parameters

- **df** – DataFrame containing the named observation column.
- **weights** – Observation weights.
- **estimate** – Optional current distribution estimate.

Raises

ValueError – If the accumulator has no column name.

Return type

None

Parameters

- **df** (*DataFrame*)
- **weights** (*Iterable[float]*)
- **estimate** (*ProbabilityDistribution[Any, Any, Any]* | None)

class dmx.bstats.pdist.**DataFrameEncodableDistribution**

Bases: *ProbabilityDistribution*[X, P, V]

Distribution that scores observations in a named DataFrame column.

class dmx.bstats.pdist.**DataSequenceEncoder**

Bases: *Generic*[X, E]

Convert observations to encoded sequence data.

seq_encode(*x*)

Encode independent observations for vectorized operations.

Parameters

x – Observations to encode.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type

dmx.bstats.pdist.EncodedDataSequence[*typing.TypeVar*(E)]

Parameters

x (*Iterable[X]*)

class dmx.bstats.pdist.**DistributionSampler**(*dist*, *seed=None*)

Bases: *Generic*[X]

Base contract for sampling observations from a distribution.

Parameters

- **dist** (*Any*)
- **seed** (*int* | *None*)

new_seed()

Draw a child seed from this sampler's random state.

Return type
int

sample(*size=None*)

Draw one or more observations.

Parameters

size – Number of observations, or *None* for one observation.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type
typing.Any

Parameters

size (*int* | *None*)

class `dmx.bstats.pdist.EncodedDataSequence`(*data*)

Bases: *Generic*[*E*]

Contain distribution-specific data produced by a sequence encoder.

Parameters

data (*E*)

class `dmx.bstats.pdist.ParameterEstimator`

Bases: *Generic*[*X*, *P*, *V*, *SS*]

Estimate a distribution from sufficient statistics.

Implementations accept either `estimate(suff_stat)` or the legacy `estimate(nobs, suff_stat)` form. The sufficient-statistic tuple remains authoritative when it already contains accumulated observation weight.

accumulatorFactory()

Return the factory using the legacy method spelling.

Raises

NotImplementedError – If neither method spelling is implemented.

Return type

`dmx.bstats.pdist.StatisticAccumulatorFactory`[*typing.TypeVar*(*X*), *typing.TypeVar*(*SS*), *typing.TypeVar*(*V*)]

accumulator_factory()

Create an accumulator factory, including legacy implementations.

Raises

NotImplementedError – If neither method spelling is implemented.

Return type

`dmx.bstats.pdist.StatisticAccumulatorFactory`[*typing.TypeVar*(*X*), *typing.TypeVar*(*SS*), *typing.TypeVar*(*V*)]

estimate(*args)

Estimate using either supported legacy call form.

Parameters

***args** – Either `suff_stat` or `nobs`, `suff_stat`. Concrete estimators document whether `nobs` is used.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[`typing.TypeVar(X)`, `typing.TypeVar(P)`, `typing.TypeVar(V)`]

Parameters

args (*Any*)

get_prior()

Return the estimator prior.

Raises

NotImplementedError – If the estimator does not expose a prior.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[`typing.Any`, `typing.Any`, `typing.Any`]

set_prior(prior)

Set the estimator prior.

Parameters

prior – Prior distribution for subsequent estimates.

Raises

NotImplementedError – If the estimator does not support a prior.

Return type

`None`

Parameters

prior (`ProbabilityDistribution`[*Any*, *Any*, *Any*])

class dmx.bstats.pdist.ProbabilityDistribution

Bases: `Generic[X, P, V]`

Base contract for a Bayesian probability distribution.

Concrete likelihoods may attach a conjugate distribution through `get_prior()`; during variational fitting that object contains the current prior or posterior hyperparameters. Prior-only distributions may instead treat their `prior` attribute as metadata, as documented by the concrete class.

add_parent(dist)

Register a distribution that contains this distribution.

Parameters

dist – Parent distribution to register.

Return type

`None`

Parameters**dist** (`ProbabilityDistribution`[*Any*, *Any*, *Any*])**cross_entropy**(*dist*)

Return the cross-entropy relative to another distribution.

Parameters**dist** – Distribution used for the cross-entropy expectation.**Raises****NotImplementedError** – If no generic implementation exists.**Return type**

float

Parameters**dist** (`ProbabilityDistribution`[*Any*, *Any*, *Any*])**density**(*x*)

Evaluate the probability density at an observation.

Parameters**x** – Observation to evaluate.**Return type**

float

ReturnsProbability density at *x*.**Parameters****x** (*X*)**df_log_density**(*df*)

Evaluate log-density for the named column of a DataFrame.

Parameters**df** – DataFrame containing the named observation column.**Return type**`pandas.core.series.Series`**Returns**Log-density indexed like *df*.**Raises****ValueError** – If the distribution has no column name.**Parameters****df** (`DataFrame`)**dist_to_encoder**()

Create the sequence encoder associated with this distribution.

Raises**NotImplementedError** – If the distribution has no separate encoder.**Return type**`dmx.bstats.pdist.DataSequenceEncoder`[`typing.TypeVar`(*X*), `typing.TypeVar`(*V*)]

entropy()

Return the distribution entropy.

Raises

NotImplementedError – If entropy has no generic implementation.

Return type

float

estimator()

Create a parameter estimator for the distribution.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type

`dmx.bstats.pdist.ParameterEstimator`[typing.TypeVar(X), typing.TypeVar(P),
typing.TypeVar(V), typing.Any]

expected_log_density(x)

Evaluate the posterior expected log-density.

Concrete Bayesian likelihoods override this method to average over their current parameter posterior. The base fallback has no separate posterior calculation and evaluates the plug-in parameters.

Parameters

x – Observation to evaluate.

Return type

float

Returns

The plug-in log-density by default.

Parameters

x (*X*)

get_name()

Return the optional distribution name.

Return type

typing.Optional[str]

get_parameters()

Return the distribution parameters.

Return type

typing.TypeVar(P)

get_prior()

Return the prior distribution.

Raises

NotImplementedError – If this distribution does not expose a prior.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[typing.Any, typing.Any, typing.
Any]

log_density(x)

Evaluate the log-density at an observation.

Parameters

x – Observation to evaluate.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type

float

Parameters

x (*X*)

sampler(*seed=None*)

Create a sampler for the distribution.

Parameters

seed – Optional random seed.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type

dmx.bstats.pdist.DistributionSampler[typing.TypeVar(*X*)]

Parameters

seed (*int* | *None*)

seq_encode(*x*)

Encode observations, preserving the input sequence by default.

Parameters

x – Observations to encode.

Return type

typing.TypeVar(*V*)

Returns

Encoded observations.

Parameters

x (*Iterable*[*X*])

seq_expected_log_density(*x*)

Evaluate expected log-densities for an encoded sequence.

The base implementation iterates over **x** and applies *expected_log_density()*; concrete distributions may vectorize it.

Parameters

x – Encoded observations.

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Returns

One-dimensional array containing one expected log-density per observation.

Parameters

x (*V*)

seq_log_density(*x*)

Evaluate log-densities for an iterable encoded sequence.

The base implementation requires *x* itself to be iterable and calls `log_density()` once per item. Concrete encoders may override this contract with a vectorized representation.

Parameters

x – Encoded observations.

Return type

`numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]`

Returns

One-dimensional array containing one log-density per observation.

Parameters

x (*V*)

set_name(*name*)

Set the optional distribution name.

Parameters

name – Name used by DataFrame-based helpers.

Return type

None

Parameters

name (*str* | *None*)

set_parameters(*value*)

Replace the distribution parameters.

Parameters

value – New parameter value.

Return type

None

Parameters

value (*P*)

set_prior(*prior*)

Set the prior distribution.

Parameters

prior – Prior distribution to associate with this distribution.

Raises

NotImplementedError – If this distribution does not support a prior.

Return type

None

Parameters

prior (`ProbabilityDistribution`[*Any*, *Any*, *Any*])

class dmx.bstats.pdist.ProbabilityDistributionFactory

Bases: `Generic[X, P, V]`

Factory for distributions with a shared parameter type.

make(*params*)

Create a distribution from parameters.

Parameters

params – Parameters for the new distribution.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[`typing.TypeVar(X)`, `typing.TypeVar(P)`, `typing.TypeVar(V)`]

Parameters

params (*P*)

class `dmx.bstats.pdist.SequenceEncodableAccumulator`

Bases: `StatisticAccumulator`[*X*, *SS*, *V*]

Accumulator with vectorized sequence update operations.

get_seq_lambda()

Return the sequence update callable for legacy helpers.

Return type

`list[collections.abc.Callable[... , None]]`

seq_initialize(*x*, *weights*, *rng*)

Initialize statistics from an encoded sequence.

Parameters

- **x** – Encoded observations.
- **weights** – Observation weights.
- **rng** – Random state for randomized initialization.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type

`None`

Parameters

- **x** (*V*)
- **weights** (`ndarray[Any, Any]`)
- **rng** (`RandomState`)

seq_update(*x*, *weights*, *estimate*)

Update statistics from an encoded sequence.

Parameters

- **x** – Encoded observations.
- **weights** – Observation weights.
- **estimate** – Optional current distribution estimate.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type

None

Parameters

- **x** (*V*)
- **weights** (*ndarray[Any, Any]*)
- **estimate** (*ProbabilityDistribution[Any, Any, Any]* | None)

class `dmx.bstats.pdist.SequenceEncodableDistribution`

Bases: *ProbabilityDistribution*[X, P, V]

Distribution with default sequence-oriented operations.

seq_encode(x)

Preserve an already sequence-compatible input.

Return type

typing.TypeVar(V)

Parameters

x (*Iterable*[X])

seq_log_density_lambda()

Return the sequence log-density callable for legacy helpers.

Return type

list[*collections.abc.Callable*[[*typing.TypeVar*(V)], *numpy.ndarray*[*typing.Any*, *typing.Any*]]]

class `dmx.bstats.pdist.StatisticAccumulator`

Bases: *Generic*[X, SS, V]

Base contract for accumulating sufficient statistics.

acc_to_encoder()

Create the sequence encoder associated with this accumulator.

Raises

NotImplementedError – If the accumulator has no separate encoder.

Return type

dmx.bstats.pdist.DataSequenceEncoder[*typing.TypeVar*(X), *typing.TypeVar*(V)]

combine(suff_stat)

Merge sufficient statistics into this accumulator.

Parameters

suff_stat – Sufficient statistics to merge.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type

dmx.bstats.pdist.StatisticAccumulator[*typing.TypeVar*(X), *typing.TypeVar*(SS), *typing.TypeVar*(V)]

Parameters

suff_stat (SS)

from_value(*x*)

Restore sufficient statistics from a value.

Parameters

x – Sufficient statistics to restore.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type

`dmx.bstats.pdist.StatisticAccumulator`[`typing.TypeVar(X)`, `typing.TypeVar(SS)`, `typing.TypeVar(V)`]

Parameters

x (*SS*)

initialize(*x*, *weight*, *rng*)

Initialize statistics from one weighted observation.

Parameters

- **x** – Observation to accumulate.
- **weight** – Observation weight.
- **rng** – Random state available to randomized initializers.

Return type

None

Parameters

- **x** (*X*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge statistics shared through named keys.

Parameters

stats_dict – Mutable mapping of shared statistic values.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type

None

Parameters

stats_dict (*MutableMapping[str, Any]*)

key_replace(*stats_dict*)

Replace statistics from values shared through named keys.

Parameters

stats_dict – Mutable mapping of shared statistic values.

Raises

NotImplementedError – Always, unless implemented by a subclass.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**update**(*x*, *weight*, *estimate*)

Accumulate one weighted observation.

Parameters

- **x** – Observation to accumulate.
- **weight** – Observation weight.
- **estimate** – Optional current distribution estimate.

Raises**NotImplementedError** – Always, unless implemented by a subclass.**Return type**

None

Parameters

- **x** (*X*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return the accumulated sufficient statistics.

Raises**NotImplementedError** – Always, unless implemented by a subclass.**Return type***typing.TypeVar*(*SS*)**class** *dmx.bstats.pdist.StatisticAccumulatorFactory*Bases: *Generic*[*X*, *SS*, *V*]

Factory for sufficient-statistic accumulators.

make()

Create a fresh accumulator.

Raises**NotImplementedError** – Always, unless implemented by a subclass.**Return type***dmx.bstats.pdist.StatisticAccumulator*[*typing.TypeVar*(*X*), *typing.TypeVar*(*SS*), *typing.TypeVar*(*V*)]**Estimation workflows**

Local variational estimation helpers for Bayesian models.

Initialization uses the established per-observation Bernoulli subsample and delegates randomized allocation to each estimator accumulator. `optimize` reports data log likelihood (LL), its change (dLL), model-prior and entropy terms (MLL/dMLL), and validation log likelihood (VLL). For a DPM, LL is the block variational objective returned by its sequence scorer. A proposed update is retained only when LL does not decrease, and the returned model is the retained estimate with the best validation score.

```
dmx.bstats.bestimation.best_of(data, vdata, est, trials, max_its, init_p, delta, rng, init_estimator=None,
                               enc_data=None, enc_vdata=None, out=sys.stdout, print_iter=1)
```

Run randomized variational fits and return the best validation model.

Each trial initializes a model from a Bernoulli subsample of the training data. Updates stop when training log-likelihood improvement is below `delta`; `None` disables both that stopping rule and rejection of likelihood-decreasing updates.

Parameters

- **data** – Raw training observations.
- **vdata** – Raw validation observations used to select the returned model.
- **est** – Estimator used for every variational update.
- **trials** – Number of independent randomized initializations.
- **max_its** – Maximum updates per trial.
- **init_p** – Probability that an observation participates in initialization.
- **delta** – Minimum training log-likelihood improvement, or `None` to run every update.
- **rng** – Random state for initialization, or `None` for a fresh state.
- **init_estimator** – Optional estimator used only during initialization.
- **enc_data** – Optional pre-encoded training chunks of the form `(observation_count, encoded_data)`.
- **enc_vdata** – Optional pre-encoded validation chunks in the same form.
- **out** – Text stream receiving progress messages.
- **print_iter** – Emit training progress every this many updates.

Return type

`tuple[float, dmx.bstats.pdist.ProbabilityDistribution[typing.Any, typing.Any, typing.Any]]`

Returns

Best validation log-likelihood and its fitted model.

Raises

- **ValueError** – If `trials` is not positive or `init_p` is outside `(0, 1]`.
- **RuntimeError** – If no trial produces a selectable model.

Parameters

- **data** (`Sequence[T]`)
- **vdata** (`Sequence[T]`)
- **est** (`ParameterEstimator[Any, Any, Any, Any]`)
- **trials** (`int`)
- **max_its** (`int`)
- **init_p** (`float`)
- **delta** (`float | None`)
- **rng** (`RandomState | None`)
- **init_estimator** (`ParameterEstimator[Any, Any, Any, Any] | None`)

- **enc_data** (*Sequence[tuple[int, Any]] | None*)
- **enc_vdata** (*Sequence[tuple[int, Any]] | None*)
- **out** (*IO[str]*)
- **print_iter** (*int*)

`dmx.bstats.bestimation.empirical_kl_divergence(dist1, dist2, enc_data)`

Estimate KL divergence on encoded support shared by two models.

Returns the discrete KL estimate and the number of non-finite log scores produced by each distribution. At least one jointly finite score is required.

Parameters

- **dist1** – Distribution defining the empirical reference probabilities.
- **dist2** – Distribution compared with **dist1**.
- **enc_data** – Sequence of (count, encoded_data) chunks accepted by both distributions.

Return type

`tuple[float, int, int]`

Returns

Tuple containing the empirical divergence, the number of non-finite scores from **dist1**, and the number from **dist2**.

Raises

ValueError – If no observation has finite log-density under both models.

Parameters

- **dist1** (*ProbabilityDistribution[Any, Any, Any]*)
- **dist2** (*ProbabilityDistribution[Any, Any, Any]*)
- **enc_data** (*Sequence[tuple[int, Any]]*)

`dmx.bstats.bestimation.hill_climb(data, vdata, estimator, prev_estimate, max_its, metric_lambda, best_estimate=None, enc_data=None, enc_vdata=None, out=sys.stdout, print_iter=1)`

Return the update with best metric, breaking ties by validation LL.

Parameters

- **data** – Raw training observations.
- **vdata** – Raw validation observations passed to **metric_lambda**.
- **estimator** – Estimator used for each update.
- **prev_estimate** – Model from which to begin updating.
- **max_its** – Number of updates to evaluate.
- **metric_lambda** – Callable returning a score to maximize for a validation data and model pair.
- **best_estimate** – Optional incumbent model included in the comparison.
- **enc_data** – Optional pre-encoded training chunks of the form (observation_count, encoded_data).
- **enc_vdata** – Optional pre-encoded validation chunks in the same form.

- **out** – Text stream receiving progress messages.
- **print_iter** – Emit progress every this many updates.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[typing.Any, typing.Any, typing.Any]

Returns

Model with the largest metric value, using validation log-likelihood to break exact metric ties.

Parameters

- **data** (`Sequence`[`T`])
- **vdata** (`Sequence`[`T`])
- **estimator** (`ParameterEstimator`[`Any`, `Any`, `Any`, `Any`])
- **prev_estimate** (`ProbabilityDistribution`[`Any`, `Any`, `Any`])
- **max_its** (`int`)
- **metric_lambda** (`Callable`[[`Sequence`[`T`], `ProbabilityDistribution`[`Any`, `Any`, `Any`]], `float`])
- **best_estimate** (`ProbabilityDistribution`[`Any`, `Any`, `Any`] | `None`)
- **enc_data** (`Sequence`[`tuple`[`int`, `Any`]] | `None`)
- **enc_vdata** (`Sequence`[`tuple`[`int`, `Any`]] | `None`)
- **out** (`IO`[`str`])
- **print_iter** (`int`)

`dmx.bstats.bestimation.iterate`(`data`, `estimator`, `max_its`, `prev_estimate=None`, `init_p=0.1`, `rng=None`, `out=sys.stdout`, `is_encoded=False`, `init_estimator=None`, `print_iter=1`)

Run exactly `max_its` updates and report mean iteration time.

Parameters

- **data** – Raw observations, or encoded chunks when `is_encoded` is true.
- **estimator** – Estimator used for initialization and every update.
- **max_its** – Number of variational updates to run.
- **prev_estimate** – Existing model to update instead of initializing one.
- **init_p** – Probability that an observation participates in initialization.
- **rng** – Random state for initialization, or `None` for a fresh state.
- **out** – Text stream receiving timing messages.
- **is_encoded** – Whether data already contains encoded chunks.
- **init_estimator** – Optional estimator used only during initialization.
- **print_iter** – Emit mean elapsed time every this many updates.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[typing.Any, typing.Any, typing.Any]

Returns

Model produced by the final update.

Raises

ValueError – If encoded data is supplied without `prev_estimate`, or initialization is needed and `init_p` is outside `(0, 1]`.

Parameters

- `data` (*Sequence*[*T*] | *Sequence*[*tuple*[*int*, *Any*]])
- `estimator` (*ParameterEstimator*[*Any*, *Any*, *Any*, *Any*])
- `max_its` (*int*)
- `prev_estimate` (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)
- `init_p` (*float*)
- `rng` (*RandomState* | *None*)
- `out` (*IO*[*str*])
- `is_encoded` (*bool*)
- `init_estimator` (*ParameterEstimator*[*Any*, *Any*, *Any*, *Any*] | *None*)
- `print_iter` (*int*)

`dmx.bstats.bestimation.k_fold_split_index(sz, k, rng)`

Return a randomized fold identifier for each observation index.

Parameters

- `sz` – Number of observations.
- `k` – Number of folds.
- `rng` – Random state used to permute observations.

Return type

`numpy.ndarray`[`typing.Any`, `typing.Any`]

Returns

Integer array of shape `(sz,)` with fold identifiers in `[0, k)`.

Raises

ValueError – If `k` is not positive.

Parameters

- `sz` (*int*)
- `k` (*int*)
- `rng` (*RandomState*)

`dmx.bstats.bestimation.optimize(data, estimator, max_its=10, delta=1.0e-6, init_estimator=None, init_p=0.1, rng=None, prev_estimate=None, vdata=None, enc_data=None, enc_vdata=None, out=sys.stdout, print_iter=1)`

Optimize locally until LL improvement is below `delta`.

Progress lines expose LL, dLL, MLL, dMLL, and VLL; a line prefixed by `Terminating` records convergence before returning the best retained validation model.

Parameters

- `data` – Raw training observations.

- **estimator** – Estimator used for initialization and updates. It must also provide `model_log_density(model)` for the reported MLL.
- **max_its** – Maximum number of variational updates.
- **delta** – Minimum training log-likelihood improvement, or `None` to run every update and retain likelihood-decreasing proposals.
- **init_estimator** – Optional estimator used only during initialization.
- **init_p** – Probability that an observation participates in initialization.
- **rng** – Random state for initialization, or `None` for a fresh state.
- **prev_estimate** – Existing model to update instead of initializing one.
- **vdata** – Validation observations, defaulting to `data`.
- **enc_data** – Optional pre-encoded training chunks of the form `(observation_count, encoded_data)`.
- **enc_vdata** – Optional pre-encoded validation chunks in the same form.
- **out** – Text stream receiving progress messages.
- **print_iter** – Emit progress every this many updates.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[`typing.Any`, `typing.Any`, `typing.Any`]

Returns

Model with the highest validation log-likelihood among retained estimates.

Raises

- **ValueError** – If initialization is needed and `init_p` is outside `(0, 1]`.
- **AttributeError** – If estimator has no `model_log_density` method.

Parameters

- **data** (`Sequence`[`T`])
- **estimator** (`ParameterEstimator`[`Any`, `Any`, `Any`, `Any`])
- **max_its** (`int`)
- **delta** (`float` | `None`)
- **init_estimator** (`ParameterEstimator`[`Any`, `Any`, `Any`, `Any`] | `None`)
- **init_p** (`float`)
- **rng** (`RandomState` | `None`)
- **prev_estimate** (`ProbabilityDistribution`[`Any`, `Any`, `Any`] | `None`)
- **vdata** (`Sequence`[`T`] | `None`)
- **enc_data** (`Sequence`[`tuple`[`int`, `Any`]] | `None`)
- **enc_vdata** (`Sequence`[`tuple`[`int`, `Any`]] | `None`)
- **out** (`IO`[`str`])
- **print_iter** (`int`)

`dmx.bstats.bestimation.partition_data(data, pvec, rng)`

Randomly partition observations with proportions from `pvec`.

Parameters

- **data** – Observations to partition.
- **pvec** – Proportion assigned to each output partition.
- **rng** – Random state used to permute observations.

Return type

`list[list[typing.TypeVar(T)]]`

Returns

Lists of observations corresponding to `partition_data_index()`.

Parameters

- **data** (*Sequence[T]*)
- **pvec** (*Sequence[float] | ndarray[Any, Any]*)
- **rng** (*RandomState*)

`dmx.bstats.bestimation.partition_data_index(sz, pvec, rng)`

Return randomized index partitions with proportions from `pvec`.

Partition boundaries are rounded cumulative proportions of `sz`. The proportions are used as supplied and need not sum to one.

Parameters

- **sz** – Number of observation indices to partition.
- **pvec** – Proportion assigned to each output partition.
- **rng** – Random state used to permute the indices.

Return type

`list[numpy.ndarray[typing.Any, typing.Any]]`

Returns

One one-dimensional index array per requested proportion.

Parameters

- **sz** (*int*)
- **pvec** (*Sequence[float] | ndarray[Any, Any]*)
- **rng** (*RandomState*)

1.4.2 Prior Distributions

Beta

Define the beta prior used by scalar Bernoulli likelihoods.

The distribution uses the conventional shape parameters `a` and `b` and has support $0 < x < 1$. In Bayesian likelihood modules an instance records the current prior or posterior hyperparameters; its optional `prior` field is metadata and is not folded into density, entropy, or cross-entropy calculations.

class dmx.bstats.beta.**BetaDistribution**(*a*, *b*, *name=None*, *prior=None*)

Bases: *ProbabilityDistribution*[float, tuple[float, float], Any]

Represent a prior or posterior over a Bernoulli probability.

Positive shapes *a* and *b* parameterize the distribution on the open interval $(0, 1)$.

Parameters

- **a** (*float*)
- **b** (*float*)
- **name** (*str* | *None*)
- **prior** (*ProbabilityDistribution*[Any, Any, Any])

cross_entropy(*dist*)

Return $-E_{\text{self}}[\log(\text{dist})]$.

The beta-to-beta case is analytic. Other distributions are integrated numerically over this distribution's support.

Parameters

dist – Distribution whose log-density is averaged.

Return type

float

Returns

Cross-entropy from this distribution to *dist*.

Parameters

dist (*ProbabilityDistribution*[Any, Any, Any])

density(*x*)

Evaluate the density, returning zero outside $0 < x < 1$.

Return type

float

Parameters

x (*float*)

entropy()

Return the differential entropy of the beta distribution.

Return type

float

get_parameters()

Return the (*a*, *b*) shape parameters.

Return type

tuple[float, float]

log_density(*x*)

Evaluate the log-density, returning $-\text{inf}$ outside the support.

Return type

float

Parameters

x (*float*)

sampler(*seed=None*)

Create a beta sampler using an optional deterministic seed.

Return type

`dmx.bstats.beta.BetaSampler`

Parameters

seed (*int* | *None*)

set_parameters(*value*)

Replace both shape parameters and refresh the normalizer.

Parameters

value – Positive (a, b) shape parameters.

Raises

ValueError – If either shape is not finite and positive.

Return type

None

Parameters

value (*tuple*[*float*, *float*])

class `dmx.bstats.beta.BetaSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`[*float*]

Draw independent values from a `BetaDistribution`.

Parameters

- **dist** (`BetaDistribution`)
- **seed** (*Optional*[*int*])

sample(*size=None*)

Draw one float or an array of size beta values.

Return type

`typing.Any`

Parameters

size (*int* | *None*)

Dictionary Dirichlet

Dictionary-keyed Dirichlet priors for categorical probability mappings.

A mapping assigns a concentration to each fixed key and supports dictionary- shaped sampling. A scalar is the compatibility form used by categorical defaults and applies to whatever keys an observation contains. With no stored keys, that form cannot be sampled or have an entropy computed.

class `dmx.bstats.catdirichlet.DictDirichletDistribution`(*alpha*)

Bases: `ProbabilityDistribution`[`Mapping`[*Any*, *float*], `dict`[*Any*, *float*] | *float*, *Any*]

Represent a prior or posterior over keyed categorical probabilities.

Mapping parameters fix the category set and assign one positive concentration per key. A scalar parameter applies the same concentration to the keys of each scored observation, but does not provide enough support information for sampling or entropy.

Parameters

alpha (*Union*[*Mapping*[*Key*, *float*], *float*])

concentrations_for_keys(*keys*)

Return dense concentrations in the requested key order.

Parameters

keys – Ordered category keys for the returned vector.

Return type

`numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]`

Returns

Concentration array of shape `(len(keys),)`.

Raises

ValueError – If keys differ from fixed mapping parameters.

Parameters

keys (*list* [*Any*])

cross_entropy(*dist*)

Return `-E_self[log(dist)]` for a compatible keyed distribution.

Parameters

dist – Distribution whose log-density is averaged.

Return type

`float`

Returns

Analytic cross-entropy from this distribution to **dist**.

Raises

- **ValueError** – If neither distribution defines keys or their fixed key sets differ.
- **NotImplementedError** – If **dist** is not dictionary Dirichlet.

Parameters

dist (*Any*)

entropy()

Return differential entropy for fixed dictionary parameters.

Raises

ValueError – If scalar parameters do not define category keys.

Return type

`float`

get_parameters()

Return mapping concentrations or the scalar compatibility value.

Return type

`typing.Union[dict[typing.Any, float], float]`

log_density(*x*)

Evaluate the log-density of a keyed simplex observation.

Parameters

x – Mapping of category keys to finite, strictly positive probabilities summing to one.

Return type

`float`

Returns

Log-density, or `-inf` when the probabilities are outside the simplex.

Raises

ValueError – If observation keys differ from fixed mapping parameters.

Parameters

x (*Mapping*[*Any*, *float*])

sampler(*seed=None*)

Create a keyed sampler for fixed dictionary parameters.

Parameters

seed – Optional deterministic random seed.

Return type

dmx.bstats.catdirichlet.DictDirichletSampler

Returns

Sampler preserving the parameter mapping's key order.

Raises

ValueError – If scalar parameters do not define category keys.

Parameters

seed (*int* | *None*)

set_parameters(*value*)

Replace fixed-key or dimension-free concentrations.

Parameters

value – Nonempty concentration mapping or positive shared scalar.

Raises

ValueError – If concentrations are empty, non-finite, or nonpositive.

Return type

None

Parameters

value (*Mapping*[*Any*, *float*] | *float*)

class *dmx.bstats.catdirichlet.DictDirichletSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*[*Mapping*[*Any*, *float*]]

Draw keyed simplex mappings from a dictionary Dirichlet prior.

Parameters

- **dist** (*DictDirichletDistribution*)
- **seed** (*Optional*[*int*])

sample(*size=None*)

Draw one mapping or a list of *size* mappings.

Return type

typing.Any

Parameters

size (*int* | *None*)

Dirichlet

Dirichlet priors for dense categorical probability vectors.

Vector parameters define an ordinary fixed-dimensional Dirichlet distribution. A scalar parameter is a compatibility form for a symmetric prior whose dimension is inferred from each scored observation. Scalar parameters therefore support density evaluation but not sampling, entropy, or estimation until a dimension is known.

class `dmx.bstats.dirichlet.DirichletAccumulator`(*dim*, *keys=None*)

Bases: [SequenceEncodableAccumulator](#)[Sequence[float] | ndarray[Any, dtype[float64]], tuple[float, ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]]], tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]]]

Accumulate weighted statistics for Dirichlet estimation.

The public statistic is (weight, sum_log_x, sum_x, sum_x_squared). Each vector has shape (d,) and contains coordinate-wise weighted sums.

Parameters

- **dim** (*int*)
- **keys** (*Optional[str]*)

combine(*suff_stat*)

Merge another sufficient-statistic tuple.

Parameters

suff_stat – (weight, sum_log_x, sum_x, sum_x_squared) to add.

Return type

[dmx.bstats.dirichlet.DirichletAccumulator](#)

Returns

This mutated accumulator.

Parameters

suff_stat (tuple[float, ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]])

from_value(*x*)

Restore a sufficient-statistic tuple.

Return type

[dmx.bstats.dirichlet.DirichletAccumulator](#)

Parameters

x (tuple[float, ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]])

key_merge(*stats_dict*)

Merge statistics under the optional shared key.

Return type

None

Parameters

stats_dict (*MutableMapping[str, Any]*)

key_replace(*stats_dict*)

Restore statistics from the optional shared key.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**seq_update**(*x*, *weights*, *estimate*)

Add encoded observations with corresponding weights.

Parameters

- **x** – Encoded tuple whose arrays have shape (n, d).
- **weights** – Observation weights of shape (n,).
- **estimate** – Current estimate; unused for Dirichlet statistics.

Return type

None

Parameters

- **x** (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]])
- **weights** (*ndarray*[*Any*, *dtype*[*float64*]])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Add one weighted simplex observation.

Parameters

- **x** – Dense probability vector of shape (d,).
- **weight** – Weight applied to the observation.
- **estimate** – Current estimate; unused for Dirichlet statistics.

Return type

None

Parameters

- **x** (*Sequence*[*float*] | *ndarray*[*Any*, *dtype*[*float64*]])
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return (weight, sum_log_x, sum_x, sum_x_squared) statistics.

Return type

tuple[*float*, *numpy.ndarray*[*typing.Any*, *numpy.dtype*[*numpy.float64*]], *numpy.ndarray*[*typing.Any*, *numpy.dtype*[*numpy.float64*]], *numpy.ndarray*[*typing.Any*, *numpy.dtype*[*numpy.float64*]]]

class dmx.bstats.dirichlet.**DirichletAccumulatorFactory**(*dim*, *keys=None*)

Bases: *StatisticAccumulatorFactory*[*Sequence*[*float*] | *ndarray*[*Any*, *dtype*[*float64*]], *tuple*[*float*, *ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]], *tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]]

Create Dirichlet sufficient-statistic accumulators.

Parameters

- **dim** (*int*)
- **keys** (*Optional[str]*)

make()

Create a zeroed accumulator.

Return type

dmx.bstats.dirichlet.DirichletAccumulator

class *dmx.bstats.dirichlet.DirichletDistribution*(*alpha*)

Bases: *ProbabilityDistribution*[*Sequence[float]* | *ndarray[Any, dtype[float64]]*, *float* | *ndarray[Any, dtype[float64]]*, *tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]]*, *ndarray[Any, dtype[float64]]*]

Represent a prior or posterior over dense categorical probabilities.

A vector *alpha* fixes dimension *d* and supplies one positive concentration per coordinate. A scalar supplies the same concentration to every coordinate and infers *d* from each scored observation. The support is the interior of the *d*-dimensional probability simplex.

Parameters

alpha (*Union[float, Sequence[float], Array]*)

concentrations_for_dimension(*dimension*)

Return dense concentrations for a requested dimension.

Parameters

dimension – Required number of simplex coordinates.

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Returns

Concentration vector of shape (*dimension*,).

Raises

ValueError – If fixed vector parameters have another dimension.

Parameters

dimension (*int*)

cross_entropy(*dist*)

Return $-E_{\text{self}}[\log(\text{dist})]$ for another Dirichlet distribution.

Parameters

dist – Distribution whose log-density is averaged.

Return type

float

Returns

Analytic cross-entropy from this distribution to *dist*.

Raises

- **ValueError** – If both distributions are dimension-free or their fixed dimensions differ.
- **NotImplementedError** – If *dist* is not a Dirichlet distribution.

Parameters

dist (*ProbabilityDistribution[Any, Any, Any]*)

entropy()

Return differential entropy for fixed-dimensional parameters.

Raises

ValueError – If the distribution has scalar, dimension-free parameters.

Return type

float

estimator(*pseudo_count=None*)

Create an estimator for fixed-dimensional dense parameters.

Parameters

pseudo_count – Optional regularization weight for a log-statistic derived from the current concentrations.

Return type

dmx.bstats.dirichlet.DirichletEstimator

Returns

Estimator configured for this distribution's dimension.

Raises

ValueError – If the distribution has scalar, dimension-free parameters.

Parameters

pseudo_count (*float* | *None*)

get_parameters()

Return the scalar or dense concentration parameters.

Return type

typing.Union[float, *numpy.ndarray*[*typing.Any*, *numpy.dtype*[*numpy.float64*]]]

log_density(*x*)

Evaluate the log-density at a dense simplex point.

Parameters

x – Probability vector of shape (d,) with finite, strictly positive entries summing to one.

Return type

float

Returns

Log-density, or -inf when x is outside the simplex.

Raises

ValueError – If x conflicts with fixed vector parameters.

Parameters

x (*Sequence*[float] | *ndarray*[*Any*, *dtype*[float64]])

sampler(*seed=None*)

Create a sampler for fixed-dimensional dense parameters.

Parameters

seed – Optional deterministic random seed.

Return type

dmx.bstats.dirichlet.DirichletSampler

Returns

Sampler for this distribution.

Raises

ValueError – If the distribution has scalar, dimension-free parameters.

Parameters

`seed` (*int* | *None*)

seq_encode(*x*)

Encode observations as logs, values, and squared values.

Parameters

`x` – Iterable of *n* dense observations with dimension *d*.

Return type

tuple[numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]], numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]], numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]]

Returns

Tuple of log values, values, and squared values, each shaped (*n*, *d*). Logs are clipped at the smallest positive float.

Parameters

`x` (*Iterable[Sequence[float]* | *ndarray[Any, dtype[float64]]*)

seq_log_density(*x*)

Evaluate log-densities for encoded simplex observations.

Parameters

`x` – Tuple of log values, values, and squared values, each shaped (*n*, *d*) as returned by [seq_encode\(\)](#).

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Returns

Log-density array of shape (*n*,). Rows outside the simplex receive `-inf`.

Raises

ValueError – If encoded values are not a matrix or their dimension conflicts with fixed vector parameters.

Parameters

`x` (*tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]]*)

set_parameters(*value*)

Replace concentrations and refresh cached dimensional state.

Parameters

`value` – Positive scalar or nonempty vector of positive concentrations.

Raises

ValueError – If concentrations are empty, non-finite, nonpositive, or not scalar or one-dimensional.

Return type

None

Parameters

`value` (*float* | *Sequence[float]* | *ndarray[Any, dtype[float64]]*)

```
class dmx.bstats.dirichlet.DirichletEstimator(dim, pseudo_count=None, suff_stat=None,
                                             delta=1.0e-8, keys=None, use_mpe=False)
```

Bases: [ParameterEstimator](#)[Sequence[float] | ndarray[Any, dtype[float64]], float | ndarray[Any, dtype[float64]], tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], tuple[float, ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]]]

Estimate dense Dirichlet concentrations from sufficient statistics.

With `pseudo_count`, the estimator adds weighted prior log-statistics before solving the concentration fixed-point equation. Without it, the empirical mean initializes the solver.

Parameters

- **dim** (*int*)
- **pseudo_count** (*Optional[float]*)
- **suff_stat** (*Optional[Array]*)
- **delta** (*float*)
- **keys** (*Optional[str]*)
- **use_mpe** (*bool*)

accumulator_factory()

Create an accumulator factory for this estimator.

Return type

[dmx.bstats.dirichlet.DirichletAccumulatorFactory](#)

estimate(*args)

Estimate concentrations using either legacy estimator call form.

Parameters

***args** – Either one sufficient-statistic tuple, or nobs followed by that tuple. nobs is accepted for protocol compatibility; the tuple’s accumulated weight is authoritative.

Return type

[dmx.bstats.dirichlet.DirichletDistribution](#)

Returns

Fixed-dimensional Dirichlet distribution with fitted concentrations.

Raises

- **TypeError** – If called with any other number of arguments.
- **ValueError** – If accumulated observation weight is not positive.

Parameters

args (*Any*)

```
class dmx.bstats.dirichlet.DirichletSampler(dist, seed=None)
```

Bases: [DistributionSampler](#)[Sequence[float] | ndarray[Any, dtype[float64]]]

Draw dense simplex vectors from fixed-dimensional parameters.

Parameters

- **dist** ([DirichletDistribution](#))
- **seed** (*Optional[int]*)

sample(*size=None*)

Draw one vector or an array shaped (size, dimension).

Return type

typing.Any

Parameters

size (*int* | *None*)

`dmx.bstats.dirichlet.alpha_seq_lambda(mean_log_p)`

Create the fixed-point update for Dirichlet concentrations.

Parameters

mean_log_p – Mean log probabilities of shape (d,).

Return type

`collections.abc.Callable[[numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]], numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]]`

Returns

Callable mapping a concentration vector of shape (d,) to its next fixed-point iterate.

Parameters

mean_log_p (*ndarray[Any, dtype[float64]]*)

`dmx.bstats.dirichlet.dirichlet_param_solve(alpha, mean_log_p, delta)`

Solve the Dirichlet fixed-point equation.

Parameters

- **alpha** – Initial concentration vector of shape (d,).
- **mean_log_p** – Mean log probabilities of shape (d,). Non-finite coordinates are excluded from the iteration.
- **delta** – Convergence threshold for relative absolute parameter change.

Return type

`tuple[numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]], int]`

Returns

Estimated concentrations of shape (d,) and iteration count. Excluded coordinates are returned as zero.

Parameters

- **alpha** (*ndarray[Any, dtype[float64]]*)
- **mean_log_p** (*ndarray[Any, dtype[float64]]*)
- **delta** (*float*)

`dmx.bstats.dirichlet.find_alpha(current_alpha, mean_log_p, threshold)`

Estimate concentrations using accelerated fixed-point iteration.

Parameters

- **current_alpha** – Initial concentration vector of shape (d,).
- **mean_log_p** – Mean log probabilities of shape (d,).
- **threshold** – Absolute-residual convergence threshold.

Return type

`tuple[numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]], int]`

Returns

Estimated concentrations of shape (d,) and update count.

Parameters

- **current_alpha** (*ndarray*[Any, dtype[*float64*]])
- **mean_log_p** (*ndarray*[Any, dtype[*float64*]])
- **threshold** (*float*)

`dmx.bstats.dirichlet.mpe(initial, update, epsilon)`

Accelerate fixed-point iteration with polynomial extrapolation.

Parameters

- **initial** – Initial parameter vector of shape (d,).
- **update** – Callable performing one fixed-point update on that shape.
- **epsilon** – Absolute-residual convergence threshold.

Return type

`tuple[numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]], int]`

Returns

Extrapolated parameter vector of shape (d,) and update count.

Parameters

- **initial** (*ndarray*[Any, dtype[*float64*]])
- **update** (*Callable[[ndarray[Any, dtype[*float64*]]], ndarray[Any, dtype[*float64*]]]*)
- **epsilon** (*float*)

Multivariate normal-gamma

Define independent normal-gamma priors for diagonal Gaussian parameters.

For every coordinate, $\tau_{i_i} \sim \text{Gamma}(a_i, \text{scale}=1 / b_i)$ and $x_i | \tau_{i_i} \sim \text{Normal}(\mu_i, \text{variance}=1 / (\text{lam}_i * \tau_{i_i}))$. The joint support contains finite location vectors and strictly positive precision vectors. Diagonal Gaussian likelihoods use these arrays as current prior or posterior hyperparameters; the optional prior attribute is metadata only.

```
class dmx.bstats.mvngamma.MultivariateNormalGammaDistribution(mu, lam, a, b, name=None,
                                                             prior=None)
```

Bases: `ProbabilityDistribution[tuple[Sequence[float] | ndarray[Any, dtype[Any]], Sequence[float] | ndarray[Any, dtype[Any]], tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], Any]`

Represent independent normal-gamma factors for diagonal parameters.

Each hyperparameter is a vector of shape (d,). Observations are pairs (location, precision) of that shape, with finite locations and strictly positive finite precisions. Instances represent current prior or posterior hyperparameters for a diagonal Gaussian likelihood.

Parameters

- **mu** (*np.ndarray*[Any, Any])
- **lam** (*np.ndarray*[Any, Any])
- **a** (*np.ndarray*[Any, Any])

- **b** (*np.ndarray*[Any, Any])
- **name** (*str* | *None*)
- **prior** (*ProbabilityDistribution*[Any, Any, Any])

cross_entropy(*dist*)

Return $-E_{\text{self}}[\log(\text{dist})]$ for the same distribution family.

There is no generic numerical fallback for arbitrary multivariate distributions because their dimension and integration contract are not available through the base interface.

Parameters

dist – Multivariate normal-gamma distribution to compare.

Return type

float

Returns

Sum of coordinate-wise analytic cross-entropies.

Raises

NotImplementedError – If *dist* is from another family.

Parameters

dist (*ProbabilityDistribution*[Any, Any, Any])

density(*x*)

Evaluate the joint density, returning zero outside the support.

Return type

float

Parameters

x (*tuple*[*Sequence*[float] | *ndarray*[Any, dtype[Any]], *Sequence*[float] | *ndarray*[Any, dtype[Any]]])

entropy()

Return the summed differential entropy of all coordinates.

Return type

float

get_parameters()

Return (*mu*, *lam*, *a*, *b*) parameter arrays.

Return type

tuple[*numpy.ndarray*[*typing.Any*, *numpy.dtype*[*numpy.float64*]], *numpy.ndarray*[*typing.Any*, *numpy.dtype*[*numpy.float64*]], *numpy.ndarray*[*typing.Any*, *numpy.dtype*[*numpy.float64*]], *numpy.ndarray*[*typing.Any*, *numpy.dtype*[*numpy.float64*]]]

log_density(*x*)

Evaluate the joint log-density, returning $-\infty$ off support.

Return type

float

Parameters

x (*tuple*[*Sequence*[float] | *ndarray*[Any, dtype[Any]], *Sequence*[float] | *ndarray*[Any, dtype[Any]]])

sampler(*seed=None*)

Create a joint sampler using an optional deterministic seed.

Return type

`dmx.bstats.mvngamma.MultivariateNormalGammaSampler`

Parameters

seed (*int* | *None*)

set_parameters(*value*)

Replace all vector normal-gamma hyperparameters.

Parameters

value – Tuple of (*mu*, *lam*, *a*, *b*) arrays with matching shapes.

Raises

ValueError – If arrays differ in shape or contain invalid parameters.

Return type

None

Parameters

value (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]])

class `dmx.bstats.mvngamma.MultivariateNormalGammaSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`[*tuple*[*Sequence*[*float*] | *ndarray*[*Any*, *dtype*[*Any*]], *Sequence*[*float*] | *ndarray*[*Any*, *dtype*[*Any*]]]

Draw vector locations and precisions from normal-gamma factors.

Parameters

- **dist** (`MultivariateNormalGammaDistribution`)
- **seed** (*Optional*[*int*])

sample(*size=None*)

Draw one pair or a list of size vector pairs.

Return type

typing.Any

Parameters

size (*int* | *None*)

Normal-gamma

Define the normal-gamma prior for a Gaussian mean and precision.

The parameterization is $\tau \sim \text{Gamma}(a, \text{scale}=1 / b)$ and $x \mid \tau \sim \text{Normal}(\mu, \text{variance}=1 / (\text{lam} * \tau))$. Thus *b* is a rate and the joint support is a real location paired with positive precision. Instances are used as prior or posterior hyperparameters by Gaussian likelihoods; the optional *prior* attribute is metadata only.

class `dmx.bstats.normgamma.NormalGammaDistribution`(*mu*, *lam*, *a*, *b*, *name=None*, *prior=None*)

Bases: `ProbabilityDistribution`[*tuple*[*float*, *float*], *tuple*[*float*, *float*, *float*, *float*], *Any*]

Represent a prior or posterior over a Gaussian mean and precision.

Observations are (*location*, *precision*) pairs with finite location and strictly positive finite precision. The gamma factor uses shape *a* and rate *b*; conditional location variance is $1 / (\text{lam} * \text{precision})$.

Parameters

- **mu** (*float*)
- **lam** (*float*)
- **a** (*float*)
- **b** (*float*)
- **name** (*str* | *None*)
- **prior** ([ProbabilityDistribution](#)[*Any*, *Any*, *Any*])

cross_entropy(*dist*)

Return $-E_{\text{self}}[\log(\text{dist})]$.

The normal-gamma case is analytic. Other distributions are integrated numerically over real locations and positive precisions.

Parameters

dist – Distribution whose joint log-density is averaged.

Return type

float

Returns

Cross-entropy from this distribution to *dist*.

Parameters

dist ([ProbabilityDistribution](#)[*Any*, *Any*, *Any*])

density(*x*)

Evaluate the joint density, returning zero for invalid precision.

Return type

float

Parameters

x (*tuple*[*float*, *float*])

entropy()

Return the differential entropy of the joint distribution.

Return type

float

get_parameters()

Return (mu, lam, a, b) in the documented parameterization.

Return type

tuple[*float*, *float*, *float*, *float*]

log_density(*x*)

Evaluate the joint log-density.

A nonfinite location or nonpositive/nonfinite precision is outside the support and returns $-\text{inf}$.

Return type

float

Parameters

x (*tuple*[*float*, *float*])

sampler(*seed=None*)

Create a joint sampler using an optional deterministic seed.

Return type

dmx.bstats.normgamma.NormalGammaSampler

Parameters

seed (*int* | *None*)

set_parameters(*value*)

Replace all normal-gamma hyperparameters.

Parameters

value – Tuple (*mu*, *lam*, *a*, *b*).

Raises

ValueError – If the parameters do not define a finite distribution.

Return type

None

Parameters

value (*tuple*[*float*, *float*, *float*, *float*])

class *dmx.bstats.normgamma.NormalGammaSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*[*tuple*[*float*, *float*]]

Draw locations and precisions from a normal-gamma distribution.

Parameters

- **dist** (*NormalGammaDistribution*)
- **seed** (*Optional*[*int*])

sample(*size=None*)

Draw one pair or a list of *size* location-precision pairs.

Return type

typing.Any

Parameters

size (*int* | *None*)

Symmetric Dirichlet

Symmetric Dirichlet priors for categorical probability vectors.

Density evaluation infers dimension from its observation. Sampling and information measures require *ndim* because no observation is available.

class *dmx.bstats.symdirichlet.SymmetricDirichletDistribution*(*alpha*, *ndim=None*)

Bases: *SequenceEncodableDistribution*[*Sequence*[*float*], *float*, *Sequence*[*Sequence*[*float*]]]

Represent a symmetric prior or posterior over probability vectors.

alpha is repeated across every simplex coordinate. The support is the interior of the simplex; *ndim* fixes its dimension when supplied and is otherwise inferred only for density evaluation.

Parameters

- **alpha** (*float*)
- **ndim** (*Optional*[*int*])

cross_entropy(*dist*)

Return $-E_{\text{self}}[\log(\text{dist})]$ for a compatible symmetric prior.

Parameters

dist – Distribution whose log-density is averaged.

Return type

float

Returns

Analytic cross-entropy from this distribution to **dist**.

Raises

- **ValueError** – If this distribution has no explicit dimension or the configured dimensions differ.
- **NotImplementedError** – If **dist** is not symmetric Dirichlet.

Parameters

dist (*Any*)

dimension()

Return the dimension required by non-scoring operations.

Raises

ValueError – If no explicit dimension was configured.

Return type

int

entropy()

Return differential entropy for the explicit dimension.

Raises

ValueError – If no explicit dimension was configured.

Return type

float

get_parameters()

Return the shared scalar concentration.

Return type

float

log_density(*x*)

Evaluate the log-density, inferring dimension from **x**.

Parameters

x – Finite, strictly positive probability vector summing to one.

Return type

float

Returns

Log-density, or $-\infty$ outside the configured simplex support.

Parameters

x (*Sequence[float]*)

sampler(*seed=None*)

Create a sampler, requiring an explicit dimension.

Parameters

seed – Optional deterministic random seed.

Return type

dmx.bstats.syndirichlet.SymmetricDirichletSampler

Returns

Sampler for this fixed-dimensional distribution.

Raises

ValueError – If no explicit dimension was configured.

Parameters

seed (*int* | *None*)

set_parameters(*value*)

Replace the shared concentration.

Parameters

value – Positive finite concentration.

Raises

ValueError – If value is non-finite or nonpositive.

Return type

None

Parameters

value (*float*)

class *dmx.bstats.syndirichlet.SymmetricDirichletSampler*(*dist, seed=None*)

Bases: *DistributionSampler*[*Sequence*[*float*]]

Draw vectors from a fixed-dimensional symmetric Dirichlet prior.

Parameters

- **dist** (*SymmetricDirichletDistribution*)
- **seed** (*Optional*[*int*])

sample(*size=None*)

Draw one vector or an array shaped (size, dimension).

Return type

typing.Any

Parameters

size (*int* | *None*)

1.4.3 Likelihood Distributions

Bernoulli

Bayesian Bernoulli likelihoods for binary observations.

BernoulliDistribution(*p*) assigns probability *p* to True and 1 - *p* to False. The default *Beta*(1.0000001, 1.0000001) prior is nearly uniform while retaining a unique interior MAP estimate. Accumulators store weighted (*true_count*, *false_count*) statistics. With a beta prior, expected log-density integrates the parameter under that prior; otherwise it falls back to the fixed-parameter log-density.

class `dmx.bstats.bernoulli.BernoulliDataEncoder`

Bases: `DataSequenceEncoder`[`bool`, `ndarray`[`Any`, `dtype`[`bool`]]]

Encode Bernoulli observations as booleans.

seq_encode(*x*)

Encode observations in a typed container.

Return type

`dmx.bstats.bernoulli.BernoulliEncodedData`

Parameters

x (`Iterable`[`bool`])

class `dmx.bstats.bernoulli.BernoulliDistribution`(*p*, *name=None*, *prior=default_prior*, *keys=None*)

Bases: `ProbabilityDistribution`[`bool`, `float`, `ndarray`[`Any`, `dtype`[`bool`]]]

Represent a Bernoulli likelihood on boolean observations.

p is the probability of True and must lie in `[0, 1]`. A beta prior enables posterior-expected scoring; any other prior is retained as metadata and fixed-parameter scoring is used.

Parameters

- **p** (`float`)
- **name** (`str` | `None`)
- **prior** (`ProbabilityDistribution`[`Any`, `Any`, `Any`])
- **keys** (`Optional`[`str`])

cross_entropy(*dist*)

Return `-E_self[log(dist)]` over the two outcomes.

Return type

`float`

Parameters

dist (`ProbabilityDistribution`[`Any`, `Any`, `Any`])

dist_to_encoder()

Create the Bernoulli sequence encoder.

Return type

`dmx.bstats.bernoulli.BernoulliDataEncoder`

entropy()

Return the Bernoulli Shannon entropy.

Return type

`float`

estimator()

Create an estimator retaining metadata and the current prior.

Return type

`dmx.bstats.bernoulli.BernoulliEstimator`

expected_log_density(*x*)

Evaluate log mass averaged under a beta prior when available.

Return type

`float`

Parameters**x** (*bool*)**get_data_type()**

Return the intended observation type.

Return typetype[*bool*]**get_parameters()**

Return the success probability.

Return type

float

get_prior()

Return the current beta or nonconjugate prior.

Return type*dmx.bstats.pdist.ProbabilityDistribution*[typing.Any, typing.Any, typing.Any]**log_density(x)**

Evaluate the log mass for a boolean observation.

Return type

float

Parameters**x** (*bool*)**moment(order)**

Return the raw moment for a nonnegative order.

Return type

float

Parameters**order** (*int*)**sampler(seed=None)**

Create a repeatable Bernoulli sampler.

Return type*dmx.bstats.bernoulli.BernoulliSampler***Parameters****seed** (*int* | *None*)**seq_encode(x)**

Encode n observations as a boolean array of shape (n,).

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.bool]]

Parameters**x** (*Iterable[bool]*)**seq_expected_log_density(x)**

Evaluate expected log masses for encoded observations.

Return type`numpy.ndarray[typing.Any, typing.Any]`**Parameters**`x (ndarray[Any, dtype[bool]])`**seq_log_density(x)**

Evaluate fixed-parameter log masses for encoded observations.

Return type`numpy.ndarray[typing.Any, typing.Any]`**Parameters**`x (ndarray[Any, dtype[bool]])`**set_parameters(value)**

Replace the success probability and cached logarithms.

Return type`None`**Parameters**`value (float)`**set_prior(prior)**

Replace the prior and cache beta expected-log parameters.

Return type`None`**Parameters**`prior (ProbabilityDistribution[Any, Any, Any])`**class dmx.bstats.bernoulli.BernoulliEncodedData(data)**

Bases: [EncodedDataSequence](#)[`ndarray[Any, dtype[bool]]`]

Contain an encoded Bernoulli sequence.

Parameters`data (BernoulliEncoded)`**class dmx.bstats.bernoulli.BernoulliEstimator(name=None, keys=None, prior=default_prior)**

Bases: [ParameterEstimator](#)[`bool`, `float`, `ndarray[Any, dtype[bool]]`, `tuple[float, float]`]

Estimate a Bernoulli probability from weighted outcome counts.

A beta prior is updated to its posterior and the returned probability is its interior mode when defined. Another non-null prior produces a numerical MAP estimate; a null prior produces the weighted maximum-likelihood value.

Parameters

- `name` (*Optional*[`str`])
- `keys` (*Optional*[`str`])
- `prior` (*Model*)

accumulator_factory()

Create a compatible accumulator factory.

Return type`dmx.bstats.bernoulli.BernoulliEstimatorAccumulatorFactory`

estimate(*args)

Estimate from (true_count, false_count) statistics.

Parameters

***args** – Either the statistic tuple alone or legacy nobs followed by the tuple. nobs is ignored.

Return type

`dmx.bstats.bernoulli.BernoulliDistribution`

Returns

Bernoulli likelihood carrying the updated posterior when the prior is conjugate.

Parameters

args (*Any*)

get_prior()

Return the estimator prior.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[*typing.Any*, *typing.Any*, *typing.Any*]

set_prior(prior)

Replace the estimator prior and update prior flags.

Return type

None

Parameters

prior (`ProbabilityDistribution`[*Any*, *Any*, *Any*])

class `dmx.bstats.bernoulli.BernoulliEstimatorAccumulator`(name, keys)

Bases: `StatisticAccumulator`[*bool*, *tuple*[*float*, *float*], *ndarray*[*Any*, *dtype*[*bool*]]]

Accumulate weighted (true_count, false_count) statistics.

Parameters

- **name** (*Optional*[*str*])
- **keys** (*Optional*[*str*])

acc_to_encoder()

Create the compatible Bernoulli encoder.

Return type

`dmx.bstats.bernoulli.BernoulliDataEncoder`

combine(suff_stat)

Merge true and false counts.

Return type

`dmx.bstats.bernoulli.BernoulliEstimatorAccumulator`

Parameters

suff_stat (*tuple*[*float*, *float*])

from_value(x)

Restore (true_count, false_count).

Return type

`dmx.bstats.bernoulli.BernoulliEstimatorAccumulator`

Parameters**x** (*tuple*[*float*, *float*])**initialize**(*x*, *weight*, *rng*)

Accumulate one observation during initialization.

Return type

None

Parameters

- **x** (*bool*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge statistics through the configured sharing key.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**key_replace**(*stats_dict*)

Replace statistics through the configured sharing key.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *rng*)

Accumulate encoded observations during initialization.

Return type

None

Parameters

- **x** (*ndarray*[*Any*, *dtype*[*bool*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Add encoded booleans with corresponding weights.

Return type

None

Parameters

- **x** (*ndarray*[*Any*, *dtype*[*bool*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Add one weighted boolean observation.

Return type

None

Parameters

- **x** (*bool*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return (true_count, false_count).

Return type

tuple[float, float]

class dmx.bstats.bernoulli.**BernoulliEstimatorAccumulatorFactory**(*name*, *keys*)

Bases: *StatisticAccumulatorFactory*[bool, tuple[float, float], ndarray[*Any*, dtype[bool]]]

Create Bernoulli accumulators with shared metadata.

Parameters

- **name** (*Optional*[*str*])
- **keys** (*Optional*[*str*])

make()

Create an empty Bernoulli accumulator.

Return type

dmx.bstats.bernoulli.BernoulliEstimatorAccumulator

class dmx.bstats.bernoulli.**BernoulliSampler**(*dist*, *seed*=*None*)

Bases: *DistributionSampler*[bool]

Draw independent boolean Bernoulli observations.

Parameters

- **dist** (*BernoulliDistribution*)
- **seed** (*Optional*[*int*])

sample(*size*=*None*)

Draw one boolean or a list of *size* booleans.

Return type

typing.Any

Parameters

size (*int* | *None*)

Categorical

Bayesian categorical likelihoods on arbitrary hashable observations.

The probability mapping defines the sampled support. Observations missing from that mapping receive `default_value / (1 + default_value)` mass, matching the legacy escape-mass convention. The default dimension-free dictionary Dirichlet prior uses concentration $1 + 1e-12$ per observed category. Accumulators store a

weighted category-count mapping and its total. Expected log-density uses Dirichlet expectations on known categories and the fixed escape mass for unknown categories.

class `dmx.bstats.categorical.CategoricalDataEncoder`

Bases: `DataSequenceEncoder`[Any, tuple[ndarray[Any, Any], ndarray[Any, Any]]]

Encode arbitrary categorical observations.

seq_encode(*x*)

Encode observations in a typed container.

Return type

`dmx.bstats.categorical.CategoricalEncodedData`

Parameters

x (`Iterable`[Any])

class `dmx.bstats.categorical.CategoricalDistribution`(*prob_map*, *default_value*=0.0, *name*=None, *prior*=*default_prior*)

Bases: `ProbabilityDistribution`[Any, dict[Any, float], tuple[ndarray[Any, Any], ndarray[Any, Any]]]

Represent a categorical likelihood over arbitrary hashable labels.

prob_map defines the explicit sampling support and sums to one before legacy escape-mass normalization. A dictionary Dirichlet prior enables posterior-expected scoring on the explicit categories; another prior uses fixed-parameter scoring.

Parameters

- **prob_map** (`dict`[Any, float])
- **default_value** (float)
- **name** (`str` | None)
- **prior** (`ProbabilityDistribution`[Any, Any, Any])

cross_entropy(*dist*)

Return $-E_{\text{self}}[\log(\text{dist})]$ over explicit categories.

Return type

float

Parameters

dist (`ProbabilityDistribution`[Any, Any, Any])

dist_to_encoder()

Create the categorical sequence encoder.

Return type

`dmx.bstats.categorical.CategoricalDataEncoder`

entropy()

Return Shannon entropy over the explicit sampled support.

Return type

float

estimator()

Create an estimator retaining the current prior.

Return type*dmx.bstats.categorical.CategoricalEstimator***expected_log_density(*x*)**

Evaluate the prior-averaged log mass when conjugate.

Return type

float

Parameters**x** (*Any*)**get_parameters()**

Return the category probability mapping.

Return type

dict[typing.Any, float]

get_prior()

Return the current dictionary Dirichlet or alternate prior.

Return type*dmx.bstats.pdist.ProbabilityDistribution*[typing.Any, typing.Any, typing.Any]**log_density(*x*)**

Evaluate known-category or escape log mass.

Return type

float

Parameters**x** (*Any*)**sampler(*seed=None*)**

Create a repeatable categorical sampler.

Return type*dmx.bstats.categorical.CategoricalSampler***Parameters****seed** (*int* | *None*)**seq_encode(*x*)**

Encode values as inverse indices and unique levels.

For *n* observations and *m* distinct labels, returns integer indices of shape (*n*,) and levels of shape (*m*,).**Return type**

tuple[numpy.ndarray[typing.Any, typing.Any], numpy.ndarray[typing.Any, typing.Any]]

Parameters**x** (*Iterable*[*Any*])**seq_expected_log_density(*x*)**

Evaluate expected log masses for encoded category indices.

Return type

numpy.ndarray[typing.Any, typing.Any]

Parameters**x** (*tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]])**seq_log_density**(*x*)

Evaluate log masses for encoded category indices.

Return type*numpy.ndarray*[*typing.Any*, *typing.Any*]**Parameters****x** (*tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]])**set_parameters**(*value*)

Replace the category probability mapping.

Return type*None***Parameters****value** (*dict*[*Any*, *float*])**set_prior**(*prior*)

Replace the prior and cache expected log probabilities.

Return type*None***Parameters****prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)**class** *dmx.bstats.categorical.CategoricalEncodedData*(*data*)Bases: *EncodedDataSequence*[*tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]]]

Contain inverse indices and unique categorical levels.

Parameters**data** (*CategoricalEncoded*)**class** *dmx.bstats.categorical.CategoricalEstimator*(*default_value=0.0*, *name=None*,
prior=default_prior, *keys=(None,)*)Bases: *ParameterEstimator*[*Any*, *dict*[*Any*, *float*], *tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]],
tuple[*dict*[*Any*, *float*], *float*]]

Estimate categorical probabilities from weighted category counts.

A dictionary Dirichlet prior is updated to posterior concentrations and probabilities are set to its mode when defined, otherwise its mean. With another prior, normalized observed counts determine the probabilities and the prior is retained unchanged.

Parameters

- **default_value** (*float*)
- **name** (*Optional*[*str*])
- **prior** (*Optional*[*Model*])
- **keys** (*tuple*[*Optional*[*str*], ...])

accumulator_factory()

Create a compatible accumulator factory.

Return type*dmx.bstats.categorical.CategoricalEstimatorAccumulatorFactory*

estimate(*args)

Estimate probabilities from a weighted category-count mapping.

Parameters

***args** – Either (count_map, total_weight) alone or legacy nobs followed by that tuple. Both count fields come from the accumulator; nobs and total_weight are ignored here.

Return type

`dmx.bstats.categorical.CategoricalDistribution`

Returns

Categorical likelihood on the observed and prior-defined keys.

Parameters

args (Any)

get_prior()

Return the estimator prior.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[typing.Any, typing.Any, typing.Any]

set_prior(prior)

Replace the estimator prior.

Return type

None

Parameters

prior (`ProbabilityDistribution`[Any, Any, Any] | None)

class `dmx.bstats.categorical.CategoricalEstimatorAccumulator`(name, keys)

Bases: `SequenceEncodableAccumulator`[Any, tuple[dict[Any, float], float], tuple[ndarray[Any, Any], ndarray[Any, Any]]], `DataFrameEncodableAccumulator`[Any, tuple[dict[Any, float], float], tuple[ndarray[Any, Any], ndarray[Any, Any]]]

Accumulate a weighted category-count mapping and total weight.

Parameters

- **name** (str | None)
- **keys** (tuple[Optional[str], ...])

acc_to_encoder()

Create the compatible categorical encoder.

Return type

`dmx.bstats.categorical.CategoricalDataEncoder`

combine(suff_stat)

Merge a category-count mapping and total.

Return type

`dmx.bstats.categorical.CategoricalEstimatorAccumulator`

Parameters

suff_stat (tuple[dict[Any, float], float])

from_value(*x*)

Restore a category-count mapping and total count.

Return type

dmx.bstats.categorical.CategoricalEstimatorAccumulator

Parameters

x (*tuple*[*dict*[*Any*, *float*], *float*])

initialize(*x*, *weight*, *rng*)

Accumulate one observation during initialization.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **rng** (*RandomState*)

seq_initialize(*x*, *weights*, *rng*)

Accumulate encoded observations during initialization.

Return type

None

Parameters

- **x** (*tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Add encoded category observations with weights.

Return type

None

Parameters

- **x** (*tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Add one weighted category observation.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return a copied count mapping and total count.

Return type

`tuple[dict[typing.Any, float], float]`

class `dmx.bstats.categorical.CategoricalEstimatorAccumulatorFactory`(*name*, *keys*)

Bases: `StatisticAccumulatorFactory`[*Any*, `tuple[dict[Any, float], float]`, `tuple[ndarray[Any, Any], ndarray[Any, Any]]`]

Create categorical accumulators with shared metadata.

Parameters

- **name** (*Optional*[*str*])
- **keys** (*tuple*[*Optional*[*str*], ...])

make()

Create an empty categorical accumulator.

Return type

`dmx.bstats.categorical.CategoricalEstimatorAccumulator`

class `dmx.bstats.categorical.CategoricalSampler`(*dist*, *seed*=*None*)

Bases: `DistributionSampler`[*Any*]

Draw independent values from an explicit categorical support.

Parameters

- **dist** (`CategoricalDistribution`)
- **seed** (*Optional*[*int*])

sample(*size*=*None*)

Draw one category or a list of *size* categories.

Return type

`typing.Any`

Parameters

size (*int* | *None*)

Dirac

Fixed point-mass likelihoods.

`DiracDistribution`(*value*) has singleton support at *value*: matching observations have log-density zero and all others have log-density $-\infty$. It has no learned parameter or nontrivial sufficient statistic. Sampling and estimation always reproduce the constructor value, and expected log-density is identical to fixed log-density.

class `dmx.bstats.dirac.DiracAccumulator`

Bases: `StatisticAccumulator`[*Any*, *None*, `ndarray[Any, dtype[object_]]`]

Discard observations because a point mass has no learned statistics.

acc_to_encoder()

Create the compatible point-mass encoder.

Return type

`dmx.bstats.dirac.DiracDataEncoder`

combine(*suff_stat*)

Combine the sole empty statistic.

Return type*dmx.bstats.dirac.DiracAccumulator***Parameters****suff_stat** (*None*)**from_value**(*x*)

Restore the sole empty sufficient statistic.

Return type*dmx.bstats.dirac.DiracAccumulator***Parameters****x** (*None*)**initialize**(*x*, *weight*, *rng*)

Discard one observation during initialization.

Return type*None***Parameters**

- **x** (*Any*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Leave shared statistics unchanged.

Return type*None***Parameters****stats_dict** (*MutableMapping[str, Any]*)**key_replace**(*stats_dict*)

Leave shared statistics unchanged.

Return type*None***Parameters****stats_dict** (*MutableMapping[str, Any]*)**seq_initialize**(*x*, *weights*, *rng*)

Discard encoded observations during initialization.

Return type*None***Parameters**

- **x** (*ndarray[Any, dtype[object_]]*)
- **weights** (*ndarray[Any, Any]*)
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Discard encoded weighted observations.

Return type

None

Parameters

- **x** (*ndarray*[*Any*, *dtype*[*object_*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Discard one weighted observation.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return the sole empty sufficient statistic.

Return type

None

class *dmx.bstats.dirac.DiracAccumulatorFactory*

Bases: *StatisticAccumulatorFactory*[*Any*, *None*, *ndarray*[*Any*, *dtype*[*object_*]]]

Create empty point-mass accumulators.

make()

Create a point-mass accumulator.

Return type

dmx.bstats.dirac.DiracAccumulator

class *dmx.bstats.dirac.DiracDataEncoder*

Bases: *DataSequenceEncoder*[*Any*, *ndarray*[*Any*, *dtype*[*object_*]]]

Encode arbitrary point-mass observations as object arrays.

seq_encode(*x*)

Encode observations in a typed container.

Return type

dmx.bstats.dirac.DiracEncodedData

Parameters

x (*Iterable*[*Any*])

class *dmx.bstats.dirac.DiracDistribution*(*value*)

Bases: *ProbabilityDistribution*[*Any*, *Any*, *ndarray*[*Any*, *dtype*[*object_*]]]

Represent a point mass supported at one fixed value.

Equality is scalar equality for ordinary objects and exact element-wise equality for NumPy arrays. Attached prior objects are metadata only; scoring and estimation always retain the fixed support point.

Parameters

value (*Any*)

density(*x*)

Return one at the support point and zero elsewhere.

Return type

float

Parameters

x (*Any*)

dist_to_encoder()

Create the point-mass sequence encoder.

Return type

dmx.bstats.dirac.DiracDataEncoder

estimator()

Create an estimator that retains the fixed point.

Return type

dmx.bstats.dirac.DiracEstimator

expected_log_density(*x*)

Return fixed log-density because the point is not random.

Return type

float

Parameters

x (*Any*)

get_parameters()

Return the fixed support point.

Return type

typing.Any

get_prior()

Return the neutral prior associated with a fixed point mass.

Return type

dmx.bstats.pdist.ProbabilityDistribution[typing.Any, typing.Any, typing.Any]

log_density(*x*)

Return zero at the support point and $-\infty$ elsewhere.

Return type

float

Parameters

x (*Any*)

sampler(*seed=None*)

Create a deterministic sampler for the fixed point.

Return type*dmx.bstats.dirac.DiracSampler***Parameters****seed** (*int* | *None*)**seq_encode(x)**

Encode observations in an object array along its leading axis.

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.object_]]

Parameters**x** (*Iterable*[*Any*])**seq_expected_log_density(x)**

Return vectorized fixed log-densities.

Return type

numpy.ndarray[typing.Any, typing.Any]

Parameters**x** (*ndarray*[*Any*, *dtype*[*object_*]])**seq_log_density(x)**

Evaluate point-mass log-densities for encoded observations.

Return type

numpy.ndarray[typing.Any, typing.Any]

Parameters**x** (*ndarray*[*Any*, *dtype*[*object_*]])**set_parameters(value)**

Replace the fixed support point.

Return type

None

Parameters**value** (*Any*)**set_prior(prior)**

Associate prior metadata without changing the fixed value.

Return type

None

Parameters**prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*])**class dmx.bstats.dirac.DiracEncodedData(data)**Bases: *EncodedDataSequence*[*ndarray*[*Any*, *dtype*[*object_*]]]

Contain an encoded point-mass sequence.

Parameters**data** (*DiracEncoded*)**class dmx.bstats.dirac.DiracEstimator(value, prior=null_dist, keys=None)**Bases: *ParameterEstimator*[*Any*, *Any*, *ndarray*[*Any*, *dtype*[*object_*]], *None*]

Return a fixed point-mass distribution for every statistic value.

Parameters

- **value** (*Any*)
- **prior** (*Model*)
- **keys** (*Optional[str]*)

accumulator_factory()

Create a compatible accumulator factory.

Return type

dmx.bstats.dirac.DiracAccumulatorFactory

estimate(*args)

Return the fixed point mass, ignoring empty statistics.

Return type

dmx.bstats.dirac.DiracDistribution

Parameters

args (*Any*)

get_prior()

Return prior metadata for the fixed point.

Return type

dmx.bstats.pdist.ProbabilityDistribution[*typing.Any*, *typing.Any*, *typing.Any*]

set_prior(prior)

Replace prior metadata without changing the fixed point.

Return type

None

Parameters

prior (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

class *dmx.bstats.dirac.DiracSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*[*Any*]

Return copies of one fixed point.

Parameters

- **dist** (*DiracDistribution*)
- **seed** (*Optional[int]*)

sample(size=None)

Return one copied point or a list of *size* copied points.

Return type

typing.Any

Parameters

size (*int* | *None*)

Diagonal Gaussian

Bayesian diagonal multivariate Gaussian likelihoods.

`DiagonalGaussianDistribution(mu, covariance)` accepts equal-length vectors of means and strictly positive diagonal variances. Its support is all finite real vectors of that dimension. The default multivariate normal-gamma prior models each coordinate mean and precision independently. Accumulators store (sum, sum_of_squares, count). Expected log-density averages over a conjugate prior and otherwise uses the fixed likelihood parameters.

class `dmx.bstats.dmvn.DiagonalGaussianAccumulator(dim=None)`

Bases: `StatisticAccumulator`[Sequence[float] | ndarray[Any, dtype[float64]], tuple[ndarray[Any, dtype[float64]] | None, ndarray[Any, dtype[float64]] | None, float], tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]]]

Accumulate coordinate sums, squared sums, and total weight.

The sufficient statistic is (sum_x, sum_x_squared, weight). The first two entries have shape (d,) or are None before dimension is known.

Parameters

dim (Optional[int])

acc_to_encoder()

Create the compatible dimensional encoder.

Return type

`dmx.bstats.dmvn.DiagonalGaussianDataEncoder`

combine(suff_stat)

Merge (sum, sum_of_squares, count) statistics.

Return type

`dmx.bstats.dmvn.DiagonalGaussianAccumulator`

Parameters

suff_stat (tuple[ndarray[Any, dtype[float64]] | None, ndarray[Any, dtype[float64]] | None, float)

from_value(x)

Restore sums, squared sums, and weighted count.

Return type

`dmx.bstats.dmvn.DiagonalGaussianAccumulator`

Parameters

x (tuple[ndarray[Any, dtype[float64]] | None, ndarray[Any, dtype[float64]] | None, float)

initialize(x, weight, rng)

Accumulate one vector during initialization.

Return type

None

Parameters

- **x** (Sequence[float] | ndarray[Any, dtype[float64]])
- **weight** (float)
- **rng** (RandomState)

key_merge(*stats_dict*)

Leave shared statistics unchanged; this accumulator has no keys.

Return type

None

Parameters

stats_dict (*MutableMapping*[*str*, *Any*])

key_replace(*stats_dict*)

Leave shared statistics unchanged; this accumulator has no keys.

Return type

None

Parameters

stats_dict (*MutableMapping*[*str*, *Any*])

seq_initialize(*x*, *weights*, *rng*)

Accumulate encoded vectors during initialization.

Return type

None

Parameters

- **x** (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]])
- **weights** (*ndarray*[*Any*, *Any*])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Accumulate encoded vectors with corresponding weights.

Return type

None

Parameters

- **x** (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]])
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Accumulate one weighted vector.

Return type

None

Parameters

- **x** (*Sequence*[*float*] | *ndarray*[*Any*, *dtype*[*float64*]])
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return sums, squared sums, and weighted count.

Return type

`tuple[typing.Optional[numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]], typing.Optional[numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]], float]`

class `dmx.bstats.dmvn.DiagonalGaussianAccumulatorFactory`(*dim*)

Bases: `StatisticAccumulatorFactory`[`Sequence[float]` | `ndarray[Any, dtype[float64]]`, `tuple[ndarray[Any, dtype[float64]]` | `None`, `ndarray[Any, dtype[float64]]` | `None`, `float`], `tuple[ndarray[Any, dtype[float64]]`, `ndarray[Any, dtype[float64]]`]

Create diagonal Gaussian accumulators of a fixed dimension.

Parameters

dim (*Optional[int]*)

make()

Create an empty diagonal Gaussian accumulator.

Return type

`dmx.bstats.dmvn.DiagonalGaussianAccumulator`

class `dmx.bstats.dmvn.DiagonalGaussianDataEncoder`(*dim*)

Bases: `DataSequenceEncoder`[`Sequence[float]` | `ndarray[Any, dtype[float64]]`, `tuple[ndarray[Any, dtype[float64]]`, `ndarray[Any, dtype[float64]]`]

Encode vectors as values and coordinate-wise squares.

Parameters

dim (*int*)

seq_encode(x)

Encode observations in a typed container.

Return type

`dmx.bstats.dmvn.DiagonalGaussianEncodedData`

Parameters

x (*Iterable[Sequence[float] | ndarray[Any, dtype[float64]]]*)

class `dmx.bstats.dmvn.DiagonalGaussianDistribution`(*mu*, *covariance*, *name=None*, *prior=None*)

Bases: `ProbabilityDistribution`[`Sequence[float]` | `ndarray[Any, dtype[float64]]`, `tuple[ndarray[Any, dtype[float64]]`, `ndarray[Any, dtype[float64]]`, `tuple[ndarray[Any, dtype[float64]]`, `ndarray[Any, dtype[float64]]`]

Represent a multivariate Gaussian likelihood with diagonal covariance.

`mu` and `covariance` are vectors of shape `(d,)`; covariance entries are variances, not standard deviations. Observations are finite vectors of the same shape. A multivariate normal-gamma prior enables posterior-expected scoring, while another prior uses plug-in parameters.

Parameters

- **mu** (*Sequence[float] | Array*)
- **covariance** (*Sequence[float] | Array*)
- **name** (*str | None*)
- **prior** (*ProbabilityDistribution[Any, Any, Any]*)

dist_to_encoder()

Create a sequence encoder for this dimensionality.

Return type

dmx.bstats.dmvn.DiagonalGaussianDataEncoder

estimator()

Create an estimator retaining dimension, name, and prior.

Return type

dmx.bstats.dmvn.DiagonalGaussianEstimator

expected_log_density(x)

Evaluate prior-averaged log-density when conjugate.

Return type

float

Parameters

x (*Sequence[float] | ndarray[Any, dtype[float64]]*)

get_parameters()

Return mean and diagonal variance arrays.

Return type

tuple[numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]], numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]]

get_prior()

Return the current multivariate normal-gamma or alternate prior.

Return type

dmx.bstats.pdist.ProbabilityDistribution[typing.Any, typing.Any, typing.Any]

log_density(x)

Evaluate log-density, returning -inf outside vector support.

Return type

float

Parameters

x (*Sequence[float] | ndarray[Any, dtype[float64]]*)

sampler(seed=None)

Create a repeatable diagonal Gaussian sampler.

Return type

dmx.bstats.dmvn.DiagonalGaussianSampler

Parameters

seed (*int | None*)

seq_encode(x)

Encode observations as values and coordinate-wise squares.

Parameters

x – Iterable of n observations, each with shape (d,).

Return type

tuple[numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]], numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]]

Returns

Value and squared-value matrices, each shaped (n, d).

Parameters

x (*Iterable*[*Sequence*[*float*] | *ndarray*[*Any*, *dtype*[*float64*]]])

seq_expected_log_density(x)

Evaluate prior-averaged log-densities from encoded observations.

Parameters

x – Value and squared-value matrices, each shaped (n, d).

Return type

numpy.ndarray[*typing.Any*, *numpy.dtype*[*numpy.float64*]]

Returns

Expected log-density array of shape (n,).

Parameters

x (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]])

seq_log_density(x)

Evaluate fixed-parameter log-densities from encoded observations.

Parameters

x – Value and squared-value matrices, each shaped (n, d).

Return type

numpy.ndarray[*typing.Any*, *numpy.dtype*[*numpy.float64*]]

Returns

Log-density array of shape (n,).

Parameters

x (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]])

set_parameters(value)

Replace mean and diagonal variance and refresh cached terms.

Return type

None

Parameters

value (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]])

set_prior(prior)

Replace the prior and cache expected natural parameters.

Return type

None

Parameters

prior (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

class dmx.bstats.dmvn.DiagonalGaussianEncodedData(data)

Bases: *EncodedDataSequence*[*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]])

Contain an encoded diagonal Gaussian sequence.

Parameters

data (*DiagonalGaussianEncoded*)

class `dmx.bstats.dmvn.DiagonalGaussianEstimator`(*dim=None, name=None, prior=None*)

Bases: [`ParameterEstimator`](#)[`Sequence[float]` | `ndarray[Any, dtype[float64]]`, `tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]]`, `tuple[ndarray[Any, dtype[float64]]`, `tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]]`, `tuple[ndarray[Any, dtype[float64]] | None, ndarray[Any, dtype[float64]] | None, float]`]

Estimate diagonal Gaussian parameters from weighted coordinate sums.

A multivariate normal-gamma prior is updated coordinate-wise and attached as the posterior of the returned likelihood. Without that conjugate prior, the returned mean and variance are weighted maximum-likelihood estimates.

Parameters

- **dim** (*Optional[int]*)
- **name** (*Optional[str]*)
- **prior** (*Optional[Model]*)

accumulator_factory()

Create a compatible accumulator factory.

Return type

[`dmx.bstats.dmvn.DiagonalGaussianAccumulatorFactory`](#)

estimate(*args)

Estimate from coordinate sums, squared sums, and total weight.

Parameters

***args** – Either (`sum_x`, `sum_x_squared`, `weight`) alone or legacy `nobs` followed by that tuple. `nobs` is ignored; statistic vectors have shape `(d,)`.

Return type

[`dmx.bstats.dmvn.DiagonalGaussianDistribution`](#)

Returns

Fitted diagonal Gaussian likelihood with an updated conjugate posterior when applicable.

Raises

ValueError – If the statistics contain no coordinate arrays, or if an unconjugated estimate has nonpositive total weight.

Parameters

args (*Any*)

get_prior()

Return the estimator prior.

Return type

`typing.Any`

set_prior(prior)

Replace the estimator prior and update its conjugacy flag.

Return type

`None`

Parameters

prior ([`ProbabilityDistribution`](#)[*Any, Any, Any*] | *None*)

class `dmx.bstats.dmvn.DiagonalGaussianSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`[`Sequence`[`float`] | `ndarray`[`Any`, `dtype`[`float64`]]]

Draw independent vectors from a diagonal Gaussian likelihood.

Parameters

- **dist** (*Any*)
- **seed** (*int* | *None*)

sample(*size=None*)

Draw one vector or a list of *size* vectors.

Return type

`typing.Any`

Parameters

size (*int* | *None*)

Exponential

Bayesian exponential likelihoods for nonnegative real observations.

`ExponentialDistribution(lam)` uses the rate parameterization, so its mean is $1 / \text{lam}$ and its support is $x \geq 0$. The default gamma prior has shape `1.0001` and scale `1e6`. Accumulators store weighted (`count`, `sum`) statistics. Under a gamma prior, expected log-density integrates $\log(\text{lam})$ and lam ; with another prior it falls back to fixed-parameter scoring.

class `dmx.bstats.exponential.ExponentialAccumulator`(*keys=(None,)*)

Bases: `StatisticAccumulator`[`float`, `tuple`[`float`, `float`], `ndarray`[`Any`, `dtype`[`float64`]]]

Accumulate weighted observation count and sum.

Parameters

keys (*tuple*[*Optional*[*str*], ...])

acc_to_encoder()

Create the compatible exponential encoder.

Return type

`dmx.bstats.exponential.ExponentialDataEncoder`

combine(*suff_stat*)

Merge (`count`, `sum`) sufficient statistics.

Return type

`dmx.bstats.exponential.ExponentialAccumulator`

Parameters

suff_stat (*tuple*[*float*, *float*])

from_value(*x*)

Restore (`count`, `sum`) sufficient statistics.

Return type

`dmx.bstats.exponential.ExponentialAccumulator`

Parameters

x (*tuple*[*float*, *float*])

initialize(*x*, *weight*, *rng*)

Accumulate one observation during initialization.

Return type

None

Parameters

- **x** (*float*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge statistics through the configured shared key.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**key_replace**(*stats_dict*)

Replace statistics through the configured shared key.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *rng*)

Accumulate a sequence during initialization.

Return type

None

Parameters

- **x** (*ndarray*[*Any*, *dtype*[*float64*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Accumulate encoded observations and corresponding weights.

Return type

None

Parameters

- **x** (*ndarray*[*Any*, *dtype*[*float64*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Accumulate one weighted observation on the support.

Return type

None

Parameters

- **x** (*float*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return (count, sum) sufficient statistics.

Return type

tuple[*float*, *float*]

class *dmx.bstats.exponential.ExponentialAccumulatorFactory*(*keys*=(*None*,))

Bases: *StatisticAccumulatorFactory*[*float*, *tuple*[*float*, *float*], *ndarray*[*Any*, *dtype*[*float64*]]]

Create exponential accumulators with a shared key.

Parameters

keys (*tuple*[*Optional*[*str*], ...])

make()

Create an empty exponential accumulator.

Return type

dmx.bstats.exponential.ExponentialAccumulator

class *dmx.bstats.exponential.ExponentialDataEncoder*

Bases: *DataSequenceEncoder*[*float*, *ndarray*[*Any*, *dtype*[*float64*]]]

Encode exponential observations as floating-point arrays.

seq_encode(x)

Encode observations in a typed container.

Return type

dmx.bstats.exponential.ExponentialEncodedData

Parameters

x (*Iterable*[*float*])

class *dmx.bstats.exponential.ExponentialDistribution*(*lam*, *name*=*None*, *prior*=*default_prior*)

Bases: *ProbabilityDistribution*[*float*, *float*, *ndarray*[*Any*, *dtype*[*float64*]]]

Represent an exponential likelihood parameterized by a positive rate.

The support is finite values $x \geq 0$ and the mean is $1 / \text{lam}$. A gamma shape-scale prior enables posterior-expected scoring; other priors are retained but scoring uses the fixed rate.

Parameters

- **lam** (*float*)
- **name** (*str* | *None*)
- **prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

dist_to_encoder()

Create the exponential sequence encoder.

Return type

dmx.bstats.exponential.ExponentialDataEncoder

estimator()

Create an estimator retaining the current prior and name.

Return type

dmx.bstats.exponential.ExponentialEstimator

expected_log_density(x)

Evaluate prior-averaged log-density when the prior is gamma.

Return type

float

Parameters

x (*float*)

get_parameters()

Return the exponential rate.

Return type

float

get_prior()

Return the current gamma or alternate prior.

Return type

dmx.bstats.pdist.ProbabilityDistribution[typing.Any, typing.Any, typing.Any]

log_density(x)

Evaluate log-density, returning $-\infty$ outside $x \geq 0$.

Return type

float

Parameters

x (*float*)

sampler(seed=None)

Create a repeatable exponential sampler.

Return type

dmx.bstats.exponential.ExponentialSampler

Parameters

seed (*int* | *None*)

seq_encode(x)

Encode n observations as a float array of shape (n,).

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Parameters

x (*Iterable[float]*)

seq_expected_log_density(x)

Evaluate prior-averaged log-densities for encoded observations.

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Parameters**x** (*ndarray*[*Any*, *dtype*[*float64*]])**seq_log_density**(*x*)

Evaluate fixed-parameter log-densities for encoded observations.

Return type*numpy.ndarray*[*typing.Any*, *numpy.dtype*[*numpy.float64*]]**Parameters****x** (*ndarray*[*Any*, *dtype*[*float64*]])**set_parameters**(*value*)

Replace the positive finite rate and refresh its logarithm.

Return type*None***Parameters****value** (*float*)**set_prior**(*prior*)

Replace the prior and cache gamma shape-rate parameters.

Return type*None***Parameters****prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*])**value**()

Return the legacy one-element parameter list.

Return type*list*[*float*]**class** *dmx.bstats.exponential.ExponentialEncodedData*(*data*)Bases: *EncodedDataSequence*[*ndarray*[*Any*, *dtype*[*float64*]]]

Contain an encoded exponential sequence.

Parameters**data** (*Array*)**class** *dmx.bstats.exponential.ExponentialEstimator*(*prior=default_prior*, *name=None*, *keys=(None,)*)Bases: *ParameterEstimator*[*float*, *float*, *ndarray*[*Any*, *dtype*[*float64*]], *tuple*[*float*, *float*]]

Estimate an exponential rate from weighted count and sum statistics.

A gamma prior, or legacy [*shape*, *rate*] pair, is updated conjugately and its posterior mode supplies the returned rate. Without recognized hyperparameters, the weighted maximum-likelihood rate is used.

Parameters

- **prior** (*Model* | *list*[*float*])
- **name** (*Optional*[*str*])
- **keys** (*tuple*[*Optional*[*str*], ...])

accumulator_factory()

Create a compatible accumulator factory.

Return type*dmx.bstats.exponential.ExponentialAccumulatorFactory***estimate(*args)**

Estimate from weighted (count, sum) statistics.

Parameters

***args** – Either the statistic tuple alone or legacy nobs followed by the tuple. nobs is ignored.

Return type*dmx.bstats.exponential.ExponentialDistribution***Returns**

Fitted exponential likelihood carrying a gamma posterior when the configured hyperparameters are conjugate.

Parameters

args (*Any*)

get_prior()

Return the estimator prior or legacy hyperparameter list.

Return type*typing.Any***set_prior(prior)**

Replace the prior and cache shape-rate hyperparameters.

Return type*None***Parameters**

prior (*Any*)

class *dmx.bstats.exponential.ExponentialSampler*(*dist, seed=None*)

Bases: *DistributionSampler*[float]

Draw independent exponential observations.

Parameters

- **dist** (*ExponentialDistribution*)
- **seed** (*Optional[int]*)

sample(size=None)

Draw one float or an array of size observations.

Return type*typing.Any***Parameters**

size (*int | None*)

Gamma

Define a gamma prior and its legacy maximum-likelihood estimator.

GammaDistribution(*k, theta*) uses shape *k* and scale *theta*, with support $x > 0$. Bayesian likelihood modules use these values as the current prior or posterior hyperparameters. The nested *prior* attribute is estimator metadata and does not alter density, entropy, or cross-entropy.

class `dmx.bstats.gamma.GammaAccumulator`(*keys=None*)

Bases: `StatisticAccumulator`[float, tuple[float, float, float], tuple[ndarray[Any, Any], ndarray[Any, Any]]]

Accumulate (*weight*, *sum_x*, *sum_log_x*) for gamma estimation.

Parameters

keys (*Optional*[str])

combine(*suff_stat*)

Merge gamma sufficient statistics.

Return type

`dmx.bstats.gamma.GammaAccumulator`

Parameters

suff_stat (tuple[float, float, float])

from_value(*x*)

Restore (*count*, *sum*, *sum_of_logs*).

Return type

`dmx.bstats.gamma.GammaAccumulator`

Parameters

x (tuple[float, float, float])

initialize(*x*, *weight*, *rng*)

Accumulate one observation during initialization.

Return type

None

Parameters

- **x** (float)
- **weight** (float)
- **rng** (RandomState)

key_merge(*stats_dict*)

Merge these statistics through the configured shared key.

Return type

None

Parameters

stats_dict (MutableMapping[str, Any])

key_replace(*stats_dict*)

Replace these statistics from the configured shared key.

Return type

None

Parameters

stats_dict (MutableMapping[str, Any])

seq_update(*x*, *weights*, *estimate*)

Accumulate an encoded sequence of weighted observations.

Return type

None

Parameters

- **x** (*tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Accumulate one weighted positive observation.

Return type

None

Parameters

- **x** (*float*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return (count, sum, sum_of_logs).

Return type*tuple*[*float*, *float*, *float*]**class** dmx.bstats.gamma.**GammaAccumulatorFactory**(*keys=None*)Bases: *StatisticAccumulatorFactory*[*float*, *tuple*[*float*, *float*, *float*], *tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]]]

Create gamma accumulators sharing one optional key.

Parameters**keys** (*Optional*[*str*])**make**()

Create an empty gamma accumulator.

Return type*dmx.bstats.gamma.GammaAccumulator***class** dmx.bstats.gamma.**GammaDistribution**(*k*, *theta*, *name=None*, *prior=null_dist*)Bases: *ProbabilityDistribution*[*float*, *tuple*[*float*, *float*], *tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]]]

Represent a gamma prior or likelihood in shape-scale form.

k is the positive shape and *theta* is the positive scale, giving mean *k* * *theta* on support *x* > 0. The optional nested prior is estimator metadata and does not affect scoring.**Parameters**

- **k** (*float*)
- **theta** (*float*)
- **name** (*str* | *None*)
- **prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

cross_entropy(*dist*)

Return $-E_{\text{self}}[\log(\text{dist})]$.

Gamma-to-gamma cross-entropy is analytic. Other distributions are integrated numerically over positive values.

Parameters

dist – Distribution whose log-density is averaged.

Return type

float

Returns

Cross-entropy from this distribution to *dist*.

Parameters

dist ([ProbabilityDistribution](#)[*Any*, *Any*, *Any*])

density(*x*)

Evaluate the density, returning zero outside $x > 0$.

Return type

float

Parameters

x (*float*)

entropy()

Return the differential entropy for shape-scale parameters.

Return type

float

estimator()

Create the legacy gamma maximum-likelihood estimator.

Return type

[dmx.bstats.gamma.GammaEstimator](#)

expected_log_density(*x*)

Return the plug-in log-density because parameters are fixed.

Return type

float

Parameters

x (*float*)

get_parameters()

Return the (shape, scale) parameters.

Return type

tuple[float, float]

log_density(*x*)

Evaluate the log-density, returning $-\text{inf}$ outside the support.

Return type

float

Parameters

x (*float*)

sampler(*seed=None*)

Create a gamma sampler using an optional deterministic seed.

Return type

dmx.bstats.gamma.GammaSampler

Parameters

seed (*int* | *None*)

seq_encode(*x*)

Encode values as arrays of values and logarithms.

Parameters

x – Iterable of *n* scalar observations.

Return type

tuple[*numpy.ndarray*[*typing.Any*, *typing.Any*], *numpy.ndarray*[*typing.Any*, *typing.Any*]]

Returns

Value and log-value arrays, each shaped (*n*,).

Parameters

x (*Iterable*[*float*])

seq_expected_log_density(*x*)

Return vectorized plug-in log-densities for encoded observations.

Return type

numpy.ndarray[*typing.Any*, *typing.Any*]

Parameters

x (*tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]])

seq_log_density(*x*)

Evaluate log-densities from encoded values and logarithms.

Parameters

x – Value and log-value arrays, each shaped (*n*,).

Return type

numpy.ndarray[*typing.Any*, *typing.Any*]

Returns

Log-density array of shape (*n*,).

Parameters

x (*tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]])

set_parameters(*value*)

Replace shape and scale and refresh the cached normalizer.

Parameters

value – Positive (*shape*, *scale*) parameters.

Raises

ValueError – If *shape* or *scale* is not finite and positive.

Return type

None

Parameters

value (*tuple*[*float*, *float*])

```
class dmx.bstats.gamma.GammaEstimator(pseudo_count=(0.0, 0.0), suff_stat=(1.0, 0.0), threshold=1.0e-8,
                                       keys=None, name=None, prior=null_dist)
```

Bases: [ParameterEstimator](#)[float, tuple[float, float], tuple[ndarray[Any, Any], ndarray[Any, Any]], tuple[float, float, float]]

Estimate gamma shape and scale from sufficient statistics.

The two pseudo-count components independently regularize the value mean and log-value mean toward `suff_stat`. `prior` is metadata only; this class performs regularized maximum-likelihood estimation rather than a conjugate posterior update.

Parameters

- **pseudo_count** (tuple[float, float])
- **suff_stat** (tuple[float, float])
- **threshold** (float)
- **keys** (Optional[str])
- **name** (Optional[str])
- **prior** (Model)

accumulator_factory()

Create a factory for compatible sufficient-statistic accumulators.

Return type

[dmx.bstats.gamma.GammaAccumulatorFactory](#)

estimate(*args)

Estimate a gamma distribution from either legacy call form.

Parameters

***args** – Either (weight, sum_x, sum_log_x) or ignored nobs followed by that tuple.

Return type

[dmx.bstats.gamma.GammaDistribution](#)

Returns

Estimated gamma distribution.

Raises

TypeError – If neither supported call form is supplied.

Parameters

args (Any)

static estimate_shape(avg_sum, avg_sum_of_logs, threshold)

Solve the gamma shape equation by Newton iteration.

Parameters

- **avg_sum** – Mean observation value.
- **avg_sum_of_logs** – Mean log observation value.
- **threshold** – Absolute parameter-change convergence tolerance.

Return type

float

Returns

Estimated positive gamma shape.

Parameters

- **avg_sum** (*float*)
- **avg_sum_of_logs** (*float*)
- **threshold** (*float*)

get_prior()

Return estimator metadata describing its prior.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[`typing.Any`, `typing.Any`, `typing.Any`]

set_prior(prior)

Replace estimator prior metadata.

Return type

`None`

Parameters

prior (`ProbabilityDistribution`[`Any`, `Any`, `Any`])

class `dmx.bstats.gamma.GammaSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`[`float`]

Draw independent values from a `GammaDistribution`.

Parameters

- **dist** (`GammaDistribution`)
- **seed** (`Optional[int]`)

sample(size=None)

Draw one float or an array of size gamma values.

Return type

`typing.Any`

Parameters

size (`int` | `None`)

Gaussian

Bayesian univariate Gaussian likelihoods.

`GaussianDistribution(mu, sigma2)` uses a mean and strictly positive variance on the real line. Its default normal-gamma prior models the mean and precision $1 / \text{sigma2}$. Accumulators retain the legacy five statistics used for separately shared location and variance estimates. Expected log-density is averaged under a normal-gamma prior and otherwise uses plug-in parameters.

class `dmx.bstats.gaussian.GaussianAccumulator`(*name=None*, *keys=(None, None)*)

Bases: `StatisticAccumulator`[`float`, `tuple`[`float`, `float`, `float`, `float`, `float`, `float`], `ndarray`[`Any`, `dtype`[`float64`]]

Accumulate legacy location and variance sufficient statistics.

The tuple is (`sum_x`, `sum_x_squared`, `variance_sum_x`, `mean_weight`, `variance_weight`). Separate keys allow mean and variance statistics to be shared independently between model components.

Parameters

- **name** (*Optional[str]*)
- **keys** (*GaussianKeys*)

acc_to_encoder()

Create the compatible Gaussian encoder.

Return type

dmx.bstats.gaussian.GaussianDataEncoder

combine(suff_stat)

Merge five legacy sufficient statistics.

Return type

dmx.bstats.gaussian.GaussianAccumulator

Parameters

suff_stat (*tuple[float, float, float, float, float]*)

df_initialize(df, weights, rng)

Accumulate a named DataFrame column during initialization.

Return type

None

Parameters

- **df** (*DataFrame*)
- **weights** (*Series*)
- **rng** (*RandomState*)

df_update(df, weights, estimate)

Accumulate a named DataFrame column and corresponding weights.

Return type

None

Parameters

- **df** (*DataFrame*)
- **weights** (*Series*)
- **estimate** (*ProbabilityDistribution[Any, Any, Any]* | None)

from_value(x)

Restore the five location and variance statistics.

Return type

dmx.bstats.gaussian.GaussianAccumulator

Parameters

x (*tuple[float, float, float, float, float]*)

initialize(x, weight, rng)

Accumulate one observation during initialization.

Return type

None

Parameters

- **x** (*float*)

- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge location and variance statistics through shared keys.

Return type

None

Parameters

stats_dict (*MutableMapping[str, Any]*)

key_replace(*stats_dict*)

Replace location and variance statistics through shared keys.

Return type

None

Parameters

stats_dict (*MutableMapping[str, Any]*)

seq_initialize(*x, weights, rng*)

Accumulate encoded observations during initialization.

Return type

None

Parameters

- **x** (*ndarray[Any, dtype[float64]]*)
- **weights** (*ndarray[Any, Any]*)
- **rng** (*RandomState*)

seq_update(*x, weights, estimate*)

Accumulate encoded observations with corresponding weights.

Return type

None

Parameters

- **x** (*ndarray[Any, dtype[float64]]*)
- **weights** (*ndarray[Any, Any]*)
- **estimate** (*ProbabilityDistribution[Any, Any, Any] | None*)

update(*x, weight, estimate*)

Accumulate one weighted value for location and variance.

Return type

None

Parameters

- **x** (*float*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution[Any, Any, Any] | None*)

value()

Return the five location and variance statistics.

Return type

`tuple[float, float, float, float, float]`

class `dmx.bstats.gaussian.GaussianDataEncoder`

Bases: `DataSequenceEncoder`[float, ndarray[Any, dtype[float64]]]

Encode Gaussian observations as floating-point arrays.

seq_encode(x)

Encode observations in a typed container.

Return type

`dmx.bstats.gaussian.GaussianEncodedData`

Parameters

x (`Iterable[float]`)

class `dmx.bstats.gaussian.GaussianDistribution(mu, sigma2, name=None, prior=default_prior)`

Bases: `ProbabilityDistribution`[float, tuple[float, float], ndarray[Any, dtype[float64]]]

Represent a univariate Gaussian likelihood by mean and variance.

`mu` is finite and `sigma2` is a strictly positive finite variance. A normal-gamma prior over mean and precision enables posterior-expected scoring; another prior uses the fixed likelihood parameters.

Parameters

- **mu** (`float`)
- **sigma2** (`float`)
- **name** (`str` | `None`)
- **prior** (`ProbabilityDistribution`[Any, Any, Any])

dist_to_encoder()

Create the Gaussian sequence encoder.

Return type

`dmx.bstats.gaussian.GaussianDataEncoder`

estimator()

Create an estimator retaining the current name and prior.

Return type

`dmx.bstats.gaussian.GaussianEstimator`

expected_log_density(x)

Evaluate prior-averaged log-density when conjugate.

Return type

`float`

Parameters

x (`float`)

get_parameters()

Return (mean, variance).

Return type

`tuple[float, float]`

get_prior()

Return the current normal-gamma or alternate prior.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[typing.Any, typing.Any, typing.Any]

log_density(x)

Evaluate the Gaussian log-density on the real line.

Return type

float

Parameters

x (float)

sampler(seed=None)

Create a repeatable Gaussian sampler.

Return type

`dmx.bstats.gaussian.GaussianSampler`

Parameters

seed (int | None)

seq_encode(x)

Encode n observations as a float array of shape (n,).

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Parameters

x (Iterable[float])

seq_expected_log_density(x)

Evaluate prior-averaged log-densities for encoded observations.

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Parameters

x (ndarray[Any, dtype[float64]])

seq_log_density(x)

Evaluate fixed-parameter log-densities for encoded observations.

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Parameters

x (ndarray[Any, dtype[float64]])

set_parameters(value)

Replace the finite mean and positive finite variance.

Return type

None

Parameters

value (tuple[float, float])

set_prior(*prior*)

Replace the prior and cache expected natural parameters.

Return type

None

Parameters

prior ([ProbabilityDistribution](#)[Any, Any, Any])

class dmx.bstats.gaussian.**GaussianEncodedData**(*data*)

Bases: [EncodedDataSequence](#)[ndarray[Any, dtype[float64]]]

Contain an encoded Gaussian sequence.

Parameters

data (Array)

class dmx.bstats.gaussian.**GaussianEstimator**(*name=None, prior=default_prior, keys=(None, None)*)

Bases: [ParameterEstimator](#)[float, tuple[float, float], ndarray[Any, dtype[float64]], tuple[float, float, float, float, float]]

Estimate Gaussian parameters from weighted first and second moments.

A normal-gamma prior is updated conjugately and attached as the posterior of the returned likelihood. Otherwise, separate weighted maximum-likelihood estimates are used for the mean and variance.

Parameters

- **name** (*Optional*[str])
- **prior** (*Model*)
- **keys** (*GaussianKeys*)

accumulator_factory()

Create a compatible accumulator factory.

Return type

[dmx.bstats.gaussian.GaussianEstimatorAccumulatorFactory](#)

estimate(*args)

Estimate from the five legacy Gaussian sufficient statistics.

Parameters

***args** – Either the statistic tuple alone or legacy nobs followed by it. nobs is ignored. The tuple contains `sum_x`, `sum_x_squared`, a separately shareable copy of `sum_x`, mean weight, and variance weight.

Return type

[dmx.bstats.gaussian.GaussianDistribution](#)

Returns

Fitted Gaussian likelihood carrying an updated normal-gamma posterior when the prior is conjugate.

Parameters

args (*Any*)

get_prior()

Return the estimator prior.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[typing.Any, typing.Any, typing.Any]

set_prior(*prior*)

Replace the estimator prior and update its conjugacy flag.

Return type

None

Parameters

prior (`ProbabilityDistribution`[Any, Any, Any])

class `dmx.bstats.gaussian.GaussianEstimatorAccumulatorFactory`(*name*, *keys*)

Bases: `StatisticAccumulatorFactory`[float, tuple[float, float, float, float, float], ndarray[Any, dtype[float64]]]

Create Gaussian accumulators with shared metadata.

Parameters

- **name** (`Optional[str]`)
- **keys** (`GaussianKeys`)

make()

Create an empty Gaussian accumulator.

Return type

`dmx.bstats.gaussian.GaussianAccumulator`

class `dmx.bstats.gaussian.GaussianSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`[float]

Draw independent Gaussian observations.

Parameters

- **dist** (`GaussianDistribution`)
- **seed** (`Optional[int]`)

sample(*size=None*)

Draw one float or an array of size observations.

Return type

typing.Any

Parameters

size (`int | None`)

Geometric

Bayesian geometric likelihoods for positive integer trial counts.

`GeometricDistribution(p)` models the number of trials through the first success, so its observation support is integers $x \geq 1$. The default prior is `Beta(1, 1)`. Accumulators store weighted (`count`, `sum`) statistics; the beta posterior adds `count` successes and `sum - count` failures. Expected log-density integrates both log-probability terms under a beta prior and otherwise falls back to fixed-parameter scoring.

class `dmx.bstats.geometric.GeometricAccumulator`(*keys*, *name*)

Bases: `StatisticAccumulator`[`int`, `tuple`[`float`, `float`], `ndarray`[`Any`, `dtype`[`float64`]]]

Accumulate weighted observation count and trial-count sum.

Parameters

- **keys** (*Optional*[`str`])
- **name** (*Optional*[`str`])

acc_to_encoder()

Create the compatible geometric encoder.

Return type

`dmx.bstats.geometric.GeometricDataEncoder`

combine(*suff_stat*)

Merge (`observation_count`, `trial_count_sum`) statistics.

Return type

`dmx.bstats.geometric.GeometricAccumulator`

Parameters

suff_stat (`tuple`[`float`, `float`])

from_value(*x*)

Restore (`observation_count`, `trial_count_sum`).

Return type

`dmx.bstats.geometric.GeometricAccumulator`

Parameters

x (`tuple`[`float`, `float`])

initialize(*x*, *weight*, *rng*)

Accumulate one observation during initialization.

Return type

`None`

Parameters

- **x** (`int`)
- **weight** (`float`)
- **rng** (`RandomState`)

key_merge(*stats_dict*)

Merge statistics through the configured sharing key.

Return type

`None`

Parameters

stats_dict (`MutableMapping`[`str`, `Any`])

key_replace(*stats_dict*)

Replace statistics through the configured sharing key.

Return type

`None`

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *rng*)

Accumulate encoded observations during initialization.

Return type

None

Parameters

- **x** (*ndarray*[*Any*, *dtype*[*float64*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Add encoded trial counts with corresponding weights.

Return type

None

Parameters

- **x** (*ndarray*[*Any*, *dtype*[*float64*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Add one weighted positive integer trial count.

Return type

None

Parameters

- **x** (*int*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return (observation_count, trial_count_sum).

Return typetuple[*float*, *float*]**class** dmx.bstats.geometric.**GeometricAccumulatorFactory**(*keys*, *name*)Bases: *StatisticAccumulatorFactory*[*int*, tuple[*float*, *float*], *ndarray*[*Any*, *dtype*[*float64*]]]

Create geometric accumulators with shared metadata.

Parameters

- **keys** (*Optional*[*str*])
- **name** (*Optional*[*str*])

make()

Create an empty geometric accumulator.

Return type*dmx.bstats.geometric.GeometricAccumulator***class** `dmx.bstats.geometric.GeometricDataEncoder`Bases: *DataSequenceEncoder*[int, ndarray[Any, dtype[float64]]]

Encode positive integer geometric observations.

seq_encode(*x*)

Encode observations in a typed container.

Return type*dmx.bstats.geometric.GeometricEncodedData***Parameters****x** (*Iterable*[int])**class** `dmx.bstats.geometric.GeometricDistribution`(*p*, *name=None*, *prior=default_prior*, *keys=None*)Bases: *ProbabilityDistribution*[int, float, ndarray[Any, dtype[float64]]]

Represent a geometric likelihood on positive integer trial counts.

p is the probability of success on each trial, and an observation is the number of trials through the first success. A beta prior enables posterior-expected scoring; another prior uses the fixed probability.

Parameters

- **p** (*float*)
- **name** (*str* | *None*)
- **prior** (*ProbabilityDistribution*[Any, Any, Any])
- **keys** (*Optional*[*str*])

cross_entropy(*dist*)Return $-E_{\text{self}}[\log(\text{dist})]$ for another geometric likelihood.**Return type**

float

Parameters**dist** (*ProbabilityDistribution*[Any, Any, Any])**dist_to_encoder**()

Create the geometric sequence encoder.

Return type*dmx.bstats.geometric.GeometricDataEncoder***entropy**()

Return the geometric Shannon entropy.

Return type

float

estimator(*pseudo_count=None*)

Create an estimator retaining metadata and the current prior.

Return type*dmx.bstats.geometric.GeometricEstimator***Parameters****pseudo_count** (*float* | *None*)

expected_log_density(*x*)

Evaluate prior-averaged log mass when the prior is beta.

Return type

float

Parameters

x (*int*)

get_parameters()

Return the success probability.

Return type

float

get_prior()

Return the current beta or alternate prior.

Return type

dmx.bstats.pdist.ProbabilityDistribution[typing.Any, typing.Any, typing.Any]

log_density(*x*)

Evaluate log mass, returning -inf outside positive integers.

Return type

float

Parameters

x (*int*)

moment(*order*)

Return a raw moment of the positive-integer trial count.

Return type

float

Parameters

order (*int*)

sampler(*seed=None*)

Create a repeatable geometric sampler.

Return type

dmx.bstats.geometric.GeometricSampler

Parameters

seed (*int* | *None*)

seq_encode(*x*)

Encode *n* trial counts as a float array of shape (*n*,).

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Parameters

x (*Iterable[int]*)

seq_expected_log_density(*x*)

Evaluate expected log masses for encoded trial counts.

Return type

`numpy.ndarray[typing.Any, typing.Any]`

Parameters

x (`ndarray[Any, dtype[float64]]`)

seq_log_density(x)

Evaluate log masses for encoded trial counts.

Return type

`numpy.ndarray[typing.Any, typing.Any]`

Parameters

x (`ndarray[Any, dtype[float64]]`)

set_parameters(value)

Replace the success probability and cached logarithms.

Return type

`None`

Parameters

value (`float`)

set_prior(prior)

Replace the prior and cache beta expected-log parameters.

Return type

`None`

Parameters

prior (`ProbabilityDistribution[Any, Any, Any]`)

class dmx.bstats.geometric.GeometricEncodedData(data)

Bases: `EncodedDataSequence[ndarray[Any, dtype[float64]]]`

Contain an encoded geometric observation sequence.

Parameters

data (`GeometricEncoded`)

class dmx.bstats.geometric.GeometricEstimator(name=None, keys=None, prior=default_prior)

Bases: `ParameterEstimator[int, float, ndarray[Any, dtype[float64]], tuple[float, float]]`

Estimate a geometric success probability from weighted trial counts.

A beta prior is updated with one success and $x - 1$ failures per observation, and its interior posterior mode is used when defined. Another non-null prior produces a numerical MAP estimate; a null prior produces the weighted maximum-likelihood value.

Parameters

- **name** (`Optional[str]`)
- **keys** (`Optional[str]`)
- **prior** (`Model`)

accumulator_factory()

Create a compatible accumulator factory.

Return type

`dmx.bstats.geometric.GeometricAccumulatorFactory`

estimate(*args)

Estimate from (observation_count, trial_count_sum) statistics.

Parameters***args** – Either the statistic tuple alone or legacy nobs followed by the tuple. nobs is ignored.**Return type***dmx.bstats.geometric.GeometricDistribution***Returns**

Geometric likelihood carrying the updated posterior when the prior is conjugate.

Parameters**args** (*Any*)**get_prior**()

Return the estimator prior.

Return type*dmx.bstats.pdist.ProbabilityDistribution*[typing.Any, typing.Any, typing.Any]**set_prior**(prior)

Replace the estimator prior and update prior flags.

Return type

None

Parameters**prior** (*ProbabilityDistribution*[Any, Any, Any])**class** dmx.bstats.geometric.GeometricSampler(*dist, seed=None*)Bases: *DistributionSampler*[int]

Draw independent positive integer geometric observations.

Parameters

- **dist** (*GeometricDistribution*)
- **seed** (*Optional*[int])

sample(*size=None*)

Draw one integer or an array of size integers.

Return type

typing.Any

Parameters**size** (*int* | *None*)

Integer categorical

Bayesian categorical likelihoods on contiguous integer ranges.

The probability vector covers integers from `min_index` through `min_index + len(prob_vec) - 1`. Outside observations receive the legacy escape mass `default_value / (1 + default_value)` (zero by default). The default dimension-free Dirichlet prior has concentration `1 + 1e-12` per supported integer. Accumulators store (`minimum`, `count_vector`) and grow the represented range as observations arrive. Expected log-density uses Dirichlet expectations on support and the fixed escape mass outside it.

```
class dmx.bstats.inrange.IntegerCategoricalAccumulator(min_val, max_val, keys=(None,))
```

Bases: [SequenceEncodableAccumulator](#)[int, tuple[int, ndarray[Any, dtype[float64]]], ndarray[Any, dtype[int64]]]

Accumulate weighted counts over a contiguous integer range.

The sufficient statistic is (minimum, count_vector), where entry *i* stores the weight for integer minimum + *i*. Unless both constructor bounds are supplied, the range grows to include observed values.

Parameters

- **min_val** (*Optional[int]*)
- **max_val** (*Optional[int]*)
- **keys** (*tuple[Optional[str], ...]*)

acc_to_encoder()

Create the compatible integer categorical encoder.

Return type

[dmx.bstats.inrange.IntegerCategoricalDataEncoder](#)

combine(suff_stat)

Merge a minimum and contiguous count vector.

Return type

[dmx.bstats.inrange.IntegerCategoricalAccumulator](#)

Parameters

suff_stat (*tuple[int, ndarray[Any, dtype[float64]]]*)

from_value(x)

Restore a minimum and contiguous count vector.

Return type

[dmx.bstats.inrange.IntegerCategoricalAccumulator](#)

Parameters

x (*tuple[int, ndarray[Any, dtype[float64]]]*)

initialize(x, weight, rng)

Accumulate one observation during initialization.

Return type

None

Parameters

- **x** (*int*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(stats_dict)

Merge statistics through the configured sharing key.

Return type

None

Parameters

stats_dict (*MutableMapping[str, Any]*)

key_replace(*stats_dict*)

Replace statistics through the configured sharing key.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *rng*)

Accumulate encoded observations during initialization.

Return type

None

Parameters

- **x** (*ndarray*[*Any*, *dtype*[*int64*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Add encoded integer observations with corresponding weights.

Return type

None

Parameters

- **x** (*ndarray*[*Any*, *dtype*[*int64*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Add one weighted integer, extending the represented range.

Return type

None

Parameters

- **x** (*int*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return (minimum, count_vector).

Return type

tuple[int, numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]]

class dmx.bstats.inrange.**IntegerCategoricalAccumulatorFactory**(*min_val*, *max_val*, *keys*=(*None*,))Bases: *StatisticAccumulatorFactory*[int, tuple[int, ndarray[*Any*, dtype[float64]]], ndarray[*Any*, dtype[int64]]

Create integer categorical accumulators for an optional fixed range.

Parameters

- **min_val** (*Optional*[*int*])

- **max_val** (*Optional[int]*)
- **keys** (*tuple[Optional[str], ...]*)

make()

Create an empty integer categorical accumulator.

Return type

dmx.bstats.inrange.IntegerCategoricalAccumulator

class *dmx.bstats.inrange.IntegerCategoricalDataEncoder*

Bases: *DataSequenceEncoder*[*int*, *ndarray*[*Any*, *dtype*[*int64*]]]

Encode integer categorical observations.

seq_encode(x)

Encode observations in a typed container.

Return type

dmx.bstats.inrange.IntegerCategoricalEncodedData

Parameters

x (*Iterable[int]*)

class *dmx.bstats.inrange.IntegerCategoricalDistribution*(*prob_vec*, *default_value=0.0*,
min_index=0, *name=None*,
prior=default_prior)

Bases: *ProbabilityDistribution*[*int*, *ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*int64*]]]

Represent a categorical likelihood on a contiguous integer range.

A probability vector of shape (k,) corresponds in order to integers *min_index* through *min_index* + *k* - 1. A dense or symmetric Dirichlet prior enables posterior-expected scoring; another prior uses the fixed probabilities. Values outside the range receive the configured escape mass.

Parameters

- **prob_vec** (*Union*[*np.ndarray*, *list*[*float*], *sp.spmatrix*])
- **default_value** (*float*)
- **min_index** (*int*)
- **name** (*str* | *None*)
- **prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

cross_entropy(dist)

Return -E_self[log(dist)] over supported integers.

Return type

float

Parameters

dist (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

dist_to_encoder()

Create the integer categorical sequence encoder.

Return type

dmx.bstats.inrange.IntegerCategoricalDataEncoder

entropy()

Return Shannon entropy over the explicit sampled support.

Return type

float

estimator()

Create an estimator retaining metadata and the current prior.

Return type

dmx.bstats.intrange.IntegerCategoricalEstimator

expected_log_density(x)

Evaluate prior-averaged log mass when conjugate.

Return type

float

Parameters

x (*int*)

get_data_type()

Return the intended observation type.

Return type

type[int]

get_parameters()

Return the probability vector.

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

get_prior()

Return the current Dirichlet or alternate prior.

Return type

dmx.bstats.pdist.ProbabilityDistribution[typing.Any, typing.Any, typing.Any]

log_density(x)

Evaluate supported or escape log mass.

Return type

float

Parameters

x (*int*)

moment(order, origin=0.0)

Return a raw moment about origin.

Return type

float

Parameters

- **order** (*int*)
- **origin** (*float*)

sampler(*seed=None*)

Create a repeatable integer categorical sampler.

Return type

dmx.bstats.intrange.IntegerCategoricalSampler

Parameters

seed (*int* | *None*)

seq_encode(*x*)

Encode *n* observations as an integer array of shape (*n*,).

Return type

numpy.ndarray[*typing.Any*, *numpy.dtype*[*numpy.int64*]]

Parameters

x (*Iterable*[*int*])

seq_expected_log_density(*x*)

Evaluate expected log masses for encoded integers.

Return type

numpy.ndarray[*typing.Any*, *typing.Any*]

Parameters

x (*ndarray*[*Any*, *dtype*[*int64*]])

seq_log_density(*x*)

Evaluate log masses for encoded integer observations.

Return type

numpy.ndarray[*typing.Any*, *typing.Any*]

Parameters

x (*ndarray*[*Any*, *dtype*[*int64*]])

set_parameters(*value*)

Replace probabilities and refresh cached support state.

Return type

None

Parameters

value (*ndarray* | *list*[*float*] | *spmatrix*)

set_prior(*prior*)

Replace the prior and cache supported expected log probabilities.

Return type

None

Parameters

prior (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

class *dmx.bstats.intrange.IntegerCategoricalEncodedData*(*data*)

Bases: *EncodedDataSequence*[*ndarray*[*Any*, *dtype*[*int64*]]]

Contain an encoded integer categorical sequence.

Parameters

data (*IntegerEncoded*)

```
class dmx.bstats.inrange.IntegerCategoricalEstimator(min_index=None, max_index=None,  
                                                    default_value=0.0, name=None,  
                                                    prior=default_prior, keys=(None,))
```

Bases: [ParameterEstimator](#)[int, ndarray[Any, dtype[float64]], ndarray[Any, dtype[int64]], tuple[int, ndarray[Any, dtype[float64]]]

Estimate contiguous integer categorical probabilities from counts.

A dense or symmetric Dirichlet prior is updated to dense posterior concentrations and probabilities are set to its mode when defined, otherwise its mean. With another prior, normalized counts determine the probabilities and the prior is retained.

Parameters

- **min_index** (*Optional[int]*)
- **max_index** (*Optional[int]*)
- **default_value** (*float*)
- **name** (*Optional[str]*)
- **prior** (*Model*)
- **keys** (*tuple[Optional[str], ...]*)

accumulator_factory()

Create a compatible accumulator factory.

Return type

[dmx.bstats.inrange.IntegerCategoricalAccumulatorFactory](#)

estimate(*args)

Estimate probabilities from (minimum, count_vector).

Parameters

***args** – Either the statistic tuple alone or legacy nobs followed by it. nobs is ignored; the vector has one weighted count per consecutive integer beginning at minimum.

Return type

[dmx.bstats.inrange.IntegerCategoricalDistribution](#)

Returns

Fitted integer categorical likelihood on the statistic's range.

Parameters

args (*Any*)

get_prior()

Return the estimator prior.

Return type

[dmx.bstats.pdist.ProbabilityDistribution](#)[typing.Any, typing.Any, typing.Any]

set_prior(prior)

Replace the estimator prior.

Return type

None

Parameters

prior ([ProbabilityDistribution](#)[Any, Any, Any])

class dmx.bstats.intrange.IntegerCategoricalSampler(*dist*, *seed=None*)

Bases: [DistributionSampler](#)[int]

Draw independent integers from a contiguous categorical support.

Parameters

- **dist** ([IntegerCategoricalDistribution](#))
- **seed** (*Optional*[int])

sample(*size=None*)

Draw one integer or a list of size integers.

Return type

`typing.Any`

Parameters

size (*int* | *None*)

Poisson

Bayesian Poisson likelihoods for nonnegative integer counts.

[PoissonDistribution](#)(*lam*) has support on integers $x \geq 0$ and returns $-\infty$ for all other observations. The default gamma prior uses shape 1.0001 and scale 1e6. Accumulators store weighted (count, sum) statistics. Under a gamma prior, expected log-density integrates both $\log(lam)$ and lam ; without a conjugate prior it uses plug-in scoring.

class dmx.bstats.poisson.PoissonDataEncoder

Bases: [DataSequenceEncoder](#)[int, tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]]]]

Encode Poisson counts and cached log-factorials.

seq_encode(*x*)

Encode observations in a typed container.

Return type

`dmx.bstats.poisson.PoissonEncodedData`

Parameters

x (*Iterable*[int])

class dmx.bstats.poisson.PoissonDistribution(*lam*, *name=None*, *prior=default_prior*, *keys=None*)

Bases: [ProbabilityDistribution](#)[int, float, tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]]]]

Represent a Poisson likelihood with a positive finite rate.

Observations are nonnegative integers. A gamma shape-scale prior over the rate enables posterior-expected scoring; another prior uses the fixed rate.

Parameters

- **lam** (*float*)
- **name** (*str* | *None*)
- **prior** ([ProbabilityDistribution](#)[Any, Any, Any])
- **keys** (*Optional*[*str*])

cross_entropy(*dist*)

Return $-E_{\text{self}}[\log(\text{dist})]$ for another Poisson likelihood.

Return type

float

Parameters

dist (*ProbabilityDistribution*[Any, Any, Any])

dist_to_encoder()

Create the Poisson sequence encoder.

Return type

dmx.bstats.poisson.PoissonDataEncoder

entropy()

Return the Poisson Shannon entropy.

Return type

float

estimator()

Create an estimator retaining metadata and the current prior.

Return type

dmx.bstats.poisson.PoissonEstimator

expected_log_density(*x*)

Evaluate prior-averaged log mass when the prior is gamma.

Return type

float

Parameters

x (*int*)

get_data_type()

Return the intended observation type.

Return type

type[int]

get_parameters()

Return the Poisson rate.

Return type

float

get_prior()

Return the current gamma or alternate prior.

Return type

dmx.bstats.pdist.ProbabilityDistribution[typing.Any, typing.Any, typing.Any]

log_density(*x*)

Evaluate log mass, returning $-\infty$ outside count support.

Return type

float

Parameters**x** (*int*)**moment**(*order*)

Return a raw integer moment using Stirling numbers.

Return type

float

Parameters**order** (*int*)**sampler**(*seed=None*)

Create a repeatable Poisson sampler.

Return type*dmx.bstats.poisson.PoissonSampler***Parameters****seed** (*int* | *None*)**seq_encode**(*x*)

Encode count values and their log-factorials.

Parameters**x** – Iterable of *n* count observations.**Return type**

tuple[numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]], numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]]

Returns

Count and log-factorial arrays, each shaped (n,).

Parameters**x** (*Iterable[int]*)**seq_expected_log_density**(*x*)

Evaluate prior-averaged log masses for encoded counts.

Return type

numpy.ndarray[typing.Any, typing.Any]

Parameters**x** (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]])**seq_log_density**(*x*)

Evaluate log masses from encoded values and log-factorials.

Parameters**x** – Count and log-factorial arrays, each shaped (n,).**Return type**

numpy.ndarray[typing.Any, typing.Any]

Returns

Log-mass array of shape (n,).

Parameters**x** (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]])

set_parameters(*value*)

Replace the positive finite rate and cached logarithm.

Return type

None

Parameters**value** (*float*)**set_prior**(*prior*)

Replace the prior and cache gamma hyperparameters.

Return type

None

Parameters**prior** ([ProbabilityDistribution](#)[*Any*, *Any*, *Any*])**class** dmx.bstats.poisson.**PoissonEncodedData**(*data*)Bases: [EncodedDataSequence](#)[*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]]

Contain Poisson values and cached log-factorials.

Parameters**data** (*PoissonEncoded*)**class** dmx.bstats.poisson.**PoissonEstimator**(*name=None*, *keys=None*, *prior=default_prior*)Bases: [ParameterEstimator](#)[*int*, *float*, *tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]], *tuple*[*float*, *float*]]

Estimate a Poisson rate from weighted count statistics.

A gamma shape-scale prior is updated conjugately, and the returned rate is its positive-clipped posterior mode. Another non-null prior produces a numerical MAP estimate; a null prior produces the weighted maximum-likelihood rate.

Parameters

- **name** (*Optional*[*str*])
- **keys** (*Optional*[*str*])
- **prior** (*Model*)

accumulator_factory()

Create a compatible accumulator factory.

Return type[dmx.bstats.poisson.PoissonEstimatorAccumulatorFactory](#)**estimate**(**args*)Estimate from (*observation_count*, *count_sum*) statistics.**Parameters*****args** – Either the statistic tuple alone or legacy *nobs* followed by it. *nobs* is ignored.**Return type**[dmx.bstats.poisson.PoissonDistribution](#)**Returns**

Poisson likelihood carrying the updated posterior when the prior is conjugate.

Parameters**args** (*Any*)

get_prior()

Return the estimator prior.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[typing.Any, typing.Any, typing.Any]

set_prior(prior)

Replace the estimator prior and update prior flags.

Return type

None

Parameters

prior (`ProbabilityDistribution`[Any, Any, Any])

class `dmx.bstats.poisson.PoissonEstimatorAccumulator`(*name*, *keys*)

Bases: `StatisticAccumulator`[int, tuple[float, float], tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]]]

Accumulate weighted (observation_count, count_sum) statistics.

Parameters

- **name** (*Optional*[str])
- **keys** (*Optional*[str])

acc_to_encoder()

Create the compatible Poisson encoder.

Return type

`dmx.bstats.poisson.PoissonDataEncoder`

combine(suff_stat)

Merge (observation_count, count_sum) statistics.

Return type

`dmx.bstats.poisson.PoissonEstimatorAccumulator`

Parameters

suff_stat (tuple[float, float])

from_value(x)

Restore (observation_count, count_sum).

Return type

`dmx.bstats.poisson.PoissonEstimatorAccumulator`

Parameters

x (tuple[float, float])

initialize(x, weight, rng)

Accumulate one observation during initialization.

Return type

None

Parameters

- **x** (int)
- **weight** (float)

- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge statistics through the configured sharing key.

Return type

None

Parameters

stats_dict (*MutableMapping*[*str*, *Any*])

key_replace(*stats_dict*)

Replace statistics through the configured sharing key.

Return type

None

Parameters

stats_dict (*MutableMapping*[*str*, *Any*])

seq_initialize(*x*, *weights*, *rng*)

Accumulate encoded observations during initialization.

Return type

None

Parameters

- **x** (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]])
- **weights** (*ndarray*[*Any*, *Any*])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Add encoded counts with corresponding weights.

Return type

None

Parameters

- **x** (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]]])
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Add one weighted nonnegative count.

Return type

None

Parameters

- **x** (*int*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return (observation_count, count_sum).

Return type

tuple[float, float]

class dmx.bstats.poisson.**PoissonEstimatorAccumulatorFactory**(name, keys)

Bases: [StatisticAccumulatorFactory](#)[int, tuple[float, float], tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]]]]

Create Poisson accumulators with shared metadata.

Parameters

- **name** (*Optional[str]*)
- **keys** (*Optional[str]*)

make()

Create an empty Poisson accumulator.

Return type

[dmx.bstats.poisson.PoissonEstimatorAccumulator](#)

class dmx.bstats.poisson.**PoissonSampler**(dist, seed=None)

Bases: [DistributionSampler](#)[int]

Draw independent nonnegative Poisson counts.

Parameters

- **dist** ([PoissonDistribution](#))
- **seed** (*Optional[int]*)

sample(size=None)

Draw one integer or an array of size integers.

Return type

typing.Any

Parameters

size (int | None)

1.4.4 Structured and Mixture Models

Composite

Bayesian product distributions for heterogeneous tuple observations.

[CompositeDistribution](#) preserves component order: observation element $x[i]$ is scored, encoded, sampled, and estimated by `dists[i]`. The joint log-density and prior-expected log-density are sums because components are conditionally independent. A composite name labels the product while child names continue to control component [DataFrame](#) operations.

The optional composite key shares the complete ordered tuple of child sufficient statistics between accumulators. Child accumulator keys are also merged and replaced independently, so composite and child keys should be distinct. Priors have the same ordered product structure as parameters.

class dmx.bstats.composite.**CompositeAccumulatorFactory**(factories, keys)

Bases: [StatisticAccumulatorFactory](#)[tuple[Any, ...], tuple[Any, ...], tuple[Any, ...]]

Create composite accumulators from ordered child factories.

Parameters

- **factories** (*Sequence*[*AccumulatorFactory*])
- **keys** (*Optional*[*str*])

make()

Create a fresh accumulator for every component.

Return type

dmx.bstats.composite.CompositeEstimatorAccumulator

class *dmx.bstats.composite.CompositeDataEncoder*(*encoders*)

Bases: *DataSequenceEncoder*[*tuple*[*Any*, ...], *tuple*[*Any*, ...]]

Encode each tuple position with its ordered child encoder.

For *n* tuple observations and *k* children, the payload is a length-*k* tuple. Item *i* is the child-specific encoding of the *n* values at observation position *i*.

Parameters

encoders – Child encoders in observation order.

Parameters

encoders (*Sequence*[*Encoder*])

seq_encode(*x*)

Encode each observation position and unwrap child containers.

Return type

dmx.bstats.composite.CompositeEncodedData

Parameters

x (*Iterable*[*tuple*[*Any*, ...]])

class *dmx.bstats.composite.CompositeDistribution*(*dists*, *name=None*, *keys=None*)

Bases: *ProbabilityDistribution*[*tuple*[*Any*, ...], *tuple*[*Any*, ...], *tuple*[*Any*, ...]]

Model an ordered tuple as a product of heterogeneous distributions.

An observation must contain one value per child. Direct and encoded sequence operations preserve that order, and the joint log density is the sum of the child log densities. The encoded payload and sufficient statistic are tuples whose item *i* belongs to *dists*[*i*].

name identifies the product itself. *keys* is copied into the estimator and controls whole-product sufficient-statistic sharing; it does not replace or reorder component-level keys.

Parameters

- **dists** – Child distributions in observation order.
- **name** – Optional identifier for the complete product.
- **keys** – Optional key used to share the complete tuple of child sufficient statistics between accumulators.

Parameters

- **dists** (*Sequence*[*Model*])
- **name** (*str* | *None*)
- **keys** (*Optional*[*str*])

cross_entropy(*dist*)

Sum component cross-entropies in their fixed order.

Return type

float

Parameters

dist (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

df_log_density(*df*)

Sum child DataFrame log-densities using component names.

Return type

`pandas.core.series.Series`

Parameters

df (*DataFrame*)

dist_to_encoder()

Create an ordered product of component encoders.

Return type

`dmx.bstats.composite.CompositeDataEncoder`

entropy()

Return the sum of independent component entropies.

Return type

float

estimator()

Create an estimator preserving component order and metadata.

Return type

`dmx.bstats.composite.CompositeEstimator`

expected_log_density(*x*)

Sum child prior-expected log-densities for one observation.

Return type

float

Parameters

x (*tuple*[*Any*, ...])

get_parameters()

Return child parameters in component order.

Return type

`tuple`[*typing.Any*, ...]

get_prior()

Return the ordered product of component priors.

Return type

`dmx.bstats.composite.CompositeDistribution`

log_density(*x*)

Sum child log-densities for one ordered tuple observation.

Return type

float

Parameters**x** (*tuple*[*Any*, ...])**sampler**(*seed*=None)

Create a sampler with one independently seeded child sampler.

Return type*dmx.bstats.composite.CompositeSampler***Parameters****seed** (*int* | *None*)**seq_encode**(*x*)

Encode each tuple position with its corresponding component.

Return type*tuple*[*typing.Any*, ...]**Parameters****x** (*Iterable*[*tuple*[*Any*, ...]])**seq_expected_log_density**(*x*)

Sum vectorized child prior-expected log-densities elementwise.

Return type*Array***Parameters****x** (*CompositeEncoded* | '*CompositeEncodedData*') **seq_log_density**(*x*)

Sum vectorized child log-densities elementwise.

Return type*Array***Parameters****x** (*CompositeEncoded* | '*CompositeEncodedData*') **set_parameters**(*value*)

Set child parameters position by position.

Return type*None***Parameters****value** (*tuple*[*Any*, ...])**set_prior**(*prior*)

Propagate an ordered composite prior to every component.

Parameters**prior** – Composite distribution whose children are component priors.**Raises****TypeError** – If prior is not a composite distribution.**Return type***None***Parameters****prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

class dmx.bstats.composite.**CompositeEncodedData**(*data*)

Bases: [EncodedDataSequence](#)[tuple[Any, ...]]

Contain component encodings in the same order as the distributions.

Parameters

data ([CompositeEncoded](#))

class dmx.bstats.composite.**CompositeEstimator**(*estimators*, *name=None*, *keys=None*)

Bases: [ParameterEstimator](#)[tuple[Any, ...], tuple[Any, ...], tuple[Any, ...], tuple[Any, ...]]

Estimate an ordered product using one estimator per component.

Each child estimator consumes the statistic at the same tuple position. *keys* shares the complete ordered statistic through the composite accumulator; sharing configured on child estimators remains active.

Parameters

- **estimators** – Child estimators in observation order.
- **name** – Optional identifier copied to the estimated product.
- **keys** – Optional key for sharing the complete child-statistic tuple.

Parameters

- **estimators** ([Sequence](#)[[Estimator](#)])
- **name** ([Optional](#)[*str*])
- **keys** ([Optional](#)[*str*])

accumulator_factory()

Create a concrete factory preserving the whole-product key.

Return type

[dmx.bstats.composite.CompositeAccumulatorFactory](#)

estimate(**args*)

Estimate each component from its corresponding sufficient statistic.

Return type

[dmx.bstats.composite.CompositeDistribution](#)

Parameters

args (*Any*)

get_prior()

Return the ordered product of child estimator priors.

Return type

[dmx.bstats.composite.CompositeDistribution](#)

model_log_density(*model*)

Score ordered model parameters under the ordered product prior.

Return type

float

Parameters

model ([CompositeDistribution](#))

set_prior(*prior*)

Propagate an ordered composite prior to child estimators.

Return type

None

Parameters

prior ([ProbabilityDistribution](#)[Any, Any, Any])

class dmx.bstats.composite.**CompositeEstimatorAccumulator**(*accumulators*, *keys*=None)

Bases: [SequenceEncodableAccumulator](#)[tuple[Any, ...], tuple[Any, ...], tuple[Any, ...]],
[DataFrameEncodableAccumulator](#)[tuple[Any, ...], tuple[Any, ...], tuple[Any, ...]]

Accumulate one ordered sufficient-statistic value per component.

A non-None composite key shares the entire tuple returned by [value\(\)](#). After that whole-product operation, supported child keys are merged or replaced as well.

Parameters

- **accumulators** ([Sequence](#)[[Accumulator](#)])
- **keys** ([Optional](#)[[str](#)])

acc_to_encoder()

Create an ordered product of child accumulator encoders.

Return type

[dmx.bstats.composite.CompositeDataEncoder](#)

combine(*suff_stat*)

Merge ordered child sufficient statistics.

Return type

[dmx.bstats.composite.CompositeEstimatorAccumulator](#)

Parameters

suff_stat ([tuple](#)[Any, ...])

df_initialize(*df*, *weights*, *rng*)

Initialize child accumulators from their named DataFrame columns.

Return type

None

Parameters

- **df** ([DataFrame](#))
- **weights** ([Iterable](#)[[float](#)])
- **rng** ([RandomState](#))

df_update(*df*, *weights*, *estimate*)

Update child accumulators from their named DataFrame columns.

Return type

None

Parameters

- **df** ([DataFrame](#))
- **weights** ([Iterable](#)[[float](#)])

- **estimate** (`ProbabilityDistribution[Any, Any, Any]` | `None`)

from_value(*x*)

Restore child sufficient statistics in component order.

Return type

`dmx.bstats.composite.CompositeEstimatorAccumulator`

Parameters

x (`tuple[Any, ...]`)

initialize(*x*, *weight*, *rng*)

Initialize every child from the matching observation position.

Return type

`None`

Parameters

- **x** (`tuple[Any, ...]`)
- **weight** (`float`)
- **rng** (`RandomState`)

key_merge(*stats_dict*)

Merge the whole product key, then independent supported child keys.

Return type

`None`

Parameters

stats_dict (`MutableMapping[str, Any]`)

key_replace(*stats_dict*)

Replace from the whole product key, then supported child keys.

Return type

`None`

Parameters

stats_dict (`MutableMapping[str, Any]`)

seq_initialize(*x*, *weights*, *rng*)

Initialize every child from its encoded sequence position.

Return type

`None`

Parameters

- **x** (`tuple[Any, ...]`)
- **weights** (`ndarray[Any, Any]`)
- **rng** (`RandomState`)

seq_update(*x*, *weights*, *estimate*)

Update every child from its encoded sequence position.

Return type

`None`

Parameters

- **x** (*tuple*[*Any*, ...])
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Update every child from the matching observation position.

Return type

None

Parameters

- **x** (*tuple*[*Any*, ...])
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return child sufficient statistics in component order.

Return type

tuple[*typing.Any*, ...]

class `dmx.bstats.composite.CompositeSampler`(*dist*, *seed=None*)

Bases: *DistributionSampler*[*tuple*[*Any*, ...]]

Draw ordered tuples from the component distributions.

Parameters

- **dist** (*CompositeDistribution*)
- **seed** (*Optional*[*int*])

sample(*size=None*)

Draw one tuple or a list of *size* tuples.

Return type

typing.Any

Parameters

size (*int* | *None*)

Conditional

Bayesian distributions selected by a hashable condition value.

An observation is (*condition*, *value*). The condition selects a child from *dmap*; when it is absent, *default_dist* is used. The shared *null_dist* means that no fallback is configured: such observations have log-density *-inf* and are ignored by sufficient-statistic accumulators.

The condition distribution supplies keys for sampling. It is not another factor in the observation log-density and is retained, rather than re-fitted, by the conditional estimator. This preserves the legacy *bstats* model.

Encoded data has the stable five-part shape (*size*, *conditions*, *encoded_values*, *indices*, *encoded_conditions*). The middle three tuples are parallel and contain one entry per distinct condition, in first-observed order. *encoded_values* contains *None* for a missing condition when no default distribution exists. *indices* maps each group back to positions in the original sequence.

```
class dmx.bstats.conditional.ConditionalDataEncoder(encoder_map, given_encoder, default_encoder,
                                                  pass_value=False)
```

Bases: [DataSequenceEncoder](#)[tuple[Hashable, Any], tuple[int, tuple[Hashable, ...], tuple[Any | None, ...], tuple[ndarray[Any, Any], ...], Any]]

Encode conditional observations into aligned per-condition groups.

The payload is (n, conditions, encoded_values, indices, encoded_conditions). The three middle tuples have one entry per distinct condition in first-observed order. Each index vector restores that group's positions among the n observations. An unknown condition without a default encoder has None as its encoded value.

Parameters

- **encoder_map** – Mapping from condition values to child encoders.
- **given_encoder** – Encoder for the condition elements themselves.
- **default_encoder** – Encoder for conditions absent from **encoder_map**.
- **pass_value** – Whether child encoders receive complete observations.

Parameters

- **encoder_map** (*Mapping[ConditionKey, Encoder]*)
- **given_encoder** (*Encoder*)
- **default_encoder** (*Encoder*)
- **pass_value** (*bool*)

```
seq_encode(x)
```

Group values by condition and encode all five payload components.

Return type

[dmx.bstats.conditional.ConditionalEncodedData](#)

Parameters

x (*Iterable[tuple[Hashable, Any]]*)

```
class dmx.bstats.conditional.ConditionalDistribution(dmap, cond_dist, default_dist=null_dist,
                                                  pass_value=False)
```

Bases: [ProbabilityDistribution](#)[tuple[Hashable, Any], Any, tuple[int, tuple[Hashable, ...], tuple[Any | None, ...], tuple[ndarray[Any, Any], ...], Any]]

Select a child distribution using the first observation element.

If **pass_value** is false, the selected child receives only the second observation element. If true, it receives the complete (condition, value) pair.

The condition model is used to sample conditions and encode them, but its density is not included in the conditional observation score. An unknown condition is handled by **default_dist**; the default **null_dist** means the observation is assigned log density $-\infty$ and contributes no child sufficient statistics.

Parameters

- **dmap** – Mapping from condition values to child distributions.
- **cond_dist** – Fixed distribution used to sample and encode conditions.
- **default_dist** – Child used for conditions absent from **dmap**.
- **pass_value** – Whether children receive the complete observation instead of only its value element.

Parameters

- **dmap** (*Mapping[ConditionKey, Model]*)
- **cond_dist** (*Model*)
- **default_dist** (*Model*)
- **pass_value** (*bool*)

dist_to_encoder()

Create an encoder with matching explicit, default, and condition paths.

Return type

dmx.bstats.conditional.ConditionalDataEncoder

estimator(*pseudo_count=None*)

Create an estimator for children while retaining the condition model.

Return type

dmx.bstats.conditional.ConditionalDistributionEstimator

Parameters

pseudo_count (*float | None*)

log_density(*x*)

Score with the selected child or the configured default.

A condition absent from **dmap** has log-density `-inf` when the default is **null_dist**.

Return type

float

Parameters

x (*tuple[Hashable, Any]*)

sampler(*seed=None*)

Create a sampler that first draws a condition, then its child value.

Return type

dmx.bstats.conditional.ConditionalDistributionSampler

Parameters

seed (*int | None*)

seq_encode(*x*)

Encode observations directly into the stable five-part payload.

Return type

tuple[int, tuple[collections.abc.Hashable, ...], tuple[typing.Optional[typing.Any], ...], tuple[numpy.ndarray[typing.Any, typing.Any], ...], typing.Any]

Parameters

x (*Iterable[tuple[Hashable, Any]]*)

seq_log_density(*x*)

Score the documented five-part conditional encoding.

Return type

Array

Parameters

x (*ConditionalEncoded | 'ConditionalEncodedData'*)


```
class dmx.bstats.conditional.ConditionalDistributionAccumulatorFactory(factory_map,
                                                                    default_factory, keys,
                                                                    given_encoder,
                                                                    pass_value)
```

Bases: [StatisticAccumulatorFactory](#)[[tuple](#)[[Hashable](#), [Any](#)], [tuple](#)[[dict](#)[[Hashable](#), [Any](#)], [Any](#) | [None](#)], [tuple](#)[[int](#), [tuple](#)[[Hashable](#), ...], [tuple](#)[[Any](#) | [None](#), ...], [tuple](#)[[ndarray](#)[[Any](#), [Any](#)], ...], [Any](#)]]

Create fresh keyed conditional accumulators.

Parameters

- **factory_map** (*Mapping*[*ConditionKey*, *AccumulatorFactory*])
- **default_factory** (*Optional*[*AccumulatorFactory*])
- **keys** (*Optional*[*str*])
- **given_encoder** (*Encoder*)
- **pass_value** (*bool*)

make()

Create an empty accumulator for every configured child.

Return type

[dmx.bstats.conditional.ConditionalDistributionEstimatorAccumulator](#)

```
class dmx.bstats.conditional.ConditionalDistributionEstimator(estimator_map,
                                                            default_estimator=None,
                                                            keys=None, cond_dist=null_dist,
                                                            pass_value=False)
```

Bases: [ParameterEstimator](#)[[tuple](#)[[Hashable](#), [Any](#)], [Any](#), [tuple](#)[[int](#), [tuple](#)[[Hashable](#), ...], [tuple](#)[[Any](#) | [None](#), ...], [tuple](#)[[ndarray](#)[[Any](#), [Any](#)], ...], [Any](#)], [tuple](#)[[dict](#)[[Hashable](#), [Any](#)], [Any](#) | [None](#)]]

Estimate explicit and default children while retaining the condition model.

The condition distribution is fixed: it supplies sampling keys and an encoder but receives no sufficient statistics. Each explicit child is estimated from the matching entry in the statistic mapping, and the optional default child is estimated from the second statistic item.

Parameters

- **estimator_map** – Mapping from condition values to child estimators.
- **default_estimator** – Estimator for conditions absent from the mapping, or [None](#) to ignore those observations during fitting.
- **keys** – Compatibility metadata retained by the accumulator factory.
- **cond_dist** – Fixed distribution used to sample and encode conditions.
- **pass_value** – Whether child estimators receive complete observations.

Parameters

- **estimator_map** (*Mapping*[*ConditionKey*, *Estimator*])
- **default_estimator** (*Optional*[*Estimator*])
- **keys** (*Optional*[*str*])
- **cond_dist** (*Model*)
- **pass_value** (*bool*)

accumulator_factory()

Create a concrete factory for explicit and default accumulators.

Return type

dmx.bstats.conditional.ConditionalDistributionAccumulatorFactory

estimate(*args)

Estimate every configured child from matching sufficient statistics.

Return type

dmx.bstats.conditional.ConditionalDistribution

Parameters

args (*Any*)

```
class dmx.bstats.conditional.ConditionalDistributionEstimatorAccumulator(accumulator_map,  
                                                                    default_accumulator,  
                                                                    keys=None,  
                                                                    given_encoder=None,  
                                                                    pass_value=False)
```

Bases: *SequenceEncodableAccumulator*[*tuple*[*Hashable*, *Any*], *tuple*[*dict*[*Hashable*, *Any*], *Any* | *None*], *tuple*[*int*, *tuple*[*Hashable*, ...], *tuple*[*Any* | *None*, ...], *tuple*[*ndarray*[*Any*, *Any*], ...], *Any*]]

Accumulate explicit child statistics and optional default statistics.

The public sufficient statistic is (*explicit*, *default*), where *explicit* maps each configured condition to its child statistic and *default* is the fallback statistic or *None*. Unknown conditions are ignored when no default accumulator is configured. Key sharing is delegated to the explicit and default children.

Parameters

- **accumulator_map** (*Mapping*[*ConditionKey*, *Accumulator*])
- **default_accumulator** (*Optional*[*Accumulator*])
- **keys** (*Optional*[*str*])
- **given_encoder** (*Optional*[*Encoder*])
- **pass_value** (*bool*)

acc_to_encoder()

Create an encoder matching accumulator child paths.

Return type

dmx.bstats.conditional.ConditionalDataEncoder

combine(suff_stat)

Merge explicit and default sufficient statistics.

Return type

dmx.bstats.conditional.ConditionalDistributionEstimatorAccumulator

Parameters

suff_stat (*tuple*[*dict*[*Hashable*, *Any*], *Any* | *None*])

from_value(x)

Restore explicit and default sufficient statistics.

Return type

dmx.bstats.conditional.ConditionalDistributionEstimatorAccumulator

Parameters**x** (*tuple*[*dict*[*Hashable*, *Any*], *Any* | *None*])**initialize**(*x*, *weight*, *rng*)

Initialize the selected explicit or default accumulator.

Return type*None***Parameters**

- **x** (*tuple*[*Hashable*, *Any*])
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Delegate keyed statistic merging to every child accumulator.

Return type*None***Parameters****stats_dict** (*MutableMapping*[*str*, *Any*])**key_replace**(*stats_dict*)

Delegate keyed statistic replacement to every child accumulator.

Return type*None***Parameters****stats_dict** (*MutableMapping*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *rng*)

Initialize accumulators from grouped encoded observations.

Return type*None***Parameters**

- **x** (*tuple*[*int*, *tuple*[*Hashable*, ...], *tuple*[*Any* | *None*, ...], *tuple*[*ndarray*[*Any*, *Any*], ...], *Any* | [ConditionalEncodedData](#))
- **weights** (*ndarray*[*Any*, *Any*])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Update accumulators from grouped encoded observations.

Return type*None***Parameters**

- **x** (*tuple*[*int*, *tuple*[*Hashable*, ...], *tuple*[*Any* | *None*, ...], *tuple*[*ndarray*[*Any*, *Any*], ...], *Any* | [ConditionalEncodedData](#))
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** ([ProbabilityDistribution](#)[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Update the selected explicit or default accumulator.

Return type

None

Parameters

- **x** (*tuple*[*Hashable*, *Any*])
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return explicit statistics followed by optional default statistics.

Return type

tuple[*dict*[*collections.abc.Hashable*, *typing.Any*], *typing.Optional*[*typing.Any*]]

class *dmx.bstats.conditional.ConditionalDistributionSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*[*tuple*[*Hashable*, *Any*]]

Draw a condition and then sample its explicit or default child.

Parameters

- **dist** (*ConditionalDistribution*)
- **seed** (*Optional*[*int*])

sample(*size=None*)

Draw one conditional observation or a list of *size* observations.

Return type

typing.Any

Parameters

size (*int* | *None*)

class *dmx.bstats.conditional.ConditionalEncodedData*(*data*)

Bases: *EncodedDataSequence*[*tuple*[*int*, *tuple*[*Hashable*, ...], *tuple*[*Any* | *None*, ...], *tuple*[*ndarray*[*Any*, *Any*], ...], *Any*]]

Contain the stable five-part conditional sequence encoding.

data is (*size*, *conditions*, *encoded_values*, *indices*, *encoded_conditions*). The condition, value, and index tuples always have equal lengths, including when an unknown condition has no default encoder.

Parameters

data (*ConditionalEncoded*)

Dirichlet-process mixture

Truncated variational Dirichlet-process mixture models.

The truncation level is the number of supplied component estimators. Each variational stick has beta parameters *g*[*k*] = (*gamma_1*, *gamma_2*). Expected log stick lengths and expected log remaining lengths are combined into ordered component log weights, then exponentiated and normalized for the finite predictive mixture stored in *w*. The legacy *v* attribute aliases these normalized predictive weights; it does not contain beta-stick means.

Component distributions carry posterior priors, while `component_priors` records the corresponding priors from before the update. Components and their priors are sorted together by decreasing effective count after every update.

class `dmx.bstats.dpm.DirichletProcessMixtureAccumulator`(*accumulators*, *keys*=(*None*, *None*))

Bases: `SequenceEncodableAccumulator`[*Any*, *tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]], *float*, *float*, *tuple*[*Any*, ...], *Any*]

Accumulate variational assignments and component statistics.

The sufficient statistic is (`component_counts`, `beta_counts`, `alpha`, `previous_log_remainder`, `child_statistics`). For truncation level *K*, the first two arrays have shapes (*K*,) and (*K*, 2) and the last item is a length-*K* tuple in component order. Updates use expected component scores and current predictive log weights to form responsibilities.

The optional keys independently share beta counts and the complete child accumulator collection. Child-level sharing remains active.

Parameters

- **accumulators** (*Sequence*[*Accumulator*])
- **keys** (*DPMKeys*)

acc_to_encoder()

Return the homogeneous encoder exposed by the first component.

Return type

`typing.Any`

combine(*suff_stat*)

Merge variational and ordered component sufficient statistics.

Return type

`dmx.bstats.dpm.DirichletProcessMixtureAccumulator`

Parameters

suff_stat (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]], *float*, *float*, *tuple*[*Any*, ...])

from_value(*x*)

Restore variational and component sufficient statistics.

Return type

`dmx.bstats.dpm.DirichletProcessMixtureAccumulator`

Parameters

x (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]], *float*, *float*, *tuple*[*Any*, ...])

initialize(*x*, *weight*, *rng*)

Randomly split one weight with the established uniform Dirichlet.

Return type

`None`

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge DPM blocks and supported child statistics by key.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**key_replace**(*stats_dict*)

Replace DPM blocks and supported child statistics by key.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**seq_update**(*x*, *weights*, *estimate*)

Allocate encoded observations by variational responsibility.

Return type

None

Parameters

- **x** (*Any*)
- **weights** (*ndarray*[*Any*, *dtype*[*float64*]])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Allocate one observation by variational responsibility.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return copied variational and component sufficient statistics.

Return type

tuple[numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]], numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]], float, float, tuple[typing.Any, ...]]

class dmx.bstats.dpm.**DirichletProcessMixtureAccumulatorFactory**(*factories*, *dim*, *keys*)Bases: [StatisticAccumulatorFactory](#)[*Any*, tuple[*ndarray*[*Any*, *dtype*[*float64*]], *ndarray*[*Any*, *dtype*[*float64*]], *float*, *float*, tuple[*Any*, ...], *Any*]

Create DPM accumulators for a fixed truncation level.

Parameters

- **factories** (*Sequence*[*AccumulatorFactory*])
- **dim** (*int*)

- **keys** (*DPMKeys*)

make()

Create one fresh accumulator per truncated component.

Return type

dmx.bstats.dpm.DirichletProcessMixtureAccumulator

class `dmx.bstats.dpm.DirichletProcessMixtureDistribution`(*components*, *w*, *a*, *g*, *component_priors*, *name=None*, *prior=default_prior*)

Bases: *ProbabilityDistribution*[*Any*, *tuple*[*float*, *ndarray*[*Any*, *dtype*[*float64*]], *list*[*Any*]], *Any*]

Finite truncation of a variational Dirichlet-process mixture.

g stores beta variational parameters and *a* the concentration. *w* is the normalized finite predictive approximation used for sampling and ordinary log-density.

There are *K* components, *w* has shape (*K*,), and *g* has shape (*K*, 2). The legacy *v* attribute is an alias for *w* rather than a vector of beta-stick means. Components must share an observation encoding.

Parameters

- **components** – Ordered homogeneous component distributions.
- **w** – Finite nonnegative predictive weights, normalized on input.
- **a** – Positive Dirichlet-process concentration parameter.
- **g** – Positive beta variational parameters shaped (*K*, 2).
- **component_priors** – Pre-update priors paired with the components.
- **name** – Optional identifier for the mixture.
- **prior** – Hyperprior for the concentration parameter.

Raises

ValueError – If component counts or shapes disagree, or concentration, weights, or beta parameters are invalid.

Parameters

- **components** (*Sequence*[*Model*])
- **w** (*Sequence*[*float*] | *np.ndarray*[*Any*, *Any*])
- **a** (*float*)
- **g** (*np.ndarray*[*Any*, *Any*])
- **component_priors** (*Sequence*[*Model*])
- **name** (*str* | *None*)
- **prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

estimator(*pseudo_count=None*)

Create a compatible estimator for the current truncation level.

Return type

dmx.bstats.dpm.DirichletProcessMixtureEstimator

Parameters

pseudo_count (*float* | *None*)

expected_log_density(*x*)

Combine component posterior expectations with predictive log weights.

Return type

float

Parameters

x (*Any*)

get_parameters()

Return concentration, legacy normalized v weights, and parameters.

Return type

tuple[float, numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]],
list[typing.Any]]

get_prior()

Return concentration, stick, and ordered component priors.

Return type

dmx.bstats.composite.CompositeDistribution

log_density(*x*)

Evaluate the normalized finite predictive mixture density.

Return type

float

Parameters

x (*Any*)

sampler(*seed=None*)

Create a repeatable sampler from normalized predictive weights.

Return type

dmx.bstats.dpm.DirichletProcessMixtureSampler

Parameters

seed (*int* | *None*)

seq_encode(*x*)

Encode homogeneous observations with the first component.

Return type

typing.Any

Parameters

x (*Iterable[Any]*)

seq_expected_log_density(*x*)

Return predictive expected scores for encoded observations.

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Parameters

x (*Any*)

seq_log_density(*x*)

Return the variational objective for one encoded data block.

The one-element result is the block evidence lower bound used by the legacy optimizer for convergence, not one predictive score per row.

Return type

`numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]`

Parameters

x (*Any*)

set_parameters(*value*)

Replace concentration, predictive weights, and component objects.

Return type

`None`

Parameters

value (`tuple[float, ndarray[Any, dtype[float64]], list[Any]]`)

set_prior(*prior*)

Propagate a complete concentration, stick, and component prior.

Return type

`None`

Parameters

prior (`ProbabilityDistribution[Any, Any, Any]`)

class `dmx.bstats.dpm.DirichletProcessMixtureEstimator`(*estimators*, *name=None*, *prior=default_prior*, *keys=(None, None)*)

Bases: `ParameterEstimator[Any, tuple[float, ndarray[Any, dtype[float64]], list[Any]], Any, tuple[ndarray[Any, dtype[float64]], ndarray[Any, dtype[float64]], float, float, tuple[Any, ...]]`

Perform the established coordinate update for a truncated DPM.

The number of component estimators fixes the truncation level. Estimation updates beta-stick parameters and the concentration, estimates every component, then stably sorts components and their pre-update priors by decreasing effective count. The resulting normalized weights are the finite predictive approximation, not beta-stick means.

Parameters

- **estimators** – Component estimators defining the truncation level.
- **name** – Optional identifier copied to the estimated mixture.
- **prior** – Hyperprior for the concentration parameter.
- **keys** – Pair (weight_key, component_key) controlling independent sufficient-statistic sharing.

Parameters

- **estimators** (`Sequence[Estimator]`)
- **name** (`Optional[str]`)
- **prior** (`Optional[Model]`)
- **keys** (`DPMKeys`)

accumulator_factory()

Create a factory retaining estimator order and sharing keys.

Return type

`dmx.bstats.dpm.DirichletProcessMixtureAccumulatorFactory`

estimate(*args)

Apply one variational update and sort components by effective count.

Return type`dmx.bstats.dpm.DirichletProcessMixtureDistribution`**Parameters****args** (*Any*)**get_prior**()

Return concentration, stick, and ordered component priors.

Return type`dmx.bstats.composite.CompositeDistribution`**model_log_density**(model)

Return prior and entropy terms for optimizer convergence output.

Return type

float

Parameters**model** (`DirichletProcessMixtureDistribution`)**set_prior**(prior)

Propagate a complete concentration, stick, and component prior.

Return type

None

Parameters**prior** (`ProbabilityDistribution`[*Any*, *Any*, *Any*])**class** `dmx.bstats.dpm.DirichletProcessMixtureSampler`(dist, seed=None)Bases: `DistributionSampler`[*Any*]

Sample component indices and observations from a truncated DPM.

Parameters

- **dist** (`DirichletProcessMixtureDistribution`)
- **seed** (`Optional`[*int*])

sample(size=None)

Draw one observation or a list of size observations.

Return type`typing.Any`**Parameters****size** (*int* | *None*)`dmx.bstats.dpm.cbg`(x, s1, s2)

Evaluate the legacy transformed beta-gamma log density helper.

Return type

float

Parameters

- **x** (*float*)
- **s1** (*float*)

- **s2** (*float*)

Ignored

Keep a Bayesian distribution fixed while retaining its scoring behavior.

An `IgnoredDistribution` delegates scoring, sampling, parameters, and priors to a wrapped distribution. Its accumulator intentionally discards observations and its estimator returns the same wrapped distribution unchanged. Thus “ignored” refers only to parameter estimation; observations are still scored normally.

class `dmx.bstats.ignored.IgnoredAccumulator`(*encoder=None*)

Bases: `StatisticAccumulator`[*Any*, *None*, *Any*]

Accumulator that deliberately discards all observations and weights.

Parameters

encoder (*Optional*[`DataSequenceEncoder`[*Any*, *Any*]])

acc_to_encoder()

Create the encoder corresponding to this accumulator.

Return type

`dmx.bstats.ignored.IgnoredDataEncoder`

combine(*suff_stat*)

Combine the sole ignored statistic value.

Return type

`dmx.bstats.ignored.IgnoredAccumulator`

Parameters

suff_stat (*None*)

from_value(*x*)

Restore the sole ignored sufficient statistic.

Return type

`dmx.bstats.ignored.IgnoredAccumulator`

Parameters

x (*None*)

initialize(*x*, *weight*, *rng*)

Discard one observation during randomized initialization.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Leave shared statistics unchanged.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**key_replace**(*stats_dict*)

Leave shared statistics unchanged.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *rng*)

Discard encoded observations during randomized initialization.

Return type

None

Parameters

- **x** (*Any*)
- **weights** (*ndarray*[*Any*, *Any*])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Discard a sequence of weighted observations.

Return type

None

Parameters

- **x** (*Any*)
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Discard one weighted observation.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return the sole ignored sufficient statistic.

Return type

None

class dmx.bstats.ignored.**IgnoredAccumulatorFactory**(*encoder*)Bases: *StatisticAccumulatorFactory*[*Any*, *None*, *Any*]

Create fresh accumulators that ignore estimation data.

Parameters**encoder** (*DataSequenceEncoder*[Any, Any])**make()**

Create an ignored accumulator.

Return type*dmx.bstats.ignored.IgnoredAccumulator***class** *dmx.bstats.ignored.IgnoredDataEncoder*(*encoder*)Bases: *DataSequenceEncoder*[Any, Any]

Wrap the sequence encoder belonging to the fixed distribution.

Parameters**encoder** (*DataSequenceEncoder*[Any, Any])**seq_encode(x)**

Encode observations using the fixed distribution's encoder.

Return type*dmx.bstats.ignored.IgnoredEncodedData***Parameters****x** (*Iterable*[Any])**class** *dmx.bstats.ignored.IgnoredDistribution*(*dist=null_dist*)Bases: *ProbabilityDistribution*[Any, Any, Any]

Wrap a fixed distribution that participates in scoring but not fitting.

Parameters**dist** (*Model*)**cross_entropy(dist)**

Delegate cross-entropy evaluation to the wrapped distribution.

Return type

float

Parameters**dist** (*ProbabilityDistribution*[Any, Any, Any])**density(x)**

Evaluate the wrapped distribution's density.

Return type

float

Parameters**x** (*Any*)**dist_to_encoder()**

Create an encoder backed by the wrapped distribution's encoder.

Return type*dmx.bstats.ignored.IgnoredDataEncoder***entropy()**

Delegate entropy evaluation to the wrapped distribution.

Return type

float

estimator()

Create an estimator that preserves the wrapped distribution.

Return type

dmx.bstats.ignored.IgnoredEstimator

expected_log_density(x)

Evaluate the wrapped distribution's expected log-density.

Return type

float

Parameters

x (*Any*)

get_parameters()

Return parameters from the wrapped fixed distribution.

Return type

typing.Any

get_prior()

Return the wrapped distribution's prior.

Return type

dmx.bstats.pdist.ProbabilityDistribution[typing.Any, typing.Any, typing.Any]

log_density(x)

Evaluate the wrapped distribution's log-density.

Return type

float

Parameters

x (*Any*)

sampler(seed=None)

Create a sampler backed by the wrapped distribution.

Return type

dmx.bstats.ignored.IgnoredSampler

Parameters

seed (*int* | *None*)

seq_encode(x)

Delegate direct sequence encoding to the wrapped distribution.

Return type

typing.Any

Parameters

x (*Iterable*[*Any*])

seq_expected_log_density(x)

Evaluate wrapped expected log-densities for encoded observations.

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Parameters**x** (*Any*)**seq_log_density(x)**

Evaluate wrapped log-densities for encoded observations.

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Parameters**x** (*Any*)**set_parameters(value)**

Set parameters on the wrapped fixed distribution.

Return type

None

Parameters**value** (*Any*)**set_prior(prior)**

Set the wrapped distribution's prior without changing ignored semantics.

Return type

None

Parameters**prior** ([ProbabilityDistribution](#)[*Any*, *Any*, *Any*])**class dmx.bstats.ignored.IgnoredEncodedData(data)**Bases: [EncodedDataSequence](#)[*Any*]

Contain encoded data produced by a fixed distribution's encoder.

Parameters**data** (*Any*)**class dmx.bstats.ignored.IgnoredEstimator(dist=null_dist, prior=null_dist, keys=None)**Bases: [ParameterEstimator](#)[*Any*, *Any*, *Any*, None]

Estimator preserving a wrapped distribution regardless of observations.

Parameters

- **dist** (*Model*)
- **prior** (*Model*)
- **keys** (*Any*)

accumulator_factory()

Create a factory whose accumulators discard all observations.

Return type[dmx.bstats.ignored.IgnoredAccumulatorFactory](#)**estimate(*args)**

Ignore either statistic call form and preserve the fixed distribution.

Return type[dmx.bstats.ignored.IgnoredDistribution](#)

Parameters**args** (*Any*)**get_prior()**

Return the fixed distribution's current prior.

Return type*dmx.bstats.pdist.ProbabilityDistribution*[*typing.Any*, *typing.Any*, *typing.Any*]**set_prior(prior)**

Set the prior of the fixed distribution.

Return type

None

Parameters**prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*])**class** *dmx.bstats.ignored.IgnoredSampler*(*dist*, *seed=None*)Bases: *DistributionSampler*[*Any*]

Sampler delegating all draws to the wrapped fixed distribution.

Parameters

- **dist** (*IgnoredDistribution*)
- **seed** (*Optional[int]*)

sample(size=None)

Draw from the wrapped distribution.

Return type*typing.Any***Parameters****size** (*int* | *None*)

Finite mixture

Finite Bayesian mixtures with ordered, homogeneous components.

Mixture weights are finite, nonnegative, normalized on input, and positionally paired with components. Scalar and sequence scoring use stable log-sum-exp. Posterior responsibilities have shape (K,) for one observation and (N, K) for a sequence. Priors are an ordered composite of the weight prior and component priors; conjugate weight priors supply expected log weights, while alternate priors use plug-in log weights.

class *dmx.bstats.mixture.MixtureDataEncoder*(*encoder*)Bases: *DataSequenceEncoder*[*Any*, *Any*]

Encode homogeneous mixture observations with one child encoder.

Every component must consume this common payload; component-specific encodings are not stored separately.

Parameters**encoder** – Encoder shared by all mixture components.**Parameters****encoder** (*Encoder*)

seq_encode(*x*)

Encode observations and unwrap the child encoder container.

Return type

dmx.bstats.mixture.MixtureEncodedData

Parameters

x (*Iterable*[*Any*])

class *dmx.bstats.mixture.MixtureDistribution*(*components*, *w*, *name=None*, *prior=None*)

Bases: *ProbabilityDistribution*[*Any*, *tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *list*[*Any*]], *Any*]

Model observations with an ordered finite mixture.

All components must accept the same observation and encoded-sequence representation. The length-K weight vector is normalized on input and paired positionally with K components. *posterior* returns a length-K responsibility vector, while *seq_posterior* returns an (N, K) matrix for N encoded observations.

A complete prior is (*weight_prior*, *component_priors*) represented by a nested *CompositeDistribution*. Passing only a weight prior leaves existing component priors unchanged.

Parameters

- **components** – Ordered homogeneous component distributions.
- **w** – Finite nonnegative component weights with at least one positive entry.
- **name** – Optional identifier for the mixture.
- **prior** – Weight prior, or a complete composite prior when set later.

Raises

ValueError – If there are no components, counts differ, or weights are invalid.

Parameters

- **components** (*Sequence*[*Model*])
- **w** (*Sequence*[*float*] | *np.ndarray*[*Any*, *Any*])
- **name** (*str* | *None*)
- **prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

dist_to_encoder()

Create the common component encoder.

Return type

dmx.bstats.mixture.MixtureDataEncoder

estimator()

Create an estimator preserving component order and priors.

Return type

dmx.bstats.mixture.MixtureEstimator

expected_log_density(*x*)

Score using component expectations and expected log weights.

Return type

float

Parameters

x (*Any*)

get_parameters()

Return weights and component parameters in component order.

Return type

`tuple[numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]], list[typing.Any]]`

get_prior()

Return ordered weight and component priors.

Return type

`dmx.bstats.composite.CompositeDistribution`

log_density(x)

Score one observation with stable weighted log-sum-exp.

Return type

`float`

Parameters

x (*Any*)

posterior(x)

Return the length-K responsibility vector for one observation.

Return type

`numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]`

Parameters

x (*Any*)

sampler(seed=None)

Create a repeatable mixture sampler.

Return type

`dmx.bstats.mixture.MixtureSampler`

Parameters

seed (*int* | *None*)

seq_component_log_density(x)

Return an (N, K) matrix of component log scores.

Return type

`numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]`

Parameters

x (*Any*)

seq_encode(x)

Encode observations with the first homogeneous component.

Return type

`typing.Any`

Parameters

x (*Iterable[Any]*)

seq_expected_log_density(x)

Return prior-expected scores for an encoded sequence.

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Parameters

x (*Any*)

seq_log_density(*x*)

Return stable mixture scores for an encoded sequence.

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Parameters

x (*Any*)

seq_posterior(*x*)

Return posterior responsibilities shaped (N, K).

Return type

numpy.ndarray[typing.Any, numpy.dtype[numpy.float64]]

Parameters

x (*Any*)

set_parameters(*value*)

Replace weights and ordered component parameters.

Return type

None

Parameters

value (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *list*[*Any*])

set_prior(*prior*)

Replace the weight prior or propagate a complete composite prior.

Return type

None

Parameters

prior (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

class dmx.bstats.mixture.**MixtureEncodedData**(*data*)

Bases: *EncodedDataSequence*[*Any*]

Contain the common component encoding for a mixture sequence.

Parameters

data (*Any*)

dmx.bstats.mixture.MixtureEncodedDataSequence

alias of *MixtureEncodedData*

class dmx.bstats.mixture.**MixtureEstimator**(*estimators*, *fixed_w*=None, *name*=None, *prior*=default_prior, *keys*=(None, None))

Bases: *ParameterEstimator*[*Any*, *tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *list*[*Any*]], *Any*, *tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *tuple*[*Any*, ...]]

Estimate ordered components and normalized finite-mixture weights.

Each child estimator consumes the statistic at its component position. With a Dirichlet weight prior, weights are posterior modes when that mode has positive mass and posterior means otherwise. Without a conjugate prior, normalized effective counts are used. *fixed_w* overrides both rules while child components are still estimated.

Parameters

- **estimators** – Ordered homogeneous component estimators.
- **fixed_w** – Optional fixed component weights, normalized on input.
- **name** – Optional identifier copied to estimated mixtures.
- **prior** – Prior for the component weights.
- **keys** – Pair (weight_key, component_key) controlling independent sufficient-statistic sharing.

Parameters

- **estimators** (*Sequence[Estimator]*)
- **fixed_w** (*Optional[Sequence[float] | np.ndarray[Any, Any]]*)
- **name** (*Optional[str]*)
- **prior** (*Model*)
- **keys** (*MixtureKeys*)

accumulator_factory()

Create a factory preserving estimator order and sharing keys.

Return type

dmx.bstats.mixture.MixtureEstimatorAccumulatorFactory

estimate(*args)

Estimate components and normalized MAP or empirical weights.

Return type

dmx.bstats.mixture.MixtureDistribution

Parameters

args (*Any*)

get_prior()

Return ordered weight and component estimator priors.

Return type

dmx.bstats.composite.CompositeDistribution

model_log_density(model)

Score mixture and component parameters under the complete prior.

Return type

float

Parameters

model (*MixtureDistribution*)

set_prior(prior)

Replace weights or propagate a complete ordered composite prior.

Return type

None

Parameters

prior (*ProbabilityDistribution[Any, Any, Any]*)

class `dmx.bstats.mixture.MixtureEstimatorAccumulator`(*accumulators*, *keys*=(*None*, *None*))

Bases: [`SequenceEncodableAccumulator`](#)[*Any*, *tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *tuple*[*Any*, ...]], *Any*]

Accumulate counts and ordered child sufficient statistics.

The sufficient statistic is (*component_counts*, *child_statistics*). *component_counts* has shape (K,) and the second item is a length-K tuple in component order. Updates distribute each observation weight according to posterior responsibilities.

The two optional keys independently share the component-count vector and the complete collection of child statistics. Child accumulator keys are also honored.

Parameters

- **accumulators** (*Sequence*[*Accumulator*])
- **keys** (*MixtureKeys*)

acc_to_encoder()

Create the common child accumulator encoder.

Return type

[`dmx.bstats.mixture.MixtureDataEncoder`](#)

combine(*suff_stat*)

Merge counts and child sufficient statistics.

Return type

[`dmx.bstats.mixture.MixtureEstimatorAccumulator`](#)

Parameters

suff_stat (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *tuple*[*Any*, ...])

from_value(*x*)

Restore counts and ordered child statistics.

Return type

[`dmx.bstats.mixture.MixtureEstimatorAccumulator`](#)

Parameters

x (*tuple*[*ndarray*[*Any*, *dtype*[*float64*]], *tuple*[*Any*, ...])

initialize(*x*, *weight*, *rng*)

Randomly split one weight with the legacy uniform Dirichlet.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge mixture blocks and supported child keys.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**key_replace**(*stats_dict*)

Replace mixture blocks and supported child keys.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *rng*)

Randomly split sequence weights over ordered components.

Return type

None

Parameters

- **x** (*Any*)
- **weights** (*ndarray*[*Any*, *dtype*[*float64*]])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Allocate a sequence by posterior responsibility.

Return type

None

Parameters

- **x** (*Any*)
- **weights** (*ndarray*[*Any*, *dtype*[*float64*]])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Allocate one observation by posterior responsibility.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return copied counts and ordered child statistics.

Return typetuple[*numpy.ndarray*[*typing.Any*, *numpy.dtype*[*numpy.float64*]], tuple[*typing.Any*, ...]]

class `dmx.bstats.mixture.MixtureEstimatorAccumulatorFactory`(*factories, dim, keys*)

Bases: `StatisticAccumulatorFactory`[Any, tuple[ndarray[Any, dtype[float64]], tuple[Any, ...]], Any]

Create mixture accumulators from ordered child factories.

Parameters

- **factories** (`Sequence`[`AccumulatorFactory`])
- **dim** (`int`)
- **keys** (`MixtureKeys`)

make()

Create one fresh accumulator per component.

Return type

`dmx.bstats.mixture.MixtureEstimatorAccumulator`

class `dmx.bstats.mixture.MixtureSampler`(*dist, seed=None*)

Bases: `DistributionSampler`[Any]

Draw observations after sampling an ordered component index.

Parameters

- **dist** (`MixtureDistribution`)
- **seed** (`Optional`[`int`])

sample(*size=None*)

Draw one observation or a list of `size` observations.

Return type

`typing.Any`

Parameters

size (`int` | `None`)

Null

Provide the neutral distribution used by Bayesian models.

`NullDistribution` represents the absence of a probabilistic factor or prior. It assigns unit density to every observation, contributes no sufficient statistics, and estimates back to the shared null distribution. Setting its prior or parameters is intentionally a no-op so it can safely fill optional prior and model slots.

class `dmx.bstats.nulldist.NullAccumulator`

Bases: `StatisticAccumulator`[Any, None, `Sequence`[Any]]

Accumulator that deliberately discards every observation.

acc_to_encoder()

Create the encoder corresponding to this accumulator.

Return type

`dmx.bstats.nulldist.NullDataEncoder`

combine(*suff_stat*)

Combine the sole null statistic value.

Return type

`dmx.bstats.nulldist.NullAccumulator`

Parameters**suff_stat** (*None*)**from_value**(*x*)

Restore the sole null sufficient statistic.

Return type*dmx.bstats.nullldist.NullAccumulator***Parameters****x** (*None*)**initialize**(*x*, *weight*, *rng*)

Discard one observation during randomized initialization.

Return type*None***Parameters**

- **x** (*Any*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Leave shared statistics unchanged.

Return type*None***Parameters****stats_dict** (*MutableMapping[str, Any]*)**key_replace**(*stats_dict*)

Leave shared statistics unchanged.

Return type*None***Parameters****stats_dict** (*MutableMapping[str, Any]*)**seq_initialize**(*x*, *weights*, *rng*)

Discard encoded observations during randomized initialization.

Return type*None***Parameters**

- **x** (*Sequence[Any]*)
- **weights** (*ndarray[Any, Any]*)
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Discard a sequence of weighted observations.

Return type*None***Parameters**

- **x** (*Sequence*[Any])
- **weights** (*ndarray*[Any, Any])
- **estimate** (*ProbabilityDistribution*[Any, Any, Any] | None)

update(*x*, *weight*, *estimate*)

Discard one weighted observation.

Return type

None

Parameters

- **x** (Any)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[Any, Any, Any] | None)

value()

Return the sole null sufficient statistic.

Return type

None

class `dmx.bstats.nulldist.NullAccumulatorFactory`

Bases: *StatisticAccumulatorFactory*[Any, None, Sequence[Any]]

Create fresh null accumulators.

make()

Create a null accumulator.

Return type

dmx.bstats.nulldist.NullAccumulator

class `dmx.bstats.nulldist.NullDataEncoder`

Bases: *DataSequenceEncoder*[Any, Sequence[Any]]

Encode observations while retaining only what neutral scoring needs.

seq_encode(*x*)

Materialize observations so sequence result length remains available.

Return type

dmx.bstats.nulldist.NullEncodedData

Parameters

x (*Iterable*[Any])

class `dmx.bstats.nulldist.NullDistribution`

Bases: *ProbabilityDistribution*[Any, None, Sequence[Any]]

Neutral distribution representing no likelihood or prior contribution.

cross_entropy(*dist*)

Return zero because the null factor has no information content.

Return type

float

Parameters**dist** (`ProbabilityDistribution`[*Any*, *Any*, *Any*])**density**(*x*)

Return unit density for every observation.

Return type

float

Parameters**x** (*Any*)**dist_to_encoder**()

Create the length-preserving null encoder.

Return type*dmx.bstats.nulldist.NullDataEncoder***entropy**()

Return zero because the null factor has no information content.

Return type

float

estimator()

Create an estimator that always returns the shared null instance.

Return type*dmx.bstats.nulldist.NullEstimator***expected_log_density**(*x*)

Return zero expected log-density for every observation.

Return type

float

Parameters**x** (*Any*)**get_parameters**()

Return the sole null parameter value.

Return type

None

get_prior()

Return this distribution as its own neutral prior.

Return type*dmx.bstats.nulldist.NullDistribution***log_density**(*x*)

Return zero log-density for every observation.

Return type

float

Parameters**x** (*Any*)

moments(*p*, *o*)

Return the neutral moment used by legacy Bayesian callers.

Return type

float

Parameters

- *p* (Any)
- *o* (Any)

sampler(*seed=None*)

Create a sampler that emits None placeholders.

Return type

dmx.bstats.nulldist.NullSampler

Parameters

seed (int | None)

seq_encode(*x*)

Materialize observations for length-preserving neutral scoring.

Return type

collections.abc.Sequence[*typing.Any*]

Parameters

x (*Iterable*[Any])

seq_expected_log_density(*x*)

Return one zero expected log-density per encoded observation.

Return type

np.ndarray[Any, *np.dtype*[*np.float64*]]

Parameters

x (*NullSequence* | *'NullEncodedData'*)

seq_log_density(*x*)

Return one zero log-density per encoded observation.

Return type

np.ndarray[Any, *np.dtype*[*np.float64*]]

Parameters

x (*NullSequence* | *'NullEncodedData'*)

set_parameters(*value*)

Accept the sole null parameter value without changing state.

Return type

None

Parameters

value (None)

set_prior(*prior*)

Ignore a prior because a null distribution has no parameters.

Return type

None

Parameters**prior** ([ProbabilityDistribution](#)[Any, Any, Any])**class** dmx.bstats.nulldist.**NullEncodedData**(data)Bases: [EncodedDataSequence](#)[Sequence[Any]]

Contain observations used only to preserve null sequence length.

Parameters**data** ([NullSequence](#))**class** dmx.bstats.nulldist.**NullEstimator**(prior=None, keys=None)Bases: [ParameterEstimator](#)[Any, None, Sequence[Any], None]

Estimator that always returns the shared neutral distribution.

Parameters

- **prior** ([Optional](#)[[Model](#)])
- **keys** ([Any](#))

accumulator_factory()

Create a factory for no-op accumulators.

Return type[dmx.bstats.nulldist.NullAccumulatorFactory](#)**estimate**(*args)

Ignore either supported statistic call form and return the null model.

Return type[dmx.bstats.nulldist.NullDistribution](#)**Parameters****args** ([Any](#))**get_prior**()

Return the shared neutral prior.

Return type[dmx.bstats.nulldist.NullDistribution](#)**set_prior**(prior)

Ignore prior replacement because this estimator is parameter-free.

Return type

None

Parameters**prior** ([ProbabilityDistribution](#)[Any, Any, Any])**class** dmx.bstats.nulldist.**NullSampler**(dist=None, seed=None)Bases: [DistributionSampler](#)[None]

Sampler returning None because a null factor has no observation model.

Parameters

- **dist** ([Optional](#)[[NullDistribution](#)])
- **seed** ([Optional](#)[int])

sample(*size=None*)

Return one or more None placeholders.

Return type

`typing.Any`

Parameters

size (*int* | *None*)

Optional

Bayesian wrapper for observations that may be missing.

Missingness keeps the legacy identity policy: the marker matches with `is`; when it is NaN, any scalar NaN also matches. The composite prior contains the missingness prior followed by the child prior. Sufficient statistics are (`missing_weight`, `present_weight`, `child_statistics`); only present values reach the child distribution and accumulator.

class `dmx.bstats.optional.OptionalDataEncoder`(*encoder*, *missing_value*)

Bases: `DataSequenceEncoder`[*Any*, *tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*]]

Encode optional observations with a child encoder.

For `n` observations the payload is (`n`, `present_indices`, `missing_indices`, `encoded_present_values`). Both index arrays are one-dimensional, and the child encoder sees only present values.

Parameters

- **encoder** – Encoder for present observations.
- **missing_value** – Marker representing the missing outcome.

Parameters

- **encoder** (`DataSequenceEncoder`[*Any*, *Any*])
- **missing_value** (*Any*)

seq_encode(*x*)

Partition observations and encode present values.

Return type

`dmx.bstats.optional.OptionalEncodedData`

Parameters

x (*Iterable*[*Any*])

class `dmx.bstats.optional.OptionalDistribution`(*dist*, *p=0.5*, *missing_value=None*, *name=None*, *prior=default_prior*, *keys=None*)

Bases: `ProbabilityDistribution`[*Any*, *tuple*[*float*, *Any*], *tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*]]

Add an explicit missing outcome to a child distribution.

`p` is the probability of the missing outcome; a present observation has probability $1 - p$ times its child density. Missing values match the configured marker by identity, except that a scalar NaN marker matches any scalar NaN. The parameter tuple is (`p`, `child_parameters`) and the prior is (`missingness_prior`, `child_prior`).

Parameters

- **dist** – Distribution for present observations.
- **p** – Probability that an observation is missing.

- **missing_value** – Marker representing the missing outcome.
- **name** – Optional identifier for the wrapper.
- **prior** – Prior for the missing probability.
- **keys** – Optional key sharing the complete optional sufficient statistic.

Raises

ValueError – If p is not finite or is outside $[0, 1]$.

Parameters

- **dist** (*Model*)
- **p** (*float*)
- **missing_value** (*Any*)
- **name** (*str* | *None*)
- **prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*])
- **keys** (*Optional*[*str*])

cross_entropy(dist)

Return cross-entropy over missing and present outcomes.

Return type

float

Parameters

dist (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

dist_to_encoder()

Create an encoder retaining marker and child encoding.

Return type

dmx.bstats.optional.OptionalDataEncoder

entropy()

Return binary missingness entropy plus weighted child entropy.

Return type

float

estimator()

Create an estimator retaining the child estimator and prior.

Return type

dmx.bstats.optional.OptionalEstimator

expected_log_density(x)

Score under beta and child priors when available.

Return type

float

Parameters

x (*Any*)

get_data_type()

Return the permissive legacy observation type.

Return type
typing.Any

get_parameters()

Return missing probability and child parameters.

Return type
tuple[float, typing.Any]

get_prior()

Return missingness and child priors in that order.

Return type
dmx.bstats.composite.CompositeDistribution

log_density(x)

Score a missing marker or present child observation.

Return type
float

Parameters
x (Any)

sampler(seed=None)

Create a repeatable optional sampler.

Return type
dmx.bstats.optional.OptionalSampler

Parameters
seed (int | None)

seq_encode(x)

Partition observations and encode only present child values.

Return type
tuple[int, numpy.ndarray[typing.Any, typing.Any], numpy.ndarray[typing.Any, typing.Any], typing.Any]

Parameters
x (Iterable[Any])

seq_expected_log_density(x)

Score encoded observations under configured priors.

Return type
numpy.ndarray[typing.Any, typing.Any]

Parameters
x (tuple[int, ndarray[Any, Any], ndarray[Any, Any], Any])

seq_log_density(x)

Score encoded observations, evaluating only present values.

Return type
numpy.ndarray[typing.Any, typing.Any]

Parameters
x (tuple[int, ndarray[Any, Any], ndarray[Any, Any], Any])

set_parameters(*value*)

Set missing probability and child parameters.

Return type

None

Parameters

value (*tuple*[*float*, *Any*])

set_prior(*prior*)

Propagate a two-part composite prior to this wrapper and child.

Return type

None

Parameters

prior (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

class dmx.bstats.optional.**OptionalEncodedData**(*data*)

Bases: *EncodedDataSequence*[*tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*]

Contain the stable four-part optional encoding.

Parameters

data (*E*)

class dmx.bstats.optional.**OptionalEstimator**(*estimator*, *missing_value*=*None*, *fixed_prob*=*None*,
name=*None*, *keys*=*None*, *prior*=*default_prior*)

Bases: *ParameterEstimator*[*Any*, *tuple*[*float*, *Any*], *tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*], *tuple*[*float*, *float*, *Any*]]

Estimate missingness and child parameters.

A beta prior produces a posterior-mode missing probability and is updated to the beta posterior stored on the result. Other priors use the empirical missing fraction. *fixed_prob* overrides either probability calculation without preventing child estimation.

Parameters

- **estimator** – Estimator for present observations.
- **missing_value** – Marker representing the missing outcome.
- **fixed_prob** – Optional fixed probability of missingness.
- **name** – Optional identifier copied to estimated distributions.
- **keys** – Optional key sharing the complete optional statistic.
- **prior** – Prior for the missing probability.

Parameters

- **estimator** (*Estimator*)
- **missing_value** (*Any*)
- **fixed_prob** (*Optional*[*float*])
- **name** (*Optional*[*str*])
- **keys** (*Optional*[*str*])
- **prior** (*Model*)

accumulator_factory()

Create a compatible accumulator factory.

Return type

dmx.bstats.optional.OptionalEstimatorAccumulatorFactory

estimate(*args)

Estimate from optional sufficient statistics.

Return type

dmx.bstats.optional.OptionalDistribution

Parameters

args (*Any*)

get_prior()

Return missingness and child priors.

Return type

dmx.bstats.composite.CompositeDistribution

set_prior(prior)

Propagate missingness and child priors.

Return type

None

Parameters

prior (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

class *dmx.bstats.optional.OptionalEstimatorAccumulator*(*accumulator*, *missing_value*, *name*, *keys*)

Bases: *SequenceEncodableAccumulator*[*Any*, *tuple*[*float*, *float*, *Any*], *tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*]

Accumulate missing/present weights and child statistics.

The sufficient statistic is (*missing_weight*, *present_weight*, *child_statistics*). Only present observations reach the child accumulator. The optional wrapper key shares this complete tuple, after which sharing configured by the child accumulator is also applied.

Parameters

- **accumulator** (*Accumulator*)
- **missing_value** (*Any*)
- **name** (*Optional*[*str*])
- **keys** (*Optional*[*str*])

acc_to_encoder()

Create the corresponding optional encoder.

Return type

dmx.bstats.optional.OptionalDataEncoder

combine(suff_stat)

Merge wrapper and child statistics.

Return type

dmx.bstats.optional.OptionalEstimatorAccumulator

Parameters**suff_stat** (*tuple*[*float*, *float*, *Any*])**from_value**(*x*)

Restore missing, present, and child statistics.

Return type*dmx.bstats.optional.OptionalEstimatorAccumulator***Parameters****x** (*tuple*[*float*, *float*, *Any*])**initialize**(*x*, *weight*, *rng*)

Initialize from one observation.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge configured wrapper and child keys.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**key_replace**(*stats_dict*)

Replace configured wrapper and child keys.

Return type

None

Parameters**stats_dict** (*MutableMapping*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *rng*)

Initialize from encoded observations.

Return type

None

Parameters

- **x** (*tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*)
- **weights** (*ndarray*[*Any*, *Any*])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Update from encoded observations.

Return type

None

Parameters

- **x** (*tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*)
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Update from one observation.

Return type

None

Parameters

- **x** (*Any*)
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return missing, present, and child statistics.

Return type

tuple[*float*, *float*, *typing.Any*]

class *dmx.bstats.optional.OptionalEstimatorAccumulatorFactory*(*acc_factory*, *missing_value*, *name*, *keys*)

Bases: *StatisticAccumulatorFactory*[*Any*, *tuple*[*float*, *float*, *Any*], *tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*]

Create optional accumulators around child accumulators.

Parameters

- **acc_factory** (*AccumulatorFactory*)
- **missing_value** (*Any*)
- **name** (*Optional*[*str*])
- **keys** (*Optional*[*str*])

make()

Create an empty optional accumulator.

Return type

dmx.bstats.optional.OptionalEstimatorAccumulator

class *dmx.bstats.optional.OptionalSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*[*Any*]

Draw missing markers or child observations.

Parameters

- **dist** (*OptionalDistribution*)
- **seed** (*Optional*[*int*])

sample(*size=None*)

Draw one optional observation or a list.

Return type

typing.Any

Parameters**size** (*int* / *None*)**Sequence**

Bayesian distribution for variable-length iid sequences.

An observation is a list whose elements are scored independently by `dist`; its length is scored by `len_dist`. Empty and unequal-length observations are valid. With `len_normalized=True` child log scores and child sufficient statistics are divided by the nonzero sequence length, while the length factor keeps its original weight. The composite prior and sufficient statistic both store child information first and length information second.

class `dmx.bstats.sequence.SequenceDataEncoder`(*encoder, len_encoder*)

Bases: `DataSequenceEncoder`[*list*[*Any*], *tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*, *Any*]]

Encode flattened child values and sequence lengths.

For *n* sequences the payload is (*indices, inverse_lengths, nonempty, child_data, length_data*). *indices* maps every flattened child value to its source row. The next two arrays have shape (*n*,); empty rows have inverse length zero. *length_data* is *None* without a length encoder.

Parameters

- **encoder** – Encoder shared by flattened child values.
- **len_encoder** – Encoder for the *n* sequence lengths, or *None*.

Parameters

- **encoder** (`DataSequenceEncoder`[*Any*, *Any*])
- **len_encoder** (*Optional*[`DataSequenceEncoder`[*Any*, *Any*]])

seq_encode(*x*)

Flatten lists and encode child values and lengths.

Return type

`dmx.bstats.sequence.SequenceEncodedData`

Parameters

x (*Iterable*[*list*[*Any*]])

class `dmx.bstats.sequence.SequenceDistribution`(*dist, len_dist=null_dist, name=None, len_normalized=False*)

Bases: `ProbabilityDistribution`[*list*[*Any*], *tuple*[*Any*, *Any*], *tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*, *Any*]]

Model a variable-length list of iid child observations.

Each list element is scored by `dist` and the integer list length by `len_dist`. A *None* or null length distribution contributes no length score. With `len_normalized=True`, nonempty sequences use the mean child log score and distribute their weight evenly over child statistics; the length contribution is never normalized.

Parameters

- **dist** – Distribution shared by all sequence elements.
- **len_dist** – Distribution for nonnegative integer sequence lengths.
- **name** – Optional identifier for the sequence model.
- **len_normalized** – Whether to average child contributions over each nonempty sequence.

Parameters

- **dist** (*Model*)
- **len_dist** (*Optional[Model]*)
- **name** (*str | None*)
- **len_normalized** (*bool*)

cross_entropy(dist)

Return sequence cross-entropy when the mean length is available.

Return type

float

Parameters

dist (*ProbabilityDistribution[Any, Any, Any]*)

dist_to_encoder()

Create an encoder combining child and length encoders.

Return type

dmx.bstats.sequence.SequenceDataEncoder

entropy()

Return sequence entropy when the mean length is available.

Return type

float

estimator()

Create an estimator retaining normalization and child estimators.

Return type

dmx.bstats.sequence.SequenceEstimator

expected_log_density(x)

Sum prior-expected child and length scores.

Return type

float

Parameters

x (*list[Any]*)

get_parameters()

Return child and length parameters.

Return type

tuple[typing.Any, typing.Any]

get_prior()

Return child and length priors in that order.

Return type

dmx.bstats.composite.CompositeDistribution

log_density(x)

Sum child scores and the score of the observed length.

Return type

float

Parameters**x** (*list* [*Any*])**sampler** (*seed*=*None*)

Create a repeatable sequence sampler.

Return type*dmx.bstats.sequence.SequenceSampler***Parameters****seed** (*int* | *None*)**seq_encode** (*x*)

Flatten variable-length lists while retaining grouping and lengths.

Return type*tuple*[*numpy.ndarray*[*typing.Any*, *typing.Any*], *numpy.ndarray*[*typing.Any*, *typing.Any*], *numpy.ndarray*[*typing.Any*, *typing.Any*], *typing.Any*, *typing.Any*]**Parameters****x** (*Iterable*[*list* [*Any*]])**seq_expected_log_density** (*x*)

Score encoded sequences under child and length priors.

Return type*numpy.ndarray*[*typing.Any*, *typing.Any*]**Parameters****x** (*tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*, *Any*])**seq_log_density** (*x*)

Score flattened encoded sequences and regroup by observation.

Return type*numpy.ndarray*[*typing.Any*, *typing.Any*]**Parameters****x** (*tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*, *Any*])**set_parameters** (*value*)

Set child and length parameters.

Return type*None***Parameters****value** (*tuple*[*Any*, *Any*])**set_prior** (*prior*)

Propagate a two-part composite prior to both child models.

Return type*None***Parameters****prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*])

class `dmx.bstats.sequence.SequenceEncodedData`(*data*)

Bases: `EncodedDataSequence`[`tuple`[`ndarray`[`Any`, `Any`], `ndarray`[`Any`, `Any`], `ndarray`[`Any`, `Any`], `Any`, `Any`]]

Contain the stable five-part variable-sequence encoding.

Parameters

data (*E*)

class `dmx.bstats.sequence.SequenceEstimator`(*estimator*, *len_estimator*=*null_estimator*,
len_normalized=*False*, *name*=*None*, *keys*=(*None*, *None*))

Bases: `ParameterEstimator`[`list`[`Any`], `tuple`[`Any`, `Any`], `tuple`[`ndarray`[`Any`, `Any`], `ndarray`[`Any`, `Any`], `ndarray`[`Any`, `Any`], `Any`, `Any`], `tuple`[`Any`, `Any`]]

Estimate child and optional length models.

The estimator consumes (`child_statistics`, `length_statistics`) and preserves the distribution's child-first prior order. When `len_normalized` is enabled, each nonempty input sequence contributes its total observation weight, rather than that weight once per element, to child estimation.

Parameters

- **estimator** – Estimator shared by all sequence elements.
- **len_estimator** – Estimator for sequence lengths, or `None`.
- **len_normalized** – Whether to normalize child sufficient-statistic weights by nonzero sequence length.
- **name** – Optional identifier copied to estimated sequence models.
- **keys** – Optional keys that each share the complete paired statistic.

Parameters

- **estimator** (*Estimator*)
- **len_estimator** (*Optional*[*Estimator*])
- **len_normalized** (*bool*)
- **name** (*Optional*[*str*])
- **keys** (*tuple*[*Optional*[*str*], *Optional*[*str*]])

accumulator_factory()

Create a factory combining child and length factories.

Return type

`dmx.bstats.sequence.SequenceEstimatorAccumulatorFactory`

estimate(**args*)

Estimate child and optional length models from paired statistics.

Return type

`dmx.bstats.sequence.SequenceDistribution`

Parameters

args (*Any*)

get_prior()

Return child and length priors.

Return type

`dmx.bstats.composite.CompositeDistribution`

model_log_density(*model*)

Score model parameters under this estimator's composite prior.

Return type

float

Parameters

model ([CompositeDistribution](#))

set_prior(*prior*)

Propagate a two-part prior to child and length estimators.

Return type

None

Parameters

prior ([ProbabilityDistribution](#)[Any, Any, Any])

class dmx.bstats.sequence.**SequenceEstimatorAccumulator**(*accumulator*, *len_normalized*,
len_accumulator, *keys*)

Bases: [SequenceEncodableAccumulator](#)[list[Any], tuple[Any, Any], tuple[ndarray[Any, Any], ndarray[Any, Any], ndarray[Any, Any], Any, Any]]

Accumulate flattened child statistics and length statistics.

The sufficient statistic is (*child_statistics*, *length_statistics*); the second item is None when no length accumulator is configured. Each non-None wrapper key aliases this complete pair, and child and length accumulator keys are additionally merged and replaced.

Parameters

- **accumulator** (*Accumulator*)
- **len_normalized** (*bool*)
- **len_accumulator** (*Optional*[*Accumulator*])
- **keys** (*tuple*[*Optional*[*str*], *Optional*[*str*]])

acc_to_encoder()

Create the corresponding child and length encoder.

Return type

[dmx.bstats.sequence.SequenceDataEncoder](#)

combine(*suff_stat*)

Merge child and length sufficient statistics.

Return type

[dmx.bstats.sequence.SequenceEstimatorAccumulator](#)

Parameters

suff_stat (*tuple*[Any, Any])

from_value(*x*)

Restore child and length sufficient statistics.

Return type

[dmx.bstats.sequence.SequenceEstimatorAccumulator](#)

Parameters

x (*tuple*[Any, Any])

initialize(*x*, *weight*, *rng*)

Initialize child elements and sequence length.

Return type

None

Parameters

- **x** (*list*[*Any*])
- **weight** (*float*)
- **rng** (*RandomState*)

key_merge(*stats_dict*)

Merge whole sequence statistics and configured child keys.

Return type

None

Parameters

stats_dict (*MutableMapping*[*str*, *Any*])

key_replace(*stats_dict*)

Replace whole sequence statistics and configured child keys.

Return type

None

Parameters

stats_dict (*MutableMapping*[*str*, *Any*])

seq_initialize(*x*, *weights*, *rng*)

Initialize from flattened encoded sequences.

Return type

None

Parameters

- **x** (*tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*, *Any*)
- **weights** (*ndarray*[*Any*, *Any*])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Update from flattened encoded sequences.

Return type

None

Parameters

- **x** (*tuple*[*ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*, *Any*)
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Update child elements and sequence length.

Return type

None

Parameters

- **x** (*list*[*Any*])
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return child and length sufficient statistics.

Return type

tuple[*typing.Any*, *typing.Any*]

class *dmx.bstats.sequence.SequenceEstimatorAccumulatorFactory*(*factory*, *len_normalized*,
len_factory, *keys*)

Bases: *StatisticAccumulatorFactory*[*list*[*Any*], *tuple*[*Any*, *Any*], *tuple*[*ndarray*[*Any*, *Any*],
ndarray[*Any*, *Any*], *ndarray*[*Any*, *Any*], *Any*, *Any*]

Create sequence accumulators from child factories.

Parameters

- **factory** (*AccumulatorFactory*)
- **len_normalized** (*bool*)
- **len_factory** (*Optional*[*AccumulatorFactory*])
- **keys** (*tuple*[*Optional*[*str*], *Optional*[*str*]])

make()

Create an empty sequence accumulator.

Return type

dmx.bstats.sequence.SequenceEstimatorAccumulator

class *dmx.bstats.sequence.SequenceSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*[*list*[*Any*]]

Draw a length followed by that many iid child observations.

Parameters

- **dist** (*SequenceDistribution*)
- **seed** (*Optional*[*int*])

sample(*size=None*)

Draw one variable-length list or a list of lists.

Return type

typing.Any

Parameters

size (*int* | *None*)

Bernoulli set

Independent-Bernoulli models for set-like observations.

Observation order does not affect scoring or sufficient statistics. For compatibility, repeated labels are not deduplicated: every occurrence contributes again. Statistics are (`weighted_label_occurrences`, `observation_weight`).

class `dmx.bstats.setdist.BernoulliSetAccumulator`

Bases: `SequenceEncodableAccumulator`[`Iterable`[`Hashable`], `tuple`[`dict`[`Hashable`, `float`], `float`], `tuple`[`int`, `ndarray`[`Any`, `Any`], `ndarray`[`Any`, `Any`], `ndarray`[`Any`, `Any`]]]

Accumulate weighted label occurrences and observation weight.

The sufficient statistic is (`label_occurrences`, `observation_weight`). Every repeated occurrence adds its observation's weight again; observations are not converted to mathematical sets before accumulation.

acc_to_encoder()

Create the corresponding encoder.

Return type

`dmx.bstats.setdist.BernoulliSetDataEncoder`

combine(`suff_stat`)

Merge occurrence counts and total observation weight.

Return type

`dmx.bstats.setdist.BernoulliSetAccumulator`

Parameters

suff_stat (`tuple`[`dict`[`Hashable`, `float`], `float`])

from_value(`x`)

Restore occurrence counts and total observation weight.

Return type

`dmx.bstats.setdist.BernoulliSetAccumulator`

Parameters

x (`tuple`[`dict`[`Hashable`, `float`], `float`])

initialize(`x`, `weight`, `rng`)

Initialize from one observation.

Return type

`None`

Parameters

- **x** (`Iterable`[`Hashable`])
- **weight** (`float`)
- **rng** (`RandomState`)

key_merge(`stats_dict`)

Leave shared statistics unchanged because this model has no key.

Return type

`None`

Parameters

stats_dict (`MutableMapping`[`str`, `Any`])

key_replace(*stats_dict*)

Leave shared statistics unchanged because this model has no key.

Return type

None

Parameters

stats_dict (*MutableMapping*[*str*, *Any*])

seq_initialize(*x*, *weights*, *rng*)

Initialize from encoded observations.

Return type

None

Parameters

- **x** (*tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **rng** (*RandomState*)

seq_update(*x*, *weights*, *estimate*)

Add weighted label occurrences from encoded observations.

Return type

None

Parameters

- **x** (*tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]])
- **weights** (*ndarray*[*Any*, *Any*])
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

update(*x*, *weight*, *estimate*)

Add one weighted set-like observation.

Return type

None

Parameters

- **x** (*Iterable*[*Hashable*])
- **weight** (*float*)
- **estimate** (*ProbabilityDistribution*[*Any*, *Any*, *Any*] | *None*)

value()

Return occurrence counts and total observation weight.

Return type

tuple[*dict*[*collections.abc.Hashable*, *float*], *float*]

class dmx.bstats.setdist.**BernoulliSetAccumulatorFactory**

Bases: *StatisticAccumulatorFactory*[*Iterable*[*Hashable*], *tuple*[*dict*[*Hashable*, *float*], *float*], *tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]]]

Create empty Bernoulli-set accumulators.

make()

Create an empty accumulator.

Return type

dmx.bstats.setdist.BernoulliSetAccumulator

class dmx.bstats.setdist.BernoulliSetDataEncoder

Bases: *DataSequenceEncoder*[Iterable[Hashable], tuple[int, ndarray[Any, Any], ndarray[Any, Any], ndarray[Any, Any]]]

Encode set-like observations into a flattened four-part tuple.

For *n* observations the payload is (*n*, *row_indices*, *levels*, *inverse*). *row_indices* maps each flattened occurrence to its source observation, *levels* contains the distinct labels, and *inverse* maps occurrences to levels. Repeated labels remain repeated occurrences.

seq_encode(*x*)

Flatten observations while retaining row and level indices.

Return type

dmx.bstats.setdist.BernoulliSetEncodedData

Parameters

x (Iterable[Iterable[Hashable]])

class dmx.bstats.setdist.BernoulliSetDistribution(*pmap*, *name*=None, *prior*=None)

Bases: *ProbabilityDistribution*[Iterable[Hashable], dict[Hashable, float], tuple[int, ndarray[Any, Any], ndarray[Any, Any], ndarray[Any, Any]]]

Model inclusion of each known label independently.

Observations are iterables of labels from *pmap*. Every known label is treated as an independent Bernoulli outcome; absent labels contribute exclusion mass. Order is irrelevant, but repeated labels are deliberately counted repeatedly for legacy compatibility.

Nonnegative map values are inclusion probabilities. A negative value is the legacy complement encoding for a probability near one: its magnitude is the exclusion probability. The common prior applies independently to every observed label.

Parameters

- **pmap** – Mapping from known labels to encoded inclusion probabilities.
- **name** – Optional identifier for the distribution.
- **prior** – Common prior for every label's inclusion probability.

Parameters

- **pmap** (*dict*[Label, float])
- **name** (*str* | None)
- **prior** (*ProbabilityDistribution*[Any, Any, Any])

dist_to_encoder()

Create the set-like sequence encoder.

Return type

dmx.bstats.setdist.BernoulliSetDataEncoder

estimator()

Create an estimator retaining metadata and prior.

Return type

dmx.bstats.setdist.BernoulliSetEstimator

get_parameters()

Return the legacy encoded inclusion probabilities.

Return type

`dict[collections.abc.Hashable, float]`

get_prior()

Return the common inclusion-probability prior.

Return type

dmx.bstats.pdist.ProbabilityDistribution[typing.Any, typing.Any, typing.Any]

log_density(*x*)

Score an order-insensitive iterable of known labels.

Return type

`float`

Parameters

x (*Iterable*[*Hashable*])

sampler(*seed=None*)

Create a repeatable inclusion sampler.

Return type

dmx.bstats.setdist.BernoulliSetSampler

Parameters

seed (*int* | *None*)

seq_encode(*x*)

Flatten observations while retaining row and unique-level indices.

Return type

`tuple[int, numpy.ndarray[typing.Any, typing.Any], numpy.ndarray[typing.Any, typing.Any], numpy.ndarray[typing.Any, typing.Any]]`

Parameters

x (*Iterable*[*Iterable*[*Hashable*]])

seq_log_density(*x*)

Score flattened observations and regroup their label occurrences.

Return type

`numpy.ndarray[typing.Any, typing.Any]`

Parameters

x (*tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]])

set_parameters(*value*)

Replace probabilities and refresh cached log masses.

Nonnegative values encode inclusion directly. Negative values retain the legacy complement encoding used for probabilities near one.

Return type

None

Parameters**value** (*dict*[*Hashable*, *float*])**set_prior**(*prior*)

Replace the common inclusion-probability prior.

Return type

None

Parameters**prior** (*ProbabilityDistribution*[*Any*, *Any*, *Any*])**class** dmx.bstats.setdist.**BernoulliSetEncodedData**(*data*)Bases: *EncodedDataSequence*[*tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]]]

Contain the stable flattened set-like encoding.

Parameters**data** (*E*)**class** dmx.bstats.setdist.**BernoulliSetEstimator**(*name=None*, *prior=default_prior*, *keys=(None,)*)Bases: *ParameterEstimator*[*Iterable*[*Hashable*], *dict*[*Hashable*, *float*], *tuple*[*int*, *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*], *ndarray*[*Any*, *Any*]], *tuple*[*dict*[*Hashable*, *float*], *float*]]

Estimate per-label inclusion probabilities.

A beta prior yields per-label posterior modes. A finite mixture of beta distributions selects the highest-posterior prior component separately for each label before taking its mode. Other priors use empirical inclusion frequencies. Results retain the legacy negative complement encoding when appropriate.

Parameters

- **name** – Optional identifier copied to the estimated distribution.
- **prior** – Common prior for every label’s inclusion probability.
- **keys** – Compatibility placeholder; this estimator does not share sufficient statistics by key.

Parameters

- **name** (*Optional*[*str*])
- **prior** (*Model*)
- **keys** (*tuple*[*None*])

accumulator_factory()

Create a compatible accumulator factory.

Return type*dmx.bstats.setdist.BernoulliSetAccumulatorFactory***estimate**(**args*)

Estimate from label-occurrence and observation counts.

Return type*dmx.bstats.setdist.BernoulliSetDistribution***Parameters****args** (*Any*)

get_prior()

Return the common inclusion-probability prior.

Return type

`dmx.bstats.pdist.ProbabilityDistribution`[typing.Any, typing.Any, typing.Any]

set_prior(prior)

Replace the common inclusion-probability prior.

Return type

None

Parameters

prior (`ProbabilityDistribution`[Any, Any, Any])

class `dmx.bstats.setdist.BernoulliSetSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`[Iterable[Hashable]]

Sample included labels in probability-map insertion order.

Parameters

- **dist** (Any)
- **seed** (int | None)

sample(size=None)

Draw one label list or a list of label lists.

Return type

typing.Any

Parameters

size (int | None)

`dmx.bstats.setdist.bernoulli_beta_posterior_mode`(*obs_cnt*, *tot_cnt*, *beta_params*)

Return legacy encoded beta-posterior modes.

Return type

dict[collections.abc.Hashable, float]

Parameters

- **obs_cnt** (dict[Hashable, float])
- **tot_cnt** (float)
- **beta_params** (tuple[float, float])

`dmx.bstats.setdist.bernoulli_betamix_posterior_mode`(*obs_cnt*, *tot_cnt*, *weights*, *beta_params*)

Return legacy encoded modes under a beta-mixture prior.

Return type

dict[collections.abc.Hashable, float]

Parameters

- **obs_cnt** (dict[Hashable, float])
- **tot_cnt** (float)
- **weights** (ndarray[Any, Any])
- **beta_params** (ndarray[Any, Any])

1.5 dmx.torch_stats API

PyTorch-backed probability models and estimation interfaces.

This optional backend mirrors a focused subset of `dmx.stats` with tensor encodings, explicit device placement, and PyTorch-based likelihood and fitting operations. The package root re-exports the supported distributions, estimators, core protocols, and local or `torch.distributed` sequence helpers. Import implementation-specific classes from their `dmx.torch_stats` submodules only when they are not re-exported here.

Use `dmx.stats` for the broader NumPy/SciPy non-Bayesian model catalog and `dmx.bstats` for prior-aware Bayesian or variational modeling. Their model and encoded-sequence protocols cannot be mixed with this backend. PyTorch is an optional dependency: when unavailable, importing this package preserves its exported names as placeholders that raise an installation-focused error when called. Device-oriented workflow utilities live in `dmx.torch_utils`.

This optional backend requires PyTorch. Its models use tensor-specific protocols and device semantics; see [Installation](#) for installation options rather than treating the NumPy and torch backends as interchangeable.

1.5.1 Torch Protocols

Abstract classes for PyTorch-based probability distributions.

This module provides torch-specific counterparts to the `dmx.stats.pdist` protocols. They keep the same distribution, estimator, accumulator, and encoder terminology while accepting torch tensors for encoded sequence work.

Devices are explicit: `device=None` resolves to CPU, and callers may select CPU, CUDA, or MPS where PyTorch supports the requested operation. A model's `to` method records its preferred device; it does not promise that arbitrary Python-valued parameters or externally supplied encoded data are moved.

The public classes cover probability distributions, samplers, statistic accumulators and factories, parameter estimators, sequence encoders, and encoded-sequence containers.

class `dmx.torch_stats.pdist.ConditionalSampler`

Bases: object

Abstract base class for conditional distribution samplers.

Samplers that generate samples conditioned on input values.

abstractmethod `sample_given(x)`

Generate a sample conditioned on x.

Parameters

x – Conditioning value.

Return type

`typing.Any`

Returns

Generated sample conditioned on x.

Parameters

x (*Any*)

class `dmx.torch_stats.pdist.DistributionSampler`

Bases: object

Abstract base class for distribution samplers.

Samplers generate random samples from probability distributions.

abstractmethod `sample(size=None)`

Generate random samples from the distribution.

Parameters

size – Number of samples to generate. Defaults to None (single sample).

Return type

`typing.Any`

Returns

Generated samples.

Parameters

size (`int` | `None`)

class `dmx.torch_stats.pdist.TorchEncodedSequence(data, device=None)`

Bases: `object`

Container for encoded data sequences with PyTorch support.

Stores encoded data and tracks the device requested by the encoder.

The container does not coerce dtype and does not itself move data; concrete encoders construct tensors with their documented dtype and device.

Parameters

- **data** (`Any`)
- **device** (`torch.device` | `None`)

data

The encoded data.

device

PyTorch device where data resides.

class `dmx.torch_stats.pdist.TorchParameterEstimator`

Bases: `Generic[SS]`

Abstract base class for PyTorch-based parameter estimators.

Estimators compute distribution parameters from sufficient statistics using PyTorch for GPU-accelerated computation.

Type Parameters:

SS: Type of the sufficient statistics.

abstractmethod `accumulator_factory()`

Return the factory for creating compatible accumulators.

Return type

`dmx.torch_stats.pdist.TorchStatisticAccumulatorFactory`

Returns

A `TorchStatisticAccumulatorFactory` instance.

abstractmethod `estimate(nobs, suff_stat, device=None)`

Estimate distribution parameters from sufficient statistics.

Parameters

- **nobs** – Number of observations.

- **suff_stat** – Sufficient statistics.
- **device** – Target PyTorch device. Defaults to None.

Return type

dmx.torch_stats.pdist.TorchProbabilityDistribution

Returns

Estimated probability distribution.

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*SS*)
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.pdist.TorchProbabilityDistribution(device=None)`

Bases: object

Abstract base class for PyTorch-based probability distributions.

This class provides the interface for torch-based probability distributions that support GPU computation and PyTorch tensor operations.

Parameters

device (*str* | *torch.device* | *None*)

_device

The torch device (CPU or CUDA) for computations.

abstractmethod density(x)

Compute the probability density at x.

Parameters

x – Input value to evaluate density.

Return type

float

Returns

Probability density value.

Parameters

x (*Any*)

abstractmethod dist_to_encoder()

Create a sequence encoder for this distribution.

Return type

dmx.torch_stats.pdist.TorchSequenceEncoder

Returns

A TorchSequenceEncoder instance.

abstractmethod estimator(pseudo_count=None)

Create a parameter estimator for this distribution.

Parameters

pseudo_count – Regularization parameter. Defaults to None.

Return type

dmx.torch_stats.pdist.TorchParameterEstimator

Returns

A TorchParameterEstimator instance.

Parameters

pseudo_count (*float* | *None*)

abstractmethod log_density(*x*)

Compute the log probability density at *x*.

Parameters

x – Input value to evaluate log density.

Return type

float

Returns

Log probability density value.

Parameters

x (*Any*)

model_device()

Return the device used for model computations.

Return type

torch.device

Returns

The PyTorch device object.

abstractmethod sampler(*seed=None*)

Create a sampler for this distribution.

Parameters

seed – Random seed for sampling. Defaults to None.

Return type

dmx.torch_stats.pdist.DistributionSampler

Returns

A DistributionSampler instance.

Parameters

seed (*int* | *None*)

abstractmethod seq_log_density(*x*)

Compute log densities for a sequence of values.

Parameters

x – Sequence of input values.

Return type

torch.Tensor

Returns

Tensor of log density values.

Parameters

x (*Any*)

abstractmethod to(*device*)

Move the distribution to a specified device.

Parameters

device – Target PyTorch device.

Return type

dmx.torch_stats.pdist.TorchProbabilityDistribution

Returns

Distribution instance on the target device.

Parameters

device (*str* / *torch.device* / *None*)

class *dmx.torch_stats.pdist.TorchSequenceEncoder*

Bases: object

Abstract base class for PyTorch-based sequence encoders.

Encoders transform raw data sequences into encoded representations suitable for PyTorch-based processing.

abstractmethod **seq_encode** (*x*, *device=None*)

Encode a data sequence.

Parameters

- **x** – Input data sequence.
- **device** – Target PyTorch device. Defaults to None.

Return type

dmx.torch_stats.pdist.TorchEncodedSequence

Returns

Encoded sequence.

Parameters

- **x** (*Any*)
- **device** (*torch.device* / *None*)

class *dmx.torch_stats.pdist.TorchStatisticAccumulator* (*device=None*)

Bases: Generic[SS]

Abstract base class for PyTorch-based sufficient statistic accumulators.

Accumulators maintain and update sufficient statistics for parameter estimation using PyTorch tensors for GPU-accelerated computation.

Type Parameters:

SS: Type of the sufficient statistics.

Parameters

device (*str* / *torch.device* / *None*)

_device

The torch device for computations.

abstractmethod **acc_to_encoder** ()

Create a sequence encoder from this accumulator.

Return type

dmx.torch_stats.pdist.TorchSequenceEncoder

Returns

A TorchSequenceEncoder instance.

abstractmethod combine(*suff_stat*)

Combine this accumulator with another's sufficient statistics.

Parameters

suff_stat – Sufficient statistics to combine with.

Return type

dmx.torch_stats.pdist.TorchStatisticAccumulator

Returns

Updated accumulator with combined statistics.

Parameters

suff_stat (SS)

abstractmethod from_value(*x*)

Create an accumulator from sufficient statistics.

Parameters

x – Sufficient statistics value.

Return type

dmx.torch_stats.pdist.TorchStatisticAccumulator

Returns

Accumulator initialized with the given statistics.

Parameters

x (SS)

abstractmethod key_merge(*stats_dict*)

Merge statistics from a dictionary by key.

Parameters

stats_dict – Dictionary of statistics to merge.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

abstractmethod key_replace(*stats_dict*)

Replace statistics with values from a dictionary by key.

Parameters

stats_dict – Dictionary of statistics to use as replacements.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

abstractmethod seq_initialize(*x, weights, tng*)

Initialize sufficient statistics with a sequence of observations.

Parameters

- **x** – Sequence of observations.

- **weights** – Tensor of weights for each observation.
- **tng** – PyTorch random number generator.

Return type

None

Parameters

- **x** (*Any*)
- **weights** (*torch.Tensor*)
- **tng** (*Any*)

abstractmethod seq_update(*x, weights, estimate*)

Update sufficient statistics with a sequence of observations.

Parameters

- **x** – Sequence of observations.
- **weights** – Tensor of weights for each observation.
- **estimate** – Current parameter estimate.

Return type

None

Parameters

- **x** (*Any*)
- **weights** (*torch.Tensor*)
- **estimate** (*Any*)

abstractmethod value()

Return the current sufficient statistics value.

Return type*typing.TypeVar(SS)***Returns**

Current sufficient statistics.

class *dmx.torch_stats.pdist.TorchStatisticAccumulatorFactory*Bases: *object*Abstract factory for creating *TorchStatisticAccumulator* instances.

Factories provide a standard interface for creating accumulator instances on specific devices.

abstractmethod make(*device=None*)Create a new *TorchStatisticAccumulator* instance.**Parameters****device** – Target PyTorch device. Defaults to CPU if None.**Return type***dmx.torch_stats.pdist.TorchStatisticAccumulator***Returns**A new *TorchStatisticAccumulator* instance.**Parameters****device** (*torch.device* | *None*)

1.5.2 Torch Distributions

Binomial

Create, estimate, and sample torch-backed binomial distributions.

Defines the `BinomialDistribution`, `BinomialSampler`, `BinomialAccumulatorFactory`, `BinomialAccumulator`, `BinomialEstimator`, and the `BinomialDataEncoder` classes for use with pysparkplug.

Data type: `int`. Terminology and support match `dmx.stats.binomial`. Scalar methods and sampling use Python/NumPy values; encoded data are torch integer tensors, and vectorized likelihoods retain the encoded tensors' device and floating calculation dtype.

```
class dmx.torch_stats.binomial.BinomialAccumulator(max_val=None, min_val=None, keys=None,
                                                    device=None)
```

Bases: *TorchStatisticAccumulator*

Aggregate sufficient statistics of `BinomialDistribution`.

Parameters

- **max_val** (*int* / *None*)
- **min_val** (*int* / *None*)
- **keys** (*str* / *None*)
- **device** (*torch.device* / *None*)

sum

Aggregates the sum of all data observations.

Type

`float`

count

Aggregates the number of weighted-data observations used in accumulating sum.

Type

`float`

max_val

Largest integer value encountered while accumulating sufficient statistics.

Type

`Optional[int]`

min_val

Smallest integer value encountered while accumulating sufficient statistics.

Type

`Optional[int]`

key

All `BinomialAccumulators` with the same key will have suff-stats merged.

Type

`Optional[str]`

acc_to_encoder()

Return the compatible binomial encoder.

Return type

dmx.torch_stats.binomial.BinomialDataEncoder

combine(*suff_stat*)

Merge a binomial sufficient-statistic tuple.

Return type

dmx.torch_stats.binomial.BinomialAccumulator

Parameters

suff_stat (*Tuple*[*float*, *float*, *int* | *None*, *int* | *None*])

from_value(*x*)

Replace state from a binomial sufficient-statistic tuple.

Return type

dmx.torch_stats.binomial.BinomialAccumulator

Parameters

x (*Tuple*[*float*, *float*, *int* | *None*, *int* | *None*])

key_merge(*stats_dict*)

Merge this accumulator's keyed statistic into *stats_dict*.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

key_replace(*stats_dict*)

Replace state from its keyed statistic, when present.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

seq_initialize(*x*, *weights*, *tng*)

Initialize statistics from encoded observations and weights.

Return type

None

Parameters

- **x** (*BinomialTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator* | *None*)

seq_update(*x*, *weights*, *estimate*)

Accumulate encoded observations with their weights.

Return type

None

Parameters

- **x** (*BinomialTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **estimate** (*BinomialDistribution* | *None*)

value()

Return the binomial sufficient-statistic tuple.

Return type

`typing.Tuple[float, float, typing.Optional[int], typing.Optional[int]]`

class `dmx.torch_stats.binomial.BinomialAccumulatorFactory`(*max_val=None, min_val=0, keys=None, _device=None*)

Bases: *TorchStatisticAccumulatorFactory*

Creates BinomialAccumulatorFactory object.

Parameters

- **max_val** (*int* / *None*)
- **min_val** (*int* / *None*)
- **keys** (*str* / *None*)
- **_device** (*torch.device* / *None*)

max_val

Max value for binomial observations.

Type

`Optional[int]`

min_val

min value for binomial observations.

Type

`Optional[int]`

keys

Declare BinomialAccumulatorFactory objects for merging suff_stats.

Type

`Optional[str]`

make(*device=None*)

Create an accumulator associated with device.

Return type

dmx.torch_stats.binomial.BinomialAccumulator

Parameters

device (*torch.device* / *None*)

class `dmx.torch_stats.binomial.BinomialDataEncoder`

Bases: *TorchSequenceEncoder*

Encode count sequences for torch vectorized likelihood calculations.

The raw count tensor has shape (n,) and uses `vec.int_tensor` dtype and device semantics. Unique values and inverse indices form the encoded representation; the torch container is the difference from stats.

seq_encode(*x, device=None*)

Encode integer sequences for vectorized accumulator/distribution use.

Parameters

- **x** (*Sequence[int]*) – Sequence of integers.

- **device** (*Optional[device]*) – Set device for data to be encoded to.

Return type

`dmx.torch_stats.binomial.BinomialTorchEncodedSequence`

Returns

Tuple[tn.Tensor, tn.Tensor, tn.Tensor, int, int] containing unique values in x, indices of ux to reconstruct x, the tensorized x, the min value of x, and the max value of x.

Parameters

- **x** (*Sequence[int]*)
- **device** (*torch.device | None*)

class `dmx.torch_stats.binomial.BinomialDistribution(p, n, min_val=None, keys=None, device=None)`

Bases: `TorchProbabilityDistribution`

Represent a binomial distribution with trial count n and probability p.

Notes

Supports data types of int between (0, n-1) or (min_val, n-min_val-1) if min_val is not None. Log-probability mass for `BinomialDistribution(n,p)`,

$$\log(f(x|n,p)) = \log(n!) - \log((n-x)!) - \log(x!) + x*\log(p) + (1-x)*\log(1-p),$$

for x in [0,n-1) or [min_val, n-1-min_val), else -inf.

Parameters

- **p** (*float*)
- **n** (*int*)
- **min_val** (*int | None*)
- **keys** (*str | None*)
- **device** (*torch.device | None*)

p

Proportion for binomial distribution, between (0,1.0].

Type

float

log_p

Logrithm of p above.

Type

float

log_1p

Logrithm of 1-p, p defined above.

Type

float

n

Number of trials in binomial distribution, n > 0.

Type
int

min_val

Change domain of binomial from (0,n-1) to (min_val, n-min_val).

Type
Optional[int]

keys

All BinomialDistributions with same keys are same distributions.

Type
Optional[str]

density(x)

Returns the probability mass of integer value x.

If x is not an integer between [0,n) or [min_val, n-1-min_val), density is 0.0.

Parameters
x (*int*) – Integer value for density evaluation.

Return type
float

Returns

float –

Probability mass of x for binomial(n,p) with
min_val=min_val. 0.0 if x is not in support.

Parameters
x (*int*)

dist_to_encoder()

Creates a BinomialDataEncoder object for sequence encoding data.

Return type
dmx.torch_stats.binomial.BinomialDataEncoder

Returns
BinomialDataEncoder object.

estimator(pseudo_count=None)

Create a BinomialEstimator for estimating BinomialDistribution.

Parameters
pseudo_count (*Optional[float]*) – If set, inflates counts for the currently set sufficient statistic (p).

Return type
dmx.torch_stats.binomial.BinomialEstimator

Returns
BinomialEstimator object.

Parameters
pseudo_count (*float | None*)

log_density(x)

Returns the log-probability mass of integer value x.

If x is not an integer between [0,n) or [min_val, n-1-min_val), log-density is -inf.

Parameters

x (*int*) – Integer value for density evaluation.

Return type

float

Returns

float –

Log-probability mass of x for binomial(n,p) with
min_val=min_val. -inf if x is not in support.

Parameters

x (*int*)

sampler(seed=None)

Return a BinomialSampler for *BinomialDistribution(n, p, min_val)*.

Parameters

seed **Optional**[int] – Used to set seed on the random number generator for sampling.

Return type

dmx.torch_stats.binomial.BinomialSampler

Returns

BinomialSampler for BinomialDistribution with seed.

Parameters

seed (*int* | *None*)

seq_log_density(x)

Evaluate log densities for an encoded sequence.

Return type

torch.Tensor

Parameters

x (*BinomialTorchEncodedSequence*)

to(device)

Select the device used for subsequent tensor calculations.

Return type

dmx.torch_stats.binomial.BinomialDistribution

Parameters

device (*str* | *torch.device* | *None*)

class dmx.torch_stats.binomial.**BinomialEstimator**(max_val=None, min_val=0, pseudo_count=None,
suff_stat=None, keys=None)

Bases: *TorchParameterEstimator*

Create a BinomialEstimator object for estimating BinomialDistribution.

Parameters

- **max_val** (*int* | *None*)

- **min_val** (*int* | *None*)
- **pseudo_count** (*float* | *None*)
- **suff_stat** (*float* | *None*)
- **keys** (*str* | *None*)

max_val

Set max value encountered.

Type

Optional[int]

min_val

Set min value for BinomialDistribution.

Type

Optional[int]

pseudo_count

Inflate sufficient statistic (p).

Type

Optional[float]

suff_stat

Set p from prior observations.

Type

Optional[float]

keys

Assign a key so matching estimators can later be combined in aggregation.

Type

Optional[str]

accumulator_factory()

Return a factory for compatible accumulators.

Return type

dmx.torch_stats.binomial.BinomialAccumulatorFactory

estimate(*nobs*, *suff_stat*, *device=None*)

Estimate a BinomialDistribution from *suff_stat*.

Note: *nobs* is not used here. Kept for consistency with other ParameterEstimators.

Member variable *suff_stat* is simply the proportion (p) of the BinomialDistribution passed to BinomialEstimator. The *pseudo_count* is used to inflate (p) in estimation.

Parameters

- **nobs** (*Optional[float]*) – Not used.
- **suff_stat** (*Tuple[float, float, Optional[int], Optional[int]]*) – Tuple of count, sum, min_val, and max_val obtained from data aggregation.
- **device** – Set the device for the estimate to be returned to.

Return type

dmx.torch_stats.binomial.BinomialDistribution

Returns

BinomialDistribution estimated from *suff_stat*, *self.suff_stat*, and *pseudo_count*.

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*float*, *float*, *int* | *None*, *int* | *None*])
- **device** (*torch.device* | *None*)

class dmx.torch_stats.binomial.**BinomialSampler**(*dist*, *seed=None*)

Bases: [DistributionSampler](#)

Draw NumPy samples from a binomial distribution.

Parameters

- **dist** ([BinomialDistribution](#))
- **seed** (*int* | *None*)

sample(*size=None*)

Draw samples from BinomialSampler.

Parameters

size (*Optional*[*int*]) – Number of samples to draw from BinomialSampler (1 if size is None).

Return type

typing.Union[*int*, *typing.List*[*int*]]

Returns

An integer sample from *BinomialDistribution*(*n*,*p*,*min_val*), or *List*[*int*] of samples with length *size*.

Parameters

size (*int* | *None*)

class dmx.torch_stats.binomial.**BinomialTorchEncodedSequence**(*data*, *device=None*)

Bases: [TorchEncodedSequence](#)

Store unique counts, inverse indices, raw counts, and their range.

Parameters

- **data** (*Tuple*[*torch.Tensor*, *torch.Tensor*, *torch.Tensor*, *int*, *int*])
- **device** (*torch.device* | *None*)

Diagonal Gaussian

Provide torch-backed diagonal Gaussian models and estimation utilities.

Defines the [DiagonalGaussianDistribution](#), [DiagonalGaussianSampler](#), [DiagonalGaussianAccumulatorFactory](#), [DiagonalGaussianAccumulator](#), [DiagonalGaussianEstimator](#), and the [DiagonalGaussianDataEncoder](#) classes for use with pysparkplug.

The log-density of an *n*-dimensional diagonal-Gaussian observation $x = (x_1, x_2, \dots, x_n)$ with mean $\mu = (m_1, m_2, \dots, m_n)$ and diagonal covariance matrix $\text{diag}(s2_1, s2_2, \dots, s2_n)$ is

$$\log(\mathbf{p_mat}(x)) = -0.5 * \sum_{i=1}^n (x_i - m_i)^2 / s2_i$$

- $0.5 * \log(s2_i) - (n/2) * \log(\pi)$.

Scalar methods accept one vector of shape (D,) and return Python floats. Sequence methods consume a (N, D) floating-point tensor created by `DiagonalGaussianDataEncoder` and return tensors of shape (N,). Model parameters are tensors on the requested device. They use the vector-helper default dtype, normally float64 and float32 on MPS; the encoded tensor dtype controls the calculation dtype in `seq_log_density`. Unlike `dmx.stats.dmvn`, this implementation uses one optional key for all sufficient statistics and its sampler and accumulator retain NumPy arrays on the CPU.

class `dmx.torch_stats.dmvn.DiagonalGaussianAccumulator`(*dim=None, keys=None, device=None*)

Bases: `TorchStatisticAccumulator`

Aggregate sufficient statistics from iid observations.

Parameters

- **dim** (*int* / *None*)
- **keys** (*str* / *None*)
- **device** (*torch.device* / *None*)

dim

Optional dimension of Gaussian.

Type

Optional[int]

count

Used for tracking weighted observations counts.

Type

float

sum

Sum of observation vectors.

Type

np.ndarray

sum2

Sum of squared observation vectors.

Type

np.ndarray

key

If set, merge sufficient statistics with objects containing matching keys.

Type

Optional[str]

acc_to_encoder()

Create an encoder using the accumulated dimension.

Return type

`dmx.torch_stats.dmvn.DiagonalGaussianDataEncoder`

combine(*suff_stat*)

Merge (*sum_x*, *sum_x2*, *count*) sufficient statistics.

Return type

`dmx.torch_stats.dmvn.DiagonalGaussianAccumulator`

Parameters**suff_stat** (*Tuple*[*ndarray*, *ndarray*, *float*])**from_value**(*x*)

Replace the accumulator from a sufficient-statistic tuple.

Return type*dmx.torch_stats.dmvn.DiagonalGaussianAccumulator***Parameters****x** (*Tuple*[*ndarray*, *ndarray*, *float*])**key_merge**(*stats_dict*)Merge this accumulator into *stats_dict* under its key.**Return type**

None

Parameters**stats_dict** (*Dict*[*str*, *Any*])**key_replace**(*stats_dict*)Replace statistics from the matching entry in *stats_dict*.**Return type**

None

Parameters**stats_dict** (*Dict*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *tng*)Add an encoded batch during initialization; *tng* is unused.**Return type**

None

Parameters

- **x** (*DiagonalGaussianTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator* | *None*)

seq_update(*x*, *weights*, *estimate*)Add *N* encoded vectors with weights of shape (*N*,).**Return type**

None

Parameters

- **x** (*DiagonalGaussianTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **estimate** (*DiagonalGaussianDistribution* | *None*)

value()Return CPU sufficient statistics (*sum_x*, *sum_x2*, *count*).**Return type***typing.Tuple*[*numpy.ndarray*, *numpy.ndarray*, *float*]

class `dmx.torch_stats.dmvn.DiagonalGaussianAccumulatorFactory`(*dim=None, keys=None*)

Bases: [`TorchStatisticAccumulatorFactory`](#)

Create diagonal Gaussian accumulators with a shared configuration.

Parameters

- **dim** (*int* | *None*)
- **keys** (*str* | *None*)

make(*device=None*)

Create an accumulator associated with device.

Return type

[`dmx.torch\_stats.dmvn.DiagonalGaussianAccumulator`](#)

Parameters

device (*torch.device* | *None*)

class `dmx.torch_stats.dmvn.DiagonalGaussianDataEncoder`(*dim=None*)

Bases: [`TorchSequenceEncoder`](#)

Encode sequences of iid diagonal-Gaussian observations.

Parameters

dim (*int* | *None*)

dim

Dimension of the Gaussian.

Type

Optional[int]

seq_encode(*x, device=None*)

Encode *N* vectors as a floating tensor of shape (*N*, *D*).

The tensor is created on device with the default dtype selected by [`dmx.torch\_utils.vector.tensor\(\)`](#).

Return type

[`dmx.torch\_stats.dmvn.DiagonalGaussianTorchEncodedSequence`](#)

Parameters

- **x** (*Sequence[List[float]* | *ndarray*)
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.dmvn.DiagonalGaussianDistribution`(*mu, covar, keys=None, device=None*)

Bases: [`TorchProbabilityDistribution`](#)

Represent a torch-backed Gaussian with diagonal covariance.

The support is $\mathbb{R}^D$. *mu* and *covar* are length-*D* vectors, and each covariance entry is a positive variance, not a standard deviation.

Parameters

- **mu** (*Sequence[float]* | *ndarray*)
- **covar** (*Sequence[float]* | *ndarray*)
- **keys** (*str* | *None*)
- **device** (*torch.device* | *None*)

dim

Dimension of the multivariate Gaussian. Determined by the mean length.

Type

int

mu (tn.Tensor): Mean tensor of shape (D,). covar (tn.Tensor): Variance tensor of shape (D,). log_c (tn.Tensor): Scalar normalizing constant. ca (tn.Tensor): Quadratic log-density coefficients of shape (D,). cb (tn.Tensor): Linear log-density coefficients of shape (D,). cc (tn.Tensor): Scalar constant log-density term. key (Optional[str]): Key for merging sufficient statistics.

density(x)

Evaluate the density of one CPU observation of shape (D,).

Return type

float

Parameters

x (*Sequence[float] | ndarray*)

dist_to_encoder()

Create an encoder fixed to this distribution's dimension.

Return type

dmx.torch_stats.dmvn.DiagonalGaussianDataEncoder

estimator(pseudo_count=None)

Create an estimator with an optional shared mean/variance pseudo-count.

Return type

dmx.torch_stats.dmvn.DiagonalGaussianEstimator

Parameters

pseudo_count (*float | None*)

log_density(x)

Evaluate the log density using NumPy on the CPU.

Return type

float

Parameters

x (*Sequence[float] | ndarray*)

sampler(seed=None)

Create a CPU NumPy sampler, optionally initialized with seed.

Return type

dmx.torch_stats.dmvn.DiagonalGaussianSampler

Parameters

seed (*int | None*)

seq_log_density(x)

Evaluate N encoded observations and return a tensor of shape (N,).

Return type

torch.Tensor

Parameters

x (*DiagonalGaussianTorchEncodedSequence*)

to(device)

Move parameter tensors to device in place and return self.

Return type

dmx.torch_stats.dmvn.DiagonalGaussianDistribution

Parameters

device (str | torch.device | None)

class `dmx.torch_stats.dmvn.DiagonalGaussianEstimator`(dim=None, pseudo_count=(None, None),
suff_stat=(None, None), keys=None)

Bases: *TorchParameterEstimator*

Estimate diagonal Gaussian distributions from aggregated statistics.

Parameters

- **dim** (int | None)
- **pseudo_count** (Tuple[float | None, float | None])
- **suff_stat** (Tuple[ndarray | None, ndarray | None])
- **keys** (str | None)

dim

Dimension of Gaussian, either set of determined from suff_stat arg.

Type

int

prior_mu

Set from suff_stat[0].

Type

Optional[np.ndarray]

prior_covar

Set from suff_stat[1].

Type

(Optional[np.ndarray])

pseudo_count

Re-weight the sum of observations and sum of squared observations in estimation.

Type

Tuple[Optional[float], Optional[float]]

keys

Key for merging sufficient statistics.

Type

Optional[str]

accumulator_factory()

Create a factory carrying this estimator's dimension and key.

Return type

dmx.torch_stats.dmvn.DiagonalGaussianAccumulatorFactory

estimate(*nobs*, *suff_stat*, *device=None*)

Estimate parameters from (sum_x, sum_x2, count) on device.

Return type

dmx.torch_stats.dmvn.DiagonalGaussianDistribution

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *ndarray*, *float*])
- **device** (*torch.device* | *None*)

class *dmx.torch_stats.dmvn.DiagonalGaussianSampler*(*dist*, *seed=None*)

Bases: *DistributionSampler*

DiagonalGaussianSampler object for sampling from DiagonalGaussian instance.

Parameters

- **dist** (*DiagonalGaussianDistribution*)
- **seed** (*int* | *None*)

dist

Object instance to sample from.

Type

DiagonalGaussianDistribution

seed

Seed for random number generator.

Type

Optional[*int*]

sample(*size=None*)

Draw one vector or a list of size vectors, each of shape (D,).

Return type

typing.Union[*typing.Sequence*[*numpy.ndarray*], *numpy.ndarray*]

Parameters

size (*int* | *None*)

class *dmx.torch_stats.dmvn.DiagonalGaussianTorchEncodedSequence*(*data*, *device=None*)

Bases: *TorchEncodedSequence*

Store an encoded floating tensor with shape (N, D).

Parameters

- **data** (*torch.Tensor*)
- **device** (*torch.device* | *None*)

Exponential

Torch-backed exponential distributions, estimation, and sequence encoding.

This module uses the same scale (mean) beta parameterization as *dmx.stats.exponential*. Scalar methods return Python floats and sampling uses NumPy; encoded sequences and vectorized likelihoods use torch tensors.

class `dmx.torch_stats.exponential.ExponentialAccumulator`(*keys=None, device=None*)

Bases: [*TorchStatisticAccumulator*](#)

ExponentialAccumulator object used to accumulate sufficient statistics.

Parameters

- **keys** (*str* | *None*)
- **device** (*torch.device* | *None*)

sum

Tracks the sum of observation values.

Type

float

count

Tracks the sum of weighted observations used to form sum.

Type

float

key

Aggregate all sufficient statistics with same key.

Type

Optional[str]

_device

Device for tensor operations.

Type

TorchDevice

acc_to_encoder()

Return the encoder accepted by this accumulator.

Return type

[*dmx.torch_stats.exponential.ExponentialDataEncoder*](#)

combine(*suff_stat*)

Merge a (sum, count) sufficient statistic.

Return type

[*dmx.torch_stats.exponential.ExponentialAccumulator*](#)

Parameters

suff_stat (*Tuple[float, float]*)

from_value(*x*)

Replace state from a (sum, count) sufficient statistic.

Return type

[*dmx.torch_stats.exponential.ExponentialAccumulator*](#)

Parameters

x (*Tuple[float, float]*)

key_merge(*stats_dict*)

Merge this accumulator's keyed statistic into stats_dict.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**key_replace**(*stats_dict*)

Replace this accumulator's state from its keyed statistic.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**seq_initialize**(*x, weights, tng*)Initialize from encoded values and tensor weights; *tng* is unused.**Return type**

None

Parameters

- **x** ([ExponentialTorchEncodedSequence](#))
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator* / *None*)

seq_update(*x, weights, estimate*)Accumulate encoded values and weights of matching shape (*n*,).

Tensor reductions are converted to Python floats, so accumulator state is device- and dtype-independent.

Return type

None

Parameters

- **x** ([ExponentialTorchEncodedSequence](#))
- **weights** (*torch.Tensor*)
- **estimate** ([ExponentialDistribution](#) / *None*)

value(*_device=None*)

Return the (sum, count) sufficient statistic.

Return type*typing.Tuple*[float, float]**Parameters****_device** (*str* / *None*)**class** `dmx.torch_stats.exponential.ExponentialAccumulatorFactory`(*keys=None*)Bases: [TorchStatisticAccumulatorFactory](#)

Create exponential accumulators with a shared optional key.

Parameters**keys** (*str* / *None*)

make(*device=None*)

Create an accumulator associated with device.

Return type

dmx.torch_stats.exponential.ExponentialAccumulator

Parameters

device (*torch.device* | *None*)

class *dmx.torch_stats.exponential.ExponentialDataEncoder*

Bases: *TorchSequenceEncoder*

Encode positive exponential observations as a floating tensor.

Input is a list or NumPy array of shape (n,). `vec.tensor` chooses its normal floating dtype and places the tensor on device; callers needing a particular dtype should provide data that follows that utility's conversion convention. This differs from `stats` only in returning a torch-backed encoded container.

seq_encode(*x, device=None*)

Validate and encode positive observations on device.

The returned data tensor has shape (n,). CUDA, MPS, and CPU are supported to the extent supported by PyTorch and `vec.tensor`.

Return type

dmx.torch_stats.exponential.ExponentialTorchEncodedSequence

Parameters

- **x** (*List[float]* | *ndarray*)
- **device** (*torch.device* | *None*)

class *dmx.torch_stats.exponential.ExponentialDistribution(beta, device=None)*

Bases: *TorchProbabilityDistribution*

Represent an exponential distribution with mean beta.

This is the torch counterpart of `stats.ExponentialDistribution`. Its numerical parameter is kept as a Python float; to selects the device used by scalar-to-tensor and encoded-sequence work without moving a stored parameter tensor.

Parameters

- **beta** (*float*)
- **device** (*torch.device* | *None*)

beta

Scale of exponential.

Type

float

log_beta

Log of scale.

Type

float

density(*x*)

Density of Exponential distribution at observation x.

See `log_density()` for details.

Parameters

$\mathbf{x}$ (*float*) – Real-valued observation of Exponential.

Return type

float

Returns

float – Density of Exponential at x .

Parameters

$\mathbf{x}$ (*float*)

dist_to_encoder()

Returns a ExponentialDataEncoder object for encoding sequences of data.

Return type

dmx.torch_stats.exponential.ExponentialDataEncoder

estimator(pseudo_count=None)

Create an estimator, optionally regularized toward this scale.

Return type

dmx.torch_stats.exponential.ExponentialEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(x)

Log-density of Exponential distribution at observation x .

Log-density of Exponential with mean μ and variance σ^2 given by,

$\log(f(x; \mu, \sigma^2)) = -\log(2\pi\sigma^2) - (x-\mu)^2/\sigma^2$, for real-valued x .

Parameters

$\mathbf{x}$ (*float*) – Real-valued observation of Exponential.

Return type

float

Returns

float – Log-density at observation x .

Parameters

$\mathbf{x}$ (*float*)

sampler(seed=None)

Create a NumPy-backed sampler, optionally seeded.

Return type

dmx.torch_stats.exponential.ExponentialSampler

Parameters

seed (*int* | *None*)

seq_log_density(x)

Evaluate log densities for an encoded one-dimensional sequence.

$\mathbf{x}$.data and the result have shape $(n,)$ and retain the encoded tensor's device and floating dtype. Unlike scalar evaluation, this method requires the module's encoded-sequence container.

Return type

torch.Tensor

Parameters

x ([ExponentialTorchEncodedSequence](#))

to(*device*)

Select the device used for subsequent tensor calculations.

No model parameter is moved because parameters are Python floats.

Return type

[dmx.torch_stats.exponential.ExponentialDistribution](#)

Parameters

device (*str* | *torch.device* | *None*)

class [dmx.torch_stats.exponential.ExponentialEstimator](#)(*pseudo_count=None*, *suff_stat=None*, *keys=None*)

Bases: [TorchParameterEstimator](#)

Estimate exponential scale from (sum, count) statistics.

Parameters

- **pseudo_count** (*float* | *None*)
- **suff_stat** (*float* | *None*)
- **keys** (*str* | *None*)

accumulator_factory()

Return a factory for compatible accumulators.

Return type

[dmx.torch_stats.exponential.ExponentialAccumulatorFactory](#)

estimate(*nobs*, *suff_stat*, *device=None*)

Estimate [ExponentialDistribution](#) from *suff_stat* arg.

Estimate [ExponentialDistribution](#) from sufficient statistic tuple *suff_stat*, storing the weighted observation sum followed by the weighted count. If *pseudo_count* is set, this is used to re-weight the member value “*suff_stat*”, which is the scale of [ExponentialEstimator](#) object.

Parameters

- **nobs** (*Optional[float]*) – Not used. Kept for consistency with [ParameterEstimator](#).
- **suff_stat** (*Tuple[float, float]*) – Tuple of (sum, count). Both are positive real-valued floats.
- **device** (*Optional[TorchDevice]*) – Set for estimating model on GPU device

Return type

[dmx.torch_stats.exponential.ExponentialDistribution](#)

Returns

[ExponentialDistribution](#) object.

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple[float, float]*)
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.exponential.ExponentialSampler`(*dist*, *seed=None*)

Bases: [`DistributionSampler`](#)

Draw NumPy samples from an exponential distribution.

Parameters

- **dist** ([`ExponentialDistribution`](#))
- **seed** (*int* | *None*)

sample(*size=None*)

Draw ‘size’ iid samples from ExponentialSampler object.

Parameters

size (*Optional[int]*) – Treated as 1 if None is passed.

Return type

`typing.Union[float, numpy.ndarray]`

Returns

Numpy array of length ‘size’ from exponential distribution with scale beta if size not None.
Else a single sample is returned as float.

Parameters

size (*int* | *None*)

class `dmx.torch_stats.exponential.ExponentialTorchEncodedSequence`(*data*, *device=None*)

Bases: [`TorchEncodedSequence`](#)

Store a floating encoded exponential sequence of shape (n,).

Parameters

- **data** (*torch.Tensor*)
- **device** (*torch.device* | *None*)

Gamma

Torch-backed gamma distributions, estimation, and sequence encoding.

This mirrors `dmx.stats.gamma` parameterization and sufficient-statistic terminology. Scalar APIs use Python and NumPy values; encoded sequence APIs operate on torch tensors placed by the encoder.

class `dmx.torch_stats.gamma.GammaAccumulator`(*keys=None*, *device=None*)

Bases: [`TorchStatisticAccumulator`](#)

GammaAccumulator object used to accumulate sufficient statistics.

Parameters

- **keys** (*str* | *None*)
- **device** (*torch.device* | *None*)

nobs

Number of observations accumulated.

Type

`float`

sum

Weighted-sum of observations accumulated.

Type

float

sum_of_logs

log weighted sum of weighted log(observations).

Type

float

key

GammaAccumulator objects with same key merge sufficient statistics.

Type

Optional[str]

acc_to_encoder()

Return the compatible gamma encoder.

Return type

dmx.torch_stats.gamma.GammaDataEncoder

combine(suff_stat)

Merge a (count, sum, log_sum) sufficient statistic.

Return type

dmx.torch_stats.gamma.GammaAccumulator

Parameters

suff_stat (*Tuple[float, float, float]*)

from_value(x)

Replace state from a gamma sufficient-statistic tuple.

Return type

dmx.torch_stats.gamma.GammaAccumulator

Parameters

x (*Tuple[float, float, float]*)

key_merge(stats_dict)

Merge this accumulator's keyed statistic into stats_dict.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

key_replace(stats_dict)

Replace state from its keyed statistic, when present.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

seq_initialize(*x*, *weights*, *tng*)

Initialize statistics from encoded observations and weights.

Return type

None

Parameters

- **x** ([GammaTorchEncodedSequence](#))
- **weights** ([torch.Tensor](#))
- **tng** ([torch.Generator](#) | *None*)

seq_update(*x*, *weights*, *estimate*)

Accumulate encoded observations with their weights.

Return type

None

Parameters

- **x** ([GammaTorchEncodedSequence](#))
- **weights** ([torch.Tensor](#))
- **estimate** ([GammaDistribution](#) | *None*)

value()

Return the (count, sum, log_sum) sufficient statistic.

Return type

[typing.Tuple](#)[float, float, float]

class `dmx.torch_stats.gamma.GammaAccumulatorFactory`(*keys=None*)

Bases: [TorchStatisticAccumulatorFactory](#)

GammaAccumulatorFactory object for creating GammaAccumulator objects.

Parameters

keys (*str* | *None*)

keys

Used for merging sufficient statistics of GammaAccumulator.

Type

[Optional](#)[*str*]

make(*device=None*)

Create an accumulator associated with device.

Return type

[dmx.torch_stats.gamma.GammaAccumulator](#)

Parameters

device ([torch.device](#) | *None*)

class `dmx.torch_stats.gamma.GammaDataEncoder`

Bases: [TorchSequenceEncoder](#)

Encode positive gamma observations as a floating tensor of shape (n,).

The encoder preserves `vec.tensor` dtype behavior and places data on the requested CPU, CUDA, or MPS device. It differs from `stats` only by returning a torch encoded-sequence container.

seq_encode(*x*, *device=None*)

Encode iid gamma observations for vectorized `seq_` function calls.

Note: Each entry of *x* must be positive float.

Parameters

- **x** (*Union[List[float], np.ndarray]*) – IID sequence of gamma distributed observations.
- **device** (*Optional[tn.device]*) – Device to encode tensors on.

Return type

`dmx.torch_stats.gamma.GammaTorchEncodedSequence`

Returns

Tuple Tensors containing *x* and $\log(x)$.

Parameters

- **x** (*List[float] | ndarray*)
- **device** (*torch.device | None*)

class `dmx.torch_stats.gamma.GammaDistribution`(*k*, *theta*, *device=None*)

Bases: `TorchProbabilityDistribution`

Represent a gamma distribution with the stats module's parameters.

to records the preferred torch device. Parameters are Python floats, so movement affects future tensor calculations rather than stored values.

Parameters

- **k** (*float*)
- **theta** (*float*)
- **device** (*torch.device | None*)

density(*x*)

Density of gamma distribution evaluated at *x*.

See `log_density()` for details.

Parameters

x (*float*) – Positive real-valued number.

Return type

`float`

Returns

float – Density of gamma distribution evaluated at *x*.

Parameters

x (*float*)

dist_to_encoder()

Return the encoder for gamma observations.

Return type

`dmx.torch_stats.gamma.GammaDataEncoder`

estimator(*pseudo_count=None*)

Create an estimator, optionally regularized toward this model.

Return type*dmx.torch_stats.gamma.GammaEstimator***Parameters****pseudo_count** (*float* | *None*)**log_density(x)**

Log-density of gamma distribution evaluated at x.

For positive x, this evaluates the shape-scale gamma log-density. Values outside the support have log-density `-np.inf`.**Parameters****x** (*float*) – Positive real-valued number.**Return type***float***Returns***float* – Log-density of gamma distribution evaluated at x.**Parameters****x** (*float*)**sampler(seed=None)**

Create a NumPy-backed sampler, optionally seeded.

Return type*dmx.torch_stats.gamma.GammaSampler***Parameters****seed** (*int* | *None*)**seq_log_density(x)**

Evaluate log densities for an encoded sequence.

Return type*torch.Tensor***Parameters****x** (*GammaTorchEncodedSequence*)**to(device)**

Select the device used for subsequent tensor calculations.

Return type*dmx.torch_stats.gamma.GammaDistribution***Parameters****device** (*str* | *torch.device* | *None*)

```
class dmx.torch_stats.gamma.GammaEstimator(pseudo_count=(0.0, 0.0), suff_stat=(1.0, 0.0),
                                           threshold=1.0e-8, keys=None)
```

Bases: *TorchParameterEstimator*

Estimate GammaDistribution from aggregated data.

Parameters

- **pseudo_count** (*Tuple[float, float]*)
- **suff_stat** (*Tuple[float, float]*)
- **threshold** (*float*)

- **keys** (*str* | *None*)

pseudo_count

Values used to re-weight member instances of sufficient statistics.

Type

Tuple[float, float]

suff_stat

shape 'k' and scale 'theta'.

Type

Tuple[float, float]

threshold

Threshold used for estimating the shape of gamma.

Type

float

keys

Assign keys to GammaEstimator for combining sufficient statistics.

Type

Optional[str]

accumulator_factory()

Return a factory for compatible accumulators.

Return type

dmx.torch_stats.gamma.GammaAccumulatorFactory

estimate(*nobs*, *suff_stat*, *device=None*)

Obtain GammaDistribution from aggregated sufficient statistics.

Takes sufficient statistic aggregated from observed data:

suff_stat[0]: weighted sum of observations *suff_stat*[1]: weighted sum of log-observations *suff_stat*[2]: weighted observation count.

Parameters

- **nobs** (*Optional*[float]) – Not used. Kept for consistency with ParameterEstimator.
- **suff_stat** – See description above for details.
- **device** (*Optional*[*tn.device*]) – Device to declare estimate on.

Return type

dmx.torch_stats.gamma.GammaDistribution

Returns

GammaDistribution object.

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[float, float, float])
- **device** (*torch.device* | *None*)

static estimate_shape(*avg_sum*, *avg_sum_of_logs*, *threshold*)

Estimates the shape parameter of GammaDistribution.

Parameters

- **avg_sum** (*float*) – Weighted sum of gamma observations.
- **avg_sum_of_logs** (*float*) – Weighted log sum of gamma observations.
- **threshold** (*float*) – Threshold used for assessing convergence of shape estimation.

Return type

float

Returns

Estimate of shape parameter 'k'.

Parameters

- **avg_sum** (*float*)
- **avg_sum_of_logs** (*float*)
- **threshold** (*float*)

class `dmx.torch_stats.gamma.GammaSampler`(*dist*, *seed=None*)

Bases: *DistributionSampler*

GammaSampler object used to draw samples from GammaDistribution.

Parameters

- **dist** (*GammaDistribution*)
- **seed** (*int* | *None*)

rng

RandomState with seed set for sampling.

Type

RandomState

dist

GammaDistribution to sample from.

Type

GammaDistribution

seed

Used to set seed on random number generator used in sampling.

Type

Optional[int]

sample(*size=None*)

Draw 'size'-iid observations from GammaSampler.

Parameters

size (*Optional[int]*) – Number of iid samples to draw from GammaSampler.

Return type

typing.Union[float, numpy.ndarray]

Returns

Single sample (float) if size is None, else a numpy array of floats containing iid samples from GammaDistribution.

Parameters

size (*int* | *None*)

class `dmx.torch_stats.gamma.GammaTorchEncodedSequence`(*data*, *device=None*)

Bases: [*TorchEncodedSequence*](#)

Store gamma encoded tensor data and its requested device.

Parameters

- **data** (*Tuple*[*torch.Tensor*, *torch.Tensor*])
- **device** (*torch.device* | *None*)

Gaussian

Torch-backed univariate Gaussian distributions and their estimators.

The module uses the same `mu` and variance `sigma2` terminology as `dmx.stats.gaussian`. Parameters and scalar results remain Python floats; only encoded sequence operations use torch tensors.

class `dmx.torch_stats.gaussian.GaussianAccumulator`(*keys=None*, *device=None*)

Bases: [*TorchStatisticAccumulator*](#)

GaussianAccumulator object used to accumulate sufficient statistics.

Parameters

- **keys** (*str* | *None*)
- **device** (*torch.device* | *None*)

sum

Sum of weighted observations ($\sum_i w_i X_i$).

Type

float

sum2

Sum of weighted squared observations ($\sum_i w_i X_i^2$)

Type

float

count

Sum of weights for observations ($\sum_i w_i$).

Type

float

count2

Sum of weights for squared observations ($\sum_i w_i$).

Type

float

key

Key string used to aggregate all sufficient statistics with same keys values.

Type

Optional[str]

acc_to_encoder()

Return the compatible Gaussian encoder.

Return type*dmx.torch_stats.gaussian.GaussianDataEncoder***combine(suff_stat)**

Merge a (sum, sum2, count, count2) statistic.

Return type*dmx.torch_stats.gaussian.GaussianAccumulator***Parameters****suff_stat** (Tuple[float, float, float, float])**from_value(x)**

Replace accumulator state from a sufficient-statistic tuple.

Return type*dmx.torch_stats.gaussian.GaussianAccumulator***Parameters****x** (Tuple[float, float, float, float])**key_merge(stats_dict)**

Merge this accumulator's keyed statistic into stats_dict.

Return type

None

Parameters**stats_dict** (Dict[str, Any])**key_replace(stats_dict)**

Replace state from its keyed statistic, when present.

Return type

None

Parameters**stats_dict** (Dict[str, Any])**seq_initialize(x, weights, tng)**

Initialize from an encoded sequence and matching tensor weights.

Return type

None

Parameters

- **x** (*GaussianTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator* | *None*)

seq_update(x, weights, estimate)

Accumulate encoded values and weights, each with shape (n,).

Return type

None

Parameters

- **x** ([GaussianTorchEncodedSequence](#))
- **weights** ([torch.Tensor](#))
- **estimate** ([GaussianDistribution](#) | *None*)

value(*_device=None*)

Return the Gaussian sufficient-statistic tuple.

Return type

typing.Tuple[float, float, float, float]

Parameters**_device** (*str* | *None*)**class** `dmx.torch_stats.gaussian.GaussianAccumulatorFactory`(*keys=None*)Bases: [TorchStatisticAccumulatorFactory](#)

Create Gaussian accumulators with a shared optional key.

Parameters**keys** (*str* | *None*)**make**(*device=None*)

Create an accumulator that records device.

Return type[dmx.torch_stats.gaussian.GaussianAccumulator](#)**Parameters****device** (*torch.device* | *None*)**class** `dmx.torch_stats.gaussian.GaussianDataEncoder`Bases: [TorchSequenceEncoder](#)

Encode one-dimensional observations as a torch tensor of shape (n,).

`vec.tensor` determines the floating dtype and places data on the requested CPU, CUDA, or MPS device. Unlike `stats`, the result is a torch encoded-sequence container.**seq_encode**(*x, device=None*)

Encode observations as a floating tensor on device.

Return type[dmx.torch_stats.gaussian.GaussianTorchEncodedSequence](#)**Parameters**

- **x** (*List[float]* | *ndarray* | *torch.Tensor*)
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.gaussian.GaussianDistribution`(*mu, sigma2, device=None*)Bases: [TorchProbabilityDistribution](#)Represent a univariate Gaussian with mean `mu` and variance `sigma2`.`to` records the device for future tensor operations. It does not move parameters because this implementation stores them as Python floats.

Parameters

- **mu** (*float*)
- **sigma2** (*float*)
- **device** (*torch.device* | *None*)

density(*x*)

Density of Gaussian distribution at observation *x*.

Parameters

x (*float*) – Real-valued observation of Gaussian.

Return type

float

Returns

float – Density of Gaussian at *x*.

Parameters

x (*float*)

dist_to_encoder()

Return the encoder for one-dimensional Gaussian observations.

Return type

dmx.torch_stats.gaussian.GaussianDataEncoder

estimator(*pseudo_count=None*)

Create an estimator, optionally regularized toward this distribution.

Return type

dmx.torch_stats.gaussian.GaussianEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Log-density of Gaussian distribution at observation *x*.

Parameters

x (*float*) – Real-valued observation of Gaussian.

Return type

float

Returns

float – Log-density at observation *x*.

Parameters

x (*float*)

sampler(*seed=None*)

Create a NumPy-backed sampler, optionally seeded.

Return type

dmx.torch_stats.gaussian.GaussianSampler

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Evaluate log densities for an encoded tensor of shape (*n*,).

The returned tensor has the encoded tensor's device and floating dtype.

Return type

`torch.Tensor`

Parameters

x (`GaussianTorchEncodedSequence`)

to(*device*)

Select the device for later torch calculations.

Return type

`dmx.torch_stats.gaussian.GaussianDistribution`

Parameters

device (`str` | `torch.device` | `None`)

class `dmx.torch_stats.gaussian.GaussianEstimator`(*pseudo_count*=(`None`, `None`), *suff_stat*=(`None`, `None`), *keys*=`None`)

Bases: `TorchParameterEstimator`

GaussianEstimator object for estimating GaussianDistribution with torch tensors.

Parameters

- **pseudo_count** (`Tuple[float | None, float | None]`)
- **suff_stat** (`Tuple[float | None, float | None]`)
- **keys** (`str` | `None`)

pseudo_count

Pseudo count to regularize suff stats.

Type

`Tuple[Optional[float], Optional[float]]`

suff_stat

Suff stats for Gaussian.

Type

`Tuple[Optional[float], Optional[float]]`

keys

Key for distribution.

Type

`Optional[str]`

accumulator_factory()

Return a factory for compatible accumulators.

Return type

`dmx.torch_stats.gaussian.GaussianAccumulatorFactory`

estimate(*nobs*, *suff_stat*, *device*=`None`)

Estimate Gaussian parameters from the supplied sufficient statistic.

Return type

`dmx.torch_stats.gaussian.GaussianDistribution`

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*float*, *float*, *float*, *float*])
- **device** (*torch.device* | *None*)

class dmx.torch_stats.gaussian.**GaussianSampler**(*dist*, *seed=None*)

Bases: *DistributionSampler*

GaussianSampler for drawing samples from GaussianSampler instance.

Parameters

- **dist** (*GaussianDistribution*)
- **seed** (*int* | *None*)

dist

GaussianDistribution instance to sample from.

Type

GaussianDistribution

tng

RandomState with seed set to seed if passed in args.

Type

tn.Generator

sample(*size=None*)

Draw 'size' iid samples from GaussianSampler object.

Numpy array of length 'size' from Gaussian distribution with mean mu and scale sigma2 if size not None. Else a single sample is returned as float.

Parameters

size (*Optional*[*int*]) – Treated as 1 if None is passed.

Return type

typing.Union[*float*, *numpy.ndarray*]

Returns

'size' iid samples from Gaussian distribution.

Parameters

size (*int* | *None*)

class dmx.torch_stats.gaussian.**GaussianTorchEncodedSequence**(*data*, *device=None*)

Bases: *TorchEncodedSequence*

Store encoded Gaussian observations in a tensor of shape (n,).

Parameters

- **data** (*torch.Tensor*)
- **device** (*torch.device* | *None*)

data

iid observations of Gaussian

Type

tn.Tensor

device

Device that data lives on.

Type

Optional[tn.device]

Geometric

Torch-backed geometric distributions, estimation, and sequence encoding.

The support and success-probability terminology match `dmx.stats.geometric`. Scalar likelihoods and samples remain Python/NumPy values; encoded sequences are torch tensors.

class `dmx.torch_stats.geometric.GeometricAccumulator`(*keys=None, device=None*)

Bases: *TorchStatisticAccumulator*

GeometricAccumulator object used to accumulate sufficient statistics.

Parameters

- **keys** (*str* | *None*)
- **device** (*torch.device* | *None*)

sum

Aggregate weighted sum of observations.

Type

float

count

Aggregate sum of weighted observation count.

Type

float

key

Assigned from keys arg.

Type

Optional[str]

acc_to_encoder()

Return the compatible geometric encoder.

Return type

dmx.torch_stats.geometric.GeometricDataEncoder

combine(*suff_stat*)

Merge a (count, sum) sufficient statistic.

Return type

dmx.torch_stats.geometric.GeometricAccumulator

Parameters

suff_stat (*Tuple*[float, float])

from_value(*x*)

Replace state from a (count, sum) sufficient statistic.

Return type

dmx.torch_stats.geometric.GeometricAccumulator

Parameters**x** (*Tuple[float, float]*)**key_merge**(*stats_dict*)Merge this accumulator's keyed statistic into *stats_dict*.**Return type**

None

Parameters**stats_dict** (*Dict[str, Any]*)**key_replace**(*stats_dict*)

Replace state from its keyed statistic, when present.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**seq_initialize**(*x, weights, tng*)

Initialize statistics from encoded observations and weights.

Return type

None

Parameters

- **x** (*GeometricTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator* | *None*)

seq_update(*x, weights, estimate*)

Accumulate encoded observations with their weights.

Return type

None

Parameters

- **x** (*GeometricTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **estimate** (*GeometricDistribution* | *None*)

value()

Return the (count, sum) sufficient statistic.

Return type*typing.Tuple[float, float]***class** `dmx.torch_stats.geometric.GeometricAccumulatorFactory`(*keys=None*)Bases: *TorchStatisticAccumulatorFactory*

Create geometric accumulators with a shared optional key.

Parameters**keys** (*str* | *None*)

make(*device=None*)

Create an accumulator associated with *device*.

Return type

dmx.torch_stats.geometric.GeometricAccumulator

Parameters

device (*torch.device* | *None*)

class *dmx.torch_stats.geometric.GeometricDataEncoder*

Bases: *TorchSequenceEncoder*

Encode geometric observations as a torch tensor of shape (n,).

The encoder uses *vec.int_tensor* dtype semantics and places the result on its requested CPU, CUDA, or MPS device. This is the torch-container counterpart of the *stats* encoded sequence.

seq_encode(*x, device=None*)

Validate and encode positive integer observations on *device*.

Return type

dmx.torch_stats.geometric.GeometricTorchEncodedSequence

Parameters

- **x** (*Sequence[int]* | *ndarray*)
- **device** (*torch.device* | *None*)

class *dmx.torch_stats.geometric.GeometricDistribution*(*p, device=None*)

Bases: *TorchProbabilityDistribution*

Represent a geometric distribution with success probability *p*.

to changes the preferred device for future tensor work only; this model stores its probability as a Python float.

Parameters

- **p** (*float*)
- **device** (*torch.device* | *None*)

density(*x*)

Density of geometric distribution evaluated at *x*.

Notes

$P(x=k) = (k-1)*\log(1-p) + \log(p)$, for $x = 1, 2, \dots$, else 0.0.

Parameters

x (*int*) – Observed geometric value (1,2,3,...).

Return type

float

Returns

float – Density of geometric distribution evaluated at *x*.

Parameters

x (*int*)

dist_to_encoder()

Return the encoder for geometric observations.

Return type

dmx.torch_stats.geometric.GeometricDataEncoder

estimator(pseudo_count=None)

Create an estimator, optionally regularized toward this model.

Return type

dmx.torch_stats.geometric.GeometricEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(x)

Log-density of geometric distribution evaluated at x.

Notes

See density() for details.

Parameters

x (*int*) – Must be natural number (1,2,3,...).

Return type

float

Returns

float – Log-density of geometric distribution evaluated at x.

Parameters

x (*int*)

sampler(seed=None)

Create a NumPy-backed sampler, optionally seeded.

Return type

dmx.torch_stats.geometric.GeometricSampler

Parameters

seed (*int* | *None*)

seq_log_density(x)

Evaluate log densities for an encoded sequence.

Return type

torch.Tensor

Parameters

x (*GeometricTorchEncodedSequence*)

to(device)

Select the device used for subsequent tensor calculations.

Return type

dmx.torch_stats.geometric.GeometricDistribution

Parameters

device (*str* | *torch.device* | *None*)

```
class dmx.torch_stats.geometric.GeometricEstimator(pseudo_count=None, suff_stat=None,
                                                    keys=None)
```

Bases: [TorchParameterEstimator](#)

Estimate GeometricDistribution from aggregated sufficient statistics.

Parameters

- **pseudo_count** (*float* | *None*)
- **suff_stat** (*float* | *None*)
- **keys** (*str* | *None*)

pseudo_count

Assigned from pseudo_count arg.

Type

Optional[float]

suff_stat

Assigned from suff_stat arg and corrected for the [0,1] constraint.

Type

Optional[float]

keys

Assigned from keys arg.

Type

Optional[str]

accumulator_factory()

Return a factory for compatible accumulators.

Return type

[dmx.torch_stats.geometric.GeometricAccumulatorFactory](#)

estimate(nobs, suff_stat, device=None)

Estimate a geometric model from (count, sum) statistics.

Return type

[dmx.torch_stats.geometric.GeometricDistribution](#)

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*float*, *float*])
- **device** (*torch.device* | *None*)

```
class dmx.torch_stats.geometric.GeometricSampler(dist, seed=None)
```

Bases: [DistributionSampler](#)

GeometricSampler object used to draw samples from GeometricDistribution.

Parameters

- **dist** ([GeometricDistribution](#))
- **seed** (*int* | *None*)

rng

RandomState with seed set for sampling.

Type

RandomState

dist

GeometricDistribution to sample from.

Type

GeometricDistribution

sample(*size=None*)

Generate iid samples from geometric distribution.

Generates a single geometric sample (int) if size is None, else a numpy array of integers of length size, iid samples, from the geometric distribution.

Parameters

size (*Optional[int]*) – Number of iid samples to draw. If None, assumed to be 1.

Return type

`typing.Union[int, numpy.ndarray]`

Returns

If size is None, int, else size length numpy array of ints.

Parameters

size (*int | None*)

class `dmx.torch_stats.geometric.GeometricTorchEncodedSequence`(*data, device=None*)

Bases: *TorchEncodedSequence*

Store integer encoded geometric observations and their device.

Parameters

- **data** (*torch.Tensor*)
- **device** (*torch.device | None*)

Integer multinomial

Provide torch-backed multinomial counts over contiguous integer categories.

Defines the IntegerMultinomialDistribution, IntegerMultinomialSampler, IntegerMultinomialAccumulatorFactory, IntegerMultinomialAccumulator, IntegerMultinomialEstimator, and the IntegerMultinomialDataEncoder classes for use with pysparkplug.

An observation is a sparse sequence of (integer, count) pairs. Its score is the sum of `count * log(category_probability)` plus the log mass assigned to the total count by the optional length distribution.

Probability-vector entry *i* represents category `min_val + i`. Encoded sequences flatten all nonzero entries from *N* observations into tensors of shape (*M*,) plus an observation-index tensor and encoded total counts. Model tensors move in place with `to`; scalar scoring reads parameters back as Python floats, while sequence scoring returns the vector-helper floating dtype on the model device. That dtype is normally float64 and is float32 on MPS. As in `dmx.stats.intmultinomial`, the score omits the multinomial combinatorial coefficient.

class `dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator`(*min_val=None, max_val=None, keys=None, len_accumulator=None, device=None*)

Bases: *TorchStatisticAccumulator*

Accumulate sufficient statistics from observed data.

Parameters

- **min_val** (*int* | *None*)
- **max_val** (*int* | *None*)
- **keys** (*str* | *None*)
- **len_accumulator** (*TorchStatisticAccumulator* | *None*)
- **device** (*torch.device* | *None*)

min_val

Minimum value for integer multinomial.

Type

Optional[int]

max_val

Maximum value for integer multinomial.

Type

Optional[int]

len_accumulator

Optional accumulator for integer multinomial trial counts. Set to `NullAccumulator()` if `None`.

Type

Optional[*TorchStatisticAccumulator*]

count_vec

Counter for the number of values in the integer multinomial range. Initialized if *min_val* and *max_val* are passed; otherwise set to `None`.

Type

Optional[ndarray]

key

Keys for merging sufficient stats with other objects containing a matching key.

Type

Optional[str]

acc_to_encoder()

Create an encoder using the nested length encoder.

Return type

dmx.torch_stats.intmultinomial.IntegerMultinomialDataEncoder

combine(*suff_stat*)

Merge (*min_val*, *category_counts*, *length_statistics*).

Return type

dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator

Parameters

suff_stat (*Tuple*[*int*, *ndarray*, *SS* | *None*])

from_value(*x*)

Replace the accumulator from a sufficient-statistic tuple.

Return type

`dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator`

Parameters

x (`Tuple[int, ndarray, SS | None]`)

key_merge(*stats_dict*)

Merge category and nested length statistics by key.

Return type

`None`

Parameters

stats_dict (`Dict[str, Any]`)

key_replace(*stats_dict*)

Replace category and nested length statistics by key.

Return type

`None`

Parameters

stats_dict (`Dict[str, Any]`)

seq_initialize(*x, weights, tng*)

Add an encoded batch during initialization; tng is unused.

Return type

`None`

Parameters

- **x** (`IntegerMultinomialTorchSequence`)
- **weights** (`torch.Tensor`)
- **tng** (`torch.Generator | None`)

seq_update(*x, weights, estimate*)

Accumulate N encoded sparse observations with (N,) weights.

Return type

`None`

Parameters

- **x** (`IntegerMultinomialTorchSequence`)
- **weights** (`torch.Tensor`)
- **estimate** (`IntegerMultinomialDistribution | None`)

value()

Return contiguous CPU category counts and length statistics.

Return type

`typing.Tuple[int, numpy.ndarray, typing.Optional[typing.Any]]`

```
class dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulatorFactory(min_val=None,
                                                                           max_val=None,
                                                                           keys=None,
                                                                           len_factory=NoneAccumulatorFactory)
```

Bases: [TorchStatisticAccumulatorFactory](#)

Factory for IntegerMultinomialAccumulator objects.

Parameters

- **min_val** (*int* / *None*)
- **max_val** (*int* / *None*)
- **keys** (*str* / *None*)
- **len_factory** ([TorchStatisticAccumulatorFactory](#) / *None*)

min_val

Optional minimum value for IntegerMultinomialAccumulator.

Type

Optional[int]

max_val

Optional maximum value for IntegerMultinomialAccumulator.

Type

Optional[int]

keys

Optional keys for merging sufficient statistics of the object instance.

Type

Optional[str]

len_factory

Optional factory for the number-of-trials accumulator. Defaults to `NullAccumulatorFactory()`.

Type

Optional[[StatisticAccumulatorFactory](#)]

make (*device=None*)

Create an accumulator associated with `device`.

Return type

[dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator](#)

Parameters

device (*torch.device* / *None*)

```
class dmx.torch_stats.intmultinomial.IntegerMultinomialDataEncoder(len_encoder=NoneDataEncoder())
```

Bases: [TorchSequenceEncoder](#)

Encode sequences of iid integer multinomial observations.

Parameters

len_encoder ([TorchSequenceEncoder](#) / *None*)

len_encoder

[TorchSequenceEncoder](#) for encoding the number of trials in each iid integer multinomial observation. Defaults to `NullDataEncoder()` if `None` is passed.

Type*TorchSequenceEncoder***seq_encode**(*x*, *device=None*)

Flatten *N* sparse count observations into torch tensors.

The returned tuple is (*N*, *indices*, *counts*, *values*, *total_counts*). The three flat tensors have shape (*M*,); *indices* and category values are integer tensors, while *counts* use the default floating dtype.

Return type*dmx.torch_stats.intmultinomial.IntegerMultinomialTorchSequence***Parameters**

- **x** (*Sequence[Sequence[Tuple[int, float]]]*)
- **device** (*torch.device* | *None*)

```
class dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution(min_val=0, p_vec=None,  
                                                                len_dist=NoneDistribution(),  
                                                                keys=None, device=None)
```

Bases: *TorchProbabilityDistribution*

Declare a multinomial distribution on a set of integers.

Parameters

- **min_val** (*int*)
- **p_vec** (*List[float]* | *ndarray* | *torch.Tensor* | *None*)
- **len_dist** (*TorchProbabilityDistribution* | *None*)
- **keys** (*str* | *None*)
- **device** (*torch.device* | *None*)

p_vec

Probability of each integer category for a trial.

Type*tn.Tensor***min_val**

Smallest integer value for category range. Defaults to 0.

Type*int***max_val**

Largest value of category range. Set by *min_val* + *len(p_vec)* - 1.

Type*int***log_p_vec**

Log of *p_vec* member instance.

Type*tn.Tensor*

num_vals

Total number of integer valued categories.

Type

int

len_dist

Distribution for number of trials. Set to NullDistribution if None.

Type

SequenceEncodableProbabilityDistribution

keys

Keys for distribution passed when ParameterEstimator is created.

Type

Optional[str]

density(x)

Evaluate the density of IntegerMultinomialDistribution at observed value x.

Parameters

x (*Sequence[Tuple[int, float]]*) – Sequence of tuples containing the integer category value and number of successes.

Return type

float

Returns

float – Density at x.

Parameters

x (*Sequence[Tuple[int, float]]*)

dist_to_encoder()

Create an encoder compatible with the total-count distribution.

Return type

dmx.torch_stats.intmultinomial.IntegerMultinomialDataEncoder

estimator(pseudo_count=None)

Create an estimator retaining support and optional smoothing.

Return type

dmx.torch_stats.intmultinomial.IntegerMultinomialEstimator

Parameters

pseudo_count (*float | None*)

log_density(x)

Evaluate the log-density at observed value x.

Notes

Log-density given by,

$\log(p_mat(x)) = \log(n!) - \sum_k x_k \log(p_k) - \log(x_k!)$, for x having k integer categories.

n is the total number of trials in observation x and x has k integer values. p_k denotes the probability of success for integer-category x_k.

Parameters

x (*Sequence[Tuple[int, float]]*) – Sequence of tuples containing the integer category value and number of successes.

Return type

float

Returns

float – Log-density at x.

Parameters

x (*Sequence[Tuple[int, float]]*)

sampler(*seed=None*)

Create a sampler when a non-null total-count model is configured.

Return type

dmx.torch_stats.intmultinomial.IntegerMultinomialSampler

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Return one log mass per encoded observation as a tensor of shape (N,).

Return type

torch.Tensor

Parameters

x (*IntegerMultinomialTorchSequence*)

to(*device*)

Move category and length-model parameters to device in place.

Return type

dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution

Parameters

device (*str* | *torch.device* | *None*)

```
class dmx.torch_stats.intmultinomial.IntegerMultinomialEstimator(min_val=None, max_val=None,
                                                                len_estimator=NoneEstimator(),
                                                                len_dist=None,
                                                                pseudo_count=None,
                                                                suff_stat=None, keys=None)
```

Bases: *TorchParameterEstimator*

Estimate integer multinomial distributions from aggregated data.

Parameters

- **min_val** (*int* | *None*)
- **max_val** (*int* | *None*)
- **len_estimator** (*TorchParameterEstimator* | *None*)
- **len_dist** (*TorchProbabilityDistribution* | *None*)
- **pseudo_count** (*float* | *None*)
- **suff_stat** (*Tuple[int, ndarray]* | *None*)
- **keys** (*str* | *None*)

min_val

Set minimum value integer multinomial.

Type

Optional[int]

max_val

Set maximum value for integer multinomial.

Type

Optional[int]

len_estimator

ParameterEstimator for number of trials, default *NullEstimator()*.

Type

TorchParameterEstimator

len_dist

Optional TorchProbabilityDistribution for fixing distribution on number of trials.

Type

Optional[*TorchProbabilityDistribution*]

name

Set name for object instance.

Type

Optional[str]

pseudo_count

Used to re-weight sufficient statistics if suff_stat is passed.

Type

Optional[float]

suff_stat

Set minimum value and counts for categories. If *min_val* and *max_val* are both not None, this is ignored in estimation.

Type

Optional[Tuple[int, np.ndarray]]

keys

Set key for merging sufficient statistics of objects with matching keys.

Type

Optional[str]

accumulator_factory()

Create a factory using fixed, prior, or data-derived support.

Return type

dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulatorFactory

estimate(nobs, suff_stat, device=None)

Estimate probabilities and the total-count model on device.

Return type

dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*int*, *ndarray*, *SS* | *None*])
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.intmultinomial.IntegerMultinomialSampler`(*dist*, *seed=None*)

Bases: *DistributionSampler*

Sample from IntegerMultinomialDistribution.

Parameters

- **dist** (*IntegerMultinomialDistribution*)
- **seed** (*int* | *None*)

p_vec

Probability for each value.

Type

np.ndarray

min_val

Min value of multinomial

Type

int

max_val

Max value of multinomial

Type

int

rng

RandomState set with seed if passed.

Type

RandomState

len_sampler

DistributionSampler for the number of trials.

Type

DistributionSampler

sample(*size=None*)

Draw independent samples from an integer multinomial distribution.

Parameters

size (*Optional*[*int*]) – Number of samples to draw.

Return type

typing.Union[*typing.List*[*typing.Tuple*[*int*, *float*]], *typing.List*[*typing.List*[*typing.Tuple*[*int*, *float*]]]

Returns

List of length *size* containing *List*[*Tuple*[*int*, *float*]]. If *size* is *None*, returns one sample *List*[*Tuple*[*int*, *float*]].

Parameters

size (*int* | *None*)

class `dmx.torch_stats.intmultinomial.IntegerMultinomialTorchSequence`(*data*, *device=None*)

Bases: [*TorchEncodedSequence*](#)

Store flattened sparse counts and an encoded total-count sequence.

Parameters

- **data** (*Tuple[int, torch.Tensor, torch.Tensor, torch.Tensor, TorchEncodedSequence]*)
- **device** (*torch.device | None*)

Integer categorical

Provide torch-backed categorical models on contiguous integer ranges.

Defines the `IntegerCategoricalDistribution`, `IntegerCategoricalSampler`, `IntegerCategoricalAccumulatorFactory`, `IntegerCategoricalAccumulator`, `IntegerCategoricalEstimator`, and the `IntegerCategoricalDataEncoder` classes for use with `pysparkplug`.

Data type (int): The integer categorical distribution is defined through summary statistics *min_val* (int) and a probability vector *p_vec* (*np.ndarray[float]*) that sums to 1.0. The range of values is given by [*min_val*, *min_val* + *len(p_vec)* - 1]. The density is then,

$$P(x_{\text{mat}}=i) = p_vec[i]$$

for *x* in {*min_val*, *min_val*+1, ..., *min_val* + *length(p_vec)* - 1}, else 0.0.

Probability-vector entry *i* represents category *min_val* + *i*. Scalar observations are integers, while encoded sequences are integer tensors of shape (N,). Model parameters move in place with `to`; floating tensors use the vector-helper default dtype, normally float64 and float32 on MPS. Sampling and sufficient statistics remain CPU NumPy data. This is the torch counterpart of `dmx.stats.inrange`.

class `dmx.torch_stats.inrange.IntegerCategoricalAccumulator`(*min_val=None*, *max_val=None*,
keys=None, *device=None*)

Bases: [*TorchStatisticAccumulator*](#)

Accumulate sufficient statistics from observed data.

Notes

If *min_val* and *max_val* are not provided, they are obtained from the data in accumulation step.

Parameters

- **min_val** (*int | None*)
- **max_val** (*int | None*)
- **keys** (*str | None*)
- **device** (*torch.device | None*)

min_val

Minimum value of integer categorical range.

Type

Optional[TI]

max_val

Maximum value of integer categorical range.

Type

Optional[TI]

count_vec

Torch tensor of floats for tracking probability weights for each integer value in support. Set to None if min_val and max_val are both not None.

Type

Optional[tn.Tensor]

key

Key for merging sufficient statistics of integer IntegerCategoricalAccumulator objects.

Type

Optional[str]

acc_to_encoder()

Return the compatible integer categorical encoder.

Return type

dmx.torch_stats.inrange.IntegerCategoricalDataEncoder

combine(suff_stat)

Merge (min_val, contiguous_category_counts) statistics.

Return type

dmx.torch_stats.inrange.IntegerCategoricalAccumulator

Parameters

suff_stat (Tuple[int | None, ndarray | None])

from_value(x)

Replace the accumulator from contiguous category counts.

Return type

dmx.torch_stats.inrange.IntegerCategoricalAccumulator

Parameters

x (Tuple[int, ndarray])

key_merge(stats_dict)

Merge this accumulator into stats_dict under its key.

Return type

None

Parameters

stats_dict (Dict[str, Any])

key_replace(stats_dict)

Replace statistics from the matching entry in stats_dict.

Return type

None

Parameters

stats_dict (Dict[str, Any])

seq_initialize(x, weights, tng)

Add an encoded batch during initialization; tng is unused.

Return type

None

Parameters

- **x** ([IntegerCategoricalTorchSequence](#))
- **weights** ([torch.Tensor](#))
- **tng** ([torch.Generator](#))

seq_update(*x, weights, estimate*)

Accumulate an integer tensor with weights of shape (N,).

Return type

None

Parameters

- **x** ([IntegerCategoricalTorchSequence](#))
- **weights** ([torch.Tensor](#))
- **estimate** ([IntegerCategoricalDistribution](#) / None)

value()

Return the minimum category and CPU count vector of shape (K,).

Return type

[typing.Tuple](#)[int, [numpy.ndarray](#)]

class [dmx.torch_stats.inrange.IntegerCategoricalAccumulatorFactory](#)(*min_val=None, max_val=None, keys=None*)

Bases: [TorchStatisticAccumulatorFactory](#)

Factory for IntegerCategoricalAccumulator objects.

Parameters

- **min_val** (*int* / None)
- **max_val** (*int* / None)
- **keys** (*str* / None)

min_val

Minimum value of integer categorical, if None estimated from data.

Type

[Optional](#)[int]

max_val

Maximum value of integer categorical, if None estimated from data.

Type

[Optional](#)[int]

make(*device=None*)

Create an accumulator associated with *device*.

Return type

[dmx.torch_stats.inrange.IntegerCategoricalAccumulator](#)

Parameters

device ([torch.device](#) / None)

class `dmx.torch_stats.inrange.IntegerCategoricalDataEncoder`

Bases: [*TorchSequenceEncoder*](#)

Encode sequences of iid integer categorical observations.

seq_encode(*x*, *device=None*)

Encode N integers as an integer tensor of shape (N,).

Return type

[*dmx.torch_stats.inrange.IntegerCategoricalTorchSequence*](#)

Parameters

- **x** (*List[int]* | *ndarray*)
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.inrange.IntegerCategoricalDistribution`(*min_val*, *p_vec*, *device=None*)

Bases: [*TorchProbabilityDistribution*](#)

Represent a categorical distribution over consecutive integers.

Parameters

- **min_val** (*int*)
- **p_vec** (*List[float]* | *ndarray*)
- **device** (*torch.device* | *None*)

p_vec

Must sum to 1.0. First probability is probability for `p_mat(x_mat=min_val)`.

Type

`tn.Tensor`

min_val

Minimum value in support of integer categorical

Type

`int`

max_val

Maximum value in support of integer categorical set to `min_val + length(p_vec) - 1`.

Type

`int`

log_p_vec

Log of `p_vec`.

Type

`tn.Tensor`

num_vals

Total number of values in support of `IntegerCategoricalDistribution` instance.

Type

`int`

density(*x*)

Evaluate the density of the integer categorical at observation `x`.

Notes

$p_{\text{mat}}(x_{\text{mat}}=x) = p_{\text{vec}}[x]$ if x in support $[\text{min_val}, \text{max_val}]$, else 0.0.

Parameters

x (*int*) – Integer value.

Return type

float

Returns

float – Density at x .

Parameters

x (*int*)

dist_to_encoder()

Create the compatible one-dimensional integer encoder.

Return type

dmx.torch_stats.inrange.IntegerCategoricalDataEncoder

estimator(*pseudo_count=None*)

Create an estimator retaining support and optional smoothing.

Return type

dmx.torch_stats.inrange.IntegerCategoricalEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(x)

Evaluate the log-density of the integer categorical at observation x .

Notes

$\log_p(x_{\text{mat}}=x) = \log_p_{\text{vec}}[x]$ if x is in support $[\text{min_val}, \text{max_val}]$, else $-\text{np.inf}$.

Parameters

x (*int*) – Integer value.

Return type

float

Returns

float – Log-density at x .

Parameters

x (*int*)

sampler(*seed=None*)

Create a CPU NumPy sampler, optionally initialized with *seed*.

Return type

dmx.torch_stats.inrange.IntegerCategoricalSampler

Parameters

seed (*int* | *None*)

seq_log_density(x)

Return log masses for an integer tensor of shape $(N,)$.

Return type
 torch.Tensor

Parameters
x ([IntegerCategoricalTorchSequence](#))

to(device)

Move probability tensors to device in place and return self.

Return type
[dmx.torch_stats.inrange.IntegerCategoricalDistribution](#)

Parameters
device (str | torch.device | None)

class [dmx.torch_stats.inrange.IntegerCategoricalEstimator](#)(min_val=None, max_val=None, pseudo_count=None, suff_stat=None, keys=None)

Bases: [TorchParameterEstimator](#)

Estimate IntegerCategoricalDistribution from sufficient statistics.

Parameters

- **min_val** (int | None)
- **max_val** (int | None)
- **pseudo_count** (float | None)
- **suff_stat** (Tuple[int, ndarray] | None)
- **keys** (str | None)

min_val

Minimum value of integer categorical.

Type
 Optional[TI]

max_val

Maximum value of integer categorical.

Type
 Optional[TI]

pseudo_count

Used to re-weight suff_stat when merged with new aggregated data.

Type
 Optional[float]

suff_stat

See above for details.

keys

Keys for accumulating merging statistics of IntegerCategoricalAccumulator objects.

Type
 Optional[str]

accumulator_factory()

Create a factory using fixed, prior, or data-derived support.

Return type

`dmx.torch_stats.inrange.IntegerCategoricalAccumulatorFactory`

estimate(*nobs*, *suff_stat*, *device=None*)

Estimate a contiguous probability vector on device.

Return type

`dmx.torch_stats.inrange.IntegerCategoricalDistribution`

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*int*, *ndarray*] | *None*)
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.inrange.IntegerCategoricalSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`

Sample from IntegerCategoricalDistribution.

Parameters

- **dist** (`IntegerCategoricalDistribution`)
- **seed** (*int* | *None*)

p_vec

Numpy array of probs for each integer value.

Type

`np.ndarray`

min_val

min val of range

Type

`int`

max_val

max val of range.

Type

`int`

rng

RandomState object with seed set if passed.

Type

`RandomState`

sample(*size=None*)

Draw iid samples from IntegerCategoricalSampler object.

Note: If size is None, a single sample is returned as an integer. If size > 0, a List of integers with length equal to size is returned.

Parameters

size (*Optional*[*int*]) – Number of iid samples to draw.

Return type

typing.Union[int, typing.List[int]]

Returns

Integer or List[int] of iid samples from IntegerCategoricalSampler
instance.

Parameters

size (int | None)

class dmx.torch_stats.intrange.**IntegerCategoricalTorchSequence**(data, device=None)

Bases: [TorchEncodedSequence](#)

Store encoded categorical integers in a tensor of shape (N,).

Parameters

- **data** (*torch.Tensor*)
- **device** (*torch.device* | *None*)

Integer Bernoulli set

Provide torch-backed Bernoulli distributions over finite integer sets.

Defines the IntegerBernoulliSetDistribution, IntegerBernoulliSetSampler, IntegerBernoulliSetAccumulatorFactory, IntegerBernoulliSetAccumulator, IntegerBernoulliSetEstimator, and the IntegerBernoulliSetDataEncoder classes for use with pysparkplug.

Let $S = \{0, 1, 2, 3, \dots, N-1\}$ be a set of integers. Let x_mat be a random subset of S . The Bernoulli set distribution models a random subset of S as

$$p_k = p_mat(k \text{ is in } x_mat), k = 0, 2, \dots, N-1.$$

The density for an observed subset of S , $x=(x_1, x_2, \dots, x_m)$, for $m < N$ is given by

$$p_mat(x) = \sum_{k=0}^{K-1} \{ p_k * (k \text{ in } x) + (1-p_k) * (k \text{ not in } x) \}.$$

Each observation is a sequence of included integers from $[0, K)$. The encoder flattens N sets into integer tensors of observation indices and values, both of shape $(M,)$. Parameters move in place with `to`; encoded scoring returns a floating tensor of shape $(N,)$ on the model device. Floating tensors use the vector-helper default dtype, normally float64 and float32 on MPS. Sampling and sufficient statistics use CPU NumPy arrays. This mirrors `dmx.stats.intsetdist` without its optional names.

class dmx.torch_stats.intsetdist.**IntegerBernoulliSetAccumulator**(num_vals, keys=None, device=None)

Bases: [TorchStatisticAccumulator](#)

Accumulate sufficient statistics from observed data.

Parameters

- **num_vals** (*int*)
- **keys** (*str* | *None*)
- **device** (*torch.device* | *None*)

pcnt

Used for aggregating weighted counts of integers.

Type

np.ndarray

key

Keys for merging sufficient statistics with matching keyed objects.

Type

Optional[str]

num_vals

Number of values in integer range for the set.

Type

int

tot_sum

Sum of weights for observations.

Type

float

acc_to_encoder()

Create the compatible flattened set encoder.

Return type

dmx.torch_stats.intsetdist.IntegerBernoulliSetDataEncoder

combine(suff_stat)

Merge (positive_counts, total_weight) statistics.

Return type

dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulator

Parameters

suff_stat (Tuple[ndarray, float])

from_value(x)

Replace the accumulator from inclusion sufficient statistics.

Return type

dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulator

Parameters

x (Tuple[ndarray, float])

key_merge(stats_dict)

Merge sufficient statistics into stats_dict under the key.

Return type

None

Parameters

stats_dict (Dict[str, Any])

key_replace(stats_dict)

Replace statistics from the matching entry in stats_dict.

Return type

None

Parameters

stats_dict (Dict[str, Any])

seq_initialize(*x*, *weights*, *tng*)

Add encoded sets during initialization; *tng* is unused.

Return type

None

Parameters

- **x** ([IntegerBernoulliSetTorchSequence](#))
- **weights** ([torch.Tensor](#))
- **tng** ([torch.Generator](#) | None)

seq_update(*x*, *weights*, *estimate*)

Accumulate N encoded sets with weights of shape (N,).

Return type

None

Parameters

- **x** ([IntegerBernoulliSetTorchSequence](#))
- **weights** ([torch.Tensor](#))
- **estimate** ([IntegerBernoulliSetDistribution](#) | None)

value()

Return CPU inclusion counts of shape (K,) and total weight.

Return type

[typing.Tuple](#)[[numpy.ndarray](#), float]

class `dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulatorFactory`(*num_vals*, *keys*=None)

Bases: [TorchStatisticAccumulatorFactory](#)

Factory for IntegerBernoulliSetAccumulator objects.

Parameters

- **num_vals** ([int](#))
- **keys** ([str](#) | None)

keys

Keys for merging sufficient statistics with matching keyed objects.

Type

[Optional](#)[[str](#)]

num_vals

Number of values in integer range for the set.

Type

[int](#)

make(*device*=None)

Create an accumulator associated with device.

Return type

[dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulator](#)

Parameters

device ([torch.device](#) | None)

class dmx.torch_stats.intsetdist.IntegerBernoulliSetDataEncoder

Bases: [*TorchSequenceEncoder*](#)

Flatten a sequence of finite integer sets for torch operations.

seq_encode(*x*, *device=None*)

Encode *N* sets as (*N*, observation_indices, values).

Both flat integer tensors have shape (*M*,) and are created on device; *M* is the total number of included integers.

Return type

[*dmx.torch_stats.intsetdist.IntegerBernoulliSetTorchSequence*](#)

Parameters

- **x** (*Sequence[Sequence[int]]*)
- **device** (*torch.device | None*)

class dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution(*log_pvec*, *log_nvec=None*,
keys=None, *device=None*)

Bases: [*TorchProbabilityDistribution*](#)

Represent independent inclusions on integers [*0*, *K*).

log_pvec[*k*] is the log inclusion probability for integer *k*. *log_nvec*[*k*] is its log absence probability when explicitly supplied.

Parameters

- **log_pvec** (*Sequence[float] | ndarray*)
- **log_nvec** (*Sequence[float] | ndarray | None*)
- **keys** (*str | None*)
- **device** (*torch.device | None*)

log_pvec

Probability of integer *k* being in set.

Type

Tensor

log_nvec

Optional normalizing probability for each integer probability.

Type

Tensor

log_dvec

Normalized probability for each integer value.

Type

Tensor

log_nsum

Sum of normalized probabilities used to add unobserved integer values in an observation.

Type

float

key

Set keys for object instance.

Type

Optional[str]

density(*x*)

Evaluate the probability mass of one integer set.

Return type

float

Parameters

x (*Sequence[int] | ndarray*)

dist_to_encoder()

Create the compatible flattened set encoder.

Return type

dmx.torch_stats.intsetdist.IntegerBernoulliSetDataEncoder

estimator(*pseudo_count=None*)

Create an estimator initialized with this support size.

Return type

dmx.torch_stats.intsetdist.IntegerBernoulliSetEstimator

Parameters

pseudo_count (*float | None*)

log_density(*x*)

Evaluate the log mass of one set using integer category indices.

Return type

float

Parameters

x (*Sequence[int] | ndarray*)

sampler(*seed=None*)

Create a CPU NumPy sampler, optionally initialized with *seed*.

Return type

dmx.torch_stats.intsetdist.IntegerBernoulliSetSampler

Parameters

seed (*int | None*)

seq_log_density(*x*)

Return log masses for N flattened encoded set observations.

Return type

torch.Tensor

Parameters

x (*IntegerBernoulliSetTorchSequence*)

to(*device*)

Move all parameter tensors to device in place and return *self*.

Return type

dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution

Parameters**device** (*str* | *torch.device* | *None*)

```
class dmx.torch_stats.intsetdist.IntegerBernoulliSetEstimator(num_vals, min_prob=1.0e-128,  
                                                             pseudo_count=None,  
                                                             suff_stat=None, keys=None)
```

Bases: *TorchParameterEstimator*

Estimate integer Bernoulli set distributions from sufficient stats.

Parameters

- **num_vals** (*int*)
- **min_prob** (*float*)
- **pseudo_count** (*float* | *None*)
- **suff_stat** (*ndarray* | *None*)
- **keys** (*str* | *None*)

num_vals

Number of values in integer range for the set.

Type*int***keys**

Keys for merging sufficient statistics with matching keyed objects.

Type*Optional[str]***pseudo_count**

Re-weight suff stats in estimation.

Type*Optional[float]***suff_stat**

Probability for integer inclusion.

Type*Optional[np.ndarray]***min_prob**

Minimum probability for an integer in the range.

Type*float***accumulator_factory()**

Create a factory for the configured support size and key.

Return type*dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulatorFactory***estimate** (*nobs*, *suff_stat=None*, *device=None*)

Estimate inclusion probabilities on device.

Return type*dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution*

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*ndarray* | *None*)
- **device** (*torch.device* | *None*)

class dmx.torch_stats.intsetdist.**IntegerBernoulliSetSampler**(*dist, seed=None*)

Bases: [DistributionSampler](#)

Sample from an IntegerBernoulliSetDistribution instance.

Parameters

- **dist** ([IntegerBernoulliSetDistribution](#))
- **seed** (*int* | *None*)

rng

RandomState object with seed set if passed in args.

Type

RandomState

log_pvec

Log probs for each value.

Type

np.ndarray

num_vals

Number of total values.

Type

int

sample(*size=None*)

Draw one integer set or a list of size sets.

Return type

typing.Union[typing.List[typing.Sequence[int]], typing.Sequence[int]]

Parameters

- **size** (*int* | *None*)

class dmx.torch_stats.intsetdist.**IntegerBernoulliSetTorchSequence**(*data, device=None*)

Bases: [TorchEncodedSequence](#)

Store a flattened integer-set encoding and observation count.

Parameters

- **data** (*Tuple[int, torch.Tensor, torch.Tensor]*)
- **device** (*torch.device* | *None*)

Multivariate Gaussian

Provide torch-backed multivariate Gaussian models and estimation utilities.

Defines the MultivariateGaussianDistribution, MultivariateGaussianSampler, MultivariateGaussianAccumulatorFactory, MultivariateGaussianAccumulator, MultivariateGaussianEstimator, and the MultivariateGaussianDataEncoder classes for use with pysparkplug.

Data type: `np.ndarray[float]`

$x = (x_1, x_2, \dots, x_n) \sim \text{MVN}(\mu, \text{covar})$, where μ is a length- n numpy array and covar is an n by n positive definite covariance matrix.

The log-density is given by

$$\log(p(x)) = -0.5 * k * \log(2 * \pi) - 0.5 * \det(\text{covar}) - 0.5 * (x - \mu)' \text{covar}^{-1} (x - \mu).$$

Scalar methods accept one NumPy vector of shape $(D,)$ and return Python floats. Encoded methods accept a floating tensor of shape (N, D) and return a tensor of shape $(N,)$. Parameters move in place with `to` and use the vector-helper floating dtype, normally float64 and float32 on MPS. Unlike `dmx.stats.mvn`, factorization uses torch and MPS likelihood solves run on CPU before results return to the model device and encoded dtype; sampling and accumulated sufficient statistics remain CPU NumPy data.

class `dmx.torch_stats.mvn.MultivariateGaussianAccumulator` (*dim=None, keys=None, device=None*)

Bases: `TorchStatisticAccumulator`

Accumulate weighted first and uncentered second moments on the CPU.

Parameters

- **dim** (*int* / *None*)
- **keys** (*str* / *None*)
- **device** (*torch.device* / *None*)

acc_to_encoder()

Create an encoder using the accumulated dimension.

Return type

`dmx.torch_stats.mvn.MultivariateGaussianDataEncoder`

combine (*suff_stat*)

Merge (*sum_x*, *sum_xx*, *count*) sufficient statistics.

Return type

`dmx.torch_stats.mvn.MultivariateGaussianAccumulator`

Parameters

suff_stat (*Tuple[ndarray, ndarray, float]*)

from_value (*x*)

Replace the accumulator from a sufficient-statistic tuple.

Return type

`dmx.torch_stats.mvn.MultivariateGaussianAccumulator`

Parameters

x (*Tuple[ndarray, ndarray, float]*)

key_merge (*stats_dict*)

Merge this accumulator into *stats_dict* under its key.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

key_replace (*stats_dict*)

Replace statistics from the matching entry in *stats_dict*.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**seq_initialize**(*x, weights, tng*)

Add an encoded batch during initialization; tng is unused.

Return type

None

Parameters

- **x** ([MultivariateGaussianTorchSequence](#))
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator* / *None*)

seq_update(*x, weights, estimate*)

Add N encoded vectors with weights of shape (N,).

Return type

None

Parameters

- **x** ([MultivariateGaussianTorchSequence](#))
- **weights** (*torch.Tensor*)
- **estimate** ([MultivariateGaussianDistribution](#) / *None*)

value()

Return CPU statistics with shapes (D,) and (D, D) plus count.

Return typetyping.Tuple[*numpy.ndarray*, *numpy.ndarray*, *float*]**class** `dmx.torch_stats.mvn.MultivariateGaussianAccumulatorFactory`(*dim, keys=None*)Bases: [TorchStatisticAccumulatorFactory](#)

Create multivariate Gaussian accumulators with shared configuration.

Parameters

- **dim** (*int* / *None*)
- **keys** (*str* / *None*)

make(*device=None*)

Create an accumulator associated with device.

Return type[dmx.torch_stats.mvn.MultivariateGaussianAccumulator](#)**Parameters****device** (*torch.device* / *None*)**class** `dmx.torch_stats.mvn.MultivariateGaussianDataEncoder`(*dim=None*)Bases: [TorchSequenceEncoder](#)

Encode vector observations as a two-dimensional floating tensor.

Parameters**dim** (*int* | *None*)**seq_encode** (*x*, *device=None*)

Encode N vectors as a floating tensor of shape (N, D).

The tensor is created on device with the default dtype selected by `dmx.torch_utils.vector.tensor()`.**Return type**`dmx.torch_stats.mvn.MultivariateGaussianTorchSequence`**Parameters**

- **x** (*Sequence[List[float]]* | *Sequence[List[ndarray]]* | *ndarray*)
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.mvn.MultivariateGaussianDistribution` (*mu*, *covar*, *keys=None*, *device=None*)Bases: `TorchProbabilityDistribution`

Represent a torch-backed multivariate Gaussian with full covariance.

The support is $\mathbb{R}^D$. *mu* has shape (D,) and the symmetric positive-definite covariance has shape (D, D).**Parameters**

- **mu** (*List[float]* | *ndarray*)
- **covar** (*List[List[float]]* | *ndarray*)
- **keys** (*str* | *None*)
- **device** (*torch.device* | *None*)

dim

Dimension D of the multivariate Gaussian.

Type*int***mu**

Mean tensor of shape (D,).

Type*tn.Tensor***covar**

Covariance tensor of shape (D, D).

Type*tn.Tensor***chol**

Lower Cholesky factor with shape (D, D).

Type*tn.Tensor***keys**

Set keys for distribution.

Type*Optional[str]*

chol_const

Scalar log-normalization constant.

Type

tn.Tensor

density(*x*)

Evaluate the density at *x*.

Parameters

x (*np.ndarray*) – Observation from multivariate Gaussian distribution.

Return type

float

Returns

float – Density at *x*.

Parameters

x (*ndarray*)

dist_to_encoder()

Create an encoder fixed to this distribution's dimension.

Return type

dmx.torch_stats.mvn.MultivariateGaussianDataEncoder

estimator(*pseudo_count=None*)

Create an estimator centered on this distribution when regularized.

Return type

dmx.torch_stats.mvn.MultivariateGaussianEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Evaluate the log-density at *x*.

Notes

$\log(p(x)) = -0.5 * k * \log(2 * \pi) - 0.5 * \det(\text{covar}) - 0.5 * (x - \mu)' \text{covar}^{-1} (x - \mu)$.

Parameters

x (*np.ndarray*) – Observation from multivariate Gaussian distribution.

Return type

float

Returns

float – Log-density at *x*.

Parameters

x (*ndarray*)

sampler(*seed=None*)

Create a CPU NumPy sampler, optionally initialized with *seed*.

Return type

dmx.torch_stats.mvn.MultivariateGaussianSampler

Parameters**seed**(*int* | *None*)**seq_log_density**(*x*)

Return log densities for an encoded tensor of shape (N, D).

Return type`torch.Tensor`**Parameters****x** (`MultivariateGaussianTorchSequence`)**to**(*device=None*)

Move parameters to device in place and recompute normalization.

Return type`dmx.torch_stats.mvn.MultivariateGaussianDistribution`**Parameters****device** (*str* | `torch.device` | *None*)

```
class dmx.torch_stats.mvn.MultivariateGaussianEstimator(dim=None, pseudo_count=(None, None),
                                                         suff_stat=(None, None), keys=None)
```

Bases: `TorchParameterEstimator`

Estimate a multivariate normal distribution from sufficient stats.

Parameters

- **dim** (*int* | *None*)
- **pseudo_count** (`Tuple[float | None, float | None]` | *None*)
- **suff_stat** (`Tuple[ndarray | None, ndarray | None]` | *None*)
- **keys** (*str* | *None*)

dim

Dimension of multivariate normal.

Type`int`**pseudo_count**

Regularize mean and/or covariance.

Type`Optional[Tuple[Optional[float], Optional[float]]]`**prior_mu**

Mean from prior data or used to regularize.

Type`Optional[np.ndarray]`**prior_covar**

Covariance matrix from prior data or used to regularize.

Type`Optional[np.ndarray]`

key

Keys for merging sufficient statistics.

Type

Optional[str]

accumulator_factory()

Create a factory carrying this estimator's dimension and key.

Return type

`dmx.torch_stats.mvn.MultivariateGaussianAccumulatorFactory`

estimate(*nobs*, *suff_stat*, *device=None*)

Estimate mean and full covariance on device.

Return type

`dmx.torch_stats.mvn.MultivariateGaussianDistribution`

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *ndarray*, *float*])
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.mvn.MultivariateGaussianSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`

Draw independent multivariate Gaussian vectors using NumPy.

Parameters

- **dist** (`MultivariateGaussianDistribution`)
- **seed** (*int* | *None*)

sample(*size=None*)

Draw one (D,) vector or an array with leading dimension size.

Return type

`numpy.ndarray`

Parameters

- **size** (*int* | *None*)

class `dmx.torch_stats.mvn.MultivariateGaussianTorchSequence`(*data*, *device=None*)

Bases: `TorchEncodedSequence`

Store an encoded floating tensor with shape (N, D).

Parameters

- **data** (*torch.Tensor*)
- **device** (*torch.device* | *None*)

Poisson

Torch-backed Poisson distributions, estimation, and sequence encoding.

The rate `lam` and (`count`, `sum`) sufficient-statistic terms match `dmx.stats.poisson`. Vectorized likelihoods use torch tensors, while scalar methods and samplers retain their Python/NumPy behavior.

class `dmx.torch_stats.poisson.PoissonAccumulator`(*keys=None, device=None*)

Bases: `TorchStatisticAccumulator`

PoissonAccumulator object used to accumulate sufficient statistics.

Parameters

- **keys** (*str* | *None*)
- **device** (*torch.device* | *None*)

sum

Aggregate sum of weighted observations.

Type

float

count

Aggregate sum of observation weights.

Type

float

key

Key for combining sufficient statistics with an object instance containing the same key.

Type

Optional[str]

acc_to_encoder()

Return the compatible Poisson encoder.

Return type

dmx.torch_stats.poisson.PoissonDataEncoder

combine(*suff_stat*)

Merge a (count, sum) sufficient statistic.

Return type

dmx.torch_stats.poisson.PoissonAccumulator

Parameters

suff_stat (*Tuple[float, float]*)

from_value(*x*)

Replace state from a (count, sum) sufficient statistic.

Return type

dmx.torch_stats.poisson.PoissonAccumulator

Parameters

x (*Tuple[float, float]*)

key_merge(*stats_dict*)

Merge this accumulator's keyed statistic into stats_dict.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

key_replace(*stats_dict*)

Replace state from its keyed statistic, when present.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

seq_initialize(*x, weights, tng=None*)

Initialize statistics from encoded observations and weights.

Return type

None

Parameters

- **x** (*PoissonTorchSequence*)
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator | None*)

seq_update(*x, weights, estimate=None*)

Accumulate encoded observations with their weights.

Return type

None

Parameters

- **x** (*PoissonTorchSequence*)
- **weights** (*torch.Tensor*)
- **estimate** (*PoissonDistribution | None*)

value()

Return the (count, sum) sufficient statistic.

Return type

typing.Tuple[float, float]

class `dmx.torch_stats.poisson.PoissonAccumulatorFactory`(*keys=None*)

Bases: *TorchStatisticAccumulatorFactory*

Create PoissonAccumulator objects.

Parameters

keys (*str | None*)

keys

Tag for combining sufficient statistics of PoissonAccumulator objects when constructed.

Type

Optional[str]

make(*device=None*)

Create an accumulator associated with device.

Return type

dmx.torch_stats.poisson.PoissonAccumulator

Parameters**device** (*torch.device* | *None*)**class** `dmx.torch_stats.poisson.PoissonDataEncoder`Bases: *TorchSequenceEncoder*

Encode nonnegative counts as a torch integer tensor of shape (n,).

The output follows `vec.int_tensor` dtype rules and is placed on the requested CPU, CUDA, or MPS device. This is the material difference from the NumPy-backed encoder in `stats`.

seq_encode(*x*, *device=None*)Validate and encode nonnegative integer observations on *device*.**Return type***dmx.torch_stats.poisson.PoissonTorchSequence***Parameters**

- **x** (*ndarray* | *Sequence[int]*)
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.poisson.PoissonDistribution`(*lam*, *device=None*)Bases: *TorchProbabilityDistribution*Represent a Poisson distribution with rate *lam*.*to* only records the preferred device because *lam* is a Python float.**Parameters**

- **lam** (*float*)
- **device** (*torch.device* | *None*)

density(*x*)Evaluate the density of Poisson distribution at observation *x*.**Parameters****x** (*int*) – Must be a non-negative integer value (0,1,2,...).**Return type***float***Returns***float* – Density of Poisson distribution evaluated at *x*.**Parameters****x** (*int*)**dist_to_encoder**()

Return the encoder for Poisson observations.

Return type*dmx.torch_stats.poisson.PoissonDataEncoder***estimator**(*pseudo_count=None*)

Create an estimator, optionally regularized toward this rate.

Return type*dmx.torch_stats.poisson.PoissonEstimator***Parameters****pseudo_count** (*float* | *None*)

log_density(*x*)

Log-density of Poisson distribution evaluated at *x*.

Parameters

x (*int*) – Must be a non-negative integer value (0,1,2,...).

Return type

float

Returns

float – Log-density of Poisson distribution evaluated at *x*.

Parameters

x (*int*)

sampler(*seed=None*)

Create a NumPy-backed sampler, optionally seeded.

Return type

dmx.torch_stats.poisson.PoissonSampler

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Evaluate log densities for an encoded sequence.

Return type

torch.Tensor

Parameters

x (*PoissonTorchSequence*)

to(*device*)

Select the device used for subsequent tensor calculations.

Return type

dmx.torch_stats.poisson.PoissonDistribution

Parameters

device (*str* | *torch.device* | *None*)

class *dmx.torch_stats.poisson.PoissonEstimator*(*pseudo_count=None, suff_stat=None, keys=None*)

Bases: *TorchParameterEstimator*

Estimate *PoissonDistribution* from aggregated sufficient statistics.

Parameters

- **pseudo_count** (*float* | *None*)
- **suff_stat** (*float* | *None*)
- **keys** (*str* | *None*)

pseudo_count

Re-weight *suff_stat*.

Type

Optional[*float*]

suff_stat

Mean of Poisson if not None.

Type

Optional[float]

keys

String keys of PoissonEstimator instance for combining sufficient statistics.

Type

Optional[str]

accumulator_factory()

Return a factory for compatible accumulators.

Return type

dmx.torch_stats.poisson.PoissonAccumulatorFactory

estimate(nobs, suff_stat, device=None)

Estimate a Poisson model from (count, sum) statistics.

Return type

dmx.torch_stats.poisson.PoissonDistribution

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*float*, *float*])
- **device** (*torch.device* | *None*)

class dmx.torch_stats.poisson.PoissonSampler(*dist*, *seed*=None)

Bases: *DistributionSampler*

PoissonSampler object used to draw samples from PoissonDistribution.

Parameters

- **dist** (*PoissonDistribution*)
- **seed** (*int* | *None*)

rng

RandomState with seed set for sampling.

Type

RandomState

dist

PoissonDistribution to sample from.

Type

GeometricDistribution

sample(*size*=None)

Generate iid samples from Poisson distribution.

Generates a single Poisson sample (int) if size is None, else a numpy array of integers of length size containing iid samples from the Poisson distribution.

Parameters

size (*Optional*[*int*]) – Number of iid samples to draw. If None, assumed to be 1.

Return type

typing.Union[int, numpy.ndarray]

Returns

If size is None, int, else size length numpy array of ints.

Parameters**size** (int | None)**class** dmx.torch_stats.poisson.**PoissonTorchSequence**(data, device=None)Bases: [TorchEncodedSequence](#)

Store integer Poisson encoded data and its requested device.

Parameters

- **data** (Tuple[torch.Tensor, torch.Tensor])
- **device** (torch.device | None)

1.5.3 Torch Structural Models

Composite

Provide torch-backed products of positional component distributions.

A composite observation is a tuple ($x_0, \dots, x_{\{K-1\}}$) whose field k belongs to child distribution `dists[k]`. Encoding transposes a sequence of N observation tuples into K child encoded sequences, each representing the same N observations. Scoring sums child log densities and returns a tensor of shape $(N,)$ on the child/model device. Device movement, sampling, accumulation, and estimation are delegated position by position. Unlike `dmx.stats.composite`, this wrapper has no name and stores device state.

class dmx.torch_stats.composite.**CompositeAccumulator**(accumulators, keys=None, device=None)Bases: [TorchStatisticAccumulator](#)

Aggregate a separate sufficient statistic for every tuple field.

The same observation-weight tensor is passed to every child. The public statistic is a tuple in positional child order.

Parameters

- **accumulators** (Sequence[[TorchStatisticAccumulator](#)])
- **keys** (str | None)
- **device** (torch.device | None)

acc_to_encoder()

Create a positional encoder from the child accumulators.

Return type[dmx.torch_stats.composite.CompositeDataEncoder](#)**combine**(suff_stat)

Merge a tuple of child sufficient statistics position by position.

Return type[dmx.torch_stats.composite.CompositeAccumulator](#)**Parameters****suff_stat** (Tuple[Any, ...])

from_value(*x*)

Replace child statistics from a positional tuple.

Return type

dmx.torch_stats.composite.CompositeAccumulator

Parameters

x (*Tuple*[*Any*, ...])

key_merge(*stats_dict*)

Merge the whole tuple and then recurse into child keys.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

key_replace(*stats_dict*)

Replace the whole tuple and then recurse into child keys.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

seq_initialize(*x*, *weights*, *tng*)

Initialize every child from its encoded field and shared weights.

Return type

None

Parameters

- **x** (*CompositeTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator*)

seq_update(*x*, *weights*, *estimate*)

Update every child from its encoded field and shared weights.

Return type

None

Parameters

- **x** (*CompositeTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **estimate** (*CompositeDistribution* / *None*)

value()

Return the tuple of child sufficient statistics.

Return type

typing.Tuple[*typing.Any*, ...]

class dmx.torch_stats.composite.**CompositeAccumulatorFactory**(*factories, keys=None*)

Bases: *TorchStatisticAccumulatorFactory*

Create composite accumulators from positional child factories.

Parameters

- **factories** (*Sequence[TorchStatisticAccumulatorFactory]*)
- **keys** (*str | None*)

make(*device=None*)

Create child accumulators and associate the wrapper with device.

Return type

dmx.torch_stats.composite.CompositeAccumulator

Parameters

device (*torch.device | None*)

class dmx.torch_stats.composite.**CompositeDataEncoder**(*encoders*)

Bases: *TorchSequenceEncoder*

Encode observation tuples using one positional child encoder per field.

Parameters

encoders (*Sequence[TorchSequenceEncoder]*)

seq_encode(*x, device=None*)

Transpose and encode N tuples into K child sequences.

Each child encoder receives its field values in observation order and the same device argument.

Return type

dmx.torch_stats.composite.CompositeTorchEncodedSequence

Parameters

- **x** (*Sequence[Tuple[Any, ...]]*)
- **device** (*torch.device | None*)

class dmx.torch_stats.composite.**CompositeDistribution**(*dists, device=None*)

Bases: *TorchProbabilityDistribution*

Model a tuple as independent draws from positional child distributions.

The support is the Cartesian product of the child supports. Every observation must have the same number and order of fields as dists.

Parameters

- **dists** (*Sequence[TorchProbabilityDistribution]*)
- **device** (*torch.device | None*)

dists

Positional children.

Type

Sequence[TorchProbabilityDistribution]

count

Number of tuple fields and child distributions.

Type

int

density(*x*)

Evaluate the scalar density for one positional observation tuple.

Return type

float

Parameters

x (*Tuple*[*Any*, ...])

dist_to_encoder()

Create a positional encoder from the child encoders.

Return type

dmx.torch_stats.composite.CompositeDataEncoder

estimator(*pseudo_count=None*)

Create one child estimator per field using *pseudo_count*.

Return type

dmx.torch_stats.composite.CompositeEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Evaluate the sum of child log densities for one tuple.

Return type

float

Parameters

x (*Tuple*[*Any*, ...])

sampler(*seed=None*)

Create independent child samplers from seeds derived from *seed*.

Return type

dmx.torch_stats.composite.CompositeSampler

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Sum child scores into a tensor of shape (N,).

Return type

torch.Tensor

Parameters

x (*CompositeTorchEncodedSequence*)

to(*device*)

Move every child to device in place and return self.

Return type

dmx.torch_stats.composite.CompositeDistribution

Parameters**device** (*str* | *torch.device* | *None*)**class** `dmx.torch_stats.composite.CompositeEstimator`(*estimators*, *keys=None*)Bases: *TorchParameterEstimator*

Estimate every positional child from its matching statistic.

Parameters

- **estimators** (*Sequence*[*TorchParameterEstimator*])
- **keys** (*str* | *None*)

accumulator_factory()

Create a composite factory from the child estimator factories.

Return type*dmx.torch_stats.composite.CompositeAccumulatorFactory***estimate**(*nobs*, *suff_stat*, *device=None*)

Estimate each child from its positional statistic on device.

Return type*dmx.torch_stats.composite.CompositeDistribution***Parameters**

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*Any*, ...])
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.composite.CompositeSampler`(*dist*, *seed=None*)Bases: *DistributionSampler*

Draw tuples by sampling each positional child independently.

Parameters

- **dist** (*CompositeDistribution*)
- **seed** (*int* | *None*)

sample(*size=None*)

Draw one K-tuple or a list of size such tuples.

Return type*typing.Union*[*typing.List*[*typing.Tuple*[*typing.Any*, ...]], *typing.Tuple*[*typing.Any*, ...]]**Parameters****size** (*int* | *None*)**class** `dmx.torch_stats.composite.CompositeTorchEncodedSequence`(*data*, *device=None*)Bases: *TorchEncodedSequence*

Store a tuple of K child encodings for the same N observations.

Parameters

- **data** (*Tuple*[*TorchEncodedSequence*, ...])
- **device** (*torch.device* | *None*)

Conditional

Provide torch-backed distributions selected by a given observation value.

An observation is (given_value, dependent_value). dmap selects the dependent child by given_value; a default child handles unmapped values, and a given distribution optionally models the selector. The encoder groups N observations by selector and stores one child encoding and integer index tensor per group. Device movement propagates to every child. These contracts match `dmx.stats.conditional`; encoded scoring returns torch tensors.

```
class dmx.torch_stats.conditional.ConditionalDistribution(dmap, default_dist=NoneDistribution(),
                                                         given_dist=NoneDistribution(),
                                                         keys=None, device=None)
```

Bases: [TorchProbabilityDistribution](#)

Model dependent values with children keyed by given values.

A missing default gives unmapped selectors log mass $-\infty$. A null given distribution contributes zero log mass; otherwise the model represents the joint factorization $p(\text{dependent} \mid \text{given}) p(\text{given})$.

Parameters

- **dmap** (*Dict[Any, TorchProbabilityDistribution] / List[TorchProbabilityDistribution]*)
- **default_dist** (*TorchProbabilityDistribution / None*)
- **given_dist** (*TorchProbabilityDistribution / None*)
- **keys** (*str / None*)
- **device** (*torch.device / None*)

density(x)

Evaluate the density of one (given, dependent) observation.

Return type

float

Parameters

x (*Tuple[T0, T1]*)

dist_to_encoder()

Create matching mapped, default, and given encoders.

Return type

dmx.torch_stats.conditional.ConditionalDistributionDataEncoder

estimator(pseudo_count=None)

Create matching child estimators using pseudo_count.

Return type

dmx.torch_stats.conditional.ConditionalDistributionEstimator

Parameters

pseudo_count (*float / None*)

log_density(x)

Evaluate $\log p(\text{dependent} \mid \text{given}) + \log p(\text{given})$.

Return type

float

Parameters

x (*Tuple[T0, T1]*)

sampler(*seed=None*)

Create mapped, default, and given samplers from derived seeds.

Return type

dmx.torch_stats.conditional.ConditionalDistributionSampler

Parameters

seed(*int* | *None*)

seq_log_density(*x*)

Scatter grouped child scores into a tensor of shape (N,).

Return type

torch.Tensor

Parameters

x (*ConditionalTorchEncodedSequence*)

to(*device*)

Move mapped, default, and given children to device in place.

Return type

dmx.torch_stats.conditional.ConditionalDistribution

Parameters

device(*str* | *torch.device* | *None*)

class *dmx.torch_stats.conditional.ConditionalDistributionAccumulator*(*accumulator_map, default_accumulator=None, given_accumulator=None, keys=None, device=None*)

Bases: *TorchStatisticAccumulator*

Aggregate mapped, default, and given sufficient statistics.

The statistic is (mapped_stats, default_stat, given_stat). Each mapped child receives only its selector group's weights; the given accumulator receives the complete weight tensor of shape (N,).

Parameters

- **accumulator_map** (*Dict[Any, TorchStatisticAccumulator]*)
- **default_accumulator** (*TorchStatisticAccumulator* | *None*)
- **given_accumulator** (*TorchStatisticAccumulator* | *None*)
- **keys** (*str* | *None*)
- **device** (*torch.device* | *None*)

acc_to_encoder()

Create a grouped encoder from all child accumulators.

Return type

dmx.torch_stats.conditional.ConditionalDistributionDataEncoder

combine(*suff_stat*)

Merge mapped, default, and given sufficient statistics.

Return type

dmx.torch_stats.conditional.ConditionalDistributionAccumulator

Parameters**suff_stat** (*Tuple*[*Dict*[*Any*, *SS0*], *SS1* | *None*, *SS2* | *None*])**from_value**(*x*)

Replace mapped, default, and given statistics from a tuple.

Return type*dmx.torch_stats.conditional.ConditionalDistributionAccumulator***Parameters****x** (*Tuple*[*Dict*[*Any*, *SS0*], *SS1* | *None*, *SS1* | *None*])**key_merge**(*stats_dict*)

Delegate keyed merges to mapped, default, and given children.

Return type*None***Parameters****stats_dict** (*Dict*[*str*, *Any*])**key_replace**(*stats_dict*)

Delegate keyed replacement to mapped, default, and given children.

Return type*None***Parameters****stats_dict** (*Dict*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *tng*)

Initialize grouped children with deterministic child generators.

Return type*None***Parameters**

- **x** (*ConditionalTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator*)

seq_update(*x*, *weights*, *estimate*)

Update grouped children with sliced weights and matching estimates.

Return type*None***Parameters**

- **x** (*ConditionalTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **estimate** (*ConditionalDistribution*)

value()

Return (mapped_stats, default_stat, given_stat).

Return type*typing.Tuple*[*typing.Dict*[*typing.Any*, *typing.Any*], *typing.Optional*[*typing.Any*], *typing.Optional*[*typing.Any*]]

```
class dmx.torch_stats.conditional.ConditionalDistributionAccumulatorFactory(factory_map, de-
                                                                    fault_factory=NoneAccumulatorFac
                                                                    given_factory=NoneAccumulatorFac
                                                                    keys=None)
```

Bases: [TorchStatisticAccumulatorFactory](#)

Create mapped, default, and given child accumulators.

Parameters

- **factory_map** (*Dict*[*T0*, [TorchStatisticAccumulatorFactory](#)])
- **default_factory** ([TorchStatisticAccumulatorFactory](#))
- **given_factory** ([TorchStatisticAccumulatorFactory](#))
- **keys** (*str* | *None*)

```
make(device=None)
```

Create all child accumulators and associate the wrapper with device.

Return type

[dmx.torch_stats.conditional.ConditionalDistributionAccumulator](#)

Parameters

device (*torch.device* | *None*)

```
class dmx.torch_stats.conditional.ConditionalDistributionDataEncoder(encoder_map, de-
                                                                    fault_encoder=NoneDataEncoder(),
                                                                    given_encoder=NoneDataEncoder())
```

Bases: [TorchSequenceEncoder](#)

Group pairs by selector and encode dependent and given values.

Parameters

- **encoder_map** (*Dict*[*Any*, [TorchSequenceEncoder](#)])
- **default_encoder** ([TorchSequenceEncoder](#))
- **given_encoder** ([TorchSequenceEncoder](#))

```
seq_encode(x, device=None)
```

Encode N pairs into grouped dependent and selector sequences.

The data tuple is (N, values, encoded_groups, indices, given). values identifies each selector group. Every indices[j] is an integer tensor of shape (N_j,) locating that group in the original order, and encoded_groups[j] encodes its N_j dependent values. given encodes all N selectors. All child encoders receive the same device argument.

Return type

[dmx.torch_stats.conditional.ConditionalTorchEncodedSequence](#)

Parameters

- **x** (*List*[*Tuple*[*T0*, *T1*]])
- **device** (*torch.device* | *None*)

```
class dmx.torch_stats.conditional.ConditionalDistributionEstimator(estimator_map, de-
                                                                    fault_estimator=NoneEstimator(),
                                                                    given_estimator=NoneEstimator(),
                                                                    keys=None)
```

Bases: [TorchParameterEstimator](#)

Estimate mapped, default, and given child distributions separately.

Parameters

- **estimator_map** (*Dict*[Any, [TorchParameterEstimator](#)])
- **default_estimator** ([TorchParameterEstimator](#) | *None*)
- **given_estimator** ([TorchParameterEstimator](#) | *None*)
- **keys** (*str* | *None*)

accumulator_factory()

Create corresponding mapped, default, and given factories.

Return type

[dmx.torch_stats.conditional.ConditionalDistributionAccumulatorFactory](#)

estimate(*nobs*, *suff_stat*, *device=None*)

Estimate all children from the three-part statistic.

Default and given estimates receive *device*. Mapped estimators use their existing default device behavior; the returned wrapper records device.

Return type

[dmx.torch_stats.conditional.ConditionalDistribution](#)

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*Dict*[Any, *SS0*], *SS1* | *None*, *SS2* | *None*])
- **device** (*torch.device* | *None*)

class [dmx.torch_stats.conditional.ConditionalDistributionSampler](#)(*dist*, *seed=None*)

Bases: [ConditionalSampler](#), [DistributionSampler](#)

Sample complete pairs or dependent values conditional on a selector.

Parameters

- **dist** ([ConditionalDistribution](#))
- **seed** (*int* | *None*)

sample(*size=None*)

Draw one pair or a list of *size* independent pairs.

Return type

typing.Union[*typing.Tuple*[*typing.Any*, *typing.Any*], *typing.List*[*typing.Tuple*[*typing.Any*, *typing.Any*]]]

Parameters

size (*int* | *None*)

sample_given(*x*)

Draw a dependent value for selector *x* or from the default child.

Return type

typing.Any

Parameters**x** (*T0*)**single_sample()**

Draw a selector from the given model and then its dependent value.

Return type

typing.Tuple[typing.Any, typing.Any]

class dmx.torch_stats.conditional.**ConditionalTorchEncodedSequence**(*data, device=None*)

Bases: [*TorchEncodedSequence*](#)

Store grouped conditional encodings for N original observations.

Parameters

- **data** (*Tuple[int, Tuple[Any, ...], Tuple[TorchEncodedSequence, ...], Tuple[torch.Tensor, ...], TorchEncodedSequence]*)
- **device** (*torch.device | None*)

Null

Provide torch-backed neutral placeholders for absent model components.

Defines the NullDistribution, NullSampler, NullAccumulatorFactory, NullAccumulator, NullEstimator, and the NullDataEncoder classes for use with pysparkplug.

The scalar density is one and the log density is zero for any value. Encoding discards its input, and encoded scoring returns a fixed one-element zero tensor rather than preserving batch size. `to` only updates device metadata; the placeholder owns no parameter tensors. Sampling always returns `None` and the accumulator's invariant sufficient statistic is `None`. These contracts match `dmx.stats.null_dist` with torch device-aware wrappers.

class dmx.torch_stats.null_dist.**NullAccumulator**(*keys=None, device=None*)

Bases: [*TorchStatisticAccumulator*](#)

Implement accumulation with an invariant `None` statistic.

Parameters

- **keys** (*str | None*)
- **device** (*torch.device | None*)

acc_to_encoder()

Create an encoder that discards every observation.

Return type[*dmx.torch_stats.null_dist.NullDataEncoder*](#)**combine**(*suff_stat*)

Ignore another statistic and return this accumulator.

Return type[*dmx.torch_stats.null_dist.NullAccumulator*](#)**Parameters****suff_stat** (*Any | None*)**from_value**(*x*)

Ignore a supplied value and return this accumulator.

Return type`dmx.torch_stats.null_dist.NullAccumulator`**Parameters**`x` (*Any* | *None*)**key_merge**(*stats_dict*)Register *None* under the configured key if absent.**Return type***None***Parameters**`stats_dict` (*Dict*[*str*, *Any*])**key_replace**(*stats_dict*)

Leave this stateless accumulator unchanged.

Return type*None***Parameters**`stats_dict` (*Dict*[*str*, *Any*])**seq_initialize**(*x*, *weights*, *tng*)

Ignore an encoded initialization sequence and generator.

Return type*None***Parameters**

- `x` (*NullTorchEncodedSequence*)
- `weights` (*torch.Tensor*)
- `tng` (*torch.Generator* | *None*)

seq_update(*x*, *weights*, *estimate*)

Ignore an encoded sequence, weights, and estimate.

Return type*None***Parameters**

- `x` (*NullTorchEncodedSequence*)
- `weights` (*torch.Tensor*)
- `estimate` (*NullDistribution* | *None*)

value()Return the invariant *None* sufficient statistic.**Return type***None***class** `dmx.torch_stats.null_dist.NullAccumulatorFactory`(*keys=None*)Bases: *TorchStatisticAccumulatorFactory*

Create stateless null accumulators.

Parameters`keys` (*str* | *None*)

make(*device=None*)

Create a null accumulator associated with device.

Return type

dmx.torch_stats.null_dist.NullAccumulator

Parameters

device (*torch.device* | *None*)

class *dmx.torch_stats.null_dist.NullDataEncoder*

Bases: *TorchSequenceEncoder*

Discard observations while retaining encoded device metadata.

seq_encode(*x, device=None*)

Discard *x* and return an encoding containing *None*.

Return type

dmx.torch_stats.null_dist.NullTorchEncodedSequence

Parameters

- **x** (*Any*)
- **device** (*torch.device* | *None*)

class *dmx.torch_stats.null_dist.NullDistribution*(*device=None*)

Bases: *TorchProbabilityDistribution*

Provide a neutral likelihood factor for an absent distribution.

Parameters

device (*str* | *torch.device* | *None*)

density(*x*)

Return one for any scalar observation.

Return type

float

Parameters

x (*Any* | *None*)

dist_to_encoder()

Create an encoder that discards its input.

Return type

dmx.torch_stats.null_dist.NullDataEncoder

estimator(*pseudo_count=None, _device=None*)

Create an estimator whose result is another null distribution.

Return type

dmx.torch_stats.null_dist.NullEstimator

Parameters

- **pseudo_count** (*float* | *None*)
- **_device** (*str* | *None*)

log_density(*x*)

Return zero for any scalar observation.

Return type

float

Parameters**x** (*Any* | *None*)**sampler(*seed=None*)**Create a sampler that always returns *None*.**Return type***dmx.torch_stats.null_dist.NullSampler***Parameters****seed** (*int* | *None*)**seq_log_density(*x*)**

Return a one-element zero tensor on the model device.

Return type

torch.Tensor

Parameters**x** (*NullTorchEncodedSequence*)**to(*device*)**

Update device metadata in place and return self.

Return type*dmx.torch_stats.null_dist.NullDistribution***Parameters****device** (*str* | *torch.device* | *None*)**class** *dmx.torch_stats.null_dist.NullEstimator*(*pseudo_count=None*, *suff_stat=None*, *keys=None*)Bases: *TorchParameterEstimator*

Implement estimation by returning a null distribution.

Parameters

- **pseudo_count** (*float* | *None*)
- **suff_stat** (*Any* | *None*)
- **keys** (*str* | *None*)

accumulator_factory()

Create a null accumulator factory retaining the merge key.

Return type*dmx.torch_stats.null_dist.NullAccumulatorFactory***estimate(*nobs*, *suff_stat=None*, *device=None*)**

Return a null distribution on device.

Return type*dmx.torch_stats.null_dist.NullDistribution***Parameters**

- **nobs** (*float* | *None*)

- **suff_stat** (*Any* | *None*)
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.null_dist.NullSampler`(*dist*, *seed=None*)

Bases: [*DistributionSampler*](#)

Implement the sampler protocol by always returning *None*.

Parameters

- **dist** ([*NullDistribution*](#))
- **seed** (*int* | *None*)

sample(*size=None*)

Return *None* regardless of *size*.

Return type

None

Parameters

size (*int* | *None*)

class `dmx.torch_stats.null_dist.NullTorchEncodedSequence`(*data*, *device*)

Bases: [*TorchEncodedSequence*](#)

Store *None* plus the device associated with discarded input.

Parameters

- **data** (*Any* | *None*)
- **device** (*torch.device* | *None*)

Sequence

Model ordered variable-length sequences of independent observations.

dist supplies item factors and optional *len_dist* supplies the mass of the non-negative integer length. With *len_normalized=True*, only the item log-density sum is divided by sequence length. Encoding flattens *N* parent sequences containing *M* total items and records their parent indices and inverse lengths. Item and length operations remain separate throughout sampling, accumulation, and estimation, matching `dmx.stats.sequence`. Child encoders and to calls receive the requested torch device.

class `dmx.torch_stats.sequence.SequenceAccumulator`(*accumulator*,
len_accumulator=NullAccumulator(),
len_normalized=False, *keys=None*,
device=None)

Bases: [*TorchStatisticAccumulator*](#)

Accumulate separate item and length sufficient statistics.

Item observations are flattened in sequence order. When normalized, each item receives its parent weight divided by parent length; the length child always receives the original parent weight.

Parameters

- **accumulator** ([*TorchStatisticAccumulator*](#))
- **len_accumulator** ([*TorchStatisticAccumulator*](#))
- **len_normalized** (*bool* | *None*)
- **keys** (*str* | *None*)

- **device** (*str* | *torch.device* | *None*)

acc_to_encoder()

Create an encoder from the item and length accumulators.

Return type

dmx.torch_stats.sequence.SequenceDataEncoder

combine(*suff_stat*)

Merge the (*item_stat*, *length_stat*) pair.

Return type

dmx.torch_stats.sequence.SequenceAccumulator

Parameters

suff_stat (*Tuple*[*SS1*, *SS2* | *None*])

from_value(*x*)

Restore item and length statistics from a paired value.

Return type

dmx.torch_stats.sequence.SequenceAccumulator

Parameters

x (*Tuple*[*SS1*, *SS2* | *None*])

key_merge(*stats_dict*)

Merge the pair by wrapper key, then recurse into child keys.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

key_replace(*stats_dict*)

Replace the pair by wrapper key, then recurse into child keys.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

seq_initialize(*x*, *weights*, *tng*)

Initialize item and length children from N encoded sequences.

Return type

None

Parameters

- **x** (*SequenceTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator*)

seq_update(*x*, *weights*, *estimate*)

Update item and length children from N encoded sequences.

Return type

None

Parameters

- **x** ([SequenceTorchEncodedSequence](#))
- **weights** (`torch.Tensor`)
- **estimate** ([SequenceDistribution](#) | `None`)

value()

Return the item and length sufficient-statistic pair.

Return type

`typing.Tuple[typing.Any, typing.Optional[typing.Any]]`

```
class dmx.torch_stats.sequence.SequenceAccumulatorFactory(dist_factory,
                                                         len_factory=NullAccumulatorFactory(),
                                                         len_normalized=False, keys=None)
```

Bases: [TorchStatisticAccumulatorFactory](#)

SequenceAccumulatorFactory object for creating SequenceAccumulator objects.

Parameters

- **dist_factory** ([TorchStatisticAccumulatorFactory](#))
- **len_factory** ([TorchStatisticAccumulatorFactory](#))
- **len_normalized** (`bool` | `None`)
- **keys** (`str` | `None`)

make(device=None)

Create both child accumulators on device.

Return type

[dmx.torch_stats.sequence.SequenceAccumulator](#)

Parameters

device (`torch.device` | `None`)

```
class dmx.torch_stats.sequence.SequenceDataEncoder(encoders)
```

Bases: [TorchSequenceEncoder](#)

Flatten parent sequences and encode their items and lengths separately.

Parameters

encoders (`Tuple`[[TorchSequenceEncoder](#), [TorchSequenceEncoder](#)])

seq_encode(x, device=None)

Encode N sequences containing M total items.

The tuple is (indices, inverse_lengths, nonempty, items, lengths). indices has shape (M,) and maps flat items to parents; inverse_lengths and nonempty have shape (N,). The child item encoding represents M values and the length encoding represents N integers. Every tensor or child encoding receives device; floating inverse lengths use the vector-helper dtype.

Return type

[dmx.torch_stats.sequence.SequenceTorchEncodedSequence](#)

Parameters

- **x** (`Sequence`[`Sequence`[`T`]])
- **device** (`torch.device` | `None`)

```
class dmx.torch_stats.sequence.SequenceDistribution(dist, len_dist=NullDistribution(),  
                                                  len_normalized=False, device=None)
```

Bases: [*TorchProbabilityDistribution*](#)

Model a sequence using independent item and optional length factors.

Parameters

- **dist** ([*TorchProbabilityDistribution*](#))
- **len_dist** ([*TorchProbabilityDistribution*](#) / *None*)
- **len_normalized** (*bool* / *None*)
- **device** (*torch.device* / *None*)

density(*x*)

Evaluate the density of *SequenceDistribution* at observed sequence *x*.

Return type

float

Parameters

x (*Sequence*[*T*])

dist_to_encoder()

Create an encoder composed from the item and length encoders.

Return type

[*dmx.torch_stats.sequence.SequenceDataEncoder*](#)

estimator(*pseudo_count*=*None*)

Create separate item and length estimators using *pseudo_count*.

Return type

[*dmx.torch_stats.sequence.SequenceEstimator*](#)

Parameters

pseudo_count (*float* / *None*)

log_density(*x*)

Evaluate the log-density of *SequenceDistribution* at observed sequence *x*.

Return type

float

Parameters

x (*Sequence*[*T*])

sampler(*seed*=*None*)

Create item and length samplers, requiring a non-null length model.

Return type

[*dmx.torch_stats.sequence.SequenceSampler*](#)

Parameters

seed (*int* / *None*)

seq_log_density(*x*)

Aggregate flat item scores into one log density per parent sequence.

The result has shape (N,). For nonempty data it follows the item score device; the all-empty branch uses the vector helper's default device allocation.

Return type
`torch.Tensor`

Parameters
x (`SequenceTorchEncodedSequence`)

to(*device*)

Move item and length children to device in place.

Return type
`dmx.torch_stats.sequence.SequenceDistribution`

Parameters
device (`str` / `torch.device` / `None`)

```
class dmx.torch_stats.sequence.SequenceEstimator(estimator, len_estimator=NoneEstimator(),
                                                  len_dist=NoneDistribution(), len_normalized=False,
                                                  keys=None)
```

Bases: `TorchParameterEstimator`

Estimate item and length children from paired sufficient statistics.

Parameters

- **estimator** (`TorchParameterEstimator`)
- **len_estimator** (`TorchParameterEstimator` / `None`)
- **len_dist** (`TorchProbabilityDistribution` / `None`)
- **len_normalized** (`bool` / `None`)
- **keys** (`str` / `None`)

accumulator_factory()

Create a factory from the item and length estimator factories.

Return type
`dmx.torch_stats.sequence.SequenceAccumulatorFactory`

estimate(*nobs*, *suff_stat*, *device=None*)

Estimate item and length children from their paired statistics.

Length estimates receive device; item estimates use their existing default device behavior. The returned wrapper records device.

Return type
`dmx.torch_stats.sequence.SequenceDistribution`

Parameters

- **nobs** (`float` / `None`)
- **suff_stat** (`Tuple[Any, Any]` / `None`)
- **device** (`torch.device` / `None`)

```
class dmx.torch_stats.sequence.SequenceSampler(dist, len_dist, seed=None)
```

Bases: `DistributionSampler`

SequenceSampler object for sampling from an SequenceDistribution instance.

Parameters

- **dist** (`TorchProbabilityDistribution`)

- **len_dist** ([TorchProbabilityDistribution](#))
- **seed** (*int* | *None*)

sample (*size=None*)

Draw one variable-length sequence or size such sequences.

Return type

[typing.List\[typing.Any\]](#)

Parameters

size (*int* | *None*)

class `dmx.torch_stats.sequence.SequenceTorchEncodedSequence` (*data*, *device=None*)

Bases: [TorchEncodedSequence](#)

Store flattened item, parent-index, and length encodings.

Parameters

- **data** ([Tuple\[torch.Tensor, torch.Tensor, torch.Tensor, TorchEncodedSequence, TorchEncodedSequence\]](#))
- **device** (*torch.device* | *None*)

1.5.4 Torch Mixture, Topic, and Sequential Models

Heterogeneous mixture

Provide torch-backed finite mixtures with heterogeneous components.

A latent component Z is drawn from K weights and the observation is drawn from `components[Z]`. Components accept the same raw observation type but may require different encoders. Equivalent encoders are grouped so each distinct representation is computed once. Component scores and posterior responsibilities have shape (N, K) and marginal scores have shape (N,). Parameters and children move with `to`; floating tensors use the vector-helper dtype, normally float64 and float32 on MPS. Sampling and accumulated counts use CPU NumPy arrays. The filename spelling is preserved; terminology follows `dmx.stats.heterogeneous_mixture`.

class `dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulator` (*accumulators*,
keys=(None,
None),
device=None)

Bases: [TorchStatisticAccumulator](#)

Accumulate component counts and responsibility-weighted child statistics.

Parameters

- **accumulators** ([Sequence\[TorchStatisticAccumulator\]](#))
- **keys** ([Tuple\[str | None, str | None\]](#))
- **device** (*torch.device* | *None*)

acc_to_encoder ()

Create grouped encoders from all component accumulators.

Return type

[dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDataEncoder](#)

combine(*suff_stat*)

Merge component counts and the length-K child-statistic tuple.

Return type

dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulator

Parameters

suff_stat (*Tuple*[*ndarray*, *Tuple*[*Any*, ...]])

from_value(*x*)

Replace component counts and child sufficient statistics.

Return type

dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulator

Parameters

x (*Tuple*[*ndarray*, *Tuple*[*Any*, ...]])

key_merge(*stats_dict*)

Merge weight, component-list, and recursive child keys.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

key_replace(*stats_dict*)

Replace weight, component-list, and recursive child keys.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

seq_initialize(*x*, *weights*, *tng*)

Randomly split (N,) weights across components with Dirichlet draws.

Return type

None

Parameters

- **x** (*HeterogeneousMixtureTorchSequence*)
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator*)

seq_update(*x*, *weights*, *estimate*)

Update children using an (N, K) responsibility matrix.

Return type

None

Parameters

- **x** (*HeterogeneousMixtureTorchSequence*)
- **weights** (*torch.Tensor*)
- **estimate** (*HeterogeneousMixtureDistribution*)

value()

Return CPU component counts and child sufficient statistics.

Return type

`typing.Tuple[numpy.ndarray, typing.Tuple[typing.Any, ...]]`

class `dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulatorFactory`(*factories*,
keys=(None,
None))

Bases: `TorchStatisticAccumulatorFactory`

Create heterogeneous mixture accumulators from component factories.

Parameters

- **factories** (`Sequence[TorchStatisticAccumulatorFactory]`)
- **keys** (`Tuple[str | None, str | None]`)

make(*device=None*)

Create component accumulators on device when supplied.

Return type

`dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulator`

Parameters

device (`torch.device | None`)

class `dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDataEncoder`(*encoders*)

Bases: `TorchSequenceEncoder`

Encode observations once per distinct component encoder.

Encoders are grouped by their string representation. Each group stores the component indices that consume its encoded sequence.

Parameters

encoders (`List[TorchSequenceEncoder]`)

seq_encode(*x*, *device=None*)

Encode N observations once for each distinct encoder group.

The result is (`component_groups`, `encoded_groups`). Each component group is a CPU NumPy index array, and each matching child encoding represents all N observations on device.

Return type

`dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureTorchSequence`

Parameters

- **x** (`Sequence[T]`)
- **device** (`torch.device | None`)

class `dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDistribution`(*components*, *w*,
device=None)

Bases: `TorchProbabilityDistribution`

Represent a finite mixture whose components use distinct encodings.

Parameters

- **components** (`Sequence[TorchProbabilityDistribution]`)

- **w** (*ndarray* | *List[float]* | *torch.Tensor*)
- **device** (*torch.device* | *None*)

component_log_density(*x*)

Return K unweighted component log densities for one observation.

Return type
torch.Tensor

Parameters
x (*T*)

density(*x*)

Evaluate the marginal density of one observation.

Return type
float

Parameters
x (*T*)

dist_to_encoder()

Create an encoder that groups equivalent component encoders.

Return type
dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDataEncoder

estimator(*pseudo_count=None*)

Create component estimators and optional weight smoothing.

Return type
dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureEstimator

Parameters
pseudo_count (*float* | *None*)

log_density(*x*)

Marginalize the latent component with a torch log-sum-exp.

Return type
float

Parameters
x (*T*)

posterior(*x*)

Return length-K posterior component responsibilities.

Return type
torch.Tensor

Parameters
x (*T*)

sampler(*seed=None*)

Create a sampler for latent components and their observations.

Return type
dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureSampler

Parameters
seed (*int* | *None*)

seq_component_log_density(*x*)

Return unweighted component log densities with shape (N, K).

Return type

torch.Tensor

Parameters**x** ([HeterogeneousMixtureTorchSequence](#))**seq_log_density**(*x*)

Return marginal mixture log densities with shape (N,).

Return type

torch.Tensor

Parameters**x** ([HeterogeneousMixtureTorchSequence](#))**seq_posterior**(*x*)

Return posterior responsibilities with shape (N, K).

Return type

torch.Tensor

Parameters**x** ([HeterogeneousMixtureTorchSequence](#))**to**(*device*)

Move weights and every component model to device in place.

Return type[dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDistribution](#)**Parameters****device** (*str* | *torch.device* | *None*)

```
class dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureEstimator(estimators,  
                                                                           fixed_weights=None,  
                                                                           suff_stat=None,  
                                                                           pseudo_count=None,  
                                                                           keys=(None, None))
```

Bases: [TorchParameterEstimator](#)

Estimate heterogeneous component models and categorical weights.

Parameters

- **estimators** (*Sequence*[[TorchParameterEstimator](#)])
- **fixed_weights** (*List*[*float*] | *torch.Tensor* | *None*)
- **suff_stat** (*ndarray* | *None*)
- **pseudo_count** (*float* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*])

accumulator_factory()

Create a factory from the component estimator factories.

Return type[dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulatorFactory](#)

estimate(*nobs*, *suff_stat*, *device=None*)

Estimate components from responsibilities and normalize weights.

Return type

`dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDistribution`

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *Tuple*[*Any*, ...]])
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`

Draw a latent component and then an observation from that component.

Parameters

- **dist** (`HeterogeneousMixtureDistribution`)
- **seed** (*int* | *None*)

sample(*size=None*)

Draw one observation or a list of *size* observations.

Return type

`typing.Union[typing.Any, typing.List[typing.Any]]`

Parameters

size (*int* | *None*)

class `dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureTorchSequence`(*data*,
device=None)

Bases: `TorchEncodedSequence`

Store component-index groups and their full observation encodings.

Parameters

- **data** (*Tuple*[*List*[*ndarray*], *List*[`TorchEncodedSequence`]])
- **device** (*torch.device* | *None*)

Hidden Markov model

Create, estimate, and sample finite-state hidden Markov models.

The torch implementation represents an observation as a Python value and an observation sequence as `List[T]`. Batched computations use `HiddenMarkovTorchSequence`, which packs variable-length sequences by time step. Model parameters and forward-backward work arrays are torch tensors; exported HMM count statistics are NumPy arrays.

The likelihood, posterior, and estimation paths implement one emission distribution per hidden state. The optional `taus` topic-mixture matrix is used by sampling only, unlike the corresponding NumPy implementation, which also supports topic mixtures in scalar likelihood calculations.

class `dmx.torch_stats.hmm.HiddenMarkovAccumulator`(*accumulators*, *len_accumulator=None*,
keys=(None, None, None), *device=None*)

Bases: `TorchStatisticAccumulator`

Accumulate sufficient statistics for HMM parameter estimation.

For K states, initial counts have shape $(K,)$, transition counts have shape (K, K) , and posterior state counts have shape $(K,)$. These counts are NumPy float64 arrays on CPU. Forward-backward work and posterior emission weights are torch tensors on the accumulator device; component and length statistics retain their own accumulator-defined representations.

`seq_initialize` assigns states randomly, while `seq_update` computes posterior state and transition weights under a current estimate. Input `weights` has shape $(N,)$ for N encoded sequences and is expanded across their observations.

Parameters

- **accumulators** (*Sequence*[*TorchStatisticAccumulator*])
- **len_accumulator** (*TorchStatisticAccumulator* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*, *str* | *None*])
- **device** (*torch.device* | *None*)

acc_to_encoder()

Create an encoder compatible with this accumulator.

Return type

dmx.torch_stats.hmm.HiddenMarkovDataEncoder

Returns

An HMM encoder built from the first emission accumulator and the length accumulator.

combine(suff_stat)

Add another HMM sufficient-statistic value in place.

Parameters

suff_stat – Tuple returned by *value()*.

Return type

dmx.torch_stats.hmm.HiddenMarkovAccumulator

Returns

This accumulator after combining statistics.

Parameters

suff_stat (*Tuple*[*int*, *ndarray*, *ndarray*, *ndarray*, *Sequence*[*T1*], *T2* | *None*])

from_value(x)

Replace accumulated statistics from a serialized value.

Parameters

x – Tuple in the format returned by *value()*.

Return type

dmx.torch_stats.hmm.HiddenMarkovAccumulator

Returns

This accumulator after replacement.

Parameters

x (*Tuple*[*int*, *ndarray*, *ndarray*, *ndarray*, *Sequence*[*T1*], *T2* | *None*])

key_merge(stats_dict)

Merge configured sufficient statistics into a shared dictionary.

Parameters

stats_dict – Mutable mapping keyed by configured statistic names.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**key_replace(stats_dict)**

Replace configured statistics from a shared dictionary.

Parameters**stats_dict** – Mapping keyed by configured statistic names.**Return type**

None

Parameters**stats_dict** (*Dict[str, Any]*)**seq_initialize(x, weights, tng)**

Initialize sufficient statistics with random hidden states.

Each packed observation receives a uniformly sampled state. Sequence weights are then accumulated into initial, state, transition, emission, and length statistics.

Parameters

- **x** – Encoded batch of N observation sequences.
- **weights** – Floating-point sequence weights with shape (N,) on a device compatible with x.
- **tng** – Torch generator used for random state assignments.

Return type

None

Parameters

- **x** ([HiddenMarkovTorchSequence](#))
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator*)

seq_update(x, weights, estimate)

Accumulate expected sufficient statistics under an HMM estimate.

A scaled forward-backward calculation produces posterior state and adjacent-state weights. It accumulates weighted expected initial counts, state occupancies, transition counts, emission statistics, and length statistics. The tensor calculations use the accumulator device; HMM count arrays are copied to CPU NumPy arrays.

Parameters

- **x** – Encoded batch of N observation sequences.
- **weights** – Floating-point sequence weights with shape (N,).
- **estimate** – Current one-emission-per-state HMM estimate.

Return type

None

Parameters

- **x** ([HiddenMarkovTorchSequence](#))

- **weights** (*torch.Tensor*)
- **estimate** (*HiddenMarkovModelDistribution*)

value()

Return the accumulated HMM sufficient statistics.

Return type

typing.Tuple[*int*, *numpy.ndarray*, *numpy.ndarray*, *numpy.ndarray*, *typing.Sequence*[*typing.Any*], *typing.Optional*[*typing.Any*]]

Returns

A tuple containing the number of states, initial counts of shape (K,), state counts of shape (K,), transition counts of shape (K, K), K emission-statistic values, and the optional length-statistic value. The three HMM count arrays are CPU NumPy float64 arrays.

```
class dmx.torch_stats.hmm.HiddenMarkovAccumulatorFactory(factories,  
                                                         len_factory=NullAccumulatorFactory(),  
                                                         keys=(None, None, None))
```

Bases: *TorchStatisticAccumulatorFactory*

Create HMM accumulators from component accumulator factories.

Parameters

- **factories** (*Sequence*[*TorchStatisticAccumulatorFactory*])
- **len_factory** (*TorchStatisticAccumulatorFactory*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*, *str* | *None*] | *None*)

make(*device*=None)

Create an HMM accumulator.

The HMM work tensors use *device*. Component factories are invoked without a *device* argument and therefore retain their own defaults.

Parameters

device – Device for HMM forward-backward work tensors.

Return type

dmx.torch_stats.hmm.HiddenMarkovAccumulator

Returns

A fresh HMM accumulator.

Parameters

device (*torch.device* | *None*)

```
class dmx.torch_stats.hmm.HiddenMarkovDataEncoder(emission_encoder,  
                                                  len_encoder=NullDataEncoder())
```

Bases: *TorchSequenceEncoder*

Encode batches of variable-length HMM observation sequences.

A batch of N sequences is packed by time step. If L is the maximum sequence length and M is the total observation count, integer indexing tensors describe the mapping between the padded (N, L) view and the packed M observations. Integer tensors use the torch utility integer dtype, currently *torch.int32*, on the requested device.

Parameters

- **emission_encoder** (*TorchSequenceEncoder*)
- **len_encoder** (*TorchSequenceEncoder* | *None*)

seq_encode(*x*, *device=None*)

Encode a batch of variable-length observation sequences.

Observations are flattened in time-major order before delegation to the emission encoder. For *N* sequences, maximum length *L*, and *M* total observations, the payload contains *len_vec* with shape (*N*,), *idx_mat* with shape (*N*, *L*), *idx_bands* with shape (*L*, 2), and *idx_vec* with shape (*M*,). *idx_mat* stores packed indices and uses -1 for padding. *has_next*[*t*] indexes the packed observations at time *t* whose sequences continue.

Parameters

- **x** – Batch of observation sequences.
- **device** – Device for index tensors and delegated encodings. *None* uses the called encoders' default behavior while the returned container records CPU.

Return type

dmx.torch_stats.hmm.HiddenMarkovTorchSequence

Returns

The packed HMM batch, including delegated emission and length encodings.

Parameters

- **x** (*List[List[T]]*)
- **device** (*torch.device* | *None*)

```
class dmx.torch_stats.hmm.HiddenMarkovEstimator(estimators, len_estimator=None,
                                              pseudo_count=(None, None), keys=(None, None, None))
```

Bases: *TorchParameterEstimator*

Estimate HMM parameters from aggregated sufficient statistics.

Emission estimators receive their posterior state counts as effective observation counts. The length estimator receives the caller's *nobs*. Initial and transition probabilities are normalized from CPU NumPy count arrays, with optional additive pseudo-counts.

Parameters

- **estimators** (*List[TorchParameterEstimator]*)
- **len_estimator** (*TorchParameterEstimator* | *None*)
- **pseudo_count** (*Tuple[float | None, float | None]* | *None*)
- **keys** (*Tuple[str | None, str | None, str | None]* | *None*)

accumulator_factory()

Create a factory for compatible HMM accumulators.

Return type

dmx.torch_stats.hmm.HiddenMarkovAccumulatorFactory

Returns

A factory wrapping the emission and length accumulator factories.

estimate(*nobs*, *suff_stat*, *device=None*)

Estimate an HMM from aggregated sufficient statistics.

Initial counts are normalized globally. Transition counts are normalized by row; without smoothing, a row with no transitions remains all zeros. A supplied initial pseudo-count is spread uniformly over states, and

a transition pseudo-count uniformly over the full K by K matrix. Topic mixtures are not estimated, so the returned model always has `taus=None`.

Parameters

- **nobs** – Effective number of sequences, forwarded to the length estimator.
- **suff_stat** – Tuple from `HiddenMarkovAccumulator.value()`.
- **device** – Device for the returned HMM parameters and forwarded length estimate. `None` selects CPU for the HMM.

Return type

`dmx.torch_stats.hmm.HiddenMarkovModelDistribution`

Returns

The estimated one-emission-per-state HMM.

Parameters

- **nobs** (`float` | `None`)
- **suff_stat** (`Tuple[int, ndarray, ndarray, ndarray, List[T1], T2 | None]`)
- **device** (`torch.device` | `None`)

```
class dmx.torch_stats.hmm.HiddenMarkovModelDistribution(topics, w, transitions, taus=None,
                                                         len_dist=None, terminal_values=None,
                                                         device=None)
```

Bases: `TorchProbabilityDistribution`

Represent a finite-state HMM with observations of generic type `T`.

For K hidden states, `w[k]` is the initial-state probability and `transitions[i, j]` is the probability of moving from state `i` to state `j`. The vectorized likelihood, posterior, Viterbi, and estimation methods interpret `topics[k]` as the emission distribution for state `k` and therefore require `len(topics) == K`. If `taus` is supplied, sampling instead uses row `k` as topic-mixture weights for state `k`.

Floating-point parameters use the dtype selected by `dmx.torch_utils.vector`: normally `torch.float64`, but `torch.float32` on MPS. They are created on device and moved by `to()`. The length distribution is constructed on that device when it is omitted, but a supplied length distribution is not moved by `to()`.

Parameters

- **topics** (`Sequence[TorchProbabilityDistribution]`)
- **w** (`Sequence[float]` | `ndarray`)
- **transitions** (`List[List[float]]` | `ndarray`)
- **taus** (`List[List[float]]` | `ndarray` | `None`)
- **len_dist** (`TorchProbabilityDistribution` | `None`)
- **terminal_values** (`Set[T]` | `None`)
- **device** (`torch.device` | `None`)

topics

Emission distributions, normally one per hidden state.

n_topics

Number of emission distributions.

n_states

Number of hidden states, inferred from `w`.

w

Initial-state probabilities with shape $(K,)$.

log_w

Elementwise logarithm of `w`, with shape $(K,)$.

transitions

Transition probabilities with shape (K, K) .

log_transitions

Elementwise logarithm of `transitions`.

taus

Optional sampling-only topic weights, normally shape (K, n_topics) .

log_taus

Elementwise logarithm of `taus`, when present.

has_topics

Whether `taus` was supplied.

len_dist

Distribution for observed sequence lengths.

terminal_values

Optional emissions that terminate sampling when no usable length sampler is present.

density(*x*)

Evaluate the density of one observed sequence.

Parameters

`x` – Observed emission sequence.

Return type

float

Returns

The marginal sequence density as a Python float.

Parameters

`x` (*List*[*T*])

dist_to_encoder()

Create an encoder for batches of HMM observation sequences.

Return type

`dmx.torch_stats.hmm.HiddenMarkovDataEncoder`

Returns

An encoder using the first emission distribution's encoder and the length distribution's encoder.

estimator(*pseudo_count=None*)

Create an estimator with matching emission and length estimators.

The same scalar pseudo-count is used for initial-state probabilities, transition probabilities, and delegated component estimators.

Parameters

pseudo_count – Optional additive smoothing mass.

Return type

[`dmx.torch\_stats.hmm.HiddenMarkovEstimator`](#)

Returns

An HMM parameter estimator.

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Evaluate the HMM log density of one observed sequence.

The hidden-state path is marginalized by the batched forward routine. An empty sequence contributes only its length log density.

Parameters

x – Observed emission sequence.

Return type

float

Returns

The marginal log density as a Python float.

Parameters

x (*List*[*T*])

sampler(*seed=None*)

Create a seeded sampler for independent HMM sequences.

Parameters

seed – Optional NumPy random seed.

Return type

[`dmx.torch\_stats.hmm.HiddenMarkovSampler`](#)

Returns

A sampler bound to this distribution.

Raises

RuntimeError – If neither sequence lengths nor terminal emissions can stop sampling.

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Evaluate log densities for a batch of observation sequences.

A scaled forward recursion marginalizes each hidden-state path. For a batch of *N* sequences, the returned tensor has shape (*N*,) on the model device and uses the model's floating-point dtype. Sequence length log densities are included. This path assumes one emission distribution per state and does not use *taus*.

Parameters

x – Batch encoded by [`HiddenMarkovDataEncoder`](#).

Return type

`torch.Tensor`

Returns

Marginal log densities, one per input sequence.

Raises

TypeError – If `x` is not a `HiddenMarkovTorchSequence`.

Parameters

x (`HiddenMarkovTorchSequence`)

to(*device*)

Move HMM parameters and emission distributions to a device.

The initial and transition tensors, optional topic weights, and all emission distributions are moved. The length distribution is retained as-is. Parameter dtypes are preserved.

Parameters

device – Target device. `None` retains the current model device.

Return type

`dmx.torch_stats.hmm.HiddenMarkovModelDistribution`

Returns

This distribution after the in-place move.

Parameters

device (`str` | `torch.device` | `None`)

viterbi(*x*)

Return per-time maximizing states from Viterbi score vectors.

The output is a floating-point tensor of shape `(L,)` on the model device for a length-`L` input. Each entry is the index maximizing the dynamic-programming score at that time. This implementation does not retain backpointers or perform traceback, so the result need not be the globally maximizing state path. The sequence-length distribution and optional `taus` matrix are not used.

Parameters

x – One nonempty observed emission sequence.

Return type

`torch.Tensor`

Returns

Per-time hidden-state indices stored in a floating-point tensor.

Parameters

x (`List[T]`)

class `dmx.torch_stats.hmm.HiddenMarkovSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`

Generate observation sequences from an HMM.

Hidden states are sampled with the NumPy-backed Markov-chain sampler. Each state's observation sampler is either its corresponding emission sampler or, when `taus` is present, a mixture over all topic samplers. Sampled values are ordinary Python objects rather than torch tensors.

Parameters

- **dist** (`HiddenMarkovModelDistribution`)
- **seed** (`int` | `None`)

sample(*size=None*)

Draw independent HMM observation sequences.

Parameters

size – Number of sequences. `None` returns one sequence.

Return type

`typing.Union[typing.List[typing.Any], typing.List[typing.List[typing.Any]]]`

Returns

One observation list, or a list of observation lists when `size` is provided.

Raises

RuntimeError – If the sampler has no stopping mechanism.

Parameters

size (`int` | `None`)

sample_seq(*size=None*)

Sample independent sequences using random sequence lengths.

Parameters

size – Number of sequences. `None` returns one sequence.

Return type

`typing.Union[typing.List[typing.Any], typing.List[typing.List[typing.Any]]]`

Returns

One observation list, or a list of observation lists when `size` is provided.

Parameters

size (`int` | `None`)

sample_terminal(*terminal_set*)

Sample through the first emission in a terminal set.

Parameters

terminal_set – Emission values that stop the sequence.

Return type

`typing.List[typing.TypeVar(T)]`

Returns

A sequence including its terminal emission.

Parameters

terminal_set (`Set[T]`)

class `dmx.torch_stats.hmm.HiddenMarkovTorchSequence`(*data, device=None*)

Bases: [`TorchEncodedSequence`](#)

Store a packed batch of variable-length HMM sequences.

`data` is `((M, L, idx_bands, has_next, len_vec, idx_mat, idx_vec, enc_data), len_enc)` for total observation count `M` and maximum length `L`. Index tensors reside on `device` and use `torch.int32`; the delegated emission and length encodings control their own tensor dtypes. The container records a `device` but does not move its payload.

Parameters

- **data** (`Tuple[Tuple[int, int, torch.Tensor, List[torch.Tensor], torch.Tensor, torch.Tensor, torch.Tensor, TorchEncodedSequence], TorchEncodedSequence]`)
- **device** (`torch.device` | `None`)

Integer probabilistic latent semantic indexing

Provide torch-backed integer probabilistic latent semantic indexing models.

An observation is a document identifier and sparse (term, count) pairs. Each token has a latent topic; document-topic and topic-term probabilities produce its marginal term probability. Encoded batches flatten M nonzero term entries from N documents, retaining document and observation indices. Model tensors move with `to` and use the vector-helper floating dtype, normally float64 and float32 on MPS; sufficient statistics and sampling are CPU NumPy based. This mirrors `dmx.stats.int_plsi`.

```
class dmx.torch_stats.int_plsi.IntegerPLSIAccumulator(num_vals, num_states, num_docs,
                                                    len_acc=NullAccumulator(), keys=(None,
                                                    None, None), device=None)
```

Bases: *TorchStatisticAccumulator*

Accumulate topic-term, document-topic, document, and length statistics.

Parameters

- **num_vals** (*int*)
- **num_states** (*int*)
- **num_docs** (*int*)
- **len_acc** (*TorchStatisticAccumulator* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*, *str* | *None*] | *None*)
- **device** (*str* | *torch.device* | *None*)

acc_to_encoder()

Create a sparse document encoder from the length accumulator.

Return type

dmx.torch_stats.int_plsi.IntegerPLSIDataEncoder

combine(suff_stat)

Merge topic-term, document-topic, document, and length statistics.

Return type

dmx.torch_stats.int_plsi.IntegerPLSIAccumulator

Parameters

suff_stat (*Tuple*[*ndarray*, *ndarray*, *ndarray*, *SS1* | *None*])

from_value(x)

Replace all PLSI sufficient statistics from a tuple.

Return type

dmx.torch_stats.int_plsi.IntegerPLSIAccumulator

Parameters

x (*Tuple*[*ndarray*, *ndarray*, *ndarray*, *SS1* | *None*])

key_merge(stats_dict)

Merge separately keyed topic-term, document-topic, and document counts.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

key_replace(*stats_dict*)

Replace separately keyed statistics and recurse into the length child.

Return type

None

Parameters

stats_dict (*Dict*[*str*, *Any*])

seq_initialize(*x*, *weights*, *tng*)

Randomly allocate each flat term count across topics for initialization.

Return type

None

Parameters

- **x** (*IntegerPLSITorchSequence*)
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator*)

seq_update(*x*, *weights*, *estimate*)

Update counts from posterior token-topic responsibilities.

Return type

None

Parameters

- **x** (*IntegerPLSITorchSequence*)
- **weights** (*torch.Tensor*)
- **estimate** (*IntegerPLSIDistribution*)

value()

Return CPU statistics (topic_terms, doc_topics, docs, length).

Return type

typing.Tuple[*numpy.ndarray*, *numpy.ndarray*, *numpy.ndarray*, *typing.Optional*[*typing.Any*]]

```
class dmx.torch_stats.int_plsi.IntegerPLSIAccumulatorFactory(num_vals, num_states, num_docs,  
                                                         len_factory=NullAccumulatorFactory(),  
                                                         keys=(None, None, None),  
                                                         _device=None)
```

Bases: *TorchStatisticAccumulatorFactory*

Create PLSI accumulators with a fixed document/topic/term shape.

Parameters

- **num_vals** (*int*)
- **num_states** (*int*)
- **num_docs** (*int*)
- **len_factory** (*TorchStatisticAccumulatorFactory* | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*, *str* | *None*] | *None*)
- **_device** (*torch.device* | *None*)

make(*device=None*)

Create a PLSI accumulator and length child associated with device.

Return type

`dmx.torch_stats.int_plsi.IntegerPLSIAccumulator`

Parameters

device (`torch.device` | `None`)

class `dmx.torch_stats.int_plsi.IntegerPLSIDataEncoder`(*len_encoder=NoneDataEncoder()*,
_device=None)

Bases: `TorchSequenceEncoder`

Flatten sparse document term counts and encode document lengths.

Parameters

- **len_encoder** (`TorchSequenceEncoder` | `None`)
- **_device** (`str` | `None`)

seq_encode(*x*, *device=None*)

Encode *N* sparse documents with *M* total nonzero entries.

The flat tensors are (terms, counts, document_ids, observation_ids, lengths, document_ids_per_observation). Terms, document IDs, and observation IDs are integer tensors of shape (*M*,); counts also have shape (*M*,). Lengths and document IDs per observation have shape (*N*,). All tensors and the length encoding are created on device.

Return type

`dmx.torch_stats.int_plsi.IntegerPLSITorchSequence`

Parameters

- **x** (`Sequence[Tuple[int, Sequence[Tuple[int, float]]]]`)
- **device** (`torch.device` | `None`)

class `dmx.torch_stats.int_plsi.IntegerPLSIDistribution`(*state_word_mat*, *doc_state_mat*, *doc_vec*,
len_dist=NoneDistribution(), *device=None*)

Bases: `TorchProbabilityDistribution`

Represent a document-topic-term model with an optional length child.

Parameters

- **state_word_mat** (`List[List[float]]` | `ndarray`)
- **doc_state_mat** (`List[List[float]]` | `ndarray`)
- **doc_vec** (`List[float]` | `ndarray`)
- **len_dist** (`TorchProbabilityDistribution` | `None`)
- **device** (`torch.device` | `None`)

component_log_density(*x*)

Return per-topic token log densities with shape (*K*,).

Return type

`torch.Tensor`

Parameters

x (`Tuple[int, Sequence[Tuple[int, float]]]`)

density(*x*)

Evaluate the marginal density of one sparse document observation.

Return type

float

Parameters

x (*Tuple*[int, *Sequence*[*Tuple*[int, float]]])

dist_to_encoder()

Create a sparse document encoder with the length child encoder.

Return type

dmx.torch_stats.int_plsi.IntegerPLSIDataEncoder

estimator(*pseudo_count=None*)

Create an estimator with optional smoothing of all probability tables.

Return type

dmx.torch_stats.int_plsi.IntegerPLSEstimator

Parameters

pseudo_count (*float* | *None*)

log_density(*x*)

Marginalize latent topics for one sparse document observation.

Return type

float

Parameters

x (*Tuple*[int, *Sequence*[*Tuple*[int, float]]])

sampler(*seed=None*)

Create a CPU sampler for documents, topics, terms, and lengths.

Return type

dmx.torch_stats.int_plsi.IntegerPLSISampler

Parameters

seed (*int* | *None*)

seq_log_density(*x*)

Return marginal document log densities with shape (N,).

Return type

torch.Tensor

Parameters

x (*IntegerPLSITorchSequence*)

to(*device*)

Move model and length-child tensors to device in place.

Return type

dmx.torch_stats.int_plsi.IntegerPLSIDistribution

Parameters

device (*str* | *torch.device* | *None*)

```
class dmx.torch_stats.int_plsi.IntegerPLSEstimator(num_vals, num_states, num_docs,
                                                    len_estimator=NoneEstimator(),
                                                    pseudo_count=(None, None, None),
                                                    suff_stat=(None, None, None), keys=(None,
                                                    None, None))
```

Bases: [TorchParameterEstimator](#)

Estimate term-topic, document-topic, and document distributions.

Parameters

- **num_vals** (*int*)
- **num_states** (*int*)
- **num_docs** (*int*)
- **len_estimator** ([TorchParameterEstimator](#) | *None*)
- **pseudo_count** (*Tuple*[*float* | *None*, *float* | *None*, *float* | *None*] | *None*)
- **suff_stat** (*Tuple*[*ndarray* | *None*, *ndarray* | *None*, *ndarray* | *None*] | *None*)
- **keys** (*Tuple*[*str* | *None*, *str* | *None*, *str* | *None*] | *None*)

accumulator_factory()

Create an accumulator factory retaining dimensions and keys.

Return type

[dmx.torch_stats.int_plsi.IntegerPLSIAccumulatorFactory](#)

estimate(nobs, suff_stat, device=None)

Normalize aggregated counts into PLSI probabilities on device.

Return type

[dmx.torch_stats.int_plsi.IntegerPLSIDistribution](#)

Parameters

- **nobs** (*float* | *None*)
- **suff_stat** (*Tuple*[*ndarray*, *ndarray*, *ndarray*, *SS1* | *None*])
- **device** (*torch.device* | *None*)

```
class dmx.torch_stats.int_plsi.IntegerPLSISampler(dist, seed=None)
```

Bases: [DistributionSampler](#)

Sample document identifiers, token topics, terms, and total lengths.

Parameters

- **dist** ([IntegerPLSIDistribution](#))
- **seed** (*int* | *None*)

sample(size=None)

Draw one sparse document observation or size observations.

Return type

typing.Union[*typing.Tuple*[*int*, *typing.Sequence*[*typing.Tuple*[*int*, *float*]]],
typing.Sequence[*typing.Tuple*[*int*, *typing.Sequence*[*typing.Tuple*[*int*,
float]]]]]

Parameters**size** (*int* | *None*)**class** `dmx.torch_stats.int_plsi.IntegerPLSITorchSequence`(*data*, *device=None*)Bases: [*TorchEncodedSequence*](#)

Store a length encoding and flattened sparse document-count tensors.

Parameters

- **data** (*Tuple*[[*TorchEncodedSequence*](#), *Tuple*[*torch.Tensor*, *torch.Tensor*, *torch.Tensor*, *torch.Tensor*]])
- **device** (*torch.device* | *None*)

Joint mixture

Provide torch-backed coupled mixtures for pairs of observation families.

For (*x1*, *x2*), latent state *Z1* selects a component in *components1* using *w1*; conditional transition *taus12*[*Z1*, :] selects *Z2* for a component in *components2*. Encoded pairs retain two child encodings for the same *N* pairs. Internally, responsibility calculations use (*N*, *K1*, *K2*) joint weights and return marginal likelihoods of shape (*N*,). Parameters are torch tensors moved by *to*; samplers and sufficient statistics are CPU NumPy based. This mirrors `dmx.stats.jmixture` without its optional name.

class `dmx.torch_stats.jmixture.JointMixtureDataEncoder`(*encoder1*, *encoder2*)Bases: [*TorchSequenceEncoder*](#)Encode each coordinate of *N* observation pairs with its own child encoder.**Parameters**

- **encoder1** ([*TorchSequenceEncoder*](#))
- **encoder2** ([*TorchSequenceEncoder*](#))

seq_encode(*x*, *device=None*)Encode pairs as first and second coordinate encodings on *device*.**Return type**[*dmx.torch_stats.jmixture.JointMixtureTorchEncodedSequence*](#)**Parameters**

- **x** (*Sequence*[*Tuple*[*T0*, *T1*]])
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.jmixture.JointMixtureDistribution`(*components1*, *components2*, *w1*, *w2*, *taus12*, *taus21*, *keys=(None, None, None)*, *device=None*)Bases: [*TorchProbabilityDistribution*](#)Represent coupled latent mixtures for observations (*x1*, *x2*).*w1* has shape (*K1*,) and *taus12*/*taus21* have shape (*K1*, *K2*). Each child family is homogeneous within itself.**Parameters**

- **components1** (*Sequence*[[*TorchProbabilityDistribution*](#)])
- **components2** (*Sequence*[[*TorchProbabilityDistribution*](#)])
- **w1** (*Sequence*[*float*] | *ndarray*)

- **w2** (*Sequence[float] | ndarray*)
- **taus12** (*List[List[float]] | ndarray*)
- **taus21** (*List[List[float]] | ndarray*)
- **keys** (*Tuple[str | None, str | None, str | None] | None*)
- **device** (*torch.device | None*)

density(*x*)

Evaluate the joint marginal density of one observation pair.

Return type

float

Parameters

x (*Tuple[T0, T1]*)

dist_to_encoder()

Create one child encoder for each observation coordinate.

Return type

dmx.torch_stats.jmixture.JointMixtureDataEncoder

estimator(*pseudo_count=None*)

Create estimators for both component families and transition counts.

Return type

dmx.torch_stats.jmixture.JointMixtureEstimator

Parameters

pseudo_count (*float | None*)

log_density(*x*)

Marginalize coupled latent states for one observation pair.

Return type

float

Parameters

x (*Tuple[T0, T1]*)

sampler(*seed=None*)

Create a CPU sampler for coupled latent states and observations.

Return type

dmx.torch_stats.jmixture.JointMixtureSampler

Parameters

seed (*int | None*)

seq_log_density(*x*)

Return joint marginal log densities with shape (N,).

Return type

torch.Tensor

Parameters

x (*JointMixtureTorchEncodedSequence*)

to(device)

Move parameter tensors to device in place and return self.

Return type

`dmx.torch_stats.jmixture.JointMixtureDistribution`

Parameters

device (`str` | `torch.device` | `None`)

```
class dmx.torch_stats.jmixture.JointMixtureEstimator(estimators1, estimators2, suff_stat=None,
                                                    pseudo_count=None, keys=(None, None,
                                                                    None))
```

Bases: `TorchParameterEstimator`

Estimate both component families and coupled latent transition matrices.

Parameters

- **estimators1** (`Sequence`[`TorchParameterEstimator`])
- **estimators2** (`Sequence`[`TorchParameterEstimator`])
- **suff_stat** (`Tuple`[`ndarray`, `ndarray`, `ndarray`, `Tuple`[`E0`, ...], `Tuple`[`E1`, ...]] | `None`)
- **pseudo_count** (`Tuple`[`float`, `float`, `float`] | `None`)
- **keys** (`Tuple`[`str` | `None`, `str` | `None`, `str` | `None`] | `None`)

accumulator_factory()

Create a coupled accumulator factory from both estimator families.

Return type

`dmx.torch_stats.jmixture.JointMixtureEstimatorAccumulatorFactory`

estimate(nobs, suff_stat, device=None)

Estimate component models, marginals, and conditional transitions.

Return type

`dmx.torch_stats.jmixture.JointMixtureDistribution`

Parameters

- **nobs** (`float` | `None`)
- **suff_stat** (`Tuple`[`ndarray`, `ndarray`, `ndarray`, `Tuple`[`E0`, ...], `Tuple`[`E1`, ...]])
- **device** (`torch.device` | `None`)

```
class dmx.torch_stats.jmixture.JointMixtureEstimatorAccumulator(accumulators1, accumulators2,
                                                                keys=(None, None, None),
                                                                device=None)
```

Bases: `TorchStatisticAccumulator`

Accumulate marginal and joint latent-state sufficient statistics.

The public statistic contains K1 counts, K2 counts, a (K1, K2) transition-count matrix, and the two positional child-statistic tuples.

Parameters

- **accumulators1** (`Sequence`[`TorchStatisticAccumulator`])
- **accumulators2** (`Sequence`[`TorchStatisticAccumulator`])

- **keys** (*Tuple[str | None, str | None, str | None] | None*)
- **device** (*torch.device | None*)

acc_to_encoder()

Create coordinate encoders from the first child accumulators.

Return type

dmx.torch_stats.jmixture.JointMixtureDataEncoder

combine(suff_stat)

Merge marginal counts, transition counts, and child statistics.

Return type

dmx.torch_stats.jmixture.JointMixtureEstimatorAccumulator

Parameters

suff_stat (*Tuple[ndarray, ndarray, ndarray, Tuple[E0, ...], Tuple[E1, ...]]*)

from_value(x)

Replace marginal counts, transition counts, and child statistics.

Return type

dmx.torch_stats.jmixture.JointMixtureEstimatorAccumulator

Parameters

x (*Tuple[ndarray, ndarray, ndarray, Tuple[E0, ...], Tuple[E1, ...]]*)

key_merge(stats_dict)

Merge keyed state counts and child statistics where configured.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

key_replace(stats_dict)

Replace keyed state counts and child statistics where configured.

Return type

None

Parameters

stats_dict (*Dict[str, Any]*)

seq_initialize(x, weights, tng)

Initialize random hard latent assignments for an encoded batch.

Return type

None

Parameters

- **x** (*JointMixtureTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator*)

seq_update(*x*, *weights*, *estimate*)

Update children using (N, K1, K2) joint responsibilities.

Return type

None

Parameters

- **x** ([JointMixtureTorchEncodedSequence](#))
- **weights** ([torch.Tensor](#))
- **estimate** ([JointMixtureDistribution](#))

value()

Return marginal and joint counts plus both child-statistic tuples.

Return type

[typing.Tuple](#)[[numpy.ndarray](#), [numpy.ndarray](#), [numpy.ndarray](#), [typing.Tuple](#)[[typing.Any](#), ...], [typing.Tuple](#)[[typing.Any](#), ...]]

```
class dmx.torch_stats.jmixture.JointMixtureEstimatorAccumulatorFactory(factories1,factories2,  
                                                                    keys=(None, None,  
                                                                    None))
```

Bases: [TorchStatisticAccumulatorFactory](#)

Create coupled-mixture accumulators from two component-factory sequences.

Parameters

- **factories1** ([Sequence](#)[[TorchStatisticAccumulatorFactory](#)])
- **factories2** ([Sequence](#)[[TorchStatisticAccumulatorFactory](#)])
- **keys** ([Tuple](#)[[str](#) | None, [str](#) | None, [str](#) | None] | None)

make(*device*=None)

Create both child-accumulator families on device.

Return type

[dmx.torch_stats.jmixture.JointMixtureEstimatorAccumulator](#)

Parameters

device ([torch.device](#) | None)

```
class dmx.torch_stats.jmixture.JointMixtureSampler(dist, seed=None)
```

Bases: [DistributionSampler](#)

Sample Z1, then Z2 | Z1, then the two observations.

Parameters

- **dist** ([JointMixtureDistribution](#))
- **seed** ([int](#) | None)

sample(*size*=None)

Draw one observation pair or a sequence of size pairs.

Return type

[typing.Union](#)[[typing.Tuple](#)[[typing.Any](#), [typing.Any](#)], [typing.Sequence](#)[[typing.Tuple](#)[[typing.Any](#), [typing.Any](#)]]]

Parameters

size ([int](#) | None)

```
class dmx.torch_stats.jmixture.JointMixtureTorchEncodedSequence(data, device=None)
```

Bases: [*TorchEncodedSequence*](#)

Store the observation count and two coordinate encodings for paired data.

Parameters

- **data** (*Tuple*[*int*, [*TorchEncodedSequence*](#), [*TorchEncodedSequence*](#)])
- **device** (*torch.device* | *None*)

Mixture

Provide torch-backed finite mixtures with homogeneous components.

A latent component Z is drawn from K mixture weights and the observed value is drawn from `components[Z]`. All components accept the same data type and share one encoded observation sequence. Encoded component log densities and posterior responsibilities have shape (N, K) ; mixture log densities have shape $(N,)$. Parameters and children move with `to`. Floating tensors use the vector-helper dtype, normally float64 and float32 on MPS, while sampled weights and accumulated component counts use CPU NumPy arrays. This mirrors `dmx.stats.mixture` without its optional name.

```
class dmx.torch_stats.mixture.MixtureAccumulator(accumulators, keys=(None, None), device=None)
```

Bases: [*TorchStatisticAccumulator*](#)

Accumulate component counts and responsibility-weighted child statistics.

Parameters

- **accumulators** (*Sequence*[[*TorchStatisticAccumulator*](#)])
- **keys** (*Tuple*[*str* | *None*, *str* | *None*])
- **device** (*torch.device* | *None*)

```
acc_to_encoder()
```

Create the shared encoder from the first child accumulator.

Return type

[*dmx.torch_stats.mixture.MixtureDataEncoder*](#)

```
combine(suff_stat)
```

Merge component counts and the length- K child-statistic tuple.

Return type

[*dmx.torch_stats.mixture.MixtureAccumulator*](#)

Parameters

suff_stat (*Tuple*[*ndarray*, *Tuple*[T_2 , ...]])

```
from_value(x)
```

Replace component counts and child sufficient statistics.

Return type

[*dmx.torch_stats.mixture.MixtureAccumulator*](#)

Parameters

x (*Tuple*[*ndarray*, *Tuple*[T_2 , ...]])

```
key_merge(stats_dict)
```

Merge weight, component-list, and recursive child keys.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**key_replace**(*stats_dict*)

Replace weight, component-list, and recursive child keys.

Return type

None

Parameters**stats_dict** (*Dict[str, Any]*)**seq_initialize**(*x, weights, tng*)

Randomly split (N,) weights across components with Dirichlet draws.

Return type

None

Parameters

- **x** (*MixtureTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **tng** (*torch.Generator*)

seq_update(*x, weights, estimate*)

Update children using an (N, K) responsibility matrix.

Return type

None

Parameters

- **x** (*MixtureTorchEncodedSequence*)
- **weights** (*torch.Tensor*)
- **estimate** (*MixtureDistribution*)

value()

Return CPU component counts and child sufficient statistics.

Return type*typing.Tuple[*numpy.ndarray*, *typing.Tuple[typing.Any, ...]*]***class** *dmx.torch_stats.mixture.MixtureAccumulatorFactory*(*factories, keys=(None, None)*)Bases: *TorchStatisticAccumulatorFactory*

Create mixture accumulators from K component factories.

Parameters

- **factories** (*Sequence[TorchStatisticAccumulatorFactory]*)
- **keys** (*Tuple[str | None, str | None]*)

make(*device=None*)

Create component accumulators on device when it is explicit.

Return type*dmx.torch_stats.mixture.MixtureAccumulator*

Parameters**device** (*torch.device* | *None*)**class** `dmx.torch_stats.mixture.MixtureDataEncoder`(*encoder*)Bases: *TorchSequenceEncoder*

Encode observations once for all homogeneous mixture components.

Parameters**encoder** (*TorchSequenceEncoder*)**seq_encode**(*x*, *device=None*)Encode *N* observations once on *device* for all components.**Return type***dmx.torch_stats.mixture.MixtureTorchEncodedSequence***Parameters**

- **x** (*Sequence*[*T*])
- **device** (*torch.device* | *None*)

class `dmx.torch_stats.mixture.MixtureDistribution`(*components*, *w*, *device=None*)Bases: *TorchProbabilityDistribution*Represent a finite mixture of *K* homogeneous child distributions.**Parameters**

- **components** (*Sequence*[*TorchProbabilityDistribution*])
- **w** (*ndarray* | *List*[*float*] | *torch.Tensor*)
- **device** (*torch.device* | *None*)

component_log_density(*x*)Return *K* unweighted component log densities for one observation.**Return type***torch.Tensor***Parameters****x** (*T*)**density**(*x*)

Evaluate the marginal density of one observation.

Return type*float***Parameters****x** (*T*)**dist_to_encoder**()

Create the shared encoder from the first homogeneous component.

Return type*dmx.torch_stats.mixture.MixtureDataEncoder***estimator**(*pseudo_count=None*)

Create component estimators and optional weight smoothing.

Return type*dmx.torch_stats.mixture.MixtureEstimator***Parameters****pseudo_count** (*float* | *None*)**log_density(*x*)**

Marginalize the latent component with a torch log-sum-exp.

Return type*float***Parameters*****x*** (*T*)**posterior(*x*)**

Return length-K posterior component responsibilities.

Return type*torch.Tensor***Parameters*****x*** (*T*)**sampler(*seed=None*)**

Create a sampler for latent components and their observations.

Return type*dmx.torch_stats.mixture.MixtureSampler***Parameters****seed** (*int* | *None*)**seq_component_log_density(*x*)**

Return unweighted component log densities with shape (N, K).

Return type*torch.Tensor***Parameters*****x*** (*MixtureTorchEncodedSequence*)**seq_log_density(*x*)**

Return marginal mixture log densities with shape (N,).

Return type*torch.Tensor***Parameters*****x*** (*MixtureTorchEncodedSequence*)**seq_posterior(*x*)**

Return posterior responsibilities with shape (N, K).

Return type*torch.Tensor***Parameters*****x*** (*MixtureTorchEncodedSequence*)

to(device)

Move weights and all component models to device in place.

Return type

`dmx.torch_stats.mixture.MixtureDistribution`

Parameters

device (`str` | `torch.device` | `None`)

class `dmx.torch_stats.mixture.MixtureEstimator`(*estimators*, *fixed_weights=None*, *suff_stat=None*, *pseudo_count=None*, *keys=(None, None)*)

Bases: `TorchParameterEstimator`

Estimate component models and categorical mixture weights.

Parameters

- **estimators** (`Sequence`[`TorchParameterEstimator`])
- **fixed_weights** (`List`[`float`] | `torch.Tensor` | `None`)
- **suff_stat** (`ndarray` | `None`)
- **pseudo_count** (`float` | `None`)
- **keys** (`Tuple`[`str` | `None`, `str` | `None`])

accumulator_factory()

Create a factory from the component estimator factories.

Return type

`dmx.torch_stats.mixture.MixtureAccumulatorFactory`

estimate(*nobs*, *suff_stat*, *device=None*)

Estimate components from responsibilities and normalize weights.

Return type

`dmx.torch_stats.mixture.MixtureDistribution`

Parameters

- **nobs** (`float` | `None`)
- **suff_stat** (`Tuple`[`ndarray`, `Tuple`[`Any`, ...]])
- **device** (`torch.device` | `None`)

class `dmx.torch_stats.mixture.MixtureSampler`(*dist*, *seed=None*)

Bases: `DistributionSampler`

Draw a latent component and then an observation from that component.

Parameters

- **dist** (`MixtureDistribution`)
- **seed** (`int` | `None`)

sample(*size=None*)

Draw one observation or a list of size observations.

Return type

`typing.Union`[`typing.List`[`typing.Any`], `typing.Any`]

Parameters

size (`int` | `None`)

```
class dmx.torch_stats.mixture.MixtureTorchEncodedSequence(data, device=None)
```

Bases: [TorchEncodedSequence](#)

Wrap one child encoding shared by every homogeneous component.

Parameters

- **data** ([TorchEncodedSequence](#))
- **device** (`torch.device` / `None`)

1.6 dmx.utils API

Backend-neutral and NumPy-oriented modeling utilities.

The submodules provide estimation loops, automatic estimator selection, embedding helpers, option handling, and vector operations used primarily with [dmx.stats](#) and, where documented, [dmx.bstats](#). The package root declares only `optsutil` and `vector` as its star-import surface; import other helpers from their defining `dmx.utils` submodules.

Tensor device and fitting concerns are outside this package and belong to [dmx.torch_utils](#). MPI orchestration likewise belongs to [dmx.mpi4py.utils](#); some optional embedding helpers here may still require dependencies beyond the core installation.

These helpers primarily support the NumPy/SciPy and Bayesian backends. Some embedding functions require optional dependencies; see [Installation](#) for installation options.

1.6.1 Model Construction and Estimation

Automatic model construction

Infer estimators from heterogeneous data and fit mixture models.

The helpers inspect nested observations, select matching `dmx.stats` or `dmx.bstats` estimators, and support the mixture-model preprocessing used by the heterogeneous embedding utilities.

```
class dmx.utils.automatic.DatumNode(parent=None, data=None)
```

Summarize values at one position in nested observations.

Tuples, lists, and non-string iterables create positional child nodes. Scalar values are counted by value and classified for automatic estimator selection.

Parameters

- **parent** ([DatumNode](#) / `None`)
- **data** (`Sequence[Any]` / `None`)

children

Positional summaries for nested values.

parent

Parent summary, or `None` at the root.

vdict

Counts of scalar values.

count

Total values added at this node.

none_count

Number of `None` values.

nan_count

Number of floating-point NaN values.

inf_count

Number of infinite values.

str_count

Number of string values.

float_count

Number of finite, non-integral floating-point values.

int_count

Number of integer-valued numeric values.

bool_count

Number of boolean values classified as such.

obj_count

Number of values not covered by another scalar category.

neg_count

Number of negative finite floating-point values.

zero_count

Number of floating-point zeros.

add_data(x)

Add each value from an iterable to the summary.

Parameters

x – Values to summarize.

Return type

None

Parameters

x (*Iterable[Any]*)

add_datum(x)

Add one scalar or nested value to the summary.

Parameters

x – Value to summarize. Non-string iterables are expanded into positional child nodes.

Return type

None

Parameters

x (*Any*)

copy()

Copy the node's children and scalar value counts.

Return type

dmx.utils.automatic.DatumNode

Returns

A new node with recursively copied children and `vdict`.

get_estimator(*pseudo_count=1.0, emp_suff_stat=True, use_bstats=False*)

Infer an estimator from the summarized observation structure.

Scalar strings and nonnegative integers become categorical fields; non-integral floats become Gaussian fields. Fixed-width nested values become composites, variable-width values become sequences, unsupported values are ignored, and observed `None` or `NaN` values add optional wrappers.

Parameters

- **pseudo_count** – Smoothing mass for inferred `dmx.stats` estimators.
- **emp_suff_stat** – Whether to initialize supported estimators from empirical counts.
- **use_bstats** – Select Bayesian estimators from `dmx.bstats`.

Return type

`typing.Any`

Returns

An estimator matching the inferred scalar or nested structure.

Parameters

- **pseudo_count** (*float*)
- **emp_suff_stat** (*bool*)
- **use_bstats** (*bool*)

merge(*x*)

Merge another summary into this node in place.

Parameters

x – Node whose counts and children are added.

Return type

`dmx.utils.automatic.DatumNode`

Returns

This mutated node.

Parameters

x (`DatumNode`)

`dmx.utils.automatic.encode_mixture_data`(*data, mix_model*)

Encode observations using the API implemented by a mixture model.

Parameters

- **data** – Observations accepted by the model's encoder.
- **mix_model** – A `dmx.stats` or `dmx.bstats` mixture distribution.

Return type

`typing.Any`

Returns

The model-specific encoded sequence. Its structure depends on the component distribution.

Raises

TypeError – If `mix_model` is not a supported mixture distribution.

Parameters

- **data** (*Sequence[`Any`]*)

- **mix_model** (*MixtureDistribution* / *MixtureDistribution*)

`dmx.utils.automatic.get_categorical_estimator(vdict, pseudo_count=None, emp_suff_stat=True, use_bstats=False)`

Create a categorical estimator from weighted observed values.

For `dmx.stats`, empirical sufficient statistics are normalized from `vdict` when requested. The `dmx.bstats` estimator is returned with its defaults and does not consume `pseudo_count` or `emp_suff_stat`.

Parameters

- **vdict** – Mapping from category values to nonnegative observation weights.
- **pseudo_count** – Smoothing mass for the `dmx.stats` estimator.
- **emp_suff_stat** – Whether to initialize `dmx.stats` probabilities from normalized observed weights.
- **use_bstats** – Select the Bayesian `dmx.bstats` implementation.

Return type

`typing.Any`

Returns

A categorical estimator from the selected package.

Parameters

- **vdict** (*Mapping*[*Any*, *float*])
- **pseudo_count** (*float* | *None*)
- **emp_suff_stat** (*bool*)
- **use_bstats** (*bool*)

`dmx.utils.automatic.get_composite_estimator(ests, use_bstats=False)`

Combine field estimators into a fixed-width composite estimator.

Parameters

- **ests** – Estimators in the same order as fields in each observation.
- **use_bstats** – Select the Bayesian `dmx.bstats` implementation.

Return type

`typing.Any`

Returns

A composite estimator from the selected package.

Parameters

- **ests** (*Sequence*[*Any*])
- **use_bstats** (*bool*)

`dmx.utils.automatic.get_dpm_mixture(data, estimator=None, max_comp=20, rng=None, max_its=1000, print_iter=100, mix_threshold_count=0.5)`

Fit and prune a finite Dirichlet-process mixture model.

The Bayesian optimizer initializes and updates at most `max_comp` copies of the component estimator. Components whose fitted weight is less than `mix_threshold_count / len(data)` are removed. The optimizer and this function write progress and retained weights to standard output.

Parameters

- **data** – Nonempty observations to model.
- **estimator** – Component estimator. When absent, one is inferred from **data**.
- **max_comp** – Maximum number of components used during fitting.
- **rng** – Random state used by mixture initialization. When absent, the Bayesian optimizer supplies its default random state.
- **max_its** – Maximum number of optimizer iterations.
- **print_iter** – Interval for optimizer progress output.
- **mix_threshold_count** – Effective-count threshold for retaining a component.

Return type

dmx.bstats.mixture.MixtureDistribution

Returns

A Bayesian mixture containing the retained components and their weights.

Raises

ZeroDivisionError – If data is empty.

Parameters

- **data** (*Sequence*[*Any*])
- **estimator** (*Any* | *None*)
- **max_comp** (*int*)
- **rng** (*RandomState* | *None*)
- **max_its** (*int*)
- **print_iter** (*int*)
- **mix_threshold_count** (*float*)

`dmx.utils.automatic.get_estimator(data, pseudo_count=1.0, emp_suff_stat=True, use_bstats=True)`

Infer an estimator from a sequence of heterogeneous observations.

Parameters

- **data** – Observations whose scalar or nested structure is inspected.
- **pseudo_count** – Smoothing mass for inferred `dmx.stats` estimators.
- **emp_suff_stat** – Whether to initialize supported estimators from empirical counts.
- **use_bstats** – Select Bayesian estimators from `dmx.bstats`.

Return type

`typing.Any`

Returns

An estimator matching the inferred data structure.

Parameters

- **data** (*Sequence*[*Any*])
- **pseudo_count** (*float*)
- **emp_suff_stat** (*bool*)
- **use_bstats** (*bool*)

`dmx.utils.automatic.get_gaussian_estimator(vdict, pseudo_count=None, emp_suff_stat=True, use_bstats=False)`

Create a Gaussian estimator from weighted observed values.

The empirical sufficient statistics are the weighted mean and population variance of finite keys. Bayesian estimators are returned with defaults.

Parameters

- **vdict** – Mapping from numeric values to observation weights.
- **pseudo_count** – Smoothing mass for both Gaussian sufficient statistics.
- **emp_suff_stat** – Whether to initialize from the weighted finite values.
- **use_bstats** – Select the Bayesian `dmx.bstats` implementation.

Return type

`typing.Any`

Returns

A Gaussian estimator from the selected package.

Parameters

- **vdict** (*Mapping[Any, float]*)
- **pseudo_count** (*float | None*)
- **emp_suff_stat** (*bool*)
- **use_bstats** (*bool*)

`dmx.utils.automatic.get_ignored_estimator(use_bstats=False)`

Create an estimator that ignores its input field.

Parameters

use_bstats – Select the Bayesian `dmx.bstats` implementation.

Return type

`typing.Any`

Returns

An ignored estimator from the selected package.

Parameters

use_bstats (*bool*)

`dmx.utils.automatic.get_optional_estimator(est, missing_value, use_bstats=False)`

Wrap an estimator so a designated missing value is modeled separately.

Parameters

- **est** – Base estimator for non-missing values.
- **missing_value** – Value treated as missing, including `None` or `NaN`.
- **use_bstats** – Select the Bayesian `dmx.bstats` implementation.

Return type

`typing.Any`

Returns

An optional estimator from the selected package.

Parameters

- **est** (*ParameterEstimator* / *ParameterEstimator*)
- **missing_value** (*Any* / *None*)
- **use_bstats** (*bool*)

`dmx.utils.automatic.get_poisson_estimator(vdict, pseudo_count=None, emp_suff_stat=True, use_bstats=False)`

Create a Poisson estimator from weighted observed values.

The empirical sufficient statistic is the weighted mean of finite keys. Bayesian estimators are returned with their defaults.

Parameters

- **vdict** – Mapping from numeric values to observation weights.
- **pseudo_count** – Smoothing mass for the `dmx.stats` estimator.
- **emp_suff_stat** – Whether to initialize from the weighted finite values.
- **use_bstats** – Select the Bayesian `dmx.bstats` implementation.

Return type

`typing.Any`

Returns

A Poisson estimator from the selected package.

Parameters

- **vdict** (*Mapping[Any, float]*)
- **pseudo_count** (*float* / *None*)
- **emp_suff_stat** (*bool*)
- **use_bstats** (*bool*)

`dmx.utils.automatic.get_sequence_estimator(est, use_bstats=False)`

Wrap an estimator to model variable-length sequences.

Parameters

- **est** – Estimator for individual sequence elements.
- **use_bstats** – Select the Bayesian `dmx.bstats` implementation.

Return type

`typing.Any`

Returns

A sequence estimator from the selected package.

Parameters

- **est** (*ParameterEstimator* / *ParameterEstimator*)
- **use_bstats** (*bool*)

`dmx.utils.automatic.prepare_mixture_model(data, rng, max_components=30, mix_threshold_count=0.5, max_its=1000, print_iter=100, comp_estimator=None, mix_model=None)`

Fit or reuse a mixture model and compute component posteriors.

When `mix_model` is absent, a Dirichlet-process mixture is fitted using `rng` and the supplied optimization controls. A supplied model is used unchanged. In either case, the observations are encoded with that model.

Parameters

- **data** – Observations to encode and, when needed, fit.
- **rng** – Random state used to initialize mixture fitting.
- **max_components** – Maximum number of mixture components; must be an integer greater than one even when a model is supplied.
- **mix_threshold_count** – Minimum effective component count retained after fitting.
- **max_its** – Maximum mixture-fitting iterations.
- **print_iter** – Interval for mixture-fitting progress output.
- **comp_estimator** – Optional estimator for a single mixture component.
- **mix_model** – Optional pre-fitted mixture model.

Return type

tuple[typing.Union[[dmx.stats.mixture.MixtureDistribution](#), [dmx.bstats.mixture.MixtureDistribution](#)], typing.Any, numpy.ndarray]

Returns

The mixture model, its model-specific encoded data, and an array of component posterior probabilities with shape (n_observations, n_components).

Raises

- **ValueError** – If max_components is not an integer greater than one.
- **RuntimeError** – If the resulting mixture has no components.
- **TypeError** – If the resulting model type cannot encode data.

Parameters

- **data** (*Sequence[Any]*)
- **rng** (*RandomState*)
- **max_components** (*int*)
- **mix_threshold_count** (*float*)
- **max_its** (*int*)
- **print_iter** (*int*)
- **comp_estimator** (*Any | None*)
- **mix_model** (*MixtureDistribution | MixtureDistribution | None*)

Spark input builders

Build field parsers and estimators for comma-delimited Spark input.

`dmx.utils.builder.get_indexed_rdd_pne(field_info=None, filename=None)`

Create a composite estimator and parser from field specifications.

Each specification has the form [index, name, mapping, estimator]. Estimator and mapping expressions are evaluated as trusted Python input in the public [dmx.stats](#) namespace. The returned parser splits input on literal commas and returns None when a line has too few fields.

Parameters

- **field_info** – Field specifications. Used directly when provided.

- **filename** – File passed to `read_index_csv()` when `field_info` is absent.

Return type

`typing.Tuple[typing.Any, typing.Callable[[str], typing.Optional[typing.Tuple[typing.Any, ...]]]]`

Returns

A composite estimator containing the non-null estimator expressions and a callable that converts a line to the corresponding value tuple.

Raises

- **ValueError** – If neither `field_info` nor `filename` supplies field specifications.
- **OSError** – If `filename` cannot be read.
- **SyntaxError** – If a trusted mapping or estimator expression is invalid.
- **NameError** – If an expression references an unavailable name.

Parameters

- **field_info** (`List[List[str]] | None`)
- **filename** (`str | None`)

`dmx.utils.builder.read_index_csv(filename)`

Read field specifications from a comma-delimited text file.

Text after `#` on each line is discarded. Only lines that split into exactly four fields (index, name, mapping expression, and estimator expression) are returned; blank and malformed lines are ignored.

Parameters

filename – Path to the UTF-8 encoded field specification.

Return type

`typing.List[typing.List[str]]`

Returns

The four string fields from each valid line, without further conversion.

Raises

OSError – If the file cannot be opened or read.

Parameters

filename (`str`)

Estimation workflows

Estimate and validate sequence-encodable models from observed data.

The module provides randomized data partitioning and iterative EM helpers for local encoded chunks or Spark RDDs. Callers may provide raw observations, pre-encoded data, or a previous model depending on the helper.

`dmx.utils.estimation.best_of(data, vdata, est, trials, max_its, init_p, delta, rng, init_estimator=None, enc_data=None, enc_vdata=None, out=sys.stdout, print_iter=1)`

Run EM from multiple randomized starts and return the best fit.

Every trial initializes from a random subset controlled by `rng`. A proposed EM update is retained only when training log likelihood does not decrease (unless `delta` is `None`), and a trial stops when its change is less than `delta`. Models are compared by validation log likelihood when validation data is available, otherwise by training log likelihood. Progress is written to `out`.

Parameters

- **data** – Raw training observations, or None when `enc_data` is supplied.
- **vdata** – Optional raw validation observations.
- **est** – Estimator used for EM updates.
- **trials** – Number of randomized starts; values below one are treated as one.
- **max_its** – Maximum updates per trial; values below one are treated as one.
- **init_p** – Fraction of observations participating in randomized initialization, in $(0, 1]$.
- **delta** – Stop a trial when the signed training log-likelihood change is less than this value.
- **rng** – Random state used for every trial and advanced in place.
- **init_estimator** – Optional estimator used only for encoding and initialization.
- **enc_data** – Pre-encoded local chunks or Spark RDD. Takes precedence over `data`.
- **enc_vdata** – Pre-encoded validation chunks. Takes precedence over `vdata`.
- **out** – Text stream receiving iteration and trial summaries.
- **print_iter** – Positive interval between iteration summaries.

Return type

`typing.Tuple[float, dmx.stats.pdist.SequenceEncodableProbabilityDistribution]`

Returns

Best validation log likelihood and its fitted model.

Raises

- **ValueError** – If both training representations are absent or `init_p` is outside $(0, 1]$.
- **RuntimeError** – If no trial produces a selectable model.

Parameters

- **data** (`Sequence[T] | None`)
- **vdata** (`Sequence[T] | None`)
- **est** ([ParameterEstimator](#))
- **trials** (`int`)
- **max_its** (`int`)
- **init_p** (`float`)
- **delta** (`float`)
- **rng** (`RandomState`)
- **init_estimator** ([ParameterEstimator](#) | `None`)
- **enc_data** (`List[Tuple[int, EncodedDataSequence]] | RDD[Any] | None`)
- **enc_vdata** (`List[Tuple[int, EncodedDataSequence]] | RDD[Any] | None`)
- **out** (`IO`)
- **print_iter** (`int`)

`dmx.utils.estimation.empirical_kl_divergence(dist1, dist2, enc_data)`

Estimate KL divergence between two models on encoded observations.

The log densities are normalized over the supplied observations and the discrete divergence from `dist1` to `dist2` is computed. Both models must accept the same encoding and at least one observation must have a finite, non-NaN log density under both models.

Parameters

- **dist1** – First distribution, which defines the KL weighting.
- **dist2** – Second distribution using the same encoded representation.
- **enc_data** – Local or Spark chunks represented as (`chunk_size`, `encoded_sequence`) pairs.

Return type

`typing.Tuple[float, float, float]`

Returns

The empirical KL estimate, followed by the counts of non-finite or NaN log densities under `dist1` and `dist2`.

Raises

ValueError – If there are no jointly valid log-density values.

Parameters

- **dist1** (`SequenceEncodableProbabilityDistribution`)
- **dist2** (`SequenceEncodableProbabilityDistribution`)
- **enc_data** (`List[Tuple[int, EncodedDataSequence]] | RDD[Any]`)

`dmx.utils.estimation.hill_climb(data, vdata, estimator, prev_estimate, max_its, metric_lambda, best_estimate=None, enc_data=None, enc_vdata=None, out=sys.stdout, print_iter=1)`

Run fixed EM updates and retain the best metric-scoring model.

Every proposal becomes the starting point for the next update, regardless of its score. The returned model maximizes `metric_lambda` on `vdata`; validation log likelihood breaks exact score ties. Progress is written to `out`.

Parameters

- **data** – Raw training observations, encoded only when `enc_data` is absent.
- **vdata** – Raw validation observations used by `metric_lambda` and encoded only when `enc_vdata` is absent.
- **estimator** – Estimator used for each EM update.
- **prev_estimate** – Starting model and source of encoders.
- **max_its** – Number of EM proposals to evaluate.
- **metric_lambda** – Callable receiving (`vdata`, `model`) and returning a scalar score to maximize.
- **best_estimate** – Optional incumbent model; defaults to `prev_estimate`.
- **enc_data** – Pre-encoded training chunks.
- **enc_vdata** – Pre-encoded validation chunks.
- **out** – Text stream receiving progress messages.

- **print_iter** – Positive interval between progress messages.

Return type

dmx.stats.pdist.SequenceEncodableProbabilityDistribution

Returns

The incumbent or proposal with the best metric and likelihood tie break.

Parameters

- **data** (*List*[*T*])
- **vdata** (*List*[*T*])
- **estimator** (*ParameterEstimator*)
- **prev_estimate** (*SequenceEncodableProbabilityDistribution*)
- **max_its** (*int*)
- **metric_lambda** (*Callable*[[*List*[*T*], *SequenceEncodableProbabilityDistribution*], *float*])
- **best_estimate** (*SequenceEncodableProbabilityDistribution* | *None*)
- **enc_data** (*List*[*Tuple*[*int*, *EncodedDataSequence*]] | *RDD*[*Any*] | *None*)
- **enc_vdata** (*List*[*Tuple*[*int*, *EncodedDataSequence*]] | *RDD*[*Any*] | *None*)
- **out** (*IO*)
- **print_iter** (*int*)

```
dmx.utils.estimation.iterate(data, estimator, max_its, prev_estimate=None, init_p=0.1,
                             rng=RandomState(), out=sys.stdout, enc_data=None, init_estimator=None,
                             print_iter=1)
```

Perform a fixed number of EM updates and return the final estimate.

Unlike *optimize()*, this helper performs no likelihood-based stopping or model selection. It optionally caches Spark encoded data and writes average elapsed time at the requested interval.

Parameters

- **data** – Raw training observations. Ignored when **enc_data** is supplied.
- **estimator** – Estimator used for updates, or *None* to use **init_estimator**.
- **max_its** – Exact number of updates to perform when positive.
- **prev_estimate** – Optional starting model. Otherwise randomized initialization is performed.
- **init_p** – Initialization sampling fraction in *(0, 1]*.
- **rng** – Random state used for initialization and advanced in place. *None* creates a fresh un-seeded state; the default instance is shared across calls.
- **out** – Text stream receiving timing messages.
- **enc_data** – Pre-encoded local chunks or Spark RDD.
- **init_estimator** – Fallback estimator used when **estimator** is absent.
- **print_iter** – Positive interval between timing messages.

Return type

dmx.stats.pdist.SequenceEncodableProbabilityDistribution

Returns

The model after the requested updates.

Raises

ValueError – If neither data representation is supplied, no estimator is available, or randomized initialization receives an invalid `init_p`.

Parameters

- **data** (*List*[*T*])
- **estimator** (*ParameterEstimator* | *None*)
- **max_its** (*int*)
- **prev_estimate** (*SequenceEncodableProbabilityDistribution* | *None*)
- **init_p** (*float*)
- **rng** (*RandomState* | *None*)
- **out** (*IO*)
- **enc_data** (*List*[*Tuple*[*int*, *EncodedDataSequence*]] | *RDD*[*Any*] | *None*)
- **init_estimator** (*ParameterEstimator* | *None*)
- **print_iter** (*int*)

`dmx.utils. estimation.k_fold_split_index(sz, k, rng)`

Assign observations to approximately balanced random folds.

Random values drawn from `rng` determine a permutation, after which fold identifiers are assigned round-robin. The random-state is advanced.

Parameters

- **sz** – Number of observations.
- **k** – Number of folds.
- **rng** – Random state controlling the permutation.

Return type

`numpy.ndarray`

Returns

Integer array of shape `(sz,)` whose entry is the observation's zero-based fold identifier.

Parameters

- **sz** (*int*)
- **k** (*int*)
- **rng** (*RandomState*)

`dmx.utils. estimation.optimize(data, estimator, max_its=10, delta=1.0e-9, init_estimator=None, init_p=0.1, rng=RandomState(), prev_estimate=None, vdata=None, enc_data=None, enc_vdata=None, out=sys.stdout, print_iter=1, num_chunks=1)`

Fit a model with EM and return the best validation-scoring estimate.

Raw data is encoded once, using `prev_estimate` when supplied or the initialization estimator otherwise. Without a previous estimate, model initialization samples observations according to `init_p` using `rng`. Updates that decrease training log likelihood are rejected unless `delta` is `None`. A non-`None` `delta` stops optimization when

the signed change is smaller than the threshold. Progress is written to out and model selection uses validation likelihood when available.

Parameters

- **data** – Raw training observations, or None when `enc_data` is supplied.
- **estimator** – Estimator used for each EM update.
- **max_its** – Maximum number of EM updates.
- **delta** – Optional stopping threshold for signed training log-likelihood improvement. None disables early stopping and accepts decreasing updates.
- **init_estimator** – Optional estimator used for encoding and randomized initialization; estimator is used by default.
- **init_p** – Initialization sampling fraction in $(0, 1]$. Ignored when `prev_estimate` is supplied.
- **rng** – Random state used for initialization and advanced in place. The default instance is shared across calls.
- **prev_estimate** – Optional starting model, which also supplies the encoder.
- **vdata** – Optional raw validation observations.
- **enc_data** – Pre-encoded local chunks or Spark RDD. Takes precedence over `data`.
- **enc_vdata** – Pre-encoded validation chunks. Takes precedence over `vdata`.
- **out** – Text stream receiving progress messages.
- **print_iter** – Positive interval between progress messages.
- **num_chunks** – Number of local encoded chunks created from raw data.

Return type

`dmx.stats.pdist.SequenceEncodableProbabilityDistribution`

Returns

The fitted model with the greatest observed validation likelihood, or training likelihood when no validation data is supplied.

Raises

ValueError – If both training representations are absent, or if a new model is requested with `init_p` outside $(0, 1]$.

Parameters

- **data** (`Sequence[T] | None`)
- **estimator** (`ParameterEstimator`)
- **max_its** (`int`)
- **delta** (`float | None`)
- **init_estimator** (`ParameterEstimator | None`)
- **init_p** (`float`)
- **rng** (`RandomState`)
- **prev_estimate** (`SequenceEncodableProbabilityDistribution | None`)
- **vdata** (`Sequence[T] | None`)

- **enc_data** (*List[Tuple[int, EncodedDataSequence]] | RDD[Any] | None*)
- **enc_vdata** (*List[Tuple[int, EncodedDataSequence]] | RDD[Any] | None*)
- **out** (*IO*)
- **print_iter** (*int*)
- **num_chunks** (*int*)

`dmx.utils.estimation.partition_data(data, pvec, rng)`

Randomly partition observations according to proportions.

Parameters

- **data** – Sequence of observations.
- **pvec** – One-dimensional sequence of partition proportions. Values are passed unchanged to [partition_data_index\(\)](#).
- **rng** – Random state controlling the shuffled ordering; it is advanced.

Return type

`typing.List[typing.List[typing.TypeVar(T)]]`

Returns

Materialized data partitions in the order of **pvec**.

Parameters

- **data** (*Sequence[T]*)
- **pvec** (*List[float] | ndarray*)
- **rng** (*RandomState*)

`dmx.utils.estimation.partition_data_index(sz, pvec, rng)`

Randomly partition observation indexes according to proportions.

The random state is advanced once per observation. Boundaries are obtained by rounding cumulative proportions times **sz**; **pvec** is not normalized or validated, so proportions summing to less than one omit trailing indexes.

Parameters

- **sz** – Number of observations.
- **pvec** – One-dimensional sequence of partition proportions.
- **rng** – Random state controlling the shuffled ordering.

Return type

`typing.List[numpy.ndarray]`

Returns

One integer index array per requested partition.

Parameters

- **sz** (*int*)
- **pvec** (*List[float] | ndarray*)
- **rng** (*RandomState*)

1.6.2 Embedding Utilities

Heterogeneous t-SNE

Embed heterogeneous observations with mixture-derived t-SNE affinities.

`dmx.utils.htsne.adj_perplexity(x, ss)`

Convert a distance vector to probabilities at a given scale.

Parameters

- **x** – One-dimensional distance array of shape `(n_neighbors,)`.
- **ss** – Positive distance scale. Its reciprocal acts as inverse temperature.

Return type

`typing.Tuple[float, numpy.ndarray]`

Returns

Natural-log entropy and normalized probabilities with the same shape as **x**.

Parameters

- **x** (*ndarray*)
- **ss** (*float*)

`dmx.utils.htsne.dpmsne(P=None, emb_dim=2, alpha=1.0, Y=None, max_its=1000, print_iter=100, eta=500, momentum=0.8, min_gain=0.01, min_value=1.0e-128, optimize_alpha=False, min_alpha=1.0e-6, max_alpha_its=3, seed=None)`

Embed a precomputed affinity matrix with the t-SNE optimizer.

P is converted with `numpy.asarray()` and normalized in place when possible. When Y is absent, `seed` controls Gaussian initialization, followed by 100 early-exaggeration updates and `max_its` regular updates. A supplied NumPy Y may be updated in place. Progress is printed to standard output.

Parameters

- **P** – Floating-point square affinity matrix with shape `(n_samples, n_samples)`.
- **emb_dim** – Output dimensionality when Y is absent.
- **alpha** – Initial Student-t degrees-of-freedom parameter.
- **Y** – Optional initial coordinates of shape `(n_samples, emb_dim)`.
- **max_its** – Number of regular embedding updates.
- **print_iter** – Positive interval for standard-output progress messages.
- **eta** – Gradient learning rate.
- **momentum** – Momentum for regular updates.
- **min_gain** – Lower bound for adaptive gradient gains.
- **min_value** – Numerical floor for affinities.
- **optimize_alpha** – Optimize alpha after each embedding update.
- **min_alpha** – Lower bound used during alpha optimization.
- **max_alpha_its** – Maximum alpha updates per embedding iteration.
- **seed** – Seed for Gaussian initialization. Ignored when Y is supplied.

Return type

`numpy.ndarray`

Returns

Centered embedding coordinates with shape (n_samples, emb_dim).

Parameters

- **P** (*ndarray* | *None*)
- **emb_dim** (*int*)
- **alpha** (*float*)
- **Y** (*ndarray* | *None*)
- **max_its** (*int*)
- **print_iter** (*int*)
- **eta** (*int*)
- **momentum** (*float*)
- **min_gain** (*float*)
- **min_value** (*float*)
- **optimize_alpha** (*bool*)
- **min_alpha** (*float*)
- **max_alpha_its** (*int*)
- **seed** (*int* | *None*)

`dmx.utils.htsne.fix_row_perplexity(P, a)`

Rescale each affinity row to a target perplexity.

Diagonal entries are excluded, then restored as zeros. The input is read but not modified.

Parameters

- **P** – Square positive affinity matrix of shape (n_samples, n_samples).
- **a** – Positive target perplexity.

Return type

`numpy.ndarray`

Returns

Row-normalized matrix with the same shape as P.

Parameters

- **P** (*ndarray*)
- **a** (*float*)

`dmx.utils.htsne.get_pmat(posterior_mat, ll_mat, targ_perplexity=None, vlen=False)`

Construct high-dimensional affinities from mixture responsibilities.

Parameters

- **posterior_mat** – Component posteriors with shape (n_samples, n_components).
- **ll_mat** – Component log densities with the same shape.
- **targ_perplexity** – Optional row perplexity target.
- **vlen** – Use the variable-length normalization implemented by `get_pmat_vlen()`.

Return type

numpy.ndarray

Returns

Directed affinity matrix of shape (n_samples, n_samples), scaled by 1 / n_samples.

Parameters

- **posterior_mat** (ndarray)
- **ll_mat** (ndarray)
- **targ_perplexity** (float | None)
- **vlen** (bool)

dmx.utils.htsne.get_pmat_vlen(*posterior_mat*, *ll_mat*, *targ_perplexity*=None)

Construct affinities for variable-length mixture observations.

Parameters

- **posterior_mat** – Component posteriors with shape (n_samples, n_components).
- **ll_mat** – Component log densities with the same shape.
- **targ_perplexity** – Optional row perplexity target.

Return type

numpy.ndarray

Returns

Directed affinity matrix of shape (n_samples, n_samples), scaled by 1 / n_samples and with a zero diagonal unless numerical flooring replaces it.

Parameters

- **posterior_mat** (ndarray)
- **ll_mat** (ndarray)
- **targ_perplexity** (float | None)

dmx.utils.htsne.htsne(*data*, *emb_dim*=2, *alpha*=1.0, *max_components*=30, *mix_threshold_count*=0.5, *Y*=None, *perplexity*=None, *max_its*=1000, *print_iter*=100, *eta*=500, *momentum*=0.8, *min_gain*=0.01, *min_value*=1.0e-128, *optimize_alpha*=False, *min_alpha*=1.0e-6, *max_alpha_its*=3, *seed*=None, *comp_estimator*=None, *mix_model*=None, *variable_length*=False)

Embed heterogeneous observations using mixture-derived affinities.

A supplied mixture is reused; otherwise a pruned Dirichlet-process mixture is fitted. Its component posteriors and log densities define the target affinity matrix. When *Y* is absent, *seed* controls both mixture fitting and a small Gaussian coordinate initialization, followed by 100 early-exaggeration updates and then *max_its* regular updates. A supplied NumPy *Y* may be updated in place. Progress is printed to standard output.

Parameters

- **data** – Nonempty heterogeneous observations.
- **emb_dim** – Output dimensionality when *Y* is absent.
- **alpha** – Initial Student-t degrees-of-freedom parameter.
- **max_components** – Maximum components used when fitting a mixture.
- **mix_threshold_count** – Minimum effective component count retained after mixture fitting.

- **Y** – Optional initial coordinates of shape `(len(data), emb_dim)`.
- **perplexity** – Optional target row perplexity for affinity construction.
- **max_its** – Number of regular embedding updates.
- **print_iter** – Positive interval for standard-output progress messages.
- **eta** – Gradient learning rate.
- **momentum** – Momentum for regular updates.
- **min_gain** – Lower bound for adaptive gradient gains.
- **min_value** – Numerical floor for affinities.
- **optimize_alpha** – Optimize alpha after each embedding update.
- **min_alpha** – Lower bound used during alpha optimization.
- **max_alpha_its** – Maximum alpha updates per embedding iteration.
- **seed** – Seed for a local legacy NumPy random state. `None` uses OS entropy.
- **comp_estimator** – Optional estimator for one mixture component.
- **mix_model** – Optional pre-fitted mixture distribution.
- **variable_length** – Use variable-length affinity normalization.

Return type

`numpy.ndarray`

Returns

Centered embedding coordinates with shape `(len(data), emb_dim)`.

Raises

- **ValueError** – If `max_components` is not an integer greater than one.
- **RuntimeError** – If mixture fitting produces no components.

Parameters

- **data** (*Sequence*[*T*])
- **emb_dim** (*int*)
- **alpha** (*float*)
- **max_components** (*int*)
- **mix_threshold_count** (*float*)
- **Y** (*ndarray* | *None*)
- **perplexity** (*int* | *None*)
- **max_its** (*int*)
- **print_iter** (*int*)
- **eta** (*int*)
- **momentum** (*float*)
- **min_gain** (*float*)
- **min_value** (*float*)
- **optimize_alpha** (*bool*)

- **min_alpha** (*float*)
- **max_alpha_its** (*int*)
- **seed** (*int* | *None*)
- **comp_estimator** (*Any* | *None*)
- **mix_model** (*MixtureDistribution* | *MixtureDistribution* | *None*)
- **variable_length** (*bool*)

`dmx.utils.htsne.row_perplexity_solve(x, a, s0, s1, d=10)`

Bisect a scale interval to approach a target entropy.

At most *d* recursive bisections are performed. If the target is outside the endpoint entropies, the corresponding endpoint is returned.

Parameters

- **x** – One-dimensional distance array.
- **a** – Target natural-log entropy.
- **s0** – Lower scale bound.
- **s1** – Upper scale bound.
- **d** – Remaining bisection depth.

Return type

float

Returns

Selected scale in the closed interval [*s0*, *s1*].

Parameters

- **x** (*ndarray*)
- **a** (*float*)
- **s0** (*float*)
- **s1** (*float*)
- **d** (*int*)

`dmx.utils.htsne.t_cond_prob_mat(tx, alpha)`

Compute normalized Student-t affinities in an embedding.

Parameters

- **tx** – Embedding coordinates of shape (*n_samples*, *n_dimensions*).
- **alpha** – Positive degrees-of-freedom parameter.

Return type

typing.Tuple[*numpy.ndarray*, *numpy.ndarray*]

Returns

A pair of (*Q*, *kernel*) matrices, each with shape (*n_samples*, *n_samples*). *Q* sums to one and has a zero diagonal.

Parameters

- **tx** (*ndarray*)

- **alpha** (*float*)

`dmx.utils.htsne.t_cond_prob_mat_alpha(tx, alpha)`

Compute Student-t affinities and squared distances for alpha updates.

Parameters

- **tx** – Embedding coordinates of shape (n_samples, n_dimensions).
- **alpha** – Positive degrees-of-freedom parameter.

Return type

`typing.Tuple[numpy.ndarray, numpy.ndarray]`

Returns

Normalized affinity and squared-distance matrices, both with shape (n_samples, n_samples).

Parameters

- **tx** (*ndarray*)
- **alpha** (*float*)

`dmx.utils.htsne.update_alpha(P, Y, alpha, min_alpha, min_value, max_its=30, step=0.1, eps=1.0e-4)`

Optimize the Student-t degrees-of-freedom parameter.

A bounded Newton-style update minimizes KL divergence, with each step limited to a relative change of `step`. Iteration stops at `max_its` or when the KL reduction is less than `eps`.

Parameters

- **P** – Target affinities of shape (n_samples, n_samples).
- **Y** – Coordinates of shape (n_samples, n_dimensions).
- **alpha** – Starting value.
- **min_alpha** – Lower bound for returned values.
- **min_value** – Numerical floor for low-dimensional affinities.
- **max_its** – Maximum alpha updates.
- **step** – Maximum relative change per update.
- **eps** – Minimum KL reduction required to continue.

Return type

float

Returns

Optimized alpha value, no smaller than `min_alpha`.

Parameters

- **P** (*ndarray*)
- **Y** (*ndarray*)
- **alpha** (*float*)
- **min_alpha** (*float*)
- **min_value** (*float*)
- **max_its** (*int*)

- **step** (*float*)
- **eps** (*float*)

`dmx.utils.htsne.update_embed(P, Y, iY, gains, momentum, eta, alpha, min_gain, min_value=1.0e-128)`

Apply one momentum gradient update to an embedding.

The coordinates are updated in place and recentered to zero column means. Sign changes between the gradient and previous increments increase gains; matching signs decay them, subject to `min_gain`.

Parameters

- **P** – Target affinities of shape `(n_samples, n_samples)`.
- **Y** – Coordinates of shape `(n_samples, n_dimensions)`.
- **iY** – Previous coordinate increments with the same shape as `Y`.
- **gains** – Per-coordinate gains with the same shape as `Y`.
- **momentum** – Multiplier applied to previous increments.
- **eta** – Gradient learning rate.
- **alpha** – Positive degrees-of-freedom parameter.
- **min_gain** – Lower bound for adaptive gains.
- **min_value** – Numerical floor applied to low-dimensional affinities.

Return type

`typing.Tuple[numpy.ndarray, numpy.ndarray, numpy.ndarray, numpy.ndarray]`

Returns

Updated coordinates, increments, gains, and an affinity matrix of shape `(n_samples, n_samples)`.

Parameters

- **P** (*ndarray*)
- **Y** (*ndarray*)
- **iY** (*ndarray*)
- **gains** (*ndarray*)
- **momentum** (*float*)
- **eta** (*float*)
- **alpha** (*float*)
- **min_gain** (*float*)
- **min_value** (*float*)

`dmx.utils.htsne.vec_perplexity(x, s)`

Compute entropy for a distance vector at a given scale.

Parameters

- **x** – One-dimensional distance array.
- **s** – Scale passed to [`adj\_perplexity\(\)`](#).

Return type

`float`

Returns

Natural-log entropy of the normalized probabilities.

Parameters

- **x** (*ndarray*)
- **s** (*float*)

Heterogeneous UMAP

Embed heterogeneous observations using mixture posteriors and UMAP.

```
dmx.utils.humap.humap(data, max_components=30, mix_threshold_count=0.5, max_its=1000, print_iter=100,  
                      seed=None, comp_estimator=None, mix_model=None, umap_kwargs=None)
```

Fit UMAP to mixture-component posterior probabilities.

A supplied mixture is reused; otherwise a pruned Dirichlet-process mixture is fitted. `seed` controls only mixture fitting. UMAP randomness is controlled independently through `umap_kwargs` (for example, `random_state`). Missing default UMAP options are inserted into a caller-supplied dictionary in place.

Parameters

- **data** – Nonempty heterogeneous observations.
- **max_components** – Maximum components used when fitting a mixture.
- **mix_threshold_count** – Minimum effective component count retained after mixture fitting.
- **max_its** – Maximum mixture-fitting iterations.
- **print_iter** – Interval for mixture-fitting progress output.
- **seed** – Seed for the local legacy NumPy random state used in mixture fitting. `None` uses OS entropy.
- **comp_estimator** – Optional estimator for one mixture component.
- **mix_model** – Optional pre-fitted mixture distribution.
- **umap_kwargs** – Options passed to `umap.UMAP`. Defaults are added for missing keys.

Return type

```
typing.Tuple[typing.Any, typing.Union[dmx.stats.mixture.  
MixtureDistribution, dmx.bstats.mixture.MixtureDistribution], umap.UMAP,  
numpy.ndarray]
```

Returns

The embedding with shape `(len(data), n_components)`, fitted mixture model, fitted UMAP object, and posterior matrix with shape `(len(data), n_mixture_components)`.

Raises

- **ValueError** – If `max_components` is not an integer greater than one, or UMAP rejects its inputs or options.
- **RuntimeError** – If mixture fitting produces no components.

Parameters

- **data** (*Sequence* [*DATUM_TYPE*])
- **max_components** (*int*)
- **mix_threshold_count** (*float*)

- **max_its** (*int*)
- **print_iter** (*int*)
- **seed** (*int* | *None*)
- **comp_estimator** (*Any* | *None*)
- **mix_model** (*MixtureDistribution* | *MixtureDistribution* | *None*)
- **umap_kwargs** (*Dict[str, Any]* | *None*)

1.6.3 Data and Mathematical Utilities

Metrics

Functions for classification evaluation.

Create ROC curves and search depth rankings.

`dmx.utils.metrics.classify(data, model, labels=None)`

Rank true labels using conditional model log densities.

For every observation value, the model scores a copy paired with each candidate label. Scores are normalized with a softmax. Labels must be sortable, and an explicit `labels` sequence must include every observed true label.

Parameters

- **data** – (*true_label*, *value*) pairs for independent observations.
- **model** – Joint or conditional distribution that accepts the same pairs.
- **labels** – Candidate labels. By default, use the sorted unique true labels.

Return type

`typing.Tuple[numpy.ndarray, numpy.ndarray, numpy.ndarray, typing.Dict[typing.Any, numpy.ndarray]]`

Returns

Four objects – zero-based true-label ranks with shape `(n_samples,)`; true-label probabilities with shape `(n_samples,)`; true labels with shape `(n_samples,)`; and a mapping from each candidate label to its probability vector of shape `(n_samples,)`.

Parameters

- **data** (*Sequence[Tuple[Any, Any]]*)
- **model** (*SequenceEncodableProbabilityDistribution*)
- **labels** (*Sequence[Any]* | *None*)

`dmx.utils.metrics.ranking_depth(x, k=None, comp_func=lambda a, b: ...)`

Find zero-based ranks of matching candidates in scored lists.

Candidates for each target are sorted by descending score before matching.

Parameters

- **x** – Pairs of a target value and a list of (*candidate*, *score*) pairs.
- **k** – Number of matching ranks to retain. *None* retains all matches.
- **comp_func** – Predicate applied as `comp_func(target, candidate)`.

Return type

`typing.Union[numpy.ndarray, typing.List[numpy.ndarray]]`

Returns

When `k` is given, a float array of shape `(len(x), k)` padded with NaN. Otherwise, a list of variable-length integer rank arrays.

Parameters

- `x` (`List[Tuple[Any, List[Tuple[Any, float]]]]`)
- `k` (`int | None`)
- `comp_func` (`Callable[[Any, Any], bool]`)

`dmx.utils.metrics.roc_curve(pos_x, neg_x)`

Create an empirical ROC curve from positive and negative scores.

Scores are sorted in descending order; each returned position corresponds to accepting one additional sample at that threshold.

Parameters

- `pos_x` – One-dimensional scores for positive examples.
- `neg_x` – One-dimensional scores for negative examples.

Return type

`typing.Tuple[numpy.ndarray, numpy.ndarray]`

Returns

Cumulative true-positive and false-positive rates, each with shape `(len(pos_x) + len(neg_x),)`.

Parameters

- `pos_x` (`List[float] | ndarray`)
- `neg_x` (`List[float] | ndarray`)

`dmx.utils.metrics.roc_percentiles(pos_x, neg_x, perc_points)`

Sample an empirical ROC curve at target true-positive rates.

For each target, the greatest attainable true-positive rate not exceeding that target is selected. Targets below the first attainable rate are omitted rather than represented by a placeholder row.

Parameters

- `pos_x` – One-dimensional scores for positive examples.
- `neg_x` – One-dimensional scores for negative examples.
- `perc_points` – Target true-positive rates, normally in `[0, 1]`.

Return type

`numpy.ndarray`

Returns

Array with up to `len(perc_points)` rows and two columns containing `[false_positive_rate, true_positive_rate]`. If no target is attainable, the current implementation returns an empty array with shape `(0,)`.

Parameters

- `pos_x` (`List[float] | ndarray`)

- **neg_x** (*List[float] | ndarray*)
- **perc_points** (*List[float] | ndarray*)

Option utilities

Utility functions for data pre/post processing.

`dmx.utils.optsutil.count_by_value(x)`

Count the number of observations of a given value in arg 'x'.

Parameters

x (*Sequence[T]*) – A sequence of type T or numpy array of type T.

Return type

`typing.Dict[typing.TypeVar(T), int]`

Returns

Dictionary mapping value (type T) to value-count.

Parameters

x (*Sequence[T] | ndarray*)

`dmx.utils.optsutil.flat_map(f, x)`

Map values of x under mapping f().

Parameters

- **f** (*Callable[[T], T1]*) – Maps values of x to sequence of type T1.
- **x** (*Sequence[T]*) – Sequence to be mapped under f.

Return type

`typing.List[typing.TypeVar(T1)]`

Returns

List of mapped type T1.

Parameters

- **f** (*Callable[[T], Sequence[T1]]*)
- **x** (*Sequence[T]*)

`dmx.utils.optsutil.get_inv_map(val_map)`

Obtain the inverse dictionary mapping of key/value pairs.

Parameters

val_map (*Dict[T1, T]*) – Dictionary mapping keys to values.

Return type

`typing.Dict[typing.TypeVar(T1), typing.TypeVar(T)]`

Returns

Inverse mapping of val_map.

Parameters

val_map (*Dict[T, T1]*)

`dmx.utils.optsutil.get_parent_directory(filepath, levels=0)`

Get the parent directory of a file.

Parameters

- **filepath** (*str*) – The full path of the file

- **levels** (*int*) – Number of levels to move up to reach parent.

Return type

`pathlib._local.Path`

Returns

Path object containing path to parent directory.

Parameters

- **filepath** (*str*)
- **levels** (*int*)

`dmx.utils.optsutil.group_by(f, x)`

Group values in *x* using the key returned by *f*.

Parameters

- **f** (*Callable[[T], T1]*) – Function mapping type *T* to its group key of type *T1*.
- **x** (*Sequence[T]*) – Sequence of values to be grouped.

Return type

`typing.Dict[typing.TypeVar(T1), typing.List[typing.TypeVar(T)]]`

Returns

Dictionary mapping group id ‘keys’ (type *T1*) to Lists of values (type *T*).

Parameters

- **f** (*Callable[[T], T1]*)
- **x** (*Sequence[T]*)

`dmx.utils.optsutil.group_by_key(x)`

Group keys and aggregate their values into lists.

Parameters

x (*Sequence[Tuple[T, T1]]*) – A sequence of tuples of key and value pairs.

Return type

`typing.Dict[typing.TypeVar(T), typing.List[typing.TypeVar(T1)]]`

Returns

Dictionary mapping keys to list of values for respective keys.

Parameters

x (*Sequence[Tuple[T, T1]]*)

`dmx.utils.optsutil.least_occurring(x, count=None, percent=None, keep_freq=True)`

Identifies the least occurring elements in a sequence.

This function finds the least frequently occurring elements in a given sequence based on the specified *count* or *percent*. Optionally, it can return the filtered sequence with only the least occurring elements or just the elements themselves.

Parameters

- **x** (*Sequence[T]*) – The input sequence to analyze.
- **count** (*Optional[int], optional*) – The maximum number of least occurring elements to return. If specified, the function will return up to *count* least occurring elements. Defaults to *None*.

- **percent** (*Optional[float], optional*) – The percentage, as a float between 0 and 1, of least occurring elements to return. If specified, the function will return the least occurring elements that make up this percentage of the total unique elements. Defaults to *None*.
- **keep_freq** (*bool, optional*) – If *True*, returns the filtered sequence containing only the least occurring elements. If *False*, returns a list of the least occurring elements themselves. Defaults to *True*.

Return type

`typing.Union[typing.List[typing.TypeVar(T)], typing.Callable]`

Returns

Union[List[T], Callable] –

- If *keep_freq* is *True*, returns a filtered sequence containing only the least occurring elements.
- If *keep_freq* is *False*, returns a list of the least occurring elements.
- If neither *count* nor *percent* is specified, the original sequence *x* is returned.

Parameters

- **x** (*Sequence[T]*)
- **count** (*int | None*)
- **percent** (*float | None*)
- **keep_freq** (*bool*)

`dmx.utils.optsutil.map_to_integers(x, val_map)`

Map sequence of type T to integers.

Parameters

- **x** (*Sequence[T]*) – Sequence of type T to be mapped to integers.
- **val_map** (*Dict[T, int]*) – Dictionary mappings for type T to unique integer values.

Return type

`typing.List[int]`

Returns

Returns x mapped to a list of integers.

Parameters

- **x** (*Sequence[T]*)
- **val_map** (*Dict[T, int]*)

`dmx.utils.optsutil.reduce_by_key(f, x)`

Reduce sequence of tuple of key value pairs under grouping function f.

Parameters

- **f** (*Callable[[T1, T1], T1]*) – Function for reducing keys.
- **x** (*Sequence[Tuple[T, T1]]*) – A sequence of key/value pairs.

Return type

`typing.Dict[typing.TypeVar(T), typing.TypeVar(T1)]`

Returns

Dictionary mapping key types T to value types T1.

Parameters

- **f** (*Callable*[[*T1*, *T1*], *T1*])
- **x** (*Sequence*[*Tuple*[*T*, *T1*]])

`dmx.utils.optsutil.sum_by_key(x)`

Sum values and return dictionary of items with their respective summed values.

Parameters

x (*Sequence*[*Tuple*[*T*, *T1*]]) – A sequence of tuples of key and value pairs.

Return type

`typing.Dict[typing.TypeVar(T), typing.TypeVar(T1)]`

Returns

Dictionary of keys with summed values.

Parameters

x (*Sequence*[*Tuple*[*T*, *T1*]])

`dmx.utils.optsutil.text_file(f)`

Open a file and split by newline.

Args

f: File to be read-in and parsed.

Return type

`typing.List[str]`

Returns

List of strings split on newline character.

Parameters

f (*str*)

P-values

P-value utilities for approximating composite binomial ranks.

`dmx.utils.pvalues.binomial_rank(log_p_vec, log_p1_vec=None, count_vec=None, ll_eps=1.0e-4, max_len=None)`

Approximate a composite-binomial log-density histogram.

Each input position defines a binomial count and success probability in log space. Individual discrete log-density histograms are placed on a shared grid and convolved. Entries with zero counts or deterministic probabilities are skipped. At least one nondegenerate entry is required.

Parameters

- **log_p_vec** – One-dimensional log success probabilities of shape (*m*,).
- **log_p1_vec** – Optional log failure probabilities with shape (*m*,). When absent, they are computed with `log1p(-exp(log_p_vec))`.
- **count_vec** – Optional nonnegative, integer-valued binomial counts with shape (*m*,); defaults to one per entry.
- **ll_eps** – Maximum grid-alignment remainder used while halving automatic bin spacing.
- **max_len** – Optional approximate upper bound used to choose bin spacing.

Return type

`typing.Tuple[numpy.ndarray, numpy.ndarray, typing.Tuple[float, float, float]]`

Returns

Grid log densities and their normalized probabilities, both of shape `(n_bins,)`, plus `(grid_origin, grid_spacing, total_count)`.

Raises

ValueError – If no nondegenerate binomial entry remains or array shapes cannot be combined.

Parameters

- **log_p_vec** (*Sequence[float] | ndarray*)
- **log_p1_vec** (*Sequence[float] | ndarray | None*)
- **count_vec** (*Sequence[float] | ndarray | None*)
- **ll_eps** (*float*)
- **max_len** (*int | None*)

Special functions

Defines log-pseudo-determinant and special-function helpers.

`dmx.utils.special.beta(*args, **kwargs)`

Proxy *scipy.special.beta* through a lazy import.

Return type

`typing.Any`

Parameters

- **args** (*Any*)
- **kwargs** (*Any*)

`dmx.utils.special.betaln(*args, **kwargs)`

Proxy *scipy.special.betaln* through a lazy import.

Return type

`typing.Any`

Parameters

- **args** (*Any*)
- **kwargs** (*Any*)

`dmx.utils.special.digamma(*args, **kwargs)`

Proxy *scipy.special.digamma* through a lazy import.

Return type

`typing.Any`

Parameters

- **args** (*Any*)
- **kwargs** (*Any*)

`dmx.utils.special.digammainv(y, out=None)`

Approximate the inverse digamma function with five Newton steps.

Array inputs preserve their shape. Positive infinity maps to positive infinity; other non-finite array entries remain zero. The `out` argument is retained for compatibility but is ignored.

Parameters

- `y` – Scalar or NumPy array of digamma values.
- `out` – Deprecated and ignored.

Return type

`typing.Union[numpy.ndarray, float]`

Returns

Approximate inverse values, with the same shape as an array input.

Parameters

- `y (ndarray | float)`
- `out (ndarray | None)`

`dmx.utils.special.gamma(*args, **kwargs)`

Proxy `scipy.special.gamma` through a lazy import.

Return type

`typing.Any`

Parameters

- `args (Any)`
- `kwargs (Any)`

`dmx.utils.special.gammaln(*args, **kwargs)`

Proxy `scipy.special.gammaln` through a lazy import.

Return type

`typing.Any`

Parameters

- `args (Any)`
- `kwargs (Any)`

`dmx.utils.special.ive(*args, **kwargs)`

Proxy `scipy.special.ive` through a lazy import.

Return type

`typing.Any`

Parameters

- `args (Any)`
- `kwargs (Any)`

`dmx.utils.special.logpdet(x_mat)`

Compute the log-pseudo-determinant of a symmetric dense matrix.

Parameters

`x_mat` – Square matrix of shape `(n, n)`.

Return type

float

Returns

Sum of logarithms of nonzero absolute eigenvalues, or negative infinity when all eigenvalues are zero.

Raises

np.linalg.LinAlgError – If eigenvalue computation does not converge.

Parameters

x_mat (*ndarray*)

`dmx.utils.special.polygamma_loc(n, y, out=None)`

Evaluate a polygamma function through Hurwitz zeta.

The result is $(-1)^{(n+1)} * \gamma(n+1) * \zeta(n+1, y)$.

Parameters

- **n** – Nonnegative derivative order.
- **y** – Scalar evaluation point.
- **out** – Optional output array accepted by SciPy's zeta ufunc.

Return type

`typing.Union[numpy.ndarray, float]`

Returns

out after in-place evaluation when provided, otherwise a scalar.

Parameters

- **n** (*int*)
- **y** (*float*)
- **out** (*ndarray* | *None*)

`dmx.utils.special.stirling2(n, k)`

Compute a Stirling number of the second kind recursively.

$S(n, k)$ counts partitions of n labeled elements into k nonempty unlabeled subsets.

Parameters

- **n** – Positive number of elements.
- **k** – Positive number of subsets.

Return type

int

Returns

The integer $S(n, k)$.

Raises

AssertionError – If n or k is not positive.

Parameters

- **n** (*int*)
- **k** (*int*)

`dmx.utils.special.trigamma(y, out=None)`

Evaluate the trigamma function elementwise.

Parameters

- **y** – Scalar or array-like evaluation points.
- **out** – Optional array receiving results in place.

Return type

`typing.Union[numpy.ndarray, float]`

Returns

A scalar for scalar input or an array broadcast to the shape of *y*.

Parameters

- **y** (`ndarray | int | float | Iterable | List[float]`)
- **out** (`ndarray | None`)

`dmx.utils.special.zeta(*args, **kwargs)`

Proxy *scipy.special.zeta* through a lazy import.

Return type

`typing.Any`

Parameters

- **args** (*Any*)
- **kwargs** (*Any*)

Vector utilities

Vector utilities for estimation and evaluation in dmx-learn.

`dmx.utils.vector.cho_solve(a_mat, b)`

Solve $a_mat x = b$ using a Cholesky factorization.

Parameters

- **a_mat** (`Tuple[np.ndarray, bool]`) – Cholesky factorization of *a*, as returned by *cho_factor*.
- **b** (`np.ndarray`) – Right-hand side `np.ndarray` in $a_mat * x = b$.

Return type

`numpy.ndarray`

Returns

The solution to the system $a_mat * x = b$.

Parameters

- **a_mat** (`Tuple[ndarray, bool]`)
- **b** (`ndarray`)

`dmx.utils.vector.cholesky(x_mat)`

Compute the Cholesky decomposition of a matrix, to use in *cho_solve*.

Returns a matrix containing the Cholesky decomposition of a Hermitian positive-definite matrix. The return value can be used directly as the first parameter to *cho_solve*.

Parameters

x_mat (`np.ndarray`) – Square `np.ndarray` of matrix to be decomposed.

Return type

`typing.Optional[typing.Tuple[numpy.ndarray, bool]]`

Returns

Optional[Tuple[np.ndarray, bool]] – Cholesky factorization when it can be computed, otherwise *None*.

Parameters

x_mat (*ndarray*)

`dmx.utils.vector.diag(x)`

Extract a diagonal or construct a diagonal array.

Note

If x is 2D, return its diagonal. If x is 1D, return a 2D diagonal matrix with x on the diagonal.

See the more detailed documentation for `numpy.diagonal` if you use this function to extract a diagonal and wish to write to the resulting array; whether it returns a copy or a view depends on what version of numpy you are using.

Parameters

x – 2-D array, or 1-D array.

Return type

`numpy.ndarray`

Returns

The extracted diagonal or constructed diagonal array.

Parameters

x (*ndarray*)

`dmx.utils.vector.dot(x, y)`

Compute the dot product of x and y .

Parameters

- **x** – Numpy array, array-like, or scalar.
- **y** – Numpy array, array-like, or scalar.

Return type

`typing.Union[numpy.ndarray, float]`

Returns

float | np.ndarray – Scalar if both inputs are 1D vectors, a 1D array if exactly one input is scalar-like, and a matrix otherwise.

Parameters

- **x** (*ndarray | Iterable | int | float*)
- **y** (*ndarray | Iterable | int | float*)

`dmx.utils.vector.gammaln(x)`

Return the logarithm of the gamma function.

Returns $\log(\text{abs}(\text{Gamma}(x)))$.

Parameters

x (*Union[np.ndarray, float, int]*) – Scalar or array-like numeric input.

Return type`typing.Union[numpy.ndarray, float]`**Returns***float* | *np.ndarray* – Scalar output for scalar input, otherwise a NumPy array.**Parameters****x** (*ndarray* | *float* | *int*)`dmx.utils.vector.log_posterior(x)`

Compute the log posterior for component log-likelihoods.

If $x[j] = \log(p(\text{obs_i} \mid \text{theta_j}))$, the returned array contains $\log(p(\text{theta_j} \mid \text{obs_i}))$ values up to normalization.**Parameters****x** (*np.ndarray*) – Log-density values for each component or parameter value.**Return type**`numpy.ndarray`**Returns***np.ndarray* – Log-posterior values for each component. Returns a uniform log distribution if *x* contains *nan* or *inf*.**Parameters****x** (*ndarray*)`dmx.utils.vector.log_posterior_sum(x)`

Compute log posterior values and their log normalizing constant.

This returns the log posterior vector together with the log normalizing constant.

Parameters**x** (*np.ndarray*) – Log-density values for each component or parameter value.**Return type**`typing.Tuple[numpy.ndarray, float]`**Returns***Tuple[*np.ndarray*, *float*]* – Log-posterior values and the log normalizing constant. Returns a uniform log distribution if *x* contains *nan* or *-np.inf*.**Parameters****x** (*ndarray*)`dmx.utils.vector.log_sum(x)`Compute $\log(\text{sum}(\exp(x)))$ for a 1D NumPy array.**Parameters****x** (*ndarray*) – Numpy array on log-scale. E.g. $x_i = \log(y_i)$.**Return type**`float`**Returns**Float value $\log(\text{sum}(\exp(x)))$, or $-\text{np.inf}$ if $\max(x)$ is $-\text{np.inf}$.**Parameters****x** (*ndarray*)`dmx.utils.vector.make(x)`Convert the array *x* into a numpy array.

Parameters

x (*Union*[*np.ndarray*, *Sequence*[*Union*[*int*, *float*, *str*]]]) – Array-like object that can be converted to a NumPy array.

Return type

numpy.ndarray

Returns

Numpy array conversion of *x*.

Parameters

x (*ndarray* | *Sequence*[*int* | *float* | *str*] | *List*[*ndarray*])

`dmx.utils.vector.make_pdf(x)`

Normalize log-density values on the log scale.

The returned array *rv* satisfies *np.exp(rv).sum() == 1.0*.

Parameters

x (*Union*[*np.ndarray*, *Sequence*[*float*], *List*[*np.ndarray*]]) – Array-like object with float-like values.

Return type

numpy.ndarray

Returns

np.ndarray – Log-scale normalized density values.

Parameters

x (*ndarray* | *Sequence*[*float*] | *List*[*ndarray*])

`dmx.utils.vector.mat_inv(x)`

Computes the inverse of a square matrix *x*.

Arg *x* data type is *Union*[*List*[*List*[*Union*[*float*, *int*]]], *List*[*np.ndarray*], *np.ndarray*].

Parameters

x (*See above*) – List of lists, list of NumPy arrays, or a 2D square NumPy array.

Return type

numpy.ndarray

Returns

Inverse of *x* as 2-d numpy array.

Parameters

x (*List*[*List*[*float* | *int*]] | *List*[*ndarray*] | *ndarray*)

`dmx.utils.vector.matrix_log_posteriors(x, u_mat, u)`

Compute row posteriors, outer posteriors, and log-likelihood.

This function calculates posterior probabilities for rows and columns based on input matrices and vectors. It also computes the log-likelihood of the data given the model.

Parameters

- **x** (*np.ndarray*) – A 2D array with shape (*m*, *z*), where *m* is the number of rows and *z* is the number of columns.
- **u_mat** (*np.ndarray*) – A 2D array with shape (*h*, *w*) representing the matrix of prior probabilities or weights.
- **u** (*np.ndarray*) – A 1D array with shape (*h*,) representing additional prior probabilities for each row.

Return type

`typing.Tuple[numpy.ndarray, numpy.ndarray, float]`

Returns

Tuple[np.ndarray, np.ndarray, float] –

- *row_posteriors* (np.ndarray): A 3D array with shape (h, w, z) containing posterior probabilities for each row and column.
- *outer_posterior* (np.ndarray): A 1D array with shape $(h,)$ containing normalized posterior probabilities for each row.
- *ll* (float): The log-likelihood of the data given the model.

Raises

ValueError – If the shapes of *x*, *u_mat*, or *u* are incompatible.

Parameters

- **x** (ndarray)
- **u_mat** (ndarray)
- **u** (ndarray)

Examples

```
>>> x = np.array([[1, 2], [3, 4]])
>>> u_mat = np.array([[0.5, 0.2], [0.1, 0.7]])
>>> u = np.array([0.3, 0.4])
>>> row_posteriors, outer_posterior, ll = matrix_log_posteriors(x, u_mat, u)
>>> row_posteriors.shape
(2, 2, 2)
>>> outer_posterior.shape
(2,)
>>> print(ll)
-1.234
```

`dmx.utils.vector.maximum(x, y, output=None)`

Element-wise maximum of array elements.

Compare two arrays and returns a new array containing the element-wise maxima. If one of the elements being compared is a NaN, then that element is returned. If both elements are NaNs then the first is returned. The latter distinction is important for complex NaNs, which are defined as at least one of the real or imaginary parts being a NaN. The net effect is that NaNs are propagated.

Parameters

- **x** (array-like) – Values to compare. If *x.shape* $\neq$ *y.shape*, the inputs must be broadcastable to a common shape.
- **y** (array-like) – Values to compare. If *x.shape* $\neq$ *y.shape*, the inputs must be broadcastable to a common shape.
- **output** – Optional np.ndarray of float to output results to.

Return type

`typing.Union[float, int, numpy.ndarray]`

Returns

ndarray or scalar – The element-wise maximum of x and y . This is a scalar if both inputs are scalars.

Parameters

- **x** (*float | int | Iterable | ndarray*)
- **y** (*float | int | Iterable | ndarray*)
- **output** (*float | int | ndarray | None*)

`dmx.utils.vector.outer(x, y)`

Compute the outer product of two vectors.

Parameters

- **x** – (M,) array_like
- **y** – (N,) array_like

Parameters

- **x** (*ndarray | Iterable | int | float*)
- **y** (*ndarray | Iterable | int | float*)

Return type

ndarray

Returns: (M, N) ndarray.

Return type

`numpy.ndarray`

Parameters

- **x** (*ndarray | Iterable | int | float*)
- **y** (*ndarray | Iterable | int | float*)

`dmx.utils.vector.posterior(log_x, out=None, log_sum=False)`

Compute posterior probabilities from component log-likelihoods.

If $\log_x[j] = \log(p(\text{obs}_i | \theta_{\text{eta}_j}))$, the returned array contains the normalized posterior probabilities over components.

Parameters

- **log_x** (*ndarray*) – Log-density values for each component or parameter value.
- **out** (*Optional[ndarray]*) – Optional numpy array to store returned value.
- **log_sum** (*Optional[bool]*) – If true, also return the log normalizing constant.

Return type

`typing.Union[numpy.ndarray, typing.Tuple[numpy.ndarray, float]]`

Returns

np.ndarray | Tuple[np.ndarray, float] – Posterior probabilities, and optionally the log normalizing constant.

Parameters

- **log_x** (*ndarray*)
- **out** (*ndarray | None*)

- **log_sum** (*bool* | *None*)

`dmx.utils.vector.reshape(x, sz)`

Gives a new shape to an array without changing its data.

Parameters

- **x** (*np.ndarray*) – Array to be reshaped.
- **sz** (*Tuple[int, ...]*) – Shape compatible with size of array x.

Return type

`numpy.ndarray`

Returns

Reshaped array containing elements of x with shape = sz.

Parameters

- **x** (*ndarray*)
- **sz** (*SupportsIndex* | *Sequence[SupportsIndex]*)

`dmx.utils.vector.row_choice(p_mat, rng)`

Vectorized choice using row-wise sampling weights.

Choice is called on the range $[0, S)$, where each row of *p_mat* contains sampling weights.

Parameters

- **p_mat** (*np.ndarray*) – N x S numpy array.
- **rng** (*Optional[np.random.RandomState]*) – RandomState for sampling.

Return type

`numpy.ndarray`

Returns

N dim numpy array of ints.

Parameters

- **p_mat** (*ndarray*)
- **rng** (*RandomState* | *None*)

`dmx.utils.vector.sorted_dict_merge_add(k_vec1, c_vec1, k_vec2, c_vec2)`

Merge two sorted key/count arrays and add counts for shared keys.

Returns the merge sorted keys and corresponding counts.

Parameters

- **k_vec1** (*ndarray*) – Numpy array of sorted dictionary keys.
- **c_vec1** (*ndarray*) – Numpy array of counts for keys in vector k_vec1.
- **k_vec2** (*ndarray*) – Numpy array of sorted dictionary keys.
- **c_vec2** (*ndarray*) – Numpy array of counts for keys in vector k_vec2.

Return type

`typing.Tuple[numpy.ndarray, numpy.ndarray]`

Returns

Tuple[np.ndarray, np.ndarray] – Merged sorted keys and their corresponding counts.

Parameters

- **k_vec1** (*ndarray*)
- **c_vec1** (*ndarray*)
- **k_vec2** (*ndarray*)
- **c_vec2** (*ndarray*)

`dmx.utils.vector.sorted_merge(a, b)`

Merge two sorted arrays into one sorted array.

Parameters

- **a** (*ndarray*) – Sorted numpy array.
- **b** (*ndarray*) – Sorted numpy array.

Return type

`numpy.ndarray`

Returns

Sorted NumPy array containing the merged contents of *a* and *b*.

Parameters

- **a** (*ndarray*)
- **b** (*ndarray*)

`dmx.utils.vector.weighted_log_posterior(x, w)`

Compute weighted log posterior values.

The weights are assumed to be on the log scale.

Parameters

- **x** (*ndarray*) – Numpy array of log-density values for each component or parameter value.
- **w** (*ndarray*) – Numpy array of log weights for each parameter value.

Return type

`typing.List[float]`

Returns

`List[float]` – Weighted log-posterior value for each component.

Parameters

- **x** (*ndarray*)
- **w** (*ndarray*)

`dmx.utils.vector.weighted_log_posterior_sum(x, w)`

Compute weighted log posterior values and their normalizer.

The weights are assumed to be on the log scale.

Parameters

- **x** (*np.ndarray*) – Log-density values for each component or parameter value.
- **w** (*np.ndarray*) – `List[float]` or NumPy array of log weights for each parameter value.

Return type

`typing.Tuple[typing.List[float], float]`

Returns

`Tuple[List[float], float]` – Weighted log-posterior values and the log normalizing constant.

Parameters

- **x** (*ndarray*)
- **w** (*ndarray*)

`dmx.utils.vector.weighted_log_sum(x, w)`

Compute a numerically stable weighted log-sum-of-exponentials.

This uses $weights = \exp(w)$ on observation values $y = \exp(x)$, returning $\log(\sum(\exp(x) * \exp(w)))$.

Note: The weights are on the log-scale.

Parameters

- **x** (*ndarray*) – Numpy array on log-scale. E.g. $x_i = \log(y_i)$.
- **w** (*ndarray*) – Numpy array of log-weights for $y_i = \exp(x_i)$.

Return type

float

Returns

Float value $\log(\sum(\exp(x)*\exp(w)))$, or `-np.inf` if any `x` or `w` are `-np.inf`.

Parameters

- **x** (*ndarray*)
- **w** (*ndarray*)

`dmx.utils.vector.zeros(n)`

Return numpy array of shape `n`, with default `dtype=float64`.

Parameters

n (*Union[int, Iterable, Tuple[int]]*) – Shape tuple of ints, Iterable, or int.

Return type

`numpy.ndarray`

Returns

Return numpy array of shape `n`, with default `dtype=float64`.

Parameters

n (*int | Iterable | Tuple[int]*)

1.7 dmx.torch_utils API

Utilities for configuring and fitting PyTorch-backed models.

This package contains device, tensor-vector, and estimation helpers intended for `dmx.torch_stats`. The package root publicly exposes `detect_device()`; import fitting and vector helpers from the `dmx.torch_utils.estimation` and `dmx.torch_utils.vector` submodules. These utilities are separate from the NumPy-oriented `dmx.utils` package and operate on the tensor protocols of the PyTorch backend.

PyTorch is optional. Importing this package succeeds without it so callers can inspect the package, but calling `detect_device()` raises `ImportError` until PyTorch is installed.

`dmx.torch_utils.detect_device()`

Detect device available for torch

Return type

str

Returns

str – Device name ('cuda', 'mps', or 'cpu')

Raises

ImportError – If torch is not installed

These utilities require PyTorch when invoked and are intended for `dmx.torch_stats` models. See [Installation](#) for installation options.

1.7.1 Torch Estimation Utilities

Torch-backed estimation helpers for local and distributed EM fitting.

`dmx.torch_utils.estimation.empirical_kl_divergence(dist1, dist2, enc_data)`

Compute the empirical KL-divergence between two densities.

Compute the KL-divergence between *dist1* and *dist2* for an encoded sequence of data. Both distributions must use the same encodings, and the encoded tensors must already be on devices compatible with the two distributions. Only the first encoded chunk in *enc_data* is evaluated.

Parameters

- **dist1** (*TorchProbabilityDistribution*) – Distribution compatible with *enc_data*.
- **dist2** (*TorchProbabilityDistribution*) – Distribution compatible with *enc_data*.
- **enc_data** (*List[Tuple[int, TorchEncodedSequence]]*) – List of tuples containing chunk size and *TorchEncodedSequence*.

Return type

`typing.Tuple[float, float, float]`

Returns

Tuple[float, float, float] – KL-divergence estimate, number of bad likelihood values for *dist1*, and number of bad likelihood values for *dist2*.

Parameters

- **dist1** (*TorchProbabilityDistribution*)
- **dist2** (*TorchProbabilityDistribution*)
- **enc_data** (*List[Tuple[int, TorchEncodedSequence]]*)

`dmx.torch_utils.estimation.optimize(data, estimator, seed=None, max_its=10, delta=1.0e-9, init_estimator=None, init_p=0.1, device=None, prev_estimate=None, vdata=None, enc_data=None, enc_vdata=None, out=sys.stdout, print_iter=1, num_chunks=1)`

Estimate *estimator* via EM until convergence or *max_its*.

With *device=None*, the target device is auto-detected in CUDA, MPS, CPU order. The module-local default float dtype is set to *float64*, except MPS uses *float32*. Raw data are encoded on the target device; caller-supplied encoded data are used as-is. A previous estimate is moved to the target device in place before fitting.

Parameters

- **data** (*Optional[Sequence[T]]*) – Observed data of type *T*. Must be compatible with the estimator.
- **estimator** (*TorchParameterEstimator*) – Estimator used to specify the distribution for observed data.
- **seed** (*Optional[int]*) – Seed for initialization. If *None*, a seed is drawn from NumPy's global random state.

- **max_its** (*int*) – Maximum number of EM iterations to be performed. Default value is 10 iterations.
- **delta** (*Optional[float]*) – Stop when the signed likelihood improvement *new_loglikelihood - old_loglikelihood* is less than *delta*. If *None*, likelihood-decreasing updates are accepted.
- **init_estimator** (*Optional[TorchParameterEstimator]*) – Estimator used to initialize EM parameters. If *None*, *estimator* is used.
- **init_p** (*float*) – Proportion of data points used in initialization. Values at or below zero become *0.1*; values above one are clamped to one.
- **device** (*DeviceLike*) – Device used for tensor calculations. Strings are resolved to torch devices; *None* defaults to auto-detection.
- **prev_estimate** (*Optional[TorchProbabilityDistribution]*) – Optional model estimate from prior fitting. Must be consistent with *estimator*.
- **vdata** (*Optional[Sequence[T]]*) – Optional validation set.
- **enc_data** (*Optional[List[Tuple[int, Any]]]*) – Optional encoded chunks. Formed from *data* when *None*.
- **enc_vdata** (*Optional[List[Tuple[int, Any]]]*) – Optional encoded validation chunks.
- **out** (*IO*) – IO stream to write EM progress.
- **print_iter** (*int*) – Print likelihood progress every *print_iter* iterations.
- **num_chunks** (*int*) – Number of chunks for encoded data.

Return type

dmx.torch_stats.pdist.TorchProbabilityDistribution

Returns

TorchProbabilityDistribution – Best model by validation likelihood, or training likelihood when no validation data are supplied.

Raises

ValueError – If both *data* and *enc_data* are *None*.

Parameters

- **data** (*Sequence[T] | None*)
- **estimator** (*TorchParameterEstimator*)
- **seed** (*int | None*)
- **max_its** (*int*)
- **delta** (*float | None*)
- **init_estimator** (*TorchParameterEstimator | None*)
- **init_p** (*float*)
- **device** (*str | torch.device | None*)
- **prev_estimate** (*TorchProbabilityDistribution | None*)
- **vdata** (*Sequence[T] | None*)
- **enc_data** (*List[Tuple[int, Any]] | None*)
- **enc_vdata** (*List[Tuple[int, Any]] | None*)
- **out** (*IO*)

- **print_iter** (*int*)
- **num_chunks** (*int*)

```
dmx.torch_utils.estimate.optimize_mp(world_rank, world_size, data, estimator, max_its=10,
                                     delta=1.0e-9, init_estimator=None, init_p=0.1, seed=None,
                                     prev_estimate=None, vdata=None, enc_data=None,
                                     enc_vdata=None, out=sys.stdout, print_iter=1, num_chunks=1)
```

Estimate *estimator* via distributed torch EM until convergence.

This helper assumes an initialized *torch.distributed* process group. Every rank must call it in the same collective order, and *world_rank* and *world_size* must match that process group. Raw data are scattered from rank 0 when encoded chunks are not supplied; pre-encoded chunks are expected to be local to each rank and already on compatible devices. Rank 0 makes update, stopping, and validation-selection decisions and broadcasts those decisions to the other ranks.

Parameters

- **world_rank** (*int*) – Rank of this worker in the process group.
- **world_size** (*int*) – Number of workers in the process group.
- **data** (*Optional[Sequence[T]]*) – Observed data compatible with the estimator. Required on rank 0 unless local *enc_data* is supplied.
- **estimator** (*TorchParameterEstimator*) – Estimator used to specify the distribution for observed data. It should be equivalent on all ranks.
- **max_its** (*int*) – Maximum number of EM iterations to be performed. Default value is 10 iterations.
- **delta** (*Optional[float]*) – Stop when rank 0 observes signed likelihood improvement below *delta*. If *None*, likelihood-decreasing updates are accepted.
- **init_estimator** (*Optional[TorchParameterEstimator]*) – Estimator used to initialize EM parameters. If *None*, *estimator* is used.
- **init_p** (*float*) – Proportion of data points used in initialization. Values at or below zero become *0.1*; values above one are clamped to one.
- **seed** (*Optional[int]*) – Seed for initialization. If *None*, each rank draws from its NumPy global random state before the collective initialization call, so callers should synchronize this value when reproducibility across ranks matters.
- **prev_estimate** (*Optional[TorchProbabilityDistribution]*) – Optional model estimate from prior fitting, expected to be available consistently on all ranks.
- **vdata** (*Optional[Sequence[T]]*) – Optional validation set.
- **enc_data** (*Optional[Sequence[Tuple[int, Any]]]*) – Optional rank-local encoded chunks. Formed from *data* when *None*.
- **enc_vdata** (*Optional[Sequence[Tuple[int, Any]]]*) – Optional rank-local encoded validation chunks.
- **out** (*IO*) – Rank 0 output stream for EM progress.
- **print_iter** (*int*) – Print likelihood progress every *print_iter* iterations.
- **num_chunks** (*int*) – Currently unused in the distributed encode path.

Return type

dmx.torch_stats.pdist.TorchProbabilityDistribution

Returns

TorchProbabilityDistribution – Best model by validation likelihood, or training likelihood when no validation data are supplied, on each rank.

Raises

- **ValueError** – If both *data* and *enc_data* are *None*.
- **RuntimeError** – If a distributed reduction or estimation step fails to return the expected result.

Parameters

- **world_rank** (*int*)
- **world_size** (*int*)
- **data** (*Sequence*[*T*] | *None*)
- **estimator** (*TorchParameterEstimator*)
- **max_its** (*int*)
- **delta** (*float* | *None*)
- **init_estimator** (*TorchParameterEstimator* | *None*)
- **init_p** (*float*)
- **seed** (*int* | *None*)
- **prev_estimate** (*TorchProbabilityDistribution* | *None*)
- **vdata** (*Sequence*[*T*] | *None*)
- **enc_data** (*Sequence*[*Tuple*[*int*, *Any*]] | *None*)
- **enc_vdata** (*Sequence*[*Tuple*[*int*, *Any*]] | *None*)
- **out** (*IO*)
- **print_iter** (*int*)
- **num_chunks** (*int*)

1.7.2 Torch Data Utilities

Option utilities

Counting helpers for Python, NumPy, and PyTorch containers.

`dmx.torch_utils.optsutil.bincount1(xv, w, nv)`

Bincount batched weights by one-dimensional ids.

Parameters

- **xv** (*Tensor*) – One-dimensional integer tensor with shape (*n*,).
- **w** (*Tensor*) – Weight tensor with shape (*s*, *n*). It must be on a device compatible with *xv*.
- **nv** (*int*) – Number of bins in the result.

Return type

`torch.Tensor`

Returns

Tensor – Tensor with shape (*s*, *nv*). Dtype and device follow the behavior of *torch.bincount* for the supplied weights.

Parameters

- **xv** (*torch.Tensor*)
- **w** (*torch.Tensor*)
- **nv** (*int*)

`dmx.torch_utils.optsutil.count_by_value(x)`

Count the number of observations of a given value in arg ‘x’.

Values are used as yielded by the input container. Iterating a tensor yields scalar tensor keys rather than converting them to Python scalars.

Parameters

x (*Union[Sequence[T], numpy.ndarray, torch.Tensor]*) – Values to count.

Return type

`typing.Dict[typing.TypeVar(T), int]`

Returns

Dict[T, int] – Dictionary mapping each observed value to its count.

Parameters

x (*Sequence[T] | ndarray | torch.Tensor*)

`dmx.torch_utils.optsutil.int_count_by_value(x)`

Count the number of observations of a given value in arg ‘x’.

Each value is converted with *int* before counting. For tensor inputs, this converts scalar tensor elements to host Python integers and may synchronize the device.

Parameters

x (*Union[Sequence[T], numpy.ndarray, torch.Tensor]*) – Values to count.

Return type

`typing.Dict[typing.TypeVar(T), int]`

Returns

Dict[int, int] – Dictionary mapping integer values to counts.

Parameters

x (*Sequence[T] | ndarray | torch.Tensor*)

Vector utilities

Vector utilities backed by PyTorch tensors.

`dmx.torch_utils.vector.bincount_2d(x, w, minlength)`

Bincount rows of a 2-d tensor by shared integer ids.

Parameters

- **x** (*Tensor*) – One-dimensional integer tensor of length *n* containing bin ids.
- **w** (*Tensor*) – Two-dimensional weight tensor with shape (*s*, *n*).
- **minlength** (*int*) – Minimum number of bins per row.

Return type

torch.Tensor

Returns

Tensor with shape (*s*, *minlength*) on *w.device*.

Parameters

- **x** (*torch.Tensor*)
- **w** (*torch.Tensor*)
- **minlength** (*int*)

`dmx.torch_utils.vector.choice(size, states, tng, p=None)`

Sample state indices with replacement.

Parameters

- **size** (*int*) – Number of state indices to sample.
- **states** (*int*) – Number of possible states when *p* is omitted.
- **tng** (*torch.Generator*) – Generator whose device receives the sampling weights and whose state is advanced.
- **p** (*Optional[torch.Tensor]*) – Optional unnormalized weights. If supplied on another device, the weights are copied to the generator device.

Return type

torch.Tensor

Returns

torch.Tensor – Sampled indices with shape (*size*,) on the generator device.

Parameters

- **size** (*int*)
- **states** (*int*)
- **tng** (*torch.Generator*)
- **p** (*torch.Tensor* | *None*)

`dmx.torch_utils.vector.float_dtype_for_device(device)`

Return float32 for MPS (no float64 support), float64 for all others.

Return type

torch.dtype

Parameters

device (*torch.device*)

`dmx.torch_utils.vector.int_tensor(x, device=None, dtype=DINT)`

Convert array-like input to an integer tensor.

Parameters

- **x** – Tensor, NumPy array, or integer sequence to convert.
- **device** (*DeviceLike*) – Optional target device.
- **dtype** (*Optional[torch.dtype]*) – Integer dtype to request. Defaults to this module's *int32* dtype.

Return type

torch.Tensor

Returns

torch.Tensor – Converted tensor using *tensor* conversion semantics.

Parameters

- **x** (*List[int] | List[List[int]] | Sequence[int] | ndarray | torch.Tensor*)
- **device** (*str | torch.device | None*)
- **dtype** (*torch.dtype | None*)

`dmx.torch_utils.vector.int_vec(size, device=None)`

Create an integer tensor of zeros.

Parameters

- **size** (*Union[int, Tuple[int, ...]]*) – Output tensor shape.
- **device** (*DeviceLike*) – Optional output device. When *None*, PyTorch’s default tensor device is used.

Return type

`torch.Tensor`

Returns

torch.Tensor – *int32* tensor of zeros.

Parameters

- **size** (*int | Tuple[int, ...]*)
- **device** (*str | torch.device | None*)

`dmx.torch_utils.vector.ones(size, device=None, dtype=None)`

Create a floating point tensor of ones.

Parameters

- **size** (*Union[int, Tuple[int, ...]]*) – Output tensor shape.
- **device** (*DeviceLike*) – Optional output device. When *None*, PyTorch’s default tensor device is used rather than *resolve_device* auto-detection.
- **dtype** (*Optional[torch.dtype]*) – Optional output dtype. When omitted, the module-local float dtype is used, except MPS allocations use *float32*.

Return type

`torch.Tensor`

Returns

torch.Tensor – Tensor of ones with the requested shape, device, and dtype.

Parameters

- **size** (*int | Tuple[int, ...]*)
- **device** (*str | torch.device | None*)
- **dtype** (*torch.dtype | None*)

`dmx.torch_utils.vector.randint(size, low, high, tng, dtype=DINT)`

Sample random integers on a generator’s device.

Parameters

- **size** (*int*) – Number of integers to sample.
- **low** (*int*) – Inclusive lower bound.
- **high** (*int*) – Exclusive upper bound.

- **tng** (*torch.Generator*) – Generator whose state is advanced.
- **dtype** (*Optional[torch.dtype]*) – Output integer dtype. Defaults to *int32*.

Return type

torch.Tensor

Returns

torch.Tensor – Random integer tensor with shape (*size*,) on *tng.device*.

Parameters

- **size** (*int*)
- **low** (*int*)
- **high** (*int*)
- **tng** (*torch.Generator*)
- **dtype** (*torch.dtype* | *None*)

`dmx.torch_utils.vector.resolve_device(device=None)`

Convert string or None to torch.device. Auto-detects CUDA > MPS > CPU if None.

Return type

torch.device

Parameters

device (*str* | *torch.device* | *None*)

`dmx.torch_utils.vector.sample_dirichlet(alpha, size, tng)`

Sample from a Dirichlet distribution with shape *alpha*.

CPU and CUDA paths use and advance the supplied generator. MPS sampling is performed on CPU from a temporary generator seeded with *tng.initial_seed()* and copied back to the generator device.

Parameters

- **alpha** (*tn.Tensor*) – Shape parameter with length equal to simplex dimension.
- **size** (*int*) – Number of samples to draw.
- **tng** (*Generator*) – Generator identifying the target device.

Return type

torch.Tensor

Returns

Tensor with shape (*size*, *len(alpha)*), dtype matching *alpha*, and device matching *tng.device*.

Parameters

- **alpha** (*torch.Tensor*)
- **size** (*int*)
- **tng** (*torch.Generator*)

`dmx.torch_utils.vector.seed_sample(n, tng)`

Sample integer seeds from a generator.

Parameters

- **n** (*int*) – Number of seeds to draw.
- **tng** (*torch.Generator*) – Generator whose state is advanced by sampling.

Return type`torch.Tensor`**Returns**

torch.Tensor – An *int32* tensor on the generator device. When $n == 1$, the return value is a scalar tensor; otherwise it has shape $(n,)$.

Parameters

- **n** (*int*)
- **tng** (*torch.Generator*)

`dmx.torch_utils.vector.seed_tng(seed, device=None)`

Create a seeded PyTorch generator.

Parameters

- **seed** (*int*) – Seed value, coerced to *int*.
- **device** (*DeviceLike*) – Optional generator device. When *None*, no auto-detection is performed and PyTorch creates its default generator, normally on CPU.

Return type`torch.Generator`**Returns**

torch.Generator – Generator seeded with *seed*.

Parameters

- **seed** (*int*)
- **device** (*str* | *torch.device* | *None*)

`dmx.torch_utils.vector.set_default_float_dtype(dtype)`

Set the module-local default floating point dtype.

This setting is used by the tensor factories in this module when no dtype is supplied. It does not change PyTorch's global default dtype.

Parameters

dtype (*torch.dtype*) – Floating point dtype to use for subsequent helper allocations.

Return type`None`**Parameters**

dtype (*torch.dtype*)

`dmx.torch_utils.vector.tensor(x, device=None, dtype=None)`

Convert array-like input to a detached tensor.

Tensor inputs are cloned and detached. When *device* is omitted, tensor inputs keep their current device and non-tensor inputs use PyTorch's default tensor device. Explicit dtype always wins; otherwise the module-local float dtype is used, except MPS uses *float32*.

Parameters

- **x** – Tensor, NumPy array, or numeric sequence to convert.
- **device** (*DeviceLike*) – Optional target device.
- **dtype** (*Optional[torch.dtype]*) – Optional target dtype.

Return type`torch.Tensor`**Returns***torch.Tensor* – Converted tensor on the requested or inferred device.**Parameters**

- **x** (*List[int]* | *List[float]* | *Sequence[int]* | *Sequence[float]* | *torch.Tensor* | *List[List[int]]* | *List[List[float]]* | *ndarray*)
- **device** (*str* | *torch.device* | *None*)
- **dtype** (*torch.dtype* | *None*)

`dmx.torch_utils.vector.zeros(size, device=None, dtype=None)`

Create a floating point tensor of zeros.

Parameters

- **size** (*Union[int, Tuple[int, ...]]*) – Output tensor shape.
- **device** (*DeviceLike*) – Optional output device. When *None*, PyTorch’s default tensor device is used rather than *resolve_device* auto-detection.
- **dtype** (*Optional[torch.dtype]*) – Optional output dtype. When omitted, the module-local float dtype is used, except MPS allocations use *float32*.

Return type`torch.Tensor`**Returns***torch.Tensor* – Tensor of zeros with the requested shape, device, and dtype.**Parameters**

- **size** (*int* | *Tuple[int, ...]*)
- **device** (*str* | *torch.device* | *None*)
- **dtype** (*torch.dtype* | *None*)

`dmx.torch_utils.vector.zeros_like(x)`

Create a floating point zero tensor shaped like another tensor.

Parameters**x** (*torch.Tensor*) – Tensor providing shape and device.**Return type**`torch.Tensor`**Returns***torch.Tensor* – Zero tensor with the same shape and device as *x*. The dtype is this module’s default float dtype, except MPS uses *float32*.**Parameters****x** (*torch.Tensor*)

1.8 MPI API

The MPI helpers are optional and require `mpi4py` plus a working MPI runtime. Every participating rank must enter collective operations consistently. See [Installation](#) for installation options and [MPI4PY Estimation Example](#) for a complete workflow.

1.8.1 MPI Helpers for `dmx.stats`

Collective MPI primitives for `dmx.stats` models.

The public `*_mpi` functions parallelize sequence encoding, initialization, estimation, and log-density evaluation for the NumPy/SciPy model protocols in `dmx.stats.pdist`. They extend the local package-level operations in `dmx.stats`; higher-level MPI optimization loops live in `dmx.mpi4py.utils.estimation`.

Every rank in `MPI.COMM_WORLD` must participate in the collectives in the same order. Raw observations, encoders, and models are generally supplied on rank 0, while encoded chunks are rank-local and fitted models are generally returned only on rank 0. Importing this package requires the optional `mpi4py` dependency. The functions do not accept the distinct `dmx.bstats` or `dmx.torch_stats` protocols.

`dmx.mpi4py.stats.seq_encode_mpi(data, encoder=None, estimator=None, model=None, num_chunks=1, chunk_size=None)`

Distributes and encodes sequence data across MPI workers.

Parameters

- **data** (*Optional[Sequence[Any]]*) – The sequence data to encode.
- **encoder** (*Optional[DataSequenceEncoder]*, *optional*) – Encoder to use for encoding data.
- **estimator** (*Optional[ParameterEstimator]*, *optional*) – Estimator to derive encoder if not provided.
- **model** (*Optional[SequenceEncodableProbabilityDistribution]*, *optional*): Model to derive encoder if not provided.
- **num_chunks** (*int*, *optional*) – Number of data chunks to split for parallel processing. Defaults to 1.
- **chunk_size** (*Optional[int]*, *optional*) – Size of each data chunk. If specified, overrides `num_chunks`.

Return type

`typing.List[typing.Tuple[int, typing.Any]]`

Returns

List[Tuple[int, Any]] –

List of tuples containing chunk size and encoded data for each chunk.

Raises

ValueError – If neither encoder, estimator, nor model is provided on rank 0.

Parameters

- **data** (*Sequence[Any] | None*)
- **encoder** (*DataSequenceEncoder | None*)
- **estimator** (*ParameterEstimator | None*)
- **model** (*SequenceEncodableProbabilityDistribution | None*)
- **num_chunks** (*int*)
- **chunk_size** (*int | None*)

`dmx.mpi4py.stats.seq_estimate_mpi(enc_data, estimator=None, prev_estimate=None)`

Estimates model parameters in parallel using encoded data and a previous estimate.

Parameters

- **enc_data** (*List[Tuple[int, EncodedDataSequence]]*) – List of tuples containing chunk size and encoded data.
- **estimator** (*Optional[ParameterEstimator], optional*) – Parameter estimator for the model.
- **prev_estimate** (**Optional[SequenceEncodableProbabilityDistribution]**, – optional): Previous model estimate.

Return type

typing.Optional[*dmx.stats.pdist.SequenceEncodableProbabilityDistribution*]

Returns

Optional[SequenceEncodableProbabilityDistribution] –

Estimated model

parameters (on rank 0), None otherwise.

Raises

ValueError – If estimator or prev_estimate is not provided on rank 0.

Parameters

- **enc_data** (*List[Tuple[int, EncodedDataSequence]]*)
- **estimator** (*ParameterEstimator | None*)
- **prev_estimate** (*SequenceEncodableProbabilityDistribution | None*)

`dmx.mpi4py.stats.seq_initialize_mpi(enc_data, estimator, rng, p)`

Initializes model parameters in parallel using encoded data.

Parameters

- **enc_data** (*List[Tuple[int, EncodedDataSequence]]*) – List of tuples containing chunk size and encoded data.
- **estimator** (*ParameterEstimator*) – Parameter estimator for the model.
- **rng** (*RandomState*) – Random number generator for initialization.
- **p** (*float*) – Probability for subsampling data during initialization.

Return type

typing.Optional[*dmx.stats.pdist.SequenceEncodableProbabilityDistribution*]

Returns

Optional[SequenceEncodableProbabilityDistribution] –

Estimated model

parameters (on rank 0), None otherwise.

Parameters

- **enc_data** (*List[Tuple[int, EncodedDataSequence]]*)
- **estimator** (*ParameterEstimator*)
- **rng** (*RandomState*)
- **p** (*float*)

`dmx.mpi4py.stats.seq_log_density_mpi(enc_data, estimate=None, is_list=False)`

Computes log densities of encoded data in parallel.

Parameters

- **enc_data** (*Sequence[Tuple[int, EncodedDataSequence]]*) – Encoded data to compute log densities for.
- **estimate** (*Optional[SequenceEncodableProbabilityDistribution]*, – optional): Model estimate for log density computation.
- **is_list** (*bool, optional*) – If True, computes log densities for a list of estimates.

Return type

`typing.List[numpy.ndarray]`

Returns

List[np.ndarray] –

List of arrays containing log densities for each
data chunk.

Raises

ValueError – If estimate is not provided on rank 0.

Parameters

- **enc_data** (*Sequence[Tuple[int, EncodedDataSequence]]*)
- **estimate** (*SequenceEncodableProbabilityDistribution / Sequence[SequenceEncodableProbabilityDistribution] / None*)
- **is_list** (*bool*)

`dmx.mpi4py.stats.seq_log_density_sum_mpi(enc_data, estimate=None)`

Computes sum of log densities and total number of observations in parallel.

Parameters

- **enc_data** (*Sequence[Tuple[int, EncodedDataSequence]]*) – Encoded data to compute log densities for.
- **estimate** (*Optional[SequenceEncodableProbabilityDistribution]*, – optional): Model estimate for log density computation.

Return type

`typing.Tuple[int, float]`

Returns

Tuple[int, float] –

Total number of observations and sum of log
densities across all data.

Raises

ValueError – If estimate is not provided on rank 0.

Parameters

- **enc_data** (*Sequence[Tuple[int, EncodedDataSequence]]*)
- **estimate** (*SequenceEncodableProbabilityDistribution / None*)

1.8.2 MPI Helpers for `dmx.bstats`

Collective MPI primitives for `dmx.bstats` models.

The functions exported here distribute local observations from rank 0, combine Bayesian sufficient statistics, and return fitted models or likelihood results according to their documented rank semantics. They complement the local and PySpark-aware helpers in `dmx.bstats`; higher-level MPI optimization loops belong to `dmx.mpi4py.utils.bestimation`.

Every rank in `MPI.COMM_WORLD` must call these operations in the same order. Raw data and models are generally supplied on rank 0, and fitted models are generally returned only there. Importing this package requires the optional `mpi4py` dependency, and its objects must follow the `dmx.bstats` protocols, not the separate `dmx.stats` or `dmx.torch_stats` protocols.

`dmx.mpi4py.bstats.initialize_mpi(data, estimator, rng, p)`

Initialize MPI-based parallel data processing and estimate parameters.

This function distributes data processing across MPI processes, computes sufficient statistics locally, and combines them at the root process to estimate parameters using the provided estimator.

Parameters

- **data** (*Sequence[Any]*) – The input data to be processed.
- **estimator** (*ParameterEstimator*) – The estimator object for parameter estimation.
- **rng** (*RandomState*) – The random number generator for sampling.
- **p** (*float*) – The probability threshold for weighting observations.

Return type

`typing.Optional[dmx.bstats.pdist.ProbabilityDistribution]`

Returns

Optional[ProbabilityDistribution] –

The estimated model if called from
the root process (rank 0), otherwise *None*.

Parameters

- **data** (*Sequence[Any] | None*)
- **estimator** (*ParameterEstimator*)
- **rng** (*RandomState*)
- **p** (*float*)

`dmx.mpi4py.bstats.seq_encode_mpi(data, model, num_chunks=1, chunk_size=None)`

Encode data sequentially using MPI for parallel processing.

This function distributes data across MPI processes, performs encoding on each process, and collects the encoded results.

Parameters

- **data** (*Sequence[Any]*) – The input data to be encoded.
- **model** (*ProbabilityDistribution*) – The model object that provides the `seq_encode` method for encoding.
- **num_chunks** (*int, optional*) – The number of chunks to divide the data into. Defaults to 1.

- **chunk_size** (*Optional[int], optional*) – The size of each chunk. If provided, it overrides *num_chunks*.

Return type

`typing.List[typing.Tuple[int, typing.Any]]`

Returns

List[Tuple[int, Any]] –

A list of tuples, where each tuple contains:

- The size of the data chunk.
- The encoded data for that chunk.

Parameters

- **data** (*Sequence[Any] | None*)
- **model** (*ProbabilityDistribution | None*)
- **num_chunks** (*int*)
- **chunk_size** (*int | None*)

`dmx.mpi4py.bstats.seq_estimate_mpi(enc_data, estimator, prev_estimate)`

Estimate parameters using MPI-based parallel processing.

This function distributes encoded data across MPI processes, updates accumulators locally, and combines sufficient statistics at the root process to compute the final estimate.

Parameters

- **enc_data** (*List[Tuple[int, Any]]*) – Encoded data, where each tuple contains: - The size of the data chunk (*int*). - The encoded data (*EncodedDataSequence*).
- **estimator** (*ParameterEstimator*) – The estimator object for parameter estimation.
- **prev_estimate** (*ProbabilityDistribution*) – The previous estimate to be used during updates.

Return type

`typing.Optional[dmx.bstats.pdist.ProbabilityDistribution]`

Returns

Optional[ProbabilityDistribution] –

The estimated model if called from

the root process (rank 0), otherwise *None*.

Parameters

- **enc_data** (*List[Tuple[int, Any]]*)
- **estimator** (*ParameterEstimator*)
- **prev_estimate** (*ProbabilityDistribution | None*)

`dmx.mpi4py.bstats.seq_log_density_mpi(enc_data, estimate, is_list=False)`

Compute log densities for encoded data sequences in parallel using MPI.

Parameters

- **enc_data** (*Sequence[Tuple[int, Any]]*) – Encoded data sequences, where each tuple contains: - The size of the data chunk (*int*). - The encoded data sequence (*EncodedDataSequence*).

- **estimate** (*ProbabilityDistribution*) – The probability distribution object used to compute log densities.
- **is_list** (*bool, optional*) – Whether to compute log densities for multiple probability distributions. Defaults to *False*.

Return type

`typing.List[numpy.ndarray]`

Returns

List[np.ndarray] – A list of log densities for the encoded data sequences.

Parameters

- **enc_data** (*Sequence[Tuple[int, Any]]*)
- **estimate** (*ProbabilityDistribution | None*)
- **is_list** (*bool*)

`dmx.mpi4py.bstats.seq_log_density_sum_mpi(enc_data, estimate)`

Compute the total number of observations and the sum of log densities in parallel using MPI.

Parameters

- **enc_data** (*Sequence[Tuple[int, Any]]*) – Encoded data sequences, where each tuple contains:
 - The size of the data chunk (*int*).
 - The encoded data sequence (*EncodedDataSequence*).
- **estimate** (*ProbabilityDistribution*) – The probability distribution object used to compute log densities.

Return type

`typing.Tuple[int, float]`

Returns

Tuple[int, float] –

A tuple containing:

- *nobs* (*int*): The total number of observations.
- *ll* (*float*): The sum of log densities for all encoded data sequences.

Parameters

- **enc_data** (*Sequence[Tuple[int, Any]]*)
- **estimate** (*ProbabilityDistribution | None*)

1.8.3 MPI Workflow Utilities

Shared infrastructure for MPI-enabled modeling workflows.

This package separates orchestration utilities from the low-level collectives in `dmx.mpi4py.stats` and `dmx.mpi4py.bstats`. Import optimization, automatic-modeling, embedding, and option helpers from their defining `dmx.mpi4py.utils` submodules. The package root intentionally exposes only `get_runtime_attr()`, which supports lazy access to optional MPI-dependent objects and avoids importing those workflow modules eagerly.

The submodule functions are collective operations unless documented otherwise; all MPI ranks must enter them consistently, and callers must install the optional MPI dependencies before invoking them.

`dmx.mpi4py.utils.get_runtime_attr(module_name, attr_name)`

Load an attribute lazily from a module.

Return type`typing.Any`**Parameters**

- **module_name** (*str*)
- **attr_name** (*str*)

These modules build higher-level fitting and embedding workflows on the low-level collective operations. They share the package-level MPI participation requirements described in [MPI API](#).

MPI Estimation Workflows**Automatic Bayesian mixtures**

Automatic MPI estimation helpers used by heterogeneous embedding workflows.

```
dmx.mpi4py.utils.automatic.get_dpm_mixture_mpi(data, estimator=None, max_comp=20, rng=None,
                                                max_its=1000, print_iter=100,
                                                mix_threshold_count=0.5)
```

Fit and prune a Dirichlet process mixture with MPI.

This is collective over `MPI.COMM_WORLD`; every rank must call it in the same order. Raw *data* is required only on rank 0, where the component estimator is inferred when omitted. The estimator is broadcast to workers, DPM fitting is performed collectively, pruning is computed on rank 0, and the pruned mixture is broadcast back to every rank. Rank 0 prints the retained weights and component count.

Parameters

- **data** (*Optional[Sequence[Any]]*) – Data to model. Must be defined on rank 0.
- **estimator** (*Optional[ParameterEstimator]*) – Base estimator to use. If omitted, rank 0 infers it from *data*.
- **max_comp** (*int*) – Maximum number of components in the mixture.
- **rng** (*Optional[numpy.random.RandomState]*) – Random number generator used during mixture optimization.
- **max_its** (*int*) – Maximum number of iterations for optimization.
- **print_iter** (*int*) – Frequency of printing iteration progress.
- **mix_threshold_count** (*float*) – Component-count threshold. On rank 0 this is divided by `len(data)` to form the minimum retained weight.

Return type`dmx.bstats.mixture.MixtureDistribution`**Returns**

MixtureDistribution – Pruned mixture distribution on every rank.

Raises

- **ValueError** – If *data* is *None* on rank 0.
- **RuntimeError** – If collective DPM optimization does not return a model on rank 0.

Parameters

- **data** (*Sequence[Any] | None*)
- **estimator** (*ParameterEstimator | None*)
- **max_comp** (*int*)

- **rng** (*RandomState* | *None*)
- **max_its** (*int*)
- **print_iter** (*int*)
- **mix_threshold_count** (*float*)

Bayesian estimation

Estimation functions for bstats with mpi4py support.

```
dmx.mpi4py.utils.bestimation.optimize_mpi(data, estimator, max_its=10, delta=1.0e-9,
                                           init_estimator=None, init_p=0.1, rng=RandomState(),
                                           prev_estimate=None, vdata=None, enc_data=None,
                                           enc_vdata=None, out=sys.stdout, print_iter=1,
                                           num_chunks=1)
```

Run MPI EM estimation for a *dmx.bstats* estimator.

This is collective over *MPI.COMM_WORLD*; every rank must call it in the same order. Raw *data* and *vdata* are required only on rank 0, but pre-encoded *enc_data* and *enc_vdata* must be supplied on every rank because their availability is checked with an all-reduce. Iterative model objects live on rank 0, while worker ranks generally return *None* from the lower-level estimation helpers. Likelihood sums are reduced across all ranks and progress is written only by rank 0.

Parameters

- **data** (*Optional[Sequence[DATUM_TYPE]]*) – Observed data compatible with *estimator*. Required on rank 0 unless encoded chunks are supplied on all ranks.
- **estimator** (*ParameterEstimator*) – Estimator for the target model. Rank 0 must receive a usable estimator.
- **max_its** (*int*) – Maximum number of EM iterations to perform.
- **delta** (*Optional[float]*) – Stop when the signed likelihood improvement is less than *delta*. If *None*, likelihood-decreasing updates are accepted.
- **init_estimator** (*Optional[ParameterEstimator]*) – Estimator used for initialization. If *None*, use *estimator*.
- **init_p** (*float*) – Initialization proportion in $(0, 1]$.
- **rng** (*RandomState*) – *RandomState* used to set seeds for initialization. The default object is process-local and is advanced across calls.
- **prev_estimate** (*Optional[ProbabilityDistribution]*) – Optional previous model estimate, meaningful on rank 0 and consistent with *estimator*.
- **vdata** (*Optional[Sequence[DATUM_TYPE]]*) – Optional validation set on rank 0.
- **enc_data** (*Optional[List[Tuple[int, ENCODED_SEQ]]]*) – Optional rank-local encoded chunks. Formed from root *data* if omitted.
- **enc_vdata** (*Optional[List[Tuple[int, ENCODED_SEQ]]]*) – Optional rank-local encoded validation chunks.
- **out** (*IO*) – Rank 0 stream for EM progress.
- **print_iter** (*int*) – Print progress every *print_iter* iterations.
- **num_chunks** (*int*) – Number of chunks for encoded data.

Return type

`typing.Optional[dmx.bstats.pdist.ProbabilityDistribution]`

Returns

Optional[ProbabilityDistribution] – Estimated model on rank 0 and *None* on worker ranks.

Raises

- **ValueError** – If rank 0 has no raw data and encoded data are not present on every rank, or if *init_p* is outside *(0, 1]*.
- **RuntimeError** – If encoded data expected on a rank are missing.

Parameters

- **data** (*Sequence[DATUM_TYPE] | None*)
- **estimator** (*ParameterEstimator*)
- **max_its** (*int*)
- **delta** (*float | None*)
- **init_estimator** (*ParameterEstimator | None*)
- **init_p** (*float*)
- **rng** (*RandomState*)
- **prev_estimate** (*ProbabilityDistribution | None*)
- **vdata** (*Sequence[DATUM_TYPE] | None*)
- **enc_data** (*List[Tuple[int, ENCODED_SEQ]] | None*)
- **enc_vdata** (*List[Tuple[int, ENCODED_SEQ]] | None*)
- **out** (*IO*)
- **print_iter** (*int*)
- **num_chunks** (*int*)

Non-Bayesian estimation

MPI-enabled estimation helpers for *dmx.stats* models.

```
dmx.mpi4py.utils.estimation.best_of_mpi(data, vdata, est, trials, max_its, max_its_cnt, init_p, delta, rng,
                                         init_estimator=None, enc_data=None, enc_vdata=None,
                                         out=sys.stdout, print_iter=1)
```

Run multiple MPI EM initializations and keep the best model.

This is collective over *MPI.COMM_WORLD*; every rank must call it in the same order. Raw data are needed only on rank 0 unless every rank supplies its local encoded chunks. Each trial starts from a randomized initialization, runs at least one EM iteration, scores by validation likelihood when validation data are supplied, and then refines the best model through *optimize_mpi* for *max_its_cnt* additional iterations. Progress and the returned model are root-only.

Parameters

- **data** (*Optional[Sequence[T]]*) – Data of type *T*. If *None* on rank 0, *enc_data* must be provided on every rank.
- **vdata** (*Optional[Sequence[T]]*) – Optional validation set on rank 0.
- **est** (*ParameterEstimator*) – Estimator for model to be estimated.

- **trials** (*int*) – Number of randomized initial conditions to try; coerced to at least one.
- **max_its** (*int*) – Maximum number of EM iterations per trial; coerced to at least one.
- **max_its_cnt** (*int*) – Number of additional *optimize_mpi* iterations for the best trial model.
- **init_p** (*float*) – Initialization proportion in $(0, 1]$.
- **delta** (*float*) – Stop when signed likelihood improvement is less than *delta*.
- **rng** (*RandomState*) – *RandomState* used to set trial seeds. It is advanced across calls.
- **init_estimator** (*Optional[ParameterEstimator]*) – Optional estimator used for fitting.
- **enc_data** (*Optional[List[Tuple[int, EncodedDataSequence]]]*) – Optional rank-local encoded chunks.
- **enc_vdata** (*Optional[Sequence[Tuple[int, EncodedDataSequence]]]*) – Optional rank-local encoded validation chunks.
- **out** (*IO*) – Rank 0 text output stream.
- **print_iter** (*int*) – Print progress every *print_iter* iterations.

Return type

typing.Optional[[dmx.stats.pdist.SequenceEncodableProbabilityDistribution](#)]

Returns

Optional[SequenceEncodableProbabilityDistribution] – Best fitting model on rank 0 and *None* on worker ranks.

Raises

- **ValueError** – If rank 0 has no raw data and encoded data are not present on every rank, or if *init_p* is outside $(0, 1]$.
- **RuntimeError** – If encoded data expected on a rank are missing.

Parameters

- **data** (*Sequence[T] | None*)
- **vdata** (*Sequence[T] | None*)
- **est** ([ParameterEstimator](#))
- **trials** (*int*)
- **max_its** (*int*)
- **max_its_cnt** (*int*)
- **init_p** (*float*)
- **delta** (*float*)
- **rng** (*RandomState*)
- **init_estimator** ([ParameterEstimator](#) | *None*)
- **enc_data** (*List[Tuple[int, EncodedDataSequence]] | None*)
- **enc_vdata** (*Sequence[Tuple[int, EncodedDataSequence]] | None*)
- **out** (*IO*)
- **print_iter** (*int*)

```
dmx.mpi4py.utils.estimate.optimize_mpi(data, estimator, max_its=10, delta=1.0e-9,
                                       init_estimator=None, init_p=0.1, rng=RandomState(),
                                       prev_estimate=None, vdata=None, enc_data=None,
                                       enc_vdata=None, out=sys.stdout, print_iter=1,
                                       num_chunks=1)
```

Run EM estimation with MPI until convergence or *max_its*.

This is collective over *MPI.COMM_WORLD*; every rank must call it in the same order. Raw *data* and *vdata* are required only on rank 0, but pre-encoded *enc_data* and *enc_vdata* must be supplied on every rank because their availability is checked with an all-reduce. Iterative model objects live on rank 0, while worker ranks generally receive and return *None* from the lower-level estimation helpers. Likelihood sums are reduced across all ranks and progress is written only by rank 0.

Parameters

- **data** (*Optional[Sequence[T]]*) – Observed data compatible with *estimator*. Required on rank 0 unless encoded chunks are supplied on all ranks.
- **estimator** (*ParameterEstimator*) – Estimator for the target model. Rank 0 must receive a usable estimator.
- **max_its** (*int*) – Maximum number of EM iterations to perform.
- **delta** (*Optional[float]*) – Stop when the signed likelihood improvement is less than *delta*. If *None*, likelihood-decreasing updates are accepted.
- **init_estimator** (*Optional[ParameterEstimator]*) – Estimator used for initialization. If *None*, use *estimator*.
- **init_p** (*float*) – Initialization proportion in $(0, 1]$.
- **rng** (*RandomState*) – *RandomState* used to set seeds for EM initialization. The default object is process-local and is advanced across calls.
- **vdata** (*Optional[Sequence[T]]*) – Optional validation set.
- **prev_estimate** (*Optional[SequenceEncodableProbabilityDistribution]*) – Optional previous model estimate, meaningful on rank 0 and consistent with *estimator*.
- **enc_data** (*Optional[List[Tuple[int, EncodedDataSequence]]]*) – Optional rank-local encoded chunks. Formed from root *data* if omitted.
- **enc_vdata** (*Optional[List[Tuple[int, EncodedDataSequence]]]*) – Optional rank-local encoded validation chunks.
- **out** (*IO*) – Rank 0 stream for EM progress.
- **print_iter** (*int*) – Print progress every *print_iter* iterations.
- **num_chunks** (*int*) – Number of chunks for encoded data.

Return type

`typing.Optional[dmx.stats.pdist.SequenceEncodableProbabilityDistribution]`

Returns

Optional[SequenceEncodableProbabilityDistribution] – Estimated model on rank 0 and *None* on worker ranks.

Raises

- **ValueError** – If rank 0 has no raw data and encoded data are not present on every rank, or if *init_p* is outside $(0, 1]$.
- **RuntimeError** – If encoded data expected on a rank are missing.

Parameters

- **data** (*Sequence*[*T*] | *None*)
- **estimator** (*ParameterEstimator*)
- **max_its** (*int*)
- **delta** (*float* | *None*)
- **init_estimator** (*ParameterEstimator* | *None*)
- **init_p** (*float*)
- **rng** (*RandomState*)
- **prev_estimate** (*SequenceEncodableProbabilityDistribution* | *None*)
- **vdata** (*Sequence*[*T*] | *None*)
- **enc_data** (*List*[*Tuple*[*int*, *EncodedDataSequence*]] | *None*)
- **enc_vdata** (*List*[*Tuple*[*int*, *EncodedDataSequence*]] | *None*)
- **out** (*IO*)
- **print_iter** (*int*)
- **num_chunks** (*int*)

MPI Embedding Utilities

MPI helpers for heterogeneous UMAP embeddings.

```
dmx.mpi4py.utils.humap.humap_mpi(data, max_components=30, mix_threshold_count=0.5, max_its=1000,  
                                  print_iter=100, seed=None, comp_estimator=None, mix_model=None,  
                                  umap_kwargs=None)
```

Fit UMAP on posteriors of a DPM mixture model.

This function participates in MPI collectives only when it fits a mixture model. In that branch, every rank must call it in the same order and raw *data* is required only on rank 0. If *mix_model* is precomputed, pass it consistently on all ranks so every process follows the same collective branch. Mixture fitting returns a model on all ranks, but posterior evaluation and UMAP fitting happen only on rank 0.

Parameters

- **data** (*Optional*[*Sequence*[*T*]]) – Input data sequence. Must be defined on rank 0.
- **max_components** (*int*) – Maximum number of components for the mixture model.
- **mix_threshold_count** (*float*) – Threshold for mixture component selection.
- **max_its** (*int*) – Maximum number of DPM fitting iterations.
- **print_iter** (*int*) – Number of iteration to print while fitting the DPM.
- **seed** (*Optional*[*int*]) – Random seed for mixture fitting. UMAP randomness is controlled through *umap_kwargs*, such as *random_state*.
- **comp_estimator** (*Optional*[*ParameterEstimator*]) – Component estimator for mixture model.
- **mix_model** (*Optional*[*MIX_TYPE*]) – Precomputed mixture model. Supplying one skips collective mixture fitting.
- **umap_kwargs** (*Optional*[*Dict*[*str*, *Any*]]) – Kwargs for UMAP fit. The dictionary is mutated in place to set *metric*=*"hellinger"*.

Return type

`typing.Optional[typing.Tuple[typing.Any, typing.Union[dmx.stats.mixture.MixtureDistribution, dmx.bstats.mixture.MixtureDistribution], umap.UMAP, numpy.ndarray]]`

Returns

Optional[Tuple[Any, MIX_TYPE, UMAP, numpy.ndarray]] – On rank 0, returns embeddings with shape $(len(data), n_components)$, the mixture model, fitted UMAP object, and posterior matrix with shape $(len(data), mix_model.num_components)$. Worker ranks return *None*.

Raises

- **ValueError** – If *data* is missing on rank 0, *max_components* ≤ 1 , or the mixture has no components.
- **RuntimeError** – If collective mixture fitting does not return a model.

Parameters

- **data** (*Sequence[T] | None*)
- **max_components** (*int*)
- **mix_threshold_count** (*float*)
- **max_its** (*int*)
- **print_iter** (*int*)
- **seed** (*int | None*)
- **comp_estimator** (*ParameterEstimator | None*)
- **mix_model** (*MixtureDistribution | MixtureDistribution | None*)
- **umap_kwargs** (*Dict[str, Any] | None*)

MPI Option Utilities

Helper functions for mpi4py.

`dmx.mpi4py.utils.optsutil.pickle_on_master(x, filename)`

Write an object to a pickle file only on rank 0.

This helper is not collective; it only checks the caller's *MPI.COMM_WORLD* rank. Rank 0 requires a non-*None* object and opens *filename* in overwrite mode. Worker ranks return without opening the file. Standard pickle security rules apply: only unpickle files from trusted sources.

Parameters

- **x** (*Any*) – Object to pickle on rank 0.
- **filename** (*str*) – Destination file path.

Raises

- **ValueError** – If *x* is *None* on rank 0.
- **OSError** – If rank 0 cannot open or write *filename*.
- **pickle.PickleError** – If rank 0 cannot pickle *x*.

Return type

None

Parameters

- `x` (*Any*)
- `filename` (*str*)

1.9 User Defined Classes

```
import sys
import os

sys.path.insert(0, os.path.abspath('path/to/dmx-learn'))

import numpy as np
from numpy.random import RandomState
from typing import Optional, Sequence, Dict, Union, Tuple, Any
from dmx.arithmetic import *
from dmx.stats.pdist import (SequenceEncodableProbabilityDistribution,
                             ParameterEstimator,
                             DistributionSampler,
                             StatisticAccumulatorFactory,
                             SequenceEncodableStatisticAccumulator,
                             DataSequenceEncoder,
                             EncodedDataSequence)
from dmx.utils.estimation import optimize
import matplotlib.pyplot as plt
```

1.9.1 Outline: User-Defined dmx-learn Class

1.9.2 SequenceEncodableProbabilityDistribution

1. **Log-likelihood:** First we work out the log-likelihood for a single observation.
2. **Vectorize the log-likelihood:** Next we think about how we can format our data for fast vectorized updates.
3. **Create DataEncodedSequence:** Stores encoded data.
4. **Write the DataSequenceEncoder:** This processes our data for use with vectorized calls.
5. **Write a Sampler:** This allows us to draw samples from the distribution.

1.9.3 SequenceEncodableStatisticAccumulator

1. **Determine Sufficient Statistics:** Follows from exponential family, defines the member variables of the object.
2. **Write Key Functionality:** Allows us to share parameters with other distribution instances.
3. **Update:** Define method for sufficient statistic accumulation.
4. **Initialize:** Define initialization for the sufficient statistics.
5. **Write a factory object:** Standard factory object for the accumulator.

1.9.4 ParameterEstimator

1. **Define the wrapper:** This is what users most commonly interact with to estimate distributions.
2. **Form estimates:** Use sufficient statistics gathered in the accumulator to estimate the distribution.

Once we have filled out our univariate Gaussian distribution, we can compare it with the standard dmx-learn style mixture wrapper.

1.9.5 One-Dimensional Gaussian Mixture Model (GMM)

1.9.6 Introduction

A One-Dimensional Gaussian Mixture Model (GMM) is a probabilistic model that assumes that the data is generated from a mixture of several one-dimensional Gaussian distributions with unknown parameters. This model is useful for clustering and density estimation in one-dimensional data.

1.9.7 Mathematical Formulation

A one-dimensional GMM is defined as follows:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \sigma_k^2)$$

where: - $\mathbf{K}$ is the number of Gaussian components. - π_k is the weight of the k -th Gaussian component, satisfying $\sum_{k=1}^K \pi_k = 1$. - $\mathcal{N}(x | \mu_k, \sigma_k^2)$ is the one-dimensional Gaussian distribution with mean μ_k and variance σ_k^2 .

The one-dimensional Gaussian distribution is given by:

$$\mathcal{N}(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

where: - σ^2 is the variance of the Gaussian distribution.

1.9.8 SequenceEncodableProbabilityDistribution

We first define a skeleton of the SequenceEncodableProbabilityDistribution. We know that the distribution requires parameters μ , σ^2 , and mixing weights π . These must be passed to the constructor. Note that the argument **name** must also be included. We won't get into the reason for this, but make sure to include it for consistency with other dmx-learn distributions.

```
class GmmDistribution(SequenceEncodableProbabilityDistribution):
    def __init__(self, mu: Union[Sequence[float], np.ndarray], sigma2: Union[Sequence[float], np.ndarray], w: Union[Sequence[float], np.ndarray], name: Optional[str] = None):
        self.mu = np.asarray(mu)
        self.sigma2 = np.asarray(sigma2)
        self.w = np.asarray(w)
        self.name = name

        self.log_const = -0.5 * np.log(2.0 * np.pi)

    def __str__(self) -> str:
        return 'GmmDistribution(mu=%s, sigma2=%s, w=%s, name=%s)' % (repr(self.mu.tolist()), repr(self.sigma2.tolist()), repr(self.w.tolist()), repr(self.name))

    def log_density(self, x: float) -> float:
        pass

    def density(self, x: float) -> float:
```

(continues on next page)

(continued from previous page)

```

    return np.exp(self.log_density(x))

def seq_log_density(self, x) -> np.ndarray:
    pass

def sampler(self, seed: Optional[int] = None):
    pass

def dist_to_encoder(self):
    pass

def estimator(self, pseudo_count: Optional[float] = None):
    pass

```

1.9.9 Log Density of a Univariate Gaussian Mixture Model

The next step is generally to define the likelihood on the log scale in terms of the parameters set as member variables for the `SequenceEncodableProbabilityDistribution`. This is detailed below.

A univariate Gaussian mixture model (GMM) can be expressed as:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \sigma_k^2)$$

where:

- K is the number of Gaussian components.
- π_k is the mixing weight for component k (with $\sum_{k=1}^K \pi_k = 1$).
- $\mathcal{N}(x|\mu_k, \sigma_k^2)$ is the Gaussian density function given by:

$$\mathcal{N}(x|\mu_k, \sigma_k^2) = \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(x - \mu_k)^2}{2\sigma_k^2}\right)$$

To evaluate the log density of the GMM, we can use the *logsumexp* trick to avoid numerical underflow or overflow when dealing with exponentials. The log density can be computed as follows:

1. Compute the log densities for each component:

$$\log p_k(x) = \log \pi_k + \log \mathcal{N}(x|\mu_k, \sigma_k^2)$$

This expands to:

$$\log p_k(x) = \log \pi_k - \frac{1}{2} \log(2\pi\sigma_k^2) - \frac{(x - \mu_k)^2}{2\sigma_k^2}$$

2. Use `logsumexp` to compute the log density of the mixture:

The log density of the GMM can be computed as:

$$\log p(x) = \log \left(\sum_{k=1}^K \pi_k \mathcal{N}(x|\mu_k, \sigma_k^2) \right)$$

Using the *logsumexp* function, we can rewrite this as:

$$\log p(x) = \log \left(\sum_{k=1}^K \exp(\log \pi_k + \log \mathcal{N}(x|\mu_k, \sigma_k^2)) \right)$$

This can be expressed as:

$$\log p(x) = \log \left(\sum_{k=1}^K \exp \left(\log \pi_k - \frac{1}{2} \log(2\pi\sigma_k^2) - \frac{(x - \mu_k)^2}{2\sigma_k^2} \right) \right)$$

3. Final Expression:

Therefore, the log density of the univariate Gaussian mixture model can be computed using:

$$\log p(x) = \log \left(\sum_{k=1}^K \exp \left(\log \pi_k - \frac{1}{2} \log(2\pi\sigma_k^2) - \frac{(x - \mu_k)^2}{2\sigma_k^2} \right) \right)$$

This formulation allows for stable computation of the log density of a Gaussian mixture model using the *logsumexp* trick, which is particularly useful in practice to avoid numerical issues.

```
class GmmDistribution(SequenceEncodableProbabilityDistribution):
    def __init__(self, mu: Union[Sequence[float], np.ndarray], sigma2: Union[Sequence[float], np.ndarray], w: Union[Sequence[float], np.ndarray], name: Optional[str] = None):
        self.mu = np.asarray(mu)
        self.sigma2 = np.asarray(sigma2)
        self.w = np.asarray(w)
        self.name = name

        self.log_const = -0.5*np.log(2.0 * np.pi)

    def __str__(self) -> str:
        return 'GmmDistribution(mu=%s, sigma2=%s, w=%s, name=%s)' % (repr(self.mu.tolist()), repr(self.sigma2.tolist()), repr(self.w.tolist()), repr(self.name))

    def log_density(self, x: float) -> float:
        # eval log-density for each component
        ll = self.log_const - 0.50*(x-self.mu) ** 2 / self.sigma2 - 0.5*np.log(self.sigma2) + np.log(self.w)
        max_ = np.max(ll)
        # subtract max and exponentiate
        np.exp(ll-max_, out=ll)
        # finish log-sum-exp
        rv = np.log(np.sum(ll)) + max_
        return rv

    def density(self, x: float) -> float:
        return np.exp(self.log_density(x))

    def seq_log_density(self, x) -> np.ndarray:
        pass

    def sampler(self, seed: Optional[int] = None):
        pass

    def dist_to_encoder(self):
        pass
```

(continues on next page)

(continued from previous page)

```
def estimator(self, pseudo_count: Optional[float] = None):
    pass
```

1.9.10 EncodedDataSequence and the DataSequenceEncoder objects

To make calculations fast, we want to think of vectorizing our `log_density` function call. Under the hood, this requires us to encode our data. Put another way, we want to pre-process the data passed to our object so we can perform fast vectorized operations. The `DataSequenceEncoder` object is responsible for encoding our data into a format useful for repeated vectorized operations. The output data is stored in an `EncodedDataSequence` object. This object also allows for type checking if desired.

A good way to think about how this will all look is to first consider a vectorized form of the `log_density` function. Assume the data is a one-dimensional numpy array (this is the form the encoded data will take). We can write out the density in vectorized form as seen below.

```
def seq_log_density(x: np.ndarray) -> np.ndarray:

    ll = -0.5 * (x[:, None] - self.mu) ** 2 / self.sigma2 - 0.5 * np.log(self.sigma2) +
    ↪ self.log_const + np.log(self.w)
    max_ = np.max(ll, axis=1, keepdims=True)
    np.exp(ll - max_, out=ll)
    ll = np.log(np.sum(ll, axis=1, keepdims=False))
    ll += max_.flatten()

    return ll
```

The `EncodedDataSequence` should store the processed data (which happens to be a numpy array for floats).

```
class GmmEncodedDataSequence(EncodedDataSequence):

    def __init__(self, data: np.ndarray):
        super().__init__(data=data)

    def __repr__(self) -> str:
        return f'GmmEncodedDataSequence(data={self.data})'
```

The `DataSequenceEncoder` object must implement `__str__()`, `__eq__()`, and `seq_encode()`. The method `seq_encode()` should take the data and encode it. The result is returned as an `EncodedDataSequence` object. Note that this is also the place to check for data compatibility (i.e. GMM can't handle NaN or inf values).

The `__eq__()` method is implemented to check if two `DataSequenceEncoder` objects are the same. This helps with avoiding multiple encodings under the hood when nesting dmx-learn functions.

```
class GmmDataEncoder(DataSequenceEncoder):

    def __str__(self) -> str:
        return 'GmmDataEncoder'

    def __eq__(self, other) -> bool:
        return isinstance(other, GmmDataEncoder)

    def seq_encode(self, x: Union[Sequence[float], np.ndarray]) ->
```

(continues on next page)

(continued from previous page)

```

→ 'GmmEncodedDataSequence':
    rv = np.asarray(x, dtype=float)

    if np.any(np.isnan(rv)) or np.any(np.isinf(rv)):
        raise Exception('GmmDistribution requires support x in (-inf,inf).')

    return GmmEncodedDataSequence(data=rv)

```

We can now fill in the `seq_log_density()` function with proper type hints. We also fill out `dist_to_encoder()`, which returns the appropriate `DataSequenceEncoder` object for encoding data.

```

class GmmDistribution(SequenceEncodableProbabilityDistribution):

    def __init__(self, mu: Union[Sequence[float], np.ndarray], sigma2:
→ Union[Sequence[float], np.ndarray], w: Union[Sequence[float], np.ndarray], name:
→ Optional[str] = None):
        self.mu = np.asarray(mu)
        self.sigma2 = np.asarray(sigma2)
        self.w = np.asarray(w)
        self.name = name

        self.log_const = -0.5 * np.log(2.0 * np.pi)

    def __str__(self) -> str:
        return 'GmmDistribution(mu=%s, sigma2=%s, w=%s, name=%s)' % (repr(self.mu.
→ tolist()), repr(self.sigma2.tolist()), repr(self.w.tolist()), repr(self.name))

    def log_density(self, x: float) -> float:
        # eval log-density for each component
        ll = self.log_const - 0.5 * (x - self.mu) ** 2 / self.sigma2 - 0.5 * np.log(self.
→ sigma2) + np.log(self.w)
        max_ = np.max(ll)
        # subtract max and exponentiate
        np.exp(ll - max_, out=ll)
        # finish log-sum-exp
        rv = np.log(np.sum(ll)) + max_
        return rv

    def density(self, x: float) -> float:
        return np.exp(self.log_density(x))

    def seq_log_density(self, x: GmmEncodedDataSequence) -> np.ndarray:
        # Type check
        if not isinstance(x, GmmEncodedDataSequence):
            raise Exception('GmmEncodedDataSequence requires for seq_log_density.')

        # Evaluate the vectorized log-density as before
        ll = -0.5 * (x.data[:, None] - self.mu) ** 2 / self.sigma2 - 0.5 * np.log(self.
→ sigma2) + self.log_const + np.log(self.w)
        max_ = np.max(ll, axis=1, keepdims=True)
        np.exp(ll - max_, out=ll)
        ll = np.log(np.sum(ll, axis=1, keepdims=False))

```

(continues on next page)

(continued from previous page)

```

    ll += max_.flatten()

    return ll

def dist_to_encoder(self) -> GmmDataEncoder:
    return GmmDataEncoder()

def sampler(self, seed: Optional[int] = None):
    pass

def estimator(self, pseudo_count: Optional[float] = None):
    pass

```

1.9.11 DistributionSampler

Next we create the `DistributionSampler`. The sampler allows us to draw samples from a fitted distribution. `DistributionSampler` objects are generally realized through the method `sampler` in the `SequenceEncodableProbabilityDistribution`.

1.9.12 Sampling the GMM

1. Draw a label from the mixture weights: $z_i \sim \pi$
2. Sample an observation conditioned on the label drawn: $x_i | z_i = k \sim N(\mu_k, \sigma_k^2)$

Below is a vectorized implementation of GMM sampling in the `sample` method.

```

class GmmSampler(DistributionSampler):

    def __init__(self, dist: GmmDistribution, seed: Optional[int] = None):
        self.rng = RandomState(seed)
        self.dist = dist

    def sample(self, size: Optional[int] = None) -> Union[float, np.ndarray]:
        ncomps = len(self.dist.w)
        if size:
            rv = np.zeros(size)
            idx = np.arange(size)
            self.rng.shuffle(idx)
            z = self.rng.choice(ncomps, p=self.dist.w, replace=True, size=size)
            z = np.bincount(z, minlength=ncomps)

            i0 = 0
            for xi, xc in enumerate(z):
                if xc > 0:
                    i1 = i0 + xc
                    rv[idx[i0:i1]] = self.rng.normal(loc=self.dist.mu[xi], scale=np.
→ sqrt(self.dist.sigma2[xi]), size=xc)
                    i0 += xc

            return rv
        else:
            z = self.rng.choice(ncomps, p=self.dist.w)

```

(continues on next page)

(continued from previous page)

```

rv = self.rng.randn() * np.sqrt(self.dist.sigma2[z]) + self.dist.mu[z]

return float(rv)

```

We can now update the sampler function in the SequenceEncodableProbabilityDistribution.

```

class GmmDistribution(SequenceEncodableProbabilityDistribution):

    def __init__(self, mu: Union[Sequence[float], np.ndarray], sigma2:
↳ Union[Sequence[float], np.ndarray], w: Union[Sequence[float], np.ndarray], name:
↳ Optional[str] = None):
        self.mu = np.asarray(mu)
        self.sigma2 = np.asarray(sigma2)
        self.w = np.asarray(w)
        self.name = name

        self.log_const = -0.5*np.log(2.0 * np.pi)

    def __str__(self) -> str:
        return 'GmmDistribution(mu=%s, sigma2=%s, w=%s, name=%s)' % (repr(self.mu.
↳ tolist(), repr(self.sigma2.tolist(), repr(self.w.tolist(), repr(self.name))

    def log_density(self, x: float) -> float:
        # eval log-density for each component
        ll = self.log_const - 0.5*(x-self.mu) ** 2 / self.sigma2 - 0.5*np.log(self.
↳ sigma2) + np.log(self.w)
        max_ = np.max(ll)
        # subtract max and exponentiate
        np.exp(ll-max_, out=ll)
        # finish log-sum-exp
        rv = np.log(np.sum(ll)) + max_
        return rv

    def density(self, x: float) -> float:
        return np.exp(self.log_density(x))

    def seq_log_density(self, x: GmmEncodedDataSequence) -> np.ndarray:
        # Type check
        if not isinstance(x, GmmEncodedDataSequence):
            raise Exception('GmmEncodedDataSequence requires for seq_log_density.')

        # Evaluate the vetorized log-density as before
        ll = -0.5*(x.data[:, None] - self.mu)**2 / self.sigma2 - 0.5*np.log(self.sigma2)
↳ + self.log_const + np.log(self.w)
        max_ = np.max(ll, axis=1, keepdims=True)
        np.exp(ll-max_, out=ll)
        ll = np.log(np.sum(ll, axis=1, keepdims=False))
        ll += max_.flatten()

        return ll

    def dist_to_encoder(self) -> GmmDataEncoder:

```

(continues on next page)

(continued from previous page)

```

return GmmDataEncoder()

def sampler(self, seed: Optional[int] = None) -> GmmSampler:
    return GmmSampler(dist=self, seed=seed)

def estimator(self, pseudo_count: Optional[float] = None):
    pass

```

We need to write the estimator to complete the distribution. We will return to this later.

1.9.13 SequenceEncodableStatisticAccumulator

Next we will write the `SequenceEncodableStatisticAccumulator` which is used to aggregate sufficient statistics. To identify the sufficient statistics and the calculation involved in tracking them, we can refer to the exponential family form of the distribution. In the case of the univariate GMM, it is easier to refer back to the E-step of the EM algorithm.

1.9.14 Expectation-Maximization Algorithm

To estimate the parameters of a one-dimensional GMM, we typically use the Expectation-Maximization (EM) algorithm, which consists of two steps:

1. **Expectation Step (E-step):** Calculate the expected value of the log-likelihood function, given the current estimates of the parameters.

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(x_n | \mu_k, \sigma_k^2)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_n | \mu_j, \sigma_j^2)}$$

where γ_{nk} is the responsibility that component k takes for data point n .

2. **Maximization Step (M-step):** Update the parameters using the expected values computed in the E-step.

$$\pi_k = \frac{N_k}{N}$$

$$\mu_k = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} x_n$$

$$\sigma_k^2 = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} (x_n - \mu_k)^2 = \gamma_{nk} x_n^2 - 2x_n \mu_k + \mu_k^2$$

where $N_k = \sum_{n=1}^N \gamma_{nk}$ is the effective number of points assigned to component k .

1.9.15 Sufficient Statistics

In `dmx-learn`, the accumulator tracks the sufficient statistics, which are used to perform the estimation step. Following the above, we see that

$$\sum_{n=1}^N \gamma_{nk}$$

$$\sum_{n=1}^N \gamma_{nk} x_n$$

$$\sum_{n=1}^N \gamma_{nk} x_n^2$$

are required to estimate π_k, μ_k, σ_k^2 . Our accumulator class will aggregate these sufficient statistics. We will denote the sufficient statistics with member variables: $\text{comp_counts} = \sum_{n=1}^N \gamma_{nk}$, $\mathbf{x} = \sum_{n=1}^N \gamma_{nk} x_n$, and $\mathbf{x2} = \sum_{n=1}^N \gamma_{nk} x_n^2$. Note that each of these member variables are k dimensional vectors where k is the number of mixture components.

Below is a skeleton of the `SequenceEncodableStatisticAccumulator`. The methods `value`, `combine`, and `from_value` all must be implemented. They return the sufficient stats from the accumulator, combine the current suff stats of the object instance with another set of suff stats, and assign the Accumulator instance suff stats respectively.

Another interesting thing to note is the passing of the variable `keys`. For the Gaussian Mixture model implementation, we allow the user to pass keys specifying whether the mixture weights or means and variances should be shared across any other distributions (with matching keys). You must implement the two methods `key_merge` and `key_replace`.

The member function `acc_to_encoder` is similar to `dist_to_encoder` from the distribution class. It must be included here, as we currently require data encodings to be available in the initialization step.

```
class GmmAccumulator(SequenceEncodableStatisticAccumulator):

    def __init__(self, num_comps: int, keys: Optional[Tuple[Optional[str],
↳Optional[str]]] = None, name: Optional[str] = None):
        self.x = np.zeros(num_comps)
        self.x2 = np.zeros(num_comps)
        self.comp_counts = np.zeros(num_comps)
        self.ncomps = num_comps
        self.weight_keys = keys[0] if keys else None
        self.param_keys = keys[1] if keys else None

    def initialize(self, x: float, weight: float, rng: Optional[RandomState]):
        pass

    def update(self, x: float, weight: float, estimate: GmmDistribution):
        pass

    def seq_initialize(self, x: GmmEncodedDataSequence, weights: np.ndarray, rng:
↳Optional[RandomState]):
        pass

    def seq_update(self, x: GmmEncodedDataSequence, weights: np.ndarray, estimate:
↳GmmDistribution):
        pass

    # Return the sufficient statistics
    def value(self) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
        return self.comp_counts, self.x, self.x2

    # Combine suff stats with the accumulators
    def combine(self, x: Tuple[np.ndarray, np.ndarray, np.ndarray]):
        self.comp_counts += x[0]
        self.x += x[1]
        self.x2 += x[2]

        return self
```

(continues on next page)

(continued from previous page)

```

# assign sufficient statistics from a value
def from_value(self, x: Tuple[np.ndarray, np.ndarray, np.ndarray]):
    self.comp_counts = x[0]
    self.x = x[1]
    self.x2 = x[2]

# This allows for merging of suff stats with parameters that have the same keys
def key_merge(self, stats_dict: Dict[str, Any]):
    if self.weight_keys is not None:
        if self.weight_keys in stats_dict:
            self.comp_counts += stats_dict[self.weight_keys]
        else:
            stats_dict[self.weight_keys] = self.comp_counts

    if self.param_keys is not None:
        if self.param_keys in stats_dict:
            x, x2 = stats_dict[self.param_keys]
            self.x += x
            self.x2 += x2
        else:
            stats_dict[self.param_keys] = (self.x, self.x2)

# Set the sufficient statistics of the accumulator to suff stats with matching keys.
def key_replace(self, stats_dict: Dict[str, Any]):
    if self.weight_keys is not None:
        if self.weight_keys in stats_dict:
            self.comp_counts = stats_dict[self.weight_keys]

    if self.param_keys is not None:
        if self.param_keys in stats_dict:
            self.param_keys = stats_dict[self.param_keys]

# Create a DataSequenceEncoder object for seq initialize encodings.
def acc_to_encoder(self):
    return GmmDataEncoder()

```

1.9.16 Implementing Update

Recall: Expectation Step (E-step): Calculate the expected value of the log-likelihood function, given the current estimates of the parameters.

$$\gamma_{nk} = \frac{\pi_k \mathcal{N}(x_n | \mu_k, \sigma_k^2)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_n | \mu_j, \sigma_j^2)}$$

where γ_{nk} is the responsibility that component k takes for data point n .

For the update function, we must calculate the posterior γ_{nk} for each observation x_n . This is done using a log-sum-exp trick. Once we have γ_{nk} , we simply update the accumulators sufficient stats $\mathbf{x}$, $\mathbf{x2}$, and `comp_counts` accordingly. Note that `weight` is also multiplied to the γ_{nk} values, as this allows for nesting with other dmx-learn classes.

We must also implement the vectorized `seq_update`, which takes the `GmmEncodedDataSequence` previously defined.

```

class GmmAccumulator(SequenceEncodableStatisticAccumulator):

    def __init__(self, num_comps: int, keys: Optional[Tuple[Optional[str],
↳Optional[str]]] = None, name: Optional[str] = None):
        self.x = np.zeros(num_comps)
        self.x2 = np.zeros(num_comps)
        self.comp_counts = np.zeros(num_comps)
        self.ncomps = num_comps
        self.weight_keys = keys[0] if keys else None
        self.param_keys = keys[1] if keys else None

    def update(self, x: float, weight: float, estimate: GmmDistribution):
        mu, s2, w = estimate.mu, estimate.sigma2, estimate.w

        gamma = -0.5*(x-mu)**2 / s2 - 0.5*np.log(s2) + np.log(w)
        max_ = np.max(gamma)

        if not np.isinf(max_):
            # log-sum-exp back to exp
            gamma = np.exp(gamma-max_, out=gamma)
            gamma /= np.sum(gamma)
            # multiply by weight to allow for down stream nesting with other dmx classes
            gamma *= weight
            self.comp_counts += gamma
            self.x += x*gamma
            self.x2 += x**2*gamma

    def seq_update(self, x: GmmEncodedDataSequence, weights: np.ndarray, estimate:
↳GmmDistribution):
        mu, s2, log_w = estimate.mu, estimate.sigma2, np.log(estimate.w)
        gammas = -0.5*(x.data[:, None] - mu)**2 / s2 - 0.5*np.log(s2)
        gammas += log_w[None, :]

        # check for 0 weights
        zw = np.isinf(log_w)
        if np.any(zw):
            gammas[:, zw] = -np.inf

        max_ = np.max(gammas, axis=1, keepdims=True)

        # correct for any posterior containing all -np.inf values.
        bad_rows = np.all(np.isinf(gammas), axis=1).flatten()
        gammas[bad_rows, :] = log_w.copy()
        max_[bad_rows] = np.max(log_w)

        # logsumexp and multiply by weights passed
        gammas -= max_
        np.exp(gammas, out=gammas)
        np.sum(gammas, axis=1, keepdims=True, out=max_)
        np.divide(weights[:, None], max_, out=max_)
        gammas *= max_

        # update the sufficient stats

```

(continues on next page)

(continued from previous page)

```

        wsum = gammas.sum(axis=0)
        self.comp_counts += wsum
        self.x += np.dot(x.data, gammas)
        self.x2 += np.dot(x.data**2, gammas)

    def initialize(self, x: float, weight: float, rng: Optional[RandomState]):
        pass

    def seq_initialize(self, x: GmmEncodedDataSequence, weights: np.ndarray, rng:
↳ Optional[RandomState]):
        pass

    # Return the sufficient statistics
    def value(self) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
        return self.comp_counts, self.x, self.x2

    # Combine suff stats with the accumulators
    def combine(self, x: Tuple[np.ndarray, np.ndarray, np.ndarray]):
        self.comp_counts += x[0]
        self.x += x[1]
        self.x2 += x[2]

        return self

    # assign sufficient statistics from a value
    def from_value(self, x: Tuple[np.ndarray, np.ndarray, np.ndarray]):
        self.comp_counts = x[0]
        self.x = x[1]
        self.x2 = x[2]

    # This allows for merging of suff stats with parameters that have the same keys
    def key_merge(self, stats_dict: Dict[str, Any]):
        if self.weight_keys is not None:
            if self.weight_keys in stats_dict:
                self.comp_counts += stats_dict[self.weight_keys]
            else:
                stats_dict[self.weight_keys] = self.comp_counts

        if self.param_keys is not None:
            if self.param_keys in stats_dict:
                x, x2 = stats_dict[self.param_keys]
                self.x += x
                self.x2 += x2
            else:
                stats_dict[self.param_keys] = (self.x, self.x2)

    # Set the sufficient statistics of the accumulator to suff stats with matching keys.
    def key_replace(self, stats_dict: Dict[str, Any]):
        if self.weight_keys is not None:
            if self.weight_keys in stats_dict:
                self.comp_counts = stats_dict[self.weight_keys]

```

(continues on next page)

(continued from previous page)

```

    if self.param_keys is not None:
        if self.param_keys in stats_dict:
            self.param_keys = stats_dict[self.param_keys]

    # Create a DataSequenceEncoder object for seq initialize encodings.
    def acc_to_encoder(self):
        return GmmDataEncoder()

```

1.9.17 Initialize

The sufficient statistics must be initialized. The *SequenceEncodableStatisticAccumulator* object allows for randomized initialization of the sufficient statistics for the observed data. The methods required here are *initialize* and the vectorized version *seq_initialize*. It is up to you to define how these methods are implemented. Below we outline the method for the case of the univariate Gaussian mixture model.

1.9.18 Initialization of GMM sufficient statistics

1. Draw $\gamma_i \sim \text{Dirichlet}((\frac{1}{k}, \frac{1}{k}, \dots, \frac{1}{k}))$.
2. Multiply by passed *weight* (this is for nesting with other dmx-learn distributions):

$$\gamma_i = \gamma_i * \text{weight}_i$$

3. Update the sufficient statistic member variables of the accumulator:

$$\text{comp_counts}[k] += \gamma_{i,k}$$

$$x[k] += \gamma_{i,k} * x_i$$

$$x2[k] += \gamma_{i,k} * x_i^2$$

This is quite trivial to vectorize and implement in *seq_initialize* using our encoded data previously defined as *GmmEncodedDataSequence*. The code is filled out below. One small comment, the value *c* is used in the initialization to avoid numeric issues encountered when sampling from a Dirichlet distribution.

```

class GmmAccumulator(SequenceEncodableStatisticAccumulator):

    def __init__(self, num_comps: int, keys: Optional[Tuple[Optional[str], ...Optional[str]]] = None, name: Optional[str] = None):

        self.x = np.zeros(num_comps)
        self.x2 = np.zeros(num_comps)
        self.comp_counts = np.zeros(num_comps)
        self.ncomps = num_comps
        self.weight_keys = keys[0] if keys else None
        self.param_keys = keys[1] if keys else None

    def update(self, x: float, weight: float, estimate: GmmDistribution):
        mu, s2, w = estimate.mu, estimate.sigma2, estimate.w

        gamma = -0.5*(x-mu)**2 / s2 - 0.5*np.log(s2) + np.log(w)
        max_ = np.max(gamma)

```

(continues on next page)

(continued from previous page)

```

    if not np.isinf(max_):
        # log-sum-exp back to exp
        gamma = np.exp(gamma-max_, out=gamma)
        gamma /= np.sum(gamma)
        # multiply by weight to allow for down stream nesting with other dmx classes
        gamma *= weight
        self.comp_counts += gamma
        self.x += x*gamma
        self.x2 += x**2*gamma

    def seq_update(self, x: GmmEncodedDataSequence, weights: np.ndarray, estimate: GmmDistribution):

        mu, s2, log_w = estimate.mu, estimate.sigma2, np.log(estimate.w)
        gammas = -0.5*(x.data[:, None] - mu)**2 / s2 - 0.5*np.log(s2)
        gammas += log_w[None, :]

        # check for 0 weights
        zw = np.isinf(log_w)
        if np.any(zw):
            gammas[:, zw] = -np.inf

        max_ = np.max(gammas, axis=1, keepdims=True)

        # correct for any posterior containing all -np.inf values.
        bad_rows = np.isinf(max_.flatten())
        gammas[bad_rows, :] = log_w.copy()
        max_[bad_rows] = np.max(log_w)

        # logsumexp and multiply by weights passed
        gammas -= max_
        np.exp(gammas, out=gammas)
        np.sum(gammas, axis=1, keepdims=True, out=max_)
        np.divide(weights[:, None], max_, out=max_)
        gammas *= max_

        # update the sufficient stats
        wsum = gammas.sum(axis=0)
        self.comp_counts += wsum
        self.x += np.dot(x.data, gammas)
        self.x2 += np.dot(x.data**2, gammas)

    def initialize(self, x: float, weight: float, rng: RandomState):

        # generate random posterior values
        c = 20 ** 2 if self.ncomps > 20 else self.ncomps**2
        ww = rng.dirichlet(np.ones(self.ncomps) / c)
        ww *= weight

        # update suff stats
        self.x += x * ww

```

(continues on next page)

(continued from previous page)

```

self.x2 += x**2 * ww
self.comp_counts += ww

def seq_initialize(self, x: GmmEncodedDataSequence, weights: np.ndarray, rng:
↳ Optional[RandomState]):

    # only generate random posteriors for weights that are non-zero
    sz = len(weights)
    c = 20 ** 2 if self.ncomps > 20 else self.ncomps ** 2

    ww = rng.dirichlet(np.ones(self.ncomps) / c, size=sz)
    ww *= weights[:, None]
    w_sum = ww.sum(axis=0)

    # initialize suff stats
    self.comp_counts += w_sum
    self.x += np.dot(x.data, ww)
    self.x2 += np.dot(x.data ** 2, ww)

# Return the sufficient statistics
def value(self) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    return self.comp_counts, self.x, self.x2

# Combine suff stats with the accumulators
def combine(self, x: Tuple[np.ndarray, np.ndarray, np.ndarray]):
    self.comp_counts += x[0]
    self.x += x[1]
    self.x2 += x[2]

    return self

# assign sufficient statistics from a value
def from_value(self, x: Tuple[np.ndarray, np.ndarray, np.ndarray]):
    self.comp_counts = x[0]
    self.x = x[1]
    self.x2 = x[2]

# This allows for merging of suff stats with parameters that have the same keys
def key_merge(self, stats_dict: Dict[str, Any]):

    if self.weight_keys is not None:
        if self.weight_keys in stats_dict:
            self.comp_counts += stats_dict[self.weight_keys]
        else:
            stats_dict[self.weight_keys] = self.comp_counts

    if self.param_keys is not None:
        if self.param_keys in stats_dict:
            x, x2 = stats_dict[self.param_keys]
            self.x += x

```

(continues on next page)

(continued from previous page)

```

        self.x2 += x2
    else:
        stats_dict[self.param_keys] = (self.x, self.x2)

# Set the sufficient statistics of the accumulator to suff stats with matching keys.
def key_replace(self, stats_dict: Dict[str, Any]):
    if self.weight_keys is not None:
        if self.weight_keys in stats_dict:
            self.comp_counts = stats_dict[self.weight_keys]

    if self.param_keys is not None:
        if self.param_keys in stats_dict:
            self.param_keys = stats_dict[self.param_keys]

# Create a DataSequenceEncoder object for seq initialize encodings.
def acc_to_encoder(self):
    return GmmDataEncoder()

```

1.9.19 StatisticAccumulatorFactory

In programming, a factory object is a design pattern used to create instances of objects. Instead of calling a constructor directly to create an object, a factory provides a method that returns an instance of a class. We define the *StatisticAccumulatorFactory* as a method for creating *SequenceEncodableStatisticAccumulator* objects.

```

class GmmAccumulatorFactory(StatisticAccumulatorFactory):

    # same constructor as the GmmAccumulator object
    def __init__(self, num_comps: int, keys: Optional[Tuple[Optional[str],
↳Optional[str]]] = None, name: Optional[str] = None):
        self.num_comps = num_comps
        self.keys = keys
        self.name = name

    # creates a GmmAccumulator object
    def make (self) -> 'GmmAccumulator':
        return GmmAccumulator(num_comps=self.num_comps, keys=self.keys, name=self.name)

```

The only thing left to do is to implement the *ParameterEstimator* class and fill out the *estimator* function call in the *SequenceEncodableProbabilityDistribution*. So let's implement the *ParameterEstimator* and tie everything together.

```

class GmmEstimator(ParameterEstimator):

    def __init__(self, num_comps: int, suff_stat: Tuple[Optional[np.ndarray], Optional[np.
↳ndarray], Optional[np.ndarray]] = (None, None, None), pseudo_count:
↳Tuple[Optional[float], Optional[float], Optional[float]] = (None, None, None), keys:
↳Tuple[Optional[str], Optional[str]] = (None, None)):
        self.ncomps = num_comps
        self.suff_stat = suff_stat
        self.pseudo_count = pseudo_count
        self.keys = keys

    def accumulator_factory(self) -> GmmAccumulatorFactory:

```

(continues on next page)

(continued from previous page)

```

return GmmAccumulatorFactory(num_comps=self.ncomps, keys=self.keys)

def estimate(self, nobss: Optional[float], suff_stat: Tuple[np.ndarray, np.ndarray, np.
↳ ndarray]) -> GmmDistribution:
    counts, xw, x2w = suff_stat

    # regularize weights without suff stat passed to it
    if self.pseudo_count[0] and not self.suff_stat[0]:
        p = self.pseudo_count[0] / self.ncomps
        w = counts + p
        w /= w.sum()
    # regularize weights with suff stat passed
    elif self.pseudo_count[0] and self.suff_stat[0]:
        w = (counts + self.suff_stat[0]*self.pseudo_count[0]) / (counts.sum() + self.
↳ pseudo_count[0]*self.suff_stat[0].sum())
    # dont regularize weights
    else:
        wsum = counts.sum()

        if wsum == 0.0:
            w = np.ones(self.ncomps) / float(self.ncomps)
        else:
            w = counts.copy() / wsum

    # flatten the mean estimates
    if self.pseudo_count[1] is not None and self.suff_stat[1] is not None:
        mu = (xw + self.pseudo_count[1] * self.suff_stat[1]) / (counts + self.pseudo_
↳ count * np.sum(self.suff_stats[1]))
    else:
        wsum = counts.copy()
        wsum[wsum==0.0] = 1.0
        mu = xw / wsum

    # flatten/regularize the variance estimates
    if self.pseudo_count[2] and self.suff_stat[2]:
        s2 = (x2w - mu**2 * counts * self.pseudo_count[2] * self.suff_stat[2]) /
↳ (counts + self.pseudo_count[2] * np.sum(self.suff_stat[2]))
    else:
        wsum = counts.copy()
        wsum[wsum==0.0] = 1.0
        s2 = x2w / wsum - mu * mu

    return GmmDistribution(mu=mu, sigma2=s2, w=w)

```

1.9.20 Completed SequenceEncodableProbabilityDistribution

We can now fill out *estimate* in the *SequenceEncodableProbabilityDistribution*. This completes the GMM class for use in dmx-learn!

```

class GmmDistribution(SequenceEncodableProbabilityDistribution):

    def __init__(self, mu: Union[Sequence[float], np.ndarray], sigma2:

```

(continues on next page)

(continued from previous page)

```

↪ Union[Sequence[float], np.ndarray], w: Union[Sequence[float], np.ndarray], name:
↪ Optional[str] = None):
    self.mu = np.asarray(mu)
    self.sigma2 = np.asarray(sigma2)
    self.w = np.asarray(w)
    self.name = name

    self.log_const = -0.5*np.log(2.0 * np.pi)

    def __str__(self) -> str:
        return 'GmmDistribution(mu=%s, sigma2=%s, w=%s, name=%s)' % (repr(self.mu.
↪ tolist()), repr(self.sigma2.tolist()), repr(self.w.tolist()), repr(self.name))

    def log_density(self, x: float) -> float:
        # eval log-density for each component
        ll = self.log_const - 0.5*(x-self.mu) ** 2 / self.sigma2 - 0.5*np.log(self.
↪ sigma2) + np.log(self.w)
        max_ = np.max(ll)
        # subtract max and exponentiate
        np.exp(ll-max_, out=ll)
        # finish log-sum-exp
        rv = np.log(np.sum(ll)) + max_
        return rv

    def density(self, x: float) -> float:
        return np.exp(self.log_density(x))

    def seq_log_density(self, x: GmmEncodedDataSequence) -> np.ndarray:
        # Type check
        if not isinstance(x, GmmEncodedDataSequence):
            raise Exception('GmmEncodedDataSequence requires for seq_log_density.')

        # Evaluate the vetorized log-density as before
        ll = -0.5*(x.data[:, None] - self.mu)**2 / self.sigma2 + self.log_const + np.
↪ log(self.w) - 0.5*np.log(self.sigma2)
        max_ = np.max(ll, axis=1, keepdims=True)
        np.exp(ll-max_, out=ll)
        ll = np.log(np.sum(ll, axis=1, keepdims=False))
        ll += max_.flatten()

        return ll

    def dist_to_encoder(self) -> GmmDataEncoder:
        return GmmDataEncoder()

    def sampler(self, seed: Optional[int] = None) -> GmmSampler:
        return GmmSampler(dist=self, seed=seed)

    def estimator(self, pseudo_count: Optional[float] = None):
        pc = (pseudo_count, pseudo_count, pseudo_count)
        return GmmEstimator(num_comps=len(self.w), pseudo_count=pc)

```

1.9.21 Proof of Concept

Let's walk through the standard dmx-learn pipeline. First we declare the model and simulate some data. We then declare the estimator and fit the model using *optimize*.

```
N = 1000
k = 3
w = np.ones(k) / float(k)
mu = np.linspace(-5, 5, k)
sigma2 = np.ones(k) / 1.

dist = GmmDistribution(mu=mu, sigma2=sigma2, w=w)

sampler = dist.sampler(seed=1)
data = sampler.sample(N)

est = GmmEstimator(num_comps=k)
fit = optimize(data=data, estimator=est, max_its=10000, print_iter=100,
               rng=RandomState(1))
```

This wraps things up. Keep in mind you are free to add other member functions to the `SequenceEncodableProbabilityDistribution` class that improve your quality of life. One thing you can try out is implementing a posterior and vectorized version `seq_posterior` that computes the posterior probability of component membership.

1.10 MPI4PY Estimation Example

This guide explains how to run *estimation_example.py* to fit statistical models in parallel using *mpi4py* and the dmx-learn package. Each step below includes the relevant Python code snippet. The file is found in *examples/mpi4py_examples*.

1.10.1 Running the Script

To launch the script with 4 MPI processes:

```
mpiexec -n 4 python3 examples/mpi4py_examples/estimation_example.py
```

1.10.2 Step 1: Import Libraries and Set Up MPI

```
import os
os.environ['NUMBA_DISABLE_JIT'] = '1'

from mpi4py import MPI
from numpy.random import RandomState
import pickle
from dmx.stats import *
from dmx.mpi4py.stats import *
from dmx.mpi4py.utils.estimation import optimize_mpi
from dmx.mpi4py.utils.optsutil import pickle_on_master

comm = MPI.COMM_WORLD
world_rank = comm.Get_rank()
world_size = comm.Get_size()
```

1.10.3 Step 2: Simulate Data on the Master Node

Note that the data simulation is only performed on the master node (rank 0). Other nodes will receive *None* for data. We use dmx-learn's `DistributionSampler` object to sample from the two state composite mixture distribution.

```
if world_rank == 0:
    d00 = GaussianDistribution(mu=0.0, sigma2=1.0)
    d01 = CategoricalDistribution({'a': 0.3, 'b': 0.7})
    d0 = CompositeDistribution([d00, d01])

    d10 = GaussianDistribution(mu=3.0, sigma2=1.0)
    d11 = CategoricalDistribution({'a': 0.7, 'b': 0.3})
    d1 = CompositeDistribution([d10, d11])

    dist = MixtureDistribution([d0, d1], w=[0.25, 0.75])

    data = dist.sampler(seed=1).sample(1000)
else:
    data = None
```

1.10.4 Step 3: Define the Estimator

The estimator can be defined on all the nodes. This object is lightweight and is later broadcasted to all nodes. Here we define a composite estimator that combines Gaussian and Categorical estimators, and then wrap it in a mixture estimator.

```
e0 = CompositeEstimator([GaussianEstimator(), CategoricalEstimator()])
est = MixtureEstimator([e0]*2)
```

1.10.5 Step 4: Fit the Model in Parallel

The data is passed to the `optimize_mpi` function along with the estimator and a random number generator. This function will handle the dissemination of data and parallel fitting of the model across all MPI processes.

```
rng = RandomState(1)
fit = optimize_mpi(data=data, estimator=est, rng=rng)
```

1.10.6 Step 5: Check Model Presence on Each Node

The snippets below are included to demonstrate to the user that only the master node will have the fitted model. Each node prints whether it has the fitted model or not.

```
print(f"Rank {world_rank}: Model is None == {fit is None}")
```

1.10.7 Step 6: Save the Model on the Master Node

The fitted model is pickled and saved to a file only on the master node (rank 0) using `pickle_on_master`. This ensures that the model is not duplicated across all nodes, which would be inefficient.

```
pickle_on_master(fit, "mpi4py_model_fit.pkl")

if world_rank == 0:
    print(f"Wrote file ./mpi4py_model_fit.pkl")
```

1.10.8 Full Script Example

Here is the complete script for reference:

```
import os
os.environ['NUMBA_DISABLE_JIT'] = '1'

from mpi4py import MPI
from numpy.random import RandomState
import pickle
from dmx.stats import *
from dmx.mpi4py.stats import *
from dmx.mpi4py.utils.estimation import optimize_mpi
from dmx.mpi4py.utils.optsutil import pickle_on_master

comm = MPI.COMM_WORLD
world_rank = comm.Get_rank()
world_size = comm.Get_size()

if __name__ == "__main__":
    if world_rank == 0:
        d00 = GaussianDistribution(mu=0.0, sigma2=1.0)
        d01 = CategoricalDistribution({'a': 0.3, 'b': 0.7})
        d0 = CompositeDistribution([d00, d01])

        d10 = GaussianDistribution(mu=3.0, sigma2=1.0)
        d11 = CategoricalDistribution({'a': 0.7, 'b': 0.3})
        d1 = CompositeDistribution([d10, d11])

        dist = MixtureDistribution([d0, d1], w=[0.25, 0.75])

        data = dist.sampler(seed=1).sample(1000)
    else:
        data = None

    e0 = CompositeEstimator([GaussianEstimator(), CategoricalEstimator()])
    est = MixtureEstimator([e0]*2)

    rng = RandomState(1)
    fit = optimize_mpi(data=data, estimator=est, rng=rng)

    print(f"Rank {world_rank}: Model is None == {fit is None}")

    pickle_on_master(fit, "mpi4py_model_fit.pkl")

    if world_rank == 0:
        print(f"Wrote file ./mpi4py_model_fit.pkl")
```

1.10.9 Notes

- Only the master node (rank 0) will have the fitted model and write the output file.
- You can modify the script to read your own data instead of simulating it.

1.10.10 References

- *mpi4py* documentation: <https://mpi4py.readthedocs.io/>

PYTHON MODULE INDEX

d

`dmx.bstats`, 372
`dmx.bstats.bernoulli`, 407
`dmx.bstats.bestimation`, 383
`dmx.bstats.beta`, 389
`dmx.bstats.catdirichlet`, 391
`dmx.bstats.categorical`, 413
`dmx.bstats.composite`, 467
`dmx.bstats.conditional`, 474
`dmx.bstats.dirac`, 419
`dmx.bstats.dirichlet`, 394
`dmx.bstats.dmvn`, 425
`dmx.bstats.dpm`, 480
`dmx.bstats.exponential`, 431
`dmx.bstats.gamma`, 436
`dmx.bstats.gaussian`, 442
`dmx.bstats.geometric`, 448
`dmx.bstats.ignored`, 487
`dmx.bstats.intrange`, 454
`dmx.bstats.mixture`, 492
`dmx.bstats.mvngamma`, 401
`dmx.bstats.normgamma`, 403
`dmx.bstats.nulldist`, 499
`dmx.bstats.optional`, 505
`dmx.bstats.pdist`, 372
`dmx.bstats.poisson`, 461
`dmx.bstats.sequence`, 512
`dmx.bstats.setdist`, 519
`dmx.bstats.symdirichlet`, 405
`dmx.mpi4py.bstats`, 706
`dmx.mpi4py.stats`, 703
`dmx.mpi4py.utils`, 708
`dmx.mpi4py.utils.automatic`, 709
`dmx.mpi4py.utils.bestimation`, 710
`dmx.mpi4py.utils.estimation`, 711
`dmx.mpi4py.utils.humap`, 714
`dmx.mpi4py.utils.optsutil`, 715
`dmx.stats`, 7
`dmx.stats.dirac_length`, 186
`dmx.stats.dmvn_mixture`, 242
`dmx.stats.gmm`, 254
`dmx.stats.hidden_association`, 294
`dmx.stats.icltree`, 301
`dmx.stats.int_edit_setdist`, 196
`dmx.stats.int_edit_stepsetdist`, 205
`dmx.stats.int_hidden_association`, 307
`dmx.stats.int_hidden_markov`, 318
`dmx.stats.int_markovchain`, 334
`dmx.stats.int_plsi`, 266
`dmx.stats.lda`, 279
`dmx.stats.look_back_hmm`, 345
`dmx.stats.null_dist`, 210
`dmx.stats.rdd_sampler`, 370
`dmx.stats.select`, 216
`dmx.stats.sparse_markov_transform`, 353
`dmx.stats.spearman_rho`, 221
`dmx.stats.ss_mixture`, 288
`dmx.stats.tree_hmm`, 360
`dmx.stats.vmf`, 228
`dmx.stats.weighted`, 236
`dmx.torch_stats`, 525
`dmx.torch_stats.binomial`, 532
`dmx.torch_stats.composite`, 603
`dmx.torch_stats.conditional`, 608
`dmx.torch_stats.dmvn`, 539
`dmx.torch_stats.exponential`, 545
`dmx.torch_stats.gamma`, 551
`dmx.torch_stats.gaussian`, 558
`dmx.torch_stats.geometric`, 564
`dmx.torch_stats.heterogenous_mixture`, 622
`dmx.torch_stats.hmm`, 627
`dmx.torch_stats.int_plsi`, 637
`dmx.torch_stats.intmultinomial`, 569
`dmx.torch_stats.intrange`, 578
`dmx.torch_stats.intsetdist`, 585
`dmx.torch_stats.jmixture`, 642
`dmx.torch_stats.mixture`, 647
`dmx.torch_stats.mvn`, 591
`dmx.torch_stats.null_dist`, 613
`dmx.torch_stats.pdist`, 525
`dmx.torch_stats.poisson`, 597
`dmx.torch_stats.sequence`, 617
`dmx.torch_utils`, 692
`dmx.torch_utils.estimation`, 693

- `dmx.torch_utils.optsutil`, 696
- `dmx.torch_utils.vector`, 697
- `dmx.utils`, 652
 - `dmx.utils.automatic`, 652
 - `dmx.utils.builder`, 659
 - `dmx.utils.estimation`, 660
 - `dmx.utils.htsne`, 667
 - `dmx.utils.humap`, 674
 - `dmx.utils.metrics`, 675
 - `dmx.utils.optsutil`, 677
 - `dmx.utils.pvalues`, 680
 - `dmx.utils.special`, 681
 - `dmx.utils.vector`, 684

INDEX

Symbols

<code>__init__()</code> (<i>dmx.stats.binomial.BinomialDistribution</i> method), 20	<code>__init__()</code> (<i>dmx.stats.geometric.GeometricEstimator</i> method), 26
<code>__init__()</code> (<i>dmx.stats.binomial.BinomialEstimator</i> method), 23	<code>__init__()</code> (<i>dmx.stats.geometric.GeometricEstimator</i> method), 28
<code>__init__()</code> (<i>dmx.stats.categorical.CategoricalDistribution</i> method), 83	<code>__init__()</code> (<i>dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution</i> method), 155
<code>__init__()</code> (<i>dmx.stats.categorical.CategoricalEstimator</i> method), 85	<code>__init__()</code> (<i>dmx.stats.heterogeneous_mixture.HeterogeneousMixtureEstimator</i> method), 159
<code>__init__()</code> (<i>dmx.stats.catmultinomial.MultinomialDistribution</i> method), 104	<code>__init__()</code> (<i>dmx.stats.hidden_markov.HiddenMarkovEstimator</i> method), 182
<code>__init__()</code> (<i>dmx.stats.catmultinomial.MultinomialEstimator</i> method), 107	<code>__init__()</code> (<i>dmx.stats.hidden_markov.HiddenMarkovModelDistribution</i> method), 178
<code>__init__()</code> (<i>dmx.stats.composite.CompositeDistribution</i> method), 118	<code>__init__()</code> (<i>dmx.stats.hmixture.HierarchicalMixtureDistribution</i> method), 169
<code>__init__()</code> (<i>dmx.stats.composite.CompositeEstimator</i> method), 121	<code>__init__()</code> (<i>dmx.stats.hmixture.HierarchicalMixtureEstimator</i> method), 174
<code>__init__()</code> (<i>dmx.stats.conditional.ConditionalDistribution</i> method), 130	<code>__init__()</code> (<i>dmx.stats.ignored.IgnoredDistribution</i> method), 142
<code>__init__()</code> (<i>dmx.stats.conditional.ConditionalDistributionEstimator</i> method), 133	<code>__init__()</code> (<i>dmx.stats.ignored.IgnoredEstimator</i> method), 144
<code>__init__()</code> (<i>dmx.stats.dirichlet.DirichletDistribution</i> method), 77	<code>__init__()</code> (<i>dmx.stats.int_spike.SpikeAndSlabDistribution</i> method), 41
<code>__init__()</code> (<i>dmx.stats.dirichlet.DirichletEstimator</i> method), 80	<code>__init__()</code> (<i>dmx.stats.int_spike.SpikeAndSlabEstimator</i> method), 43
<code>__init__()</code> (<i>dmx.stats.dmvn.DiagonalGaussianDistribution</i> method), 66	<code>__init__()</code> (<i>dmx.stats.intmultinomial.IntegerMultinomialDistribution</i> method), 98
<code>__init__()</code> (<i>dmx.stats.dmvn.DiagonalGaussianEstimator</i> method), 69	<code>__init__()</code> (<i>dmx.stats.intmultinomial.IntegerMultinomialEstimator</i> method), 101
<code>__init__()</code> (<i>dmx.stats.exponential.ExponentialDistribution</i> method), 50	<code>__init__()</code> (<i>dmx.stats.intrange.IntegerCategoricalDistribution</i> method), 35
<code>__init__()</code> (<i>dmx.stats.exponential.ExponentialEstimator</i> method), 53	<code>__init__()</code> (<i>dmx.stats.intrange.IntegerCategoricalEstimator</i> method), 38
<code>__init__()</code> (<i>dmx.stats.gamma.GammaDistribution</i> method), 55	<code>__init__()</code> (<i>dmx.stats.intsetdist.IntegerBernoulliSetDistribution</i> method), 88
<code>__init__()</code> (<i>dmx.stats.gamma.GammaEstimator</i> method), 58	<code>__init__()</code> (<i>dmx.stats.intsetdist.IntegerBernoulliSetEstimator</i> method), 90
<code>__init__()</code> (<i>dmx.stats.gaussian.GaussianDistribution</i> method), 45	<code>__init__()</code> (<i>dmx.stats.jmixture.JointMixtureDistribution</i> method), 163
<code>__init__()</code> (<i>dmx.stats.gaussian.GaussianEstimator</i> method), 48	<code>__init__()</code> (<i>dmx.stats.jmixture.JointMixtureEstimator</i> method), 166
<code>__init__()</code> (<i>dmx.stats.geometric.GeometricDistribution</i> method), 26	<code>__init__()</code> (<i>dmx.stats.log_gaussian.LogGaussianDistribution</i> method), 61
	<code>__init__()</code> (<i>dmx.stats.log_gaussian.LogGaussianEstimator</i> method), 61

[illegible]

`acc_to_encoder()` (`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator` method), 243
`acc_to_encoder()` (`dmx.stats.gmm.GaussianMixtureAccumulator` method), 256
`acc_to_encoder()` (`dmx.stats.hidden_association.HiddenAssociationAccumulator` method), 295
`acc_to_encoder()` (`dmx.stats.icltree.ICLTreeAccumulator` method), 301
`acc_to_encoder()` (`dmx.stats.int_edit_setdist.IntegerBernoulliEditDistanceAccumulator` method), 197
`acc_to_encoder()` (`dmx.stats.int_edit_stepsetdist.IntegerStepSetDistanceAccumulator` method), 205
`acc_to_encoder()` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator` method), 307
`acc_to_encoder()` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator` method), 320
`acc_to_encoder()` (`dmx.stats.int_markovchain.IntegerMarkovChainAccumulator` method), 336
`acc_to_encoder()` (`dmx.stats.int_plsi.IntegerPLSIAccumulator` method), 268
`acc_to_encoder()` (`dmx.stats.lda.LDAEstimatorAccumulator` method), 284
`acc_to_encoder()` (`dmx.stats.look_back_hmm.LookbackHiddenMarkovAccumulator` method), 349
`acc_to_encoder()` (`dmx.stats.null_dist.NullAccumulator` method), 210
`acc_to_encoder()` (`dmx.stats.pdist.SequenceEncodableStatisticAccumulator` method), 16
`acc_to_encoder()` (`dmx.stats.select.SelectEstimatorAccumulator` method), 219
`acc_to_encoder()` (`dmx.stats.sparse_markov_transform.SparseMarkovTransformAccumulator` method), 353
`acc_to_encoder()` (`dmx.stats.spearman_rho.SpearmanRankingAccumulator` method), 222
`acc_to_encoder()` (`dmx.stats.ss_mixture.SemiSupervisedDiracMixtureAccumulator` method), 292
`acc_to_encoder()` (`dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator` method), 360
`acc_to_encoder()` (`dmx.stats.vmf.VonMisesFisherAccumulator` method), 228
`acc_to_encoder()` (`dmx.stats.weighted.WeightedAccumulator` method), 236
`acc_to_encoder()` (`dmx.torch_stats.binomial.BinomialAccumulator` method), 532
`acc_to_encoder()` (`dmx.torch_stats.composite.CompositeAccumulator` method), 603
`acc_to_encoder()` (`dmx.torch_stats.conditional.ConditionalDistributionAccumulator` method), 609
`acc_to_encoder()` (`dmx.torch_stats.dmvn.DiagonalGaussianAccumulator` method), 540
`acc_to_encoder()` (`dmx.torch_stats.exponential.ExponentialAccumulator` method), 546
`acc_to_encoder()` (`dmx.torch_stats.gamma.GammaAccumulator` method), 552
`acc_to_encoder()` (`dmx.torch_stats.gaussian.GaussianAccumulator` method), 559
`acc_to_encoder()` (`dmx.torch_stats.geometric.GeometricAccumulator` method), 564
`acc_to_encoder()` (`dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulator` method), 622
`acc_to_encoder()` (`dmx.torch_stats.hmm.HiddenMarkovAccumulator` method), 628
`acc_to_encoder()` (`dmx.torch_stats.int_plsi.IntegerPLSIAccumulator` method), 637
`acc_to_encoder()` (`dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator` method), 570
`acc_to_encoder()` (`dmx.torch_stats.intrange.IntegerCategoricalAccumulator` method), 579
`acc_to_encoder()` (`dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulator` method), 586
`acc_to_encoder()` (`dmx.torch_stats.jmixture.JointMixtureEstimatorAccumulator` method), 645
`acc_to_encoder()` (`dmx.torch_stats.mixture.MixtureAccumulator` method), 647
`acc_to_encoder()` (`dmx.torch_stats.mvn.MultivariateGaussianAccumulator` method), 592
`acc_to_encoder()` (`dmx.torch_stats.null_dist.NullAccumulator` method), 613
`acc_to_encoder()` (`dmx.torch_stats.pdist.TorchStatisticAccumulator` method), 529
`acc_to_encoder()` (`dmx.torch_stats.poisson.PoissonAccumulator` method), 598
`acc_to_encoder()` (`dmx.torch_stats.sequence.SequenceAccumulator` method), 618
`acc_to_encoder()` (`dmx.torch_stats.dirac.DiracMixtureAccumulator` attribute), 186
`acc_to_encoder()` (`dmx.stats.weighted.WeightedAccumulator` attribute), 236
`accumulator_factory()` (`dmx.bstats.bernoulli.BernoulliEstimator` method), 410
`accumulator_factory()` (`dmx.bstats.categorical.CategoricalEstimator` method), 416
`accumulator_factory()` (`dmx.bstats.composite.CompositeEstimator` method), 471
`accumulator_factory()` (`dmx.bstats.conditional.ConditionalDistributionEstimator` method), 477
`accumulator_factory()` (`dmx.bstats.dirac.DiracEstimator` method), 483
`accumulator_factory()` (`dmx.bstats.dirichlet.DirichletEstimator` method), 399
`accumulator_factory()` (`dmx.bstats.dmvn.DiagonalGaussianEstimator` method), 540

- method*), 430
- `accumulator_factory()`
(*dmx.bstats.dpm.DirichletProcessMixtureEstimator method*), 485
- `accumulator_factory()`
(*dmx.bstats.exponential.ExponentialEstimator method*), 435
- `accumulator_factory()`
(*dmx.bstats.gamma.GammaEstimator method*), 441
- `accumulator_factory()`
(*dmx.bstats.gaussian.GaussianEstimator method*), 447
- `accumulator_factory()`
(*dmx.bstats.geometric.GeometricEstimator method*), 453
- `accumulator_factory()`
(*dmx.bstats.ignored.IgnoredEstimator method*), 491
- `accumulator_factory()`
(*dmx.bstats.inrange.IntegerCategoricalEstimator method*), 460
- `accumulator_factory()`
(*dmx.bstats.mixture.MixtureEstimator method*), 496
- `accumulator_factory()`
(*dmx.bstats.nulldist.NullEstimator method*), 504
- `accumulator_factory()`
(*dmx.bstats.optional.OptionalEstimator method*), 508
- `accumulator_factory()`
(*dmx.bstats.pdist.ParameterEstimator method*), 374
- `accumulator_factory()`
(*dmx.bstats.poisson.PoissonEstimator method*), 464
- `accumulator_factory()`
(*dmx.bstats.sequence.SequenceEstimator method*), 515
- `accumulator_factory()`
(*dmx.bstats.setdist.BernoulliSetEstimator method*), 523
- `accumulator_factory()`
(*dmx.stats.binomial.BinomialEstimator method*), 24
- `accumulator_factory()`
(*dmx.stats.categorical.CategoricalEstimator method*), 86
- `accumulator_factory()`
(*dmx.stats.catmultinomial.MultinomialEstimator method*), 108
- `accumulator_factory()`
(*dmx.stats.composite.CompositeEstimator method*), 121
- `accumulator_factory()`
(*dmx.stats.conditional.ConditionalDistributionEstimator method*), 134
- `accumulator_factory()`
(*dmx.stats.dirac_length.DiracMixtureEstimator method*), 195
- `accumulator_factory()`
(*dmx.stats.dirichlet.DirichletEstimator method*), 80
- `accumulator_factory()`
(*dmx.stats.dmvn.DiagonalGaussianEstimator method*), 69
- `accumulator_factory()`
(*dmx.stats.dmvn_mixture.DiagonalGaussianMixtureEstimator method*), 252
- `accumulator_factory()`
(*dmx.stats.exponential.ExponentialEstimator method*), 53
- `accumulator_factory()`
(*dmx.stats.gamma.GammaEstimator method*), 58
- `accumulator_factory()`
(*dmx.stats.gaussian.GaussianEstimator method*), 48
- `accumulator_factory()`
(*dmx.stats.geometric.GeometricEstimator method*), 28
- `accumulator_factory()`
(*dmx.stats.gmm.GaussianMixtureEstimator method*), 264
- `accumulator_factory()`
(*dmx.stats.heterogeneous_mixture.HeterogeneousMixtureEstimator method*), 160
- `accumulator_factory()`
(*dmx.stats.hidden_association.HiddenAssociationEstimator method*), 300
- `accumulator_factory()`
(*dmx.stats.hidden_markov.HiddenMarkovEstimator method*), 183
- `accumulator_factory()`
(*dmx.stats.hmixture.HierarchicalMixtureEstimator method*), 174
- `accumulator_factory()`
(*dmx.stats.icltree.ICLTreeEstimator method*), 306
- `accumulator_factory()`
(*dmx.stats.ignored.IgnoredEstimator method*), 145
- `accumulator_factory()`
(*dmx.stats.int_edit_setdist.IntegerBernoulliEditEstimator method*), 203
- `accumulator_factory()`
(*dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditEstimator method*), 203

`method)`, 209
`accumulator_factory()`
`(dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator`
`method)`, 315
`accumulator_factory()`
`(dmx.stats.int_hidden_markov.IntegerHiddenMarkovEstimator`
`method)`, 325
`accumulator_factory()`
`(dmx.stats.int_markovchain.IntegerMarkovChainEstimator`
`method)`, 127
`method)`, 344
`accumulator_factory()`
`(dmx.stats.int_plsi.IntegerPLSEstimator`
`method)`, 275
`accumulator_factory()`
`(dmx.stats.int_spike.SpikeAndSlabEstimator`
`method)`, 43
`accumulator_factory()`
`(dmx.stats.intmultinomial.IntegerMultinomialEstimator`
`method)`, 102
`accumulator_factory()`
`(dmx.stats.intrange.IntegerCategoricalEstimator`
`method)`, 38
`accumulator_factory()`
`(dmx.stats.intsetdist.IntegerBernoulliSetEstimator`
`method)`, 91
`accumulator_factory()`
`(dmx.stats.jmixture.JointMixtureEstimator`
`method)`, 166
`accumulator_factory()` `(dmx.stats.lda.LDAEstimator`
`method)`, 283
`accumulator_factory()`
`(dmx.stats.log_gaussian.LogGaussianEstimator`
`method)`, 63
`accumulator_factory()`
`(dmx.stats.look_back_hmm.LookbackHiddenMarkovEstimator`
`method)`, 348
`accumulator_factory()`
`(dmx.stats.markovchain.MarkovChainEstimator`
`method)`, 115
`accumulator_factory()`
`(dmx.stats.mixture.MixtureEstimator`
`method)`, 152
`accumulator_factory()`
`(dmx.stats.mvn.MultivariateGaussianEstimator`
`method)`, 75
`accumulator_factory()`
`(dmx.stats.null_dist.NullEstimator`
`method)`, 215
`accumulator_factory()`
`(dmx.stats.optional.OptionalEstimator`
`method)`, 140
`accumulator_factory()`
`(dmx.stats.pdist.ParameterEstimator`
`method)`, 18
`accumulator_factory()`
`(dmx.stats.poisson.PoissonEstimator`
`method)`, 315
`accumulator_factory()`
`(dmx.stats.select.SelectEstimator`
`method)`, 218
`accumulator_factory()`
`(dmx.stats.sequence.SequenceEstimator`
`method)`, 127
`accumulator_factory()`
`(dmx.stats.setdist.BernoulliSetEstimator`
`method)`, 96
`accumulator_factory()`
`(dmx.stats.sparse_markov_transform.SparseMarkovAssociationE`
`method)`, 359
`accumulator_factory()`
`(dmx.stats.spearman_rho.SpearmanRankingEstimator`
`method)`, 227
`accumulator_factory()`
`(dmx.stats.ss_mixture.SemiSupervisedMixtureEstimator`
`method)`, 291
`accumulator_factory()`
`(dmx.stats.tree_hmm.TreeHiddenMarkovEstimator`
`method)`, 363
`accumulator_factory()`
`(dmx.stats.vmf.VonMisesFisherEstimator`
`method)`, 234
`accumulator_factory()`
`(dmx.stats.weighted.WeightedEstimator`
`method)`, 241
`accumulator_factory()`
`(dmx.torch_stats.binomial.BinomialEstimator`
`method)`, 538
`accumulator_factory()`
`(dmx.torch_stats.composite.CompositeEstimator`
`method)`, 607
`accumulator_factory()`
`(dmx.torch_stats.conditional.ConditionalDistributionEstimator`
`method)`, 612
`accumulator_factory()`
`(dmx.torch_stats.dmvn.DiagonalGaussianEstimator`
`method)`, 544
`accumulator_factory()`
`(dmx.torch_stats.exponential.ExponentialEstimator`
`method)`, 550
`accumulator_factory()`
`(dmx.torch_stats.gamma.GammaEstimator`
`method)`, 556
`accumulator_factory()`
`(dmx.torch_stats.gaussian.GaussianEstimator`
`method)`, 562
`accumulator_factory()`
`(dmx.torch_stats.geometric.GeometricEstimator`
`method)`, 568

accumulator_factory()
 (dm_x.torch_stats.heterogenous_mixture.HeterogeneousMixtureEstimator method), 626
 accumulator_factory()
 (dm_x.torch_stats.hmm.HiddenMarkovEstimator method), 631
 accumulator_factory()
 (dm_x.torch_stats.int_plsi.IntegerPLSEstimator method), 641
 accumulator_factory()
 (dm_x.torch_stats.intmultinomial.IntegerMultinomialEstimator method), 576
 accumulator_factory()
 (dm_x.torch_stats.intrange.IntegerCategoricalEstimator method), 583
 accumulator_factory()
 (dm_x.torch_stats.intsetdist.IntegerBernoulliSetEstimator method), 590
 accumulator_factory()
 (dm_x.torch_stats.jmixture.JointMixtureEstimator method), 644
 accumulator_factory()
 (dm_x.torch_stats.mixture.MixtureEstimator method), 651
 accumulator_factory()
 (dm_x.torch_stats.mvn.MultivariateGaussianEstimator method), 597
 accumulator_factory()
 (dm_x.torch_stats.null_dist.NullEstimator method), 616
 accumulator_factory()
 (dm_x.torch_stats.pdist.TorchParameterEstimator method), 526
 accumulator_factory()
 (dm_x.torch_stats.poisson.PoissonEstimator method), 602
 accumulator_factory()
 (dm_x.torch_stats.sequence.SequenceEstimator method), 621
 accumulatorFactory()
 (dm_x.bstats.pdist.ParameterEstimator method), 374
 add_data() (dm_x.utils.automatic.DatumNode method), 653
 add_datum() (dm_x.utils.automatic.DatumNode method), 653
 add_parent() (dm_x.bstats.pdist.ProbabilityDistribution method), 375
 adj_perplexity() (in module dm_x.utils.htsne), 667
 all_vals (dm_x.stats.markovchain.MarkovChainDistribution attribute), 110
 alpha (dm_x.stats.dirichlet.DirichletDistribution attribute), 76
 alpha (dm_x.stats.int_hidden_association.IntegerHiddenAssociationDistribution attribute), 311
 alpha (dm_x.stats.int_hidden_association.IntegerHiddenAssociationEstimator attribute), 314
 alpha (dm_x.stats.lda.LDADistribution attribute), 281
 alpha (dm_x.stats.sparse_markov_transform.SparseMarkovAssociationDistribution attribute), 356
 alpha (dm_x.stats.sparse_markov_transform.SparseMarkovAssociationEstimator attribute), 358
 alpha_ma (dm_x.stats.dirichlet.DirichletDistribution attribute), 76
 alpha_seq_lambda() (in module dm_x.bstats.dirichlet), 400
 alpha_seq_lambda() (in module dm_x.stats.lda), 286
B
 bernoulli_beta_posterior_mode() (in module dm_x.bstats.setdist), 524
 bernoulli_betamix_posterior_mode() (in module dm_x.bstats.setdist), 524
 BernoulliDataEncoder (class in dm_x.bstats.bernoulli), 407
 BernoulliDistribution (class in dm_x.bstats.bernoulli), 408
 BernoulliEncodedData (class in dm_x.bstats.bernoulli), 410
 BernoulliEstimator (class in dm_x.bstats.bernoulli), 410
 BernoulliEstimatorAccumulator (class in dm_x.bstats.bernoulli), 411
 BernoulliEstimatorAccumulatorFactory (class in dm_x.bstats.bernoulli), 413
 BernoulliSampler (class in dm_x.bstats.bernoulli), 413
 BernoulliSetAccumulator (class in dm_x.bstats.setdist), 519
 BernoulliSetAccumulatorFactory (class in dm_x.bstats.setdist), 520
 BernoulliSetDataEncoder (class in dm_x.bstats.setdist), 521
 BernoulliSetDistribution (class in dm_x.bstats.setdist), 521
 BernoulliSetDistribution (class in dm_x.stats.setdist), 92
 BernoulliSetEncodedData (class in dm_x.bstats.setdist), 523
 BernoulliSetEstimator (class in dm_x.bstats.setdist), 523
 BernoulliSetEstimator (class in dm_x.stats.setdist), 95
 BernoulliSetSampler (class in dm_x.bstats.setdist), 524
 BernoulliSetSampler (class in dm_x.stats.setdist), 96
 best_of() (in module dm_x.bstats.bestimation), 383
 best_of() (in module dm_x.utils.estimation), 660
 best_of_mpi() (in module dm_x.mpi4py.utils.estimation), 711

- beta (*dmx.stats.exponential.ExponentialDistribution* attribute), 50
- beta (*dmx.torch_stats.exponential.ExponentialDistribution* attribute), 548
- beta() (in module *dmx.utils.special*), 681
- BetaDistribution (class in *dmx.bstats.beta*), 389
- betaIn() (in module *dmx.utils.special*), 681
- BetaSampler (class in *dmx.bstats.beta*), 391
- bincount() (in module *dmx.stats.int_plsi*), 276
- bincount1() (in module *dmx.torch_utils.optsutil*), 696
- bincount_2d() (in module *dmx.torch_utils.vector*), 697
- binomial_rank() (in module *dmx.utils.pvalues*), 680
- BinomialAccumulator (class in *dmx.torch_stats.binomial*), 532
- BinomialAccumulatorFactory (class in *dmx.torch_stats.binomial*), 534
- BinomialDataEncoder (class in *dmx.torch_stats.binomial*), 534
- BinomialDistribution (class in *dmx.stats.binomial*), 19
- BinomialDistribution (class in *dmx.torch_stats.binomial*), 535
- BinomialEstimator (class in *dmx.stats.binomial*), 22
- BinomialEstimator (class in *dmx.torch_stats.binomial*), 537
- BinomialSampler (class in *dmx.stats.binomial*), 24
- BinomialSampler (class in *dmx.torch_stats.binomial*), 539
- BinomialTorchEncodedSequence (class in *dmx.torch_stats.binomial*), 539
- bool_count (*dmx.utils.automatic.DatumNode* attribute), 653
- ## C
- ca (*dmx.stats.dmvn.DiagonalGaussianDistribution* attribute), 65
- CategoricalDataEncoder (class in *dmx.bstats.categorical*), 414
- CategoricalDistribution (class in *dmx.bstats.categorical*), 414
- CategoricalDistribution (class in *dmx.stats.categorical*), 82
- CategoricalEncodedData (class in *dmx.bstats.categorical*), 416
- CategoricalEstimator (class in *dmx.bstats.categorical*), 416
- CategoricalEstimator (class in *dmx.stats.categorical*), 84
- CategoricalEstimatorAccumulator (class in *dmx.bstats.categorical*), 417
- CategoricalEstimatorAccumulatorFactory (class in *dmx.bstats.categorical*), 419
- CategoricalSampler (class in *dmx.bstats.categorical*), 419
- CategoricalSampler (class in *dmx.stats.categorical*), 86
- cb (*dmx.stats.dmvn.DiagonalGaussianDistribution* attribute), 66
- cbg() (in module *dmx.bstats.dpm*), 486
- cc (*dmx.stats.dmvn.DiagonalGaussianDistribution* attribute), 66
- children (*dmx.utils.automatic.DatumNode* attribute), 652
- cho_solve() (in module *dmx.utils.vector*), 684
- choice() (in module *dmx.torch_utils.vector*), 698
- chol (*dmx.stats.mvn.MultivariateGaussianDistribution* attribute), 71
- chol (*dmx.torch_stats.mvn.MultivariateGaussianDistribution* attribute), 594
- chol_const (*dmx.stats.mvn.MultivariateGaussianDistribution*.self attribute), 72
- chol_const (*dmx.torch_stats.mvn.MultivariateGaussianDistribution* attribute), 594
- cholesky() (in module *dmx.utils.vector*), 684
- classify() (in module *dmx.utils.metrics*), 675
- combine() (*dmx.bstats.bernoulli.BernoulliEstimatorAccumulator* method), 411
- combine() (*dmx.bstats.categorical.CategoricalEstimatorAccumulator* method), 417
- combine() (*dmx.bstats.composite.CompositeEstimatorAccumulator* method), 472
- combine() (*dmx.bstats.conditional.ConditionalDistributionEstimatorAccumulator* method), 478
- combine() (*dmx.bstats.dirac.DiracAccumulator* method), 419
- combine() (*dmx.bstats.dirichlet.DirichletAccumulator* method), 394
- combine() (*dmx.bstats.dmvn.DiagonalGaussianAccumulator* method), 425
- combine() (*dmx.bstats.dpm.DirichletProcessMixtureAccumulator* method), 481
- combine() (*dmx.bstats.exponential.ExponentialAccumulator* method), 431
- combine() (*dmx.bstats.gamma.GammaAccumulator* method), 437
- combine() (*dmx.bstats.gaussian.GaussianAccumulator* method), 443
- combine() (*dmx.bstats.geometric.GeometricAccumulator* method), 449
- combine() (*dmx.bstats.ignored.IgnoredAccumulator* method), 487
- combine() (*dmx.bstats.intrange.IntegerCategoricalAccumulator* method), 455
- combine() (*dmx.bstats.mixture.MixtureEstimatorAccumulator* method), 497
- combine() (*dmx.bstats.nulldist.NullAccumulator* method), 499
- combine() (*dmx.bstats.optional.OptionalEstimatorAccumulator* method), 499

method), 509
 combine() (dmx.bstats.pdist.StatisticAccumulator method), 381
 combine() (dmx.bstats.poisson.PoissonEstimatorAccumulator method), 465
 combine() (dmx.bstats.sequence.SequenceEstimatorAccumulator method), 516
 combine() (dmx.bstats.setdist.BernoulliSetAccumulator method), 519
 combine() (dmx.stats.dirac_length.DiracMixtureAccumulator method), 187
 combine() (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator method), 243
 combine() (dmx.stats.gmm.GaussianMixtureAccumulator method), 256
 combine() (dmx.stats.hidden_association.HiddenAssociationAccumulator method), 295
 combine() (dmx.stats.icltree.ICLTreeAccumulator method), 301
 combine() (dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator method), 197
 combine() (dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditAccumulator method), 205
 combine() (dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator method), 307
 combine() (dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator method), 320
 combine() (dmx.stats.int_markovchain.IntegerMarkovChainAccumulator method), 336
 combine() (dmx.stats.int_plsi.IntegerPLSIAccumulator method), 268
 combine() (dmx.stats.lda.LDAEstimatorAccumulator method), 284
 combine() (dmx.stats.look_back_hmm.LookbackHiddenMarkovAccumulator method), 349
 combine() (dmx.stats.null_dist.NullAccumulator method), 211
 combine() (dmx.stats.pdist.StatisticAccumulator method), 14
 combine() (dmx.stats.select.SelectEstimatorAccumulator method), 219
 combine() (dmx.stats.sparse_markov_transform.SparseMarkovTransformAccumulator method), 353
 combine() (dmx.stats.spearman_rho.SpearmanRankingAccumulator method), 222
 combine() (dmx.stats.ss_mixture.SemiSupervisedMixtureEstimatorAccumulator method), 292
 combine() (dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator method), 360
 combine() (dmx.stats.vmf.VonMisesFisherAccumulator method), 229
 combine() (dmx.stats.weighted.WeightedAccumulator method), 237
 combine() (dmx.torch_stats.binomial.BinomialAccumulator method), 533
 combine() (dmx.torch_stats.composite.CompositeAccumulator method), 603
 combine() (dmx.torch_stats.conditional.ConditionalDistributionAccumulator method), 609
 combine() (dmx.torch_stats.dmvn.DiagonalGaussianAccumulator method), 540
 combine() (dmx.torch_stats.exponential.ExponentialAccumulator method), 546
 combine() (dmx.torch_stats.gamma.GammaAccumulator method), 552
 combine() (dmx.torch_stats.gaussian.GaussianAccumulator method), 559
 combine() (dmx.torch_stats.geometric.GeometricAccumulator method), 564
 combine() (dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulator method), 622
 combine() (dmx.torch_stats.hmm.HiddenMarkovAccumulator method), 628
 combine() (dmx.torch_stats.int_plsi.IntegerPLSIAccumulator method), 637
 combine() (dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator method), 570
 combine() (dmx.torch_stats.intrange.IntegerCategoricalAccumulator method), 579
 combine() (dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulator method), 586
 combine() (dmx.torch_stats.jmixture.JointMixtureEstimatorAccumulator method), 645
 combine() (dmx.torch_stats.mixture.MixtureAccumulator method), 647
 combine() (dmx.torch_stats.mvn.MultivariateGaussianAccumulator method), 592
 combine() (dmx.torch_stats.null_dist.NullAccumulator method), 613
 combine() (dmx.torch_stats.pdist.TorchStatisticAccumulator method), 530
 combine() (dmx.torch_stats.poisson.PoissonAccumulator method), 598
 combine() (dmx.torch_stats.sequence.SequenceAccumulator method), 618
 combine() (dmx.stats.int_plsi.IntegerPLSIAccumulator attribute), 267
 comp_accounts (dmx.stats.dirac_length.DiracMixtureAccumulator attribute), 186
 comp_accounts (dmx.stats.gmm.GaussianMixtureAccumulator attribute), 255
 comp_key (dmx.stats.dirac_length.DiracMixtureAccumulator attribute), 186
 comp_key (dmx.stats.gmm.GaussianMixtureAccumulator attribute), 255
 comp_sampler1 (dmx.stats.jmixture.JointMixtureSampler attribute), 167
 comp_sampler2 (dmx.stats.jmixture.JointMixtureSampler

<i>attribute</i>), 167			<i>dmx.torch_stats.composite</i>), 605	
<code>comp_samplers</code> (<i>dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution</i> <i>attribute</i>), 160			<code>CompositeDistribution</code> (class in <i>dmx.bstats.composite</i>), 468	
<code>comp_samplers</code> (<i>dmx.stats.mixture.MixtureSampler</i> <i>attribute</i>), 153			<code>CompositeDistribution</code> (class in <i>dmx.stats.composite</i>), 118	
<code>component_log_density()</code> (<i>dmx.stats.dirac_length.DiracMixtureDistribution</i> <i>method</i>), 191			<code>CompositeDistribution</code> (class in <i>dmx.torch_stats.composite</i>), 605	
<code>component_log_density()</code> (<i>dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution</i> <i>method</i>), 248			<code>CompositeEncodedData</code> (class in <i>dmx.bstats.composite</i>), 470	
<code>component_log_density()</code> (<i>dmx.stats.gmm.GaussianMixtureDistribution</i> <i>method</i>), 260			<code>CompositeEstimator</code> (class in <i>dmx.bstats.composite</i>), 471	
<code>component_log_density()</code> (<i>dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution</i> <i>method</i>), 155			<code>CompositeEstimator</code> (class in <i>dmx.stats.composite</i>), 120	
<code>component_log_density()</code> (<i>dmx.stats.hmixture.HierarchicalMixtureDistribution</i> <i>method</i>), 170			<code>CompositeEstimator</code> (class in <i>dmx.torch_stats.composite</i>), 607	
<code>component_log_density()</code> (<i>dmx.stats.int_plsi.IntegerPLSIDistribution</i> <i>method</i>), 272			<code>CompositeEstimatorAccumulator</code> (class in <i>dmx.bstats.composite</i>), 472	
<code>component_log_density()</code> (<i>dmx.stats.mixture.MixtureDistribution</i> <i>method</i>), 148			<code>CompositeSampler</code> (class in <i>dmx.bstats.composite</i>), 474	
<code>component_log_density()</code> (<i>dmx.torch_stats.heterogeneous_mixture.HeterogeneousMixtureDistribution</i> <i>method</i>), 625			<code>CompositeSampler</code> (class in <i>dmx.stats.composite</i>), 122	
<code>component_log_density()</code> (<i>dmx.torch_stats.int_plsi.IntegerPLSIDistribution</i> <i>method</i>), 639			<code>CompositeSampler</code> (class in <i>dmx.torch_stats.composite</i>), 607	
<code>component_log_density()</code> (<i>dmx.torch_stats.mixture.MixtureDistribution</i> <i>method</i>), 649			<code>CompositeTorchEncodedSequence</code> (class in <i>dmx.torch_stats.composite</i>), 607	
<code>components</code> (<i>dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution</i> <i>attribute</i>), 154			<code>concentrations_for_dimension()</code> (<i>dmx.bstats.dirichlet.DirichletDistribution</i> <i>method</i>), 396	
<code>components</code> (<i>dmx.stats.mixture.MixtureDistribution</i> <i>attribute</i>), 146			<code>concentrations_for_keys()</code> (<i>dmx.bstats.dirichlet.DictDirichletDistribution</i> <i>method</i>), 391	
<code>components</code> (<i>dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution</i> <i>attribute</i>), 288			<code>cond_dist</code> (<i>dmx.stats.hidden_association.HiddenAssociationDistribution</i> <i>attribute</i>), 297	
<code>components1</code> (<i>dmx.stats.jmixture.JointMixtureDistribution</i> <i>attribute</i>), 162			<code>cond_dist</code> (<i>dmx.stats.int_markovchain.IntegerMarkovChainDistribution</i> <i>attribute</i>), 340	
<code>components2</code> (<i>dmx.stats.jmixture.JointMixtureDistribution</i> <i>attribute</i>), 162			<code>cond_estimator</code> (<i>dmx.stats.hidden_association.HiddenAssociationEstimator</i> <i>attribute</i>), 299	
<code>CompositeAccumulator</code> (class in <i>dmx.torch_stats.composite</i>), 603			<code>cond_estimator</code> (<i>dmx.stats.sparse_markov_transform.SparseMarkovAssociation</i> <i>attribute</i>), 356	
<code>CompositeAccumulatorFactory</code> (class in <i>dmx.bstats.composite</i>), 467			<code>cond_weights</code> (<i>dmx.stats.int_hidden_association.IntegerHiddenAssociation</i> <i>attribute</i>), 311	
<code>CompositeAccumulatorFactory</code> (class in <i>dmx.torch_stats.composite</i>), 604			<code>ConditionalDensities</code> (<i>dmx.stats.icltree.ICLTreeDistribution</i> <i>attribute</i>), 304	
<code>CompositeDataEncoder</code> (class in <i>dmx.bstats.composite</i>), 468			<code>conditional_log_densities</code> (<i>dmx.stats.icltree.ICLTreeDistribution</i> <i>attribute</i>), 304	
<code>CompositeDataEncoder</code> (class in <i>dmx.torch_stats.composite</i>), 608			<code>ConditionalDataEncoder</code> (class in <i>dmx.bstats.conditional</i>), 474	
			<code>ConditionalDistribution</code> (class in <i>dmx.bstats.conditional</i>), 475	
			<code>ConditionalDistribution</code> (class in <i>dmx.stats.conditional</i>), 129	
			<code>ConditionalDistribution</code> (class in <i>dmx.torch_stats.conditional</i>), 608	
			<code>ConditionalDistributionAccumulator</code> (class in <i>dmx.bstats.conditional</i>), 475	

`dmx.torch_stats.conditional`), 609
`ConditionalDistributionAccumulatorFactory` (class in `dmx.bstats.conditional`), 477
`ConditionalDistributionAccumulatorFactory` (class in `dmx.torch_stats.conditional`), 610
`ConditionalDistributionDataEncoder` (class in `dmx.torch_stats.conditional`), 611
`ConditionalDistributionEstimator` (class in `dmx.bstats.conditional`), 477
`ConditionalDistributionEstimator` (class in `dmx.stats.conditional`), 132
`ConditionalDistributionEstimator` (class in `dmx.torch_stats.conditional`), 611
`ConditionalDistributionEstimatorAccumulator` (class in `dmx.bstats.conditional`), 478
`ConditionalDistributionSampler` (class in `dmx.bstats.conditional`), 480
`ConditionalDistributionSampler` (class in `dmx.stats.conditional`), 134
`ConditionalDistributionSampler` (class in `dmx.torch_stats.conditional`), 612
`ConditionalEncodedData` (class in `dmx.bstats.conditional`), 480
`ConditionalSampler` (class in `dmx.stats.pdist`), 14
`ConditionalSampler` (class in `dmx.torch_stats.pdist`), 525
`ConditionalTorchEncodedSequence` (class in `dmx.torch_stats.conditional`), 613
`const` (`dmx.stats.gaussian.GaussianDistribution` attribute), 45
`const` (`dmx.stats.log_gaussian.LogGaussianDistribution` attribute), 61
`copy()` (`dmx.utils.automatic.DatumNode` method), 653
`count` (`dmx.stats.composite.CompositeDistribution` attribute), 118
`count` (`dmx.stats.composite.CompositeEstimator` attribute), 121
`count` (`dmx.stats.spearman_rho.SpearmanRankingAccumulator` attribute), 221
`count` (`dmx.stats.vmf.VonMisesFisherAccumulator` attribute), 228
`count` (`dmx.torch_stats.binomial.BinomialAccumulator` attribute), 532
`count` (`dmx.torch_stats.composite.CompositeDistribution` attribute), 605
`count` (`dmx.torch_stats.dmvn.DiagonalGaussianAccumulator` attribute), 540
`count` (`dmx.torch_stats.exponential.ExponentialAccumulator` attribute), 546
`count` (`dmx.torch_stats.gaussian.GaussianAccumulator` attribute), 558
`count` (`dmx.torch_stats.geometric.GeometricAccumulator` attribute), 564
`count` (`dmx.torch_stats.poisson.PoissonAccumulator` attribute), 598
`count` (`dmx.torch_stats.gaussian.GaussianAccumulator` attribute), 558
`count2` (`dmx.torch_stats.gaussian.GaussianAccumulator` attribute), 558
`count_by_value()` (in module `dmx.torch_utils.optsutil`), 697
`count_by_value()` (in module `dmx.utils.optsutil`), 677
`count_vec` (`dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator` attribute), 570
`count_vec` (`dmx.torch_stats.intrange.IntegerCategoricalAccumulator` attribute), 579
`covar` (`dmx.stats.dmvn.DiagonalGaussianDistribution` attribute), 65
`covar` (`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution` attribute), 247
`covar` (`dmx.stats.mvn.MultivariateGaussianDistribution` attribute), 71
`covar` (`dmx.torch_stats.mvn.MultivariateGaussianDistribution` attribute), 594
`cross_entropy()` (`dmx.bstats.bernoulli.BernoulliDistribution` method), 408
`cross_entropy()` (`dmx.bstats.beta.BetaDistribution` method), 390
`cross_entropy()` (`dmx.bstats.catdirichlet.DictDirichletDistribution` method), 392
`cross_entropy()` (`dmx.bstats.categorical.CategoricalDistribution` method), 414
`cross_entropy()` (`dmx.bstats.composite.CompositeDistribution` method), 468
`cross_entropy()` (`dmx.bstats.dirichlet.DirichletDistribution` method), 396
`cross_entropy()` (`dmx.bstats.gamma.GammaDistribution` method), 438
`cross_entropy()` (`dmx.bstats.geometric.GeometricDistribution` method), 451
`cross_entropy()` (`dmx.bstats.ignored.IgnoredDistribution` method), 489
`cross_entropy()` (`dmx.bstats.intrange.IntegerCategoricalDistribution` method), 457
`cross_entropy()` (`dmx.bstats.mvngamma.MultivariateNormalGammaDistribution` method), 402
`cross_entropy()` (`dmx.bstats.normgamma.NormalGammaDistribution` method), 404
`cross_entropy()` (`dmx.bstats.nulldist.NullDistribution` method), 501
`cross_entropy()` (`dmx.bstats.optional.OptionalDistribution` method), 506
`cross_entropy()` (`dmx.bstats.pdist.ProbabilityDistribution` method), 376
`cross_entropy()` (`dmx.bstats.poisson.PoissonDistribution` method), 461
`cross_entropy()` (`dmx.bstats.sequence.SequenceDistribution` method), 513
`cross_entropy()` (`dmx.bstats.symdirichlet.SymmetricDirichletDistribution` method), 513

method), 405

D

`data (dmx.stats.dirac_length.DiracMixtureEncodedDataSequence attribute)`, 194

`data (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureEncodedDataSequence attribute)`, 251

`data (dmx.stats.gmm.GaussianMixtureEncodedDataSequence attribute)`, 263

`data (dmx.stats.hidden_association.HiddenAssociationEncodedDataSequence attribute)`, 299

`data (dmx.stats.icltree.ICLTreeEncodedDataSequence attribute)`, 305

`data (dmx.stats.int_hidden_association.IntegerHiddenAssociationEncodedDataSequence attribute)`, 313

`data (dmx.stats.int_hidden_markov.IntegerHiddenMarkovEncodedDataSequence attribute)`, 324

`data (dmx.stats.int_markovchain.IntegerMarkovChainEncodedDataSequence attribute)`, 342

`data (dmx.stats.int_plsi.IntegerPLSIEncodedDataSequence attribute)`, 274

`data (dmx.stats.null_dist.NullEncodedDataSequence attribute)`, 214

`data (dmx.stats.pdist.EncodedDataSequence attribute)`, 19

`data (dmx.stats.spearman_rho.SpearmanRankingEncodedDataSequence attribute)`, 226

`data (dmx.stats.ss_mixture.SemiSupervisedMixtureEncodedDataSequence attribute)`, 290

`data (dmx.stats.vmf.VonMisesFisherEncodedDataSequence attribute)`, 233

`data (dmx.stats.weighted.WeightedEncodedDataSequence attribute)`, 241

`data (dmx.torch_stats.gaussian.GaussianTorchEncodedSequence attribute)`, 563

`data (dmx.torch_stats.pdist.TorchEncodedSequence attribute)`, 526

`DataFrameEncodableAccumulator (class in dmx.bstats.pdist)`, 372

`DataFrameEncodableDistribution (class in dmx.bstats.pdist)`, 373

`DataSequenceEncoder (class in dmx.bstats.pdist)`, 373

`DataSequenceEncoder (class in dmx.stats.pdist)`, 18

`DatumNode (class in dmx.utils.automatic)`, 652

`dc_key (dmx.stats.int_plsi.IntegerPLSIAccumulator attribute)`, 267

`default_dist (dmx.stats.conditional.ConditionalDistribution attribute)`, 129

`default_estimator (dmx.stats.conditional.ConditionalDistribution attribute)`, 133

`default_sampler (dmx.stats.conditional.ConditionalDistribution attribute)`, 134

`default_value (dmx.stats.categorical.CategoricalDistribution attribute)`, 82

`default_value (dmx.stats.categorical.CategoricalEstimator attribute)`, 85

`default_value (dmx.stats.markovchain.MarkovChainDistribution attribute)`, 110

`delta (dmx.stats.dirichlet.DirichletEstimator attribute)`, 79

`density() (dmx.bstats.beta.BetaDistribution method)`, 390

`density() (dmx.bstats.dirac.DiracDistribution method)`, 422

`density() (dmx.bstats.gamma.GammaDistribution method)`, 439

`density() (dmx.bstats.ignored.IgnoredDistribution method)`, 489

`density() (dmx.bstats.mvngamma.MultivariateNormalGammaDistribution method)`, 402

`density() (dmx.bstats.normgamma.NormalGammaDistribution method)`, 404

`density() (dmx.bstats.nulldist.NullDistribution method)`, 502

`density() (dmx.bstats.pdist.ProbabilityDistribution method)`, 376

`density() (dmx.stats.binomial.BinomialDistribution method)`, 21

`density() (dmx.stats.categorical.CategoricalDistribution method)`, 83

`density() (dmx.stats.catmultinomial.MultinomialDistribution method)`, 104

`density() (dmx.stats.composite.CompositeDistribution method)`, 119

`density() (dmx.stats.conditional.ConditionalDistribution method)`, 131

`density() (dmx.stats.dirac_length.DiracMixtureDistribution method)`, 192

`density() (dmx.stats.dirichlet.DirichletDistribution method)`, 77

`density() (dmx.stats.dmvn.DiagonalGaussianDistribution method)`, 66

`density() (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution method)`, 248

`density() (dmx.stats.exponential.ExponentialDistribution method)`, 51

`density() (dmx.stats.gamma.GammaDistribution method)`, 56

`density() (dmx.stats.gaussian.GaussianDistribution method)`, 46

`density() (dmx.stats.geometric.GeometricDistribution method)`, 26

`density() (dmx.stats.gmm.GaussianMixtureDistribution method)`, 261

`density() (dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution method)`, 155

`density() (dmx.stats.hidden_association.HiddenAssociationDistribution method)`, 298

`density()` (`dmx.stats.hidden_markov.HiddenMarkovModelDistribution` method), 94
`method`), 179
`density()` (`dmx.stats.hmixture.HierarchicalMixtureDistribution` method), 357
`method`), 170
`density()` (`dmx.stats.ictree.ICLTreeDistribution` method), 304
`density()` (`dmx.stats.ignored.IgnoredDistribution` method), 142
`density()` (`dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution` method), 364
`method`), 201
`density()` (`dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditDistribution` method), 232
`method`), 208
`density()` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution` method), 240
`method`), 312
`density()` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution` method), 536
`method`), 327
`density()` (`dmx.stats.int_markovchain.IntegerMarkovChainDistribution` method), 606
`method`), 341
`density()` (`dmx.stats.int_plsi.IntegerPLSIDistribution` method), 272
`density()` (`dmx.stats.int_spike.SpikeAndSlabDistribution` method), 41
`density()` (`dmx.stats.intmultinomial.IntegerMultinomialDistribution` method), 548
`method`), 98
`density()` (`dmx.stats.intrange.IntegerCategoricalDistribution` method), 554
`method`), 36
`density()` (`dmx.stats.intsetdist.IntegerBernoulliSetDistribution` method), 561
`method`), 89
`density()` (`dmx.stats.jmixture.JointMixtureDistribution` method), 164
`density()` (`dmx.stats.lda.LDADistribution` method), 281
`density()` (`dmx.stats.log_gaussian.LogGaussianDistribution` method), 61
`density()` (`dmx.stats.look_back_hmm.LookbackHiddenMarkovModelDistribution` method), 346
`density()` (`dmx.stats.markovchain.MarkovChainDistribution` method), 112
`density()` (`dmx.stats.mixture.MixtureDistribution` method), 148
`density()` (`dmx.stats.mvn.MultivariateGaussianDistribution` method), 72
`density()` (`dmx.stats.null_dist.NullDistribution` method), 213
`density()` (`dmx.stats.optional.OptionalDistribution` method), 137
`density()` (`dmx.stats.pdist.ProbabilityDistribution` method), 11
`density()` (`dmx.stats.poisson.PoissonDistribution` method), 31
`density()` (`dmx.stats.select.SelectDistribution` method), 217
`density()` (`dmx.stats.sequence.SequenceDistribution` method), 124
`density()` (`dmx.stats.setdist.BernoulliSetDistribution` method), 179
`density()` (`dmx.stats.sparse_markov_transform.SparseMarkovAssociationDistribution` method), 357
`density()` (`dmx.stats.spearman_rho.SpearmanRankingDistribution` method), 225
`density()` (`dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution` method), 289
`density()` (`dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution` method), 364
`density()` (`dmx.stats.vmf.VonMisesFisherDistribution` method), 232
`density()` (`dmx.stats.weighted.WeightedDistribution` method), 240
`density()` (`dmx.torch_stats.binomial.BinomialDistribution` method), 536
`density()` (`dmx.torch_stats.composite.CompositeDistribution` method), 606
`density()` (`dmx.torch_stats.conditional.ConditionalDistribution` method), 608
`density()` (`dmx.torch_stats.dmvn.DiagonalGaussianDistribution` method), 543
`density()` (`dmx.torch_stats.exponential.ExponentialDistribution` method), 548
`density()` (`dmx.torch_stats.gamma.GammaDistribution` method), 554
`density()` (`dmx.torch_stats.gaussian.GaussianDistribution` method), 561
`density()` (`dmx.torch_stats.geometric.GeometricDistribution` method), 566
`density()` (`dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDistribution` method), 625
`density()` (`dmx.torch_stats.hmm.HiddenMarkovModelDistribution` method), 633
`density()` (`dmx.torch_stats.int_plsi.IntegerPLSIDistribution` method), 639
`density()` (`dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution` method), 574
`density()` (`dmx.torch_stats.intrange.IntegerCategoricalDistribution` method), 581
`density()` (`dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution` method), 589
`density()` (`dmx.torch_stats.jmixture.JointMixtureDistribution` method), 643
`density()` (`dmx.torch_stats.mixture.MixtureDistribution` method), 649
`density()` (`dmx.torch_stats.mvn.MultivariateGaussianDistribution` method), 595
`density()` (`dmx.torch_stats.null_dist.NullDistribution` method), 615
`density()` (`dmx.torch_stats.pdist.TorchProbabilityDistribution` method), 527
`density()` (`dmx.torch_stats.poisson.PoissonDistribution` method), 600
`density()` (`dmx.torch_stats.sequence.SequenceDistribution` method), 124

method), 620
 dependency_list (dmx.stats.icltree.ICLTreeDistribution attribute), 304
 detect_device() (in module dmx.torch_utils), 692
 device (dmx.torch_stats.gaussian.GaussianTorchEncodedSequence attribute), 563
 device (dmx.torch_stats.pdist.TorchEncodedSequence attribute), 526
 df_initialize() (dmx.bstats.composite.CompositeEstimatorAccumulator method), 472
 df_initialize() (dmx.bstats.gaussian.GaussianAccumulator method), 443
 df_initialize() (dmx.bstats.pdist.DataFrameEncodableAccumulator method), 372
 df_log_density() (dmx.bstats.composite.CompositeDistribution method), 469
 df_log_density() (dmx.bstats.pdist.ProbabilityDistribution method), 376
 df_update() (dmx.bstats.composite.CompositeEstimatorAccumulator method), 472
 df_update() (dmx.bstats.gaussian.GaussianAccumulator method), 443
 df_update() (dmx.bstats.pdist.DataFrameEncodableAccumulator method), 373
 diag() (in module dmx.utils.vector), 685
 DiagonalGaussianAccumulator (class in dmx.bstats.dmvn), 425
 DiagonalGaussianAccumulator (class in dmx.torch_stats.dmvn), 540
 DiagonalGaussianAccumulatorFactory (class in dmx.bstats.dmvn), 427
 DiagonalGaussianAccumulatorFactory (class in dmx.torch_stats.dmvn), 541
 DiagonalGaussianDataEncoder (class in dmx.bstats.dmvn), 427
 DiagonalGaussianDataEncoder (class in dmx.torch_stats.dmvn), 542
 DiagonalGaussianDistribution (class in dmx.bstats.dmvn), 427
 DiagonalGaussianDistribution (class in dmx.stats.dmvn), 65
 DiagonalGaussianDistribution (class in dmx.torch_stats.dmvn), 542
 DiagonalGaussianEncodedData (class in dmx.bstats.dmvn), 429
 DiagonalGaussianEstimator (class in dmx.bstats.dmvn), 429
 DiagonalGaussianEstimator (class in dmx.stats.dmvn), 68
 DiagonalGaussianEstimator (class in dmx.torch_stats.dmvn), 544
 DiagonalGaussianMixtureAccumulator (class in dmx.stats.dmvn_mixture), 242
 DiagonalGaussianMixtureAccumulatorFactory (class in dmx.stats.dmvn_mixture), 245
 DiagonalGaussianMixtureDataEncoder (class in dmx.stats.dmvn_mixture), 246
 DiagonalGaussianMixtureDistribution (class in dmx.stats.dmvn_mixture), 247
 DiagonalGaussianMixtureEncodedDataSequence (class in dmx.stats.dmvn_mixture), 251
 DiagonalGaussianMixtureEstimator (class in dmx.stats.dmvn_mixture), 251
 DiagonalGaussianMixtureSampler (class in dmx.stats.dmvn_mixture), 253
 DiagonalGaussianSampler (class in dmx.bstats.dmvn), 420
 DiagonalGaussianSampler (class in dmx.stats.dmvn), 70
 DiagonalGaussianSampler (class in dmx.torch_stats.dmvn), 545
 DiagonalGaussianTorchEncodedSequence (class in dmx.torch_stats.dmvn), 545
 DictDirichletDistribution (class in dmx.bstats.catdirichlet), 391
 DictDirichletSampler (class in dmx.bstats.catdirichlet), 393
 digamma() (in module dmx.utils.special), 681
 digammainv() (in module dmx.utils.special), 681
 dim (dmx.stats.dirichlet.DirichletDistribution attribute), 76
 dim (dmx.stats.dirichlet.DirichletEstimator attribute), 79
 dim (dmx.stats.dmvn.DiagonalGaussianDistribution attribute), 65
 dim (dmx.stats.dmvn.DiagonalGaussianEstimator attribute), 68
 dim (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator attribute), 242
 dim (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulatorFactory attribute), 246
 dim (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution attribute), 247
 dim (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureEstimator attribute), 252
 dim (dmx.stats.mvn.MultivariateGaussianDistribution attribute), 71
 dim (dmx.stats.mvn.MultivariateGaussianEstimator attribute), 73
 dim (dmx.stats.spearman_rho.SpearmanRankingAccumulatorFactory attribute), 223
 dim (dmx.stats.spearman_rho.SpearmanRankingDistribution attribute), 225
 dim (dmx.stats.spearman_rho.SpearmanRankingEstimator attribute), 226
 dim (dmx.stats.vmf.VonMisesFisherAccumulator attribute), 228
 dim (dmx.stats.vmf.VonMisesFisherAccumulatorFactory attribute), 230

`dim (dmx.stats.vmf.VonMisesFisherDistribution attribute), 231`
`dim (dmx.stats.vmf.VonMisesFisherEstimator attribute), 233`
`dim (dmx.torch_stats.dmvn.DiagonalGaussianAccumulator attribute), 540`
`dim (dmx.torch_stats.dmvn.DiagonalGaussianDataEncoder attribute), 542`
`dim (dmx.torch_stats.dmvn.DiagonalGaussianDistribution attribute), 543`
`dim (dmx.torch_stats.dmvn.DiagonalGaussianEstimator attribute), 544`
`dim (dmx.torch_stats.mvn.MultivariateGaussianDistribution attribute), 594`
`dim (dmx.torch_stats.mvn.MultivariateGaussianEstimator attribute), 596`
`dimension() (dmx.bstats.symdirichlet.SymmetricDirichletDistribution method), 406`
`DiracAccumulator (class in dmx.bstats.dirac), 419`
`DiracAccumulatorFactory (class in dmx.bstats.dirac), 421`
`DiracDataEncoder (class in dmx.bstats.dirac), 421`
`DiracDistribution (class in dmx.bstats.dirac), 421`
`DiracEncodedData (class in dmx.bstats.dirac), 423`
`DiracEstimator (class in dmx.bstats.dirac), 423`
`DiracMixtureAccumulator (class in dmx.stats.dirac_length), 186`
`DiracMixtureAccumulatorFactory (class in dmx.stats.dirac_length), 189`
`DiracMixtureDataEncoder (class in dmx.stats.dirac_length), 190`
`DiracMixtureDistribution (class in dmx.stats.dirac_length), 190`
`DiracMixtureEncodedDataSequence (class in dmx.stats.dirac_length), 194`
`DiracMixtureEstimator (class in dmx.stats.dirac_length), 194`
`DiracMixtureSampler (class in dmx.stats.dirac_length), 195`
`DiracSampler (class in dmx.bstats.dirac), 424`
`dirichlet_param_solve() (in module dmx.bstats.dirichlet), 400`
`DirichletAccumulator (class in dmx.bstats.dirichlet), 394`
`DirichletAccumulatorFactory (class in dmx.bstats.dirichlet), 395`
`DirichletDistribution (class in dmx.bstats.dirichlet), 396`
`DirichletDistribution (class in dmx.stats.dirichlet), 76`
`DirichletEstimator (class in dmx.bstats.dirichlet), 398`
`DirichletEstimator (class in dmx.stats.dirichlet), 79`
`DirichletProcessMixtureAccumulator (class in dmx.bstats.dpm), 481`
`DirichletProcessMixtureAccumulatorFactory (class in dmx.bstats.dpm), 482`
`DirichletProcessMixtureDistribution (class in dmx.bstats.dpm), 483`
`DirichletProcessMixtureEstimator (class in dmx.bstats.dpm), 485`
`DirichletProcessMixtureSampler (class in dmx.bstats.dpm), 486`
`DirichletSampler (class in dmx.bstats.dirichlet), 399`
`DirichletSampler (class in dmx.stats.dirichlet), 81`
`dist (dmx.stats.binomial.BinomialSampler attribute), 24`
`dist (dmx.stats.catmultinomial.MultinomialDistribution attribute), 103`
`dist (dmx.stats.catmultinomial.MultinomialSampler attribute), 108`
`dist (dmx.stats.composite.CompositeSampler attribute), 122`
`dist (dmx.stats.conditional.ConditionalDistributionSampler attribute), 134`
`dist (dmx.stats.dirac_length.DiracMixtureDistribution attribute), 191`
`dist (dmx.stats.dirichlet.DirichletSampler attribute), 81`
`dist (dmx.stats.dmvn.DiagonalGaussianSampler attribute), 70`
`dist (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureSampler attribute), 253`
`dist (dmx.stats.exponential.ExponentialSampler attribute), 54`
`dist (dmx.stats.gamma.GammaSampler attribute), 59`
`dist (dmx.stats.gaussian.GaussianSampler attribute), 49`
`dist (dmx.stats.geometric.GeometricSampler attribute), 29`
`dist (dmx.stats.gmm.GaussianMixtureSampler attribute), 265`
`dist (dmx.stats.heterogeneous_mixture.HeterogeneousMixtureSampler attribute), 160`
`dist (dmx.stats.hidden_markov.HiddenMarkovSampler attribute), 184`
`dist (dmx.stats.hmixture.HierarchicalMixtureSampler attribute), 175`
`dist (dmx.stats.ictree.ICLTreeSampler attribute), 306`
`dist (dmx.stats.ignored.IgnoredDistribution attribute), 142`
`dist (dmx.stats.ignored.IgnoredEstimator attribute), 144`
`dist (dmx.stats.int_edit_setdist.IntegerBernoulliEditSampler attribute), 204`
`dist (dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler attribute), 329`
`dist (dmx.stats.int_markovchain.IntegerMarkovChainSampler attribute), 344`
`dist (dmx.stats.int_plsi.IntegerPLSISampler attribute), 276`
`dist (dmx.stats.int_spike.SpikeAndSlabSampler attribute), 276`

- tribute), 44
- `dist (dmx.stats.intmultinomial.IntegerMultinomialSampler attribute)`, 102
- `dist (dmx.stats.intrange.IntegerCategoricalSampler attribute)`, 39
- `dist (dmx.stats.intsetdist.IntegerBernoulliSetSampler attribute)`, 92
- `dist (dmx.stats.jmixture.JointMixtureSampler attribute)`, 167
- `dist (dmx.stats.log_gaussian.LogGaussianSampler attribute)`, 64
- `dist (dmx.stats.mixture.MixtureSampler attribute)`, 153
- `dist (dmx.stats.mvn.MultivariateGaussianSampler attribute)`, 75
- `dist (dmx.stats.null_dist.NullSampler attribute)`, 216
- `dist (dmx.stats.optional.OptionalDistribution attribute)`, 136
- `dist (dmx.stats.optional.OptionalSampler attribute)`, 141
- `dist (dmx.stats.pdist.DistributionSampler attribute)`, 13
- `dist (dmx.stats.poisson.PoissonSampler attribute)`, 34
- `dist (dmx.stats.sequence.SequenceDistribution attribute)`, 123
- `dist (dmx.stats.sequence.SequenceSampler attribute)`, 128
- `dist (dmx.stats.setdist.BernoulliSetSampler attribute)`, 96
- `dist (dmx.stats.spearman_rho.SpearmanRankingSampler attribute)`, 227
- `dist (dmx.stats.vmf.VonMisesFisherSampler attribute)`, 234
- `dist (dmx.stats.weighted.WeightedDistribution attribute)`, 239
- `dist (dmx.torch_stats.dmvn.DiagonalGaussianSampler attribute)`, 545
- `dist (dmx.torch_stats.gamma.GammaSampler attribute)`, 557
- `dist (dmx.torch_stats.gaussian.GaussianSampler attribute)`, 563
- `dist (dmx.torch_stats.geometric.GeometricSampler attribute)`, 569
- `dist (dmx.torch_stats.poisson.PoissonSampler attribute)`, 602
- `dist_sampler (dmx.stats.catmultinomial.MultinomialSampler attribute)`, 108
- `dist_sampler (dmx.stats.dirac_length.DiracMixtureSampler attribute)`, 196
- `dist_sampler (dmx.stats.ignored.IgnoredSampler attribute)`, 145
- `dist_sampler (dmx.stats.sequence.SequenceSampler attribute)`, 128
- `dist_samplers (dmx.stats.composite.CompositeSampler attribute)`, 122
- `dist_to_encoder() (dmx.bstats.bernoulli.BernoulliDistribution method)`, 408
- `dist_to_encoder() (dmx.bstats.categorical.CategoricalDistribution method)`, 414
- `dist_to_encoder() (dmx.bstats.composite.CompositeDistribution method)`, 469
- `dist_to_encoder() (dmx.bstats.conditional.ConditionalDistribution method)`, 476
- `dist_to_encoder() (dmx.bstats.dirac.DiracDistribution method)`, 422
- `dist_to_encoder() (dmx.bstats.dmvn.DiagonalGaussianDistribution method)`, 427
- `dist_to_encoder() (dmx.bstats.exponential.ExponentialDistribution method)`, 433
- `dist_to_encoder() (dmx.bstats.gaussian.GaussianDistribution method)`, 445
- `dist_to_encoder() (dmx.bstats.geometric.GeometricDistribution method)`, 451
- `dist_to_encoder() (dmx.bstats.ignored.IgnoredDistribution method)`, 489
- `dist_to_encoder() (dmx.bstats.intrange.IntegerCategoricalDistribution method)`, 457
- `dist_to_encoder() (dmx.bstats.mixture.MixtureDistribution method)`, 493
- `dist_to_encoder() (dmx.bstats.null_dist.NullDistribution method)`, 502
- `dist_to_encoder() (dmx.bstats.optional.OptionalDistribution method)`, 506
- `dist_to_encoder() (dmx.bstats.pdist.ProbabilityDistribution method)`, 376
- `dist_to_encoder() (dmx.bstats.poisson.PoissonDistribution method)`, 462
- `dist_to_encoder() (dmx.bstats.sequence.SequenceDistribution method)`, 513
- `dist_to_encoder() (dmx.bstats.setdist.BernoulliSetDistribution method)`, 521
- `dist_to_encoder() (dmx.stats.binomial.BinomialDistribution method)`, 21
- `dist_to_encoder() (dmx.stats.categorical.CategoricalDistribution method)`, 83
- `dist_to_encoder() (dmx.stats.catmultinomial.MultinomialDistribution method)`, 105
- `dist_to_encoder() (dmx.stats.composite.CompositeDistribution method)`, 119
- `dist_to_encoder() (dmx.stats.conditional.ConditionalDistribution method)`, 131
- `dist_to_encoder() (dmx.stats.dirac_length.DiracMixtureDistribution method)`, 192
- `dist_to_encoder() (dmx.stats.dirichlet.DirichletDistribution method)`, 77
- `dist_to_encoder() (dmx.stats.dmvn.DiagonalGaussianDistribution method)`, 66
- `dist_to_encoder() (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution method)`, 248
- `dist_to_encoder() (dmx.stats.exponential.ExponentialDistribution method)`, 51

`dist_to_encoder()` (`dmx.stats.gamma.GammaDistribution`
`method`), 56

`dist_to_encoder()` (`dmx.stats.gaussian.GaussianDistribution`
`method`), 46

`dist_to_encoder()` (`dmx.stats.geometric.GeometricDistribution`
`method`), 26

`dist_to_encoder()` (`dmx.stats.gmm.GaussianMixtureDistribution`
`method`), 261

`dist_to_encoder()` (`dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution`
`method`), 156

`dist_to_encoder()` (`dmx.stats.hidden_association.HiddenAssociationDistribution`
`method`), 298

`dist_to_encoder()` (`dmx.stats.hidden_markov.HiddenMarkovModelDistribution`
`method`), 179

`dist_to_encoder()` (`dmx.stats.hmixture.HierarchicalMixtureDistribution`
`method`), 170

`dist_to_encoder()` (`dmx.stats.ictree.ICLTreeDistribution`
`method`), 304

`dist_to_encoder()` (`dmx.stats.ignored.IgnoredDistribution`
`method`), 143

`dist_to_encoder()` (`dmx.stats.int_edit_setdist.IntegerBernoulliSetDistribution`
`method`), 201

`dist_to_encoder()` (`dmx.stats.int_edit_stepsetdist.IntegerBernoulliStepSetDistribution`
`method`), 208

`dist_to_encoder()` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution`
`method`), 312

`dist_to_encoder()` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution`
`method`), 328

`dist_to_encoder()` (`dmx.stats.int_markovchain.IntegerMarkovChainDistribution`
`method`), 341

`dist_to_encoder()` (`dmx.stats.int_plsi.IntegerPLSIDistribution`
`method`), 273

`dist_to_encoder()` (`dmx.stats.int_spike.SpikeAndSlabDistribution`
`method`), 41

`dist_to_encoder()` (`dmx.stats.intmultinomial.IntegerMultinomialDistribution`
`method`), 99

`dist_to_encoder()` (`dmx.stats.intrange.IntegerCategoricalDistribution`
`method`), 36

`dist_to_encoder()` (`dmx.stats.intsetdist.IntegerBernoulliSetDistribution`
`method`), 89

`dist_to_encoder()` (`dmx.stats.jmixture.JointMixtureDistribution`
`method`), 164

`dist_to_encoder()` (`dmx.stats.lda.LDADistribution`
`method`), 281

`dist_to_encoder()` (`dmx.stats.log_gaussian.LogGaussianDistribution`
`method`), 61

`dist_to_encoder()` (`dmx.stats.look_back_hmm.LookbackHiddenMarkovModelDistribution`
`method`), 346

`dist_to_encoder()` (`dmx.stats.markovchain.MarkovChainDistribution`
`method`), 112

`dist_to_encoder()` (`dmx.stats.mixture.MixtureDistribution`
`method`), 148

`dist_to_encoder()` (`dmx.stats.mvn.MultivariateGaussianDistribution`
`method`), 72

`dist_to_encoder()` (`dmx.stats.null_dist.NullDistribution`
`method`), 213

`dist_to_encoder()` (`dmx.stats.optional.OptionalDistribution`
`method`), 138

`dist_to_encoder()` (`dmx.stats.pdist.SequenceEncodableProbabilityDistribution`
`method`), 12

`dist_to_encoder()` (`dmx.stats.poisson.PoissonDistribution`
`method`), 31

`dist_to_encoder()` (`dmx.stats.select.SelectDistribution`
`method`), 217

`dist_to_encoder()` (`dmx.stats.sequence.SequenceDistribution`
`method`), 125

`dist_to_encoder()` (`dmx.stats.setdist.BernoulliSetDistribution`
`method`), 94

`dist_to_encoder()` (`dmx.stats.sparse_markov_transform.SparseMarkovTransformDistribution`
`method`), 357

`dist_to_encoder()` (`dmx.stats.spearman_rho.SpearmanRankingDistribution`
`method`), 225

`dist_to_encoder()` (`dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution`
`method`), 289

`dist_to_encoder()` (`dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution`
`method`), 364

`dist_to_encoder()` (`dmx.stats.vmf.VonMisesFisherDistribution`
`method`), 232

`dist_to_encoder()` (`dmx.stats.weighted.WeightedDistribution`
`method`), 240

`dist_to_encoder()` (`dmx.stats.binomial.BinomialDistribution`
`method`), 536

`dist_to_encoder()` (`dmx.torch_stats.composite.CompositeDistribution`
`method`), 606

`dist_to_encoder()` (`dmx.torch_stats.conditional.ConditionalDistribution`
`method`), 608

`dist_to_encoder()` (`dmx.torch_stats.dmvn.DiagonalGaussianDistribution`
`method`), 543

`dist_to_encoder()` (`dmx.torch_stats.exponential.ExponentialDistribution`
`method`), 549

`dist_to_encoder()` (`dmx.torch_stats.gamma.GammaDistribution`
`method`), 554

`dist_to_encoder()` (`dmx.torch_stats.gaussian.GaussianDistribution`
`method`), 561

`dist_to_encoder()` (`dmx.torch_stats.geometric.GeometricDistribution`
`method`), 566

`dist_to_encoder()` (`dmx.torch_stats.heterogeneous_mixture.HeterogeneousMixtureDistribution`
`method`), 625

`dist_to_encoder()` (`dmx.torch_stats.hmm.HiddenMarkovModelDistribution`
`method`), 633

`dist_to_encoder()` (`dmx.torch_stats.int_plsi.IntegerPLSIDistribution`
`method`), 640

`dist_to_encoder()` (`dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution`
`method`), 574

`dist_to_encoder()` (`dmx.torch_stats.intrange.IntegerCategoricalDistribution`
`method`), 582

`dist_to_encoder()` (`dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution`
`method`), 589

<code>dist_to_encoder()</code> (<i>dmx.torch_stats.jmixture.JointMixtureDistribution</i> module), 643	<code>dmx.bstats.geometric</code> module, 448
<code>dist_to_encoder()</code> (<i>dmx.torch_stats.mixture.MixtureDistribution</i> module), 649	<code>dmx.bstats.ignored</code> module, 487
<code>dist_to_encoder()</code> (<i>dmx.torch_stats.mvn.MultivariateGaussianDistribution</i> module), 595	<code>dmx.bstats.intrange</code> module, 454
<code>dist_to_encoder()</code> (<i>dmx.torch_stats.null_dist.NullDistribution</i> module), 615	<code>dmx.bstats.mixture</code> module, 492
<code>dist_to_encoder()</code> (<i>dmx.torch_stats.pdist.TorchProbabilityDistribution</i> module), 527	<code>dmx.bstats.mvngamma</code> module, 401
<code>dist_to_encoder()</code> (<i>dmx.torch_stats.poisson.PoissonDistribution</i> module), 600	<code>dmx.bstats.normgamma</code> module, 403
<code>dist_to_encoder()</code> (<i>dmx.torch_stats.sequence.SequenceDistribution</i> module), 620	<code>dmx.bstats.nulldist</code> module, 499
<code>DistributionSampler</code> (class in <i>dmx.bstats.pdist</i>), 373	<code>dmx.bstats.optional</code> module, 505
<code>DistributionSampler</code> (class in <i>dmx.stats.pdist</i>), 13	
<code>DistributionSampler</code> (class in <i>dmx.torch_stats.pdist</i>), 525	<code>dmx.bstats.pdist</code> module, 372
<code>dists</code> (<i>dmx.stats.composite.CompositeDistribution</i> attribute), 118	<code>dmx.bstats.poisson</code> module, 461
<code>dists</code> (<i>dmx.torch_stats.composite.CompositeDistribution</i> attribute), 605	<code>dmx.bstats.sequence</code> module, 512
<code>dmap</code> (<i>dmx.stats.conditional.ConditionalDistribution</i> attribute), 129	<code>dmx.bstats.setdist</code> module, 519
<code>dmx.bstats</code> module, 372	<code>dmx.bstats.symdirichlet</code> module, 405
<code>dmx.bstats.bernoulli</code> module, 407	<code>dmx.mpi4py.bstats</code> module, 706
<code>dmx.bstats.bestimation</code> module, 383	<code>dmx.mpi4py.stats</code> module, 703
<code>dmx.bstats.beta</code> module, 389	<code>dmx.mpi4py.utils</code> module, 708
<code>dmx.bstats.catdirichlet</code> module, 391	<code>dmx.mpi4py.utils.automatic</code> module, 709
<code>dmx.bstats.categorical</code> module, 413	<code>dmx.mpi4py.utils.bestimation</code> module, 710
<code>dmx.bstats.composite</code> module, 467	<code>dmx.mpi4py.utils.estimation</code> module, 711
<code>dmx.bstats.conditional</code> module, 474	<code>dmx.mpi4py.utils.humap</code> module, 714
<code>dmx.bstats.dirac</code> module, 419	<code>dmx.mpi4py.utils.optsutil</code> module, 715
<code>dmx.bstats.dirichlet</code> module, 394	<code>dmx.stats</code> module, 7
<code>dmx.bstats.dmvn</code> module, 425	<code>dmx.stats.dirac_length</code> module, 186
<code>dmx.bstats.dpm</code> module, 480	<code>dmx.stats.dmvn_mixture</code> module, 242
<code>dmx.bstats.exponential</code> module, 431	<code>dmx.stats.gmm</code> module, 254
<code>dmx.bstats.gamma</code> module, 436	<code>dmx.stats.hidden_association</code> module, 294
<code>dmx.bstats.gaussian</code> module, 442	<code>dmx.stats.icltree</code> module, 301

<code>dmx.stats.int_edit_setdist</code> module, 196	<code>dmx.torch_stats.hmm</code> module, 627
<code>dmx.stats.int_edit_stepsetdist</code> module, 205	<code>dmx.torch_stats.int_plsi</code> module, 637
<code>dmx.stats.int_hidden_association</code> module, 307	<code>dmx.torch_stats.intmultinomial</code> module, 569
<code>dmx.stats.int_hidden_markov</code> module, 318	<code>dmx.torch_stats.intrange</code> module, 578
<code>dmx.stats.int_markovchain</code> module, 334	<code>dmx.torch_stats.intsetdist</code> module, 585
<code>dmx.stats.int_plsi</code> module, 266	<code>dmx.torch_stats.jmixture</code> module, 642
<code>dmx.stats.lda</code> module, 279	<code>dmx.torch_stats.mixture</code> module, 647
<code>dmx.stats.look_back_hmm</code> module, 345	<code>dmx.torch_stats.mvn</code> module, 591
<code>dmx.stats.null_dist</code> module, 210	<code>dmx.torch_stats.null_dist</code> module, 613
<code>dmx.stats.rdd_sampler</code> module, 370	<code>dmx.torch_stats.pdist</code> module, 525
<code>dmx.stats.select</code> module, 216	<code>dmx.torch_stats.poisson</code> module, 597
<code>dmx.stats.sparse_markov_transform</code> module, 353	<code>dmx.torch_stats.sequence</code> module, 617
<code>dmx.stats.spearman_rho</code> module, 221	<code>dmx.torch_utils</code> module, 692
<code>dmx.stats.ss_mixture</code> module, 288	<code>dmx.torch_utils.estimation</code> module, 693
<code>dmx.stats.tree_hmm</code> module, 360	<code>dmx.torch_utils.optsutil</code> module, 696
<code>dmx.stats.vmf</code> module, 228	<code>dmx.torch_utils.vector</code> module, 697
<code>dmx.stats.weighted</code> module, 236	<code>dmx.utils</code> module, 652
<code>dmx.torch_stats</code> module, 525	<code>dmx.utils.automatic</code> module, 652
<code>dmx.torch_stats.binomial</code> module, 532	<code>dmx.utils.builder</code> module, 659
<code>dmx.torch_stats.composite</code> module, 603	<code>dmx.utils.estimation</code> module, 660
<code>dmx.torch_stats.conditional</code> module, 608	<code>dmx.utils.htsne</code> module, 667
<code>dmx.torch_stats.dmvn</code> module, 539	<code>dmx.utils.humap</code> module, 674
<code>dmx.torch_stats.exponential</code> module, 545	<code>dmx.utils.metrics</code> module, 675
<code>dmx.torch_stats.gamma</code> module, 551	<code>dmx.utils.optsutil</code> module, 677
<code>dmx.torch_stats.gaussian</code> module, 558	<code>dmx.utils.pvalues</code> module, 680
<code>dmx.torch_stats.geometric</code> module, 564	<code>dmx.utils.special</code> module, 681
<code>dmx.torch_stats.heterogenous_mixture</code> module, 622	<code>dmx.utils.vector</code> module, 684

- `doc_count` (*dmx.stats.int_plsi.IntegerPLSIAccumulator attribute*), 267
- `doc_vec` (*dmx.stats.int_plsi.IntegerPLSIDistribution attribute*), 271
- `dot()` (*in module dmx.utils.vector*), 685
- `dpm_sne()` (*in module dmx.utils.htsne*), 667
- ## E
- `empirical_kl_divergence()` (*in module dmx.bstats.bestimation*), 385
- `empirical_kl_divergence()` (*in module dmx.torch_utils.estimation*), 693
- `empirical_kl_divergence()` (*in module dmx.utils.estimation*), 661
- `encode_mixture_data()` (*in module dmx.utils.automatic*), 654
- `EncodedDataSequence` (*class in dmx.bstats.pdist*), 374
- `EncodedDataSequence` (*class in dmx.stats.pdist*), 19
- `encoder` (*dmx.stats.dirac_length.DiracMixtureDataEncoder attribute*), 190
- `encoder` (*dmx.stats.weighted.WeightedDataEncoder attribute*), 239
- `entropy()` (*dmx.bstats.bernoulli.BernoulliDistribution method*), 408
- `entropy()` (*dmx.bstats.beta.BetaDistribution method*), 390
- `entropy()` (*dmx.bstats.catdirichlet.DictDirichletDistribution method*), 392
- `entropy()` (*dmx.bstats.categorical.CategoricalDistribution method*), 414
- `entropy()` (*dmx.bstats.composite.CompositeDistribution method*), 469
- `entropy()` (*dmx.bstats.dirichlet.DirichletDistribution method*), 396
- `entropy()` (*dmx.bstats.gamma.GammaDistribution method*), 439
- `entropy()` (*dmx.bstats.geometric.GeometricDistribution method*), 451
- `entropy()` (*dmx.bstats.ignored.IgnoredDistribution method*), 489
- `entropy()` (*dmx.bstats.inrange.IntegerCategoricalDistribution method*), 457
- `entropy()` (*dmx.bstats.mvngamma.MultivariateNormalGammaDistribution method*), 402
- `entropy()` (*dmx.bstats.normgamma.NormalGammaDistribution method*), 404
- `entropy()` (*dmx.bstats.nulldist.NullDistribution method*), 502
- `entropy()` (*dmx.bstats.optional.OptionalDistribution method*), 506
- `entropy()` (*dmx.bstats.pdist.ProbabilityDistribution method*), 376
- `entropy()` (*dmx.bstats.poisson.PoissonDistribution method*), 462
- `entropy()` (*dmx.bstats.sequence.SequenceDistribution method*), 513
- `entropy()` (*dmx.bstats.symdirichlet.SymmetricDirichletDistribution method*), 406
- `est_prob` (*dmx.stats.optional.OptionalEstimator attribute*), 139
- `estimate()` (*dmx.bstats.bernoulli.BernoulliEstimator method*), 410
- `estimate()` (*dmx.bstats.categorical.CategoricalEstimator method*), 417
- `estimate()` (*dmx.bstats.composite.CompositeEstimator method*), 471
- `estimate()` (*dmx.bstats.conditional.ConditionalDistributionEstimator method*), 478
- `estimate()` (*dmx.bstats.dirac.DiracEstimator method*), 424
- `estimate()` (*dmx.bstats.dirichlet.DirichletEstimator method*), 399
- `estimate()` (*dmx.bstats.dmvn.DiagonalGaussianEstimator method*), 430
- `estimate()` (*dmx.bstats.dpm.DirichletProcessMixtureEstimator method*), 485
- `estimate()` (*dmx.bstats.exponential.ExponentialEstimator method*), 436
- `estimate()` (*dmx.bstats.gamma.GammaEstimator method*), 441
- `estimate()` (*dmx.bstats.gaussian.GaussianEstimator method*), 447
- `estimate()` (*dmx.bstats.geometric.GeometricEstimator method*), 453
- `estimate()` (*dmx.bstats.ignored.IgnoredEstimator method*), 491
- `estimate()` (*dmx.bstats.inrange.IntegerCategoricalEstimator method*), 460
- `estimate()` (*dmx.bstats.mixture.MixtureEstimator method*), 496
- `estimate()` (*dmx.bstats.nulldist.NullEstimator method*), 504
- `estimate()` (*dmx.bstats.optional.OptionalEstimator method*), 509
- `estimate()` (*dmx.bstats.pdist.ParameterEstimator method*), 374
- `estimate()` (*dmx.bstats.poisson.PoissonEstimator method*), 464
- `estimate()` (*dmx.bstats.sequence.SequenceEstimator method*), 515
- `estimate()` (*dmx.bstats.setdist.BernoulliSetEstimator method*), 523
- `estimate()` (*dmx.stats.binomial.BinomialEstimator method*), 24
- `estimate()` (*dmx.stats.categorical.CategoricalEstimator method*), 86
- `estimate()` (*dmx.stats.catmultinomial.MultinomialEstimator method*), 108

`estimate()` (`dmx.stats.composite.CompositeEstimator` method), 121
`estimate()` (`dmx.stats.conditional.ConditionalDistributionEstimator` method), 134
`estimate()` (`dmx.stats.dirac_length.DiracMixtureEstimator` method), 195
`estimate()` (`dmx.stats.dirichlet.DirichletEstimator` method), 80
`estimate()` (`dmx.stats.dmvn.DiagonalGaussianEstimator` method), 69
`estimate()` (`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureEstimator` method), 252
`estimate()` (`dmx.stats.exponential.ExponentialEstimator` method), 53
`estimate()` (`dmx.stats.gamma.GammaEstimator` method), 58
`estimate()` (`dmx.stats.gaussian.GaussianEstimator` method), 49
`estimate()` (`dmx.stats.geometric.GeometricEstimator` method), 28
`estimate()` (`dmx.stats.gmm.GaussianMixtureEstimator` method), 265
`estimate()` (`dmx.stats.heterogeneous_mixture.HeterogeneousMixtureEstimator` method), 160
`estimate()` (`dmx.stats.hidden_association.HiddenAssociationEstimator` method), 300
`estimate()` (`dmx.stats.hidden_markov.HiddenMarkovEstimator` method), 183
`estimate()` (`dmx.stats.hmixture.HierarchicalMixtureEstimator` method), 175
`estimate()` (`dmx.stats.icltree.ICLTreeEstimator` method), 306
`estimate()` (`dmx.stats.ignored.IgnoredEstimator` method), 145
`estimate()` (`dmx.stats.int_edit_setdist.IntegerBernoulliEditEstimator` method), 203
`estimate()` (`dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditEstimator` method), 209
`estimate()` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator` method), 315
`estimate()` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovEstimator` method), 325
`estimate()` (`dmx.stats.int_markovchain.IntegerMarkovChainEstimator` method), 344
`estimate()` (`dmx.stats.int_plsi.IntegerPLSEstimator` method), 275
`estimate()` (`dmx.stats.int_spike.SpikeAndSlabEstimator` method), 44
`estimate()` (`dmx.stats.intmultinomial.IntegerMultinomialEstimator` method), 102
`estimate()` (`dmx.stats.intrange.IntegerCategoricalEstimator` method), 38
`estimate()` (`dmx.stats.intsetdist.IntegerBernoulliSetEstimator` method), 91
`estimate()` (`dmx.stats.jmixture.JointMixtureEstimator` method), 166
`estimate()` (`dmx.stats.lda.LDAEstimator` method), 283
`estimate()` (`dmx.stats.log_gaussian.LogGaussianEstimator` method), 64
`estimate()` (`dmx.stats.look_back_hmm.LookbackHiddenMarkovEstimator` method), 348
`estimate()` (`dmx.stats.markovchain.MarkovChainEstimator` method), 115
`estimate()` (`dmx.stats.mixture.MixtureEstimator` method), 152
`estimate()` (`dmx.stats.mvn.MultivariateGaussianEstimator` method), 75
`estimate()` (`dmx.stats.null_dist.NullEstimator` method), 215
`estimate()` (`dmx.stats.optional.OptionalEstimator` method), 140
`estimate()` (`dmx.stats.pdist.ParameterEstimator` method), 18
`estimate()` (`dmx.stats.poisson.PoissonEstimator` method), 33
`estimate()` (`dmx.stats.select.SelectEstimator` method), 252
`estimate()` (`dmx.stats.sequence.SequenceEstimator` method), 127
`estimate()` (`dmx.stats.setdist.BernoulliSetEstimator` method), 96
`estimate()` (`dmx.stats.sparse_markov_transform.SparseMarkovAssociationEstimator` method), 359
`estimate()` (`dmx.stats.spearman_rho.SpearmanRankingEstimator` method), 227
`estimate()` (`dmx.stats.ss_mixture.SemiSupervisedMixtureEstimator` method), 291
`estimate()` (`dmx.stats.tree_hmm.TreeHiddenMarkovEstimator` method), 363
`estimate()` (`dmx.stats.vmf.VonMisesFisherEstimator` method), 234
`estimate()` (`dmx.stats.weighted.WeightedEstimator` method), 220
`estimate()` (`dmx.torch_stats.binomial.BinomialEstimator` method), 538
`estimate()` (`dmx.torch_stats.composite.CompositeEstimator` method), 607
`estimate()` (`dmx.torch_stats.conditional.ConditionalDistributionEstimator` method), 612
`estimate()` (`dmx.torch_stats.dmvn.DiagonalGaussianEstimator` method), 544
`estimate()` (`dmx.torch_stats.exponential.ExponentialEstimator` method), 550
`estimate()` (`dmx.torch_stats.gamma.GammaEstimator` method), 556
`estimate()` (`dmx.torch_stats.gaussian.GaussianEstimator` method), 562
`estimate()` (`dmx.torch_stats.geometric.GeometricEstimator`

method), 568
 estimate() (dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureEstimator
 method), 626
 estimate() (dmx.torch_stats.hmm.HiddenMarkovEstimator method), 631
 estimate() (dmx.torch_stats.int_plsi.IntegerPLSEstimator method), 641
 estimate() (dmx.torch_stats.intmultinomial.IntegerMultinomialEstimator method), 576
 estimate() (dmx.torch_stats.intrange.IntegerCategoricalEstimator method), 584
 estimate() (dmx.torch_stats.intsetdist.IntegerBernoulliSetEstimator method), 590
 estimate() (dmx.torch_stats.jmixture.JointMixtureEstimator method), 644
 estimate() (dmx.torch_stats.mixture.MixtureEstimator method), 651
 estimate() (dmx.torch_stats.mvn.MultivariateGaussianEstimator method), 597
 estimate() (dmx.torch_stats.null_dist.NullEstimator method), 616
 estimate() (dmx.torch_stats.pdist.TorchParameterEstimator method), 526
 estimate() (dmx.torch_stats.poisson.PoissonEstimator method), 602
 estimate() (dmx.torch_stats.sequence.SequenceEstimator method), 621
 estimate() (in module dmx.stats), 8
 estimate0() (dmx.stats.markovchain.MarkovChainEstimator method), 115
 estimate1() (dmx.stats.markovchain.MarkovChainEstimator method), 115
 estimate_shape() (dmx.bstats.gamma.GammaEstimator static method), 441
 estimate_shape() (dmx.stats.gamma.GammaEstimator static method), 59
 estimate_shape() (dmx.torch_stats.gamma.GammaEstimator static method), 556
 estimator (dmx.stats.catmultinomial.MultinomialEstimator attribute), 106
 estimator (dmx.stats.dirac_length.DiracMixtureEstimator attribute), 194
 estimator (dmx.stats.optional.OptionalEstimator attribute), 139
 estimator (dmx.stats.sequence.SequenceEstimator attribute), 126
 estimator (dmx.stats.weighted.WeightedEstimator attribute), 241
 estimator() (dmx.bstats.bernoulli.BernoulliDistribution method), 408
 estimator() (dmx.bstats.categorical.CategoricalDistribution method), 414
 estimator() (dmx.bstats.composite.CompositeDistribution method), 469
 estimator() (dmx.bstats conditional.ConditionalDistribution method), 476
 estimator() (dmx.bstats.dirac.DiracDistribution method), 422
 estimator() (dmx.bstats.dirichlet.DirichletDistribution method), 397
 estimator() (dmx.bstats.dmvn.DiagonalGaussianDistribution method), 428
 estimator() (dmx.bstats.dpm.DirichletProcessMixtureDistribution method), 483
 estimator() (dmx.bstats.exponential.ExponentialDistribution method), 433
 estimator() (dmx.bstats.gamma.GammaDistribution method), 439
 estimator() (dmx.bstats.gaussian.GaussianDistribution method), 445
 estimator() (dmx.bstats.geometric.GeometricDistribution method), 451
 estimator() (dmx.bstats.ignored.IgnoredDistribution method), 489
 estimator() (dmx.bstats.intrange.IntegerCategoricalDistribution method), 458
 estimator() (dmx.bstats.mixture.MixtureDistribution method), 493
 estimator() (dmx.bstats.nulldist.NullDistribution method), 502
 estimator() (dmx.bstats.optional.OptionalDistribution method), 506
 estimator() (dmx.bstats.pdist.ProbabilityDistribution method), 377
 estimator() (dmx.bstats.poisson.PoissonDistribution method), 462
 estimator() (dmx.bstats.sequence.SequenceDistribution method), 513
 estimator() (dmx.bstats.setdist.BernoulliSetDistribution method), 521
 estimator() (dmx.stats.binomial.BinomialDistribution method), 21
 estimator() (dmx.stats.categorical.CategoricalDistribution method), 83
 estimator() (dmx.stats.catmultinomial.MultinomialDistribution method), 105
 estimator() (dmx.stats.composite.CompositeDistribution method), 119
 estimator() (dmx.stats conditional.ConditionalDistribution method), 131
 estimator() (dmx.stats.dirac_length.DiracMixtureDistribution method), 192
 estimator() (dmx.stats.dirichlet.DirichletDistribution method), 77
 estimator() (dmx.stats.dmvn.DiagonalGaussianDistribution method), 67
 estimator() (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution method), 249

`estimator()` (`dmx.stats.exponential.ExponentialDistribution` method), 51
`estimator()` (`dmx.stats.gamma.GammaDistribution` method), 56
`estimator()` (`dmx.stats.gaussian.GaussianDistribution` method), 46
`estimator()` (`dmx.stats.geometric.GeometricDistribution` method), 26
`estimator()` (`dmx.stats.gmm.GaussianMixtureDistribution` method), 261
`estimator()` (`dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution` method), 156
`estimator()` (`dmx.stats.hidden_association.HiddenAssociationDistribution` method), 298
`estimator()` (`dmx.stats.hidden_markov.HiddenMarkovModelDistribution` method), 179
`estimator()` (`dmx.stats.hmixture.HierarchicalMixtureDistribution` method), 170
`estimator()` (`dmx.stats.ictree.ICLTreeDistribution` method), 305
`estimator()` (`dmx.stats.ignored.IgnoredDistribution` method), 143
`estimator()` (`dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution` method), 201
`estimator()` (`dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditDistribution` method), 208
`estimator()` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution` method), 312
`estimator()` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution` method), 328
`estimator()` (`dmx.stats.int_markovchain.IntegerMarkovChainDistribution` method), 341
`estimator()` (`dmx.stats.int_plsi.IntegerPLSIDistribution` method), 273
`estimator()` (`dmx.stats.int_spike.SpikeAndSlabDistribution` method), 41
`estimator()` (`dmx.stats.intmultinomial.IntegerMultinomialDistribution` method), 99
`estimator()` (`dmx.stats.intrange.IntegerCategoricalDistribution` method), 36
`estimator()` (`dmx.stats.intsetdist.IntegerBernoulliSetDistribution` method), 89
`estimator()` (`dmx.stats.jmixture.JointMixtureDistribution` method), 164
`estimator()` (`dmx.stats.lda.LDADistribution` method), 281
`estimator()` (`dmx.stats.log_gaussian.LogGaussianDistribution` method), 62
`estimator()` (`dmx.stats.look_back_hmm.LookbackHiddenMarkovModelDistribution` method), 347
`estimator()` (`dmx.stats.markovchain.MarkovChainDistribution` method), 112
`estimator()` (`dmx.stats.mixture.MixtureDistribution` method), 148
`estimator()` (`dmx.stats.mvn.MultivariateGaussianDistribution` method), 72
`estimator()` (`dmx.stats.null_dist.NullDistribution` method), 213
`estimator()` (`dmx.stats.optional.OptionalDistribution` method), 138
`estimator()` (`dmx.stats.pdist.ProbabilityDistribution` method), 12
`estimator()` (`dmx.stats.poisson.PoissonDistribution` method), 31
`estimator()` (`dmx.stats.select.SelectDistribution` method), 217
`estimator()` (`dmx.stats.sequence.SequenceDistribution` method), 125
`estimator()` (`dmx.stats.setdist.BernoulliSetDistribution` method), 94
`estimator()` (`dmx.stats.sparse_markov_transform.SparseMarkovAssociationDistribution` method), 357
`estimator()` (`dmx.stats.spearman_rho.SpearmanRankingDistribution` method), 225
`estimator()` (`dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution` method), 289
`estimator()` (`dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution` method), 365
`estimator()` (`dmx.stats.vmf.VonMisesFisherDistribution` method), 232
`estimator()` (`dmx.stats.weighted.WeightedDistribution` method), 240
`estimator()` (`dmx.stats.binomial.BinomialDistribution` method), 536
`estimator()` (`dmx.torch_stats.composite.CompositeDistribution` method), 606
`estimator()` (`dmx.torch_stats.conditional.ConditionalDistribution` method), 608
`estimator()` (`dmx.torch_stats.dmvn.DiagonalGaussianDistribution` method), 543
`estimator()` (`dmx.torch_stats.exponential.ExponentialDistribution` method), 549
`estimator()` (`dmx.torch_stats.gamma.GammaDistribution` method), 554
`estimator()` (`dmx.torch_stats.gaussian.GaussianDistribution` method), 561
`estimator()` (`dmx.torch_stats.geometric.GeometricDistribution` method), 567
`estimator()` (`dmx.torch_stats.heterogeneous_mixture.HeterogeneousMixtureDistribution` method), 625
`estimator()` (`dmx.torch_stats.hmm.HiddenMarkovModelDistribution` method), 633
`estimator()` (`dmx.torch_stats.int_plsi.IntegerPLSIDistribution` method), 640
`estimator()` (`dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution` method), 574
`estimator()` (`dmx.torch_stats.intrange.IntegerCategoricalDistribution` method), 582

estimator() (dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution method), 434
 method), 589 expected_log_density()
 estimator() (dmx.torch_stats.jmixture.JointMixtureDistribution (dmx.bstats.gamma.GammaDistribution method), 643 method), 439
 estimator() (dmx.torch_stats.mixture.MixtureDistribution expected_log_density() (dmx.bstats.gaussian.GaussianDistribution method), 649 method), 445
 estimator() (dmx.torch_stats.mvn.MultivariateGaussianDistribution method), 445
 method), 595 expected_log_density()
 estimator() (dmx.torch_stats.null_dist.NullDistribution (dmx.bstats.geometric.GeometricDistribution method), 615 method), 451
 estimator() (dmx.torch_stats.pdist.TorchProbabilityDistribution expected_log_density() (dmx.bstats.ignored.IgnoredDistribution method), 527 method), 490
 estimator() (dmx.torch_stats.poisson.PoissonDistribution method), 490
 method), 600 expected_log_density()
 estimator() (dmx.torch_stats.sequence.SequenceDistribution (dmx.bstats.inrange.IntegerCategoricalDistribution method), 620 method), 458
 estimator_map (dmx.stats.conditional.ConditionalDistribution expected_log_density() (dmx.bstats.mixture.MixtureDistribution attribute), 132 method), 493
 estimators (dmx.stats.composite.CompositeEstimator method), 121 expected_log_density()
 estimators (dmx.stats.heterogeneous_mixture.HeterogeneousMixtureEstimator (dmx.bstats.nulldist.NullDistribution method), attribute), 158 502
 estimators (dmx.stats.hidden_markov.HiddenMarkovEstimator expected_log_density() (dmx.bstats.optional.OptionalDistribution attribute), 182 method), 506
 estimators (dmx.stats.hmixture.HierarchicalMixtureEstimator expected_log_density() (dmx.bstats.pdist.ProbabilityDistribution attribute), 173 method), 377
 estimators (dmx.stats.mixture.MixtureEstimator (dmx.bstats.poisson.PoissonDistribution attribute), 151 method), 462
 estimators (dmx.stats.ss_mixture.SemiSupervisedMixtureEstimator expected_log_density() (dmx.bstats.poisson.PoissonDistribution attribute), 291 method), 462
 estimators1 (dmx.stats.jmixture.JointMixtureEstimator expected_log_density() (dmx.bstats.sequence.SequenceDistribution attribute), 165 method), 513
 estimators2 (dmx.stats.jmixture.JointMixtureEstimator (dmx.bstats.sequence.SequenceDistribution attribute), 165 method), 513
 expected_log_density() ExponentialAccumulator (class in (dmx.bstats.bernoulli.BernoulliDistribution dmx.bstats.exponential), 431 method), 408 ExponentialAccumulator (class in dmx.torch_stats.exponential), 545
 expected_log_density() ExponentialAccumulatorFactory (class in (dmx.bstats.categorical.CategoricalDistribution dmx.bstats.exponential), 433 method), 415 ExponentialAccumulatorFactory (class in dmx.torch_stats.exponential), 547
 expected_log_density() ExponentialDataEncoder (class in (dmx.bstats.composite.CompositeDistribution dmx.torch_stats.exponential), 547 method), 469 ExponentialDataEncoder (class in dmx.bstats.exponential), 433
 expected_log_density() ExponentialDataEncoder (class in (dmx.bstats.dirac.DiracDistribution method), 422 dmx.torch_stats.exponential), 548
 expected_log_density() ExponentialDistribution (class in (dmx.bstats.dmvn.DiagonalGaussianDistribution dmx.bstats.exponential), 433 method), 428 ExponentialDistribution (class in dmx.stats.exponential), 50
 expected_log_density() ExponentialDistribution (class in (dmx.bstats.dpm.DirichletProcessMixtureDistribution dmx.torch_stats.exponential), 548 method), 483 ExponentialDistribution (class in dmx.torch_stats.exponential), 548
 expected_log_density() ExponentialEncodedData (class in (dmx.bstats.exponential.ExponentialDistribution dmx.bstats.exponential), 435 method), 483

ExponentialEstimator (class in <i>dmx.bstats.exponential</i>), 435	in float_dtype_for_device() (in module <i>dmx.torch_utils.vector</i>), 698
ExponentialEstimator (class in <i>dmx.stats.exponential</i>), 52	in forward_backward() (in module <i>dmx.stats.int_hidden_markov</i>), 332
ExponentialEstimator (class in <i>dmx.torch_stats.exponential</i>), 550	in from_value() (<i>dmx.bstats.bernoulli.BernoulliEstimatorAccumulator</i> method), 411
ExponentialSampler (class in <i>dmx.bstats.exponential</i>), 436	in from_value() (<i>dmx.bstats.categorical.CategoricalEstimatorAccumulator</i> method), 417
ExponentialSampler (class in <i>dmx.stats.exponential</i>), 54	in from_value() (<i>dmx.bstats.composite.CompositeEstimatorAccumulator</i> method), 473
ExponentialSampler (class in <i>dmx.torch_stats.exponential</i>), 550	in from_value() (<i>dmx.bstats.conditional.ConditionalDistributionEstimatorAccumulator</i> method), 478
ExponentialTorchEncodedSequence (class in <i>dmx.torch_stats.exponential</i>), 551	in from_value() (<i>dmx.bstats.dirac.DiracAccumulator</i> method), 420
F	from_value() (<i>dmx.bstats.dirichlet.DirichletAccumulator</i> method), 394
factory (<i>dmx.stats.dirac_length.DiracMixtureAccumulatorFactory</i> attribute), 189	from_value() (<i>dmx.bstats.dmvn.DiagonalGaussianAccumulator</i> method), 425
factory (<i>dmx.stats.weighted.WeightedAccumulatorFactory</i> attribute), 238	from_value() (<i>dmx.bstats.dpm.DirichletProcessMixtureAccumulator</i> method), 481
fast_log_density() (in module <i>dmx.stats.int_hidden_markov</i>), 331	from_value() (<i>dmx.bstats.exponential.ExponentialAccumulator</i> method), 431
fast_seq_component_log_density() (in module <i>dmx.stats.int_plsi</i>), 277	from_value() (<i>dmx.bstats.gamma.GammaAccumulator</i> method), 437
fast_seq_initialize() (in module <i>dmx.stats.int_hidden_markov</i>), 332	from_value() (<i>dmx.bstats.gaussian.GaussianAccumulator</i> method), 443
fast_seq_log_density() (in module <i>dmx.stats.int_plsi</i>), 277	from_value() (<i>dmx.bstats.geometric.GeometricAccumulator</i> method), 449
fast_seq_posterior() (<i>dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution</i> method), 249	from_value() (<i>dmx.bstats.ignored.IgnoredAccumulator</i> method), 487
fast_seq_posterior() (in module <i>dmx.stats.dmvn_mixture</i>), 253	from_value() (<i>dmx.bstats.intrange.IntegerCategoricalAccumulator</i> method), 455
fast_seq_update() (in module <i>dmx.stats.int_plsi</i>), 277	from_value() (<i>dmx.bstats.mixture.MixtureEstimatorAccumulator</i> method), 497
feature_order (<i>dmx.stats.icltree.ICLTreeDistribution</i> attribute), 304	from_value() (<i>dmx.bstats.nulldist.NullAccumulator</i> method), 500
find_alpha() (in module <i>dmx.bstats.dirichlet</i>), 400	from_value() (<i>dmx.bstats.optional.OptionalEstimatorAccumulator</i> method), 510
find_alpha() (in module <i>dmx.stats.lda</i>), 286	from_value() (<i>dmx.bstats.pdist.StatisticAccumulator</i> method), 382
find_level() (in module <i>dmx.stats.tree_hmm</i>), 366	from_value() (<i>dmx.bstats.poisson.PoissonEstimatorAccumulator</i> method), 465
fix_row_perplexity() (in module <i>dmx.utils.htsne</i>), 668	from_value() (<i>dmx.bstats.sequence.SequenceEstimatorAccumulator</i> method), 516
fixed_p_vec (<i>dmx.stats.dirac_length.DiracMixtureEstimator</i> attribute), 195	from_value() (<i>dmx.bstats.setdist.BernoulliSetAccumulator</i> method), 519
fixed_weights (<i>dmx.stats.dmvn_mixture.DiagonalGaussianMixtureEstimator</i> attribute), 252	from_value() (<i>dmx.stats.dirac_length.DiracMixtureAccumulator</i> method), 187
fixed_weights (<i>dmx.stats.gmm.GaussianMixtureEstimator</i> attribute), 264	from_value() (<i>dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator</i> method), 243
fixed_weights (<i>dmx.stats.heterogeneous_mixture.HeterogeneousMixtureEstimator</i> attribute), 159	from_value() (<i>dmx.stats.gmm.GaussianMixtureAccumulator</i> method), 256
fixed_weights (<i>dmx.stats.mixture.MixtureEstimator</i> attribute), 151	from_value() (<i>dmx.stats.hidden_association.HiddenAssociationAccumulator</i> method), 295
flat_map() (in module <i>dmx.utils.optsutil</i>), 677	
float_count (<i>dmx.utils.automatic.DatumNode</i> attribute), 653	

from_value() (dmx.stats.icltree.ICLTreeAccumulator method), 301
 from_value() (dmx.stats.int_edit_setdist.IntegerBernoulliSetAccumulator method), 197
 from_value() (dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliSetAccumulator method), 205
 from_value() (dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator method), 308
 from_value() (dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator method), 320
 from_value() (dmx.stats.int_markovchain.IntegerMarkovChainAccumulator method), 336
 from_value() (dmx.stats.int_plsi.IntegerPLSIAccumulator method), 268
 from_value() (dmx.stats.lda.LDAEstimatorAccumulator method), 284
 from_value() (dmx.stats.look_back_hmm.LookbackHiddenMarkovAccumulator method), 349
 from_value() (dmx.stats.null_dist.NullAccumulator method), 211
 from_value() (dmx.stats.pdist.StatisticAccumulator method), 15
 from_value() (dmx.stats.select.SelectEstimatorAccumulator method), 219
 from_value() (dmx.stats.sparse_markov_transform.SparseMarkovAssociationAccumulator method), 354
 from_value() (dmx.stats.spearman_rho.SpearmanRankingAccumulator method), 222
 from_value() (dmx.stats.ss_mixture.SemiSupervisedMixtureEstimatorAccumulator method), 292
 from_value() (dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator method), 360
 from_value() (dmx.stats.vmf.VonMisesFisherAccumulator method), 229
 from_value() (dmx.stats.weighted.WeightedAccumulator method), 237
 from_value() (dmx.torch_stats.binomial.BinomialAccumulator method), 533
 from_value() (dmx.torch_stats.composite.CompositeAccumulator method), 603
 from_value() (dmx.torch_stats.conditional.ConditionalDistributionAccumulator method), 610
 from_value() (dmx.torch_stats.dmvn.DiagonalGaussianAccumulator method), 541
 from_value() (dmx.torch_stats.exponential.ExponentialAccumulator method), 546
 from_value() (dmx.torch_stats.gamma.GammaAccumulator method), 552
 from_value() (dmx.torch_stats.gaussian.GaussianAccumulator method), 559
 from_value() (dmx.torch_stats.geometric.GeometricAccumulator method), 564
 from_value() (dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulator method), 623
 from_value() (dmx.torch_stats.hmm.HiddenMarkovAccumulator method), 628
 from_value() (dmx.torch_stats.int_plsi.IntegerPLSIAccumulator method), 637
 from_value() (dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator method), 570
 from_value() (dmx.torch_stats.intrange.IntegerCategoricalAccumulator method), 579
 from_value() (dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulator method), 586
 from_value() (dmx.torch_stats.jmixture.JointMixtureEstimatorAccumulator method), 645
 from_value() (dmx.torch_stats.mixture.MixtureAccumulator method), 647
 from_value() (dmx.torch_stats.mvn.MultivariateGaussianAccumulator method), 592
 from_value() (dmx.torch_stats.null_dist.NullAccumulator method), 613
 from_value() (dmx.torch_stats.pdist.TorchStatisticAccumulator method), 530
 from_value() (dmx.torch_stats.poisson.PoissonAccumulator method), 598
 from_value() (dmx.torch_stats.sequence.SequenceAccumulator method), 618

G

gamma() (in module dmx.utils.special), 682
 gamma_threshold (dmx.stats.lda.LDADistribution attribute), 284
 GammaAccumulator (class in dmx.bstats.gamma), 436
 GammaAccumulator (class in dmx.torch_stats.gamma), 551
 GammaAccumulatorFactory (class in dmx.bstats.gamma), 438
 GammaAccumulatorFactory (class in dmx.torch_stats.gamma), 553
 GammaDataEncoder (class in dmx.torch_stats.gamma), 553
 GammaDistribution (class in dmx.bstats.gamma), 438
 GammaDistribution (class in dmx.stats.gamma), 55
 GammaDistribution (class in dmx.torch_stats.gamma), 554
 GammaEstimator (class in dmx.bstats.gamma), 440
 GammaEstimator (class in dmx.stats.gamma), 57
 GammaEstimator (class in dmx.torch_stats.gamma), 555
 gammaln() (in module dmx.utils.special), 682
 gammaln() (in module dmx.utils.vector), 685
 GammaSampler (class in dmx.bstats.gamma), 442
 GammaSampler (class in dmx.stats.gamma), 59
 GammaSampler (class in dmx.torch_stats.gamma), 557
 GammaTorchEncodedSequence (class in dmx.torch_stats.gamma), 558
 GaussianAccumulator (class in dmx.bstats.gaussian), 442

GaussianAccumulator	(class in <i>dmx.torch_stats.gaussian</i>), 558	in	GeometricDistribution	(class in <i>dmx.bstats.geometric</i>), 451	in
GaussianAccumulatorFactory	(class in <i>dmx.torch_stats.gaussian</i>), 560	in	GeometricDistribution	(class in <i>dmx.stats.geometric</i>), 25	in
GaussianDataEncoder	(class in <i>dmx.bstats.gaussian</i>), 445		GeometricDistribution	(class in <i>dmx.torch_stats.geometric</i>), 566	in
GaussianDataEncoder	(class in <i>dmx.torch_stats.gaussian</i>), 560	in	GeometricEncodedData	(class in <i>dmx.bstats.geometric</i>), 453	in
GaussianDistribution	(class in <i>dmx.bstats.gaussian</i>), 445		GeometricEstimator	(class in <i>dmx.bstats.geometric</i>), 453	
GaussianDistribution	(class in <i>dmx.stats.gaussian</i>), 45		GeometricEstimator	(class in <i>dmx.stats.geometric</i>), 27	
GaussianDistribution	(class in <i>dmx.torch_stats.gaussian</i>), 560	in	GeometricEstimator	(class in <i>dmx.torch_stats.geometric</i>), 567	in
GaussianEncodedData	(class in <i>dmx.bstats.gaussian</i>), 447		GeometricSampler	(class in <i>dmx.bstats.geometric</i>), 454	
GaussianEstimator	(class in <i>dmx.bstats.gaussian</i>), 447		GeometricSampler	(class in <i>dmx.stats.geometric</i>), 29	
GaussianEstimator	(class in <i>dmx.stats.gaussian</i>), 47		GeometricSampler	(class in <i>dmx.torch_stats.geometric</i>), 568	in
GaussianEstimator	(class in <i>dmx.torch_stats.gaussian</i>), 562	in	GeometricTorchEncodedSequence	(class in <i>dmx.torch_stats.geometric</i>), 569	in
GaussianEstimatorAccumulatorFactory	(class in <i>dmx.bstats.gaussian</i>), 448	in	get_categorical_estimator()	(in module <i>dmx.utils.automatic</i>), 655	
GaussianMixtureAccumulator	(class in <i>dmx.stats.gmm</i>), 254	in	get_composite_estimator()	(in module <i>dmx.utils.automatic</i>), 655	
GaussianMixtureAccumulatorFactory	(class in <i>dmx.stats.gmm</i>), 258	in	get_data_type()	(<i>dmx.bstats.bernoulli.BernoulliDistribution</i> method), 409	
GaussianMixtureDataEncoder	(class in <i>dmx.stats.gmm</i>), 259	in	get_data_type()	(<i>dmx.bstats.intrange.IntegerCategoricalDistribution</i> method), 458	
GaussianMixtureDistribution	(class in <i>dmx.stats.gmm</i>), 259	in	get_data_type()	(<i>dmx.bstats.optional.OptionalDistribution</i> method), 506	
GaussianMixtureEncodedDataSequence	(class in <i>dmx.stats.gmm</i>), 263	in	get_data_type()	(<i>dmx.bstats.poisson.PoissonDistribution</i> method), 462	
GaussianMixtureEstimator	(class in <i>dmx.stats.gmm</i>), 263		get_dpm_mixture()	(in module <i>dmx.utils.automatic</i>), 655	
GaussianMixtureSampler	(class in <i>dmx.stats.gmm</i>), 265		get_dpm_mixture_mpi()	(in module <i>dmx.mpi4py.utils.automatic</i>), 709	
GaussianSampler	(class in <i>dmx.bstats.gaussian</i>), 448		get_estimator()	(<i>dmx.utils.automatic.DatumNode</i> method), 653	
GaussianSampler	(class in <i>dmx.stats.gaussian</i>), 49		get_estimator()	(in module <i>dmx.utils.automatic</i>), 656	
GaussianSampler	(class in <i>dmx.torch_stats.gaussian</i>), 563		get_gaussian_estimator()	(in module <i>dmx.utils.automatic</i>), 656	
GaussianTorchEncodedSequence	(class in <i>dmx.torch_stats.gaussian</i>), 563	in	get_ignored_estimator()	(in module <i>dmx.utils.automatic</i>), 657	
GeometricAccumulator	(class in <i>dmx.bstats.geometric</i>), 448	in	get_indexed_rdd_pne()	(in module <i>dmx.utils.builder</i>), 659	
GeometricAccumulator	(class in <i>dmx.torch_stats.geometric</i>), 564	in	get_inv_map()	(in module <i>dmx.utils.optsutil</i>), 677	
GeometricAccumulatorFactory	(class in <i>dmx.bstats.geometric</i>), 450	in	get_name()	(<i>dmx.bstats.pdist.ProbabilityDistribution</i> method), 377	
GeometricAccumulatorFactory	(class in <i>dmx.torch_stats.geometric</i>), 565	in	get_optional_estimator()	(in module <i>dmx.utils.automatic</i>), 657	
GeometricDataEncoder	(class in <i>dmx.bstats.geometric</i>), 451	in	get_parameters()	(<i>dmx.bstats.bernoulli.BernoulliDistribution</i> method), 409	
GeometricDataEncoder	(class in <i>dmx.torch_stats.geometric</i>), 566	in	get_parameters()	(<i>dmx.bstats.beta.BetaDistribution</i> method), 390	
			get_parameters()	(<i>dmx.bstats.catdirichlet.DictDirichletDistribution</i> method), 390	

method), 392

get_parameters() (dmx.bstats.categorical.CategoricalDistribution method), 415

get_parameters() (dmx.bstats.composite.CompositeDistribution method), 469

get_parameters() (dmx.bstats.dirac.DiracDistribution method), 422

get_parameters() (dmx.bstats.dirichlet.DirichletDistribution method), 397

get_parameters() (dmx.bstats.dmvn.DiagonalGaussianDistribution method), 428

get_parameters() (dmx.bstats.dpm.DirichletProcessMixtureDistribution method), 484

get_parameters() (dmx.bstats.exponential.ExponentialDistribution method), 434

get_parameters() (dmx.bstats.gamma.GammaDistribution method), 439

get_parameters() (dmx.bstats.gaussian.GaussianDistribution method), 445

get_parameters() (dmx.bstats.geometric.GeometricDistribution method), 452

get_parameters() (dmx.bstats.ignored.IgnoredDistribution method), 490

get_parameters() (dmx.bstats.intrange.IntegerCategoricalDistribution method), 458

get_parameters() (dmx.bstats.mixture.MixtureDistribution method), 493

get_parameters() (dmx.bstats.mvngamma.MultivariateNormalGammaDistribution method), 402

get_parameters() (dmx.bstats.normgamma.NormalGammaDistribution method), 404

get_parameters() (dmx.bstats.nulldist.NullDistribution method), 502

get_parameters() (dmx.bstats.optional.OptionalDistribution method), 507

get_parameters() (dmx.bstats.pdist.ProbabilityDistribution method), 377

get_parameters() (dmx.bstats.poisson.PoissonDistribution method), 462

get_parameters() (dmx.bstats.sequence.SequenceDistribution method), 513

get_parameters() (dmx.bstats.setdist.BernoulliSetDistribution method), 522

get_parameters() (dmx.bstats.symdirichlet.SymmetricDirichletDistribution method), 406

get_parent_directory() (in module dmx.utils.optsutil), 677

get_pmat() (in module dmx.utils.htsne), 668

get_pmat_vlen() (in module dmx.utils.htsne), 669

get_poisson_estimator() (in module dmx.utils.automatic), 658

get_prior() (dmx.bstats.bernoulli.BernoulliDistribution method), 409

get_prior() (dmx.bstats.bernoulli.BernoulliEstimator method), 411

get_prior() (dmx.bstats.categorical.CategoricalDistribution method), 415

get_prior() (dmx.bstats.categorical.CategoricalEstimator method), 417

get_prior() (dmx.bstats.composite.CompositeDistribution method), 469

get_prior() (dmx.bstats.composite.CompositeEstimator method), 471

get_prior() (dmx.bstats.dirac.DiracDistribution method), 422

get_prior() (dmx.bstats.dirac.DiracEstimator method), 424

get_prior() (dmx.bstats.dmvn.DiagonalGaussianDistribution method), 428

get_prior() (dmx.bstats.dmvn.DiagonalGaussianEstimator method), 430

get_prior() (dmx.bstats.dpm.DirichletProcessMixtureDistribution method), 484

get_prior() (dmx.bstats.dpm.DirichletProcessMixtureEstimator method), 486

get_prior() (dmx.bstats.exponential.ExponentialDistribution method), 434

get_prior() (dmx.bstats.exponential.ExponentialEstimator method), 436

get_prior() (dmx.bstats.gamma.GammaEstimator method), 442

get_prior() (dmx.bstats.gaussian.GaussianDistribution method), 446

get_prior() (dmx.bstats.gaussian.GaussianEstimator method), 447

get_prior() (dmx.bstats.geometric.GeometricDistribution method), 452

get_prior() (dmx.bstats.geometric.GeometricEstimator method), 454

get_prior() (dmx.bstats.ignored.IgnoredDistribution method), 490

get_prior() (dmx.bstats.ignored.IgnoredEstimator method), 492

get_prior() (dmx.bstats.intrange.IntegerCategoricalDistribution method), 458

get_prior() (dmx.bstats.intrange.IntegerCategoricalEstimator method), 460

get_prior() (dmx.bstats.mixture.MixtureDistribution method), 494

get_prior() (dmx.bstats.mixture.MixtureEstimator method), 496

get_prior() (dmx.bstats.nulldist.NullDistribution method), 502

get_prior() (dmx.bstats.nulldist.NullEstimator method), 504

get_prior() (dmx.bstats.optional.OptionalDistribution method), 507

get_prior() (dmx.bstats.optional.OptionalEstimator method), 509

method), 509

get_prior() (dmx.bstats.pdist.ParameterEstimator method), 375

get_prior() (dmx.bstats.pdist.ProbabilityDistribution method), 377

get_prior() (dmx.bstats.poisson.PoissonDistribution method), 462

get_prior() (dmx.bstats.poisson.PoissonEstimator method), 465

get_prior() (dmx.bstats.sequence.SequenceDistribution method), 513

get_prior() (dmx.bstats.sequence.SequenceEstimator method), 515

get_prior() (dmx.bstats.setdist.BernoulliSetDistribution method), 522

get_prior() (dmx.bstats.setdist.BernoulliSetEstimator method), 523

get_runtime_attr() (in module dmx.mpi4py.utils), 708

get_seq_lambda() (dmx.bstats.pdist.SequenceEncodableAccumulator method), 380

get_seq_lambda() (dmx.stats.pdist.SequenceEncodableStatisticAccumulator method), 17

get_sequence_estimator() (in module dmx.utils.automatic), 658

given_dist (dmx.stats.conditional.ConditionalDistribution attribute), 130

given_dist (dmx.stats.hidden_association.HiddenAssociationDistribution attribute), 298

given_estimator (dmx.stats.conditional.ConditionalDistributionEstimator attribute), 133

given_estimator (dmx.stats.hidden_association.HiddenAssociationEstimator attribute), 299

given_sampler (dmx.stats.conditional.ConditionalDistributionSampler attribute), 134

group_by() (in module dmx.utils.optsutil), 678

group_by_key() (in module dmx.utils.optsutil), 678

H

has_default (dmx.stats.conditional.ConditionalDistribution attribute), 130

has_default_sampler (dmx.stats.conditional.ConditionalDistributionSampler attribute), 134

has_given (dmx.stats.conditional.ConditionalDistribution attribute), 130

has_given_sampler (dmx.stats.conditional.ConditionalDistributionSampler attribute), 135

has_invalid (dmx.stats.dirichlet.DirichletDistribution attribute), 76

has_p (dmx.stats.optional.OptionalDistribution attribute), 136

has_prev_dist (dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution attribute), 311

has_topics (dmx.stats.hidden_markov.HiddenMarkovModelDistribution attribute), 177

has_topics (dmx.torch_stats.hmm.HiddenMarkovModelDistribution attribute), 633

HeterogeneousMixtureAccumulator (class in dmx.torch_stats.heterogenous_mixture), 622

HeterogeneousMixtureAccumulatorFactory (class in dmx.torch_stats.heterogenous_mixture), 624

HeterogeneousMixtureDataEncoder (class in dmx.torch_stats.heterogenous_mixture), 624

HeterogeneousMixtureDistribution (class in dmx.stats.heterogeneous_mixture), 154

HeterogeneousMixtureDistribution (class in dmx.torch_stats.heterogenous_mixture), 624

HeterogeneousMixtureEstimator (class in dmx.stats.heterogeneous_mixture), 158

HeterogeneousMixtureEstimator (class in dmx.torch_stats.heterogenous_mixture), 626

HeterogeneousMixtureSampler (class in dmx.stats.heterogeneous_mixture), 160

HeterogeneousMixtureSampler (class in dmx.torch_stats.heterogenous_mixture), 627

HeterogeneousMixtureTorchSequence (class in dmx.torch_stats.heterogenous_mixture), 627

HiddenAssociationAccumulator (class in dmx.stats.hidden_association), 294

HiddenAssociationAccumulatorFactory (class in dmx.stats.hidden_association), 296

HiddenAssociationDataEncoder (class in dmx.stats.hidden_association), 297

HiddenAssociationDistribution (class in dmx.stats.hidden_association), 297

HiddenAssociationEncodedDataSequence (class in dmx.stats.hidden_association), 299

HiddenAssociationEstimator (class in dmx.stats.hidden_association), 299

HiddenAssociationSampler (class in dmx.stats.hidden_association), 300

HiddenMarkovAccumulator (class in dmx.torch_stats.hmm), 627

HiddenMarkovAccumulatorFactory (class in dmx.torch_stats.hmm), 630

HiddenMarkovDataEncoder (class in dmx.torch_stats.hmm), 630

HiddenMarkovEstimator (class in dmx.stats.hidden_markov), 181

HiddenMarkovEstimator (class in dmx.torch_stats.hmm), 631

HiddenMarkovModelDistribution (class in dmx.stats.hidden_markov), 176

HiddenMarkovModelDistribution (class in dmx.torch_stats.hmm), 632

HiddenMarkovSampler (class in dmx.stats.hidden_markov), 184

HiddenMarkovSampler (class in *dmx.torch_stats.hmm*), 635
 HiddenMarkovTorchSequence (class in *dmx.torch_stats.hmm*), 636
 HierarchicalMixtureDistribution (class in *dmx.stats.hmixture*), 168
 HierarchicalMixtureEstimator (class in *dmx.stats.hmixture*), 172
 HierarchicalMixtureSampler (class in *dmx.stats.hmixture*), 175
 hill_climb() (in module *dmx.bstats.bestimation*), 385
 hill_climb() (in module *dmx.utils.estimation*), 662
 htsne() (in module *dmx.utils.htsne*), 669
 humap() (in module *dmx.utils.humap*), 674
 humap_mpi() (in module *dmx.mpi4py.utils.humap*), 714
 |
 ICLTreeAccumulator (class in *dmx.stats.icltree*), 301
 ICLTreeAccumulatorFactory (class in *dmx.stats.icltree*), 303
 ICLTreeDataEncoder (class in *dmx.stats.icltree*), 303
 ICLTreeDistribution (class in *dmx.stats.icltree*), 303
 ICLTreeEncodedDataSequence (class in *dmx.stats.icltree*), 305
 ICLTreeEstimator (class in *dmx.stats.icltree*), 305
 ICLTreeSampler (class in *dmx.stats.icltree*), 306
 IgnoredAccumulator (class in *dmx.bstats.ignored*), 487
 IgnoredAccumulatorFactory (class in *dmx.bstats.ignored*), 488
 IgnoredDataEncoder (class in *dmx.bstats.ignored*), 489
 IgnoredDistribution (class in *dmx.bstats.ignored*), 489
 IgnoredDistribution (class in *dmx.stats.ignored*), 142
 IgnoredEncodedData (class in *dmx.bstats.ignored*), 491
 IgnoredEstimator (class in *dmx.bstats.ignored*), 491
 IgnoredEstimator (class in *dmx.stats.ignored*), 144
 IgnoredSampler (class in *dmx.bstats.ignored*), 492
 IgnoredSampler (class in *dmx.stats.ignored*), 145
 index_dot() (in module *dmx.stats.int_plsi*), 278
 inf_count (*dmx.utils.automatic.DatumNode* attribute), 653
 init_acc (*dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator* attribute), 197
 init_accumulator (*dmx.stats.int_markovchain.IntegerMarkovChainAccumulator* attribute), 335
 init_counts (*dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator* attribute), 319
 init_dist (*dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution* attribute), 200
 init_dist (*dmx.stats.int_markovchain.IntegerMarkovChainDistribution* attribute), 340
 init_dist (*dmx.stats.int_markovchain.IntegerMarkovChainEstimator* attribute), 343
 init_encoder (*dmx.stats.int_edit_setdist.IntegerBernoulliEditDataEncoder* attribute), 200
 init_encoder (*dmx.stats.int_markovchain.IntegerMarkovChainDataEncoder* attribute), 339
 init_est (*dmx.stats.int_edit_setdist.IntegerBernoulliEditEstimator* attribute), 203
 init_estimator (*dmx.stats.int_markovchain.IntegerMarkovChainEstimator* attribute), 343
 init_factory (*dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator* attribute), 199
 init_factory (*dmx.stats.int_markovchain.IntegerMarkovChainAccumulator* attribute), 338
 init_key (*dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator* attribute), 319
 init_log_pvec (*dmx.stats.markovchain.MarkovChainDistribution* attribute), 111
 init_prob (*dmx.stats.markovchain.MarkovChainSampler* attribute), 116
 init_prob_map (*dmx.stats.markovchain.MarkovChainDistribution* attribute), 110
 init_prob_vec (*dmx.stats.int_hidden_association.IntegerHiddenAssociation* attribute), 312
 init_prob_vec (*dmx.stats.sparse_markov_transform.SparseMarkovAssociation* attribute), 356
 init_rng (*dmx.stats.int_edit_setdist.IntegerBernoulliEditSampler* attribute), 204
 initialize() (*dmx.bstats.bernoulli.BernoulliEstimatorAccumulator* method), 412
 initialize() (*dmx.bstats.categorical.CategoricalEstimatorAccumulator* method), 418
 initialize() (*dmx.bstats.composite.CompositeEstimatorAccumulator* method), 473
 initialize() (*dmx.bstats.conditional.ConditionalDistributionEstimatorAccumulator* method), 479
 initialize() (*dmx.bstats.dirac.DiracAccumulator* method), 420
 initialize() (*dmx.bstats.dmvn.DiagonalGaussianAccumulator* method), 425
 initialize() (*dmx.bstats.dpm.DirichletProcessMixtureAccumulator* method), 481
 initialize() (*dmx.bstats.exponential.ExponentialAccumulator* method), 431
 initialize() (*dmx.bstats.gamma.GammaAccumulator* method), 437
 initialize() (*dmx.bstats.gaussian.GaussianAccumulator* method), 443
 initialize() (*dmx.bstats.geometric.GeometricAccumulator* method), 449
 initialize() (*dmx.bstats.ignored.IgnoredAccumulator* method), 487
 initialize() (*dmx.bstats.intrange.IntegerCategoricalAccumulator* method), 455
 initialize() (*dmx.bstats.mixture.MixtureEstimatorAccumulator* method), 497

`initialize()` (`dmx.bstats.nulldist.NullAccumulator` method), 500
`initialize()` (`dmx.bstats.optional.OptionalEstimatorAccumulator` method), 510
`initialize()` (`dmx.bstats.pdist.StatisticAccumulator` method), 382
`initialize()` (`dmx.bstats.poisson.PoissonEstimatorAccumulator` method), 465
`initialize()` (`dmx.bstats.sequence.SequenceEstimatorAccumulator` method), 516
`initialize()` (`dmx.bstats.setdist.BernoulliSetAccumulator` method), 519
`initialize()` (`dmx.stats.dirac_length.DiracMixtureAccumulator` method), 187
`initialize()` (`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator` method), 243
`initialize()` (`dmx.stats.gmm.GaussianMixtureAccumulator` method), 256
`initialize()` (`dmx.stats.hidden_association.HiddenAssociationAccumulator` method), 295
`initialize()` (`dmx.stats.icltree.ICLTreeAccumulator` method), 302
`initialize()` (`dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator` method), 198
`initialize()` (`dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditAccumulator` method), 205
`initialize()` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator` method), 308
`initialize()` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator` method), 321
`initialize()` (`dmx.stats.int_markovchain.IntegerMarkovChainAccumulator` method), 336
`initialize()` (`dmx.stats.int_plsi.IntegerPLSIAccumulator` method), 268
`initialize()` (`dmx.stats.lda.LDAEstimatorAccumulator` method), 284
`initialize()` (`dmx.stats.look_back_hmm.LookbackHiddenMarkovModelAccumulator` method), 349
`initialize()` (`dmx.stats.null_dist.NullAccumulator` method), 211
`initialize()` (`dmx.stats.pdist.StatisticAccumulator` method), 15
`initialize()` (`dmx.stats.select.SelectEstimatorAccumulator` method), 219
`initialize()` (`dmx.stats.sparse_markov_transform.SparseMarkovAccumulator` method), 354
`initialize()` (`dmx.stats.spearman_rho.SpearmanRankingAccumulator` method), 222
`initialize()` (`dmx.stats.ss_mixture.SemiSupervisedMixtureAccumulator` method), 292
`initialize()` (`dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator` method), 361
`initialize()` (`dmx.stats.vmf.VonMisesFisherAccumulator` method), 229
`initialize()` (`dmx.stats.weighted.WeightedAccumulator` method), 237
`initialize()` (in module `dmx.stats`), 7
`initialize_mpi()` (in module `dmx.mpi4py.bstats`), 706
`initialize_rng()` (`dmx.stats.sparse_markov_transform.SparseMarkovAccumulator` method), 354
`int_count` (`dmx.utils.automatic.DatumNode` attribute), 653
`int_count_by_value()` (in module `dmx.torch_utils.optsutil`), 697
`int_tensor()` (in module `dmx.torch_utils.vector`), 698
`int_vec()` (in module `dmx.torch_utils.vector`), 699
`IntegerBernoulliEditAccumulator` (class in `dmx.stats.int_edit_setdist`), 196
`IntegerBernoulliEditAccumulatorFactory` (class in `dmx.stats.int_edit_setdist`), 199
`IntegerBernoulliEditDataEncoder` (class in `dmx.stats.int_edit_setdist`), 200
`IntegerBernoulliEditDistribution` (class in `dmx.stats.int_edit_setdist`), 200
`IntegerBernoulliEditEncodedDataSequence` (class in `dmx.stats.int_edit_setdist`), 202
`IntegerBernoulliEditEstimator` (class in `dmx.stats.int_edit_setdist`), 202
`IntegerBernoulliEditSampler` (class in `dmx.stats.int_edit_setdist`), 204
`IntegerBernoulliSetAccumulator` (class in `dmx.torch_stats.intsetdist`), 585
`IntegerBernoulliSetAccumulatorFactory` (class in `dmx.torch_stats.intsetdist`), 587
`IntegerBernoulliSetDataEncoder` (class in `dmx.torch_stats.intsetdist`), 587
`IntegerBernoulliSetDistribution` (class in `dmx.stats.intsetdist`), 87
`IntegerBernoulliSetDistribution` (class in `dmx.torch_stats.intsetdist`), 588
`IntegerBernoulliSetEstimator` (class in `dmx.stats.intsetdist`), 90
`IntegerBernoulliSetEstimator` (class in `dmx.torch_stats.intsetdist`), 590
`IntegerBernoulliSetSampler` (class in `dmx.stats.intsetdist`), 91
`IntegerBernoulliSetSampler` (class in `dmx.torch_stats.intsetdist`), 591
`IntegerBernoulliSetTensorSequence` (class in `dmx.torch_stats.intsetdist`), 591
`IntegerCategoricalAccumulator` (class in `dmx.bstats.intrange`), 454
`IntegerCategoricalAccumulator` (class in `dmx.torch_stats.intrange`), 578
`IntegerCategoricalAccumulatorFactory` (class in `dmx.bstats.intrange`), 456
`IntegerCategoricalAccumulatorFactory` (class in `dmx.torch_stats.intrange`), 580

IntegerCategoricalDataEncoder (class in <i>dmx.bstats.intrange</i>), 457	IntegerHiddenMarkovSampler (class in <i>dmx.stats.int_hidden_markov</i>), 329
IntegerCategoricalDataEncoder (class in <i>dmx.torch_stats.intrange</i>), 580	IntegerMarkovChainAccumulator (class in <i>dmx.stats.int_markovchain</i>), 335
IntegerCategoricalDistribution (class in <i>dmx.bstats.intrange</i>), 457	IntegerMarkovChainAccumulatorFactory (class in <i>dmx.stats.int_markovchain</i>), 338
IntegerCategoricalDistribution (class in <i>dmx.stats.intrange</i>), 34	IntegerMarkovChainDataEncoder (class in <i>dmx.stats.int_markovchain</i>), 339
IntegerCategoricalDistribution (class in <i>dmx.torch_stats.intrange</i>), 581	IntegerMarkovChainDistribution (class in <i>dmx.stats.int_markovchain</i>), 340
IntegerCategoricalEncodedData (class in <i>dmx.bstats.intrange</i>), 459	IntegerMarkovChainEncodedDataSequence (class in <i>dmx.stats.int_markovchain</i>), 342
IntegerCategoricalEstimator (class in <i>dmx.bstats.intrange</i>), 459	IntegerMarkovChainEstimator (class in <i>dmx.stats.int_markovchain</i>), 342
IntegerCategoricalEstimator (class in <i>dmx.stats.intrange</i>), 37	IntegerMarkovChainSampler (class in <i>dmx.stats.int_markovchain</i>), 344
IntegerCategoricalEstimator (class in <i>dmx.torch_stats.intrange</i>), 583	IntegerMultinomialAccumulator (class in <i>dmx.torch_stats.intmultinomial</i>), 569
IntegerCategoricalSampler (class in <i>dmx.bstats.intrange</i>), 460	IntegerMultinomialAccumulatorFactory (class in <i>dmx.torch_stats.intmultinomial</i>), 571
IntegerCategoricalSampler (class in <i>dmx.stats.intrange</i>), 39	IntegerMultinomialDataEncoder (class in <i>dmx.torch_stats.intmultinomial</i>), 572
IntegerCategoricalSampler (class in <i>dmx.torch_stats.intrange</i>), 584	IntegerMultinomialDistribution (class in <i>dmx.stats.intmultinomial</i>), 97
IntegerCategoricalTorchSequence (class in <i>dmx.torch_stats.intrange</i>), 585	IntegerMultinomialDistribution (class in <i>dmx.torch_stats.intmultinomial</i>), 573
IntegerHiddenAssociationAccumulator (class in <i>dmx.stats.int_hidden_association</i>), 307	IntegerMultinomialEstimator (class in <i>dmx.stats.intmultinomial</i>), 100
IntegerHiddenAssociationAccumulatorFactory (class in <i>dmx.stats.int_hidden_association</i>), 309	IntegerMultinomialEstimator (class in <i>dmx.torch_stats.intmultinomial</i>), 575
IntegerHiddenAssociationDataEncoder (class in <i>dmx.stats.int_hidden_association</i>), 309	IntegerMultinomialSampler (class in <i>dmx.stats.intmultinomial</i>), 102
IntegerHiddenAssociationDistribution (class in <i>dmx.stats.int_hidden_association</i>), 310	IntegerMultinomialSampler (class in <i>dmx.torch_stats.intmultinomial</i>), 577
IntegerHiddenAssociationEncodedDataSequence (class in <i>dmx.stats.int_hidden_association</i>), 313	IntegerMultinomialTorchSequence (class in <i>dmx.torch_stats.intmultinomial</i>), 577
IntegerHiddenAssociationEstimator (class in <i>dmx.stats.int_hidden_association</i>), 313	IntegerPLSIAccumulator (class in <i>dmx.stats.int_plsi</i>), 266
IntegerHiddenAssociationSampler (class in <i>dmx.stats.int_hidden_association</i>), 316	IntegerPLSIAccumulator (class in <i>dmx.torch_stats.int_plsi</i>), 637
IntegerHiddenMarkovAccumulator (class in <i>dmx.stats.int_hidden_markov</i>), 318	IntegerPLSIAccumulatorFactory (class in <i>dmx.stats.int_plsi</i>), 269
IntegerHiddenMarkovAccumulatorFactory (class in <i>dmx.stats.int_hidden_markov</i>), 322	IntegerPLSIAccumulatorFactory (class in <i>dmx.torch_stats.int_plsi</i>), 638
IntegerHiddenMarkovDataEncoder (class in <i>dmx.stats.int_hidden_markov</i>), 323	IntegerPLSIDataEncoder (class in <i>dmx.stats.int_plsi</i>), 270
IntegerHiddenMarkovEncodedDataSequence (class in <i>dmx.stats.int_hidden_markov</i>), 324	IntegerPLSIDataEncoder (class in <i>dmx.torch_stats.int_plsi</i>), 639
IntegerHiddenMarkovEstimator (class in <i>dmx.stats.int_hidden_markov</i>), 324	IntegerPLSIDistribution (class in <i>dmx.stats.int_plsi</i>), 271
IntegerHiddenMarkovModelDistribution (class in <i>dmx.stats.int_hidden_markov</i>), 326	IntegerPLSIDistribution (class in <i>dmx.torch_stats.int_plsi</i>), 639
	IntegerPLSIEncodedDataSequence (class in <i>dmx.stats.int_plsi</i>), 274

IntegerPLSIEstimator (class in <i>dmx.stats.int_plsi</i>), 274	k (dmx.stats.int_spike.SpikeAndSlabDistribution attribute), 40
IntegerPLSIEstimator (class in <i>dmx.torch_stats.int_plsi</i>), 640	k_fold_split_index() (in module <i>dmx.bstats.bestimation</i>), 387
IntegerPLSISampler (class in <i>dmx.stats.int_plsi</i>), 276	k_fold_split_index() (in module <i>dmx.utils.estimation</i>), 664
IntegerPLSISampler (class in <i>dmx.torch_stats.int_plsi</i>), 641	kappa (dmx.stats.vmf.VonMisesFisherDistribution attribute), 232
IntegerPLSTorchSequence (class in <i>dmx.torch_stats.int_plsi</i>), 642	key (dmx.stats.int_markovchain.IntegerMarkovChainEstimator attribute), 343
IntegerStepBernoulliEditAccumulator (class in <i>dmx.stats.int_edit_stepsetdist</i>), 205	key (dmx.stats.vmf.VonMisesFisherAccumulator attribute), 228
IntegerStepBernoulliEditAccumulatorFactory (class in <i>dmx.stats.int_edit_stepsetdist</i>), 207	key (dmx.torch_stats.binomial.BinomialAccumulator attribute), 532
IntegerStepBernoulliEditDataEncoder (class in <i>dmx.stats.int_edit_stepsetdist</i>), 207	key (dmx.torch_stats.dmvn.DiagonalGaussianAccumulator attribute), 540
IntegerStepBernoulliEditDistribution (class in <i>dmx.stats.int_edit_stepsetdist</i>), 207	key (dmx.torch_stats.exponential.ExponentialAccumulator attribute), 546
IntegerStepBernoulliEditEstimator (class in <i>dmx.stats.int_edit_stepsetdist</i>), 209	key (dmx.torch_stats.gamma.GammaAccumulator attribute), 552
IntegerStepBernoulliEditSampler (class in <i>dmx.stats.int_edit_stepsetdist</i>), 209	key (dmx.torch_stats.gaussian.GaussianAccumulator attribute), 558
IntegerStepBernoulliEncodedDataSequence (class in <i>dmx.stats.int_edit_stepsetdist</i>), 210	key (dmx.torch_stats.geometric.GeometricAccumulator attribute), 564
inv_key_map (dmx.stats.markovchain.MarkovChainDistribution attribute), 111	key (dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator attribute), 570
iterate() (in module <i>dmx.bstats.bestimation</i>), 386	key (dmx.torch_stats.intrange.IntegerCategoricalAccumulator attribute), 579
iterate() (in module <i>dmx.utils.estimation</i>), 663	key (dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulator attribute), 585
ive() (in module <i>dmx.utils.special</i>), 682	key (dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution attribute), 588
J	key (dmx.torch_stats.mvn.MultivariateGaussianEstimator attribute), 596
JointMixtureDataEncoder (class in <i>dmx.torch_stats.jmixture</i>), 642	key (dmx.torch_stats.poisson.PoissonAccumulator attribute), 598
JointMixtureDistribution (class in <i>dmx.stats.jmixture</i>), 161	key_map (dmx.stats.markovchain.MarkovChainDistribution attribute), 111
JointMixtureDistribution (class in <i>dmx.torch_stats.jmixture</i>), 642	key_merge() (dmx.bstats.bernoulli.BernoulliEstimatorAccumulator method), 412
JointMixtureEstimator (class in <i>dmx.stats.jmixture</i>), 165	key_merge() (dmx.bstats.composite.CompositeEstimatorAccumulator method), 473
JointMixtureEstimator (class in <i>dmx.torch_stats.jmixture</i>), 644	key_merge() (dmx.bstats.conditional.ConditionalDistributionEstimatorAccumulator method), 479
JointMixtureEstimatorAccumulator (class in <i>dmx.torch_stats.jmixture</i>), 644	key_merge() (dmx.bstats.dirac.DiracAccumulator method), 420
JointMixtureEstimatorAccumulatorFactory (class in <i>dmx.torch_stats.jmixture</i>), 646	key_merge() (dmx.bstats.dirichlet.DirichletAccumulator method), 394
JointMixtureSampler (class in <i>dmx.stats.jmixture</i>), 167	key_merge() (dmx.bstats.dmvn.DiagonalGaussianAccumulator method), 425
JointMixtureSampler (class in <i>dmx.torch_stats.jmixture</i>), 646	key_merge() (dmx.bstats.dpm.DirichletProcessMixtureAccumulator method), 481
JointMixtureTorchEncodedSequence (class in <i>dmx.torch_stats.jmixture</i>), 647	key_merge() (dmx.bstats.exponential.ExponentialAccumulator method), 432
K	
k (dmx.stats.gamma.GammaDistribution attribute), 55	

`key_merge()` (`dmx.bstats.gamma.GammaAccumulator` method), 437
`key_merge()` (`dmx.bstats.gaussian.GaussianAccumulator` method), 444
`key_merge()` (`dmx.bstats.geometric.GeometricAccumulator` method), 449
`key_merge()` (`dmx.bstats.ignored.IgnoredAccumulator` method), 487
`key_merge()` (`dmx.bstats.intrange.IntegerCategoricalAccumulator` method), 455
`key_merge()` (`dmx.bstats.mixture.MixtureEstimatorAccumulator` method), 497
`key_merge()` (`dmx.bstats.nulldist.NullAccumulator` method), 500
`key_merge()` (`dmx.bstats.optional.OptionalEstimatorAccumulator` method), 510
`key_merge()` (`dmx.bstats.pdist.StatisticAccumulator` method), 382
`key_merge()` (`dmx.bstats.poisson.PoissonEstimatorAccumulator` method), 466
`key_merge()` (`dmx.bstats.sequence.SequenceEstimatorAccumulator` method), 517
`key_merge()` (`dmx.bstats.setdist.BernoulliSetAccumulator` method), 519
`key_merge()` (`dmx.stats.dirac_length.DiracMixtureAccumulator` method), 188
`key_merge()` (`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator` method), 244
`key_merge()` (`dmx.stats.gmm.GaussianMixtureAccumulator` method), 257
`key_merge()` (`dmx.stats.hidden_association.HiddenAssociationAccumulator` method), 295
`key_merge()` (`dmx.stats.icltree.ICLTreeAccumulator` method), 302
`key_merge()` (`dmx.stats.int_edit_setdist.IntegerBernoulliEditDistanceAccumulator` method), 198
`key_merge()` (`dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditDistanceAccumulator` method), 206
`key_merge()` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator` method), 308
`key_merge()` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovChainAccumulator` method), 321
`key_merge()` (`dmx.stats.int_markovchain.IntegerMarkovChainAccumulator` method), 337
`key_merge()` (`dmx.stats.int_plsi.IntegerPLSIAccumulator` method), 268
`key_merge()` (`dmx.stats.lda.LDAEstimatorAccumulator` method), 284
`key_merge()` (`dmx.stats.look_back_hmm.LookbackHiddenMarkovChainAccumulator` method), 349
`key_merge()` (`dmx.stats.null_dist.NullAccumulator` method), 211
`key_merge()` (`dmx.stats.pdist.StatisticAccumulator` method), 15
`key_merge()` (`dmx.stats.select.SelectEstimatorAccumulator` method), 219
`key_merge()` (`dmx.stats.sparse_markov_transform.SparseMarkovAssociationAccumulator` method), 354
`key_merge()` (`dmx.stats.spearman_rho.SpearmanRankingAccumulator` method), 222
`key_merge()` (`dmx.stats.ss_mixture.SemiSupervisedMixtureEstimatorAccumulator` method), 292
`key_merge()` (`dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator` method), 361
`key_merge()` (`dmx.stats.vmf.VonMisesFisherAccumulator` method), 229
`key_merge()` (`dmx.stats.weighted.WeightedAccumulator` method), 237
`key_merge()` (`dmx.torch_stats.binomial.BinomialAccumulator` method), 533
`key_merge()` (`dmx.torch_stats.composite.CompositeAccumulator` method), 604
`key_merge()` (`dmx.torch_stats.conditional.ConditionalDistributionAccumulator` method), 610
`key_merge()` (`dmx.torch_stats.dmvn.DiagonalGaussianAccumulator` method), 541
`key_merge()` (`dmx.torch_stats.exponential.ExponentialAccumulator` method), 546
`key_merge()` (`dmx.torch_stats.gamma.GammaAccumulator` method), 552
`key_merge()` (`dmx.torch_stats.gaussian.GaussianAccumulator` method), 559
`key_merge()` (`dmx.torch_stats.geometric.GeometricAccumulator` method), 565
`key_merge()` (`dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulator` method), 623
`key_merge()` (`dmx.torch_stats.hmm.HiddenMarkovAccumulator` method), 628
`key_merge()` (`dmx.torch_stats.int_plsi.IntegerPLSIAccumulator` method), 637
`key_merge()` (`dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator` method), 571
`key_merge()` (`dmx.torch_stats.intrange.IntegerCategoricalAccumulator` method), 579
`key_merge()` (`dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulator` method), 586
`key_merge()` (`dmx.torch_stats.jmixture.JointMixtureEstimatorAccumulator` method), 645
`key_merge()` (`dmx.torch_stats.mixture.MixtureAccumulator` method), 647
`key_merge()` (`dmx.torch_stats.mvn.MultivariateGaussianAccumulator` method), 592
`key_merge()` (`dmx.torch_stats.null_dist.NullAccumulator` method), 614
`key_merge()` (`dmx.torch_stats.pdist.TorchStatisticAccumulator` method), 530
`key_merge()` (`dmx.torch_stats.poisson.PoissonAccumulator` method), 598

[key_merge\(\)](#) (*dmx.torch_stats.sequence.SequenceAccumulator* method), 618
[key_replace\(\)](#) (*dmx.bstats.bernoulli.BernoulliEstimatorAccumulator* method), 412
[key_replace\(\)](#) (*dmx.bstats.composite.CompositeEstimatorAccumulator* method), 473
[key_replace\(\)](#) (*dmx.bstats.conditional.ConditionalDistributionAccumulator* method), 479
[key_replace\(\)](#) (*dmx.bstats.dirac.DiracAccumulator* method), 420
[key_replace\(\)](#) (*dmx.bstats.dirichlet.DirichletAccumulator* method), 394
[key_replace\(\)](#) (*dmx.bstats.dmvn.DiagonalGaussianAccumulator* method), 426
[key_replace\(\)](#) (*dmx.bstats.dpm.DirichletProcessMixtureAccumulator* method), 482
[key_replace\(\)](#) (*dmx.bstats.exponential.ExponentialAccumulator* method), 432
[key_replace\(\)](#) (*dmx.bstats.gamma.GammaAccumulator* method), 437
[key_replace\(\)](#) (*dmx.bstats.gaussian.GaussianAccumulator* method), 444
[key_replace\(\)](#) (*dmx.bstats.geometric.GeometricAccumulator* method), 449
[key_replace\(\)](#) (*dmx.bstats.ignored.IgnoredAccumulator* method), 488
[key_replace\(\)](#) (*dmx.bstats.intrange.IntegerCategoricalAccumulator* method), 455
[key_replace\(\)](#) (*dmx.bstats.mixture.MixtureEstimatorAccumulator* method), 498
[key_replace\(\)](#) (*dmx.bstats.nulldist.NullAccumulator* method), 500
[key_replace\(\)](#) (*dmx.bstats.optional.OptionalEstimatorAccumulator* method), 510
[key_replace\(\)](#) (*dmx.bstats.pdist.StatisticAccumulator* method), 382
[key_replace\(\)](#) (*dmx.bstats.poisson.PoissonEstimatorAccumulator* method), 466
[key_replace\(\)](#) (*dmx.bstats.sequence.SequenceEstimatorAccumulator* method), 517
[key_replace\(\)](#) (*dmx.bstats.setdist.BernoulliSetAccumulator* method), 519
[key_replace\(\)](#) (*dmx.stats.dirac_length.DiracMixtureAccumulator* method), 188
[key_replace\(\)](#) (*dmx.stats.dmvn_mixture.DiagonalGaussianAccumulator* method), 244
[key_replace\(\)](#) (*dmx.stats.gmm.GaussianMixtureAccumulator* method), 257
[key_replace\(\)](#) (*dmx.stats.hidden_association.HiddenAssociationAccumulator* method), 296
[key_replace\(\)](#) (*dmx.stats.icltree.ICLTreeAccumulator* method), 302
[key_replace\(\)](#) (*dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator* method), 198
[key_replace\(\)](#) (*dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditAccumulator* method), 206
[key_replace\(\)](#) (*dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator* method), 308
[key_replace\(\)](#) (*dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator* method), 321
[key_replace\(\)](#) (*dmx.stats.int_markovchain.IntegerMarkovChainAccumulator* method), 337
[key_replace\(\)](#) (*dmx.stats.int_plsi.IntegerPLSIAccumulator* method), 268
[key_replace\(\)](#) (*dmx.stats.lda.LDAEstimatorAccumulator* method), 284
[key_replace\(\)](#) (*dmx.stats.look_back_hmm.LookbackHiddenMarkovEstimatorAccumulator* method), 349
[key_replace\(\)](#) (*dmx.stats.null_dist.NullAccumulator* method), 211
[key_replace\(\)](#) (*dmx.stats.pdist.StatisticAccumulator* method), 16
[key_replace\(\)](#) (*dmx.stats.select.SelectEstimatorAccumulator* method), 219
[key_replace\(\)](#) (*dmx.stats.sparse_markov_transform.SparseMarkovAssociationAccumulator* method), 354
[key_replace\(\)](#) (*dmx.stats.spearman_rho.SpearmanRankingAccumulator* method), 222
[key_replace\(\)](#) (*dmx.stats.ss_mixture.SemiSupervisedMixtureEstimatorAccumulator* method), 293
[key_replace\(\)](#) (*dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator* method), 361
[key_replace\(\)](#) (*dmx.stats.vmf.VonMisesFisherAccumulator* method), 229
[key_replace\(\)](#) (*dmx.stats.weighted.WeightedAccumulator* method), 237
[key_replace\(\)](#) (*dmx.torch_stats.binomial.BinomialAccumulator* method), 533
[key_replace\(\)](#) (*dmx.torch_stats.composite.CompositeAccumulator* method), 604
[key_replace\(\)](#) (*dmx.torch_stats.conditional.ConditionalDistributionAccumulator* method), 610
[key_replace\(\)](#) (*dmx.torch_stats.dmvn.DiagonalGaussianAccumulator* method), 541
[key_replace\(\)](#) (*dmx.torch_stats.exponential.ExponentialAccumulator* method), 547
[key_replace\(\)](#) (*dmx.torch_stats.gamma.GammaAccumulator* method), 552
[key_replace\(\)](#) (*dmx.torch_stats.gaussian.GaussianAccumulator* method), 559
[key_replace\(\)](#) (*dmx.torch_stats.geometric.GeometricAccumulator* method), 565
[key_replace\(\)](#) (*dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulator* method), 623
[key_replace\(\)](#) (*dmx.torch_stats.hmm.HiddenMarkovAccumulator* method), 629
[key_replace\(\)](#) (*dmx.torch_stats.int_plsi.IntegerPLSIAccumulator* method), 637

`key_replace()` (*dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator* attribute), 571
`key_replace()` (*dmx.torch_stats.intrange.IntegerCategoricalAccumulator* attribute), 579
`key_replace()` (*dmx.torch_stats.intsetdist.IntegerBernoulliAccumulator* attribute), 586
`key_replace()` (*dmx.torch_stats.jmixture.JointMixtureEstimator* attribute), 645
`key_replace()` (*dmx.torch_stats.mixture.MixtureAccumulator* attribute), 648
`key_replace()` (*dmx.torch_stats.mvn.MultivariateGaussianAccumulator* attribute), 592
`key_replace()` (*dmx.torch_stats.null_dist.NullAccumulator* attribute), 614
`key_replace()` (*dmx.torch_stats.pdist.TorchStatisticAccumulator* attribute), 530
`key_replace()` (*dmx.torch_stats.poisson.PoissonAccumulator* attribute), 598
`key_replace()` (*dmx.torch_stats.sequence.SequenceAccumulator* attribute), 618
`keys` (*dmx.stats.binomial.BinomialDistribution* attribute), 20
`keys` (*dmx.stats.binomial.BinomialEstimator* attribute), 23
`keys` (*dmx.stats.categorical.CategoricalDistribution* attribute), 82
`keys` (*dmx.stats.categorical.CategoricalEstimator* attribute), 85
`keys` (*dmx.stats.catmultinomial.MultinomialDistribution* attribute), 104
`keys` (*dmx.stats.catmultinomial.MultinomialEstimator* attribute), 107
`keys` (*dmx.stats.composite.CompositeDistribution* attribute), 118
`keys` (*dmx.stats.composite.CompositeEstimator* attribute), 121
`keys` (*dmx.stats.conditional.ConditionalDistribution* attribute), 130
`keys` (*dmx.stats.conditional.ConditionalDistributionEstimator* attribute), 133
`keys` (*dmx.stats.dirac_length.DiracMixtureAccumulatorFactory* attribute), 189
`keys` (*dmx.stats.dirac_length.DiracMixtureDistribution* attribute), 191
`keys` (*dmx.stats.dirac_length.DiracMixtureEstimator* attribute), 195
`keys` (*dmx.stats.dirichlet.DirichletDistribution* attribute), 77
`keys` (*dmx.stats.dirichlet.DirichletEstimator* attribute), 79
`keys` (*dmx.stats.dmvn.DiagonalGaussianDistribution* attribute), 66
`keys` (*dmx.stats.dmvn.DiagonalGaussianEstimator* attribute), 68
`keys` (*dmx.stats.int_multinomial.IntegerMultinomialAccumulator* attribute), 243
`keys` (*dmx.stats.int_multinomial.IntegerMultinomialEstimator* attribute), 246
`keys` (*dmx.stats.int_setdist.IntegerBernoulliAccumulator* attribute), 247
`keys` (*dmx.stats.int_setdist.IntegerBernoulliEstimator* attribute), 252
`keys` (*dmx.stats.exponential.ExponentialDistribution* attribute), 50
`keys` (*dmx.stats.exponential.ExponentialEstimator* attribute), 53
`keys` (*dmx.stats.gamma.GammaDistribution* attribute), 55
`keys` (*dmx.stats.gamma.GammaEstimator* attribute), 58
`keys` (*dmx.stats.gaussian.GaussianDistribution* attribute), 45
`keys` (*dmx.stats.gaussian.GaussianEstimator* attribute), 48
`keys` (*dmx.stats.geometric.GeometricDistribution* attribute), 25
`keys` (*dmx.stats.geometric.GeometricEstimator* attribute), 28
`keys` (*dmx.stats.gmm.GaussianMixtureAccumulator* attribute), 255
`keys` (*dmx.stats.gmm.GaussianMixtureAccumulatorFactory* attribute), 258
`keys` (*dmx.stats.gmm.GaussianMixtureDistribution* attribute), 260
`keys` (*dmx.stats.gmm.GaussianMixtureEstimator* attribute), 264
`keys` (*dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution* attribute), 155
`keys` (*dmx.stats.heterogeneous_mixture.HeterogeneousMixtureEstimator* attribute), 159
`keys` (*dmx.stats.hidden_association.HiddenAssociationDistribution* attribute), 298
`keys` (*dmx.stats.hidden_association.HiddenAssociationEstimator* attribute), 300
`keys` (*dmx.stats.hidden_markov.HiddenMarkovEstimator* attribute), 182
`keys` (*dmx.stats.hidden_markov.HiddenMarkovModelDistribution* attribute), 178
`keys` (*dmx.stats.hmixture.HierarchicalMixtureDistribution* attribute), 169
`keys` (*dmx.stats.hmixture.HierarchicalMixtureEstimator* attribute), 173
`keys` (*dmx.stats.icltree.ICLTreeDistribution* attribute), 304
`keys` (*dmx.stats.ignored.IgnoredDistribution* attribute), 142
`keys` (*dmx.stats.ignored.IgnoredEstimator* attribute), 144
`keys` (*dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator* attribute), 197

[keys \(dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulatorFactory attribute\), 199](#)
[keys \(dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution attribute\), 201](#)
[keys \(dmx.stats.int_edit_setdist.IntegerBernoulliEditEstimator attribute\), 203](#)
[keys \(dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution attribute\), 312](#)
[keys \(dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator attribute\), 315](#)
[keys \(dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulatorFactory attribute\), 323](#)
[keys \(dmx.stats.int_hidden_markov.IntegerHiddenMarkovEstimator attribute\), 325](#)
[keys \(dmx.stats.int_hidden_markov.IntegerHiddenMarkovMarkovChainDistribution attribute\), 327](#)
[keys \(dmx.stats.int_markovchain.IntegerMarkovChainAccumulator attribute\), 336](#)
[keys \(dmx.stats.int_markovchain.IntegerMarkovChainAccumulatorFactory attribute\), 338](#)
[keys \(dmx.stats.int_markovchain.IntegerMarkovChainDistribution attribute\), 341](#)
[keys \(dmx.stats.int_plsi.IntegerPLSIAccumulatorFactory attribute\), 270](#)
[keys \(dmx.stats.int_plsi.IntegerPLSIDistribution attribute\), 272](#)
[keys \(dmx.stats.int_plsi.IntegerPLSEstimator attribute\), 275](#)
[keys \(dmx.stats.int_spike.SpikeAndSlabDistribution attribute\), 41](#)
[keys \(dmx.stats.int_spike.SpikeAndSlabEstimator attribute\), 43](#)
[keys \(dmx.stats.intmultinomial.IntegerMultinomialDistribution attribute\), 98](#)
[keys \(dmx.stats.intmultinomial.IntegerMultinomialEstimator attribute\), 101](#)
[keys \(dmx.stats.inrange.IntegerCategoricalDistribution attribute\), 35](#)
[keys \(dmx.stats.inrange.IntegerCategoricalEstimator attribute\), 38](#)
[keys \(dmx.stats.intsetdist.IntegerBernoulliSetDistribution attribute\), 88](#)
[keys \(dmx.stats.intsetdist.IntegerBernoulliSetEstimator attribute\), 90](#)
[keys \(dmx.stats.jmixture.JointMixtureDistribution attribute\), 163](#)
[keys \(dmx.stats.jmixture.JointMixtureEstimator attribute\), 165](#)
[keys \(dmx.stats.log_gaussian.LogGaussianDistribution attribute\), 61](#)
[keys \(dmx.stats.log_gaussian.LogGaussianEstimator attribute\), 63](#)
[keys \(dmx.stats.markovchain.MarkovChainDistribution attribute\), 111](#)
[keys \(dmx.stats.markovchain.MarkovChainEstimator attribute\), 114](#)
[keys \(dmx.stats.mixture.MixtureDistribution attribute\), 147](#)
[keys \(dmx.stats.mixture.MixtureEstimator attribute\), 151](#)
[keys \(dmx.stats.mvn.MultivariateGaussianDistribution attribute\), 71](#)
[keys \(dmx.stats.mvn.MultivariateGaussianEstimator attribute\), 74](#)
[keys \(dmx.stats.null_dist.NullAccumulator attribute\), 210](#)
[keys \(dmx.stats.null_dist.NullAccumulatorFactory attribute\), 212](#)
[keys \(dmx.stats.null_dist.NullEstimator attribute\), 215](#)
[keys \(dmx.stats.optional.OptionalDistribution attribute\), 137](#)
[keys \(dmx.stats.optional.OptionalEstimator attribute\), 140](#)
[keys \(dmx.stats.poisson.PoissonDistribution attribute\), 30](#)
[keys \(dmx.stats.poisson.PoissonEstimator attribute\), 32](#)
[keys \(dmx.stats.sequence.SequenceDistribution attribute\), 124](#)
[keys \(dmx.stats.sequence.SequenceEstimator attribute\), 127](#)
[keys \(dmx.stats.setdist.BernoulliSetDistribution attribute\), 92](#)
[keys \(dmx.stats.setdist.BernoulliSetEstimator attribute\), 95](#)
[keys \(dmx.stats.sparse_markov_transform.SparseMarkovAssociationEstimator attribute\), 359](#)
[keys \(dmx.stats.spearman_rho.SpearmanRankingAccumulator attribute\), 221](#)
[keys \(dmx.stats.spearman_rho.SpearmanRankingAccumulatorFactory attribute\), 224](#)
[keys \(dmx.stats.spearman_rho.SpearmanRankingDistribution attribute\), 225](#)
[keys \(dmx.stats.spearman_rho.SpearmanRankingEstimator attribute\), 226](#)
[keys \(dmx.stats.ss_mixture.SemiSupervisedMixtureEstimator attribute\), 291](#)
[keys \(dmx.stats.vmf.VonMisesFisherAccumulatorFactory attribute\), 230](#)
[keys \(dmx.stats.vmf.VonMisesFisherDistribution attribute\), 232](#)
[keys \(dmx.stats.vmf.VonMisesFisherEstimator attribute\), 234](#)
[keys \(dmx.stats.weighted.WeightedAccumulator attribute\), 236](#)
[keys \(dmx.stats.weighted.WeightedAccumulatorFactory attribute\), 238](#)
[keys \(dmx.stats.weighted.WeightedDistribution attribute\), 240](#)
[keys \(dmx.stats.weighted.WeightedEstimator attribute\),](#)

- 241
- keys (*dmx.torch_stats.binomial.BinomialAccumulatorFactory* attribute), 534
- keys (*dmx.torch_stats.binomial.BinomialDistribution* attribute), 536
- keys (*dmx.torch_stats.binomial.BinomialEstimator* attribute), 538
- keys (*dmx.torch_stats.dmvn.DiagonalGaussianEstimator* attribute), 544
- keys (*dmx.torch_stats.gamma.GammaAccumulatorFactory* attribute), 553
- keys (*dmx.torch_stats.gamma.GammaEstimator* attribute), 556
- keys (*dmx.torch_stats.gaussian.GaussianEstimator* attribute), 562
- keys (*dmx.torch_stats.geometric.GeometricEstimator* attribute), 568
- keys (*dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulatorFactory* attribute), 572
- keys (*dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution* attribute), 574
- keys (*dmx.torch_stats.intmultinomial.IntegerMultinomialEstimator* attribute), 576
- keys (*dmx.torch_stats.intrange.IntegerCategoricalEstimator* attribute), 583
- keys (*dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulatorFactory* attribute), 587
- keys (*dmx.torch_stats.intsetdist.IntegerBernoulliSetEstimator* attribute), 590
- keys (*dmx.torch_stats.mvn.MultivariateGaussianDistribution* attribute), 594
- keys (*dmx.torch_stats.poisson.PoissonAccumulatorFactory* attribute), 599
- keys (*dmx.torch_stats.poisson.PoissonEstimator* attribute), 602
- L**
- lag (*dmx.stats.int_markovchain.IntegerMarkovChainAccumulator* attribute), 335
- lag (*dmx.stats.int_markovchain.IntegerMarkovChainAccumulatorFactory* attribute), 338
- lag (*dmx.stats.int_markovchain.IntegerMarkovChainDataEncoder* attribute), 339
- lag (*dmx.stats.int_markovchain.IntegerMarkovChainDistribution* attribute), 340
- lag (*dmx.stats.int_markovchain.IntegerMarkovChainEstimator* attribute), 343
- lam (*dmx.stats.poisson.PoissonDistribution* attribute), 30
- LDADataEncoder (class in *dmx.stats.lda*), 280
- LDADistribution (class in *dmx.stats.lda*), 280
- LDAEncodedDataSequence (class in *dmx.stats.lda*), 282
- LDAEstimator (class in *dmx.stats.lda*), 282
- LDAEstimatorAccumulator (class in *dmx.stats.lda*), 283
- LDAEstimatorAccumulatorFactory (class in *dmx.stats.lda*), 285
- LDASampler (class in *dmx.stats.lda*), 286
- least_occurring() (in module *dmx.utils.optsutil*), 678
- len_acc (*dmx.stats.int_plsi.IntegerPLSIAccumulator* attribute), 267
- len_accumulator (*dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator* attribute), 319
- len_accumulator (*dmx.stats.int_markovchain.IntegerMarkovChainAccumulator* attribute), 335
- len_accumulator (*dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator* attribute), 570
- len_dist (*dmx.stats.catmultinomial.MultinomialDistribution* attribute), 104
- len_dist (*dmx.stats.catmultinomial.MultinomialEstimator* attribute), 107
- len_dist (*dmx.stats.hidden_association.HiddenAssociationDistribution* attribute), 298
- len_dist (*dmx.stats.hidden_markov.HiddenMarkovModelDistribution* attribute), 178
- len_dist (*dmx.stats.hmixture.HierarchicalMixtureDistribution* attribute), 169
- len_dist (*dmx.stats.hmixture.HierarchicalMixtureEstimator* attribute), 173
- len_dist (*dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution* attribute), 311
- len_dist (*dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution* attribute), 327
- len_dist (*dmx.stats.int_markovchain.IntegerMarkovChainDistribution* attribute), 340
- len_dist (*dmx.stats.int_markovchain.IntegerMarkovChainEstimator* attribute), 343
- len_dist (*dmx.stats.int_plsi.IntegerPLSIDistribution* attribute), 272
- len_dist (*dmx.stats.intmultinomial.IntegerMultinomialDistribution* attribute), 98
- len_dist (*dmx.stats.intmultinomial.IntegerMultinomialEstimator* attribute), 100
- len_dist (*dmx.stats.lda.LDADistribution* attribute), 281
- len_dist (*dmx.stats.markovchain.MarkovChainDistribution* attribute), 110
- len_dist (*dmx.stats.sequence.SequenceDistribution* attribute), 123
- len_dist (*dmx.stats.sequence.SequenceEstimator* attribute), 126
- len_dist (*dmx.stats.sequence.SequenceSampler* attribute), 128
- len_dist (*dmx.stats.sparse_markov_transform.SparseMarkovAssociationDistribution* attribute), 356
- len_dist (*dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution* attribute), 364
- len_dist (*dmx.torch_stats.hmm.HiddenMarkovModelDistribution* attribute), 633
- len_dist (*dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution* attribute), 574

attribute), 574
 len_dist (dmx.torch_stats.intmultinomial.IntegerMultinomialSampler attribute), 576
 len_encoder (dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler attribute), 323
 len_encoder (dmx.stats.int_markovchain.IntegerMarkovChainSampler attribute), 339
 len_encoder (dmx.stats.int_plsi.IntegerPLSIDataEncoder attribute), 270
 len_encoder (dmx.torch_stats.intmultinomial.IntegerMultinomialSampler attribute), 572
 len_estimator (dmx.stats.catmultinomial.MultinomialEstimator attribute), 106
 len_estimator (dmx.stats.hidden_association.HiddenAssociationEstimator attribute), 300
 len_estimator (dmx.stats.hidden_markov.HiddenMarkovEstimator attribute), 182
 len_estimator (dmx.stats.hmixture.HierarchicalMixtureEstimator attribute), 173
 len_estimator (dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator attribute), 315
 len_estimator (dmx.stats.int_hidden_markov.IntegerHiddenMarkovEstimator attribute), 325
 len_estimator (dmx.stats.int_markovchain.IntegerMarkovChainEstimator attribute), 343
 len_estimator (dmx.stats.int_plsi.IntegerPLSIEstimator attribute), 275
 len_estimator (dmx.stats.intmultinomial.IntegerMultinomialEstimator attribute), 100
 len_estimator (dmx.stats.markovchain.MarkovChainEstimator attribute), 114
 len_estimator (dmx.stats.sequence.SequenceEstimator attribute), 126
 len_estimator (dmx.stats.sparse_markov_transform.SparseMarkovTransformEstimator attribute), 358
 len_estimator (dmx.torch_stats.intmultinomial.IntegerMultinomialEstimator attribute), 576
 len_factory (dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler attribute), 323
 len_factory (dmx.stats.int_markovchain.IntegerMarkovChainSampler attribute), 338
 len_factory (dmx.stats.int_plsi.IntegerPLSIAccumulator attribute), 270
 len_factory (dmx.torch_stats.intmultinomial.IntegerMultinomialSampler attribute), 572
 len_normalized (dmx.stats.catmultinomial.MultinomialDistribution attribute), 104
 len_normalized (dmx.stats.catmultinomial.MultinomialEstimator attribute), 107
 len_normalized (dmx.stats.sequence.SequenceDistribution attribute), 123
 len_normalized (dmx.stats.sequence.SequenceEstimator attribute), 126
 len_sampler (dmx.stats.catmultinomial.MultinomialSampler attribute), 108
 len_sampler (dmx.stats.hidden_markov.HiddenMarkovSampler attribute), 184
 len_sampler (dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler attribute), 330
 len_sampler (dmx.stats.intmultinomial.IntegerMultinomialSampler attribute), 102
 len_sampler (dmx.stats.markovchain.MarkovChainSampler attribute), 116
 len_sampler (dmx.stats.sequence.SequenceSampler attribute), 128
 len_sampler (dmx.torch_stats.intmultinomial.IntegerMultinomialSampler attribute), 577
 level_estimator (dmx.stats.tree_hmm, level_estimator prob() (in module dmx.stats.tree_hmm), 366
 levels (dmx.stats.categorical.CategoricalSampler attribute), 86
 levels (dmx.stats.markovchain.MarkovChainEstimator attribute), 114
 linkfn (dmx.stats.vmf, linkfn), 235
 lniv_h() (in module dmx.stats.vmf), 235
 lniv_h() (in module dmx.stats.vmf), 235
 log1p_default_value
 log1p (dmx.stats.categorical.CategoricalDistribution attribute), 82
 log1p_dv (dmx.stats.markovchain.MarkovChainDistribution attribute), 111
 log1p (dmx.stats.binomial.BinomialDistribution attribute), 20
 log1p (dmx.stats.geometric.GeometricDistribution attribute), 25
 log1p (dmx.stats.int_spike.SpikeAndSlabDistribution attribute), 40
 log1p (dmx.stats.intmultinomial.BinomialDistribution attribute), 535
 logbeta (dmx.stats.exponential.ExponentialDistribution attribute), 50
 logbeta (dmx.stats.exponential.ExponentialDistribution attribute), 548
 logAccur (dmx.stats.diagn.GaussianDistribution attribute), 65
 logAccur (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution attribute), 248
 logAccur (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution attribute), 260
 log2const (dmx.stats.dirichlet.DirichletDistribution attribute), 76
 logconst (dmx.stats.gamma.GammaDistribution attribute), 55
 logconst (dmx.stats.gaussian.GaussianDistribution attribute), 45
 logconst (dmx.stats.log_gaussian.LogGaussianDistribution attribute), 61
 logconst (dmx.stats.vmf.VonMisesFisherDistribution

attribute), 232
 log_default_value(dmx.stats.categorical.CategoricalDistribution, attribute), 82
 log_density()(dmx.bstats.bernoulli.BernoulliDistribution, method), 409
 log_density()(dmx.bstats.beta.BetaDistribution, method), 390
 log_density()(dmx.bstats.catdirichlet.DictDirichletDistribution, method), 392
 log_density()(dmx.bstats.categorical.CategoricalDistribution, method), 415
 log_density()(dmx.bstats.composite.CompositeDistribution, method), 469
 log_density()(dmx.bstats.conditional.ConditionalDistribution, method), 476
 log_density()(dmx.bstats.dirac.DiracDistribution, method), 422
 log_density()(dmx.bstats.dirichlet.DirichletDistribution, method), 397
 log_density()(dmx.bstats.dmvn.DiagonalGaussianDistribution, method), 428
 log_density()(dmx.bstats.dpm.DirichletProcessMixtureDistribution, method), 484
 log_density()(dmx.bstats.exponential.ExponentialDistribution, method), 434
 log_density()(dmx.bstats.gamma.GammaDistribution, method), 439
 log_density()(dmx.bstats.gaussian.GaussianDistribution, method), 446
 log_density()(dmx.bstats.geometric.GeometricDistribution, method), 452
 log_density()(dmx.bstats.ignored.IgnoredDistribution, method), 490
 log_density()(dmx.bstats.intrange.IntegerCategoricalDistribution, method), 458
 log_density()(dmx.bstats.mixture.MixtureDistribution, method), 494
 log_density()(dmx.bstats.mvngamma.MultivariateNormalGammaDistribution, method), 402
 log_density()(dmx.bstats.normgamma.NormalGammaDistribution, method), 404
 log_density()(dmx.bstats.nulldist.NullDistribution, method), 502
 log_density()(dmx.bstats.optional.OptionalDistribution, method), 507
 log_density()(dmx.bstats.pdist.ProbabilityDistribution, method), 377
 log_density()(dmx.bstats.poisson.PoissonDistribution, method), 462
 log_density()(dmx.bstats.sequence.SequenceDistribution, method), 513
 log_density()(dmx.bstats.setdist.BernoulliSetDistribution, method), 522
 log_density()(dmx.bstats.symdirichlet.SymmetricDirichletDistribution, method), 406
 log_density()(dmx.stats.binomial.BinomialDistribution, method), 21
 log_density()(dmx.stats.categorical.CategoricalDistribution, method), 84
 log_density()(dmx.stats.catmultinomial.MultinomialDistribution, method), 105
 log_density()(dmx.stats.composite.CompositeDistribution, method), 119
 log_density()(dmx.stats.conditional.ConditionalDistribution, method), 131
 log_density()(dmx.stats.dirac_length.DiracMixtureDistribution, method), 192
 log_density()(dmx.stats.dirichlet.DirichletDistribution, method), 78
 log_density()(dmx.stats.dmvn.DiagonalGaussianDistribution, method), 67
 log_density()(dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution, method), 249
 log_density()(dmx.stats.exponential.ExponentialDistribution, method), 51
 log_density()(dmx.stats.gamma.GammaDistribution, method), 56
 log_density()(dmx.stats.gaussian.GaussianDistribution, method), 47
 log_density()(dmx.stats.geometric.GeometricDistribution, method), 26
 log_density()(dmx.stats.gmm.GaussianMixtureDistribution, method), 261
 log_density()(dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution, method), 156
 log_density()(dmx.stats.hidden_association.HiddenAssociationDistribution, method), 298
 log_density()(dmx.stats.hidden_markov.HiddenMarkovModelDistribution, method), 179
 log_density()(dmx.stats.hmixture.HierarchicalMixtureDistribution, method), 170
 log_density()(dmx.stats.ictree.ICLTreeDistribution, method), 305
 log_density()(dmx.stats.ignored.IgnoredDistribution, method), 143
 log_density()(dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution, method), 202
 log_density()(dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditDistribution, method), 208
 log_density()(dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution, method), 312
 log_density()(dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution, method), 328
 log_density()(dmx.stats.int_markovchain.IntegerMarkovChainDistribution, method), 341
 log_density()(dmx.stats.int_plsi.IntegerPLSIDistribution, method), 273
 log_density()(dmx.stats.int_spike.SpikeAndSlabDistribution, method), 273

method), 42

log_density() (dmx.stats.intmultinomial.IntegerMultinomialDistribution method), 99

log_density() (dmx.stats.intrange.IntegerCategoricalDistribution method), 36

log_density() (dmx.stats.intsetdist.IntegerBernoulliSetDistribution method), 89

log_density() (dmx.stats.jmixture.JointMixtureDistribution method), 164

log_density() (dmx.stats.lda.LDADistribution method), 281

log_density() (dmx.stats.log_gaussian.LogGaussianDistribution method), 62

log_density() (dmx.stats.look_back_hmm.LookbackHiddenMarkovModelDistribution method), 347

log_density() (dmx.stats.markovchain.MarkovChainDistribution method), 113

log_density() (dmx.stats.mixture.MixtureDistribution method), 149

log_density() (dmx.stats.mvn.MultivariateGaussianDistribution method), 73

log_density() (dmx.stats.null_dist.NullDistribution method), 214

log_density() (dmx.stats.optional.OptionalDistribution method), 138

log_density() (dmx.stats.pdist.ProbabilityDistribution method), 12

log_density() (dmx.stats.poisson.PoissonDistribution method), 31

log_density() (dmx.stats.select.SelectDistribution method), 217

log_density() (dmx.stats.sequence.SequenceDistribution method), 125

log_density() (dmx.stats.setdist.BernoulliSetDistribution method), 94

log_density() (dmx.stats.sparse_markov_transform.SparseMarkovTransform method), 357

log_density() (dmx.stats.spearman_rho.SpearmanRankingDistribution method), 225

log_density() (dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution method), 289

log_density() (dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution method), 365

log_density() (dmx.stats.vmf.VonMisesFisherDistribution method), 232

log_density() (dmx.stats.weighted.WeightedDistribution method), 240

log_density() (dmx.torch_stats.binomial.BinomialDistribution method), 536

log_density() (dmx.torch_stats.composite.CompositeDistribution method), 606

log_density() (dmx.torch_stats.conditional.ConditionalDistribution method), 608

log_density() (dmx.torch_stats.dmvn.DiagonalGaussianDistribution method), 543

log_density() (dmx.torch_stats.exponential.ExponentialDistribution method), 549

log_density() (dmx.torch_stats.gamma.GammaDistribution method), 555

log_density() (dmx.torch_stats.gaussian.GaussianDistribution method), 561

log_density() (dmx.torch_stats.geometric.GeometricDistribution method), 567

log_density() (dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDistribution method), 625

log_density() (dmx.torch_stats.hmm.HiddenMarkovModelDistribution method), 634

log_density() (dmx.torch_stats.int_plsi.IntegerPLSIDistribution method), 640

log_density() (dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution method), 574

log_density() (dmx.torch_stats.intrange.IntegerCategoricalDistribution method), 582

log_density() (dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution method), 589

log_density() (dmx.torch_stats.jmixture.JointMixtureDistribution method), 643

log_density() (dmx.torch_stats.mixture.MixtureDistribution method), 650

log_density() (dmx.torch_stats.mvn.MultivariateGaussianDistribution method), 595

log_density() (dmx.torch_stats.null_dist.NullDistribution method), 615

log_density() (dmx.torch_stats.pdist.TorchProbabilityDistribution method), 528

log_density() (dmx.torch_stats.poisson.PoissonDistribution method), 601

log_density() (dmx.torch_stats.sequence.SequenceDistribution method), 620

log_density() (dmx.stats.int_plsi.IntegerPLSIDistribution attribute), 271

log_density() (dmx.stats.markovchain.MarkovChainDistribution attribute), 111

log_density() (dmx.stats.markovchain.MarkovChainDistribution attribute), 111

log_dvec (dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution attribute), 201

log_dvec (dmx.stats.intsetdist.IntegerBernoulliSetDistribution attribute), 88

log_dvec (dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution attribute), 588

log_edit_pmat (dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution attribute), 201

log_lambda (dmx.stats.poisson.PoissonDistribution attribute), 30

log_norm (dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution attribute), 201

attribute), 201
 log_nsum (dmx.stats.intsetdist.IntegerBernoulliSetDistribution attribute), 88
 log_nsum (dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution attribute), 588
 log_nvec (dmx.stats.intsetdist.IntegerBernoulliSetDistribution attribute), 88
 log_nvec (dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution attribute), 588
 log_p (dmx.stats.binomial.BinomialDistribution attribute), 20
 log_p (dmx.stats.geometric.GeometricDistribution attribute), 25
 log_p (dmx.stats.int_spike.SpikeAndSlabDistribution attribute), 40
 log_p (dmx.stats.optional.OptionalDistribution attribute), 136
 log_p (dmx.torch_stats.binomial.BinomialDistribution attribute), 535
 log_p_vec (dmx.stats.intmultinomial.IntegerMultinomialDistribution attribute), 97
 log_p_vec (dmx.stats.intrange.IntegerCategoricalDistribution attribute), 35
 log_p_vec (dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution attribute), 573
 log_p_vec (dmx.torch_stats.intrange.IntegerCategoricalDistribution attribute), 581
 log_pmat (dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution attribute), 326
 log_pn (dmx.stats.optional.OptionalDistribution attribute), 136
 log_posterior() (in module dmx.utils.vector), 686
 log_posterior_sum() (in module dmx.utils.vector), 686
 log_pvec (dmx.stats.intsetdist.IntegerBernoulliSetDistribution attribute), 88
 log_pvec (dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution attribute), 588
 log_pvec (dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution attribute), 591
 log_sum() (in module dmx.utils.vector), 686
 log_taus (dmx.stats.hidden_markov.HiddenMarkovModelDistribution attribute), 177
 log_taus (dmx.stats.hmixture.HierarchicalMixtureDistribution attribute), 169
 log_taus (dmx.torch_stats.hmm.HiddenMarkovModelDistribution attribute), 633
 log_taus12 (dmx.stats.jmixture.JointMixtureDistribution attribute), 163
 log_taus21 (dmx.stats.jmixture.JointMixtureDistribution attribute), 163
 log_transition_map (dmx.stats.markovchain.MarkovChainDistribution attribute), 110
 log_transitions (dmx.stats.hidden_markov.HiddenMarkovModelDistribution attribute), 177
 log_transitions (dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution attribute), 327
 log_transitions (dmx.torch_stats.hmm.HiddenMarkovModelDistribution attribute), 633
 log_w (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution attribute), 248
 log_w (dmx.stats.gmm.GaussianMixtureDistribution attribute), 260
 log_w (dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution attribute), 154
 log_w (dmx.stats.hidden_markov.HiddenMarkovModelDistribution attribute), 177
 log_w (dmx.stats.hmixture.HierarchicalMixtureDistribution attribute), 169
 log_w (dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution attribute), 327
 log_w (dmx.stats.mixture.MixtureDistribution attribute), 147
 log_w (dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution attribute), 289
 log_w (dmx.torch_stats.hmm.HiddenMarkovModelDistribution attribute), 633
 log_w (dmx.stats.jmixture.JointMixtureDistribution attribute), 162
 log_w (dmx.stats.jmixture.JointMixtureDistribution attribute), 162
 LogGaussianDistribution (class in dmx.stats.log_gaussian), 60
 LogGaussianEstimator (class in dmx.stats.log_gaussian), 62
 LogGaussianSampler (class in dmx.stats.log_gaussian), 64
 loginit_prob_map (dmx.stats.markovchain.MarkovChainDistribution attribute), 110
 logpdet() (in module dmx.utils.special), 682
 logsumexp_2d_rows() (in module dmx.stats.int_hidden_markov), 333
 logsumexp_numba() (in module dmx.stats.int_hidden_markov), 333
 LookbackHiddenMarkovDataEncoder (class in dmx.stats.look_back_hmm), 346
 LookbackHiddenMarkovDistribution (class in dmx.stats.look_back_hmm), 346
 LookbackHiddenMarkovEstimator (class in dmx.stats.look_back_hmm), 347
 LookbackHiddenMarkovEstimatorAccumulator (class in dmx.stats.look_back_hmm), 348
 LookbackHiddenMarkovEstimatorAccumulatorFactory (class in dmx.stats.look_back_hmm), 350
 LookbackHiddenMarkovSampler (class in dmx.stats.look_back_hmm), 351
 LookBackHMMEncodedDataSequence (class in dmx.stats.look_back_hmm), 345

`low_memory (dmx.stats.sparse_markov_transform.SparseMarkovTransform.accumulator_factory)` (attribute), 357
`low_memory (dmx.stats.sparse_markov_transform.SparseMarkovTransform.accumulator_factory)` (method), 246
`low_memory (dmx.stats.sparse_markov_transform.SparseMarkovTransform.accumulator_factory)` (method), 246
`low_memory (dmx.stats.sparse_markov_transform.SparseMarkovTransform.accumulator_factory)` (method), 259
`lower (dmx.stats.mvn.MultivariateGaussianDistribution.accumulator_factory)` (attribute), 71
`lower (dmx.stats.mvn.MultivariateGaussianDistribution.accumulator_factory)` (method), 297
`lower (dmx.stats.mvn.MultivariateGaussianDistribution.accumulator_factory)` (method), 297
`lower (dmx.stats.mvn.MultivariateGaussianDistribution.accumulator_factory)` (method), 303
`make () (dmx.bstats.bernoulli.BernoulliEstimatorAccumulatorFactory)` (method), 413
`make () (dmx.bstats.bernoulli.BernoulliEstimatorAccumulatorFactory)` (method), 200
`make () (dmx.bstats.categorical.CategoricalEstimatorAccumulatorFactory)` (method), 419
`make () (dmx.bstats.categorical.CategoricalEstimatorAccumulatorFactory)` (method), 207
`make () (dmx.bstats.composite.CompositeAccumulatorFactory)` (method), 468
`make () (dmx.bstats.composite.CompositeAccumulatorFactory)` (method), 309
`make () (dmx.bstats.conditional.ConditionalDistributionAccumulatorFactory)` (method), 477
`make () (dmx.bstats.conditional.ConditionalDistributionAccumulatorFactory)` (method), 323
`make () (dmx.bstats.dirac.DiracAccumulatorFactory)` (method), 421
`make () (dmx.bstats.dirac.DiracAccumulatorFactory)` (method), 339
`make () (dmx.bstats.dirichlet.DirichletAccumulatorFactory)` (method), 396
`make () (dmx.bstats.dirichlet.DirichletAccumulatorFactory)` (method), 270
`make () (dmx.bstats.dmvn.DiagonalGaussianAccumulatorFactory)` (method), 427
`make () (dmx.bstats.dmvn.DiagonalGaussianAccumulatorFactory)` (method), 286
`make () (dmx.bstats.dpm.DirichletProcessMixtureAccumulatorFactory)` (method), 483
`make () (dmx.bstats.dpm.DirichletProcessMixtureAccumulatorFactory)` (method), 351
`make () (dmx.bstats.exponential.ExponentialAccumulatorFactory)` (method), 433
`make () (dmx.bstats.exponential.ExponentialAccumulatorFactory)` (method), 212
`make () (dmx.bstats.gamma.GammaAccumulatorFactory)` (method), 438
`make () (dmx.bstats.gamma.GammaAccumulatorFactory)` (method), 221
`make () (dmx.bstats.gaussian.GaussianEstimatorAccumulatorFactory)` (method), 448
`make () (dmx.bstats.gaussian.GaussianEstimatorAccumulatorFactory)` (method), 355
`make () (dmx.bstats.geometric.GeometricAccumulatorFactory)` (method), 450
`make () (dmx.bstats.geometric.GeometricAccumulatorFactory)` (method), 224
`make () (dmx.bstats.ignored.IgnoredAccumulatorFactory)` (method), 489
`make () (dmx.bstats.ignored.IgnoredAccumulatorFactory)` (method), 294
`make () (dmx.bstats.inrange.IntegerCategoricalAccumulatorFactory)` (method), 457
`make () (dmx.bstats.inrange.IntegerCategoricalAccumulatorFactory)` (method), 362
`make () (dmx.bstats.mixture.MixtureEstimatorAccumulatorFactory)` (method), 499
`make () (dmx.bstats.mixture.MixtureEstimatorAccumulatorFactory)` (method), 231
`make () (dmx.bstats.nulldist.NullAccumulatorFactory)` (method), 501
`make () (dmx.bstats.nulldist.NullAccumulatorFactory)` (method), 239
`make () (dmx.bstats.optional.OptionalEstimatorAccumulatorFactory)` (method), 511
`make () (dmx.bstats.optional.OptionalEstimatorAccumulatorFactory)` (method), 534
`make () (dmx.bstats.pdist.ProbabilityDistributionFactory)` (method), 379
`make () (dmx.bstats.pdist.ProbabilityDistributionFactory)` (method), 605
`make () (dmx.bstats.pdist.StatisticAccumulatorFactory)` (method), 383
`make () (dmx.bstats.pdist.StatisticAccumulatorFactory)` (method), 611
`make () (dmx.bstats.poisson.PoissonEstimatorAccumulatorFactory)` (method), 467
`make () (dmx.bstats.poisson.PoissonEstimatorAccumulatorFactory)` (method), 542
`make () (dmx.bstats.sequence.SequenceEstimatorAccumulatorFactory)` (method), 518
`make () (dmx.bstats.sequence.SequenceEstimatorAccumulatorFactory)` (method), 547
`make () (dmx.bstats.setdist.BernoulliSetAccumulatorFactory)` (method), 520
`make () (dmx.bstats.setdist.BernoulliSetAccumulatorFactory)` (method), 553
`make () (dmx.stats.dirac_length.DiracMixtureAccumulatorFactory)` (method), 190
`make () (dmx.stats.dirac_length.DiracMixtureAccumulatorFactory)` (method), 560

`make()` (*dmx.torch_stats.geometric.GeometricAccumulatorFactory* method), 565
`make()` (*dmx.torch_stats.heterogenous_mixture.HeterogeneousMultivariateBinomial.BinomialAccumulatorFactory* method), 624
`make()` (*dmx.torch_stats.hmm.HiddenMarkovAccumulatorFactory* method), 630
`make()` (*dmx.torch_stats.int_plsi.IntegerPLSIAccumulatorFactory* method), 638
`make()` (*dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulatorFactory* method), 572
`make()` (*dmx.torch_stats.intrange.IntegerCategoricalAccumulatorFactory* method), 580
`make()` (*dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulatorFactory* method), 587
`make()` (*dmx.torch_stats.jmixture.JointMixtureEstimatorAccumulatorFactory* method), 646
`make()` (*dmx.torch_stats.mixture.MixtureAccumulatorFactory* method), 648
`make()` (*dmx.torch_stats.mvn.MultivariateGaussianAccumulatorFactory* method), 593
`make()` (*dmx.torch_stats.null_dist.NullAccumulatorFactory* method), 615
`make()` (*dmx.torch_stats.pdist.TorchStatisticAccumulatorFactory* method), 531
`make()` (*dmx.torch_stats.poisson.PoissonAccumulatorFactory* method), 599
`make()` (*dmx.torch_stats.sequence.SequenceAccumulatorFactory* method), 619
`make()` (*in module dmx.utils.vector*), 686
`make_pdf()` (*in module dmx.utils.vector*), 687
`map_to_integers()` (*in module dmx.utils.optsutil*), 679
`MarkovChainDistribution` (class in *dmx.stats.markovchain*), 109
`MarkovChainEstimator` (class in *dmx.stats.markovchain*), 113
`MarkovChainSampler` (class in *dmx.stats.markovchain*), 116
`mat_inv()` (*in module dmx.utils.vector*), 687
`matrix_log_posteriors()` (*in module dmx.utils.vector*), 687
`max_val` (*dmx.stats.binomial.BinomialEstimator* attribute), 22
`max_val` (*dmx.stats.int_spike.SpikeAndSlabDistribution* attribute), 40
`max_val` (*dmx.stats.int_spike.SpikeAndSlabEstimator* attribute), 43
`max_val` (*dmx.stats.intmultinomial.IntegerMultinomialDistribution* attribute), 97
`max_val` (*dmx.stats.intmultinomial.IntegerMultinomialEstimator* attribute), 100
`max_val` (*dmx.stats.intrange.IntegerCategoricalDistribution* attribute), 35
`max_val` (*dmx.stats.intrange.IntegerCategoricalEstimator* attribute), 37
`max_val` (*dmx.torch_stats.binomial.BinomialAccumulator* attribute), 532
`max_val` (*dmx.torch_stats.binomial.BinomialEstimator* attribute), 538
`max_val` (*dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator* attribute), 570
`max_val` (*dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution* attribute), 572
`max_val` (*dmx.torch_stats.intmultinomial.IntegerMultinomialEstimator* attribute), 576
`max_val` (*dmx.torch_stats.intmultinomial.IntegerMultinomialSampler* attribute), 577
`max_val` (*dmx.torch_stats.intrange.IntegerCategoricalAccumulator* attribute), 578
`max_val` (*dmx.torch_stats.intrange.IntegerCategoricalDistribution* attribute), 580
`max_val` (*dmx.torch_stats.intrange.IntegerCategoricalEstimator* attribute), 583
`max_val` (*dmx.torch_stats.intrange.IntegerCategoricalSampler* attribute), 584
`max_value` (*dmx.stats.int_markovchain.IntegerMarkovChainAccumulator* attribute), 335
`maximum()` (*in module dmx.utils.vector*), 688
`merge()` (*dmx.utils.automatic.DatumNode* method), 654
`min_prob` (*dmx.stats.int_edit_setdist.IntegerBernoulliEditEstimator* attribute), 203
`min_prob` (*dmx.stats.intsetdist.IntegerBernoulliSetEstimator* attribute), 90
`min_prob` (*dmx.stats.setdist.BernoulliSetDistribution* attribute), 93
`min_prob` (*dmx.stats.setdist.BernoulliSetEstimator* attribute), 95
`min_prob` (*dmx.torch_stats.intsetdist.IntegerBernoulliSetEstimator* attribute), 590
`min_val` (*dmx.stats.binomial.BinomialDistribution* attribute), 20
`min_val` (*dmx.stats.binomial.BinomialEstimator* attribute), 23
`min_val` (*dmx.stats.int_spike.SpikeAndSlabDistribution* attribute), 40
`min_val` (*dmx.stats.int_spike.SpikeAndSlabEstimator* attribute), 42
`min_val` (*dmx.stats.intmultinomial.IntegerMultinomialDistribution* attribute), 97
`min_val` (*dmx.stats.intmultinomial.IntegerMultinomialEstimator* attribute), 100
`min_val` (*dmx.stats.intrange.IntegerCategoricalDistribution* attribute), 35

<code>min_val</code> (<code>dmx.stats.inrange.IntegerCategoricalEstimator</code> attribute), 37	<code>MixtureEstimator</code> (class in <code>dmx.torch_stats.mixture</code>), 651
<code>min_val</code> (<code>dmx.torch_stats.binomial.BinomialAccumulator</code> attribute), 532	<code>MixtureEstimatorAccumulator</code> (class in <code>dmx.bstats.mixture</code>), 496
<code>min_val</code> (<code>dmx.torch_stats.binomial.BinomialAccumulatorFactory</code> attribute), 534	<code>MixtureEstimatorAccumulatorFactory</code> (class in <code>dmx.bstats.mixture</code>), 498
<code>min_val</code> (<code>dmx.torch_stats.binomial.BinomialDistribution</code> attribute), 536	<code>MixtureSampler</code> (class in <code>dmx.bstats.mixture</code>), 499
<code>min_val</code> (<code>dmx.torch_stats.binomial.BinomialEstimator</code> attribute), 538	<code>MixtureSampler</code> (class in <code>dmx.stats.mixture</code>), 153
<code>min_val</code> (<code>dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator</code> attribute), 570	<code>MixtureSampler</code> (class in <code>dmx.torch_stats.mixture</code>), 651
<code>min_val</code> (<code>dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulatorFactory</code> attribute), 572	<code>MixtureTorchEncodedSequence</code> (class in <code>dmx.torch_stats.mixture</code>), 652
<code>min_val</code> (<code>dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution</code> attribute), 573	<code>model_device()</code> (<code>dmx.torch_stats.pdist.TorchProbabilityDistribution</code> method), 538
<code>min_val</code> (<code>dmx.torch_stats.intmultinomial.IntegerMultinomialEstimator</code> attribute), 575	<code>model_log_density()</code> (<code>dmx.bstats.composite.CompositeEstimator</code> method), 471
<code>min_val</code> (<code>dmx.torch_stats.intmultinomial.IntegerMultinomialSampler</code> attribute), 577	<code>model_log_density()</code> (<code>dmx.bstats.dpm.DirichletProcessMixtureEstimator</code> method), 486
<code>min_val</code> (<code>dmx.torch_stats.inrange.IntegerCategoricalAccumulator</code> attribute), 578	<code>model_log_density()</code> (<code>dmx.bstats.mixture.MixtureEstimator</code> method), 496
<code>min_val</code> (<code>dmx.torch_stats.inrange.IntegerCategoricalAccumulatorFactory</code> attribute), 580	<code>model_log_density()</code> (<code>dmx.bstats.sequence.SequenceEstimator</code> method), 516
<code>min_val</code> (<code>dmx.torch_stats.inrange.IntegerCategoricalDistribution</code> attribute), 581	module
<code>min_val</code> (<code>dmx.torch_stats.inrange.IntegerCategoricalEstimator</code> attribute), 583	<code>dmx.bstats</code> , 372
<code>min_val</code> (<code>dmx.torch_stats.inrange.IntegerCategoricalSampler</code> attribute), 584	<code>dmx.bstats.bernoulli</code> , 407
<code>missing_value</code> (<code>dmx.stats.optional.OptionalDistribution</code> attribute), 137	<code>dmx.bstats.bestimation</code> , 383
<code>missing_value</code> (<code>dmx.stats.optional.OptionalEstimator</code> attribute), 139	<code>dmx.bstats.beta</code> , 389
<code>missing_value_is_nan</code> (<code>dmx.stats.optional.OptionalDistribution</code> attribute), 137	<code>dmx.bstats.catdirichlet</code> , 391
<code>MixtureAccumulator</code> (class in <code>dmx.torch_stats.mixture</code>), 647	<code>dmx.bstats.categorical</code> , 413
<code>MixtureAccumulatorFactory</code> (class in <code>dmx.torch_stats.mixture</code>), 648	<code>dmx.bstats.composite</code> , 467
<code>MixtureDataEncoder</code> (class in <code>dmx.bstats.mixture</code>), 492	<code>dmx.bstats.conditional</code> , 474
<code>MixtureDataEncoder</code> (class in <code>dmx.torch_stats.mixture</code>), 649	<code>dmx.bstats.dirac</code> , 419
<code>MixtureDistribution</code> (class in <code>dmx.bstats.mixture</code>), 493	<code>dmx.bstats.dirichlet</code> , 394
<code>MixtureDistribution</code> (class in <code>dmx.stats.mixture</code>), 146	<code>dmx.bstats.dmvn</code> , 425
<code>MixtureDistribution</code> (class in <code>dmx.torch_stats.mixture</code>), 649	<code>dmx.bstats.dpm</code> , 480
<code>MixtureEncodedData</code> (class in <code>dmx.bstats.mixture</code>), 495	<code>dmx.bstats.exponential</code> , 431
<code>MixtureEncodedDataSequence</code> (in module <code>dmx.bstats.mixture</code>), 495	<code>dmx.bstats.gamma</code> , 436
<code>MixtureEstimator</code> (class in <code>dmx.bstats.mixture</code>), 495	<code>dmx.bstats.gaussian</code> , 442
<code>MixtureEstimator</code> (class in <code>dmx.stats.mixture</code>), 150	<code>dmx.bstats.geometric</code> , 448
	<code>dmx.bstats.ignored</code> , 487
	<code>dmx.bstats.inrange</code> , 454
	<code>dmx.bstats.mixture</code> , 492
	<code>dmx.bstats.mvngamma</code> , 401
	<code>dmx.bstats.normgamma</code> , 403
	<code>dmx.bstats.nullldist</code> , 499
	<code>dmx.bstats.optional</code> , 505
	<code>dmx.bstats.pdist</code> , 372
	<code>dmx.bstats.poisson</code> , 461
	<code>dmx.bstats.sequence</code> , 512
	<code>dmx.bstats.setdist</code> , 519
	<code>dmx.bstats.symdirichlet</code> , 405

dmx.mpi4py.bstats, 706
 dmx.mpi4py.stats, 703
 dmx.mpi4py.utils, 708
 dmx.mpi4py.utils.automatic, 709
 dmx.mpi4py.utils.bestimation, 710
 dmx.mpi4py.utils.estimation, 711
 dmx.mpi4py.utils.humap, 714
 dmx.mpi4py.utils.optsutil, 715
 dmx.stats, 7
 dmx.stats.dirac_length, 186
 dmx.stats.dmvn_mixture, 242
 dmx.stats.gmm, 254
 dmx.stats.hidden_association, 294
 dmx.stats.icltree, 301
 dmx.stats.int_edit_setdist, 196
 dmx.stats.int_edit_stepsetdist, 205
 dmx.stats.int_hidden_association, 307
 dmx.stats.int_hidden_markov, 318
 dmx.stats.int_markovchain, 334
 dmx.stats.int_plsi, 266
 dmx.stats.lda, 279
 dmx.stats.look_back_hmm, 345
 dmx.stats.null_dist, 210
 dmx.stats.rdd_sampler, 370
 dmx.stats.select, 216
 dmx.stats.sparse_markov_transform, 353
 dmx.stats.spearman_rho, 221
 dmx.stats.ss_mixture, 288
 dmx.stats.tree_hmm, 360
 dmx.stats.vmf, 228
 dmx.stats.weighted, 236
 dmx.torch_stats, 525
 dmx.torch_stats.binomial, 532
 dmx.torch_stats.composite, 603
 dmx.torch_stats.conditional, 608
 dmx.torch_stats.dmvn, 539
 dmx.torch_stats.exponential, 545
 dmx.torch_stats.gamma, 551
 dmx.torch_stats.gaussian, 558
 dmx.torch_stats.geometric, 564
 dmx.torch_stats.heterogenous_mixture, 622
 dmx.torch_stats.hmm, 627
 dmx.torch_stats.int_plsi, 637
 dmx.torch_stats.intmultinomial, 569
 dmx.torch_stats.intrange, 578
 dmx.torch_stats.intsetdist, 585
 dmx.torch_stats.jmixture, 642
 dmx.torch_stats.mixture, 647
 dmx.torch_stats.mvn, 591
 dmx.torch_stats.null_dist, 613
 dmx.torch_stats.pdist, 525
 dmx.torch_stats.poisson, 597
 dmx.torch_stats.sequence, 617
 dmx.torch_utils, 692
 dmx.torch_utils.estimation, 693
 dmx.torch_utils.optsutil, 696
 dmx.torch_utils.vector, 697
 dmx.utils, 652
 dmx.utils.automatic, 652
 dmx.utils.builder, 659
 dmx.utils.estimation, 660
 dmx.utils.htsne, 667
 dmx.utils.humap, 674
 dmx.utils.metrics, 675
 dmx.utils.optsutil, 677
 dmx.utils.pvalues, 680
 dmx.utils.special, 681
 dmx.utils.vector, 684
 moment() (dmx.bstats.bernoulli.BernoulliDistribution method), 409
 moment() (dmx.bstats.geometric.GeometricDistribution method), 452
 moment() (dmx.bstats.intrange.IntegerCategoricalDistribution method), 458
 moment() (dmx.bstats.poisson.PoissonDistribution method), 463
 moments() (dmx.bstats.nulldist.NullDistribution method), 502
 mpe() (in module dmx.bstats.dirichlet), 401
 mpe() (in module dmx.stats.lda), 286
 mpe_update() (in module dmx.stats.lda), 287
 mu (dmx.stats.dmvn.DiagonalGaussianDistribution attribute), 65
 mu (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution attribute), 247
 mu (dmx.stats.gaussian.GaussianDistribution attribute), 45
 mu (dmx.stats.gmm.GaussianMixtureDistribution attribute), 259
 mu (dmx.stats.log_gaussian.LogGaussianDistribution attribute), 60
 mu (dmx.stats.mvn.MultivariateGaussianDistribution attribute), 71
 mu (dmx.stats.vmf.VonMisesFisherDistribution attribute), 232
 mu (dmx.torch_stats.mvn.MultivariateGaussianDistribution attribute), 594
 MultinomialDistribution (class in dmx.stats.catmultinomial), 103
 MultinomialEstimator (class in dmx.stats.catmultinomial), 106
 MultinomialSampler (class in dmx.stats.catmultinomial), 108
 MultivariateGaussianAccumulator (class in dmx.torch_stats.mvn), 592
 MultivariateGaussianAccumulatorFactory (class in dmx.torch_stats.mvn), 593
 MultivariateGaussianDataEncoder (class in

<i>dmx.torch_stats.mvn</i>), 593		<i>name</i> (<i>dmx.stats.conditional.ConditionalDistributionEstimator</i> attribute), 133
<i>MultivariateGaussianDistribution</i> (class in <i>dmx.stats.mvn</i>), 71		<i>name</i> (<i>dmx.stats.dirac_length.DiracMixtureAccumulator</i> attribute), 187
<i>MultivariateGaussianDistribution</i> (class in <i>dmx.torch_stats.mvn</i>), 594		<i>name</i> (<i>dmx.stats.dirac_length.DiracMixtureAccumulatorFactory</i> attribute), 190
<i>MultivariateGaussianEstimator</i> (class in <i>dmx.stats.mvn</i>), 73		<i>name</i> (<i>dmx.stats.dirac_length.DiracMixtureDistribution</i> attribute), 191
<i>MultivariateGaussianEstimator</i> (class in <i>dmx.torch_stats.mvn</i>), 596		<i>name</i> (<i>dmx.stats.dirac_length.DiracMixtureEstimator</i> attribute), 195
<i>MultivariateGaussianSampler</i> (class in <i>dmx.stats.mvn</i>), 75		<i>name</i> (<i>dmx.stats.dirichlet.DirichletDistribution</i> attribute), 76
<i>MultivariateGaussianSampler</i> (class in <i>dmx.torch_stats.mvn</i>), 597		<i>name</i> (<i>dmx.stats.dirichlet.DirichletEstimator</i> attribute), 79
<i>MultivariateGaussianTorchSequence</i> (class in <i>dmx.torch_stats.mvn</i>), 597		<i>name</i> (<i>dmx.stats.dmvn.DiagonalGaussianDistribution</i> attribute), 65
<i>MultivariateNormalGammaDistribution</i> (class in <i>dmx.bstats.mvngamma</i>), 401		<i>name</i> (<i>dmx.stats.dmvn.DiagonalGaussianEstimator</i> attribute), 68
<i>MultivariateNormalGammaSampler</i> (class in <i>dmx.bstats.mvngamma</i>), 403		<i>name</i> (<i>dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator</i> attribute), 243
N		<i>name</i> (<i>dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulatorFactory</i> attribute), 246
<i>n</i> (<i>dmx.stats.binomial.BinomialDistribution</i> attribute), 20		<i>name</i> (<i>dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution</i> attribute), 247
<i>n</i> (<i>dmx.torch_stats.binomial.BinomialDistribution</i> attribute), 535		<i>name</i> (<i>dmx.stats.dmvn_mixture.DiagonalGaussianMixtureEstimator</i> attribute), 252
<i>n_states</i> (<i>dmx.stats.hidden_markov.HiddenMarkovModelDistribution</i> attribute), 177		<i>name</i> (<i>dmx.stats.exponential.ExponentialDistribution</i> attribute), 50
<i>n_states</i> (<i>dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution</i> attribute), 327		<i>name</i> (<i>dmx.stats.exponential.ExponentialEstimator</i> attribute), 53
<i>n_states</i> (<i>dmx.torch_stats.hmm.HiddenMarkovModelDistribution</i> attribute), 632		<i>name</i> (<i>dmx.stats.gamma.GammaDistribution</i> attribute), 55
<i>n_topics</i> (<i>dmx.stats.hidden_markov.HiddenMarkovModelDistribution</i> attribute), 177		<i>name</i> (<i>dmx.stats.gamma.GammaEstimator</i> attribute), 58
<i>n_topics</i> (<i>dmx.torch_stats.hmm.HiddenMarkovModelDistribution</i> attribute), 632		<i>name</i> (<i>dmx.stats.gaussian.GaussianDistribution</i> attribute), 45
<i>n_words</i> (<i>dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution</i> attribute), 326		<i>name</i> (<i>dmx.stats.gaussian.GaussianEstimator</i> attribute), 48
<i>name</i> (<i>dmx.stats.binomial.BinomialDistribution</i> attribute), 20		<i>name</i> (<i>dmx.stats.geometric.GeometricDistribution</i> attribute), 25
<i>name</i> (<i>dmx.stats.binomial.BinomialEstimator</i> attribute), 23		<i>name</i> (<i>dmx.stats.geometric.GeometricEstimator</i> attribute), 28
<i>name</i> (<i>dmx.stats.categorical.CategoricalDistribution</i> attribute), 82		<i>name</i> (<i>dmx.stats.gmm.GaussianMixtureAccumulator</i> attribute), 255
<i>name</i> (<i>dmx.stats.categorical.CategoricalEstimator</i> attribute), 85		<i>name</i> (<i>dmx.stats.gmm.GaussianMixtureAccumulatorFactory</i> attribute), 259
<i>name</i> (<i>dmx.stats.catmultinomial.MultinomialDistribution</i> attribute), 104		<i>name</i> (<i>dmx.stats.gmm.GaussianMixtureDistribution</i> attribute), 260
<i>name</i> (<i>dmx.stats.catmultinomial.MultinomialEstimator</i> attribute), 107		<i>name</i> (<i>dmx.stats.gmm.GaussianMixtureEstimator</i> attribute), 264
<i>name</i> (<i>dmx.stats.composite.CompositeDistribution</i> attribute), 118		<i>name</i> (<i>dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution</i> attribute), 154
<i>name</i> (<i>dmx.stats.composite.CompositeEstimator</i> attribute), 121		<i>name</i> (<i>dmx.stats.heterogeneous_mixture.HeterogeneousMixtureEstimator</i> attribute), 159
<i>name</i> (<i>dmx.stats.conditional.ConditionalDistribution</i> attribute), 130		<i>name</i> (<i>dmx.stats.hidden_association.HiddenAssociationDistribution</i>

attribute), 298
 name (dmx.stats.hidden_association.HiddenAssociationEstimator attribute), 300
 name (dmx.stats.hidden_markov.HiddenMarkovEstimator attribute), 182
 name (dmx.stats.hidden_markov.HiddenMarkovModelDistribution attribute), 178
 name (dmx.stats.hmixture.HierarchicalMixtureDistribution attribute), 169
 name (dmx.stats.hmixture.HierarchicalMixtureEstimator attribute), 174
 name (dmx.stats.ictree.ICLTreeDistribution attribute), 304
 name (dmx.stats.ignored.IgnoredDistribution attribute), 142
 name (dmx.stats.ignored.IgnoredEstimator attribute), 144
 name (dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulatorFactory attribute), 199
 name (dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution attribute), 200
 name (dmx.stats.int_edit_setdist.IntegerBernoulliEditEstimator attribute), 203
 name (dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution attribute), 312
 name (dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator attribute), 315
 name (dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator attribute), 320
 name (dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulatorFactory attribute), 323
 name (dmx.stats.int_hidden_markov.IntegerHiddenMarkovEstimator attribute), 325
 name (dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution attribute), 327
 name (dmx.stats.int_markovchain.IntegerMarkovChainAccumulator attribute), 336
 name (dmx.stats.int_markovchain.IntegerMarkovChainAccumulatorFactory attribute), 338
 name (dmx.stats.int_markovchain.IntegerMarkovChainDistribution attribute), 340
 name (dmx.stats.int_markovchain.IntegerMarkovChainEstimator attribute), 343
 name (dmx.stats.int_plsi.IntegerPLSIAccumulator attribute), 267
 name (dmx.stats.int_plsi.IntegerPLSIAccumulatorFactory attribute), 270
 name (dmx.stats.int_plsi.IntegerPLSIDistribution attribute), 272
 name (dmx.stats.int_plsi.IntegerPLSEstimator attribute), 275
 name (dmx.stats.int_spike.SpikeAndSlabDistribution attribute), 41
 name (dmx.stats.int_spike.SpikeAndSlabEstimator attribute), 43
 name (dmx.stats.intmultinomial.IntegerMultinomialDistribution attribute), 98
 name (dmx.stats.intmultinomial.IntegerMultinomialEstimator attribute), 100
 name (dmx.stats.inrange.IntegerCategoricalDistribution attribute), 35
 name (dmx.stats.inrange.IntegerCategoricalEstimator attribute), 37
 name (dmx.stats.intsetdist.IntegerBernoulliSetDistribution attribute), 88
 name (dmx.stats.intsetdist.IntegerBernoulliSetEstimator attribute), 90
 name (dmx.stats.jmixture.JointMixtureDistribution attribute), 163
 name (dmx.stats.jmixture.JointMixtureEstimator attribute), 166
 name (dmx.stats.log_gaussian.LogGaussianDistribution attribute), 61
 name (dmx.stats.log_gaussian.LogGaussianEstimator attribute), 63
 name (dmx.stats.markovchain.MarkovChainDistribution attribute), 110
 name (dmx.stats.markovchain.MarkovChainEstimator attribute), 114
 name (dmx.stats.mixture.MixtureDistribution attribute), 147
 name (dmx.stats.mixture.MixtureEstimator attribute), 151
 name (dmx.stats.mvn.MultivariateGaussianDistribution attribute), 71
 name (dmx.stats.mvn.MultivariateGaussianEstimator attribute), 74
 name (dmx.stats.null_dist.NullDistribution attribute), 213
 name (dmx.stats.null_dist.NullEstimator attribute), 215
 name (dmx.stats.optional.OptionalDistribution attribute), 137
 name (dmx.stats.optional.OptionalEstimator attribute), 140
 name (dmx.stats.poisson.PoissonDistribution attribute), 30
 name (dmx.stats.poisson.PoissonEstimator attribute), 32
 name (dmx.stats.sequence.SequenceDistribution attribute), 123
 name (dmx.stats.sequence.SequenceEstimator attribute), 127
 name (dmx.stats.setdist.BernoulliSetDistribution attribute), 92
 name (dmx.stats.setdist.BernoulliSetEstimator attribute), 95
 name (dmx.stats.spearman_rho.SpearmanRankingAccumulator attribute), 222
 name (dmx.stats.spearman_rho.SpearmanRankingAccumulatorFactory attribute), 224
 name (dmx.stats.spearman_rho.SpearmanRankingDistribution attribute), 225

<code>name (dmx.stats.spearman_rho.SpearmanRankingEstimatorNullAccumulator (class in dmx.torch_stats.null_dist), attribute), 227</code>	613
<code>name (dmx.stats.ss_mixture.SemiSupervisedMixtureDistributionNullAccumulatorFactory (class in attribute), 289</code>	<code>dmx.bstats.null_dist), 501</code>
<code>name (dmx.stats.ss_mixture.SemiSupervisedMixtureEstimatorNullAccumulatorFactory (class in attribute), 291</code>	<code>dmx.stats.null_dist), 212</code>
<code>name (dmx.stats.vmf.VonMisesFisherAccumulator (class in attribute), 228</code>	<code>dmx.torch_stats.null_dist), 614</code>
<code>name (dmx.stats.vmf.VonMisesFisherAccumulatorFactory (class in attribute), 230</code>	<code>dmx.bstats.null_dist), 501</code>
<code>name (dmx.stats.vmf.VonMisesFisherDistribution (class in attribute), 231</code>	<code>dmx.stats.null_dist), 212</code>
<code>name (dmx.stats.vmf.VonMisesFisherEstimator (class in attribute), 234</code>	<code>dmx.torch_stats.null_dist), 615</code>
<code>name (dmx.stats.weighted.WeightedAccumulator (class in attribute), 236</code>	<code>dmx.bstats.null_dist), 501</code>
<code>name (dmx.stats.weighted.WeightedAccumulatorFactory (class in attribute), 238</code>	<code>dmx.stats.null_dist), 213</code>
<code>name (dmx.stats.weighted.WeightedDistribution (class in attribute), 240</code>	<code>dmx.torch_stats.null_dist), 615</code>
<code>name (dmx.stats.weighted.WeightedEstimator (class in attribute), 241</code>	<code>dmx.bstats.null_dist), 504</code>
<code>name (dmx.torch_stats.intmultinomial.IntegerMultinomialEstimator (class in attribute), 576</code>	<code>dmx.stats.null_dist), 214</code>
<code>nan_count (dmx.utils.automatic.DatumNode attribute), 652</code>	<code>dmx.torch_stats.null_dist), 616</code>
<code>neg_count (dmx.utils.automatic.DatumNode attribute), 653</code>	<code>dmx.bstats.null_dist), 504</code>
<code>new_seed() (dmx.bstats.pdist.DistributionSampler method), 374</code>	<code>dmx.stats.null_dist), 215</code>
<code>new_seed() (dmx.stats.pdist.DistributionSampler method), 13</code>	<code>dmx.torch_stats.null_dist), 617</code>
<code>next_rng (dmx.stats.int_edit_setdist.IntegerBernoulliEditSampler attribute), 204</code>	<code>dmx.bstats.null_dist), 504</code>
<code>nlog_sum (dmx.stats.setdist.BernoulliSetDistribution attribute), 93</code>	<code>dmx.stats.null_dist), 215</code>
<code>no_default (dmx.stats.categorical.CategoricalDistribution attribute), 82</code>	<code>dmx.torch_stats.null_dist), 617</code>
<code>nobs (dmx.torch_stats.gamma.GammaAccumulator attribute), 551</code>	<code>dmx.bstats.null_dist), 504</code>
<code>non_k (dmx.stats.int_spike.SpikeAndSlabSampler attribute), 44</code>	<code>dmx.stats.null_dist), 215</code>
<code>none_count (dmx.utils.automatic.DatumNode attribute), 652</code>	<code>dmx.torch_stats.null_dist), 617</code>
<code>NormalGammaDistribution (class in dmx.bstats.normgamma), 403</code>	<code>dmx.bstats.null_dist), 504</code>
<code>NormalGammaSampler (class in dmx.bstats.normgamma), 405</code>	<code>dmx.stats.null_dist), 215</code>
<code>null_len_dist (dmx.stats.sequence.SequenceDistribution attribute), 124</code>	<code>dmx.torch_stats.null_dist), 617</code>
<code>null_sampler (dmx.stats.ignored.IgnoredSampler attribute), 145</code>	<code>dmx.bstats.null_dist), 504</code>
<code>NullAccumulator (class in dmx.bstats.null_dist), 499</code>	<code>dmx.stats.null_dist), 215</code>
<code>NullAccumulator (class in dmx.stats.null_dist), 210</code>	<code>dmx.torch_stats.null_dist), 617</code>

attribute), 162
 num_docs (dmx.stats.int_plsi.IntegerPLSIAccumulator attribute), 266
 num_docs (dmx.stats.int_plsi.IntegerPLSIAccumulatorFactory attribute), 270
 num_docs (dmx.stats.int_plsi.IntegerPLSIDistribution attribute), 272
 num_docs (dmx.stats.int_plsi.IntegerPLSEstimator attribute), 275
 num_features (dmx.stats.ictree.ICLTreeDistribution attribute), 304
 num_keys (dmx.stats.markovchain.MarkovChainDistribution attribute), 111
 num_levels (dmx.stats.categorical.CategoricalSampler attribute), 87
 num_mixtures (dmx.stats.hmixture.HierarchicalMixtureDistribution attribute), 168
 num_mixtures (dmx.stats.hmixture.HierarchicalMixtureEstimator attribute), 173
 num_required (dmx.stats.setdist.BernoulliSetDistribution attribute), 93
 num_states (dmx.stats.hidden_markov.HiddenMarkovSampler attribute), 184
 num_states (dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution attribute), 311
 num_states (dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator attribute), 314
 num_states (dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator attribute), 319
 num_states (dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulatorFactory attribute), 323
 num_states (dmx.stats.int_hidden_markov.IntegerHiddenMarkovEstimator attribute), 325
 num_states (dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler attribute), 325
 num_states (dmx.stats.int_hidden_markov.IntegerHiddenMarkovSplice attribute), 329
 num_states (dmx.stats.int_plsi.IntegerPLSIAccumulator attribute), 266
 num_states (dmx.stats.int_plsi.IntegerPLSIAccumulatorFactory attribute), 270
 num_states (dmx.stats.int_plsi.IntegerPLSIDistribution attribute), 272
 num_states (dmx.stats.int_plsi.IntegerPLSEstimator attribute), 275
 num_states (dmx.stats.tree_hmm.TreeHiddenMarkovModel attribute), 364
 num_topics (dmx.stats.hmixture.HierarchicalMixtureDistribution attribute), 168
 num_vals (dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator attribute), 197
 num_vals (dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulatorFactory attribute), 199
 num_vals (dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution attribute), 201
 num_vals (dmx.stats.int_edit_setdist.IntegerBernoulliEditEstimator attribute), 203
 num_vals (dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator attribute), 314
 num_vals (dmx.stats.int_plsi.IntegerPLSIAccumulator attribute), 266
 num_vals (dmx.stats.int_plsi.IntegerPLSIAccumulatorFactory attribute), 270
 num_vals (dmx.stats.int_plsi.IntegerPLSIDistribution attribute), 271
 num_vals (dmx.stats.int_plsi.IntegerPLSEstimator attribute), 274
 num_vals (dmx.stats.int_spike.SpikeAndSlabDistribution attribute), 40
 num_vals (dmx.stats.intmultinomial.IntegerMultinomialDistribution attribute), 98
 num_vals (dmx.stats.intrange.IntegerCategoricalDistribution attribute), 35
 num_vals (dmx.stats.intsetdist.IntegerBernoulliSetEstimator attribute), 90
 num_vals (dmx.stats.sparse_markov_transform.SparseMarkovAssociationEstimator attribute), 358
 num_vals (dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution attribute), 573
 num_vals (dmx.stats.intrange.IntegerCategoricalDistribution attribute), 581
 num_vals (dmx.stats.intsetdist.IntegerBernoulliSetAccumulator attribute), 586
 num_vals (dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulatorFactory attribute), 587
 num_vals (dmx.stats.intsetdist.IntegerBernoulliSetEstimator attribute), 590
 num_vals (dmx.torch_stats.intsetdist.IntegerBernoulliSetSampler attribute), 591
 num_vals (dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution attribute), 311
 num_vals1 (dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator attribute), 315
 num_vals2 (dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution attribute), 311
 num_vals2 (dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator attribute), 315
 num_values (dmx.stats.int_markovchain.IntegerMarkovChainDistribution attribute), 340
 num_values (dmx.stats.int_markovchain.IntegerMarkovChainEstimator attribute), 343
 num_words (dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator attribute), 319
 num_words (dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulatorFactory attribute), 323
 num_words (dmx.stats.int_hidden_markov.IntegerHiddenMarkovEstimator attribute), 325
 numba.numba_welch() (in module dmx.stats.int_hidden_markov), 333
 numba.numba_welch() (in module

dmx.stats.look_back_hmm), 351
numba_baum_welch() (in module *dmx.stats.tree_hmm*), 367
numba_baum_welch2() (in module *dmx.stats.look_back_hmm*), 351
numba_baum_welch_alphas() (in module *dmx.stats.int_hidden_markov*), 334
numba_baum_welch_alphas() (in module *dmx.stats.look_back_hmm*), 352
numba_enc (*dmx.stats.int_hidden_association.IntegerHiddenAssociationEncodedDataSequence* attribute), 313
numba_initialize() (in module *dmx.stats.tree_hmm*), 367
numba_posteriors() (in module *dmx.stats.tree_hmm*), 368
numba_seq_log_density() (in module *dmx.stats.int_hidden_association*), 316
numba_seq_log_density() (in module *dmx.stats.look_back_hmm*), 352
numba_seq_log_density() (in module *dmx.stats.tree_hmm*), 369
numba_seq_update() (in module *dmx.stats.int_hidden_association*), 317
numba_viterbi() (in module *dmx.stats.tree_hmm*), 369

O

obj_count (*dmx.utils.automatic.DatumNode* attribute), 653
obs_samplers (*dmx.stats.hidden_markov.HiddenMarkovSampler* attribute), 184
ones() (in module *dmx.torch_utils.vector*), 699
optimize() (in module *dmx.bstats.bestimation*), 387
optimize() (in module *dmx.torch_utils.estimation*), 693
optimize() (in module *dmx.utils.estimation*), 664
optimize_mp() (in module *dmx.torch_utils.estimation*), 695
optimize_mpi() (in module *dmx.mpi4py.utils.bestimation*), 710
optimize_mpi() (in module *dmx.mpi4py.utils.estimation*), 712
OptionalDataEncoder (class in *dmx.bstats.optional*), 505
OptionalDistribution (class in *dmx.bstats.optional*), 505
OptionalDistribution (class in *dmx.stats.optional*), 136
OptionalEncodedData (class in *dmx.bstats.optional*), 508
OptionalEstimator (class in *dmx.bstats.optional*), 508
OptionalEstimator (class in *dmx.stats.optional*), 139
OptionalEstimatorAccumulator (class in *dmx.bstats.optional*), 509
OptionalEstimatorAccumulatorFactory (class in *dmx.bstats.optional*), 511

OptionalSampler (class in *dmx.bstats.optional*), 511
OptionalSampler (class in *dmx.stats.optional*), 141
orig_log_edit_pmat (*dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution* attribute), 201
outer() (in module *dmx.utils.vector*), 689

P

p (*dmx.stats.binomial.BinomialDistribution* attribute), 20
p (*dmx.stats.dirac_length.DiracMixtureDistribution* attribute), 196
p (*dmx.stats.dirac_length.DiracMixtureSampler* attribute), 196
p (*dmx.stats.geometric.GeometricDistribution* attribute), 25
p (*dmx.stats.int_spike.SpikeAndSlabDistribution* attribute), 40
p (*dmx.stats.optional.OptionalDistribution* attribute), 136
p (*dmx.torch_stats.binomial.BinomialDistribution* attribute), 535
p_vec (*dmx.stats.intmultinomial.IntegerMultinomialDistribution* attribute), 97
p_vec (*dmx.stats.inrange.IntegerCategoricalDistribution* attribute), 34
p_vec (*dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution* attribute), 573
p_vec (*dmx.torch_stats.intmultinomial.IntegerMultinomialSampler* attribute), 577
p_vec (*dmx.torch_stats.inrange.IntegerCategoricalDistribution* attribute), 581
p_vec (*dmx.torch_stats.inrange.IntegerCategoricalSampler* attribute), 584
ParameterEstimator (class in *dmx.bstats.pdist*), 374
ParameterEstimator (class in *dmx.stats.pdist*), 17
parent (*dmx.utils.automatic.DatumNode* attribute), 652
partition_data() (in module *dmx.bstats.bestimation*), 388
partition_data() (in module *dmx.utils.estimation*), 666
partition_data_index() (in module *dmx.bstats.bestimation*), 389
partition_data_index() (in module *dmx.utils.estimation*), 666
pcnt (*dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator* attribute), 197
pcnt (*dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulator* attribute), 585
perms (*dmx.stats.spearman_rho.SpearmanRankingSampler* attribute), 227
pickle_on_master() (in module *dmx.mpi4py.utils.optsutil*), 715
pmap (*dmx.stats.categorical.CategoricalDistribution* attribute), 82
pmap (*dmx.stats.setdist.BernoulliSetDistribution* attribute), 93

`pmat` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution` (class in `dmx.stats.int_hidden_markov`), 326
`PoissonAccumulator` (class in `dmx.torch_stats.poisson`), 597
`PoissonAccumulatorFactory` (class in `dmx.torch_stats.poisson`), 599
`PoissonDataEncoder` (class in `dmx.bstats.poisson`), 461
`PoissonDataEncoder` (class in `dmx.torch_stats.poisson`), 600
`PoissonDistribution` (class in `dmx.bstats.poisson`), 461
`PoissonDistribution` (class in `dmx.stats.poisson`), 30
`PoissonDistribution` (class in `dmx.torch_stats.poisson`), 600
`PoissonEncodedData` (class in `dmx.bstats.poisson`), 464
`PoissonEstimator` (class in `dmx.bstats.poisson`), 464
`PoissonEstimator` (class in `dmx.stats.poisson`), 32
`PoissonEstimator` (class in `dmx.torch_stats.poisson`), 601
`PoissonEstimatorAccumulator` (class in `dmx.bstats.poisson`), 465
`PoissonEstimatorAccumulatorFactory` (class in `dmx.bstats.poisson`), 467
`PoissonSampler` (class in `dmx.bstats.poisson`), 467
`PoissonSampler` (class in `dmx.stats.poisson`), 33
`PoissonSampler` (class in `dmx.torch_stats.poisson`), 602
`PoissonTorchSequence` (class in `dmx.torch_stats.poisson`), 603
`polygamma_loc()` (in module `dmx.utils.special`), 683
`posterior()` (`dmx.bstats.mixture.MixtureDistribution` method), 494
`posterior()` (`dmx.stats.dirac_length.DiracMixtureDistribution` method), 192
`posterior()` (`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution` method), 249
`posterior()` (`dmx.stats.gmm.GaussianMixtureDistribution` method), 262
`posterior()` (`dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution` method), 156
`posterior()` (`dmx.stats.hmixture.HierarchicalMixtureDistribution` method), 171
`posterior()` (`dmx.stats.mixture.MixtureDistribution` method), 149
`posterior()` (`dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution` method), 289
`posterior()` (`dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDistribution` method), 625
`posterior()` (`dmx.torch_stats.mixture.MixtureDistribution` method), 650
`posterior()` (in module `dmx.utils.vector`), 689
`prepare_mixture_model()` (in module `dmx.utils.automatic`), 658
`prev_dist` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution` (class in `dmx.stats.int_hidden_association`), 311
`prev_estimator` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution` (class in `dmx.stats.int_hidden_association`), 314
`prior_covar` (`dmx.stats.dmvn.DiagonalGaussianEstimator` attribute), 68
`prior_covar` (`dmx.stats.mvn.MultivariateGaussianEstimator` attribute), 74
`prior_covar` (`dmx.torch_stats.dmvn.DiagonalGaussianEstimator` attribute), 544
`prior_covar` (`dmx.torch_stats.mvn.MultivariateGaussianEstimator` attribute), 596
`prior_mu` (`dmx.stats.dmvn.DiagonalGaussianEstimator` attribute), 68
`prior_mu` (`dmx.stats.mvn.MultivariateGaussianEstimator` attribute), 74
`prior_mu` (`dmx.torch_stats.dmvn.DiagonalGaussianEstimator` attribute), 544
`prior_mu` (`dmx.torch_stats.mvn.MultivariateGaussianEstimator` attribute), 596
`prob_mat` (`dmx.stats.int_plsi.IntegerPLSIDistribution` attribute), 271
`ProbabilityDistribution` (class in `dmx.bstats.pdist`), 375
`ProbabilityDistribution` (class in `dmx.stats.pdist`), 11
`ProbabilityDistributionFactory` (class in `dmx.bstats.pdist`), 379
`probs` (`dmx.stats.categorical.CategoricalSampler` attribute), 87
`probs` (`dmx.stats.spearman_rho.SpearmanRankingSampler` attribute), 227
`pseudo_count` (`dmx.stats.binomial.BinomialEstimator` attribute), 23
`pseudo_count` (`dmx.stats.categorical.CategoricalEstimator` attribute), 85
`pseudo_count` (`dmx.stats.catmultinomial.MultinomialEstimator` attribute), 106
`pseudo_count` (`dmx.stats.dirac_length.DiracMixtureEstimator` attribute), 104
`pseudo_count` (`dmx.stats.dirichlet.DirichletEstimator` attribute), 79
`pseudo_count` (`dmx.stats.dmvn.DiagonalGaussianEstimator` attribute), 68
`pseudo_count` (`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureEstimator` attribute), 252
`pseudo_count` (`dmx.stats.exponential.ExponentialEstimator` attribute), 52
`pseudo_count` (`dmx.stats.gamma.GammaEstimator` attribute), 57
`pseudo_count` (`dmx.stats.gaussian.GaussianEstimator` attribute), 48
`pseudo_count` (`dmx.stats.geometric.GeometricEstimator` attribute), 28
`pseudo_count` (`dmx.stats.gmm.GaussianMixtureEstimator` attribute), 264

pseudo_count (dmx.stats.heterogeneous_mixture.HeterogeneousMixtureEstimator attribute), 159
 pseudo_count (dmx.stats.hidden_association.HiddenAssociationEstimator attribute), 300
 pseudo_count (dmx.stats.hidden_markov.HiddenMarkovEstimator attribute), 182
 pseudo_count (dmx.stats.hmixture.HierarchicalMixtureEstimator attribute), 173
 pseudo_count (dmx.stats.ignored.IgnoredEstimator attribute), 144
 pseudo_count (dmx.stats.int_edit_setdist.IntegerBernoulliSetDistributionEstimator attribute), 203
 pseudo_count (dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator attribute), 315
 pseudo_count (dmx.stats.int_hidden_markov.IntegerHiddenMarkovEstimator attribute), 325
 pseudo_count (dmx.stats.int_markovchain.IntegerMarkovChainEstimator attribute), 343
 pseudo_count (dmx.stats.int_plsi.IntegerPLSEstimator attribute), 275
 pseudo_count (dmx.stats.int_spike.SpikeAndSlabEstimator attribute), 42
 pseudo_count (dmx.stats.intmultinomial.IntegerMultinomialEstimator attribute), 100
 pseudo_count (dmx.stats.intrange.IntegerCategoricalEstimator attribute), 37
 pseudo_count (dmx.stats.intsetdist.IntegerBernoulliSetEstimator attribute), 90
 pseudo_count (dmx.stats.jmixture.JointMixtureEstimator attribute), 165
 pseudo_count (dmx.stats.log_gaussian.LogGaussianEstimator attribute), 63
 pseudo_count (dmx.stats.markovchain.MarkovChainEstimator attribute), 114
 pseudo_count (dmx.stats.mixture.MixtureEstimator attribute), 151
 pseudo_count (dmx.stats.mvn.MultivariateGaussianEstimator attribute), 74
 pseudo_count (dmx.stats.null_dist.NullEstimator attribute), 215
 pseudo_count (dmx.stats.optional.OptionalEstimator attribute), 140
 pseudo_count (dmx.stats.poisson.PoissonEstimator attribute), 32
 pseudo_count (dmx.stats.setdist.BernoulliSetEstimator attribute), 95
 pseudo_count (dmx.stats.sparse_markov_transform.SparseMarkovAssociationEstimator attribute), 358
 pseudo_count (dmx.stats.ss_mixture.SemiSupervisedMixtureEstimator attribute), 291
 pseudo_count (dmx.stats.vmf.VonMisesFisherEstimator attribute), 233
 pseudo_count (dmx.torch_stats.binomial.BinomialEstimator attribute), 538
 pseudo_count (dmx.torch_stats.dmvn.DiagonalGaussianEstimator attribute), 544
 pseudo_count (dmx.torch_stats.gamma.GammaEstimator attribute), 556
 pseudo_count (dmx.torch_stats.gaussian.GaussianEstimator attribute), 562
 pseudo_count (dmx.torch_stats.geometric.GeometricEstimator attribute), 568
 pseudo_count (dmx.torch_stats.intmultinomial.IntegerMultinomialEstimator attribute), 576
 pseudo_count (dmx.torch_stats.intrange.IntegerCategoricalEstimator attribute), 583
 pseudo_count (dmx.torch_stats.intsetdist.IntegerBernoulliSetEstimator attribute), 590
 pseudo_count (dmx.torch_stats.mvn.MultivariateGaussianEstimator attribute), 596
 pseudo_count (dmx.torch_stats.poisson.PoissonEstimator attribute), 601
 psuedo_count (dmx.stats.spearman_rho.SpearmanRankingEstimator attribute), 226

R

randomize() (in module dmx.torch_utils.vector), 699
 ranking_depth() (in module dmx.utils.metrics), 675
 read_index_csv() (in module dmx.utils.builder), 660
 reduce_by_key() (in module dmx.utils.optsutil), 679
 required (dmx.stats.setdist.BernoulliSetDistribution attribute), 93
 reshape() (in module dmx.utils.vector), 690
 resolve_device() (in module dmx.torch_utils.vector), 700
 rho (dmx.stats.spearman_rho.SpearmanRankingDistribution attribute), 224
 rng (dmx.stats.binomial.BinomialSampler attribute), 24
 rng (dmx.stats.categorical.CategoricalSampler attribute), 86
 rng (dmx.stats.catmultinomial.MultinomialSampler attribute), 108
 rng (dmx.stats.composite.CompositeSampler attribute), 122
 rng (dmx.stats.dirac_length.DiracMixtureSampler attribute), 196
 rng (dmx.stats.dirichlet.DirichletSampler attribute), 81
 rng (dmx.stats.dmvn.DiagonalGaussianSampler attribute), 70
 rng (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureSampler attribute), 253
 rng (dmx.stats.exponential.ExponentialSampler attribute), 54
 rng (dmx.stats.gamma.GammaSampler attribute), 59
 rng (dmx.stats.gaussian.GaussianSampler attribute), 49
 rng (dmx.stats.geometric.GeometricSampler attribute), 29

- rng (*dmx.stats.gmm.GaussianMixtureSampler* attribute), 265
 rng (*dmx.stats.heterogeneous_mixture.HeterogeneousMixtureSampler* attribute), 160
 rng (*dmx.stats.hidden_markov.HiddenMarkovSampler* attribute), 184
 rng (*dmx.stats.hmixture.HierarchicalMixtureSampler* attribute), 175
 rng (*dmx.stats.icltree.ICLTreeSampler* attribute), 306
 rng (*dmx.stats.int_edit_setdist.IntegerBernoulliEditSampler* attribute), 204
 rng (*dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler* attribute), 330
 rng (*dmx.stats.int_markovchain.IntegerMarkovChainSampler* attribute), 344
 rng (*dmx.stats.int_plsi.IntegerPLISampler* attribute), 276
 rng (*dmx.stats.int_spike.SpikeAndSlabSampler* attribute), 44
 rng (*dmx.stats.intmultinomial.IntegerMultinomialSampler* attribute), 102
 rng (*dmx.stats.intrange.IntegerCategoricalSampler* attribute), 39
 rng (*dmx.stats.intsetdist.IntegerBernoulliSetSampler* attribute), 91
 rng (*dmx.stats.jmixture.JointMixtureSampler* attribute), 167
 rng (*dmx.stats.log_gaussian.LogGaussianSampler* attribute), 64
 rng (*dmx.stats.markovchain.MarkovChainSampler* attribute), 116
 rng (*dmx.stats.mixture.MixtureSampler* attribute), 153
 rng (*dmx.stats.mvn.MultivariateGaussianSampler* attribute), 75
 rng (*dmx.stats.null_dist.NullSampler* attribute), 216
 rng (*dmx.stats.optional.OptionalSampler* attribute), 141
 rng (*dmx.stats.pdist.DistributionSampler* attribute), 13
 rng (*dmx.stats.poisson.PoissonSampler* attribute), 33
 rng (*dmx.stats.sequence.SequenceSampler* attribute), 128
 rng (*dmx.stats.spearman_rho.SpearmanRankingSampler* attribute), 227
 rng (*dmx.stats.vmf.VonMisesFisherSampler* attribute), 234
 rng (*dmx.torch_stats.gamma.GammaSampler* attribute), 557
 rng (*dmx.torch_stats.geometric.GeometricSampler* attribute), 568
 rng (*dmx.torch_stats.intmultinomial.IntegerMultinomialSampler* attribute), 577
 rng (*dmx.torch_stats.intrange.IntegerCategoricalSampler* attribute), 584
 rng (*dmx.torch_stats.intsetdist.IntegerBernoulliSetSampler* attribute), 591
 rng (*dmx.torch_stats.poisson.PoissonSampler* attribute), 602
 roc_curve() (in module *dmx.utils.metrics*), 676
 sample_percentiles() (in module *dmx.utils.metrics*), 676
 row_choice() (in module *dmx.utils.vector*), 690
 row_perplexity_solve() (in module *dmx.utils.htsne*), 671
- ## S
- sample() (*dmx.bstats.bernoulli.BernoulliSampler* method), 413
 sample() (*dmx.bstats.beta.BetaSampler* method), 391
 sample() (*dmx.bstats.catdirichlet.DictDirichletSampler* method), 393
 sample() (*dmx.bstats.categorical.CategoricalSampler* method), 419
 sample() (*dmx.bstats.composite.CompositeSampler* method), 474
 sample() (*dmx.bstats.conditional.ConditionalDistributionSampler* method), 480
 sample() (*dmx.bstats.dirac.DiracSampler* method), 424
 sample() (*dmx.bstats.dirichlet.DirichletSampler* method), 399
 sample() (*dmx.bstats.dmvn.DiagonalGaussianSampler* method), 431
 sample() (*dmx.bstats.dpm.DirichletProcessMixtureSampler* method), 486
 sample() (*dmx.bstats.exponential.ExponentialSampler* method), 436
 sample() (*dmx.bstats.gamma.GammaSampler* method), 442
 sample() (*dmx.bstats.gaussian.GaussianSampler* method), 448
 sample() (*dmx.bstats.geometric.GeometricSampler* method), 454
 sample() (*dmx.bstats.ignored.IgnoredSampler* method), 492
 sample() (*dmx.bstats.intrange.IntegerCategoricalSampler* method), 461
 sample() (*dmx.bstats.mixture.MixtureSampler* method), 499
 sample() (*dmx.bstats.mvngamma.MultivariateNormalGammaSampler* method), 403
 sample() (*dmx.bstats.normgamma.NormalGammaSampler* method), 405
 sample() (*dmx.bstats.nulldist.NullSampler* method), 504
 sample() (*dmx.bstats.optional.OptionalSampler* method), 511
 sample() (*dmx.bstats.pdist.DistributionSampler* method), 374
 sample() (*dmx.bstats.poisson.PoissonSampler* method), 467
 sample() (*dmx.bstats.sequence.SequenceSampler* method), 518

<code>sample()</code> (<i>dmx.bstats.setdist.BernoulliSetSampler method</i>), 524	<code>sample()</code> (<i>dmx.stats.int_plsi.IntegerPLISampler method</i>), 276
<code>sample()</code> (<i>dmx.bstats.syndirichlet.SymmetricDirichletSampler method</i>), 407	<code>sample()</code> (<i>dmx.stats.int_spike.SpikeAndSlabSampler method</i>), 44
<code>sample()</code> (<i>dmx.stats.binomial.BinomialSampler method</i>), 24	<code>sample()</code> (<i>dmx.stats.intmultinomial.IntegerMultinomialSampler method</i>), 102
<code>sample()</code> (<i>dmx.stats.categorical.CategoricalSampler method</i>), 87	<code>sample()</code> (<i>dmx.stats.intrange.IntegerCategoricalSampler method</i>), 39
<code>sample()</code> (<i>dmx.stats.catmultinomial.MultinomialSampler method</i>), 109	<code>sample()</code> (<i>dmx.stats.intsetdist.IntegerBernoulliSetSampler method</i>), 92
<code>sample()</code> (<i>dmx.stats.composite.CompositeSampler method</i>), 122	<code>sample()</code> (<i>dmx.stats.jmixture.JointMixtureSampler method</i>), 167
<code>sample()</code> (<i>dmx.stats.conditional.ConditionalDistributionSampler method</i>), 135	<code>sample()</code> (<i>dmx.stats.lda.LDASampler method</i>), 286
<code>sample()</code> (<i>dmx.stats.dirac_length.DiracMixtureSampler method</i>), 196	<code>sample()</code> (<i>dmx.stats.log_gaussian.LogGaussianSampler method</i>), 64
<code>sample()</code> (<i>dmx.stats.dirichlet.DirichletSampler method</i>), 81	<code>sample()</code> (<i>dmx.stats.look_back_hmm.LookbackHiddenMarkovSampler method</i>), 351
<code>sample()</code> (<i>dmx.stats.dmvn.DiagonalGaussianSampler method</i>), 70	<code>sample()</code> (<i>dmx.stats.markovchain.MarkovChainSampler method</i>), 117
<code>sample()</code> (<i>dmx.stats.dmvn_mixture.DiagonalGaussianMixtureSampler method</i>), 253	<code>sample()</code> (<i>dmx.stats.mixture.MixtureSampler method</i>), 53
<code>sample()</code> (<i>dmx.stats.exponential.ExponentialSampler method</i>), 54	<code>sample()</code> (<i>dmx.stats.mvn.MultivariateGaussianSampler method</i>), 75
<code>sample()</code> (<i>dmx.stats.gamma.GammaSampler method</i>), 60	<code>sample()</code> (<i>dmx.stats.null_dist.NullSampler method</i>), 216
<code>sample()</code> (<i>dmx.stats.gaussian.GaussianSampler method</i>), 49	<code>sample()</code> (<i>dmx.stats.optional.OptionalSampler method</i>), 141
<code>sample()</code> (<i>dmx.stats.geometric.GeometricSampler method</i>), 29	<code>sample()</code> (<i>dmx.stats.pdist.DistributionSampler method</i>), 14
<code>sample()</code> (<i>dmx.stats.gmm.GaussianMixtureSampler method</i>), 265	<code>sample()</code> (<i>dmx.stats.poisson.PoissonSampler method</i>), 34
<code>sample()</code> (<i>dmx.stats.heterogeneous_mixture.HeterogeneousMixtureSampler method</i>), 161	<code>sample()</code> (<i>dmx.stats.select.SelectSampler method</i>), 221
<code>sample()</code> (<i>dmx.stats.hidden_association.HiddenAssociationSampler method</i>), 300	<code>sample()</code> (<i>dmx.stats.sequence.SequenceSampler method</i>), 128
<code>sample()</code> (<i>dmx.stats.hidden_markov.HiddenMarkovSampler method</i>), 185	<code>sample()</code> (<i>dmx.stats.setdist.BernoulliSetSampler method</i>), 96
<code>sample()</code> (<i>dmx.stats.hmixture.HierarchicalMixtureSampler method</i>), 175	<code>sample()</code> (<i>dmx.stats.sparse_markov_transform.SparseMarkovAssociationSampler method</i>), 359
<code>sample()</code> (<i>dmx.stats.icltree.ICLTreeSampler method</i>), 306	<code>sample()</code> (<i>dmx.stats.spearman_rho.SpearmanRankingSampler method</i>), 228
<code>sample()</code> (<i>dmx.stats.ignored.IgnoredSampler method</i>), 146	<code>sample()</code> (<i>dmx.stats.ss_mixture.SemiSupervisedMixtureSampler method</i>), 294
<code>sample()</code> (<i>dmx.stats.int_edit_setdist.IntegerBernoulliEditSampler method</i>), 204	<code>sample()</code> (<i>dmx.stats.tree_hmm.TreeHiddenMarkovSampler method</i>), 366
<code>sample()</code> (<i>dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditSampler method</i>), 209	<code>sample()</code> (<i>dmx.stats.vmf.VonMisesFisherSampler method</i>), 235
<code>sample()</code> (<i>dmx.stats.int_hidden_association.IntegerHiddenAssociationSampler method</i>), 316	<code>sample()</code> (<i>dmx.torch_stats.binomial.BinomialSampler method</i>), 539
<code>sample()</code> (<i>dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler method</i>), 330	<code>sample()</code> (<i>dmx.torch_stats.composite.CompositeSampler method</i>), 607
<code>sample()</code> (<i>dmx.stats.int_markovchain.IntegerMarkovChainSampler method</i>), 344	<code>sample()</code> (<i>dmx.torch_stats.conditional.ConditionalDistributionSampler method</i>), 612
	<code>sample()</code> (<i>dmx.torch_stats.dmvn.DiagonalGaussianSampler method</i>), 545
	<code>sample()</code> (<i>dmx.torch_stats.exponential.ExponentialSampler method</i>), 545

method), 551

sample() (dmx.torch_stats.gamma.GammaSampler method), 557

sample() (dmx.torch_stats.gaussian.GaussianSampler method), 563

sample() (dmx.torch_stats.geometric.GeometricSampler method), 569

sample() (dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureSampler method), 627

sample() (dmx.torch_stats.hmm.HiddenMarkovSampler method), 635

sample() (dmx.torch_stats.int_plsi.IntegerPLSISampler method), 641

sample() (dmx.torch_stats.intmultinomial.IntegerMultinomialSampler method), 577

sample() (dmx.torch_stats.intrange.IntegerCategoricalSampler method), 584

sample() (dmx.torch_stats.intsetdist.IntegerBernoulliSetSampler method), 591

sample() (dmx.torch_stats.jmixture.JointMixtureSampler method), 646

sample() (dmx.torch_stats.mixture.MixtureSampler method), 651

sample() (dmx.torch_stats.mvn.MultivariateGaussianSampler method), 597

sample() (dmx.torch_stats.null_dist.NullSampler method), 617

sample() (dmx.torch_stats.pdist.DistributionSampler method), 525

sample() (dmx.torch_stats.poisson.PoissonSampler method), 602

sample() (dmx.torch_stats.sequence.SequenceSampler method), 622

sample_dirichlet() (in module dmx.torch_utils.vector), 700

sample_given() (dmx.stats.conditional.ConditionalDistributionSampler method), 135

sample_given() (dmx.stats.hidden_association.HiddenAssociationSampler method), 301

sample_given() (dmx.stats.int_edit_setdist.IntegerBernoulliEditSampler method), 204

sample_given() (dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditSampler method), 210

sample_given() (dmx.stats.int_hidden_association.IntegerHiddenAssociationSampler method), 316

sample_given() (dmx.stats.int_markovchain.IntegerMarkovChainSampler method), 345

sample_given() (dmx.stats.pdist.ConditionalSampler method), 14

sample_given() (dmx.torch_stats.conditional.ConditionalDistributionSampler method), 612

sample_given() (dmx.torch_stats.pdist.ConditionalSampler method), 525

sample_int() (dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler method), 330

sample_rdd() (in module dmx.stats.rdd_sampler), 370

sample_seq() (dmx.stats.hidden_markov.HiddenMarkovSampler method), 185

sample_seq() (dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler method), 331

sample_seq() (dmx.stats.markovchain.MarkovChainSampler method), 636

sample_seq() (dmx.torch_stats.hmm.HiddenMarkovSampler method), 636

sample_seq_as_rdd() (in module dmx.stats.rdd_sampler), 371

sample_state() (dmx.stats.tree_hmm.TreeHiddenMarkovSampler method), 366

sample_terminal() (dmx.stats.hidden_markov.HiddenMarkovSampler method), 185

sample_terminal() (dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler method), 331

sample_terminal() (dmx.torch_stats.hmm.HiddenMarkovSampler method), 636

sample_tree() (dmx.stats.tree_hmm.TreeHiddenMarkovSampler method), 366

sampler (dmx.stats.hmixture.HierarchicalMixtureSampler attribute), 175

sampler (dmx.stats.optional.OptionalSampler attribute), 141

sampler() (dmx.bstats.bernoulli.BernoulliDistribution method), 409

sampler() (dmx.bstats.beta.BetaDistribution method), 390

sampler() (dmx.bstats.catdirichlet.DictDirichletDistribution method), 393

sampler() (dmx.bstats.categorical.CategoricalDistribution method), 415

sampler() (dmx.bstats.composite.CompositeDistribution method), 470

sampler() (dmx.bstats.conditional.ConditionalDistribution method), 476

sampler() (dmx.bstats.dirac.DiracDistribution method), 422

sampler() (dmx.bstats.dirichlet.DirichletDistribution method), 407

sampler() (dmx.bstats.dmvn.DiagonalGaussianDistribution method), 428

sampler() (dmx.bstats.dpm.DirichletProcessMixtureDistribution method), 484

sampler() (dmx.bstats.exponential.ExponentialDistribution method), 434

sampler() (dmx.bstats.gamma.GammaDistribution method), 439

sampler() (dmx.bstats.gaussian.GaussianDistribution method), 446

sampler() (dmx.bstats.geometric.GeometricDistribution method), 452

<code>sampler()</code> (<i>dmx.bstats.ignored.IgnoredDistribution</i> method), 490	<code>sampler()</code> (<i>dmx.stats.hidden_association.HiddenAssociationDistribution</i> method), 298
<code>sampler()</code> (<i>dmx.bstats.intrange.IntegerCategoricalDistribution</i> method), 458	<code>sampler()</code> (<i>dmx.stats.hidden_markov.HiddenMarkovModelDistribution</i> method), 180
<code>sampler()</code> (<i>dmx.bstats.mixture.MixtureDistribution</i> method), 494	<code>sampler()</code> (<i>dmx.stats.hmixture.HierarchicalMixtureDistribution</i> method), 171
<code>sampler()</code> (<i>dmx.bstats.mvngamma.MultivariateNormalGammaDistribution</i> method), 402	<code>sampler()</code> (<i>dmx.stats.ictree.ICLTreeDistribution</i> method), 305
<code>sampler()</code> (<i>dmx.bstats.normgamma.NormalGammaDistribution</i> method), 404	<code>sampler()</code> (<i>dmx.stats.ignored.IgnoredDistribution</i> method), 143
<code>sampler()</code> (<i>dmx.bstats.nulldist.NullDistribution</i> method), 503	<code>sampler()</code> (<i>dmx.stats.int_edit_setdist.IntegerBernoulliEditDistribution</i> method), 202
<code>sampler()</code> (<i>dmx.bstats.optional.OptionalDistribution</i> method), 507	<code>sampler()</code> (<i>dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditDistribution</i> method), 208
<code>sampler()</code> (<i>dmx.bstats.pdist.ProbabilityDistribution</i> method), 378	<code>sampler()</code> (<i>dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution</i> method), 312
<code>sampler()</code> (<i>dmx.bstats.poisson.PoissonDistribution</i> method), 463	<code>sampler()</code> (<i>dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution</i> method), 328
<code>sampler()</code> (<i>dmx.bstats.sequence.SequenceDistribution</i> method), 514	<code>sampler()</code> (<i>dmx.stats.int_markovchain.IntegerMarkovChainDistribution</i> method), 341
<code>sampler()</code> (<i>dmx.bstats.setdist.BernoulliSetDistribution</i> method), 522	<code>sampler()</code> (<i>dmx.stats.int_plsi.IntegerPLSIDistribution</i> method), 273
<code>sampler()</code> (<i>dmx.bstats.syndirichlet.SymmetricDirichletDistribution</i> method), 406	<code>sampler()</code> (<i>dmx.stats.int_spike.SpikeAndSlabDistribution</i> method), 42
<code>sampler()</code> (<i>dmx.stats.binomial.BinomialDistribution</i> method), 22	<code>sampler()</code> (<i>dmx.stats.intmultinomial.IntegerMultinomialDistribution</i> method), 99
<code>sampler()</code> (<i>dmx.stats.categorical.CategoricalDistribution</i> method), 84	<code>sampler()</code> (<i>dmx.stats.intrange.IntegerCategoricalDistribution</i> method), 36
<code>sampler()</code> (<i>dmx.stats.catmultinomial.MultinomialDistribution</i> method), 105	<code>sampler()</code> (<i>dmx.stats.intsetdist.IntegerBernoulliSetDistribution</i> method), 89
<code>sampler()</code> (<i>dmx.stats.composite.CompositeDistribution</i> method), 120	<code>sampler()</code> (<i>dmx.stats.jmixture.JointMixtureDistribution</i> method), 164
<code>sampler()</code> (<i>dmx.stats.conditional.ConditionalDistribution</i> method), 131	<code>sampler()</code> (<i>dmx.stats.lda.LDADistribution</i> method), 281
<code>sampler()</code> (<i>dmx.stats.dirac_length.DiracMixtureDistribution</i> method), 193	<code>sampler()</code> (<i>dmx.stats.log_gaussian.LogGaussianDistribution</i> method), 62
<code>sampler()</code> (<i>dmx.stats.dirichlet.DirichletDistribution</i> method), 78	<code>sampler()</code> (<i>dmx.stats.look_back_hmm.LookbackHiddenMarkovDistribution</i> method), 347
<code>sampler()</code> (<i>dmx.stats.dmvn.DiagonalGaussianDistribution</i> method), 67	<code>sampler()</code> (<i>dmx.stats.markovchain.MarkovChainDistribution</i> method), 113
<code>sampler()</code> (<i>dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution</i> method), 250	<code>sampler()</code> (<i>dmx.stats.mixture.MixtureDistribution</i> method), 149
<code>sampler()</code> (<i>dmx.stats.exponential.ExponentialDistribution</i> method), 52	<code>sampler()</code> (<i>dmx.stats.mvn.MultivariateGaussianDistribution</i> method), 73
<code>sampler()</code> (<i>dmx.stats.gamma.GammaDistribution</i> method), 57	<code>sampler()</code> (<i>dmx.stats.null_dist.NullDistribution</i> method), 214
<code>sampler()</code> (<i>dmx.stats.gaussian.GaussianDistribution</i> method), 47	<code>sampler()</code> (<i>dmx.stats.optional.OptionalDistribution</i> method), 138
<code>sampler()</code> (<i>dmx.stats.geometric.GeometricDistribution</i> method), 27	<code>sampler()</code> (<i>dmx.stats.pdist.ProbabilityDistribution</i> method), 12
<code>sampler()</code> (<i>dmx.stats.gmm.GaussianMixtureDistribution</i> method), 262	<code>sampler()</code> (<i>dmx.stats.poisson.PoissonDistribution</i> method), 32
<code>sampler()</code> (<i>dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution</i> method), 157	<code>sampler()</code> (<i>dmx.stats.select.SelectDistribution</i> method), 18
	<code>sampler()</code> (<i>dmx.stats.sequence.SequenceDistribution</i> method), 514

method), 125

sampler() (dmx.stats.setdist.BernoulliSetDistribution method), 94

sampler() (dmx.stats.sparse_markov_transform.SparseMarkovTransform method), 357

sampler() (dmx.stats.spearman_rho.SpearmanRankingDistribution method), 225

sampler() (dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution method), 290

sampler() (dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution method), 365

sampler() (dmx.stats.vmf.VonMisesFisherDistribution method), 233

sampler() (dmx.stats.weighted.WeightedDistribution method), 240

sampler() (dmx.torch_stats.binomial.BinomialDistribution method), 537

sampler() (dmx.torch_stats.composite.CompositeDistribution method), 606

sampler() (dmx.torch_stats.conditional.ConditionalDistribution method), 609

sampler() (dmx.torch_stats.dmvn.DiagonalGaussianDistribution method), 543

sampler() (dmx.torch_stats.exponential.ExponentialDistribution method), 549

sampler() (dmx.torch_stats.gamma.GammaDistribution method), 555

sampler() (dmx.torch_stats.gaussian.GaussianDistribution method), 561

sampler() (dmx.torch_stats.geometric.GeometricDistribution method), 567

sampler() (dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDistribution method), 625

sampler() (dmx.torch_stats.hmm.HiddenMarkovModelDistribution method), 634

sampler() (dmx.torch_stats.int_plsi.IntegerPLSIDistribution method), 640

sampler() (dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution method), 575

sampler() (dmx.torch_stats.intrange.IntegerCategoricalDistribution method), 582

sampler() (dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution method), 589

sampler() (dmx.torch_stats.jmixture.JointMixtureDistribution method), 643

sampler() (dmx.torch_stats.mixture.MixtureDistribution method), 650

sampler() (dmx.torch_stats.mvn.MultivariateGaussianDistribution method), 595

sampler() (dmx.torch_stats.null_dist.NullDistribution method), 616

sampler() (dmx.torch_stats.pdist.TorchProbabilityDistribution method), 528

sampler() (dmx.torch_stats.poisson.PoissonDistribution method), 601

sampler() (dmx.torch_stats.sequence.SequenceDistribution method), 620

sampler() (dmx.torch_stats.setdist.BernoulliSetDistribution method), 94

sampler() (dmx.stats.int_plsi.IntegerPLSIAccumulator attribute), 267

seed() (dmx.stats.gamma.GammaSampler attribute), 60

seed (dmx.stats.setdist.BernoulliSetSampler attribute), 96

seed (dmx.torch_stats.dmvn.DiagonalGaussianSampler attribute), 545

seed (dmx.torch_stats.gamma.GammaSampler attribute), 557

seed_sample() (in module dmx.torch_utils.vector), 700

seed_tng() (in module dmx.torch_utils.vector), 701

SelectDataEncoder (class in dmx.stats.select), 216

SelectDistribution (class in dmx.stats.select), 217

SelectEncodedDataSequence (class in dmx.stats.select), 218

SelectEstimator (class in dmx.stats.select), 218

SelectEstimatorAccumulator (class in dmx.stats.select), 218

SelectEstimatorAccumulatorFactory (class in dmx.stats.select), 220

SelectSampler (class in dmx.stats.select), 221

SemiSupervisedMixtureDataEncoder (class in dmx.stats.ss_mixture), 288

SemiSupervisedMixtureDistribution (class in dmx.stats.ss_mixture), 288

SemiSupervisedMixtureEncodedDataSequence (class in dmx.stats.ss_mixture), 290

SemiSupervisedMixtureEstimator (class in dmx.stats.ss_mixture), 290

SemiSupervisedMixtureEstimatorAccumulator (class in dmx.stats.ss_mixture), 292

SemiSupervisedMixtureEstimatorAccumulatorFactory (class in dmx.stats.ss_mixture), 293

SemiSupervisedMixtureSampler (class in dmx.stats.ss_mixture), 294

seq_component_log_density() (dmx.bstats.mixture.MixtureDistribution method), 494

seq_component_log_density() (dmx.stats.dirac_length.DiracMixtureDistribution method), 193

seq_component_log_density() (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution method), 250

seq_component_log_density() (dmx.stats.gmm.GaussianMixtureDistribution method), 262

seq_component_log_density() (dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution method), 262

`method`), 157
`seq_component_log_density()`
 (`dmx.stats.hmixture.HierarchicalMixtureDistribution`
 `method`), 171
`seq_component_log_density()`
 (`dmx.stats.int_plsi.IntegerPLSIDistribution`
 `method`), 273
`seq_component_log_density()`
 (`dmx.stats.lda.LDADistribution` `method`),
 282
`seq_component_log_density()`
 (`dmx.stats.mixture.MixtureDistribution`
 `method`), 149
`seq_component_log_density()`
 (`dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDistribution`
 `method`), 625
`seq_component_log_density()`
 (`dmx.torch_stats.mixture.MixtureDistribution`
 `method`), 650
`seq_encode()` (`dmx.bstats.bernoulli.BernoulliDataEncoder`
 `method`), 408
`seq_encode()` (`dmx.bstats.bernoulli.BernoulliDistribution`
 `method`), 409
`seq_encode()` (`dmx.bstats.categorical.CategoricalDataEncoder`
 `method`), 414
`seq_encode()` (`dmx.bstats.categorical.CategoricalDistribution`
 `method`), 415
`seq_encode()` (`dmx.bstats.composite.CompositeDataEncoder`
 `method`), 468
`seq_encode()` (`dmx.bstats.composite.CompositeDistribution`
 `method`), 470
`seq_encode()` (`dmx.bstats.conditional.ConditionalDataEncoder`
 `method`), 475
`seq_encode()` (`dmx.bstats.conditional.ConditionalDistribution`
 `method`), 476
`seq_encode()` (`dmx.bstats.dirac.DiracDataEncoder`
 `method`), 421
`seq_encode()` (`dmx.bstats.dirac.DiracDistribution`
 `method`), 423
`seq_encode()` (`dmx.bstats.dirichlet.DirichletDistribution`
 `method`), 398
`seq_encode()` (`dmx.bstats.dmvn.DiagonalGaussianDataEncoder`
 `method`), 427
`seq_encode()` (`dmx.bstats.dmvn.DiagonalGaussianDistribution`
 `method`), 428
`seq_encode()` (`dmx.bstats.dpm.DirichletProcessMixtureDistribution`
 `method`), 484
`seq_encode()` (`dmx.bstats.exponential.ExponentialDataEncoder`
 `method`), 433
`seq_encode()` (`dmx.bstats.exponential.ExponentialDistribution`
 `method`), 434
`seq_encode()` (`dmx.bstats.gamma.GammaDistribution`
 `method`), 440
`seq_encode()` (`dmx.bstats.gaussian.GaussianDataEncoder`
 `method`), 445
`seq_encode()` (`dmx.bstats.gaussian.GaussianDistribution`
 `method`), 446
`seq_encode()` (`dmx.bstats.geometric.GeometricDataEncoder`
 `method`), 451
`seq_encode()` (`dmx.bstats.geometric.GeometricDistribution`
 `method`), 452
`seq_encode()` (`dmx.bstats.ignored.IgnoredDataEncoder`
 `method`), 489
`seq_encode()` (`dmx.bstats.ignored.IgnoredDistribution`
 `method`), 490
`seq_encode()` (`dmx.bstats.intrange.IntegerCategoricalDataEncoder`
 `method`), 457
`seq_encode()` (`dmx.bstats.intrange.IntegerCategoricalDistribution`
 `method`), 458
`seq_encode()` (`dmx.bstats.mixture.MixtureDataEncoder`
 `method`), 492
`seq_encode()` (`dmx.bstats.mixture.MixtureDistribution`
 `method`), 494
`seq_encode()` (`dmx.bstats.nulldist.NullDataEncoder`
 `method`), 501
`seq_encode()` (`dmx.bstats.nulldist.NullDistribution`
 `method`), 503
`seq_encode()` (`dmx.bstats.optional.OptionalDataEncoder`
 `method`), 505
`seq_encode()` (`dmx.bstats.optional.OptionalDistribution`
 `method`), 507
`seq_encode()` (`dmx.bstats.pdist.DataSequenceEncoder`
 `method`), 373
`seq_encode()` (`dmx.bstats.pdist.ProbabilityDistribution`
 `method`), 378
`seq_encode()` (`dmx.bstats.pdist.SequenceEncodableDistribution`
 `method`), 381
`seq_encode()` (`dmx.bstats.poisson.PoissonDataEncoder`
 `method`), 461
`seq_encode()` (`dmx.bstats.poisson.PoissonDistribution`
 `method`), 463
`seq_encode()` (`dmx.bstats.sequence.SequenceDataEncoder`
 `method`), 512
`seq_encode()` (`dmx.bstats.sequence.SequenceDistribution`
 `method`), 514
`seq_encode()` (`dmx.bstats.setdist.BernoulliSetDataEncoder`
 `method`), 521
`seq_encode()` (`dmx.bstats.setdist.BernoulliSetDistribution`
 `method`), 522
`seq_encode()` (`dmx.stats.dirac_length.DiracMixtureDataEncoder`
 `method`), 190
`seq_encode()` (`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDataEncoder`
 `method`), 246
`seq_encode()` (`dmx.stats.gmm.GaussianMixtureDataEncoder`
 `method`), 259
`seq_encode()` (`dmx.stats.hidden_association.HiddenAssociationDataEncoder`
 `method`), 297
`seq_encode()` (`dmx.stats.icltree.ICLTreeDataEncoder`

method), 303

seq_encode() (dmx.stats.int_edit_setdist.IntegerBernoulliSetDataEncoder method), 200

seq_encode() (dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliSetDataEncoder method), 207

seq_encode() (dmx.stats.int_hidden_association.IntegerHiddenAssociationDataEncoder method), 310

seq_encode() (dmx.stats.int_hidden_markov.IntegerHiddenMarkovDataEncoder method), 324

seq_encode() (dmx.stats.int_markovchain.IntegerMarkovChainDataEncoder method), 339

seq_encode() (dmx.stats.int_plsi.IntegerPLSIDataEncoder method), 271

seq_encode() (dmx.stats.lda.LDADataEncoder method), 280

seq_encode() (dmx.stats.look_back_hmm.LookbackHiddenMarkovDataEncoder method), 346

seq_encode() (dmx.stats.null_dist.NullDataEncoder method), 213

seq_encode() (dmx.stats.pdist.DataSequenceEncoder method), 18

seq_encode() (dmx.stats.select.SelectDataEncoder method), 217

seq_encode() (dmx.stats.sparse_markov_transform.SparseMarkovTransformDataEncoder method), 356

seq_encode() (dmx.stats.spearman_rho.SpearmanRankingDataEncoder method), 224

seq_encode() (dmx.stats.ss_mixture.SemiSupervisedMixtureDataEncoder method), 288

seq_encode() (dmx.stats.tree_hmm.TreeHiddenMarkovDataEncoder method), 363

seq_encode() (dmx.stats.vmf.VonMisesFisherDataEncoder method), 231

seq_encode() (dmx.stats.weighted.WeightedDataEncoder method), 239

seq_encode() (dmx.torch_stats.binomial.BinomialDataEncoder method), 534

seq_encode() (dmx.torch_stats.composite.CompositeDataEncoder method), 605

seq_encode() (dmx.torch_stats.conditional.ConditionalDistributionDataEncoder method), 611

seq_encode() (dmx.torch_stats.dmvn.DiagonalGaussianDataEncoder method), 542

seq_encode() (dmx.torch_stats.exponential.ExponentialDataEncoder method), 548

seq_encode() (dmx.torch_stats.gamma.GammaDataEncoder method), 553

seq_encode() (dmx.torch_stats.gaussian.GaussianDataEncoder method), 560

seq_encode() (dmx.torch_stats.geometric.GeometricDataEncoder method), 566

seq_encode() (dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDataEncoder method), 624

seq_encode() (dmx.torch_stats.hmm.HiddenMarkovDataEncoder method), 630

seq_encode() (dmx.torch_stats.int_plsi.IntegerPLSIDataEncoder method), 639

seq_encode() (dmx.torch_stats.intmultinomial.IntegerMultinomialDataEncoder method), 573

seq_encode() (dmx.torch_stats.intrange.IntegerCategoricalDataEncoder method), 581

seq_encode() (dmx.torch_stats.intsetdist.IntegerBernoulliSetDataEncoder method), 588

seq_encode() (dmx.torch_stats.jmixture.JointMixtureDataEncoder method), 642

seq_encode() (dmx.torch_stats.mixture.MixtureDataEncoder method), 649

seq_encode() (dmx.torch_stats.mvn.MultivariateGaussianDataEncoder method), 594

seq_encode() (dmx.torch_stats.null_dist.NullDataEncoder method), 615

seq_encode() (dmx.torch_stats.pdist.TorchSequenceEncoder method), 529

seq_encode() (dmx.torch_stats.poisson.PoissonDataEncoder method), 600

seq_encode() (dmx.torch_stats.sequence.SequenceDataEncoder method), 619

seq_encode() (in module dmx.stats), 8

seq_encode_mpi() (in module dmx.mpi4py.bstats), 706

seq_encode_mpi() (in module dmx.mpi4py.stats), 703

seq_estimate() (in module dmx.stats), 10

seq_estimate_mpi() (in module dmx.mpi4py.bstats), 707

seq_estimate_mpi() (in module dmx.mpi4py.stats), 703

seq_expected_log_density() (dmx.bstats.bernoulli.BernoulliDistribution method), 409

seq_expected_log_density() (dmx.bstats.categorical.CategoricalDistribution method), 415

seq_expected_log_density() (dmx.bstats.composite.CompositeDistribution method), 420

seq_expected_log_density() (dmx.bstats.dirac.DiracDistribution method), 423

seq_expected_log_density() (dmx.bstats.dmvn.DiagonalGaussianDistribution method), 429

seq_expected_log_density() (dmx.bstats.dpm.DirichletProcessMixtureDistribution method), 484

seq_expected_log_density() (dmx.bstats.exponential.ExponentialDistribution method), 490

seq_expected_log_density() (dmx.bstats.gamma.GammaDistribution method), 496

`method`), 440
`seq_expected_log_density()` (`dmx.bstats.gaussian.GaussianDistribution` `method`), 446
`seq_expected_log_density()` (`dmx.bstats.geometric.GeometricDistribution` `method`), 452
`seq_expected_log_density()` (`dmx.bstats.ignored.IgnoredDistribution` `method`), 490
`seq_expected_log_density()` (`dmx.bstats.inrange.IntegerCategoricalDistribution` `method`), 459
`seq_expected_log_density()` (`dmx.bstats.mixture.MixtureDistribution` `method`), 494
`seq_expected_log_density()` (`dmx.bstats.nulldist.NullDistribution` `method`), 503
`seq_expected_log_density()` (`dmx.bstats.optional.OptionalDistribution` `method`), 507
`seq_expected_log_density()` (`dmx.bstats.pdist.ProbabilityDistribution` `method`), 378
`seq_expected_log_density()` (`dmx.bstats.poisson.PoissonDistribution` `method`), 463
`seq_expected_log_density()` (`dmx.bstats.sequence.SequenceDistribution` `method`), 514
`seq_initialize()` (`dmx.bstats.bernoulli.BernoulliEstimatorAccumulator` `method`), 412
`seq_initialize()` (`dmx.bstats.categorical.CategoricalEstimatorAccumulator` `method`), 418
`seq_initialize()` (`dmx.bstats.composite.CompositeEstimatorAccumulator` `method`), 473
`seq_initialize()` (`dmx.bstats.conditional.ConditionalDistributionEstimatorAccumulator` `method`), 479
`seq_initialize()` (`dmx.bstats.dirac.DiracAccumulator` `method`), 420
`seq_initialize()` (`dmx.bstats.dmvn.DiagonalGaussianAccumulator` `method`), 426
`seq_initialize()` (`dmx.bstats.exponential.ExponentialAccumulator` `method`), 432
`seq_initialize()` (`dmx.bstats.gaussian.GaussianAccumulator` `method`), 444
`seq_initialize()` (`dmx.bstats.geometric.GeometricAccumulator` `method`), 450
`seq_initialize()` (`dmx.bstats.ignored.IgnoredAccumulator` `method`), 488
`seq_initialize()` (`dmx.bstats.inrange.IntegerCategoricalAccumulator` `method`), 456
`seq_initialize()` (`dmx.bstats.mixture.MixtureEstimatorAccumulator` `method`), 498
`seq_initialize()` (`dmx.bstats.nulldist.NullAccumulator` `method`), 500
`seq_initialize()` (`dmx.bstats.optional.OptionalEstimatorAccumulator` `method`), 510
`seq_initialize()` (`dmx.bstats.pdist.SequenceEncodableAccumulator` `method`), 380
`seq_initialize()` (`dmx.bstats.poisson.PoissonEstimatorAccumulator` `method`), 466
`seq_initialize()` (`dmx.bstats.sequence.SequenceEstimatorAccumulator` `method`), 517
`seq_initialize()` (`dmx.bstats.setdist.BernoulliSetAccumulator` `method`), 520
`seq_initialize()` (`dmx.stats.dirac_length.DiracMixtureAccumulator` `method`), 188
`seq_initialize()` (`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator` `method`), 244
`seq_initialize()` (`dmx.stats.gmm.GaussianMixtureAccumulator` `method`), 257
`seq_initialize()` (`dmx.stats.hidden_association.HiddenAssociationAccumulator` `method`), 296
`seq_initialize()` (`dmx.stats.ictree.ICLTreeAccumulator` `method`), 302
`seq_initialize()` (`dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator` `method`), 198
`seq_initialize()` (`dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditAccumulator` `method`), 206
`seq_initialize()` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator` `method`), 308
`seq_initialize()` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator` `method`), 321
`seq_initialize()` (`dmx.stats.int_markovchain.IntegerMarkovChainAccumulator` `method`), 337
`seq_initialize()` (`dmx.stats.int_plsi.IntegerPLSIAccumulator` `method`), 269
`seq_initialize()` (`dmx.stats.lda.LDAEstimatorAccumulator` `method`), 284
`seq_initialize()` (`dmx.stats.look_back_hmm.LookbackHiddenMarkovEstimatorAccumulator` `method`), 349
`seq_initialize()` (`dmx.stats.null_dist.NullAccumulator` `method`), 211
`seq_initialize()` (`dmx.stats.pdist.SequenceEncodableStatisticAccumulator` `method`), 17
`seq_initialize()` (`dmx.stats.select.SelectEstimatorAccumulator` `method`), 220
`seq_initialize()` (`dmx.stats.sparse_markov_transform.SparseMarkovAccumulator` `method`), 354
`seq_initialize()` (`dmx.stats.spearman_rho.SpearmanRankingAccumulator` `method`), 223
`seq_initialize()` (`dmx.stats.ss_mixture.SemiSupervisedMixtureEstimatorAccumulator` `method`), 293
`seq_initialize()` (`dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator` `method`), 361
`seq_initialize()` (`dmx.stats.vmf.VonMisesFisherAccumulator` `method`), 498

`method`), 229
`seq_initialize()` (`dmx.stats.weighted.WeightedAccumulator` `method`), 125
`method`), 237
`seq_initialize()` (`dmx.torch_stats.binomial.BinomialAccumulator` `method`), 410
`method`), 533
`seq_initialize()` (`dmx.torch_stats.composite.CompositeAccumulator` `method`), 416
`method`), 604
`seq_initialize()` (`dmx.torch_stats.conditional.ConditionalDistribution` `method`), 470
`method`), 610
`seq_initialize()` (`dmx.torch_stats.dmvn.DiagonalGaussianAccumulator` `method`), 476
`method`), 541
`seq_initialize()` (`dmx.torch_stats.exponential.ExponentialAccumulator` `method`), 423
`method`), 547
`seq_initialize()` (`dmx.torch_stats.gamma.GammaAccumulator` `method`), 398
`method`), 552
`seq_initialize()` (`dmx.torch_stats.gaussian.GaussianAccumulator` `method`), 429
`method`), 559
`seq_initialize()` (`dmx.torch_stats.geometric.GeometricAccumulator` `method`), 484
`method`), 565
`seq_initialize()` (`dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulator` `method`), 436
`method`), 623
`seq_initialize()` (`dmx.torch_stats.hmm.HiddenMarkovAccumulator` `method`), 440
`method`), 629
`seq_initialize()` (`dmx.torch_stats.int_plsi.IntegerPLSIAccumulator` `method`), 446
`method`), 638
`seq_initialize()` (`dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator` `method`), 444
`method`), 571
`seq_initialize()` (`dmx.torch_stats.intrange.IntegerCategoricalAccumulator` `method`), 491
`method`), 579
`seq_initialize()` (`dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulator` `method`), 459
`method`), 586
`seq_initialize()` (`dmx.torch_stats.jmixture.JointMixtureEstimator` `method`), 495
`method`), 645
`seq_initialize()` (`dmx.torch_stats.mixture.MixtureAccumulator` `method`), 503
`method`), 648
`seq_initialize()` (`dmx.torch_stats.mvn.MultivariateGaussianAccumulator` `method`), 507
`method`), 593
`seq_initialize()` (`dmx.torch_stats.null_dist.NullAccumulator` `method`), 378
`method`), 614
`seq_initialize()` (`dmx.torch_stats.pdist.TorchStatisticAccumulator` `method`), 463
`method`), 530
`seq_initialize()` (`dmx.torch_stats.poisson.PoissonAccumulator` `method`), 514
`method`), 599
`seq_initialize()` (`dmx.torch_stats.sequence.SequenceAccumulator` `method`), 522
`method`), 618
`seq_initialize()` (in module `dmx.stats`), 10
`seq_initialize_mpi()` (in module `dmx.mpi4py.stats`), 704
`seq_ld_lambda()` (`dmx.stats.gaussian.GaussianDistribution` `method`), 47
`seq_ld_lambda()` (`dmx.stats.log_gaussian.LogGaussianDistribution` `method`), 62
`seq_ld_lambda()` (`dmx.stats.pdist.SequenceEncodableProbabilityDistribution` `method`), 12
`seq_ld_lambda()` (`dmx.stats.sequence.SequenceDistribution` `method`), 125
`seq_log_density()` (`dmx.bstats.bernoulli.BernoulliDistribution` `method`), 470
`seq_log_density()` (`dmx.bstats.categorical.CategoricalDistribution` `method`), 416
`seq_log_density()` (`dmx.bstats.composite.CompositeDistribution` `method`), 470
`seq_log_density()` (`dmx.bstats.conditional.ConditionalDistribution` `method`), 476
`seq_log_density()` (`dmx.bstats.dirac.DiracDistribution` `method`), 423
`seq_log_density()` (`dmx.bstats.dirichlet.DirichletDistribution` `method`), 398
`seq_log_density()` (`dmx.bstats.dmvn.DiagonalGaussianDistribution` `method`), 429
`seq_log_density()` (`dmx.bstats.dpm.DirichletProcessMixtureDistribution` `method`), 436
`seq_log_density()` (`dmx.bstats.exponential.ExponentialDistribution` `method`), 484
`seq_log_density()` (`dmx.bstats.gamma.GammaDistribution` `method`), 440
`seq_log_density()` (`dmx.bstats.gaussian.GaussianDistribution` `method`), 446
`seq_log_density()` (`dmx.bstats.geometric.GeometricDistribution` `method`), 444
`seq_log_density()` (`dmx.bstats.ignored.IgnoredDistribution` `method`), 491
`seq_log_density()` (`dmx.bstats.intrange.IntegerCategoricalDistribution` `method`), 459
`seq_log_density()` (`dmx.bstats.mixture.MixtureDistribution` `method`), 495
`seq_log_density()` (`dmx.bstats.nulldist.NullDistribution` `method`), 378
`seq_log_density()` (`dmx.bstats.optional.OptionalDistribution` `method`), 507
`seq_log_density()` (`dmx.bstats.pdist.ProbabilityDistribution` `method`), 514
`seq_log_density()` (`dmx.bstats.poisson.PoissonDistribution` `method`), 522
`seq_log_density()` (`dmx.bstats.sequence.SequenceDistribution` `method`), 530
`seq_log_density()` (`dmx.bstats.setdist.BernoulliSetDistribution` `method`), 514
`seq_log_density()` (`dmx.stats.binomial.BinomialDistribution` `method`), 22
`seq_log_density()` (`dmx.stats.categorical.CategoricalDistribution` `method`), 84
`seq_log_density()` (`dmx.stats.catmultinomial.MultinomialDistribution` `method`), 106
`seq_log_density()` (`dmx.stats.composite.CompositeDistribution` `method`), 120
`seq_log_density()` (`dmx.stats.conditional.ConditionalDistribution` `method`), 132

seq_log_density() (dmx.stats.dirac_length.DiracMixtureDistribution method), 193

seq_log_density() (dmx.stats.dirichlet.DirichletDistribution method), 78

seq_log_density() (dmx.stats.dmvn.DiagonalGaussianDistribution method), 67

seq_log_density() (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution method), 250

seq_log_density() (dmx.stats.exponential.ExponentialDistribution method), 52

seq_log_density() (dmx.stats.gamma.GammaDistribution method), 57

seq_log_density() (dmx.stats.gaussian.GaussianDistribution method), 47

seq_log_density() (dmx.stats.geometric.GeometricDistribution method), 27

seq_log_density() (dmx.stats.gmm.GaussianMixtureDistribution method), 262

seq_log_density() (dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution method), 157

seq_log_density() (dmx.stats.hidden_association.HiddenAssociationDistribution method), 299

seq_log_density() (dmx.stats.hidden_markov.HiddenMarkovModelDistribution method), 180

seq_log_density() (dmx.stats.hmixture.HierarchicalMixtureDistribution method), 171

seq_log_density() (dmx.stats.ictree.ICLTreeDistribution method), 305

seq_log_density() (dmx.stats.ignored.IgnoredDistribution method), 143

seq_log_density() (dmx.stats.int_edit_setdist.IntegerBernoulliSetDistribution method), 202

seq_log_density() (dmx.stats.int_edit_stepsetdist.IntegerBernoulliStepSetDistribution method), 208

seq_log_density() (dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution method), 313

seq_log_density() (dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution method), 328

seq_log_density() (dmx.stats.int_markovchain.IntegerMarkovChainDistribution method), 342

seq_log_density() (dmx.stats.int_plsi.IntegerPLSIDistribution method), 274

seq_log_density() (dmx.stats.int_spike.SpikeAndSlabDistribution method), 42

seq_log_density() (dmx.stats.intmultinomial.IntegerMultinomialDistribution method), 99

seq_log_density() (dmx.stats.intrange.IntegerCategoricalDistribution method), 36

seq_log_density() (dmx.stats.intsetdist.IntegerBernoulliSetDistribution method), 89

seq_log_density() (dmx.stats.jmixture.JointMixtureDistribution method), 164

seq_log_density() (dmx.stats.lda.LDADistribution method), 282

seq_log_density() (dmx.stats.log_gaussian.LogGaussianDistribution method), 62

seq_log_density() (dmx.stats.look_back_hmm.LookbackHiddenMarkovModelDistribution method), 347

seq_log_density() (dmx.stats.markovchain.MarkovChainDistribution method), 113

seq_log_density() (dmx.stats.mixture.MixtureDistribution method), 150

seq_log_density() (dmx.stats.mvn.MultivariateGaussianDistribution method), 73

seq_log_density() (dmx.stats.null_dist.NullDistribution method), 214

seq_log_density() (dmx.stats.optional.OptionalDistribution method), 138

seq_log_density() (dmx.stats.pdist.SequenceEncodableProbabilityDistribution method), 13

seq_log_density() (dmx.stats.poisson.PoissonDistribution method), 32

seq_log_density() (dmx.stats.select.SelectDistribution method), 218

seq_log_density() (dmx.stats.sequence.SequenceDistribution method), 125

seq_log_density() (dmx.stats.setdist.BernoulliSetDistribution method), 94

seq_log_density() (dmx.stats.sparse_markov_transform.SparseMarkovTransform method), 357

seq_log_density() (dmx.stats.spearman_rho.SpearmanRankingDistribution method), 226

seq_log_density() (dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution method), 290

seq_log_density() (dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution method), 365

seq_log_density() (dmx.stats.vmf.VonMisesFisherDistribution method), 233

seq_log_density() (dmx.stats.weighted.WeightedDistribution method), 240

seq_log_density() (dmx.stats.binomial.BinomialDistribution method), 537

seq_log_density() (dmx.torch_stats.composite.CompositeDistribution method), 606

seq_log_density() (dmx.torch_stats.conditional.ConditionalDistribution method), 609

seq_log_density() (dmx.torch_stats.dmvn.DiagonalGaussianDistribution method), 543

seq_log_density() (dmx.torch_stats.exponential.ExponentialDistribution method), 549

seq_log_density() (dmx.torch_stats.gamma.GammaDistribution method), 555

seq_log_density() (dmx.torch_stats.gaussian.GaussianDistribution method), 561

seq_log_density() (dmx.torch_stats.geometric.GeometricDistribution method), 567

seq_log_density() (dmx.torch_stats.heterogeneous_mixture.HeterogeneousMixtureDistribution method), 626

seq_log_density() (dmx.torch_stats.hmm.HiddenMarkovModelDistribution method), 634

seq_log_density() (dmx.torch_stats.int_plsi.IntegerPLSISDistribution method), 640

seq_log_density() (dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution method), 575

seq_log_density() (dmx.torch_stats.intrange.IntegerCategoricalDistribution method), 582

seq_log_density() (dmx.torch_stats.intsetdist.IntegerBernoulliDistribution method), 589

seq_log_density() (dmx.torch_stats.jmixture.JointMixtureDistribution method), 643

seq_log_density() (dmx.torch_stats.mixture.MixtureDistribution method), 650

seq_log_density() (dmx.torch_stats.mvn.MultivariateGaussianDistribution method), 596

seq_log_density() (dmx.torch_stats.null_dist.NullDistribution method), 616

seq_log_density() (dmx.torch_stats.pdist.TorchProbabilityDistribution method), 528

seq_log_density() (dmx.torch_stats.poisson.PoissonDistribution method), 601

seq_log_density() (dmx.torch_stats.sequence.SequenceDistribution method), 620

seq_log_density() (in module dmx.stats), 9

seq_log_density_lambda() (dmx.bstats.pdist.SequenceEncodableDistribution method), 381

seq_log_density_lambda() (dmx.stats.pdist.SequenceEncodableProbabilityDistribution method), 13

seq_log_density_mpi() (in module dmx.mpi4py.bstats), 707

seq_log_density_mpi() (in module dmx.mpi4py.stats), 704

seq_log_density_sum() (in module dmx.stats), 9

seq_log_density_sum_mpi() (in module dmx.mpi4py.bstats), 708

seq_log_density_sum_mpi() (in module dmx.mpi4py.stats), 705

seq_posterior() (dmx.bstats.mixture.MixtureDistribution method), 495

seq_posterior() (dmx.stats.dirac_length.DiracMixtureDistribution method), 193

seq_posterior() (dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution method), 250

seq_posterior() (dmx.stats.gmm.GaussianMixtureDistribution method), 263

seq_posterior() (dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution method), 158

seq_posterior() (dmx.stats.hidden_markov.HiddenMarkovModelDistribution method), 180

seq_posterior() (dmx.stats.hmixture.HierarchicalMixtureDistribution method), 172

seq_posterior() (dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution method), 329

seq_posterior() (dmx.stats.lda.LDADistribution method), 282

seq_posterior() (dmx.stats.look_back_hmm.LookbackHiddenMarkovDistribution method), 347

seq_posterior() (dmx.stats.mixture.MixtureDistribution method), 150

seq_posterior() (dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution method), 290

seq_posterior() (dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution method), 365

seq_posterior() (dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDistribution method), 626

seq_posterior() (dmx.torch_stats.mixture.MixtureDistribution method), 650

seq_posterior2() (in module dmx.stats.lda), 287

seq_update() (dmx.bstats.bernoulli.BernoulliEstimatorAccumulator method), 412

seq_update() (dmx.bstats.categorical.CategoricalEstimatorAccumulator method), 418

seq_update() (dmx.bstats.composite.CompositeEstimatorAccumulator method), 473

seq_update() (dmx.bstats.conditional.ConditionalDistributionEstimatorAccumulator method), 479

seq_update() (dmx.bstats.dirac.DiracAccumulator method), 420

seq_update() (dmx.bstats.dirichlet.DirichletAccumulator method), 395

seq_update() (dmx.bstats.dmvn.DiagonalGaussianAccumulator method), 426

seq_update() (dmx.bstats.dpm.DirichletProcessMixtureAccumulator method), 482

seq_update() (dmx.bstats.exponential.ExponentialAccumulator method), 432

seq_update() (dmx.bstats.gamma.GammaAccumulator method), 437

seq_update() (dmx.bstats.gaussian.GaussianAccumulator method), 444

seq_update() (dmx.bstats.geometric.GeometricAccumulator method), 450

seq_update() (dmx.bstats.ignored.IgnoredAccumulator method), 488

seq_update() (dmx.bstats.intrange.IntegerCategoricalAccumulator method), 456

seq_update() (dmx.bstats.mixture.MixtureEstimatorAccumulator method), 498

seq_update() (dmx.bstats.null_dist.NullAccumulator method), 500

seq_update() (dmx.bstats.optional.OptionalEstimatorAccumulator method), 510

seq_update() (dmx.bstats.pdist.SequenceEncodableAccumulator method), 380

<code>seq_update()</code> (<code>dmx.bstats.poisson.PoissonEstimatorAccumulator</code> method), 466	<code>seq_update()</code> (<code>dmx.torch_stats.conditional.ConditionalDistributionAccumulator</code> method), 610
<code>seq_update()</code> (<code>dmx.bstats.sequence.SequenceEstimatorAccumulator</code> method), 517	<code>seq_update()</code> (<code>dmx.torch_stats.dmvn.DiagonalGaussianAccumulator</code> method), 541
<code>seq_update()</code> (<code>dmx.bstats.setdist.BernoulliSetAccumulator</code> method), 520	<code>seq_update()</code> (<code>dmx.torch_stats.exponential.ExponentialAccumulator</code> method), 547
<code>seq_update()</code> (<code>dmx.stats.dirac_length.DiracMixtureAccumulator</code> method), 188	<code>seq_update()</code> (<code>dmx.torch_stats.gamma.GammaAccumulator</code> method), 553
<code>seq_update()</code> (<code>dmx.stats.dmvn_mixture.DiagonalGaussianAccumulator</code> method), 244	<code>seq_update()</code> (<code>dmx.torch_stats.gaussian.GaussianAccumulator</code> method), 559
<code>seq_update()</code> (<code>dmx.stats.gmm.GaussianMixtureAccumulator</code> method), 257	<code>seq_update()</code> (<code>dmx.torch_stats.geometric.GeometricAccumulator</code> method), 565
<code>seq_update()</code> (<code>dmx.stats.hidden_association.HiddenAssociationAccumulator</code> method), 296	<code>seq_update()</code> (<code>dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureAccumulator</code> method), 623
<code>seq_update()</code> (<code>dmx.stats.icltree.ICLTreeAccumulator</code> method), 302	<code>seq_update()</code> (<code>dmx.torch_stats.hmm.HiddenMarkovAccumulator</code> method), 629
<code>seq_update()</code> (<code>dmx.stats.int_edit_setdist.IntegerBernoulliSetAccumulator</code> method), 198	<code>seq_update()</code> (<code>dmx.torch_stats.int_plsi.IntegerPLSIAccumulator</code> method), 638
<code>seq_update()</code> (<code>dmx.stats.int_edit_stepsetdist.IntegerStepBasedAccumulator</code> method), 206	<code>seq_update()</code> (<code>dmx.torch_stats.intmultinomial.IntegerMultinomialAccumulator</code> method), 571
<code>seq_update()</code> (<code>dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator</code> method), 308	<code>seq_update()</code> (<code>dmx.torch_stats.intrange.IntegerCategoricalAccumulator</code> method), 580
<code>seq_update()</code> (<code>dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator</code> method), 322	<code>seq_update()</code> (<code>dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulator</code> method), 587
<code>seq_update()</code> (<code>dmx.stats.int_markovchain.IntegerMarkovChainAccumulator</code> method), 337	<code>seq_update()</code> (<code>dmx.torch_stats.jmixture.JointMixtureEstimatorAccumulator</code> method), 645
<code>seq_update()</code> (<code>dmx.stats.int_plsi.IntegerPLSIAccumulator</code> method), 269	<code>seq_update()</code> (<code>dmx.torch_stats.mixture.MixtureAccumulator</code> method), 648
<code>seq_update()</code> (<code>dmx.stats.lda.LDAEstimatorAccumulator</code> method), 285	<code>seq_update()</code> (<code>dmx.torch_stats.mvn.MultivariateGaussianAccumulator</code> method), 593
<code>seq_update()</code> (<code>dmx.stats.look_back_hmm.LookbackHiddenMarkovModelDistribution</code> method), 350	<code>seq_update()</code> (<code>dmx.torch_stats.null_dist.NullAccumulator</code> method), 614
<code>seq_update()</code> (<code>dmx.stats.null_dist.NullAccumulator</code> method), 212	<code>seq_update()</code> (<code>dmx.torch_stats.pdist.TorchStatisticAccumulator</code> method), 531
<code>seq_update()</code> (<code>dmx.stats.pdist.SequenceEncodableStatisticsAccumulator</code> method), 17	<code>seq_update()</code> (<code>dmx.torch_stats.poisson.PoissonAccumulator</code> method), 599
<code>seq_update()</code> (<code>dmx.stats.select.SelectEstimatorAccumulator</code> method), 220	<code>seq_update()</code> (<code>dmx.torch_stats.sequence.SequenceAccumulator</code> method), 618
<code>seq_update()</code> (<code>dmx.stats.sparse_markov_transform.SparseMarkovTransformAccumulator</code> method), 355	<code>seq_update()</code> (<code>dmx.torch_stats.sequence.SequenceAccumulator</code> method), 618
<code>seq_update()</code> (<code>dmx.stats.spearman_rho.SpearmanRankingAccumulator</code> method), 223	<code>seq_update()</code> (<code>dmx.torch_stats.sequence.SequenceAccumulator</code> method), 618
<code>seq_update()</code> (<code>dmx.stats.ss_mixture.SemiSupervisedMixtureAccumulator</code> method), 293	<code>seq_update()</code> (<code>dmx.torch_stats.sequence.SequenceAccumulator</code> method), 618
<code>seq_update()</code> (<code>dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator</code> method), 361	<code>seq_update()</code> (<code>dmx.torch_stats.sequence.SequenceAccumulator</code> method), 618
<code>seq_update()</code> (<code>dmx.stats.vmf.VonMisesFisherAccumulator</code> method), 230	<code>seq_update()</code> (<code>dmx.torch_stats.sequence.SequenceAccumulator</code> method), 618
<code>seq_update()</code> (<code>dmx.stats.weighted.WeightedAccumulator</code> method), 238	<code>seq_update()</code> (<code>dmx.torch_stats.sequence.SequenceAccumulator</code> method), 618
<code>seq_update()</code> (<code>dmx.torch_stats.binomial.BinomialAccumulator</code> method), 533	<code>seq_update()</code> (<code>dmx.torch_stats.sequence.SequenceAccumulator</code> method), 618
<code>seq_update()</code> (<code>dmx.torch_stats.composite.CompositeAccumulator</code> method), 604	<code>seq_update()</code> (<code>dmx.torch_stats.sequence.SequenceAccumulator</code> method), 618

SequenceDistribution	(class in <i>dmx.torch_stats.sequence</i>), 619	in <i>set_parameters()</i> (<i>dmx.bstats.geometric.GeometricDistribution</i> method), 453
SequenceEncodableAccumulator	(class in <i>dmx.bstats.pdist</i>), 380	in <i>set_parameters()</i> (<i>dmx.bstats.ignored.IgnoredDistribution</i> method), 491
SequenceEncodableDistribution	(class in <i>dmx.bstats.pdist</i>), 381	in <i>set_parameters()</i> (<i>dmx.bstats.intrange.IntegerCategoricalDistribution</i> method), 459
SequenceEncodableProbabilityDistribution	(class in <i>dmx.stats.pdist</i>), 12	<i>set_parameters()</i> (<i>dmx.bstats.mixture.MixtureDistribution</i> method), 495
SequenceEncodableStatisticAccumulator	(class in <i>dmx.stats.pdist</i>), 16	<i>set_parameters()</i> (<i>dmx.bstats.mvngamma.MultivariateNormalGammaD</i> method), 403
SequenceEncodedData	(class in <i>dmx.bstats.sequence</i>), 514	<i>set_parameters()</i> (<i>dmx.bstats.normgamma.NormalGammaDistribution</i> method), 405
SequenceEstimator	(class in <i>dmx.bstats.sequence</i>), 515	<i>set_parameters()</i> (<i>dmx.bstats.nulldist.NullDistribution</i> method), 503
SequenceEstimator	(class in <i>dmx.stats.sequence</i>), 126	<i>set_parameters()</i> (<i>dmx.bstats.optional.OptionalDistribution</i> method), 507
SequenceEstimator	(class in <i>dmx.torch_stats.sequence</i>), 621	in <i>set_parameters()</i> (<i>dmx.bstats.pdist.ProbabilityDistribution</i> method), 379
SequenceEstimatorAccumulator	(class in <i>dmx.bstats.sequence</i>), 516	in <i>set_parameters()</i> (<i>dmx.bstats.poisson.PoissonDistribution</i> method), 463
SequenceEstimatorAccumulatorFactory	(class in <i>dmx.bstats.sequence</i>), 518	<i>set_parameters()</i> (<i>dmx.bstats.sequence.SequenceDistribution</i> method), 514
SequenceSampler	(class in <i>dmx.bstats.sequence</i>), 518	<i>set_parameters()</i> (<i>dmx.bstats.setdist.BernoulliSetDistribution</i> method), 522
SequenceSampler	(class in <i>dmx.stats.sequence</i>), 128	<i>set_parameters()</i> (<i>dmx.bstats.setdist.BernoulliSetDistribution</i> method), 522
SequenceSampler	(class in <i>dmx.torch_stats.sequence</i>), 621	in <i>set_parameters()</i> (<i>dmx.bstats.syndirichlet.SymmetricDirichletDistributi</i> method), 407
SequenceTorchEncodedSequence	(class in <i>dmx.torch_stats.sequence</i>), 622	in <i>set_parameters()</i> (<i>dmx.bstats.syndirichlet.SymmetricDirichletDistributi</i> method), 407
set_default_float_dtype()	(in <i>dmx.torch_utils.vector</i>), 701	in <i>module</i> <i>set_prior()</i> (<i>dmx.bstats.bernoulli.BernoulliDistribution</i> method), 410
set_name()	(<i>dmx.bstats.pdist.ProbabilityDistribution</i> method), 379	<i>set_prior()</i> (<i>dmx.bstats.bernoulli.BernoulliEstimator</i> method), 411
set_parameters()	(<i>dmx.bstats.bernoulli.BernoulliDistribution</i> method), 410	<i>set_prior()</i> (<i>dmx.bstats.bernoulli.BernoulliEstimator</i> method), 411
set_parameters()	(<i>dmx.bstats.beta.BetaDistribution</i> method), 391	<i>set_prior()</i> (<i>dmx.bstats.categorical.CategoricalDistribution</i> method), 416
set_parameters()	(<i>dmx.bstats.beta.BetaDistribution</i> method), 391	<i>set_prior()</i> (<i>dmx.bstats.categorical.CategoricalEstimator</i> method), 417
set_parameters()	(<i>dmx.bstats.catdirichlet.DictDirichletDistribution</i> method), 393	<i>set_prior()</i> (<i>dmx.bstats.categorical.CategoricalDistribution</i> method), 416
set_parameters()	(<i>dmx.bstats.catdirichlet.DictDirichletDistribution</i> method), 393	<i>set_prior()</i> (<i>dmx.bstats.categorical.CategoricalEstimator</i> method), 417
set_parameters()	(<i>dmx.bstats.categorical.CategoricalDistribution</i> method), 416	<i>set_prior()</i> (<i>dmx.bstats.categorical.CategoricalDistribution</i> method), 416
set_parameters()	(<i>dmx.bstats.categorical.CategoricalDistribution</i> method), 416	<i>set_prior()</i> (<i>dmx.bstats.categorical.CategoricalEstimator</i> method), 417
set_parameters()	(<i>dmx.bstats.composite.CompositeDistribution</i> method), 470	<i>set_prior()</i> (<i>dmx.bstats.composite.CompositeDistribution</i> method), 470
set_parameters()	(<i>dmx.bstats.composite.CompositeDistribution</i> method), 470	<i>set_prior()</i> (<i>dmx.bstats.composite.CompositeEstimator</i> method), 471
set_parameters()	(<i>dmx.bstats.composite.CompositeDistribution</i> method), 470	<i>set_prior()</i> (<i>dmx.bstats.composite.CompositeEstimator</i> method), 471
set_parameters()	(<i>dmx.bstats.composite.CompositeDistribution</i> method), 470	<i>set_prior()</i> (<i>dmx.bstats.dirac.DiracDistribution</i> method), 423
set_parameters()	(<i>dmx.bstats.dirac.DiracDistribution</i> method), 423	<i>set_prior()</i> (<i>dmx.bstats.dirac.DiracDistribution</i> method), 423
set_parameters()	(<i>dmx.bstats.dirac.DiracDistribution</i> method), 423	<i>set_prior()</i> (<i>dmx.bstats.dirac.DiracEstimator</i> method), 424
set_parameters()	(<i>dmx.bstats.dirac.DiracDistribution</i> method), 423	<i>set_prior()</i> (<i>dmx.bstats.dirac.DiracEstimator</i> method), 424
set_parameters()	(<i>dmx.bstats.dirichlet.DirichletDistribution</i> method), 398	<i>set_prior()</i> (<i>dmx.bstats.dirichlet.DirichletDistribution</i> method), 429
set_parameters()	(<i>dmx.bstats.dirichlet.DirichletDistribution</i> method), 398	<i>set_prior()</i> (<i>dmx.bstats.dirichlet.DirichletDistribution</i> method), 429
set_parameters()	(<i>dmx.bstats.dmvn.DiagonalGaussianDistribution</i> method), 429	<i>set_prior()</i> (<i>dmx.bstats.dmvn.DiagonalGaussianDistribution</i> method), 429
set_parameters()	(<i>dmx.bstats.dmvn.DiagonalGaussianDistribution</i> method), 429	<i>set_prior()</i> (<i>dmx.bstats.dmvn.DiagonalGaussianEstimator</i> method), 430
set_parameters()	(<i>dmx.bstats.dmvn.DiagonalGaussianDistribution</i> method), 429	<i>set_prior()</i> (<i>dmx.bstats.dmvn.DiagonalGaussianEstimator</i> method), 430
set_parameters()	(<i>dmx.bstats.dpm.DirichletProcessMixtureDistribution</i> method), 485	<i>set_prior()</i> (<i>dmx.bstats.dpm.DirichletProcessMixtureDistribution</i> method), 485
set_parameters()	(<i>dmx.bstats.dpm.DirichletProcessMixtureDistribution</i> method), 485	<i>set_prior()</i> (<i>dmx.bstats.dpm.DirichletProcessMixtureDistribution</i> method), 485
set_parameters()	(<i>dmx.bstats.dpm.DirichletProcessMixtureDistribution</i> method), 485	<i>set_prior()</i> (<i>dmx.bstats.dpm.DirichletProcessMixtureEstimator</i> method), 486
set_parameters()	(<i>dmx.bstats.dpm.DirichletProcessMixtureDistribution</i> method), 485	<i>set_prior()</i> (<i>dmx.bstats.dpm.DirichletProcessMixtureEstimator</i> method), 486
set_parameters()	(<i>dmx.bstats.exponential.ExponentialDistribution</i> method), 435	<i>set_prior()</i> (<i>dmx.bstats.exponential.ExponentialDistribution</i> method), 435
set_parameters()	(<i>dmx.bstats.exponential.ExponentialDistribution</i> method), 435	<i>set_prior()</i> (<i>dmx.bstats.exponential.ExponentialDistribution</i> method), 435
set_parameters()	(<i>dmx.bstats.gamma.GammaDistribution</i> method), 440	<i>set_prior()</i> (<i>dmx.bstats.exponential.ExponentialDistribution</i> method), 435
set_parameters()	(<i>dmx.bstats.gamma.GammaDistribution</i> method), 440	<i>set_prior()</i> (<i>dmx.bstats.exponential.ExponentialEstimator</i> method), 436
set_parameters()	(<i>dmx.bstats.gaussian.GaussianDistribution</i> method), 446	<i>set_prior()</i> (<i>dmx.bstats.exponential.ExponentialEstimator</i> method), 436
set_parameters()	(<i>dmx.bstats.gaussian.GaussianDistribution</i> method), 446	<i>set_prior()</i> (<i>dmx.bstats.exponential.ExponentialEstimator</i> method), 436

<code>set_prior()</code> (<i>dmx.bstats.gamma.GammaEstimator</i> method), 442	<code>single_sample()</code> (<i>dmx.stats.conditional.ConditionalDistributionSampler</i> method), 135
<code>set_prior()</code> (<i>dmx.bstats.gaussian.GaussianDistribution</i> method), 446	<code>single_sample()</code> (<i>dmx.stats.int_markovchain.IntegerMarkovChainSampler</i> method), 345
<code>set_prior()</code> (<i>dmx.bstats.gaussian.GaussianEstimator</i> method), 448	<code>single_sample()</code> (<i>dmx.torch_stats.conditional.ConditionalDistributionSampler</i> method), 613
<code>set_prior()</code> (<i>dmx.bstats.geometric.GeometricDistributionsize_rng</i> method), 453	(<i>dmx.stats.int_plsi.IntegerPLSISampler</i> attribute), 276
<code>set_prior()</code> (<i>dmx.bstats.geometric.GeometricEstimator</i> method), 454	<code>sorted_dict_merge_add()</code> (in module <i>dmx.utils.vector</i>), 690
<code>set_prior()</code> (<i>dmx.bstats.ignored.IgnoredDistribution</i> method), 491	<code>sorted_merge()</code> (in module <i>dmx.utils.vector</i>), 691
<code>set_prior()</code> (<i>dmx.bstats.ignored.IgnoredEstimator</i> method), 492	<code>SparseMarkovAssociationAccumulator</code> (class in <i>dmx.stats.sparse_markov_transform</i>), 353
<code>set_prior()</code> (<i>dmx.bstats.intrange.IntegerCategoricalDistribution</i> method), 459	<code>SparseMarkovAssociationAccumulatorFactory</code> (class in <i>dmx.stats.sparse_markov_transform</i>), 355
<code>set_prior()</code> (<i>dmx.bstats.intrange.IntegerCategoricalEstimator</i> method), 460	<code>SparseMarkovAssociationDataEncoder</code> (class in <i>dmx.stats.sparse_markov_transform</i>), 355
<code>set_prior()</code> (<i>dmx.bstats.mixture.MixtureDistribution</i> method), 495	<code>SparseMarkovAssociationDistribution</code> (class in <i>dmx.stats.sparse_markov_transform</i>), 356
<code>set_prior()</code> (<i>dmx.bstats.mixture.MixtureEstimator</i> method), 496	<code>SparseMarkovAssociationEncodedDataSequence</code> (class in <i>dmx.stats.sparse_markov_transform</i>), 357
<code>set_prior()</code> (<i>dmx.bstats.nulldist.NullDistribution</i> method), 503	<code>SparseMarkovAssociationEstimator</code> (class in <i>dmx.stats.sparse_markov_transform</i>), 358
<code>set_prior()</code> (<i>dmx.bstats.nulldist.NullEstimator</i> method), 504	<code>SparseMarkovAssociationSampler</code> (class in <i>dmx.stats.sparse_markov_transform</i>), 359
<code>set_prior()</code> (<i>dmx.bstats.optional.OptionalDistribution</i> method), 508	<code>SpearmanRankingAccumulator</code> (class in <i>dmx.stats.spearman_rho</i>), 221
<code>set_prior()</code> (<i>dmx.bstats.optional.OptionalEstimator</i> method), 509	<code>SpearmanRankingAccumulatorFactory</code> (class in <i>dmx.stats.spearman_rho</i>), 223
<code>set_prior()</code> (<i>dmx.bstats.pdist.ParameterEstimator</i> method), 375	<code>SpearmanRankingDataEncoder</code> (class in <i>dmx.stats.spearman_rho</i>), 224
<code>set_prior()</code> (<i>dmx.bstats.pdist.ProbabilityDistribution</i> method), 379	<code>SpearmanRankingDistribution</code> (class in <i>dmx.stats.spearman_rho</i>), 224
<code>set_prior()</code> (<i>dmx.bstats.poisson.PoissonDistribution</i> method), 464	<code>SpearmanRankingEncodedDataSequence</code> (class in <i>dmx.stats.spearman_rho</i>), 226
<code>set_prior()</code> (<i>dmx.bstats.poisson.PoissonEstimator</i> method), 465	<code>SpearmanRankingEstimator</code> (class in <i>dmx.stats.spearman_rho</i>), 226
<code>set_prior()</code> (<i>dmx.bstats.sequence.SequenceDistribution</i> method), 514	<code>SpearmanRankingSampler</code> (class in <i>dmx.stats.spearman_rho</i>), 227
<code>set_prior()</code> (<i>dmx.bstats.sequence.SequenceEstimator</i> method), 516	<code>SpikeAndSlabDistribution</code> (class in <i>dmx.stats.int_spike</i>), 40
<code>set_prior()</code> (<i>dmx.bstats.setdist.BernoulliSetDistribution</i> method), 523	<code>SpikeAndSlabEstimator</code> (class in <i>dmx.stats.int_spike</i>), 42
<code>set_prior()</code> (<i>dmx.bstats.setdist.BernoulliSetEstimator</i> method), 524	<code>SpikeAndSlabSampler</code> (class in <i>dmx.stats.int_spike</i>), 44
<code>sigma</code> (<i>dmx.stats.spearman_rho.SpearmanRankingDistribution</i> attribute), 224	<code>sum</code> (<i>dmx.stats.vmf.VonMisesFisherAccumulator</i> attribute), 228
<code>sigma2</code> (<i>dmx.stats.gaussian.GaussianDistribution</i> attribute), 45	<code>state_counts</code> (<i>dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator</i> attribute), 319
<code>sigma2</code> (<i>dmx.stats.gmm.GaussianMixtureDistribution</i> attribute), 260	<code>state_key</code> (<i>dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator</i> attribute), 319
<code>sigma2</code> (<i>dmx.stats.log_gaussian.LogGaussianDistribution</i> attribute), 61	<code>state_mat</code> (<i>dmx.stats.int_plsi.IntegerPLSIDistribution</i> attribute), 271

`state_prob_mat` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution` attribute), 311
`state_sampler` (`dmx.stats.hidden_markov.HiddenMarkovSampler` attribute), 32
`state_sampler` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler` attribute), 330
`StatisticAccumulator` (class in `dmx.bstats.pdist`), 381
`StatisticAccumulator` (class in `dmx.stats.pdist`), 14
`StatisticAccumulatorFactory` (class in `dmx.bstats.pdist`), 383
`stirling2()` (in module `dmx.utils.special`), 683
`str_count` (`dmx.utils.automatic.DatumNode` attribute), 653
`suff_stat` (`dmx.stats.binomial.BinomialEstimator` attribute), 23
`suff_stat` (`dmx.stats.categorical.CategoricalEstimator` attribute), 85
`suff_stat` (`dmx.stats.dirac_length.DiracMixtureEstimator` attribute), 195
`suff_stat` (`dmx.stats.dirichlet.DirichletEstimator` attribute), 79
`suff_stat` (`dmx.stats.exponential.ExponentialEstimator` attribute), 52
`suff_stat` (`dmx.stats.gamma.GammaEstimator` attribute), 57
`suff_stat` (`dmx.stats.gaussian.GaussianEstimator` attribute), 48
`suff_stat` (`dmx.stats.geometric.GeometricEstimator` attribute), 28
`suff_stat` (`dmx.stats.heterogeneous_mixture.HeterogeneousMixtureEstimator` attribute), 159
`suff_stat` (`dmx.stats.hmixture.HierarchicalMixtureEstimator` attribute), 173
`suff_stat` (`dmx.stats.ignored.IgnoredEstimator` attribute), 144
`suff_stat` (`dmx.stats.int_edit_setdist.IntegerBernoulliEditEstimator` attribute), 203
`suff_stat` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator` attribute), 315
`suff_stat` (`dmx.stats.int_spike.SpikeAndSlabEstimator` attribute), 43
`suff_stat` (`dmx.stats.intmultinomial.IntegerMultinomialEstimator` attribute), 101
`suff_stat` (`dmx.stats.intrange.IntegerCategoricalEstimator` attribute), 37
`suff_stat` (`dmx.stats.intsetdist.IntegerBernoulliSetEstimator` attribute), 90
`suff_stat` (`dmx.stats.jmixture.JointMixtureEstimator` attribute), 165
`suff_stat` (`dmx.stats.log_gaussian.LogGaussianEstimator` attribute), 63
`suff_stat` (`dmx.stats.mixture.MixtureEstimator` attribute), 151
`suff_stat` (`dmx.stats.null_dist.NullEstimator` attribute),
`suff_stat` (`dmx.stats.poisson.PoissonEstimator` attribute), 35
`suff_stat` (`dmx.stats.setdist.BernoulliSetEstimator` attribute), 95
`suff_stat` (`dmx.stats.sparse_markov_transform.SparseMarkovAssociationEstimator` attribute), 358
`suff_stat` (`dmx.stats.spearman_rho.SpearmanRankingEstimator` attribute), 226
`suff_stat` (`dmx.stats.ss_mixture.SemiSupervisedMixtureEstimator` attribute), 291
`suff_stat` (`dmx.torch_stats.binomial.BinomialEstimator` attribute), 538
`suff_stat` (`dmx.torch_stats.gamma.GammaEstimator` attribute), 556
`suff_stat` (`dmx.torch_stats.gaussian.GaussianEstimator` attribute), 562
`suff_stat` (`dmx.torch_stats.geometric.GeometricEstimator` attribute), 568
`suff_stat` (`dmx.torch_stats.intmultinomial.IntegerMultinomialEstimator` attribute), 576
`suff_stat` (`dmx.torch_stats.intrange.IntegerCategoricalEstimator` attribute), 583
`suff_stat` (`dmx.torch_stats.intsetdist.IntegerBernoulliSetEstimator` attribute), 590
`suff_stat` (`dmx.torch_stats.poisson.PoissonEstimator` attribute), 601
`sum` (`dmx.stats.spearman_rho.SpearmanRankingAccumulator` attribute), 221
`sum` (`dmx.stats.binomial.BinomialAccumulator` attribute), 532
`sum` (`dmx.torch_stats.dmvn.DiagonalGaussianAccumulator` attribute), 540
`sum` (`dmx.torch_stats.exponential.ExponentialAccumulator` attribute), 546
`sum` (`dmx.torch_stats.gamma.GammaAccumulator` attribute), 551
`sum` (`dmx.torch_stats.gaussian.GaussianAccumulator` attribute), 558
`sum` (`dmx.torch_stats.geometric.GeometricAccumulator` attribute), 564
`sum` (`dmx.torch_stats.poisson.PoissonAccumulator` attribute), 598
`sum2` (`dmx.torch_stats.dmvn.DiagonalGaussianAccumulator` attribute), 540
`sum2` (`dmx.torch_stats.gaussian.GaussianAccumulator` attribute), 558
`sum_by_key()` (in module `dmx.utils.optsutil`), 680
`sum_of_logs` (`dmx.torch_stats.gamma.GammaAccumulator` attribute), 552
`SymmetricDirichletDistribution` (class in `dmx.bstats.syndirichlet`), 405
`SymmetricDirichletSampler` (class in `dmx.bstats.syndirichlet`), 407

T

- [t_cond_prob_mat\(\)](#) (in module *dmx.utils.htsne*), 671
[t_cond_prob_mat_alpha\(\)](#) (in module *dmx.utils.htsne*), 672
[take_sample\(\)](#) (in module *dmx.stats.rdd_sampler*), 371
[taus](#) (*dmx.stats.hidden_markov.HiddenMarkovModelDistribution* attribute), 177
[taus](#) (*dmx.stats.hmixture.HierarchicalMixtureDistribution* attribute), 169
[taus](#) (*dmx.torch_stats.hmm.HiddenMarkovModelDistribution* attribute), 633
[taus12](#) (*dmx.stats.jmixture.JointMixtureDistribution* attribute), 162
[taus21](#) (*dmx.stats.jmixture.JointMixtureDistribution* attribute), 162
[tensor\(\)](#) (in module *dmx.torch_utils.vector*), 701
[terminal_level](#) (*dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution* attribute), 364
[terminal_set](#) (*dmx.stats.hidden_markov.HiddenMarkovSampler* attribute), 184
[terminal_set](#) (*dmx.stats.int_hidden_markov.IntegerHiddenMarkovSampler* attribute), 330
[terminal_values](#) (*dmx.stats.hidden_markov.HiddenMarkovModelDistribution* attribute), 178
[terminal_values](#) (*dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution* attribute), 327
[terminal_values](#) (*dmx.torch_stats.hmm.HiddenMarkovModelDistribution* attribute), 633
[text_file\(\)](#) (in module *dmx.utils.optsutil*), 680
[theta](#) (*dmx.stats.gamma.GammaDistribution* attribute), 55
[threshold](#) (*dmx.stats.gamma.GammaEstimator* attribute), 58
[threshold](#) (*dmx.torch_stats.gamma.GammaEstimator* attribute), 556
[tied](#) (*dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator* attribute), 242
[tied](#) (*dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulatorFactory* attribute), 246
[tied](#) (*dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution* attribute), 247
[tied](#) (*dmx.stats.dmvn_mixture.DiagonalGaussianMixtureEstimator* attribute), 252
[tied](#) (*dmx.stats.gmm.GaussianMixtureAccumulator* attribute), 255
[tied](#) (*dmx.stats.gmm.GaussianMixtureAccumulatorFactory* attribute), 259
[tied](#) (*dmx.stats.gmm.GaussianMixtureEstimator* attribute), 264
[tng](#) (*dmx.torch_stats.gaussian.GaussianSampler* attribute), 563
[to\(\)](#) (*dmx.torch_stats.binomial.BinomialDistribution* method), 537
[to\(\)](#) (*dmx.torch_stats.composite.CompositeDistribution* method), 606
[to\(\)](#) (*dmx.torch_stats.conditional.ConditionalDistribution* method), 609
[to\(\)](#) (*dmx.torch_stats.dmvn.DiagonalGaussianDistribution* method), 543
[to\(\)](#) (*dmx.torch_stats.exponential.ExponentialDistribution* method), 550
[to\(\)](#) (*dmx.torch_stats.gamma.GammaDistribution* method), 555
[to\(\)](#) (*dmx.torch_stats.gaussian.GaussianDistribution* method), 562
[to\(\)](#) (*dmx.torch_stats.geometric.GeometricDistribution* method), 567
[to\(\)](#) (*dmx.torch_stats.heterogenous_mixture.HeterogeneousMixtureDistribution* method), 626
[to\(\)](#) (*dmx.torch_stats.hmm.HiddenMarkovModelDistribution* method), 635
[to\(\)](#) (*dmx.torch_stats.int_plsi.IntegerPLSIDistribution* method), 640
[to\(\)](#) (*dmx.torch_stats.intmultinomial.IntegerMultinomialDistribution* method), 575
[to\(\)](#) (*dmx.torch_stats.intrange.IntegerCategoricalDistribution* method), 583
[to\(\)](#) (*dmx.torch_stats.intsetdist.IntegerBernoulliSetDistribution* method), 589
[to\(\)](#) (*dmx.torch_stats.jmixture.JointMixtureDistribution* method), 643
[to\(\)](#) (*dmx.torch_stats.mixture.MixtureDistribution* method), 650
[to\(\)](#) (*dmx.torch_stats.mvn.MultivariateGaussianDistribution* method), 596
[to\(\)](#) (*dmx.torch_stats.null_dist.NullDistribution* method), 616
[to\(\)](#) (*dmx.torch_stats.pdist.TorchProbabilityDistribution* method), 528
[to\(\)](#) (*dmx.torch_stats.poisson.PoissonDistribution* method), 601
[to\(\)](#) (*dmx.torch_stats.sequence.SequenceDistribution* method), 621
[to_mixture\(\)](#) (*dmx.stats.hmixture.HierarchicalMixtureDistribution* method), 172
[topics](#) (*dmx.stats.hidden_markov.HiddenMarkovModelDistribution* attribute), 177
[topics](#) (*dmx.stats.hmixture.HierarchicalMixtureDistribution* attribute), 168
[topics](#) (*dmx.stats.lda.LDADistribution* attribute), 280
[topics](#) (*dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution* attribute), 364
[topics](#) (*dmx.torch_stats.hmm.HiddenMarkovModelDistribution* attribute), 632
[TorchEncodedSequence](#) (class in *dmx.torch_stats.pdist*), 526
[TorchParameterEstimator](#) (class in *dmx.torch_stats.pdist*), 526

<code>TorchProbabilityDistribution</code> (class in <code>dmx.torch_stats.pdist</code>), 527	<code>update()</code> (<code>dmx.bstats.categorical.CategoricalEstimatorAccumulator</code> method), 418
<code>TorchSequenceEncoder</code> (class in <code>dmx.torch_stats.pdist</code>), 529	<code>update()</code> (<code>dmx.bstats.composite.CompositeEstimatorAccumulator</code> method), 474
<code>TorchStatisticAccumulator</code> (class in <code>dmx.torch_stats.pdist</code>), 529	<code>update()</code> (<code>dmx.bstats.conditional.ConditionalDistributionEstimatorAccumulator</code> method), 479
<code>TorchStatisticAccumulatorFactory</code> (class in <code>dmx.torch_stats.pdist</code>), 531	<code>update()</code> (<code>dmx.bstats.dirac.DiracAccumulator</code> method), 421
<code>tot_sum</code> (<code>dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator</code> attribute), 197	<code>update()</code> (<code>dmx.bstats.dirichlet.DirichletAccumulator</code> method), 395
<code>tot_sum</code> (<code>dmx.torch_stats.intsetdist.IntegerBernoulliSetAccumulator</code> attribute), 586	<code>update()</code> (<code>dmx.bstats.dmvn.DiagonalGaussianAccumulator</code> method), 426
<code>trans_count_map</code> (<code>dmx.stats.int_markovchain.IntegerMarkovChainDiscreteAccumulator</code> attribute), 335	<code>update()</code> (<code>dmx.bstats.dpm.DirichletProcessMixtureAccumulator</code> method), 482
<code>trans_counts</code> (<code>dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDiscreteAccumulator</code> attribute), 319	<code>update()</code> (<code>dmx.bstats.exponential.ExponentialAccumulator</code> method), 432
<code>trans_key</code> (<code>dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDiscreteAccumulator</code> attribute), 319	<code>update()</code> (<code>dmx.bstats.gamma.GammaAccumulator</code> method), 438
<code>trans_log_pvec</code> (<code>dmx.stats.markovchain.MarkovChainDiscreteAccumulator</code> attribute), 111	<code>update()</code> (<code>dmx.bstats.gaussian.GaussianAccumulator</code> method), 444
<code>trans_prob</code> (<code>dmx.stats.markovchain.MarkovChainSampler</code> attribute), 116	<code>update()</code> (<code>dmx.bstats.geometric.GeometricAccumulator</code> method), 450
<code>trans_sampler</code> (<code>dmx.stats.int_markovchain.IntegerMarkovChainDiscreteAccumulator</code> attribute), 344	<code>update()</code> (<code>dmx.bstats.ignored.IgnoredAccumulator</code> method), 488
<code>transition_map</code> (<code>dmx.stats.markovchain.MarkovChainDiscreteAccumulator</code> attribute), 110	<code>update()</code> (<code>dmx.bstats.intrange.IntegerCategoricalAccumulator</code> method), 456
<code>transitions</code> (<code>dmx.stats.hidden_markov.HiddenMarkovModelDiscreteAccumulator</code> attribute), 177	<code>update()</code> (<code>dmx.bstats.mixture.MixtureEstimatorAccumulator</code> method), 498
<code>transitions</code> (<code>dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDiscreteAccumulator</code> attribute), 327	<code>update()</code> (<code>dmx.bstats.nulldist.NullAccumulator</code> method), 501
<code>transitions</code> (<code>dmx.stats.tree_hmm.TreeHiddenMarkovModelDiscreteAccumulator</code> attribute), 364	<code>update()</code> (<code>dmx.bstats.optional.OptionalEstimatorAccumulator</code> method), 511
<code>transitions</code> (<code>dmx.torch_stats.hmm.HiddenMarkovModelDiscreteAccumulator</code> attribute), 633	<code>update()</code> (<code>dmx.bstats.pdist.StatisticAccumulator</code> method), 383
<code>TreeHiddenMarkovAccumulator</code> (class in <code>dmx.stats.tree_hmm</code>), 360	<code>update()</code> (<code>dmx.bstats.poisson.PoissonEstimatorAccumulator</code> method), 466
<code>TreeHiddenMarkovAccumulatorFactory</code> (class in <code>dmx.stats.tree_hmm</code>), 362	<code>update()</code> (<code>dmx.bstats.sequence.SequenceEstimatorAccumulator</code> method), 517
<code>TreeHiddenMarkovDataEncoder</code> (class in <code>dmx.stats.tree_hmm</code>), 362	<code>update()</code> (<code>dmx.bstats.setdist.BernoulliSetAccumulator</code> method), 520
<code>TreeHiddenMarkovEncodedDataSequence</code> (class in <code>dmx.stats.tree_hmm</code>), 363	<code>update()</code> (<code>dmx.stats.dirac_length.DiracMixtureAccumulator</code> method), 189
<code>TreeHiddenMarkovEstimator</code> (class in <code>dmx.stats.tree_hmm</code>), 363	<code>update()</code> (<code>dmx.stats.dmvn_mixture.DiagonalGaussianMixtureAccumulator</code> method), 245
<code>TreeHiddenMarkovModelDistribution</code> (class in <code>dmx.stats.tree_hmm</code>), 364	<code>update()</code> (<code>dmx.stats.gmm.GaussianMixtureAccumulator</code> method), 258
<code>TreeHiddenMarkovSampler</code> (class in <code>dmx.stats.tree_hmm</code>), 366	<code>update()</code> (<code>dmx.stats.hidden_association.HiddenAssociationAccumulator</code> method), 296
<code>trigamma()</code> (in module <code>dmx.utils.special</code>), 683	<code>update()</code> (<code>dmx.stats.icltree.ICLTreeAccumulator</code> method), 303
U	<code>update()</code> (<code>dmx.stats.int_edit_setdist.IntegerBernoulliEditAccumulator</code> method), 199
<code>update()</code> (<code>dmx.bstats.bernoulli.BernoulliEstimatorAccumulator</code> method), 412	<code>update()</code> (<code>dmx.stats.int_edit_stepsetdist.IntegerStepBernoulliEditAccumulator</code> method), 206

`update()` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationAccumulator` attribute), 309
`update()` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator` attribute), 322
`update()` (`dmx.stats.int_markovchain.IntegerMarkovChainAccumulator` attribute), 337
`update()` (`dmx.stats.int_plsi.IntegerPLSIAccumulator` attribute), 269
`update()` (`dmx.stats.lda.LDAEstimatorAccumulator` attribute), 285
`update()` (`dmx.stats.look_back_hmm.LookbackHiddenMarkovAccumulator` attribute), 350
`update()` (`dmx.stats.null_dist.NullAccumulator` attribute), 212
`update()` (`dmx.stats.pdist.StatisticAccumulator` attribute), 16
`update()` (`dmx.stats.select.SelectEstimatorAccumulator` attribute), 220
`update()` (`dmx.stats.sparse_markov_transform.SparseMarkovTransformAccumulator` attribute), 355
`update()` (`dmx.stats.spearman_rho.SpearmanRankingAccumulator` attribute), 223
`update()` (`dmx.stats.ss_mixture.SemiSupervisedMixtureEstimatorAccumulator` attribute), 293
`update()` (`dmx.stats.tree_hmm.TreeHiddenMarkovAccumulator` attribute), 362
`update()` (`dmx.stats.vmf.VonMisesFisherAccumulator` attribute), 230
`update()` (`dmx.stats.weighted.WeightedAccumulator` attribute), 238
`update_alpha()` (in module `dmx.stats.lda`), 287
`update_alpha()` (in module `dmx.utils.htsne`), 672
`update_embed()` (in module `dmx.utils.htsne`), 673
`use_lstsq` (`dmx.stats.mvn.MultivariateGaussianDistribution`.self attribute), 71
`use_mpe` (`dmx.stats.dirichlet.DirichletEstimator` attribute), 79
`use_numba` (`dmx.stats.hidden_markov.HiddenMarkovEstimator` attribute), 182
`use_numba` (`dmx.stats.hidden_markov.HiddenMarkovModelDistribution` attribute), 178
`use_numba` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationDistribution` attribute), 312
`use_numba` (`dmx.stats.int_hidden_association.IntegerHiddenAssociationEstimator` attribute), 315
`use_numba` (`dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution` attribute), 364

V
`v` (`dmx.stats.dirac_length.DiracMixtureAccumulator` attribute), 186
`v` (`dmx.stats.dirac_length.DiracMixtureAccumulatorFactory` attribute), 189
`value()` (`dmx.bstats.bernoulli.BernoulliEstimatorAccumulator` attribute), 413
`value()` (`dmx.bstats.categorical.CategoricalEstimatorAccumulator` attribute), 418
`value()` (`dmx.bstats.composite.CompositeEstimatorAccumulator` attribute), 474
`value()` (`dmx.bstats.conditional.ConditionalDistributionEstimatorAccumulator` attribute), 480
`value()` (`dmx.bstats.dirac.DiracAccumulator` attribute), 421
`value()` (`dmx.bstats.dirichlet.DirichletAccumulator` attribute), 395
`value()` (`dmx.bstats.dmvn.DiagonalGaussianAccumulator` attribute), 426
`value()` (`dmx.bstats.dpm.DirichletProcessMixtureAccumulator` attribute), 482
`value()` (`dmx.bstats.exponential.ExponentialAccumulator` attribute), 433
`value()` (`dmx.bstats.exponential.ExponentialDistribution` attribute), 435
`value()` (`dmx.bstats.gamma.GammaAccumulator` attribute), 438
`value()` (`dmx.bstats.gaussian.GaussianAccumulator` attribute), 444
`value()` (`dmx.bstats.geometric.GeometricAccumulator` attribute), 450
`value()` (`dmx.bstats.ignored.IgnoredAccumulator` attribute), 488
`value()` (`dmx.bstats.intrange.IntegerCategoricalAccumulator` attribute), 456
`value()` (`dmx.bstats.mixture.MixtureEstimatorAccumulator` attribute), 498
`value()` (`dmx.bstats.nulldist.NullAccumulator` attribute), 501
`value()` (`dmx.bstats.optional.OptionalEstimatorAccumulator` attribute), 483
`value()` (`dmx.bstats.pdist.StatisticAccumulator` attribute), 183
`value()` (`dmx.bstats.poisson.PoissonEstimatorAccumulator` attribute), 466
`value()` (`dmx.bstats.sequence.SequenceEstimatorAccumulator` attribute), 518
`value()` (`dmx.bstats.setdist.BernoulliSetAccumulator` attribute), 520
`value()` (`dmx.stats.dirac_length.DiracMixtureAccumulator` attribute), 189

`dmx.stats.vmf`), 230
 VonMisesFisherDataEncoder (class in `dmx.stats.vmf`), 231
 VonMisesFisherDistribution (class in `dmx.stats.vmf`), 231
 VonMisesFisherEncodedDataSequence (class in `dmx.stats.vmf`), 233
 VonMisesFisherEstimator (class in `dmx.stats.vmf`), 233
 VonMisesFisherSampler (class in `dmx.stats.vmf`), 234

W

`w` (`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution` attribute), 247

`w` (`dmx.stats.gmm.GaussianMixtureDistribution` attribute), 260

`w` (`dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution` attribute), 154

`w` (`dmx.stats.hidden_markov.HiddenMarkovModelDistribution` attribute), 177

`w` (`dmx.stats.hmixture.HierarchicalMixtureDistribution` attribute), 168

`w` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovModelDistribution` attribute), 327

`w` (`dmx.stats.mixture.MixtureDistribution` attribute), 147

`w` (`dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution` attribute), 289

`w` (`dmx.stats.tree_hmm.TreeHiddenMarkovModelDistribution` attribute), 364

`w` (`dmx.torch_stats.hmm.HiddenMarkovModelDistribution` attribute), 633

`w1` (`dmx.stats.jmixture.JointMixtureDistribution` attribute), 162

`w2` (`dmx.stats.jmixture.JointMixtureDistribution` attribute), 162

`wc_key` (`dmx.stats.int_plsi.IntegerPLSIAccumulator` attribute), 267

`wcnt` (`dmx.stats.gmm.GaussianMixtureAccumulator` attribute), 255

`wcnt2` (`dmx.stats.gmm.GaussianMixtureAccumulator` attribute), 255

`wcnts` (`dmx.stats.int_hidden_markov.IntegerHiddenMarkovAccumulator` attribute), 319

`weight_key` (`dmx.stats.dirac_length.DiracMixtureAccumulator` attribute), 186

`weight_key` (`dmx.stats.gmm.GaussianMixtureAccumulator` attribute), 255

`weighted_log_posterior()` (in module `dmx.utils.vector`), 691

`weighted_log_posterior_sum()` (in module `dmx.utils.vector`), 691

`weighted_log_sum()` (in module `dmx.utils.vector`), 692

WeightedAccumulator (class in `dmx.stats.weighted`), 236

WeightedAccumulatorFactory (class in `dmx.stats.weighted`), 238

WeightedDataEncoder (class in `dmx.stats.weighted`), 239

WeightedDistribution (class in `dmx.stats.weighted`), 239

WeightedEncodedDataSequence (class in `dmx.stats.weighted`), 241

WeightedEstimator (class in `dmx.stats.weighted`), 241

`word_count` (`dmx.stats.int_plsi.IntegerPLSIAccumulator` attribute), 266

X

`x2w` (`dmx.stats.gmm.GaussianMixtureAccumulator` attribute), 255

`xw` (`dmx.stats.gmm.GaussianMixtureAccumulator` attribute), 255

Z

`zero_count` (`dmx.utils.automatic.DatumNode` attribute), 653

`zeros()` (in module `dmx.torch_utils.vector`), 702

`zeros()` (in module `dmx.utils.vector`), 692

`zeros_like()` (in module `dmx.torch_utils.vector`), 702

`zeta()` (in module `dmx.utils.special`), 684

`zw` (`dmx.stats.dmvn_mixture.DiagonalGaussianMixtureDistribution` attribute), 248

`zw` (`dmx.stats.gmm.GaussianMixtureDistribution` attribute), 260

`zw` (`dmx.stats.heterogeneous_mixture.HeterogeneousMixtureDistribution` attribute), 154

`zw` (`dmx.stats.mixture.MixtureDistribution` attribute), 147

`zw` (`dmx.stats.ss_mixture.SemiSupervisedMixtureDistribution` attribute), 289